

ASP .Net Core Tutorial for Beginners **Part 1**

Playlist: ASP NET Core Tutorial - Kudvenkat

Съдържание

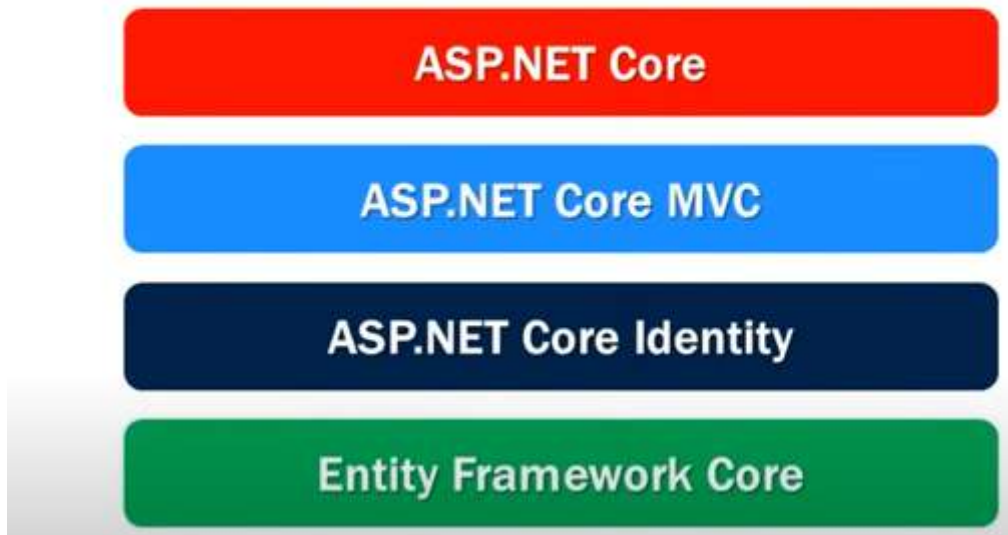
ASP .Net Core Tutorial for Beginners Part 1	1
01 - Introduction	3
02 – Setting up machine.....	5
03 – Web application	7
04 – Project File	11
05 – Main method	15
06 – In Process Hosting	17
07 – Out of process hosting.....	25
08 – Launchsettings	28
09 - Appsettings.json	31
10 - Middleware	39
11 – Configure request processing pipeline.....	43
12 – Static files.....	61
13 – Developer exception page	67
14 – Environment variables.....	73
15 – MVC tutorial.....	77
16 – Setup MVC	80
17 – Mvc vs Mvc Core	85
18 – Model	89
19 – Dependency injection.....	96
20 – Controller.....	98
21 – Views	102
22 – Customize View discovery	105
23 – Passing data to View	110
24 – ViewBag	113
25 – Strongly Typed View.....	116
26 – ViewModel.....	119
27 – List view	124
28 – Layout	128
29 – Sections in Layout Page	135
30 – ViewStart.cshtml	139

31 – ViewImports.cshtml	142
32 – Conventional Routing in MVC.....	144
33- Attribute routing.....	152
34 – Install and use Bootstrap	160
35 – Tag Helpers	172
36 – Why use TagHelpers	176
37 – ImageTagHelper	178
38 – Environment TagHelper.....	180
39 – Bootstrap Navigation Menu.....	189
40 – Form TagHelpers	198
41 – Model Binding	204
42 – Model Validation.....	212
44 – AddSingleton vs AddScoped vs AddTransient.....	217
45 – Introduction to Entity Framework	222
46 – Install Entity Framework Core.....	226
47 – DbContext in Entity Framework Core.....	230
48 – Sql Server	233
50 - Entity Framework Core migrations	245
51 - Entity framework core seed data.....	254
52 - Keeping domain models and database schema in sync.....	259
53 - File upload	264
54 -	278
55 – Edit View	286
56 – HTTP post edit action	293
57 – Handling 404 not found	301
58 - Centralised 404 error	306
59 - UseStatusCodePagesWithRedirects vs UseStatusCodePagesWithReExecute	308
60 - Global exception handling.....	313
61 – Logging.....	319
62 - Logging exceptions.....	325

Забележка: Учила съм се самостоятелно, като тези записки не са предвиждани за споделяне, а само и единствено за преговор. Проектът ми е с картинки от любимата ми художничка Josephine Wall - крайно неподходящи за бизнес-портфолио и в самия проект ще ги сменя със стандартни, но на скрийншотите все ще се виждат първоначалните.

01 - Introduction

ASP.NET Core Tutorial



What is ASP.NET Core

- ASP.NET Core is a **cross-platform, high-performance, open-source** framework for building modern, cloud-based, Internet-connected applications
- ASP.NET Core is a redesign of ASP.NET 4.x

ASP.NET Core Benefits and Features

One Unified Programming Model for MVC and Web API

Both the MVC Controller class and the ASP.NET Web API Controller class inherit from the same **Controller** base class and returns **ActionResult**



$\text{IActionResult} = \text{ViewResult} + \text{JsonResult}$

ASP.NET Core Benefits and Features

Cross Platform

Testability

One Programming
Model for MVC and
Web API

Open-Source

Dependency
Injection

Modular

ASP.NET Core Benefits and Features

Modular

- ASP.NET Core Provides Modularity with Middleware Components
- Both the request and response pipelines are composed using the middleware components
- Rich set of built-in middleware components are provided out of the box
- Custom Middleware Components can also be created

Prerequisites

- Basic HTML, CSS and C#
- Prior MVC knowledge is helpful but not required

Next : Tools required to build ASP.NET Core Applications

02 – Setting up machine

Software for .NET Core Development

An Editor - VS, VS Code, Sublime, Vim, Atom...

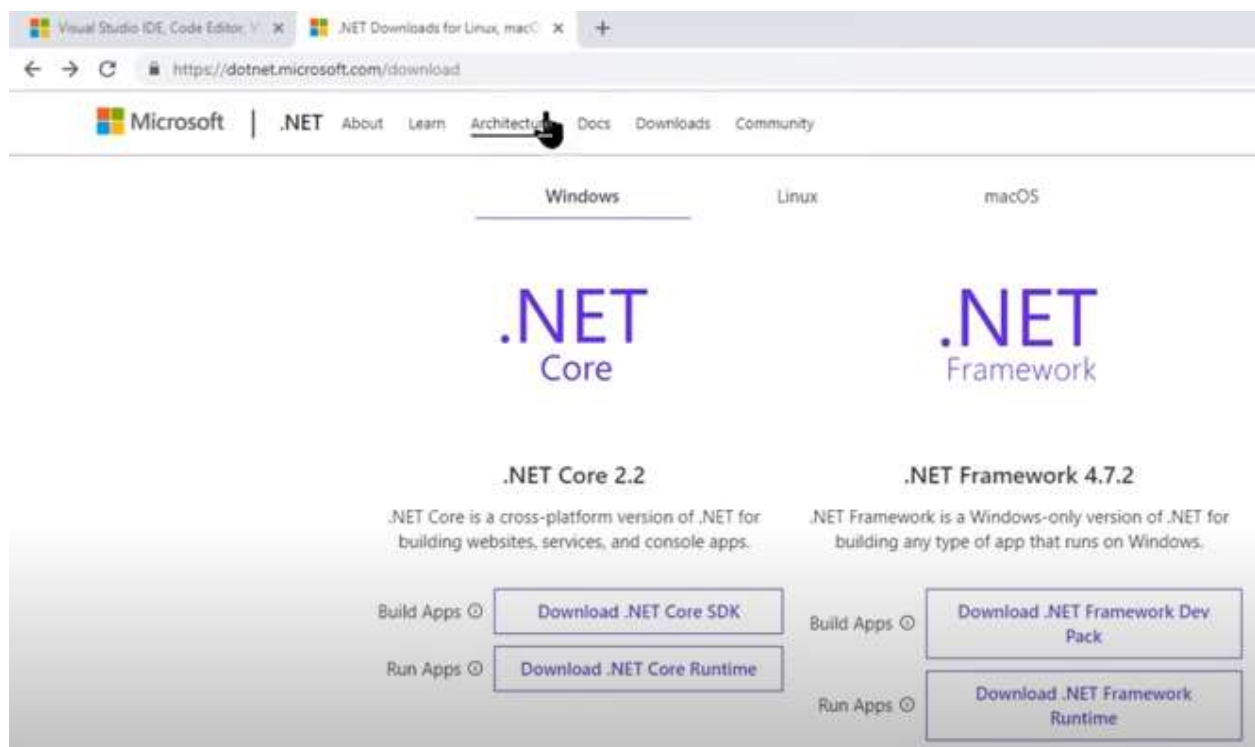
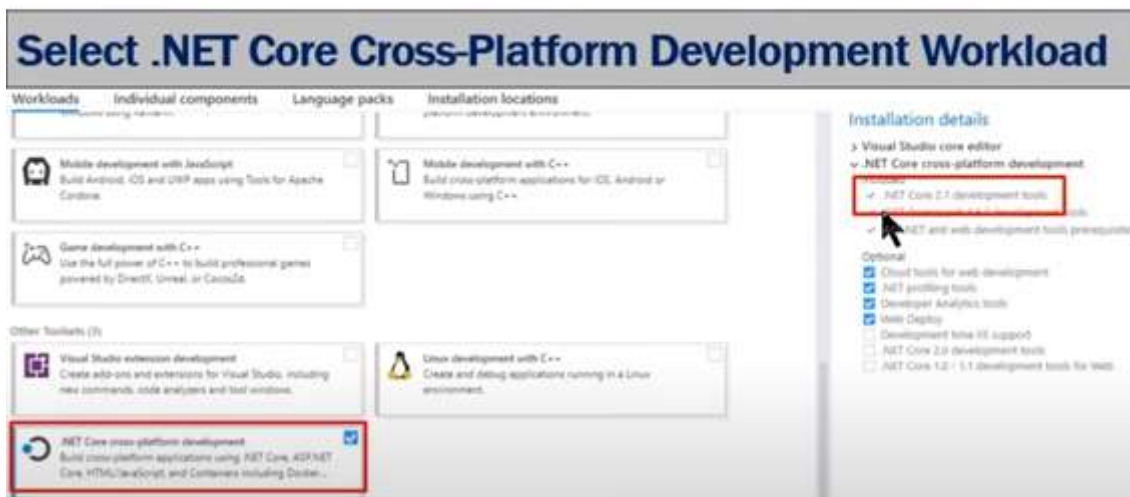
.NET Core SDK (Software Development Kit)

Install Visual Studio 2017

 **Visual Studio Community Edition is free**

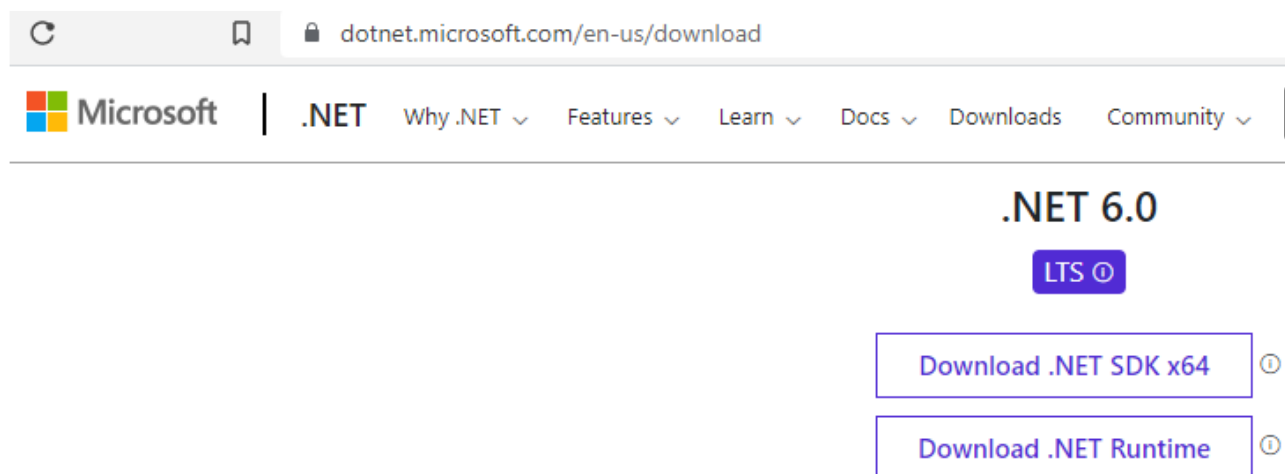
 <https://visualstudio.microsoft.com>

Install Visual Studio 2017



03 – Web application

Empty, без отметка на http.



Инсталираме .netCore SDK (създава и стартира), която включва в себе си и Runtime (само стартира готови проекти, без да може да ги създаде), затова само тя е достатъчна.

ASP.NET Core Benefits and Features

Cross Platform

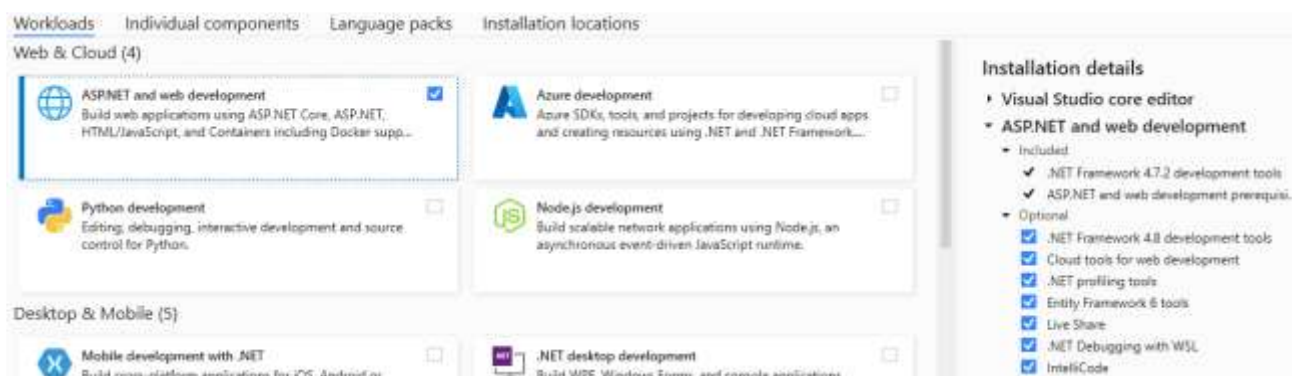
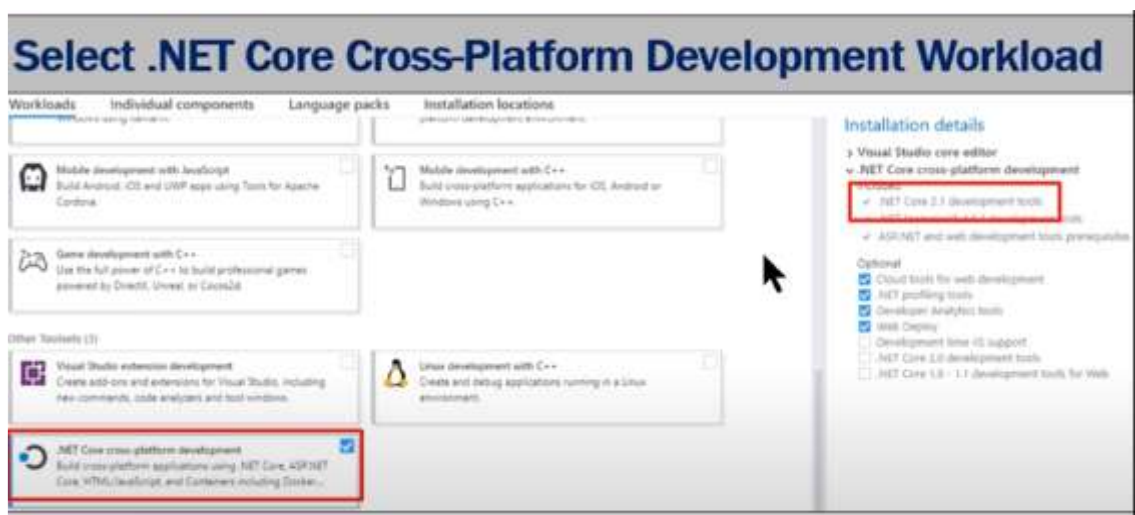
ASP.NET Core applications can be developed and run across different platforms like

- Windows
- macOS
- Linux

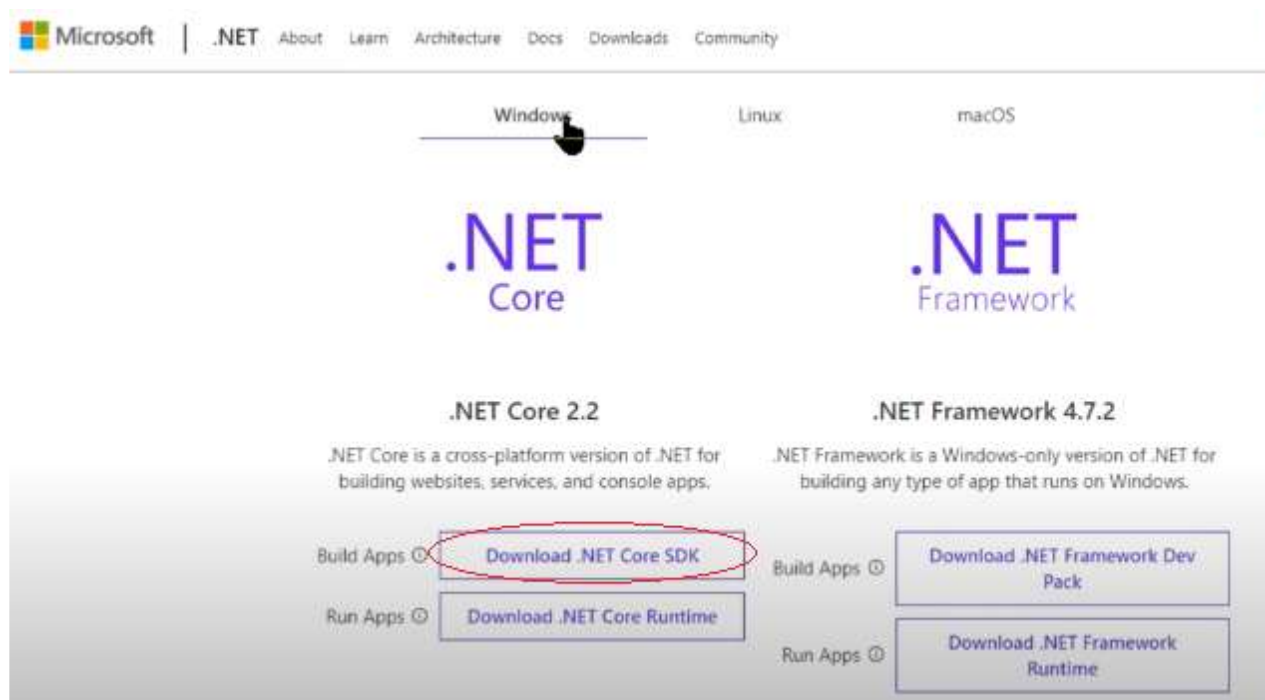
ASP.NET Core applications can be hosted on

- IIS
- Apache
- Docker
- Self-host in your own process

Инсталираме Visual Studio Community безплатно от сайта на MicroSoft като гледаме да има сложим отметка на * .Net Core Cross-Platform Development Workload.



Ако тя е избрана, с нея инсталираме автоматично и SDK.



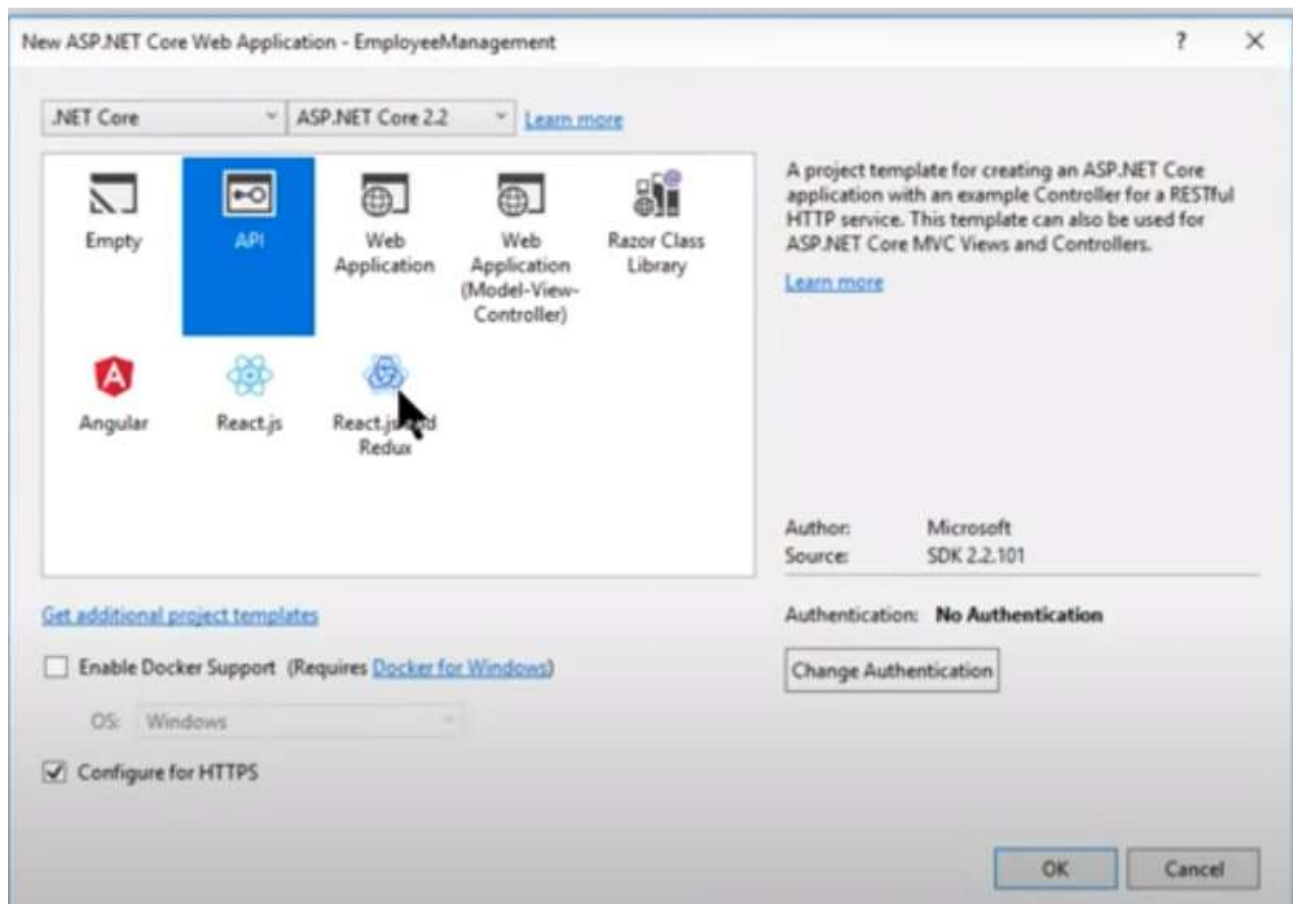
.NET Core SDK е достатъчно и включва и Runtime в себе си.
Runtime = Run only – само стартира проект

Core – проектира, и стартира, съдържа Runtime в себе си.

Templates:

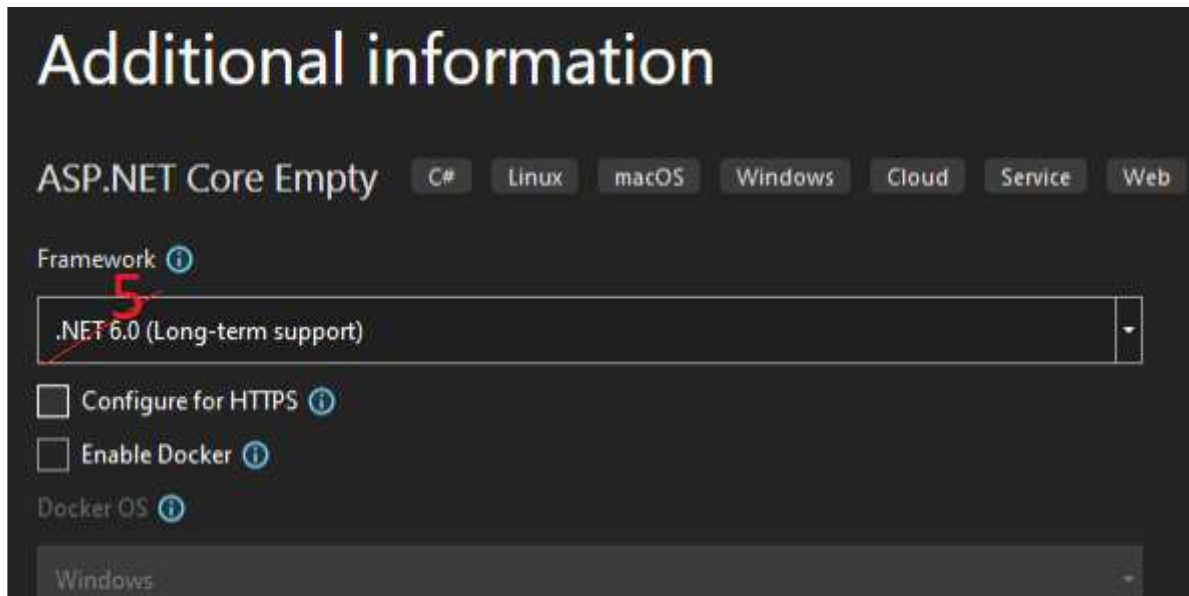
- * Empty – празен
- * MCV – всичко
- * API – Application – restful HTTP Service, без user interface (JS, CSS, View, Layout...) не се изискват – създава само контролери за обработка на данни.
- * Web App – razor – по-олекотена версия на MCV
- * Razor Class Library – reusable class library project – data models, class, pages, razor views, view components. Може да бъде използван многократно в различни приложения, като може да ги презаписва.

- * Angular –
- * React
- * React and redux



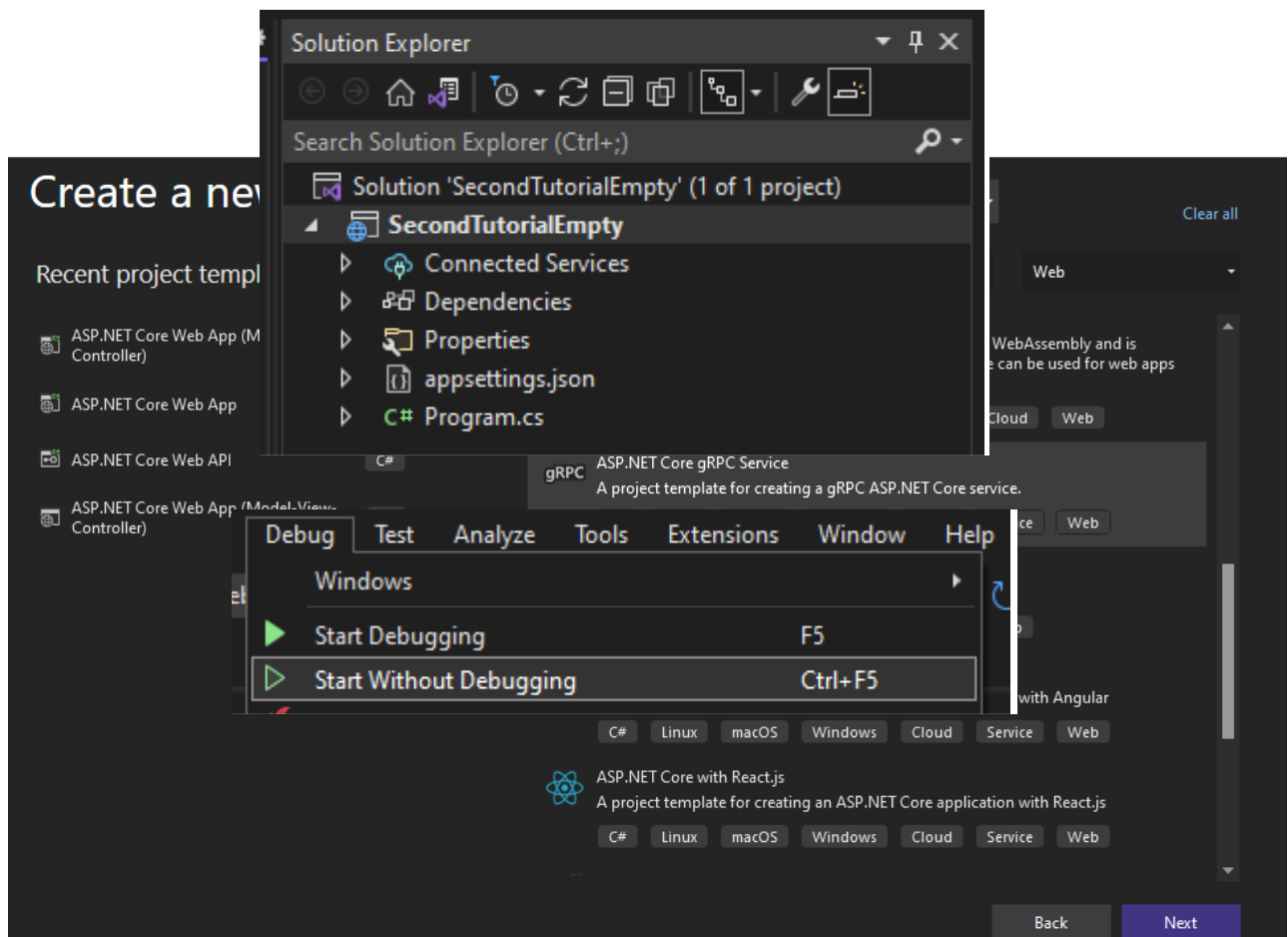
04 – Project File

Избираме Empty за демонстрация и гледаме да не е чекнат Configure for HTTPS



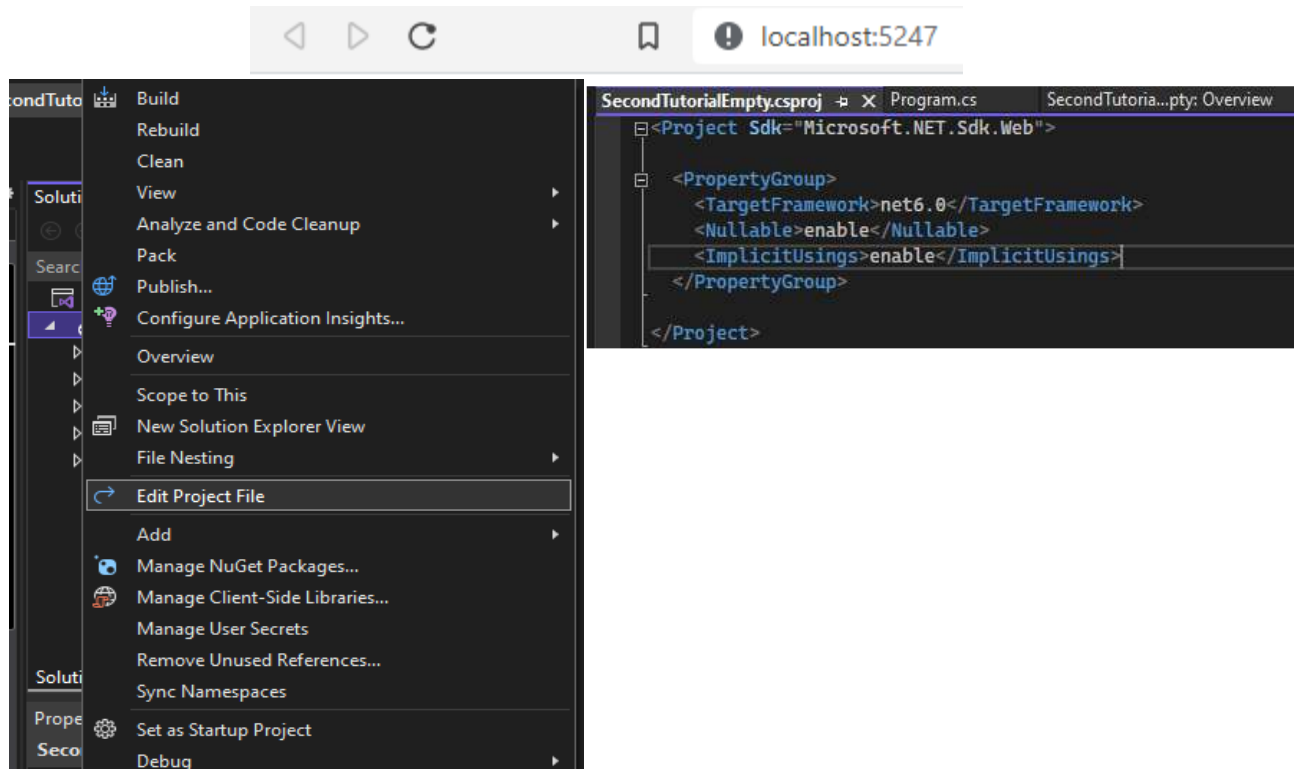
VS->Empty project-> (without HTTP mark)-> *.CsProj

Като погледнем Solution Explorer имаме минимално генериран код по образец



.csproj = c sharp project

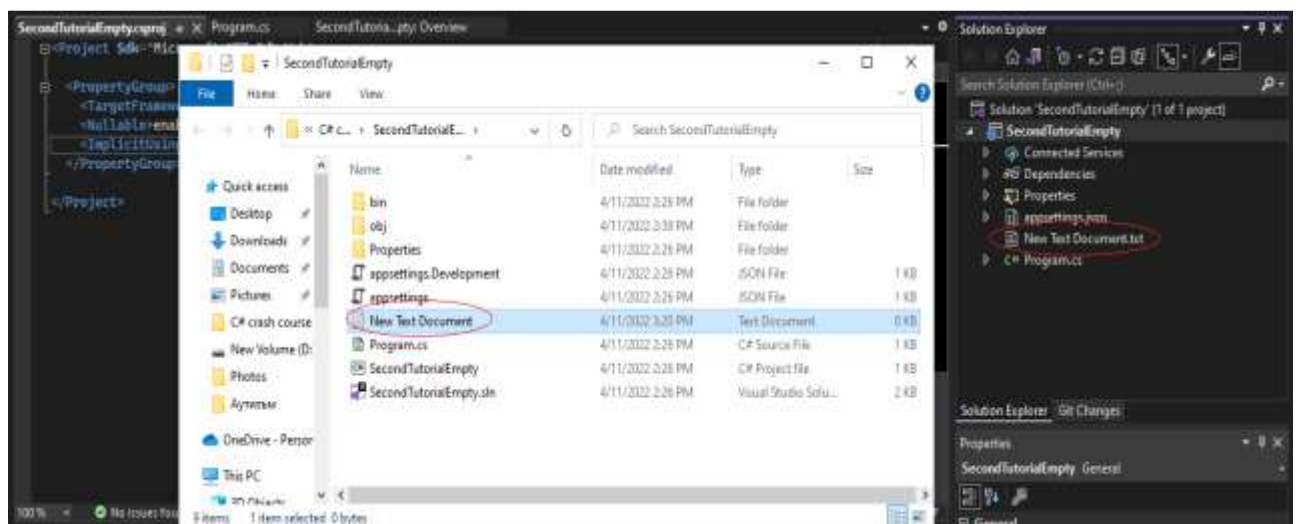
За да затворим проект преди: д.б. Unload project, после може да нанесем корекции върху самия project файл и после да го заредим отново с Reload project.



В .Net Core това не е необходимо – не е нужно да затваряме проект, за да го редактираме.

Как да добавим файл към текущ проект? – д.б. Open Folder in File Explorer.

Отиваме в локалната папка на проекта в компютъра. Добавяме обикновен текстов файл и той автоматично се добавя в Solution Explorer на програмата. Изтриваме го от едното място, съответно се изтрива и в другото. И двете са свързани. Всички файлове



и папки, които се намират в root-folder папката на проекта, автоматично стават част от него.

`<TargetFramework>` – Target Framework Moniker TFM (Прозвище)

Стари версии за .NET Core 2017 + 2.2

ASP.NET Core Project File		
TargetFramework		
➤ Specifies the target framework for the application ➤ To specify a target framework we use Target Framework Moniker (TFM)		
Name	Abbreviation	TFM
.NET Framework	net	net451
		net472
.NET Core	netcoreapp	netcoreapp1.0
		netcoreapp2.2

`<TargetFramework>`Miniker – псевдоним, версия 2.2 в случая, при мен net5.0

```
1 <Project Sdk="Microsoft.NET.Sdk.Web">
2
3   <PropertyGroup>
4     <TargetFramework>netcoreapp2.2</TargetFramework>
5     <AspNetCoreHostingModel>InProcess</AspNetCoreHostingModel>
6   </PropertyGroup>
7
8   <ItemGroup>
9     <PackageReference Include="Microsoft.AspNetCore.App" />
10    <PackageReference Include="Microsoft.AspNetCore.Razor.Design" Version="2.2.0" PrivateAsset
11  </ItemGroup>
12
13 </Project>
```

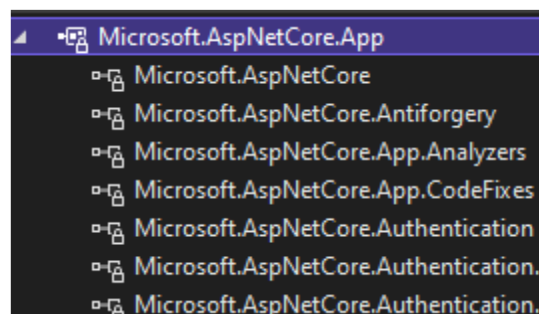
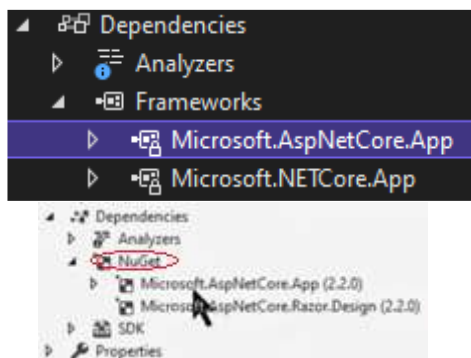
`<AspNetCoreHostingModel>` – хостване

ASP.NET Core Project File

AspNetCoreHostingModel

- Specifies how the application should be hosted
- InProcess or OutOfProcess
- InProcess hosts the app inside of the IIS worker process (w3wp.exe)
- OutOfProcess hosting model forward web requests to a backend ASP.NET Core app running the Kestrel server
- The default is OutOfProcess hosting

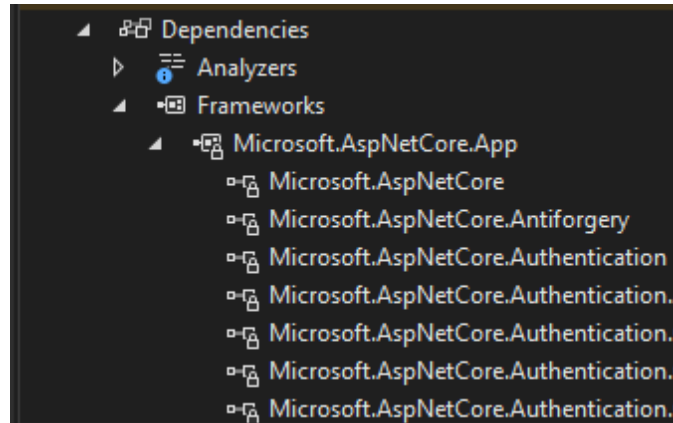
<PackageReference> МетаПакет със зависимости, които ще допълваме. При мен липсва по подразбиране.



При разлистване се показва огромен брой Мета-зависимости Dependencies. С всяка по-нова версия на .NET, все по-малко код остава видим. Мета-пакетът няма собствено съдържание, а само лист от зависимости (други пакети).

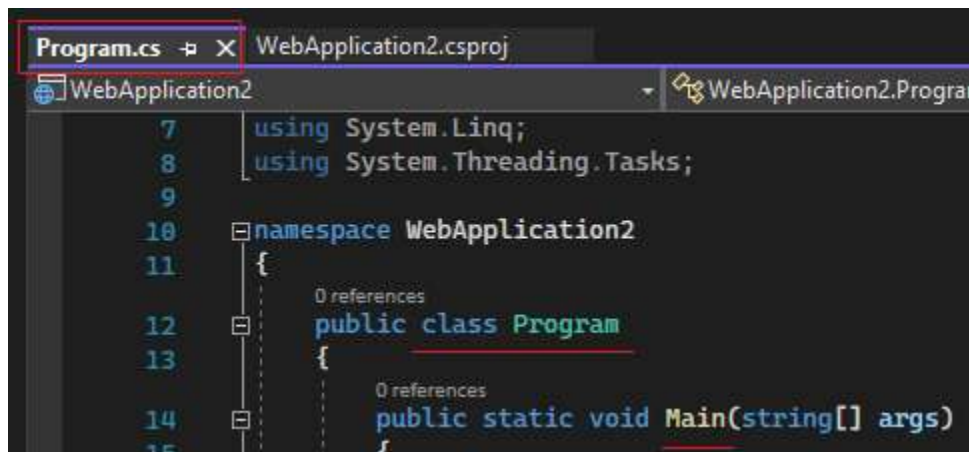
PackageReference

- Used to Include a reference to the NuGet package that is installed for the application
- Metapackage - **Microsoft.AspNetCore.App**
- A metapackage has no content of its own
- It just contains a list of dependencies (other packages)
- When the version is not specified, an implicit version is specified by the SDK
- Rely on the implicit version rather than explicitly setting the version number on the package reference

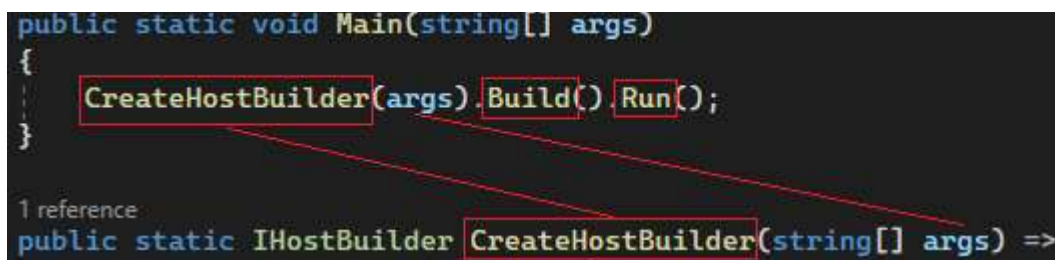


05 – Main method

Main – the entry point for the console application in class program (for Runtime)!



Самият туториал върви по версия .NET 2.2 и дори до 5.0 има промени, като това, че в програмата е заложено в старата версия CreateWebHostBuilder, а в новата само CreateHostBuilder, но тогава при друго стартиране извиква името на програмата само.



Обектът, който CreateHostBuilder връща със своите аргументи, се хоства с Build, а после стартира с Run, който слуша за идващи http заявки.


```

public static void Main(string[] args)
{
    CreateHostBuilder(args).Build().Run();
}

1 reference
public static IHostBuilder CreateHostBuilder(string[] args) =>
    Host.CreateDefaultBuilder(args)
        .ConfigureWebHostDefaults(webBuilder =>
        {
            webBuilder.UseStartup<Startup>();
        });

```

```

webBuilder.UseStartup<Startup>();
});

```

class WebApplication2.Startup

При посочване с мишката – Startup е клас, а при д.б. - Go to Definition – отвежда ни къде е дефиниран този клас. Main – оттам започва изпълнението на програмата. Във видеото е CreateWebHostBuilder, а в моята версия е CreateHostBuilder

```

namespace EmployeeManagement
{
    public class Program
    {
        public static void Main(string[] args)
        {
            CreateWebHostBuilder(args).Build().Run();
        }

        public static IWebHostBuilder CreateWebHostBuilder(string[]
            args) =>
            WebHostBuilder.CreateDefaultBuilder(args)
                .UseStartup<Startup>();
    }
}

```

```

1 reference
public class Startup
{
    // This method gets called by the runtime. Use this method to add services to the container.
    // For more information on how to configure your application, visit https://go.microsoft.com/fwlink/?
    17 public void ConfigureServices(IServiceCollection services)
    18 {
    19 }
    20
    // This method gets called by the runtime. Use this method to configure the HTTP request pipeline.
    22 public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
    23 {
    24     if (env.IsDevelopment())
    25     {
    26         app.UseDeveloperExceptionPage();
    27     }
    28     app.UseRouting();
    29     app.UseEndpoints(endpoints =>
    30     {
    31     });
    32 }
}

```

Клас Startup съдържа двата метода ConfigureServices (конфигурира услугите, които се изискват за приложението), Configure (приложението, заявките, .

Extension - <https://www.youtube.com/watch?v=VkrKNXscoto>

06 – In Process Hosting

ASP.NET Core InProcess Hosting

Some of the Tasks that CreateDefaultBuilder() performs

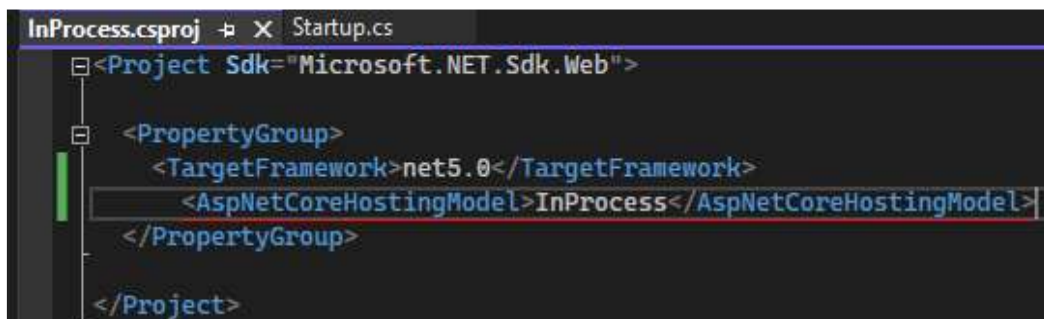
- Setting up the web server
- Loading the host and application configuration from various configuration sources and
- Configuring logging

An ASP.NET core application can be hosted

- InProcess or
- OutOfProcess

Върху папката с проекта -> д.6. -> Edit project file ->

`<AspNetCoreHostingModel>InProcess</AspNetCoreHostingModel>`



По подразбиране в новите версии този ред изцяло отсъства, а процесът е OutOfProcess.

.NetCore app starts as console application, then becomes web app.
InProcess има много по-голяма пропускателна способност от OutOfProcess.
OutOfProcess е по подразбиране. Процесът, който се изпълнява по подразбиране е iisexpress за Host + Run от единствен сървър Kestrel при разработване на проект.
OutOfProcess = 2 web servers (internal Kestrel) + (external IIS), процес iisexpress
InProcess = 1 web server _____(external IIS), процес dotnet
InProduction използваме iis
external web server = reverse proxy server

ASP.NET Core InProcess Hosting

To configure InProcess hosting

```
<AspNetCoreHostingModel>InProcess</AspNetCoreHostingModel>
```

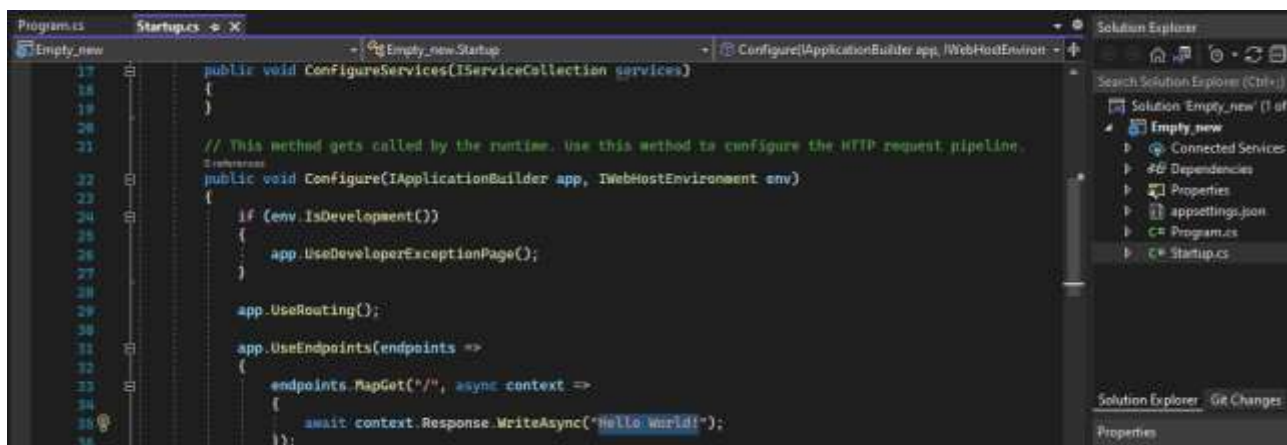
CreateDefaultBuilder() method calls UseIIS() method and host the app inside of the IIS worker process (w3wp.exe or iisexpress.exe)

InProcess hosting delivers significantly higher request throughput than OutOfProcess hosting

To get the process name executing the app

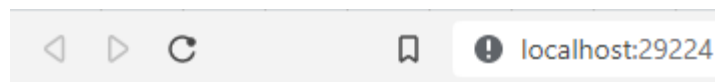
```
System.Diagnostics.Process.GetCurrentProcess().ProcessName
```

В празен проект, „Hello world” се намира в startup.cs
Вместо „Hello world!” искаме да изпишем името на процеса, който го извиква:



await context.Response
.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().ProcessName);
и натискаме Enter, за да смъкнем на долния ред без да скрулираме надясно.

```
await context.Response  
.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().ProcessName);
```



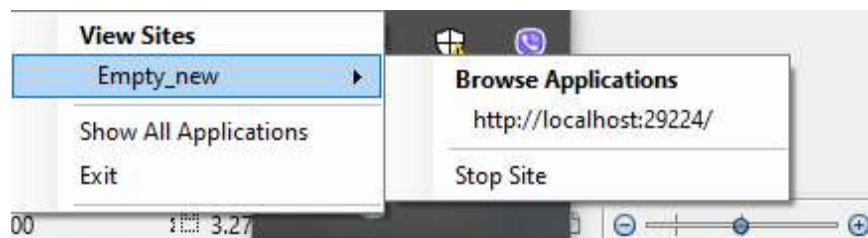
iisexpress

Резултат след стартиране с Ctrl + F5 = iisexpress (името на процеса, който VisualStudio използва по подразбиране).

Този процес не се използва InProduction – тогава използваме друг: IIS И при зареждане в брауъра ще видим W3Wp. Също може да го намерим в линията на



таскбара:



ASP.NET Core InProcess Hosting

With out of process hosting

- 2 Web Servers - Internal and External Web Server
- The internal web server is Kestrel
- The external web server can be IIS, Nginx or Apache

What is Kestrel

- Cross - Platform Web Server for ASP.NET Core
- Kestrel can be used, by itself as an edge server
- The process used to host the app is dotnet.exe

CMD = .NET command-line interface (CLI)

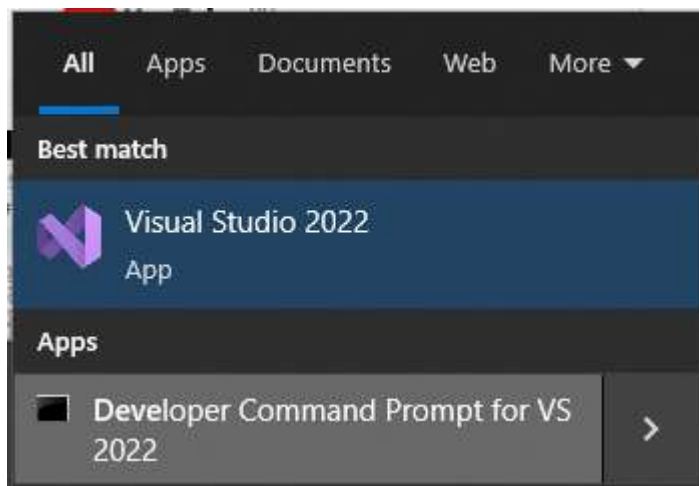
команда **dotnet --help**

виждаме всички команди, които можем да използваме в тази работна среда

cd – change directory - главна директория c: D:

ctrl + c = close console

Run: Search



Dotnet --help

Не ми дава достъп до директорията `cd d:\`,
Под windows 10 има промени:

`cd /d d:\` или `cd c:\`

```
Select Developer Command Prompt for VS 2022

*****
** Visual Studio 2022 Developer Command Prompt v17.1.3
** Copyright (c) 2022 Microsoft Corporation
*****

D:\Programs\VS>dotnet --help
.NET SDK (6.0.201)
Usage: dotnet [runtime-options] [path-to-application] [arguments]

Execute a .NET application.

runtime-options:
--additionalprobingpath <path>    Path containing probing policy and assemblies to probe for.
--additional-deps <path>          Path to additional deps.json file.
--depsfile <path>                 Path to <application>.deps.json file.
--fx-version <version>            Version of the installed Shared Framework to use to run the application.
--roll-forward <setting>          Roll forward to framework version (LatestPatch, Minor, LatestMinor, Major, LatestMajor, Disable).
--runtimeconfig <path>           Path to <application>.runtimeconfig.json file.

path-to-application:
The path to an application .dll file to execute.

Usage: dotnet [sdk-options] [command] [command-options] [arguments]

Execute a .NET SDK command.

sdk-options:
-d|--diagnostics    Enable diagnostic output.
-h|--help           Show command line help.
```

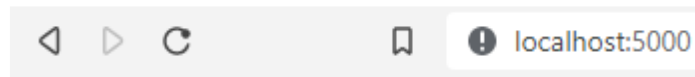
Отиваме в директорията на проекта, като използваме CD за нулиране на адреса към основните дялове C или D. CMD е сензитивен към големината на буквите. В новата версия на CLI 2022 се използва друга команда **`cd /d d:\`**

```
D:\>cd /d d:\Crash_course\Empty_new

d:\Crash_course\Empty_new>dotnet run
Building...
info: Microsoft.Hosting.Lifetime[0]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: d:\Crash_course\Empty_new
```

Така стартираме програмата от конзолата, без да участва програмата VS.

Резултатът се извежда от Kestrel сървър. <http://localhost:{PORT}>.



Empty_new

Опитах да използвам специален знак и интервали. Командите в CMD си вървят, но ако самият VS използва файловете в същия момент с CMD, връща грешки докато не се затвори.

```
j:\Debug\net5.0\Empty_new.dll" to "bin\Debug\net5.0\Empty_new.dll" because
"obj\Debug\net5.0\Empty_new.dll" to "bin\Debug\net5.0\Empty_new.dll" because
ss the file 'D:\C# crash course\Empty_new\bin\Debug\net5.0\Empty_new.dll' because
[D:\C# crash course\Empty_new\Empty_new.csproj]
C:\Program Files\dotnet\sdk\6.0.201\Microsoft.Common.Cu
Debug\net5.0\Empty_new.dll" to "bin\Debug\net5.0\Empty_
e\Empty_new\Empty_new.csproj]
C:\Program Files\dotnet\sdk\6.0.201\Microsoft.Common.Cu
"obj\Debug\net5.0\Empty_new.dll" to "bin\Debug\net5.0\Empty_new.dll" because
urce\Empty_new\bin\Debug\net5.0\Empty_new.dll' because
w\Empty_new.csproj]

The build failed. Fix the build errors and run again.

D:\C# crash course\Empty_new>dotnet run
Building...
info: Microsoft.Hosting.Lifetime[0]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\C# crash course\Empty_new
```

При мен не изкарва името на процеса **dotnet**, както във видеото и свръх-чувствителната настройка на антивирусната Avast се намесва. Изкарва ми просто името на моята програма *.exe вътре в папката със същото име. Причината е по-новата версия и има значение кой точно компилатор се стартира.

`Process.GetCurrentProcess().ProcessName` reports `w3wp/iisexpress` (in-process) or `dotnet` (out-of-process).

<https://docs.microsoft.com/en-us/aspnet/core/host-and-deploy/iis/out-of-process-hosting?view=aspnetcore-5.0>


```

C:\> Select Developer Command Prompt for VS 2022 - dotnet run

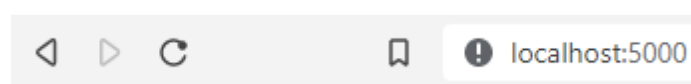
*****
** Visual Studio 2022 Developer Command Prompt v17.1.3
** Copyright (c) 2022 Microsoft Corporation
*****

D:\Programs\VS>cd d:\C# crash course\InProgress

d:\C# crash course\InProgress>dotnet run
Building...
info: Microsoft.Hosting.Lifetime[0]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: d:\C# crash course\InProgress

```

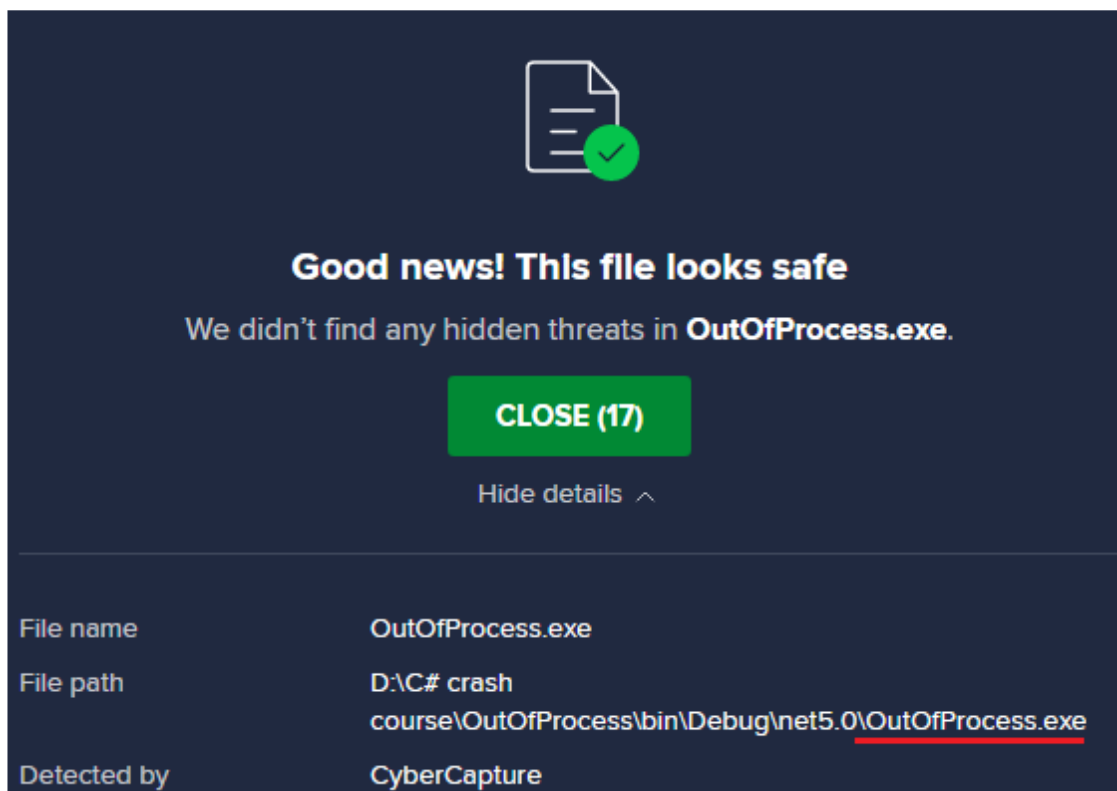
6/124 – имам проблем – вместо името на процеса dotnet, изписва името на проекта, което е вероятно предпазен механизъм на самото VS – връща го винаги, когато не може да намери конкретен процес (като при разминавания между версиите 2.2, 5.0, 6.0)



InProcess

https://www.youtube.com/watch?v=ydR2jd3ZaEA&list=PL6n9fhu94yhVkdusLaQsfERmL_Jh4XmU&index=7

Всъщност, в следващото видео и при следващия старт на антивирусната, вероятно точно тази е причината – в старите версии е било dotnet.exe еднакъв файл във папката на всеки проект, докато сега *.exe файла е със същото име като проекта, в който се намира.



При проба с OutOfProcess или изтрит ред, програмата извежда като име на процес името на проекта.exe, а когато въведа InProcess, се изпълнява вярно извежда iiexpress

<AspNetCoreHostingModel>InProcess</AspNetCoreHostingModel>

РАЗБРАХ ОТКЪДЕ ИДВА ГРЕШКАТА: след .NET 3.0 по подразбиране за OutOfProcess изкарва това .exe с името на проекта, вместо dotnet.exe както е в старата версия, на която е туториъла .net 2.2. Ако има грешка (не е затворено VS или не е стартиран през CMD dotnet run, в браузъра localhost:5000 дава, че страницата не е намерена.



This site can't be reached

localhost refused to connect.

Try:

- Checking the connection
- [Checking the proxy and the firewall](#)

ERR_CONNECTION_REFUSED

Reload

Why do I see project directory name instead of dotnet(out-of-process) while running ASP.NET Core app through dotnet CLI?

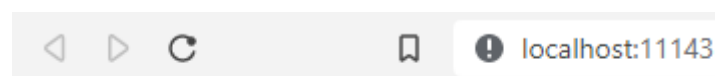
Starting with ASP.NET Core 3.0 a host exe ("apphost") is generated automatically for the platform you're currently building on (except for macOS due to its signing/notarization requirements), so the `EmployeeManagement.exe` in your bin folder will be used to run the .NET Core Runtime.

You should see a difference if you directly use `dotnet bin\netcoreapp3.1\EmployeeManagement.dll` or set the following in the csproj:

```
<PropertyGroup>
  <UseAppHost>False</UseAppHost>
</PropertyGroup>
```

```
<PropertyGroup>
  <TargetFramework>net5.0</TargetFramework>
  <AspNetCoreHostingModel>OutOfProcess</AspNetCoreHostingModel>
  <UseAppHost>False</UseAppHost>
</PropertyGroup>
```

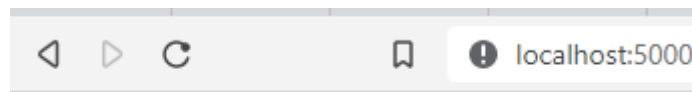
`<UseAppHost>False</UseAppHost>`



dotnet

Или чрез CMD

```
WebApplication14>dotnet bin\Release\net5.0\WebApplication14.dll
```



dotnet

HTTP – Hypertext Transfer Protocol

IIS = Information Internet Server

ASP.NET Core InProcess Hosting



With InProcess hosting

- Application is hosted inside the IIS worker process
- There is only one web server
- From a performance standpoint, InProcess hosting is better than OutOfProcess hosting

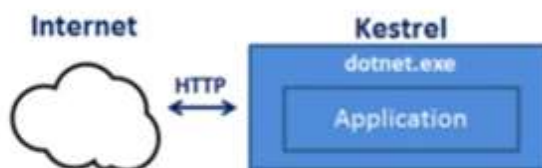
07 – Out of process hosting

*InProcess – IIS Express (w3wp.exe) по-бърз и лек

*OutOfProcess – Kestrel (dotnet.exe) по подразбиране, както и независимо от задаването на модела `<AspNetCoreHostingModel>InProcess` вътре в програмата, се изпълнява в CLI OutOfProcess (и в 5.0 показва името на проекта .exe, ако изречно не е зададен `<UseAppHost>False`), както и всеки път през localhost:5000

ASP.NET Core Out of Process Hosting

- Kestrel can be used as the internet facing web server



- Kestrel can be used in combination with a reverse proxy



Също разминаване: ако е изтрит реда за модела `<AspNetCoreHostingModel>`, по подразбиране в туториъла е `OutOfProcess` и се извежда на екрана `dotnet`, но при мен се извежда `iisexpress`, т.е. вероятно е `InProcess`, поне в старата комбинация от windows 7 + .net 5.0. При втори път на минаване на туториъла при windows 10 нямам този проблем.

Internal and external

IIS – Proxy server по подразбиране.

Защо ни трябва междинен Proxy server? - за допълнителни слоеве конфигурации и сигурност.

In Process v/s Out of Process Hosting

In Process	Out of Process
Process name is w3wp.exe or iisexpress.exe	Process name is dotnet.exe
Only one web server	Two web servers
Better for performance	Penalty of proxying requests between internal and external web server

(Out Of Process) - 2 web servers: * Internal (Kestrel) + External (IIS)

Можем ли да стартираме .Net core апликация без да използваме Kestrel? - Да, ако използваме вграден InProcess хостинг модел, апликацията се хоства вътре в iis работещ процес w3wp.exe без да се използва build-in Kestrel.

dotnet -info

По подразбиране VS използва IIS за стартиране и хостване на апликацията.

За да видим процеса, който се стартира:

Startup.cs:

```
C:\Users\Administrator>cd /d d:  
D:\>
```

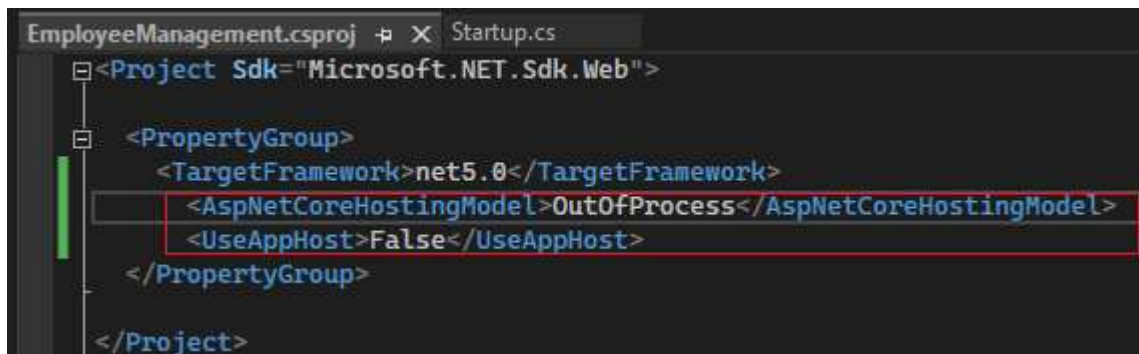
dotnet run

```
C:\Program Files\Microsoft Visual Studio\2022\Community>cd /d d:/Exercises/CS_Begin/EmployeeManagement  
d:\Exercises\CS_Begin\EmployeeManagement>dotnet run
```

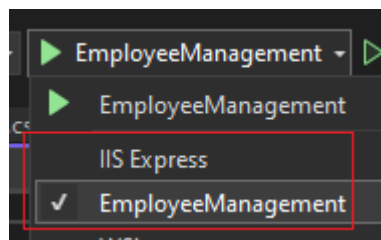
Резултат при <http://localhost:5000/> => EmployeeManagement

`Process.GetCurrentProcess().ProcessName` reports `w3wp/iisexpress` (in-process) or `dotnet` (out-of-process).

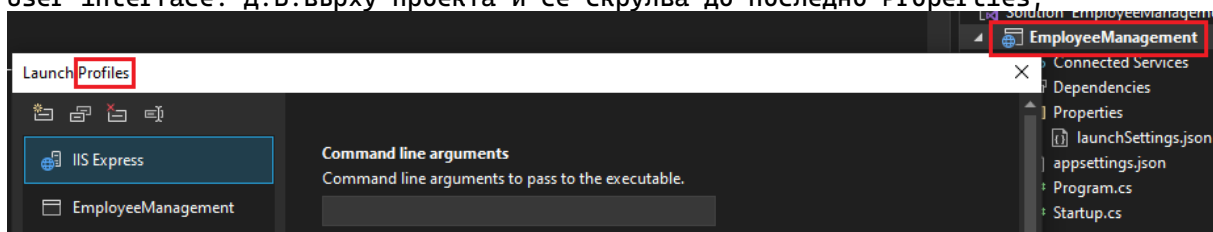
```
<AspNetCoreHostingModel>OutOfProcess</AspNetCoreHostingModel>  
<UseAppHost>False</UseAppHost>
```



Този ред стандартно отсъства, той е заложен по подразбиране. Променя се при InProcess `<AspNetCoreHostingModel>OutOfProcess</AspNetCoreHostingModel>`
Ако в програмата е InProcess, но се стартира CLI, външната CLI ще засенчи локалната стойност InProcess и ще използва по подразбиране заложената в CLI – OutOfProcess, при което в 5000 -> dotnet



При старт от се активира CLI с localhost:5000
User interface: Д.Б.върху проекта и се скрулва до последно Properties,



Спокойно мога да заключа, че най-много главоблъсканици получавах при стартиране на нова версия на VisualStudio2022 + net 5.0 върху стара платформа OS Windows 7. Не правете така у дома!

С инсталиране на същите версии върху Windows 10 автоматично много грешки и разминавания отпаднаха.

08 – Launchsettings

Solution Explorer -> Properties -> launchSettings.json – използва се само от локалната машина и не е необходим за публикуване, а за съхранение на определени настройки, независимо откъде се използва – VS/CLI

- Application configuration;
- Environment specific app settings strategies and files
// appSettings.staging.json – staging environment
appSettings.development.json - development environment
- Environment variables uses

Имаме 2 профила: единият е IISExpress, другият е с името на проекта. По-нагоре в iisSettings имаме номерът на порта, който ще се стартира като localhost:номер при стартиране на проекта с Ctrl+F5 или само F5. Съответно този номер може да бъде променен оттам.

```
"iisSettings": {
  "windowsAuthentication": false,
  "anonymousAuthentication": true,
  "iisExpress": {
    "applicationUrl": "http://localhost:4184",
    "sslPort": 0
  }
},
"profiles": {
  "IIS Express": {
    "commandName": "IISExpress",
    "launchBrowser": true,
    "environmentVariables": {
      "ASPNETCORE_ENVIRONMENT": "Development"
    }
  },
  "WebApplication15": {
    "commandName": "Project",
    "dotnetRunMessages": "true",
    "launchBrowser": true,
    "applicationUrl": "http://localhost:5000",
    "environmentVariables": {
      "ASPNETCORE_ENVIRONMENT": "Development"
    }
  }
}
```

ASPNETCORE_ENVIRONMENT със стойност Development може да бъде сменен със Staging/Production в зависимост от това в каква среда стартираме проекта – Staging/Production, също може да добавим нови променливи. Например в Startup.cs

```
22 public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
23 {
24     if (env.IsDevelopment())
25     {
26         app.UseDeveloperExceptionPage();
27     }
28     app.UseRouting();
29     app.UseEndpoints(endpoints => {
30         endpoints.MapControllerRoute(
31             "default", "{controller}/{action}/{id}");
32     });
33 }
```

(extension) IApplicationBuilder IApplicationBuilder.UseDeveloperExceptionPage() (+ 1 overload)
Captures synchronous and asynchronous Exception instances from the pipeline and generates HTML error responses.
This should only be enabled in the Development environment.
Returns:
A reference to the app after the operation has completed.

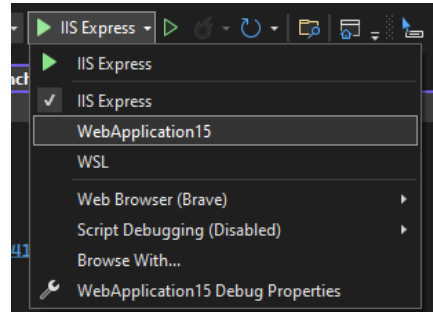
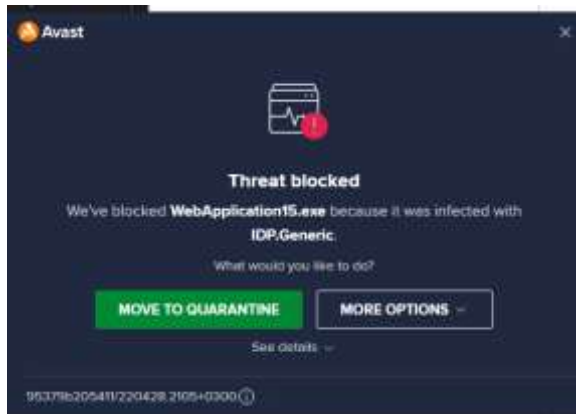
IsDevelopment() връща true ако "ASPNETCORE_ENVIRONMENT": "Development" и тогава се изпълнява UseDeveloperExceptionPage, т.е. ще се покаже тази страница с изключение

```
if (env.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
```

Имаме 2 профила: IIS Express и вторият е нашият с името, което сме избрали. Когато стартираме проект с Ctrl + F5 или само F5, стартира програмата с профила с името на проекта.

Имаме право на два избора на компилатори: IIS Express или този с името на програмата. Ако изберем първия, който е по подразбиране, минава през VS, а втория през CLI, който стартира нашата апликация. Нека го докажем. Затваряме проекта CLI и избираме стартиране с IIS Express

, тогава виждаме изпълнението на програмата с единия порт localhost, а при избор за старт с името на нашата програма, излиза същият резултат през друг адрес localhost:5000.

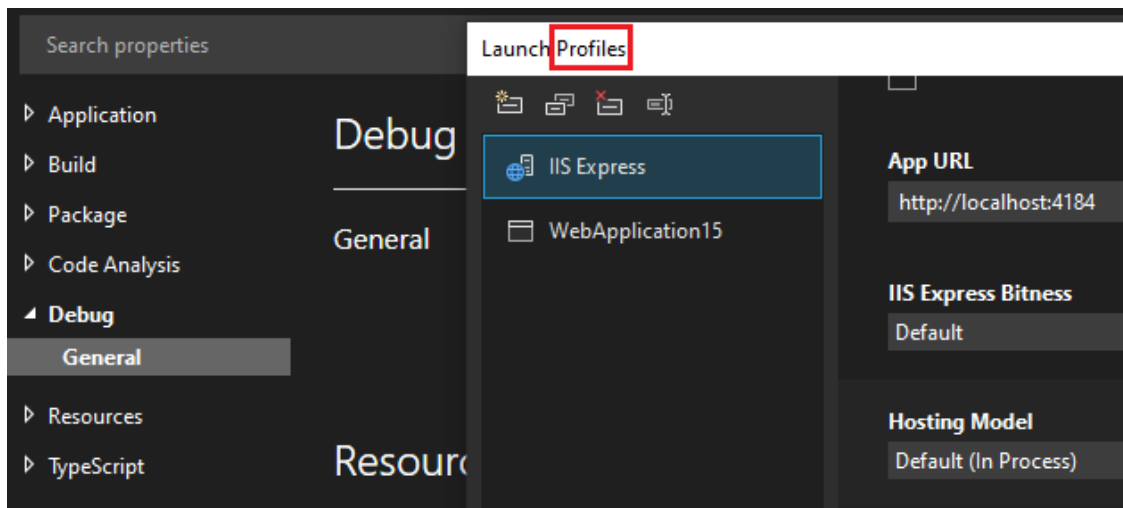


С най-голяма видимост: <pre><AspNetCoreHostingModel>InProcess <UseAppHost>False IISExpress – localhost:4184 => iisexpress WebApp15 – localhost:5000 => dotnet</pre>	По подразбиране, ако редовете отсъстват: <pre><AspNetCoreHostingModel>InProcess <UseAppHost>True IISExpress – localhost:4184 => iisexpress WebApp15 – localhost:5000 => WebApp15</pre>
<pre><AspNetCoreHostingModel>OutOfProcess <UseAppHost>False IIS Express – localhost:4184 => dotnet WebApp15 – localhost:5000 => dotnet</pre>	<pre><AspNetCoreHostingModel>OutOfProcess <UseAppHost>True IISExpress – localhost:4184 => WebApp15 WebApp15 – localhost:5000 => WebApp15 (съобщение на антивирусната)</pre>

"commandName" може да приемат стойности "IISExpress", "Project" и "IIS".
Project = WebApp15

commandName	AspNetCoreHostingModel	Internal Web Server	External Web Server
Project	Hosting Setting Ignored	Only one web server - Kestrel	
IISExpress	InProcess	Only one web server - IIS Express	
IISExpress	OutOfProcess	Kestrel	IIS Express
IIS	InProcess	Only one web server - IIS	
IIS	OutOfProcess	Kestrel	IIS

User Interface: Д.Б. върху проекта и скрул до последно на моя монитор -> properties ->



ASP.NET Core appsettings.json File

Configurations Sources in ASP.NET Core

- Files (appsettings.json, appsettings.{Environment}.json)
- User secrets
- Environment variables
- Command-line arguments

To access configuration information

- IConfiguration Service

09 - Appsettings.json

Empty project template:

```

1 {
2   "Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft": "Warning",
6       "Microsoft.Hosting.Lifetime": "Information"
7     }
8   },
9   "AllowedHosts": "*"
10 }

```

Конфигурационната информация в .net обикновено се съхранява като ключове.

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",

```



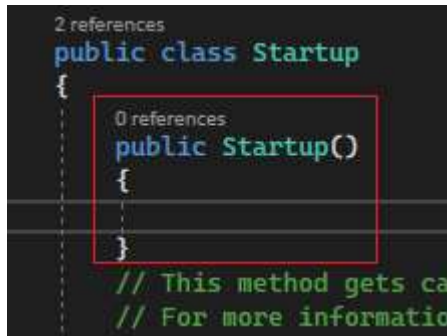
```

        "Microsoft": "Warning",
        "Microsoft.Hosting.Lifetime": "Information"
    },
    "AllowedHosts": "*",
    "MyKey": "Value of MyKey from appsettings.json"
}

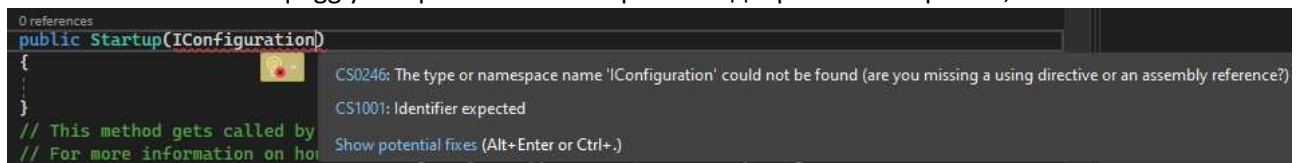
```

Конфигурационната информация в в appsettings.json обикновено се добавя с key value pairs.

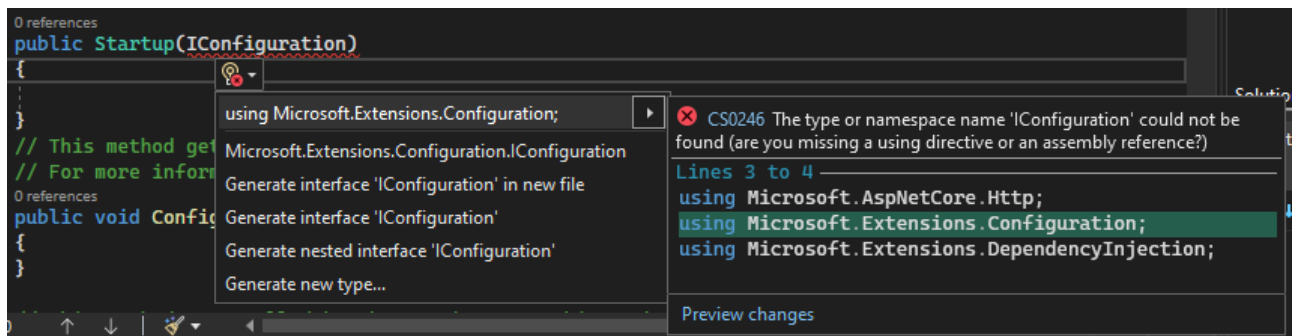
И отиваме да го декларираме в Startup.cs, и за да бъде достъпен, инжектираме IConfiguration service, предоставен от framework в Startup class за работа с ключове. За това ни трябва constructor – ctor + Tabx2 и съответно автоматично вгражда public Startup() {}



Red squiggly – червено зигзагообразно подчертаване за грешка,



което с на натиска на клавишна комбинация **Ctrl + .** (period) извежда съвети за корекциите.



Съответно при натискане на 'Enter' автоматично коригира с добавяне на липсващото namespace в началото


```

4  using Microsoft.Extensions.Configuration;
5  using Microsoft.Extensions.DependencyInjection;
6  using Microsoft.Extensions.Hosting;
7  using System;
8  using System.Collections.Generic;
9  using System.Linq;
10 using System.Threading.Tasks;
11
12 namespace WebApplication15
13 {
14     2 references
15     public class Startup
16     {
17         0 references
18         public Startup(IConfiguration)
19         {
20             //
21         }
22     }
23 }

```

interface Microsoft.Extensions.Configuration.IConfiguration
Represents a set of key/value application configuration properties.

Въвеждаме променлива config и съответно автоматично генерираме private field поле от паметта за нея с Ctrl + . Обръщаме внимание, че използвайки зависимото инжектиране, за да вмъкнем IConfiguration service, а в предишните версии на .NET (под 2.2) е използван външен framework като Ninject StructureMap и др. В .net Core, той е цялостна (интегрална) част от програмата.

```

using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace EmployeeManagement
{
    2 references
    public class Startup
    {
        private IConfiguration _config; // auto generated private field;

        0 references
        public Startup(IConfiguration config) //+ auto MS namespace, parameter named config
        {
            _config = config; // to store this parameter _config is private field
        }
    }
}

```

```

public Startup(IConfiguration config)
{
    _config = config;
}

```

Извикваме изписването на екрана:

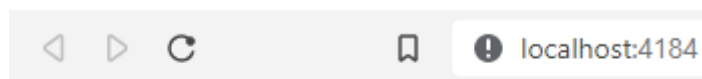
```

await context.Response.WriteAsync(_config ["MyKey"]);
await context.Response.WriteAsync(_config ["MyKey"]);

```

Резултат: Value of MyKey from appsettings.json

Резултат:



Value of myKey from appsettings.json

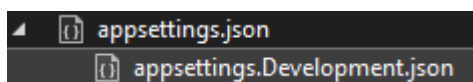
Готово – направихме конфигурационни настройки

* appsettings.json

По същият начин може да направим environment specific app settings file – appsettings.{Environment}.json, например:

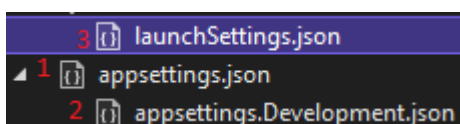
- + appsettings.Development.json,
- + appsettings.Production.json
- + appsettings.Stage.json

Важно е да запомним, че тези специфични настройки ще пренапишат основния файл appsettings.json



Т.е. свойствата, зададени в appsettings.Development.json са с по-висок приоритет от appsettings.json.

```
}  
  "MyKey": "Value of MyKey from appsettings.Development.json"  
}
```



Ако се включи и environment variable, тя става с най-висок приоритет и нея показва браузъра.

```
"ASPNETCORE_ENVIRONMENT": "Development",  
"MyKey": "Value of MyKey from launchsettings.json as Environment variable"  
}  
},
```

С максимален приоритет е CLI.

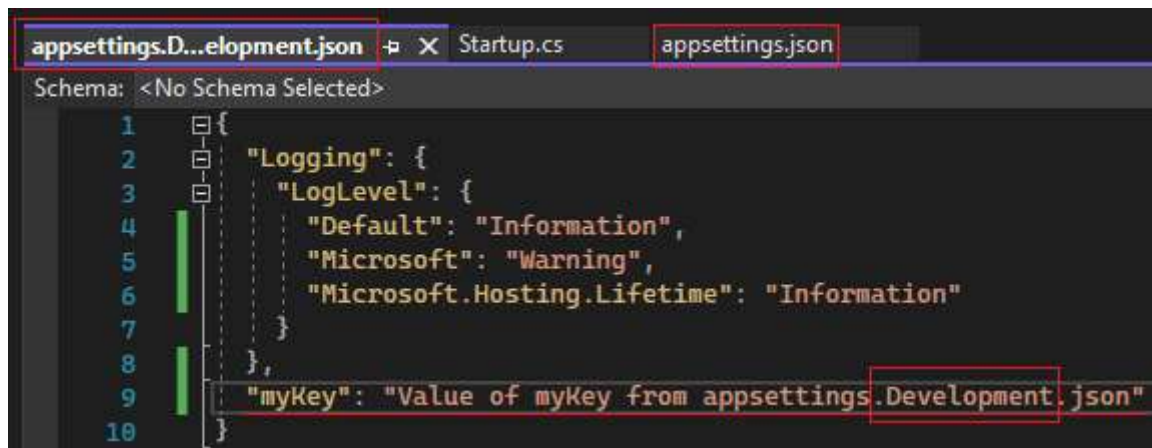
Към момента имаме едни и същи settings: MyKey на 3 места. Команда в CLI в папката на програмата:

```
dotnet run MyKey="The value from Command Line"
```

Нека докажем, че настройките във специфичния appsettings.Development.json ще пренапишат основния appsettings.json. Копираме и задаваме да се покаже стойността в Development файла

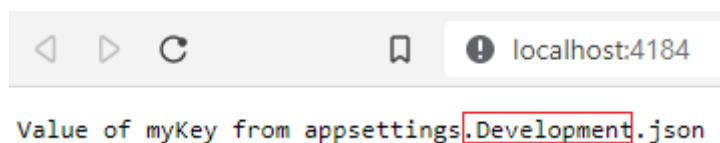
}, Слагаме запетайка преди ключа и необходимия ред код е преди последната фигурна скоба }

Тук съм скрийншотвала по-стар опит, в който вместо MyKey, съм използвала myKey.

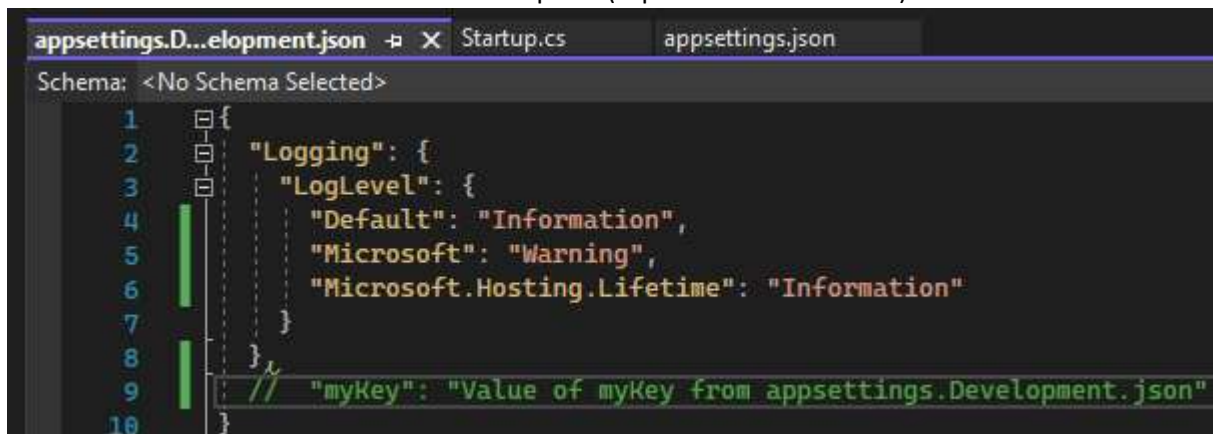


```
1 {
2   "Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft": "Warning",
6       "Microsoft.Hosting.Lifetime": "Information"
7     }
8   },
9   "myKey": "Value of myKey from appsettings.Development.json"
10 }
```

Резултат: Development закрива основния .json:

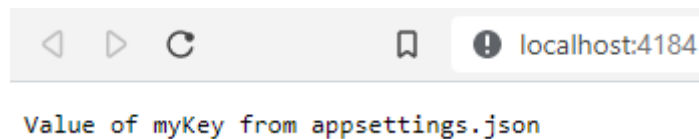


Ако го изкоментираме (скрием за изпълнение)

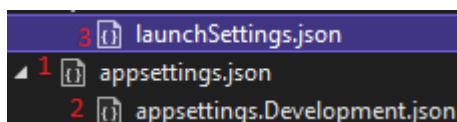


```
1 {
2   "Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft": "Warning",
6       "Microsoft.Hosting.Lifetime": "Information"
7     }
8   },
9   // "myKey": "Value of myKey from appsettings.Development.json"
10 }
```

И рефрешнем брауъра (дори без safe), appsettings.json става отново основен по подразбиране



Като допълнение към файловете, информацията за конфигурациите идва и от потребителските user secrets, променливите на средата environment variables, които съхраняваме в локалния компютър в launchSettings.json



Обръщаме внимание, че в него вече имаме променливи на средата:

```
"environmentVariables": {
  "ASPNETCORE_ENVIRONMENT": "Development"
}
```

Сега ще включим друга променлива на средата със същия ключ като използвания вече myKey и сменим източника, от който да я вземем:

```
"environmentVariables": {
  "ASPNETCORE_ENVIRONMENT": "Development",
  "myKey": "Value of myKey from Environment Variable"
}
```

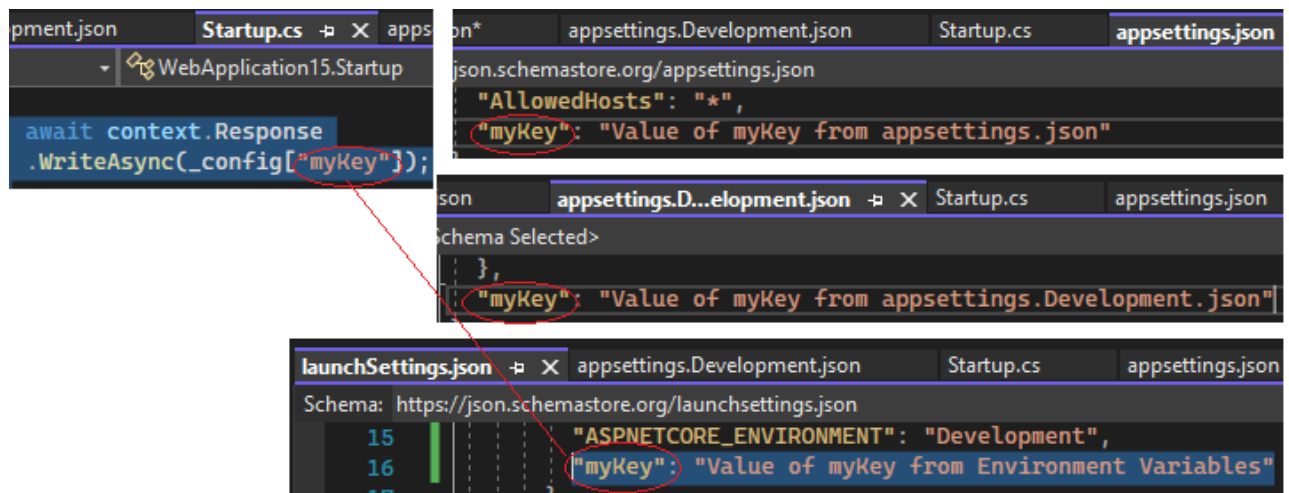
Важно е да запомним, че този IConfiguration service предоставен от framework, ще прочете конфигурационните файлове в този ред като последователност:

- 1) Files appsettings.json
 - appsettings.{Environment}.json
- 2) User secrets
- 3) Environment variables – launchsettings.json
- 4) Command-line arguments CLI (max)

Коеето означава, че Command-line arguments могат да презапишат environment variables, те от своя страна user secrets, те са с приоритет пред appsettings.{Environment}.json, а той засенчва генералния appsettings.json, т.е. по-късно създадените конфигурации се изпълняват с предимство пред вградените по подразбиране.

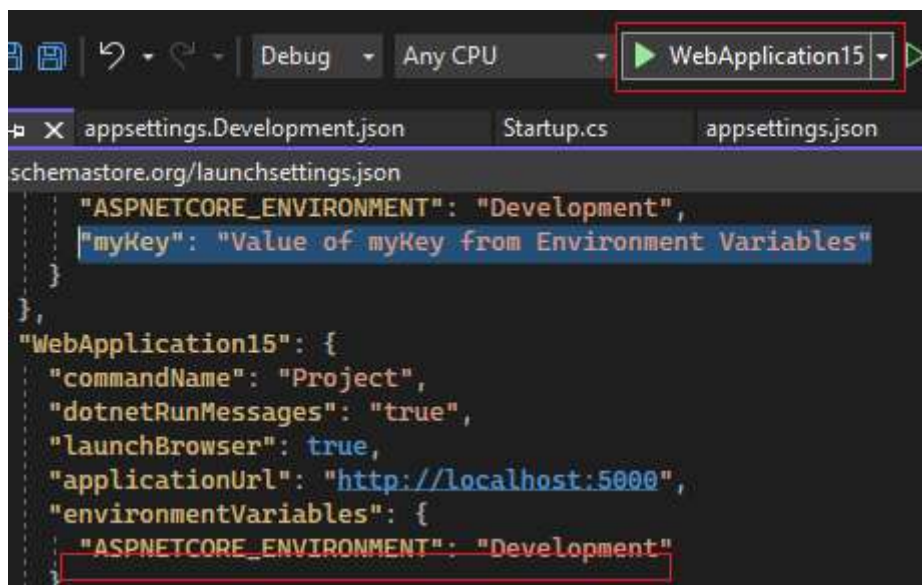
Нека го докажем:

Към момента разполагаме с 3 конфигурации от зададената последователност: appsettings.json, appsettings.{Environment}.json и environment variables. Това означава, че когато махнем коментара, Environment variable трябва да се изпълни вместо всички останали с IIS Express.



Самият .Net core явно подрежда и отворените подпрозорци по приоритет?

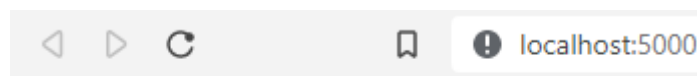
Ако същата програма стартирам с WebApp15, понеже нямам зададен ключ myKey в нея, ще се изпълни



Value of myKey from appsettings.Development.json

CLI: `dotnet run myKey="Value from Command Line"`

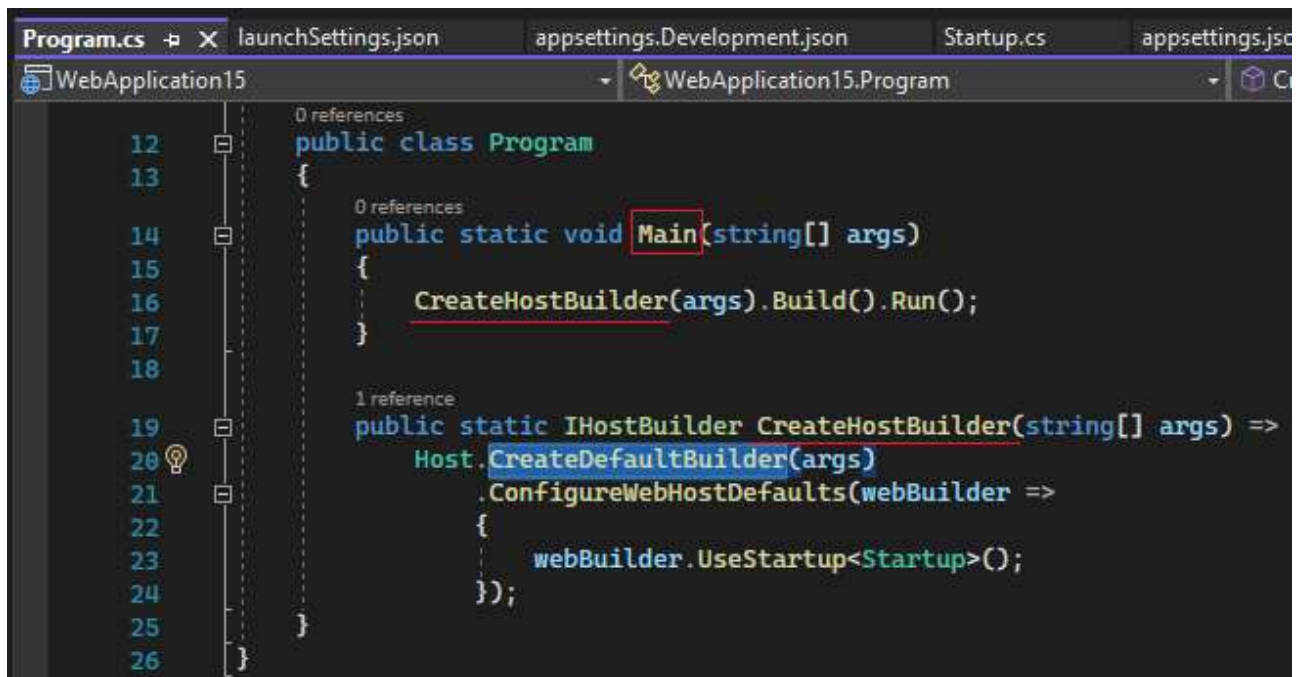
Зареждаме нов подпрозорец в брауъра с друг хостинг 5000, за резултатът от CLI:



Value from Command Line

С което доказваме, че командите в CLI са с най-висок приоритет от всички!

Program.cs файла съдържа Main-метод, който е входната точка на апликацията. Този метод извиква CreateWebHostBuilder, който от своя страна извиква CreateDefaultBuilder – това е методът, който съдържа наборът на разчитане на последователността по подразбиране на всички конфигурационни източници.



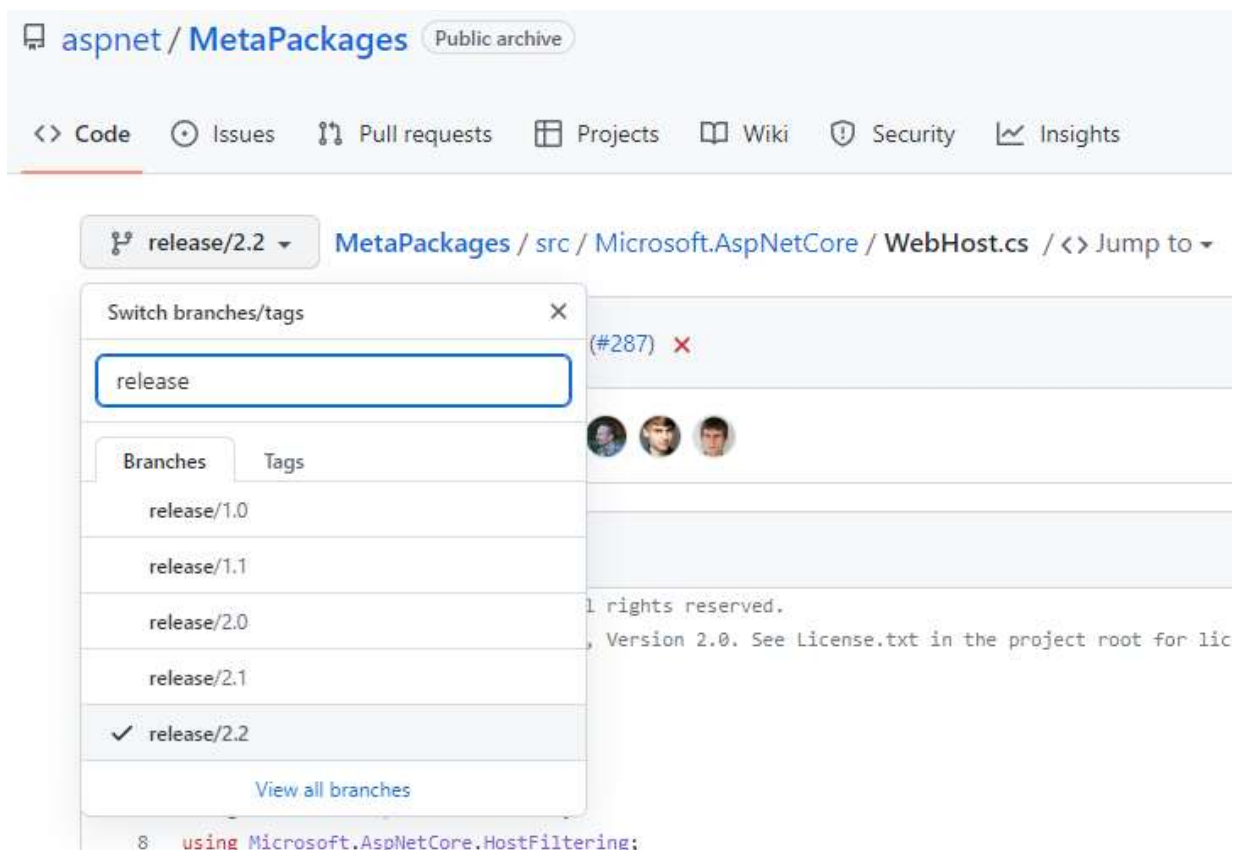
```
12 public class Program
13 {
14     0 references
15     public static void Main(string[] args)
16     {
17         CreateHostBuilder(args).Build().Run();
18     }
19     1 reference
20     public static IWebHostBuilder CreateHostBuilder(string[] args) =>
21     {
22         Host.CreateDefaultBuilder(args)
23             .ConfigureWebHostDefaults(webBuilder =>
24             {
25                 webBuilder.UseStartup<Startup>();
26             });
27     }
```

Понеже .NET core е open source, ние можем да намерим кода на този WebHostClass в github-библиотеката:

Версия 2.2 към момента на видеото:

<https://github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore.WebHost.cs>

Проверих за по-новата 5.0, но няма



aspnet / MetaPackages Public archive

<> Code Issues Pull requests Projects Wiki Security Insights

release/2.2 MetaPackages / src / Microsoft.AspNetCore.WebHost.cs / <> Jump to

Switch branches/tags

release

Branches Tags

- release/1.0
- release/1.1
- release/2.0
- release/2.1
- ✓ release/2.2

View all branches

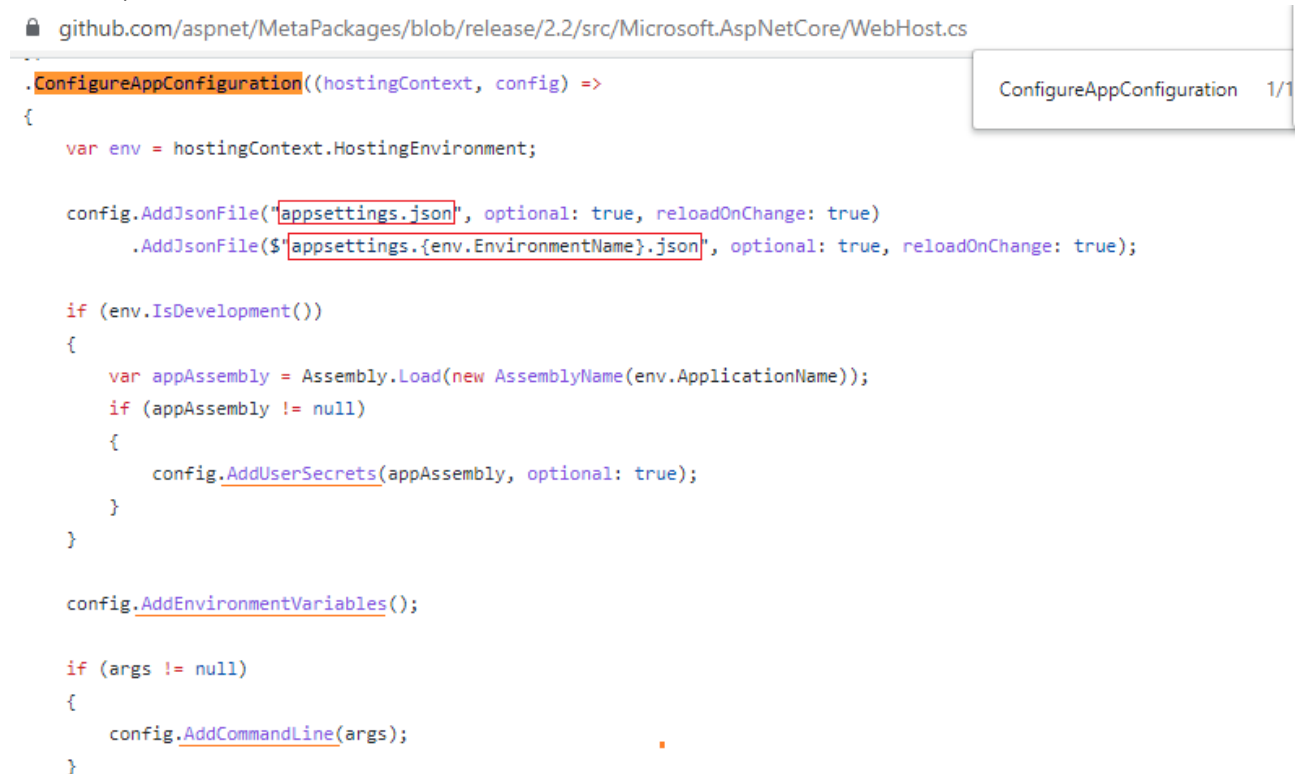
8 using Microsoft.AspNetCore.HostFiltering;

Приоритетът се определя от open-source метод `CreateDefaultBuilder` извикан във файл `Program.cs`

```
public static IHostBuilder CreateHostBuilder(string[] args) =>
    Host.CreateDefaultBuilder(args)
        .ConfigureWebHostDefaults(webBuilder =>
        {
            webBuilder.UseStartup<Startup>();
        });
```

<https://github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore.WebHost.cs>

С търсачката в страницата търсим **ConfigureAppConfiguration** – методът, в който е описана последователността, която се залага по подразбиране за разчитане на всички конфигурационни източници:

The screenshot shows the `ConfigureAppConfiguration` method in `Program.cs`. The method signature is `ConfigureAppConfiguration((hostingContext, config) => { ... })`. The code inside the method block includes:
1. `var env = hostingContext.HostingEnvironment;`
2. `config.AddJsonFile("appsettings.json", optional: true, reloadOnChange: true);`
3. `.AddJsonFile($"appsettings.{env.EnvironmentName}.json", optional: true, reloadOnChange: true);`
4. A conditional block `if (env.IsDevelopment()) { ... }` containing:
 - `var appAssembly = Assembly.Load(new AssemblyName(env.ApplicationName));`
 - `if (appAssembly != null) { ... }`
 - `config.AddUserSecrets(appAssembly, optional: true);`
5. `config.AddEnvironmentVariables();`
6. A conditional block `if (args != null) { ... }` containing `config.AddCommandLine(args);`
A tooltip on the right side of the screenshot shows the text `ConfigureAppConfiguration 1/1`.

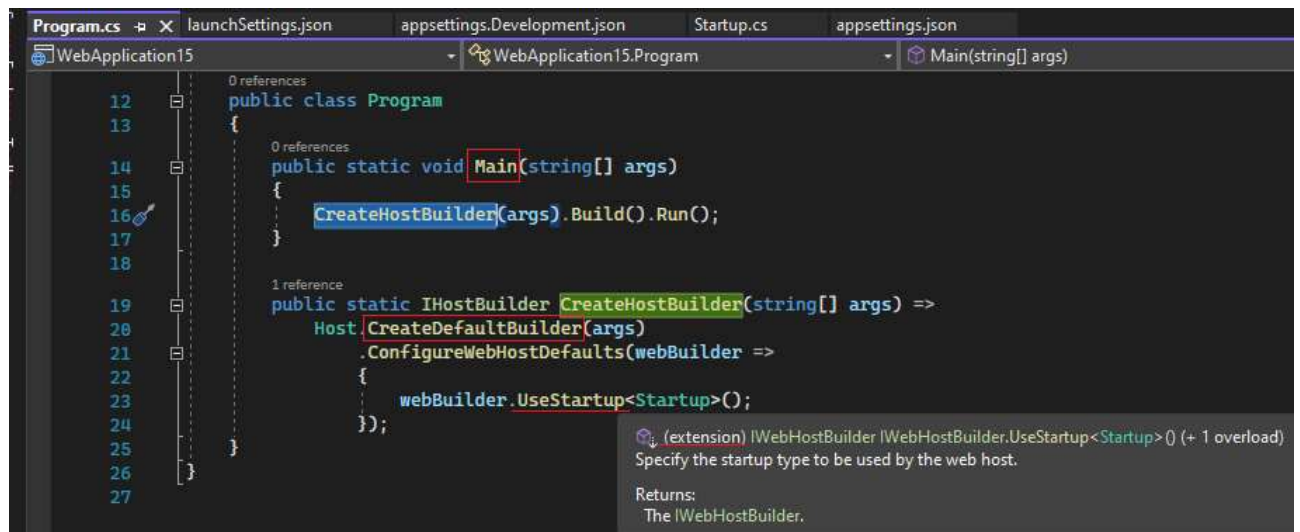
Ако искаме, можем да променим тази последователност и дори да добавим наш собствен конфигурационен източник.

10 - Middleware

Middleware – междинен софтуер, доста широк термин; парче софтуер, което може да обработи http-заявка или отговори на даден компонент със специално предназначение, например за аутентификация на потребител, друг междинен софтуер за прихващане на грешки, друг да подава статични файлове като JavaScript-файлове, CSS-файлове, картинки и т.н. Използваме междинен софтуер за настройване на съобщителния канал pipeline, който определя как да се обработват заявките.

.Net проектът стартира `Program.cs`, с отправна точка главния `Main`-метод, който извиква клас `Startup`, дефиниран във файла със същото име `Startup.cs`.

Main-методът извиква CreateDefaultBuilder, който от своя страна настройва WebHost по подразбиране. За да настроим всичко това, използваме споменатият клас <Startup>, извикан чрез .UseStartup-разширението.

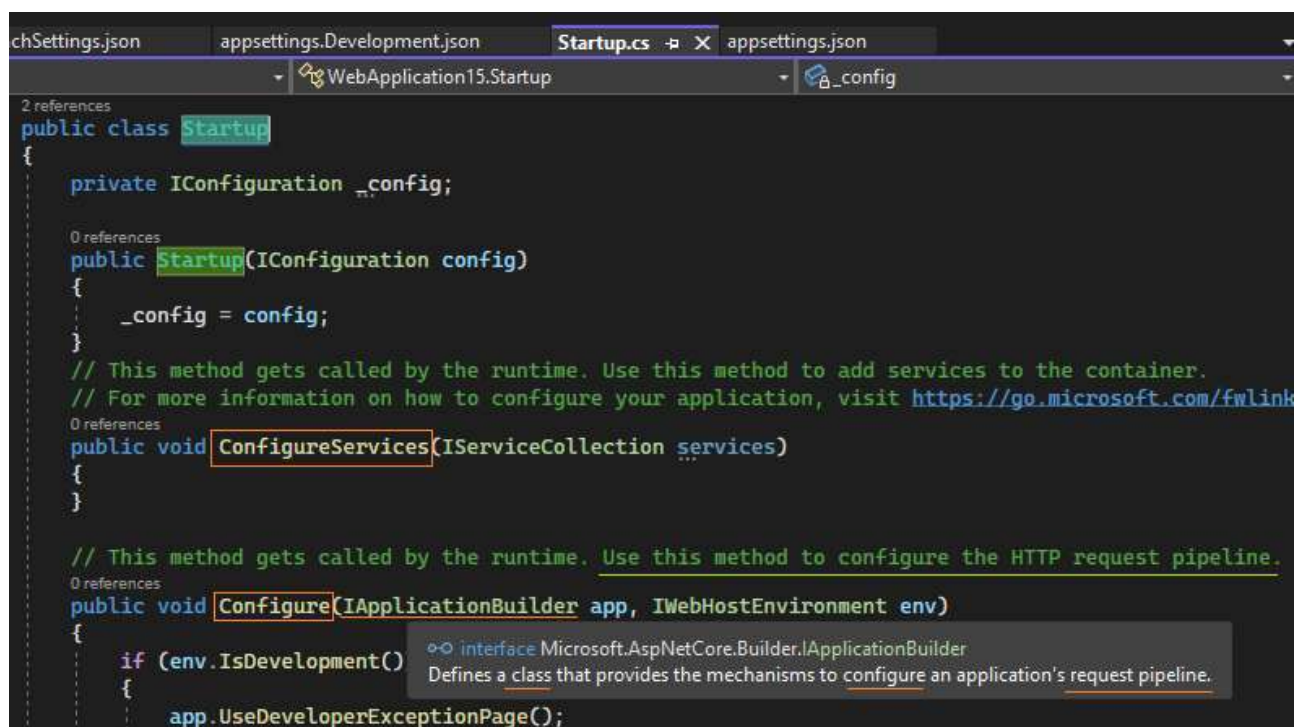


```
12 public class Program
13 {
14     public static void Main(string[] args)
15     {
16         CreateHostBuilder(args).Build().Run();
17     }
18
19     public static IHostBuilder CreateHostBuilder(string[] args) =>
20     {
21         Host.CreateDefaultBuilder(args)
22             .ConfigureWebHostDefaults(webBuilder =>
23             {
24                 webBuilder.UseStartup<Startup>();
25             });
26     }
27 }
```

(extension) IWebHostBuilder IWebHostBuilder.UseStartup<Startup>() (+ 1 overload)
Specify the startup type to be used by the web host.
Returns:
The IWebHostBuilder.

Във файла Startup.cs имаме клас с неговото име Startup, който управлява 2 метода: ConfigureServices + Configure.

Configure-методът конфигурира обработката на заявките по съобщителния канал pipeline.



```
public class Startup
{
    private IConfiguration _config;

    public Startup(IConfiguration config)
    {
        _config = config;
    }

    // This method gets called by the runtime. Use this method to add services to the container.
    // For more information on how to configure your application, visit https://go.microsoft.com/fwlink
    public void ConfigureServices(IServiceCollection services)
    {
    }

    // This method gets called by the runtime. Use this method to configure the HTTP request pipeline.
    public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
    {
        if (env.IsDevelopment())
        {
            app.UseDeveloperExceptionPage();
        }
    }
}
```

interface Microsoft.AspNetCore.Builder.IApplicationBuilder
Defines a class that provides the mechanisms to configure an application's request pipeline.

Кодът в този Configure-метод е генериран автоматично от Empty-project template. Той задава много проста обработка на заявките по съобщителния канал със само два middleware: 1) UseDeveloperExceptionPage и 2) UseEndpoints, който във 2.2 е Run в този IApplicationBuilder-интерфейс:

```

public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }

    app.UseRouting();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapGet("/", async context =>
        {
            await context.Response
                .WriteAsync(_config["myKey"]);
        });
    });
}

```

```

// This method gets called by the runtime. Use this method to configure the HTTP request pipeline.
public void Configure(IApplicationBuilder app, IHostingEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }

    app.Run(async (context) =>
    {
        await context.Response
            .WriteAsync(_config["MyKey"]);
    });
}

```

В този си вид, единственото, което може да направи нашето приложение, е да напише съобщение (чрез `_config` към `.Response`-обект) което да бъде визуализирано през браузъра.

* Middleware компонентите имат достъп до входящи заявки и изходящи отговори, така че сливащият компонент прихваща входящата заявка, придвижва я по съобщителния канал, където я обработва следващото парче код.

Middleware in ASP.NET Core



Middleware in ASP.NET Core

- Has access to both Request and Response
- May simply pass the Request to next Middleware
- May process and then pass the Request to next Middleware
- May handle the Request and short-circuit the pipeline
- May process the outgoing Response
- Middlewares are executed in the order they are added

Например Logging-middleware е първото парче код в съобщителния канал на нашата апликация, така че, когато заявката пристигне от уеб-сървъра, в това междинно парче код регистрира времето на получаване и го прехвърля към следващото междинно парче код (в нашия случай до статичните файлове за по-нататъчна обработка).

* Също може да откаже по-нататъчна обработка, ако има условията за това, което се нарича short-circuiting (къса верига) по съобщителния канал pipeline. Това скъсване често е предпочитано, защото избягва излишна обработка.

Например, ако заявката е за статичен файл (картинка, CSS, JS), ще се извика късата верига и приключи, без да се предаде за обработка към MVC кодовете.

* Може да откаже изцяло каквато и да е обработка, ако издадем заявка към localhost за намаляване на служителите, ако основно искаме да извлечем списъка на всички служители, така че, ако тази заявка пристигне от уеб-сървъра, първото парче код в нашия съобщителен канал, което е Logging, ще отчете времето на пристигане на заявката, ще маркира, че това не е статичен файл, който няма нужда от допълнителна обработка, така че ще прескочи кода, предназначен за статични файлове и ще го насочи за допълнителна обработка към активния MVC model-view-controller, който да произведе HTTP-отговор. В този случай съобщителния канал превключва обратно отговора към кода за обработка на статичните файлове, който от своя страна няма интерес да ги обработва по-нататък, така че просто оставя отговора да премине към следващото парче код, което е Logging, което просто отчита времето на връщане на отговора на обработената заявка.

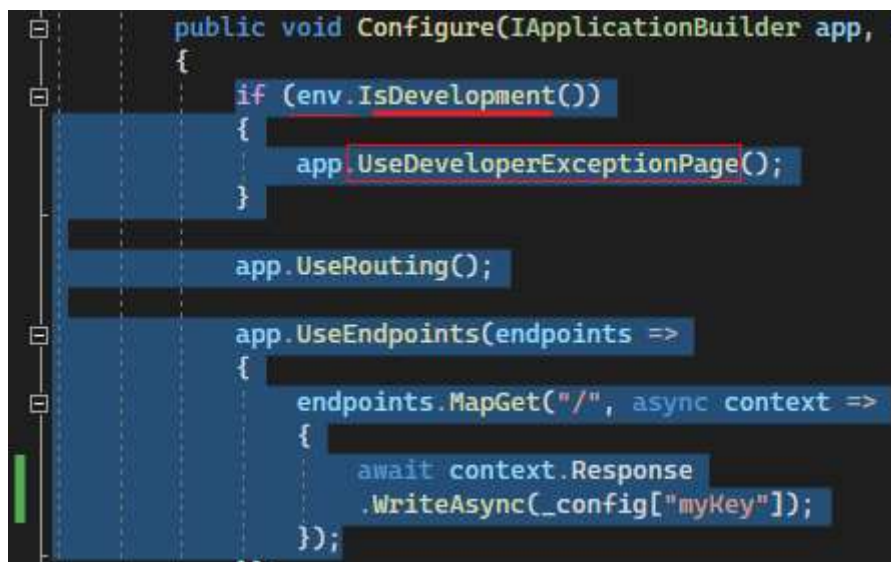
Важно е да се запомни, че Logging-кода отчита двете времена – на входящата заявка и отговорът от обработката, така че това може да се използва за изчисление на общото време, необходимо за услугата и после този отговор да се предостави на уеб-сървъра, който от своя страна да върне отговор към клиента, който е поискал това.

* Поради всичко това е важно парчетата код да се извикват едно друго в правилна последователност. Тези парчета код са достъпни като NuGet-packages, което означава, че ъпдейтите са NuGet-парчета код, позволяващи ни да обновяваме всяко парче междинен код поотделно. В зависимост от изискванията на приложението, ние можем да добавяме повече или по-малко парчета междинен код към съобщителния канал за обработка на заявки.

Например: ако изграждаме просто уеб-приложение с малко статични .html страници и картинки, тогава съобщителният канал може да съдържа само междинен код за обработката им. От друга, страна, ако изграждаме защитено с данни уеб-приложение, ще имаме нужда от няколко междинни кодови компонента за обработки на статични файлове, ауторизации, лични данни, MVC.

* Имаме пълен контрол върху съобщителния канал. Това също означава, че от гледна точка на паметта и производителността, че ще се обработват само междинните парчета код, необходими за заявките на приложението. Накратко може да обобщим, че само всяко от ангажираните междинни парчета код има достъп едновременно до входящите заявки и изходящи отговори. Междинният код може просто да прехвърли обработката до следващото парче код, може да извърши някаква обработка и тогава да го предаде нататък, или пък изцяло да откаже обработка по къса верига. Парчетата междинен код се изпълняват в последователността, която е заложена при създаването им.

11 – Configure request processing pipeline



```
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }

    app.UseRouting();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapGet("/", async context =>
        {
            await context.Response
                .WriteAsync(_config["myKey"]);
        });
    });
}
```

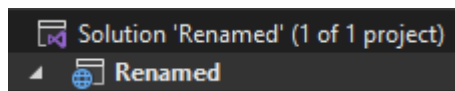
Имаме междинен код методът `UseDeveloperExceptionPage`, който ще се изпълни, ако имаме изключение и ако `Environment = IsDevelopment ("ASPNETCORE_ENVIRONMENT": "Development")` от `LaunchSettings.json`

Другият метод, който имаме `UseEndPoints` може само да изведе съобщение на екрана, използвайки `.Response` обекта. Към момента, това е парчето междинен код, което отговаря на почти всички заявки, независимо каква е частта от заявката или изискванията към нея, всичко се обработва от `.Response` и отговорът, който получаваме, е съобщението, което изписваме към обекта `_config`.

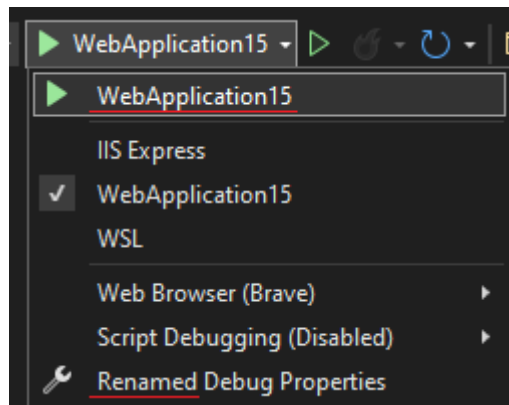
Нека опростим нещата със съобщение „Hello world” вместо ключът `myKey`.

Решавам да копирам и преименувам проекта така, че да стартира.

Copy->Paste цялата папка, после отварям с VS .sln, д.б.->rename на двете места

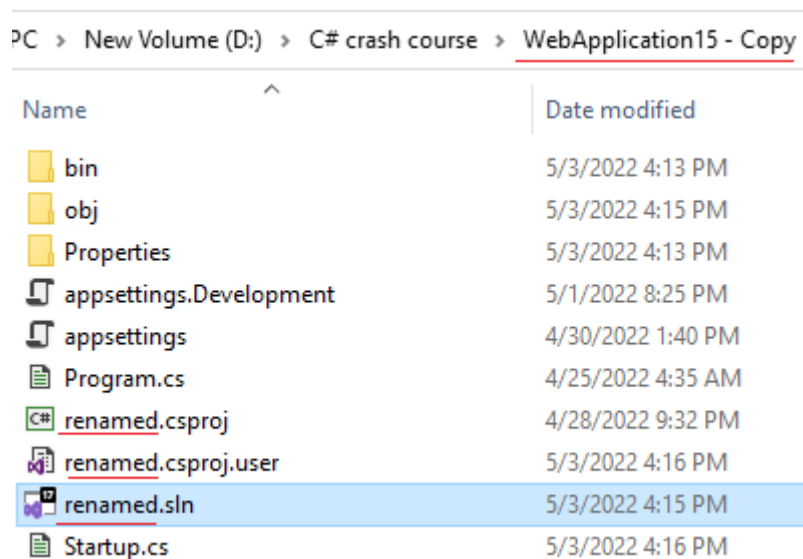


Самата папка с проекта се използва по-дълго, но тя може да се преименува извън VS. Ако преименувам ръчно чрез WExplorer файловете, .sln дава грешки при отваряне на VS.

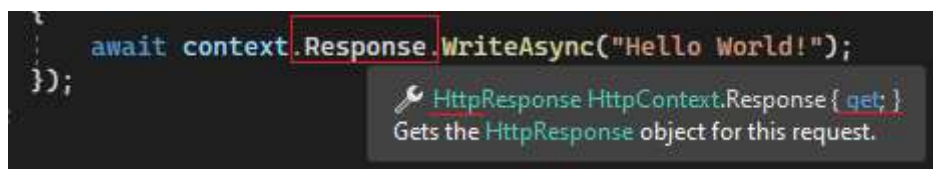


Единствено в дебъгера засега се забелязва старото име, но го оставям така.

И с двата сървъра IIS Express и WebApplication15 се стартира.



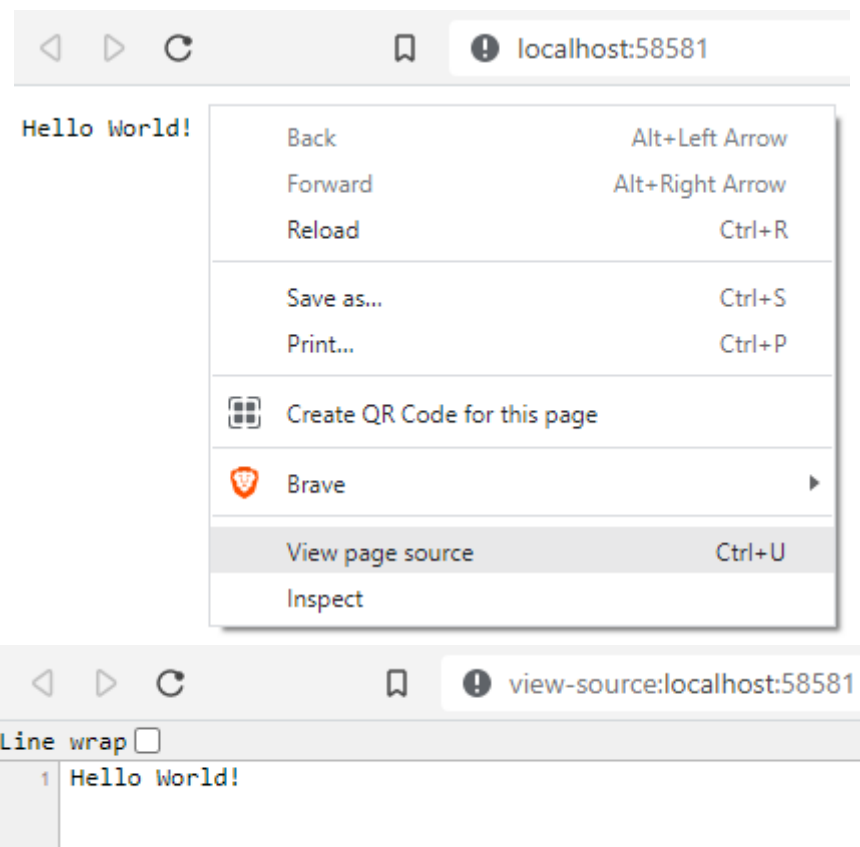
Отговорът на middleware е html-страница с обикновен текст, който, ако проверим с д.б. в браузъра, няма тагове.



* Сменяме текста с „Hello world!“ за простота и с IIS Express извежда съобщението в браузъра, при WebApplication15 обаче стои старото съобщение – ще погледна после защо.

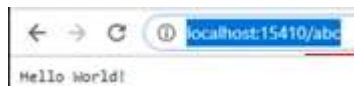
<https://www.youtube.com/watch?v=1AVl6MzHz88>

При д.б. View source в браузъра виждаме, че няма никакъв HTML код:

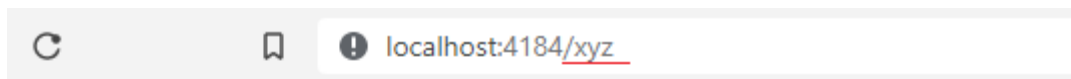


Правя нов solution с име WebApp11_pipeline и съответно заради това localhost е с различни номера, които няма в момента да коригирам.

В tutorиъла към този момент няма никакво значение какъв адрес зададем в адресния бар след базовия localhost:4184, който се изпълнява – все се вижда текстовото съобщение.



Моят браузър обаче извежда съобщение за грешка:



This localhost page can't be found

No webpage was found for the web address: <http://localhost:4184/xyz>

HTTP ERROR 404

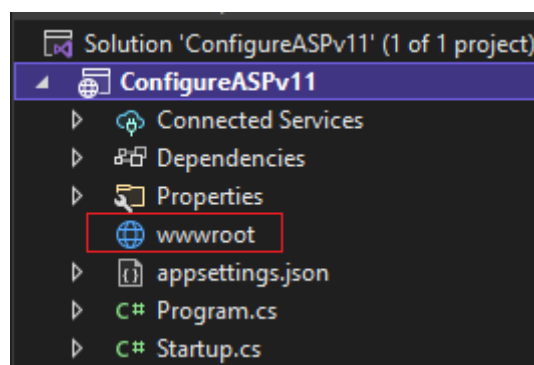
Reload

Правя си нов проект за експеримента – дава същата грешка, така че тук имаме новост във версията на браузъра или програмата, все тая. Дори при новата ОС windows 10, пак е по същият начин.

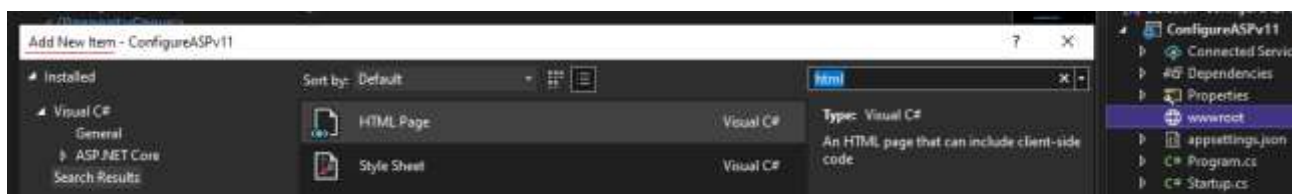
Идеята е, че това е междинно парче код, което отговаря на всеки отговор на заявка:

```
endpoints.MapGet("/", async context =>
{
    await context.Response.WriteAsync("Hello World!");
});
```

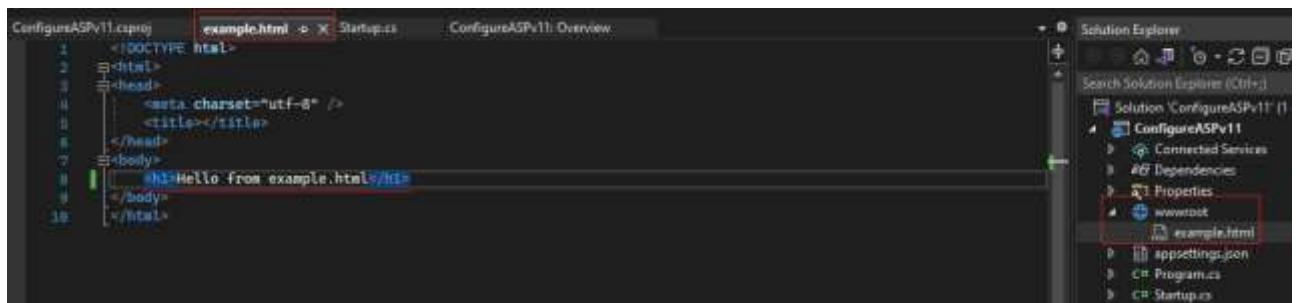
Дори ако имаме физически файл с име full.html – д.б. върху проекта във VS -> Add -> New Folder -> име wwwroot



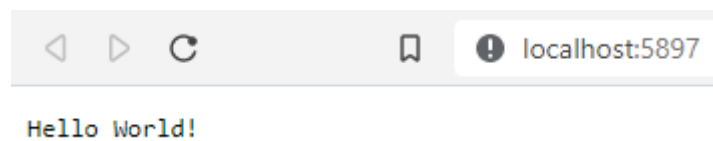
В самата папка Add -> New item -> (search bar) html -> HTML Page -> име example.html



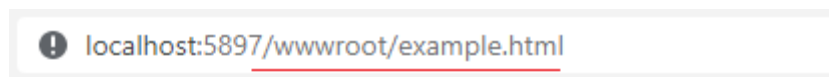
В който експериментално въвеждам текст.



При стартиране се изпълнява старото съобщение:



Независимо кой адрес задам в браузъра, знаейки, че имаме този физически файл да съществува с адрес `wwwroot/example.html`, отново изкарва съобщението от `UseEndpoints`



This localhost page can't be found

No webpage was found for the web address:

`http://localhost:5897/wwwroot/example.html`

HTTP ERROR 404

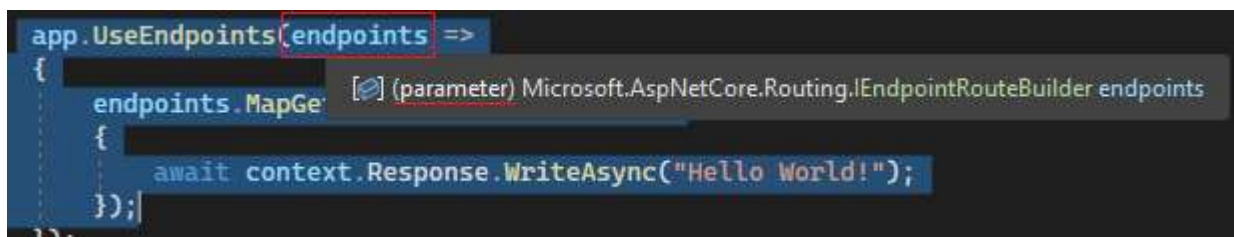
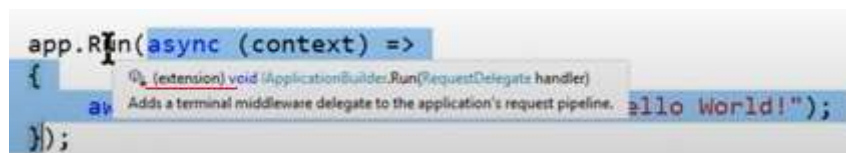
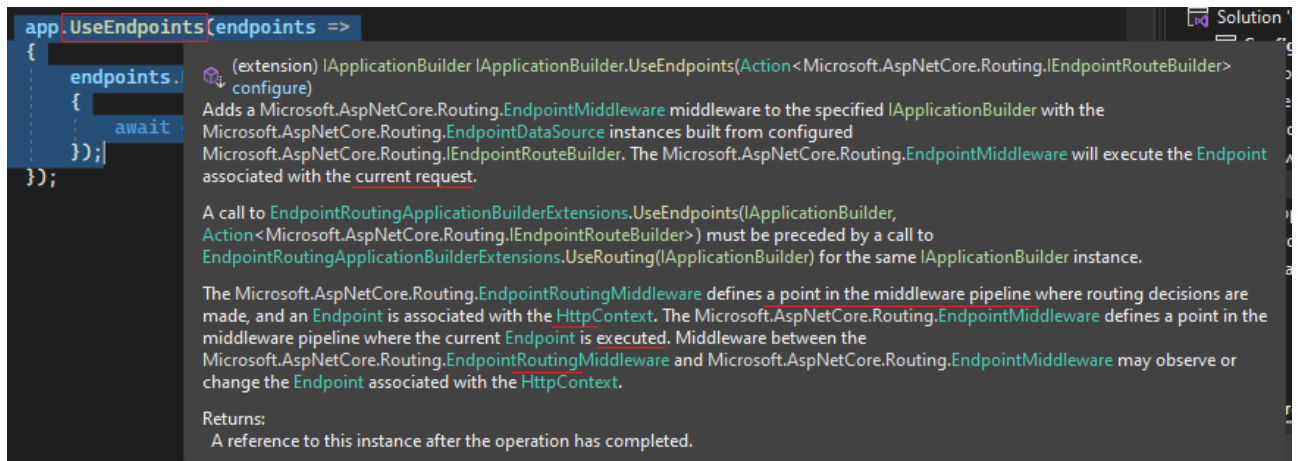
Reload

Демонстрацията е, че всяка заявка, която идва независимо откъде, се обработва от това междинното парче код `middleware`, което се намира в `startup.cs`

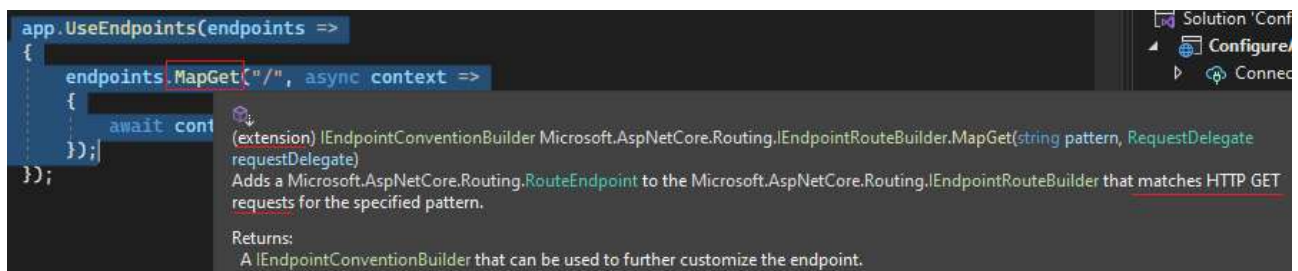
```
app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("Hello World!");
    });
});
```

Към този момент, програмата няма възможности да обработва статични файлове.

Метод UseEndpoints (старо име Run) е изходната точка с резултата от нашата заявка в html-визуализиран вид.

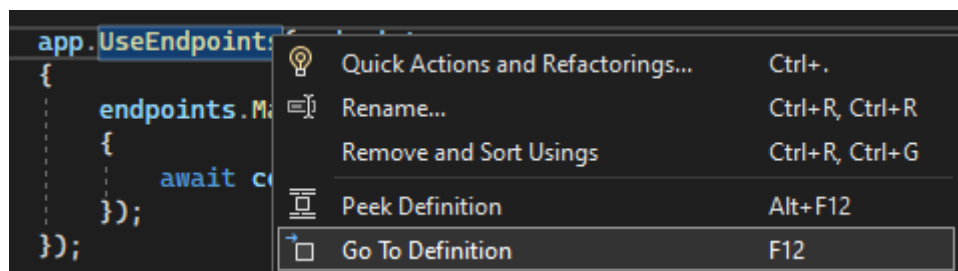


Параметърът е RequestDelegate – пратеник, който приема request HTTP context object като параметър.



```
public delegate Task RequestDelegate(HttpContext context);
```

Нека разгледаме зададената дефиниция на UseEndpoints ->д.б.->Go To Definition



Резултат:

```

76 public static IApplicationBuilder UseEndpoints(this IApplicationBuilder builder, Action<IEndpointRouteBuilder> configure)
77 {
78     (extension) IApplicationBuilder UseEndpoints(this IApplicationBuilder builder, Action<IEndpointRouteBuilder> configure)
79     if false // Decompilation Log

```

Дефинициите (Go to Definition F12) са в контекстното меню с подпроцорци с дясно подреждане и затваряне, съответно там не са позволени редакции за нов ред с Enter и има много свити описателни коментари.

```

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("Hello world!");
    });
});

```

[?] (parameter) HttpContext context

С нов д.б.->Go To Definition виждаме, че context object дава възможност за входящ и изходящ контрол на данните от заявките през http чрез .Response – property.

```

namespace Microsoft.AspNetCore.Http
{
    ...public abstract class HttpContext
    {
        ...public abstract IFeatureCollection Features{...}
        //
        // Summary:
        //     Gets the Microsoft.AspNetCore.Http.HttpRequest object for this request.
        public abstract HttpRequest Request{...}
        //
        // Summary:
        //     Gets the Microsoft.AspNetCore.Http.HttpResponse object for this request.
        public abstract HttpResponse Response{...}
    }
}

```

Към момента, това е анонимен метод, използващ Lambda expression – има отделни видеа за нея в подобен туториъл на kudvenkat.

Lambda expressions (C# reference)

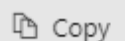
Article • 01/25/2022 • 15 minutes to read • [5 contributors](#)



You use a *lambda expression* to create an anonymous function. Use the [lambda declaration operator](#) `=>` to separate the lambda's parameter list from its body. A lambda expression can be of any of the following two forms:

- [Expression lambda](#) that has an expression as its body:

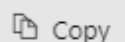
C#



```
(input-parameters) => expression
```

- [Statement lambda](#) that has a statement block as its body:

C#



```
(input-parameters) => { <sequence-of-statements> }
```

To create a lambda expression, you specify input parameters (if any) on the left side of the lambda operator and an expression or a statement block on the other side.

```
app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("Hello World!");
    });
});
app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("Hello from 2-nd middleware!");
    });
});
```

В този пример се прави демонстрация, че се изпълнява само първия `Run`, който е крайна точка в изпълнението на програмата като самостоятелен `Terminal Middleware` (финален, който не извиква за обработка следващия `Middleware`, а се връща към предходния `Middleware`) и не се стига до втория след него, който никога няма да получи възможност да се изпълни.

```
app.Run(async (context) =>
{
    await context.Response.WriteAsync("Hello from 1st Middleware!");
});
```

```
app.Run(async (context) =>
{
    await context.Response.WriteAsync("He
});
```

Terminal Middlware

в 5.0 обаче е защитено от подобни грешки.



An unhandled exception occurred while processing the request.

AmbiguousMatchException: The request matched multiple endpoints. Matches:

Когато модифицирам и дублирам само функцията endpoints, извежда същата грешка.

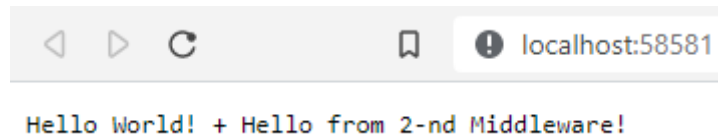
```
app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("Hello World!");
    });
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("Hello from 2-nd middleware!");
    });
});
```

Когато обаче дублирам реда за изписване на съобщение, и двете се извеждат на екрана:

```
app.UseRouting();

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("Hello World!");
        await context.Response.WriteAsync(" + Hello from 2-nd Middleware!");
    });
});
```

Резултат:



Странното е, че когато добавя метод Use, не се изпълнява изобщо Endpoints. Вероятно трябва да е след `app.UseRouting()`;

```
app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Hello from 1-nd Middleware!");
});

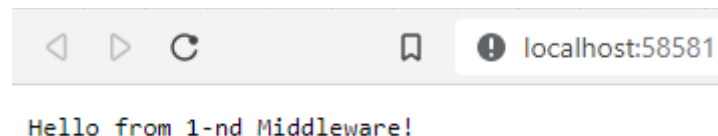
app.UseRouting();

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync(" + Hello from 2-nd Middleware!");
    });
});
```

```
app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Hello 1");
});
```

Само че след Routing, на скрийншота е сгрешено.

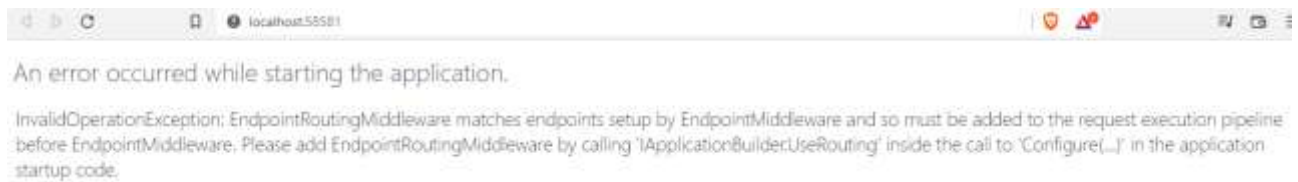
Резултат:



```
app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Hello from 1-nd Middleware!");
});

// app.UseRouting();

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync(" + Hello from 2-nd Middleware!");
    });
});
```

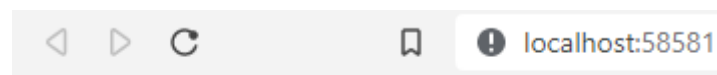



```
app.UseRouting();

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Hello from 1-nd Middleware!");
});

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync(" + Hello from 2-nd Middleware!");
    });
});
```

Отново се изпълнява първото съобщение, без второто. Ще погледна след малко в дебъгера защо.



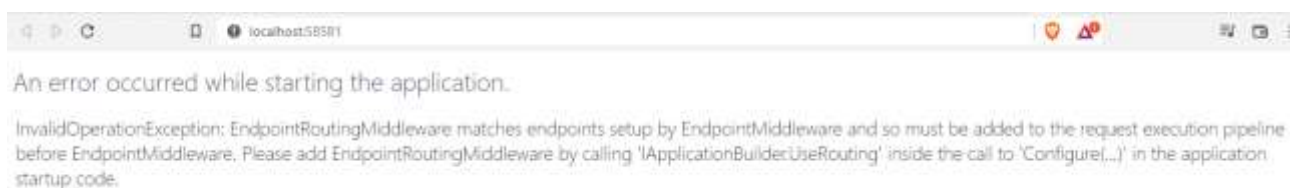
Hello from 1-nd Middleware!

Отново, ако се закоментира, извежда грешка:

```
// app.UseRouting();

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Hello from 1-nd Middleware!");
});

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync(" + Hello from 2-nd Middleware!");
    });
});
```



Това, което е липсвало е next- delegate със синтаксис `await next();`

```

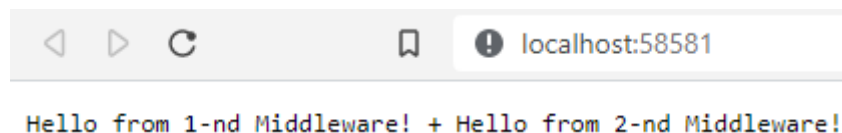
app.UseRouting();

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Hello from 1-nd Middleware!");
    await next();
});

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync(" + Hello from 2-nd Middleware!");
    });
});

```

Излиза, че без него, UseEndpoints не е терминален (финален) като Run в старите версии.



Когато имаме ъпдейти, които не присъстват като middleware components in Pipeline, те са под формата на NuGet packages.

Terminal middleware – последен, не може да се повтаря.

```

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("Hello world!");
    });
});

```

Можем да използваме обикновен Use:

```

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Hello 1");
});

```

Който в старите версии работил като content присвоява като параметър функцията middleware, която следва след него, но в новите извежда само Hello 1, без да се вижда Hello 2. За да се видят и двете в новите, е необходим `await next();`

```

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Hello 1");
    await next();
});

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync(" + Hello 2");
    });
});

```

```

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Hello 1");
    await next();
});

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync(" + Hello 2");
    });
});

```

За да усложним нещата, нека видим логване, което се извиква от Main() от Program.cs, а той извиква CreateHostBuilder

<https://github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore.WebHost.cs>

github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore.WebHost.cs

CreateDefaultBuilder(string 1/1

```

/// <returns>The initialized <see cref="IWebHostBuilder"/>.</returns>
public static IWebHostBuilder CreateDefaultBuilder(string[] args)
{
    var builder = new WebHostBuilder();

    if (string.IsNullOrEmpty(builder.GetSetting(WebHostDefaults.ContentRootKey)))
    {
        builder.UseContentRoot(Directory.GetCurrentDirectory());
    }

    if (args != null)
    {
        builder.UseConfiguration(new ConfigurationBuilder().AddCommandLine(args).Build());
    }

    builder.UseKestrel((builderContext, options) =>
    {
        options.Configure(builderContext.Configuration.GetSection("Kestrel"));
    })
    .ConfigureAppConfiguration((hostingContext, config) =>
    {

```

Виждаме и по-надолу, че конфигурира И logging

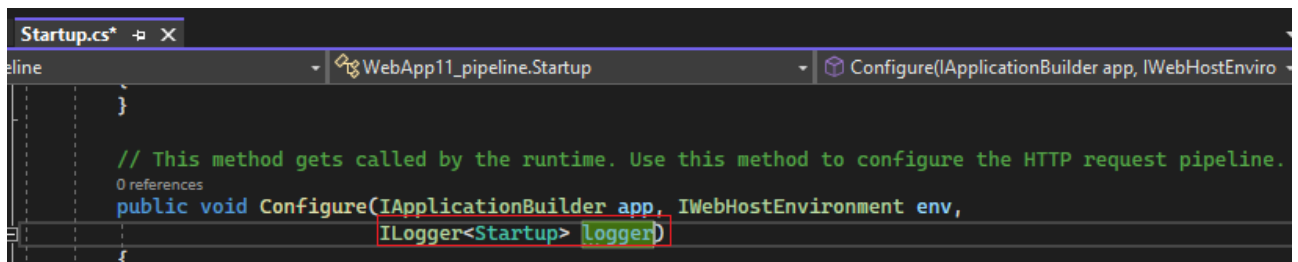
```

})
.ConfigureLogging((hostingContext, logging) =>
{
    logging.AddConfiguration(hostingContext.Configuration.GetSection("Logging"));
    logging.AddConsole();
    logging.AddDebug();
    logging.AddEventSourceLogger();
})
.ConfigureServices((hostingContext, services) =>
{
    //

```

CreateDefaultBuilder(string 1/1

Като допълнение да добавим логър за клас startup с име logger и съответно с Ctrl . - namespace.



```

public void Configure(IApplicationBuilder app, IWebHostEnvironment env,
    ILogger<Startup> logger)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }

    app.UseRouting();

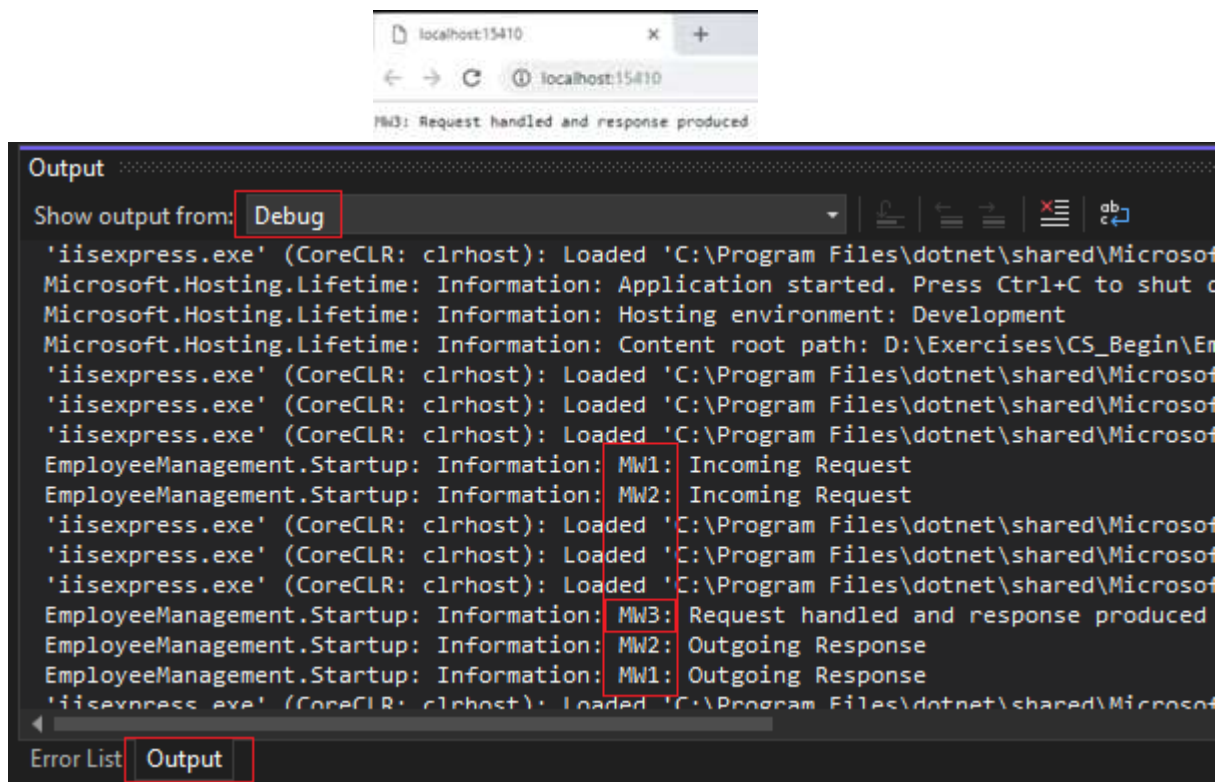
    app.Use(async (context, next) =>
    {
        logger.LogInformation("MW1: Incoming Request");
        await next();
        logger.LogInformation("MW1: Outgoing Response");
    });

    app.Use(async (context, next) =>
    {
        logger.LogInformation("MW2: Incoming Request");
        await next();
        logger.LogInformation("MW2: Outgoing Response");
    });

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapGet("/", async context =>
        {
            await context.Response.WriteAsync("MW3");
            logger.LogInformation("MW3: Request handled and response
produced");
        });
    });
}

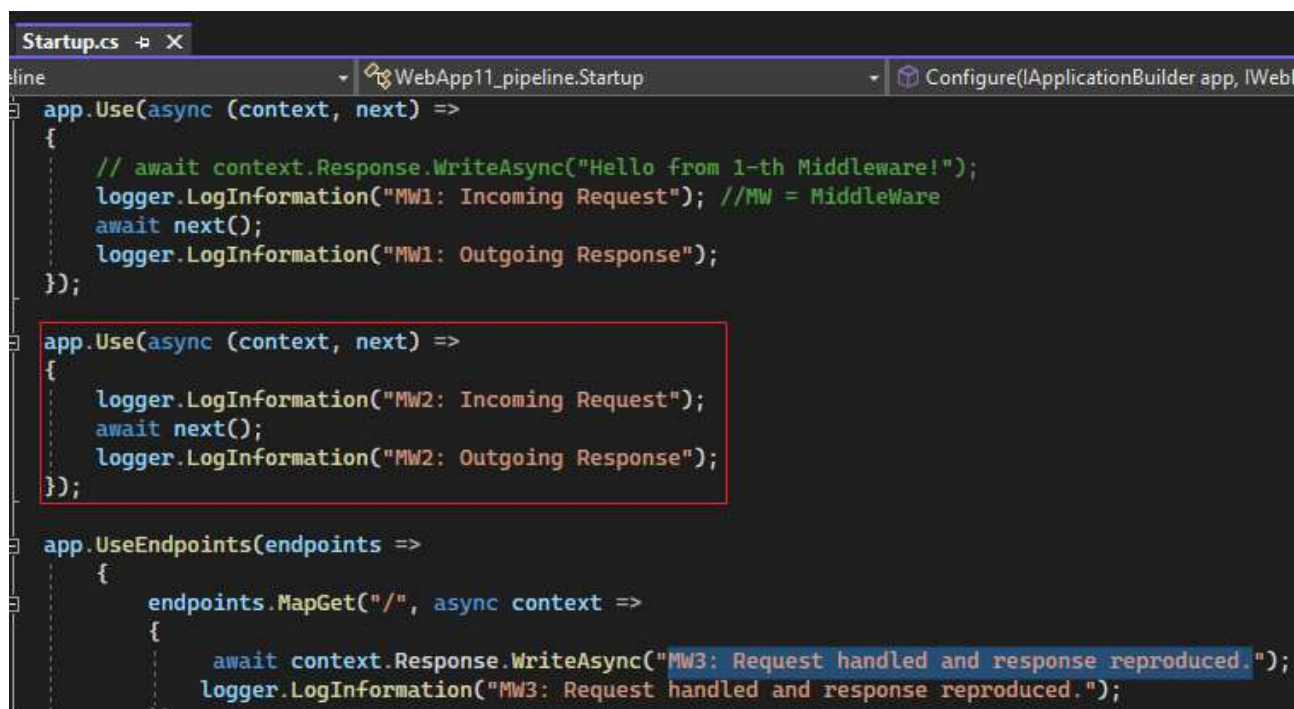
```

Резултат: MW3 (MiddleWare3), но при Debug Само с Ctrl, не Ctrl+F5:

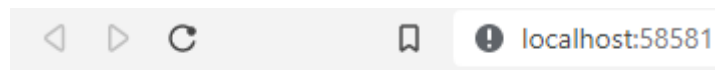


Чак сега си видях друга грешка в съобщението „1-th“ – сменила съм само цифрата “2-nd”.

Няма да скрийншотвам корекции.



Съобщението, което се извежда в брауъра е от WriteAsync:



MW3: Request handled and response reproduced.

Странно защо в дебъгера на туториъла, проследяването е много по-описателно

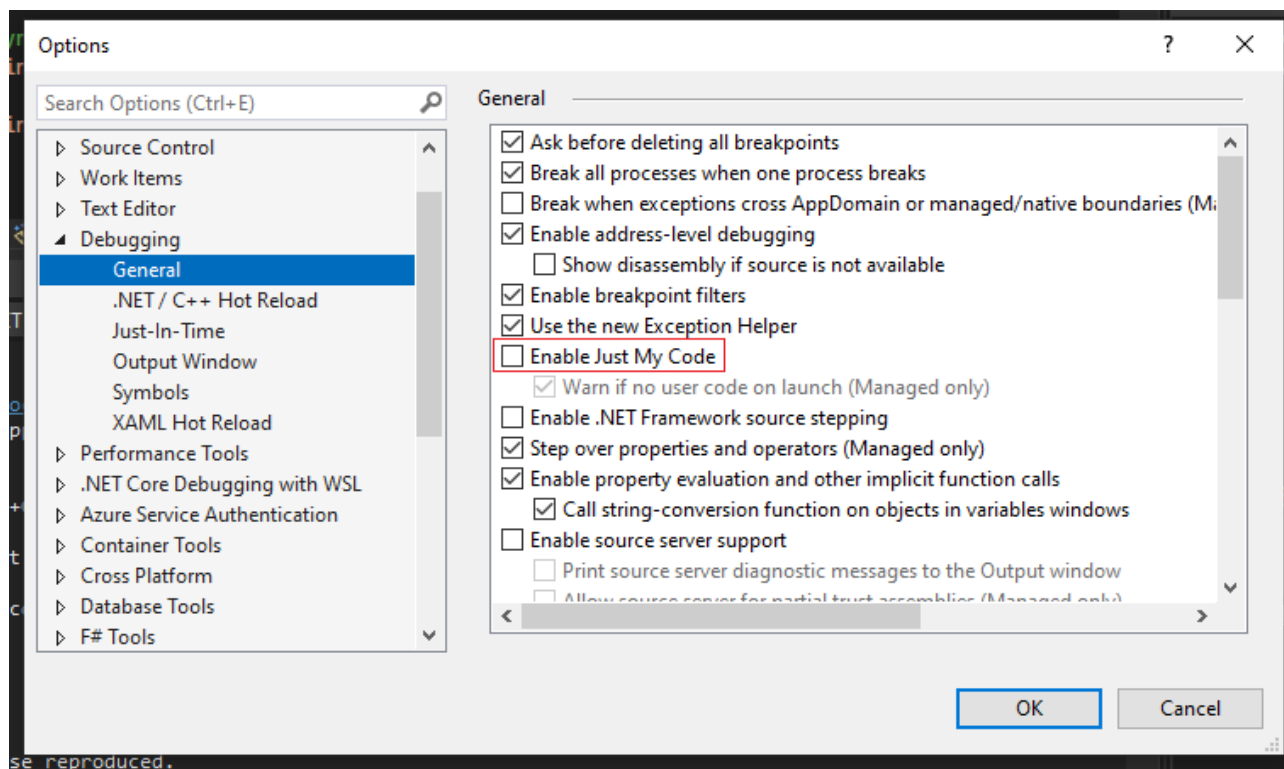
```
Output
Show output from: Debug
EmployeeManagement.Startup:Information: MW2: Outgoing Response
EmployeeManagement.Startup:Information: MW1: Outgoing Response
Microsoft.AspNetCore.Hosting.Internal.WebHost:Information: Request finished in 384.5217ms 200
Microsoft.AspNetCore.Hosting.Internal.WebHost:Information: Request starting HTTP/1.1 GET http://localhost:15410/favicon.ico
EmployeeManagement.Startup:Information: MW3: Incoming Request
EmployeeManagement.Startup:Information: MW2: Incoming Request
EmployeeManagement.Startup:Information: MW3: Request handled and response reproduced
EmployeeManagement.Startup:Information: MW2: Outgoing Response
EmployeeManagement.Startup:Information: MW1: Outgoing Response
Microsoft.AspNetCore.Hosting.Internal.WebHost:Information: Request finished in 95.8463ms 200
The thread 0x1264 has exited with code 0 (0x0).
The thread 0x265c has exited with code 0 (0x0).
The thread 0x1dd0 has exited with code 0 (0x0).
The thread 0x3508 has exited with code 0 (0x0).
The thread 0x418 has exited with code 0 (0x0).
```

Докато при мен “MW3: Request handled and response reproduced.” и всички подобни съобщения просто са заменени с „Just My Code”

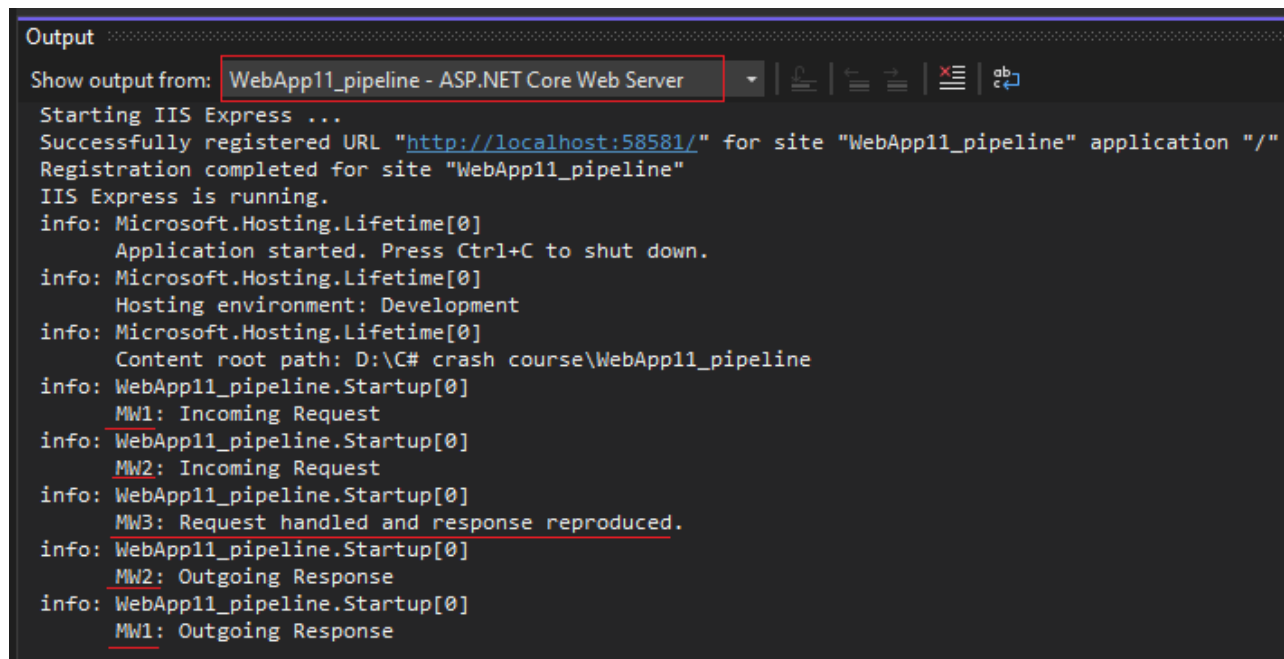
```
Output
Show output from: Debug
etime: Information: Content root path: D:\C# crash course\WebApp11_pipeline
eCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\5.0.16\System
eCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.AspNetCore.App\5.0.16\Microsoft
eCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.AspNetCore.App\5.0.16\Microsoft
eCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.AspNetCore.App\5.0.16\Microsoft
eCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\5.0.16\System
eCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\5.0.16\System
eCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\5.0.16\System
exited with code 0 (0x0).
exited with code 0 (0x0).
exited with code 0 (0x0).
exited with code 0 (0x0).
```

```
debugger option 'Just My Code' is enabled.
the debugger option 'Just My Code' is enabled.
imized and the debugger option 'Just My Code' is enabled.
ized and the debugger option 'Just My Code' is enabled.
option 'Just My Code' is enabled.
bugger option 'Just My Code' is enabled.
and the debugger option 'Just My Code' is enabled.
```

‘Just My Code’ се коригира с Debug -> Options, където се маха отметката на Enable за съобщението.



Но не виждам голям смисъл от това към момента, защото и без да е дебъгнато, с падащото меню се вижда предостатъчно информация как заявката се изпълнява в права и обратна посока:



Сменям едното съобщение, за да добия по-голяма яснота, защото в браузъра се вижда съобщението от WriteAsync, а реално логърите се засичат като изпълнение само.

```

51      {
52          await context.Response.WriteAsync("MW3: Request handled and response reproduced.");
53          logger.LogInformation("MW3: ");
54      }
55  }

```

100 % No issues found Ln: 53 Ch: 53

Output

Show output from: WebApp11_pipeline - ASP.NET Core Web Server

```

Content root path: D:\C# crash course\WebApp11_pipeline
info: WebApp11_pipeline.Startup[0]
      MW1: Incoming Request
info: WebApp11_pipeline.Startup[0]
      MW2: Incoming Request
info: WebApp11_pipeline.Startup[0]
      MW3:
info: WebApp11_pipeline.Startup[0]
      MW2: Outgoing Response
info: WebApp11_pipeline.Startup[0]
      MW1: Outgoing Response

```

Имаме 3 MiddleWare компонента в pipeline, като NEXT играе ключова роля в прескачането:

```

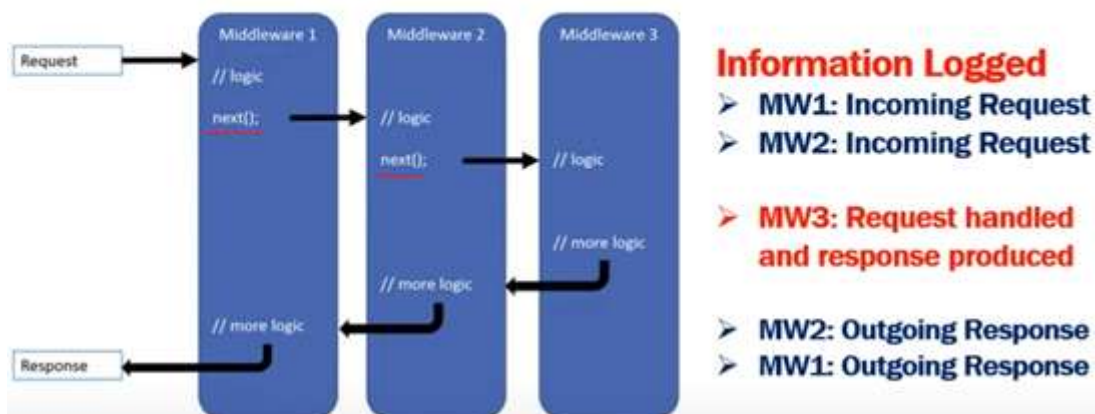
logger.LogInformation("MW1: Incoming Request"); //MW = Middleware
await next();
logger.LogInformation("MW1: Outgoing Response");
});

app.Use(async (context, next) =>
{
    logger.LogInformation("MW2: Incoming Request");
    await next();
    logger.LogInformation("MW2: Outgoing Response");
});

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("MW3: Request handled and response reproduced.");
        logger.LogInformation("MW3: ");
    });
});

```

Configure Request Processing Pipeline



Everything that happens **before the next() method** is invoked in each of the middleware components, happens as the **REQUEST** travels from middleware to middleware through the pipeline and this is represented by the **incoming arrow**

When a middleware handles the request and produces response, the request processing pipeline **starts to reverse**

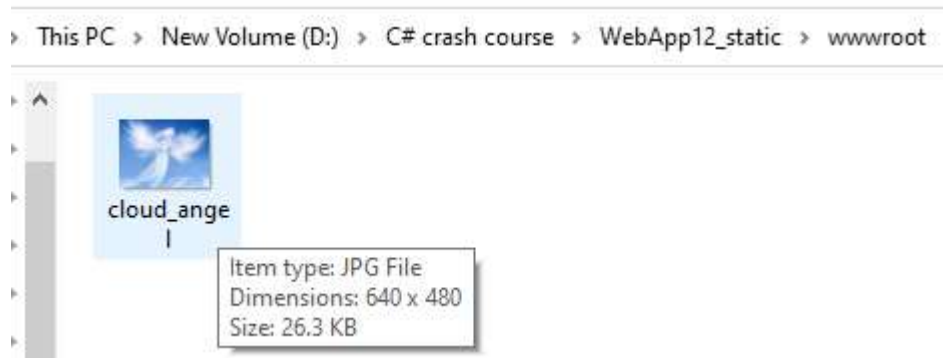
Everything that happens **after the next() method** is invoked in each of the middleware components, happens as the **RESPONSE** travels from middleware to middleware through the pipeline and this is represented by the **outgoing arrow**

12 – Static files

Изтриваме демонстративния код за съобщенията за MiddleWare. По подразбиране, .Net не обслужва статични файлове (html, css, Js, картинки дори в папка wwwroot да поставим картинка, тя няма да бъде достъпна през localhost на браузъра.

За да го прави, е необходимо да бъдат изпълнени 2 изисквания:

- 1) всички картинки да са в wwwroot папка директно в главната директория на приложението
- 2) `app.UseStaticFiles();`



Дори физически на диска да имаме картинка, на този етап не можем да я изведем в браузъра:

localhost:50322/wwwroot/cloud_angel.jpg



This localhost page can't be found

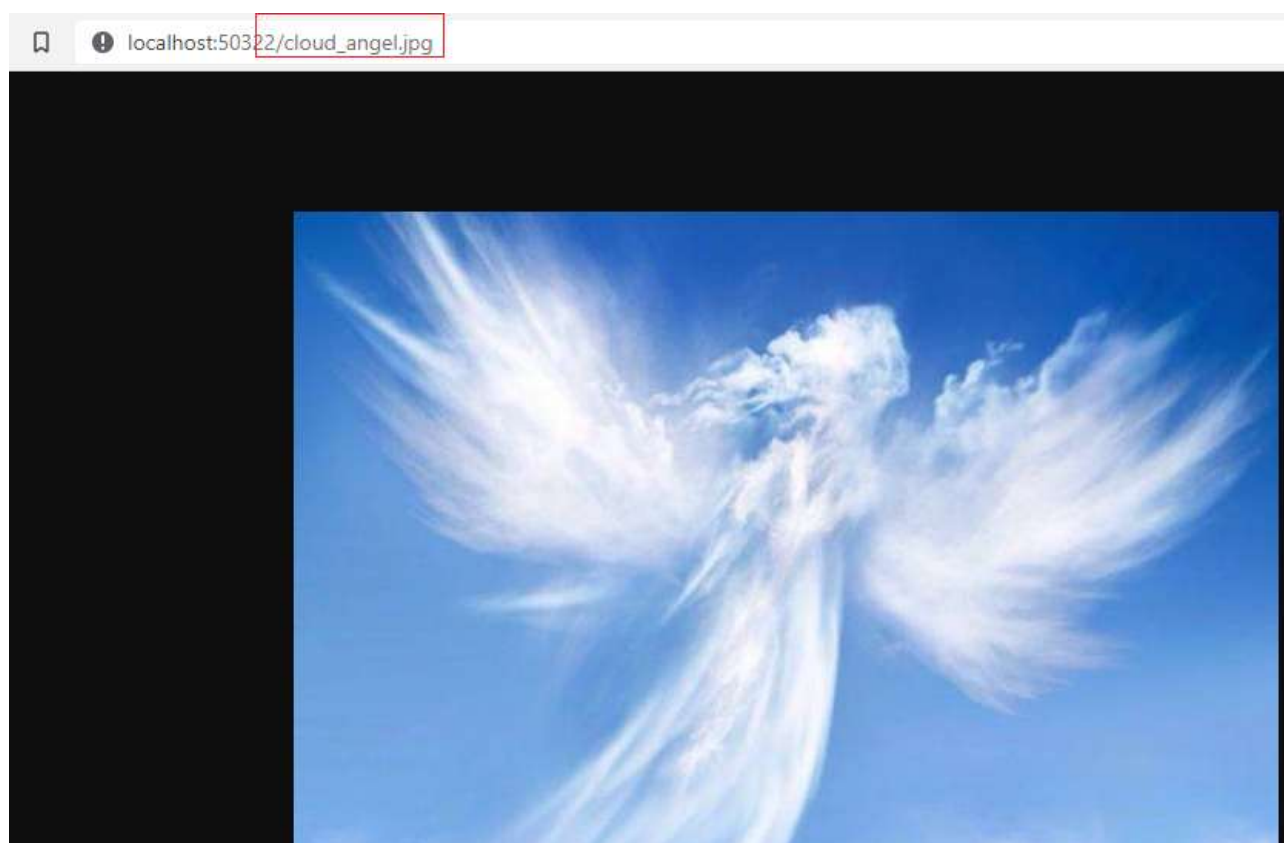
No webpage was found for the web address:

http://localhost:50322/wwwroot/cloud_angel.jpg

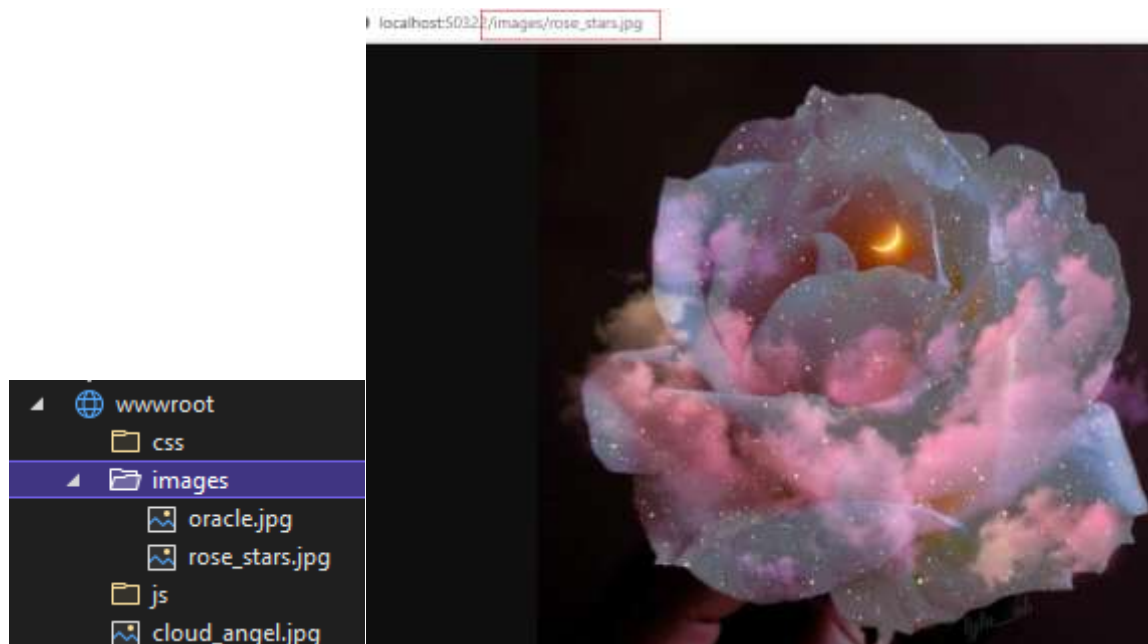
HTTP ERROR 404

Reload

Внимаваме с кеша на браузъра и обновяваме непрекъснато

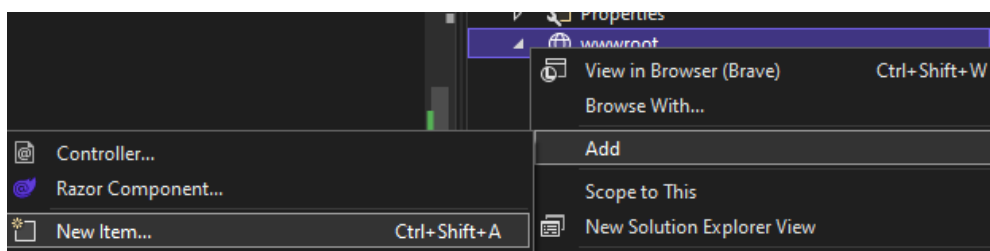


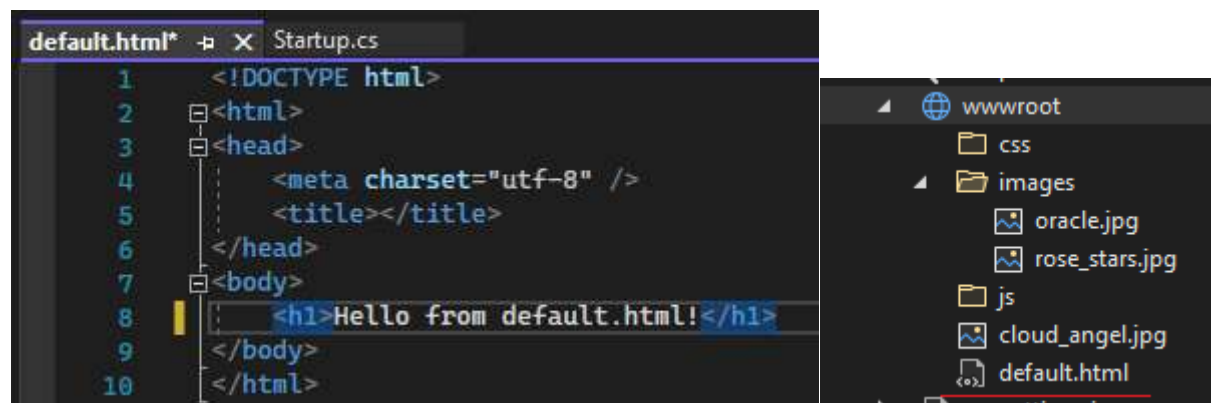
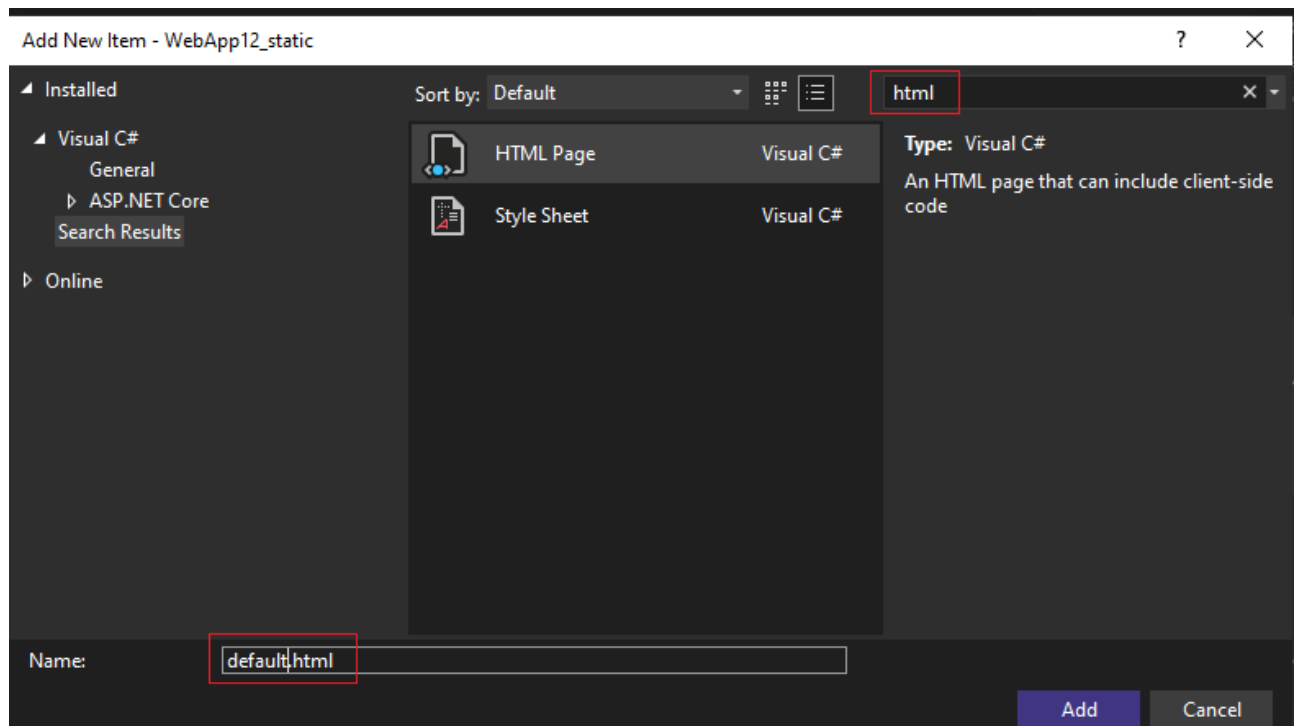
Възприето е всеки вид статични файлове да бъдат в отделни папки:



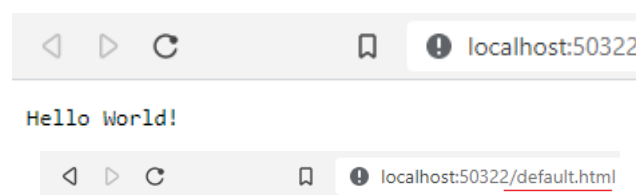
Като вече имаме свободен достъп до съдържанието след `app.UseStaticFiles();`, която по подразбиране обслужва само статични файлове в `wwwroot`-папка. Възможно е да имаме достъп и до статични файлове в други папки. Също по подразбиране се стартира статичен html файл вътре е `wwwroot`-папката, но в случая нямаме такъв и се изпълнява операцията от `app.UseEndpoints` в `startup.exe` с избор:

1. `index.htm`
2. `index.html`
3. `default.htm`
4. `default.html`





Въпреки, че вече го имаме наличен, браузърът все още изпълнява startup.cs



Hello from default.html!

Освен, ако изречно не навигираме до него в адрес-бара.

За да зареди по подразбиране статичните файлове в wwwroot, добавяме `app.UseDefaultFiles();`



Дори при това положение няма да зареди default.html по подразбиране, защото редът е много важен – трябва UseDefaultFiles да е преди UseStaticFiles – Това е много важно като подаване на заявки за обработка по pipeline: първо default, който после обслужва статичните файлове. В обратния случай обработваме статични файлове, без да имаме default и когато той бъде извикан, няма на кого да предаде заявките да обработят картинки и други, така че ги пропуска. При мен обаче, дори тогава не тръгва. Това е заради липсата на `app.UseRouting();` в старата версия.

`UseDefaultFiles` must be called before `UseStaticFiles` to serve the default file.

`UseDefaultFiles` is a URL rewriter that doesn't serve the file.

Routing in ASP.NET Core

Article • 04/13/2022 • 99 minutes to read • 38 contributors



By [Ryan Nowak](#), [Kirk Larkin](#), and [Rick Anderson](#)

Routing is responsible for matching incoming HTTP requests and dispatching those requests to the app's executable endpoints. **Endpoints** are the app's units of executable request-handling code. Endpoints are defined in the app and configured when the app starts. The endpoint matching process can extract values from the request's URL and provide those values for request processing. Using endpoint information from the app, routing is also able to generate URLs that map to endpoints.

<https://docs.microsoft.com/en-us/aspnet/core/fundamentals/routing?view=aspnetcore-5.0>



Hello from index.html!

Ако искаме по подразбиране да отваря друг файл – home.html или page1.html, задаваме го чрез `DefaultFileOptions();`

```
DefaultFileOptions defaultFileOptions = new DefaultFileOptions();
defaultFileOptions.DefaultFileNames = Tab Tab to accept

app.UseDefaultFiles(
app.UseStaticFiles()
app.UseRouting();
app.UseEndpoints(endpoints =>
```

Използваме подсказките с двоен Tab максимално.

```
DefaultFileOptions defaultFileOptions = new DefaultFileOptions();
defaultFileOptions.DefaultFileNames.Clear();
defaultFileOptions.DefaultFileNames.Add("page1.html");

app.UseDefaultFiles(defaultFileOptions);
app.UseStaticFiles();

app.UseRouting();

app.UseEndpoints(endpoints =>
```



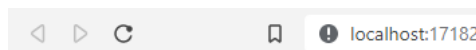
Hello from page1.html!

```
app.UseDefaultFiles();
app.UseStaticFiles();
app.UseDirectoryBrowser();
app.UseFileServer();
```

```
//app.UseDefaultFiles(defaultFileOptions);
//app.UseStaticFiles();
//app.UseDirectoryBrowser();
app.UseFileServer();

app.UseRouting();
```

Като функционалност **UseFileServer()**; може да замени 3-те изброени преди него, като връща страницата по подразбиране.



Hello from index.html!

Ако искаме да използваме персонализиран .html файл по подразбиране със UseFileServer(), вместо DefaultFileOptions, използваме FileServerOptions.

```
FileServerOptions fileServerOptions = new FileServerOptions();
fileServerOptions.DefaultFileOptions.DefaultFileNames.Clear();
fileServerOptions.DefaultFileOptions.DefaultFileNames.Add("page1.html");
app.UseFileServer(fileServerOptions);
```



Hello from page1.html!

Важно е да се запомни, че разширенията extensions (middleware) винаги започват с **Use** и за допълнителните им настройки използваме същото име като middleware без Use в началото, а с добавка **Options** в края като апендикс.

Например:

UseDeveloperExceptionPage - DeveloperExceptionPageOptions

UseDefaultFiles – DefaultFilesOptions

UseStaticFiles - StaticFilesOptions

UseFileServer – FileServerOptions

Static Files in ASP.NET Core

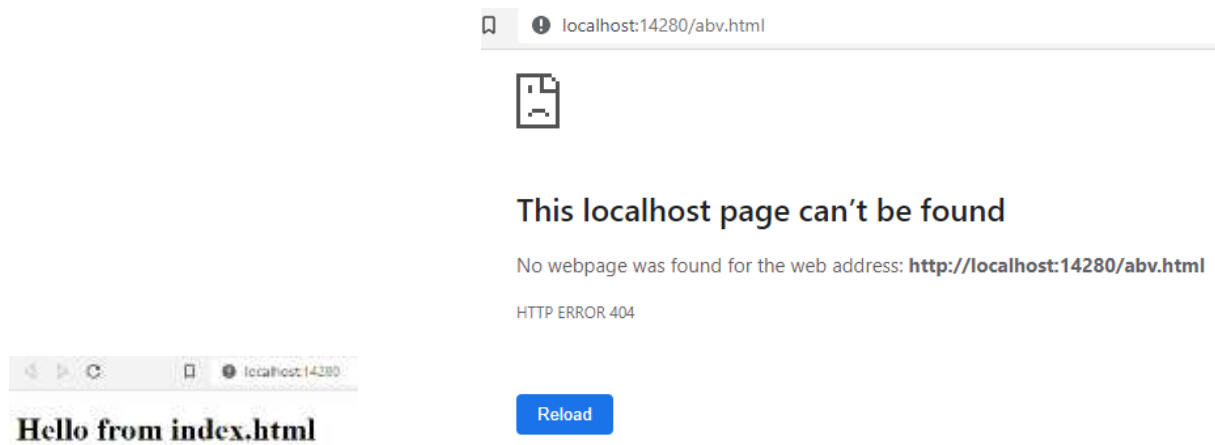
- By default an ASP.NET Core application **will not serve static files**
- The default directory for static files is **wwwroot**
- To serve static files **UseStaticFiles()** middleware is required
- To serve a default file **UseDefaultFiles()** middleware is required
- The following are the default files
 - index.htm
 - index.html
 - default.htm
 - default.html
- UseDefaultFiles() **must be registered before** UseStaticFiles()
- UseFileServer **combines the functionality** of UseStaticFiles, UseDefaultFiles and UseDirectoryBrowser middleware

13 – Developer exception page

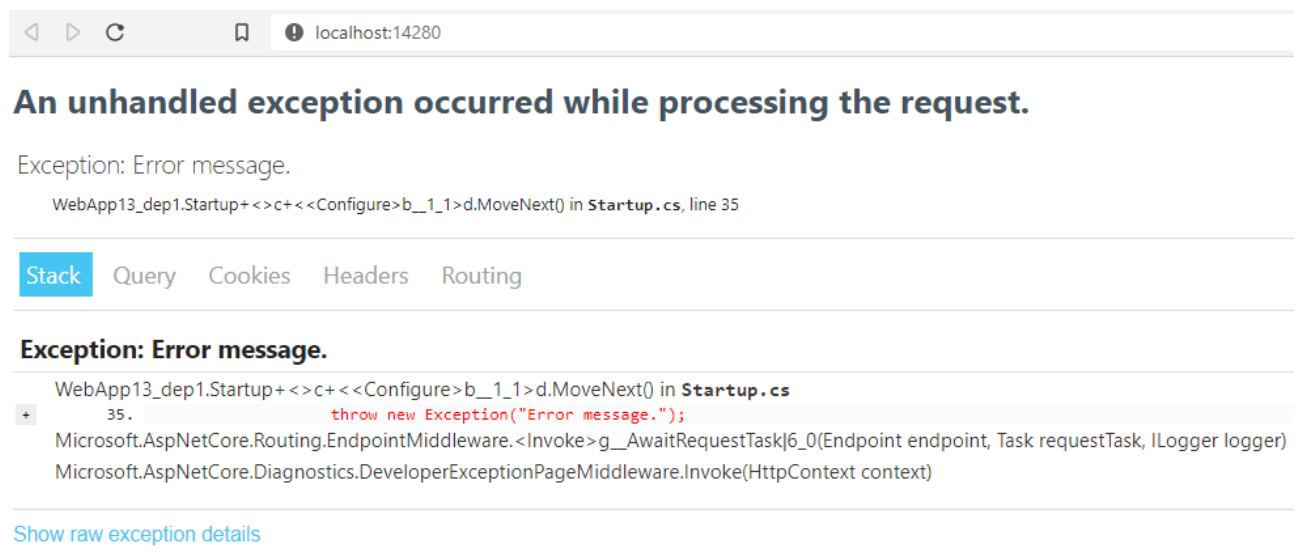
```
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }
    app.UseFileServer();
    app.UseRouting();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapGet("/", async context =>
        {
            throw new Exception("Error message.");
            await context.Response.WriteAsync("Hello World!");
        });
    });
}
```

Ако стартирам програмата при наличен index.html изпълнява:



Едва, ако липсва index.html (или преименуван index1.html) – тогава се изпълнява страницата с грешката.



В tutorиъла не е така – ако въведа несъществуващ файл в адрес-бара, се извежда съобщението за грешка, докато при мен - по подразбиране се изпълнява index.html, ако той не съществува, извежда съобщението с изключението, а ако в адрес-бара задам файл, който не съществува, просто го изписва като грешка. Забелязвам, че default.html е с по-висок приоритет от index.html и колкото и да рефрешвам, остава по-дълго в паметта на програмата и браузъра. За да продължа с упражнението, просто го преименувам index1.html

При разлистване 32-44 стр.

```

WebApp13_dep1.Startup+<>c+<<Configure>b__5_1>d.MoveNext() in Startup.cs
32.         app.UseRouting();
33.
34.         app.UseEndpoints(endpoints =>
35.         {
36.             endpoints.MapGet("/", async context =>
37.             {
38.                 throw new Exception("Error message.");
39.                 await context.Response.WriteAsync("Hello World!");
40.             });
41.         });
42.     }
43. }
44. }

```

Освен това, в раздела на заявката няма данни, но ако ние въведем променливи в адрес-бара, те ще бъдат прихванати и показани в Query:

localhost:14280/?a=10&b=20

An unhandled exception occurred while processing the request.

Exception: Error message.

WebApp13_dep1.Startup+<>c+<<Configure>b__5_1>d.MoveNext() in Startup.cs, line 38

Stack	Query	Cookies	Headers	Routing
Variable	Value			
a	10			
b	20			

Важно е да се прилага възможно по-нагоре в кода за изпълнение.

```

{
    app.UseDeveloperExceptionPage();
}

```

Също не ми позволява смяна на броя на изведените на дисплея линии код (винаги е +6 преди и след съобщението за грешка)

В tutorиъла не изисква ; след цифрата за линии код преди и след грешката, а при мен изисква и на всичкото отгоре не се изпълнява по задаването.

```

if (env.IsDevelopment())
{
    DeveloperExceptionPageOptions developerExceptionPageOptions = new DeveloperExceptionPageOptions();
    {
        SourceCodeLineCount = 1;
    };
    app.UseDeveloperExceptionPage(developerExceptionPageOptions);
}

```

Урок 13 – напълно провален.

В урок 16 всъщност разбрах откъде не се е изпълнявал правилно кодът – понеже в туториъла презаписват върху един и същи проект различни парчета код, аз правя всеки път нов с името на урока, оказва се, че `using Microsoft.Extensions.Configuration;`

```
using Microsoft.Extensions.Configuration;
```

Тогава редовете се изпълняват за windows 7, но впоследствие, в windows 10 има още промени:

```
public class Startup
{
    public void ConfigureServices(IServiceCollection services)
    {
    }
    private object _config;
    public Startup(IConfiguration config)
    {
        _config = config;
    }
    public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
    {
        if (env.IsDevelopment())
        {
            DeveloperExceptionPageOptions developerExceptionPageOptions = new DeveloperExceptionPageOptions();
            developerExceptionPageOptions.SourceCodeLineCount = 5;
            app.UseDeveloperExceptionPage(developerExceptionPageOptions);
        }
        app.UseFileServer();
        app.UseRouting();
        app.UseEndpoints(endpoints =>
        {
            endpoints.MapGet("/", async context =>
            {
                throw new Exception("Some error processing request.");
                await context.Response.WriteAsync("Hello World!");
            });
        });
    }
}
```

Кодът не е същият като в туториъла – не мога да вложа цикъл без грешки, но по този начин работи и извежда +- зададения брой редове около грешката.

Много интересно, че когато инсталирах Windows 10, една редица бъгове ги няма, защото в кода по подразбиране е заложен `app.UseRouting();` който преди трябваше да намеря и добавя аз, както и други промени с разлики във версиите.

```
DefaultFilesOptions defaultFilesOptions = new DefaultFilesOptions();
defaultFilesOptions.DefaultFileNames.Clear(); // first clear the default
defaultFilesOptions.DefaultFileNames.Add("home.html"); // name of the new

app.UseDefaultFiles(defaultFilesOptions); // use the parameter
app.UseStaticFiles();
app.UseRouting();
```

Така по подразбиране зарежда `home.html`, вместо `index/default`.

`app.UseFileServer()` обединява 3-те:
`app.UseDefaultFiles(); UseStaticFiles(); UseDirectoryBrowser();`

Ако използваме `app.UseFileServer()` ще стане:

```

FileServerOptions fileServerOptions = new FileServerOptions();
fileServerOptions.DefaultFilesOptions.DefaultFileNames.Clear();
fileServerOptions.DefaultFilesOptions.DefaultFileNames.Add("home.html");

app.UseFileServer(fileServerOptions);
app.UseRouting();

```

```

FileServerOptions fileServerOptions = new FileServerOptions();
fileServerOptions.DefaultFilesOptions.DefaultFileNames.Clear();
fileServerOptions.DefaultFilesOptions.DefaultFileNames.Add("home.html");

```

```

app.UseFileServer(fileServerOptions);
app.UseRouting();

```

Изтриваме целия този демонстративен код и оставяме само:

```

app.UseFileServer();
app.UseRouting();

```

```

public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }
    app.UseFileServer();
    app.UseRouting();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapGet("/", async context =>
        {
            throw new Exception("Error message.");
            await context.Response.WriteAsync("Hello World!");
        });
    });
}

```

Съобщението за грешка в терминала няма да се използва, т.к. преди това `UseFileServer()`; е извикал default-документ, който се е изпълнил. Ако такъв документ липсва и страницата не се зареди, ще се подаде към следващия middleware in pipeline, който е съобщението за грешка.

Стремим се да поставим `app.UseDeveloperExceptionPage()`; възможно най-горе в кода, защото в противен случай, при грешка, ще се зареди страница с много малко информация Error 500, вместо необходимата `DeveloperExceptionPage`.

```

if (env.IsDevelopment())
{
    DeveloperExceptionPageOptions developerExceptionPageOptions = new DeveloperExceptionPageOptions();
    {
        //SourceCodeLineCount = 10;
        developerExceptionPageOptions.SourceCodeLineCount = 1;
    }
    app.UseDeveloperExceptionPage(developerExceptionPageOptions);
}

```

В старите версии се използва директно `SourceCodeLineCount = 1`; като се добави `using Microsoft.Extensions.Configuration`;, но в новите не е така.

```

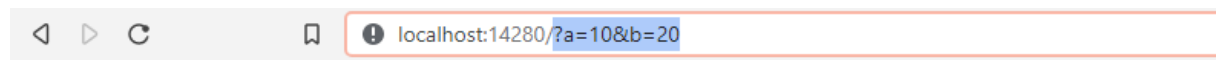
if (env.IsDevelopment())
{
    DeveloperExceptionPageOptions developerExceptionPageOptions = new
    DeveloperExceptionPageOptions();
}

```

```

        {
            developerExceptionPageOptions.SourceCodeLineCount = 1;
        }
    app.UseDeveloperExceptionPage(developerExceptionPageOptions);
}

```



Exception: Error message.

WebApp13_dep1.Startup+<>c+<<Configure>b__5_1>d.MoveNext() in Startup.cs, line 38

Stack Query Cookies Headers Routing

Variable	Value
a	10
b	20

/?a=10&b=20

На последния слайд от презентацията на урок 13, кодът който е показан, е новият, който работи при мен, а не този, който се е обяснявал в урока.

ASP.NET Core Developer Exception Page

```

if (env.IsDevelopment())
{
    DeveloperExceptionPageOptions developerExceptionPageOptions = new DeveloperExceptionPageOptions();
    developerExceptionPageOptions.SourceCodeLineCount = 10;
    app.UseDeveloperExceptionPage(developerExceptionPageOptions);
}

app.UseFileServer();

app.Run(async (context) =>
{
    throw new Exception("Some error processing the request");
    await context.Response.WriteAsync("Hello World!");
});

```

- To enable plug in **UseDeveloperExceptionPage** Middleware in the pipeline
- Must be plugged in the pipeline **as early as possible**
- Contains **Stack Trace, Query String, Cookies and HTTP headers**
- For customizing use **DeveloperExceptionPageOptions** object

14 – Environment variables

Development Environments



- ASPNETCORE_ENVIRONMENT variable sets the Runtime Environment
- On development machine we set it in **launchsettings.json** file
- On Staging or Production server we set in the **operating system**
- Use **IHostingEnvironment** service to access the runtime environment
- Runtime environment defaults to **Production** if not set explicitly
- In addition to **standard environments** (Development, Staging, Production), **custom environments** (UAT, QA etc) are also supported

Development Environment : обичайно за софтиерните разработчици + JavaScript, CSS

Staging Environment : възможно най-идентичен с Production, като се цели безопасност и производителност, особено при B2B (Business to Business). Обикновено не се дебъгва и ремонтира, също се използват минимално JS, CSS, за да не товарят производителността. Ако има непредвидена грешка, ще се покаже само генерално съобщение от user friendly page, вместо от DeveloperExceptionPage.

Production Environment : Този софтуер, който използват потребителите ежедневно с максимална производителност и сигурност, заради което минимизираме JS, CSS, а грешките дават минимално количество информация.

* Имаме и environment tag helpers in Razor View in HTML code.

"ASPNETCORE_ENVIRONMENT": variable sets the Runtime Environment.

```
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }

    app.UseStaticFiles();
    app.UseRouting();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapGet("/", async context =>
        {
            await context.Response.WriteAsync("Hosting environment: " + env.EnvironmentName);
        });
    });
}
```

Махаме новия код с грешките от предходния урок, като оставяме `app.UseStaticFiles();`

```
await context.Response.WriteAsync("Hosting environment: " + env.EnvironmentName);
```

за да изпише името на средата.

```
"profiles": {  
  "IIS Express": {  
    "commandName": "IISExpress",  
    "launchBrowser": true,  
    "environmentVariables": {  
      "ASPNETCORE_ENVIRONMENT": "Development"  
    }  
  }  
}
```

localhost:22782

Hosting environment: Development

Съответно изписва Staging или Production, когато се сменят в launchSettings.json

Когато нямаме Environment, показва системната по подразбиране, която в Windows е Production, за по-голяма сигурност и производителност като потребителски достъп.

```
"environmentVariables": {  
  // "ASPNETCORE_ENVIRONMENT": "Staging"  
}
```

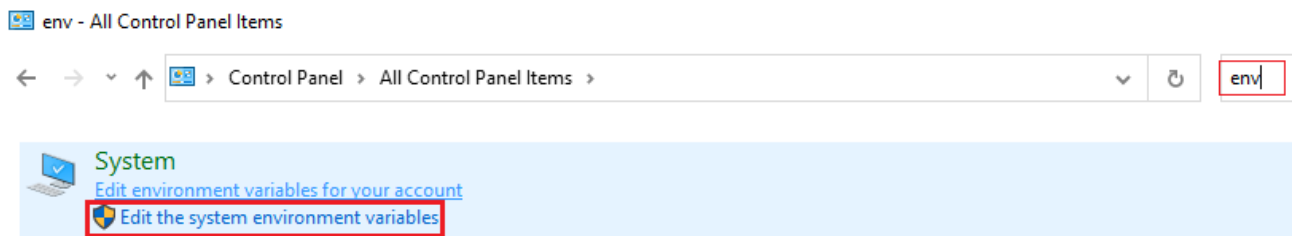
localhost:22782

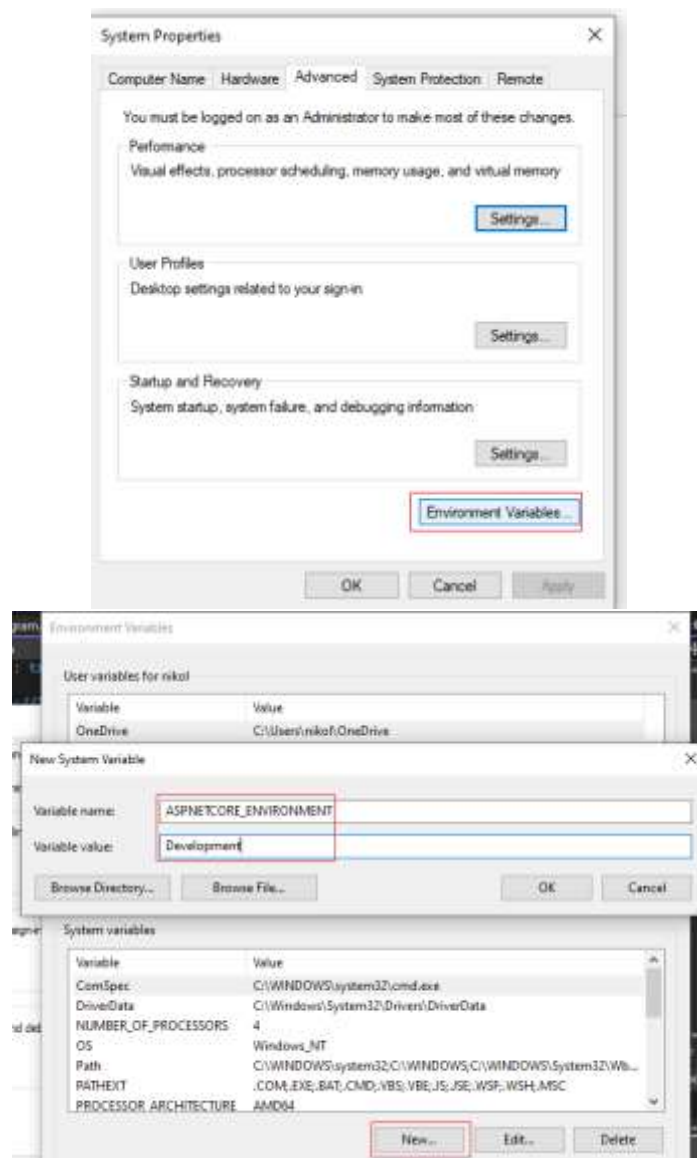
Hosting environment: Production

Properties -> launchSettings.json -> избираме Development, Staging, Production.

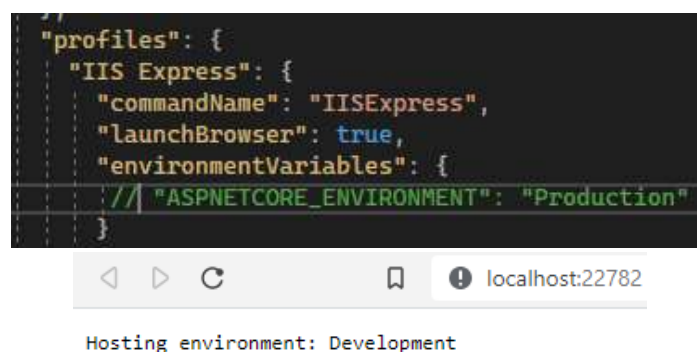
Търсачка 'control' -> C panel -> Още едно търсене: 'env' -> System -> Edit the system environment variables -> System properties -> Environment variables -> New -> ASPNETCORE_ENVIRONMENT

По подразбиране в Windows = Production.

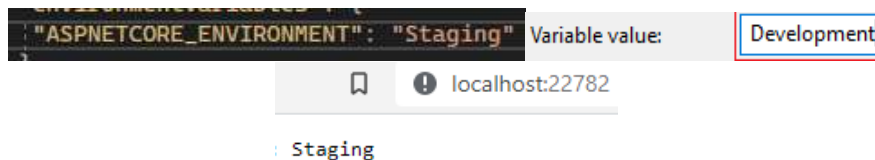




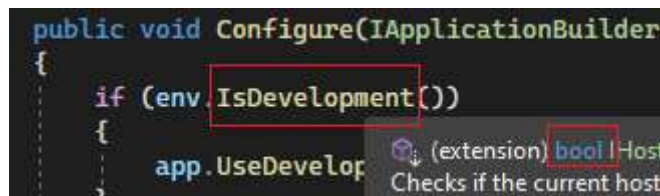
За да се отразят промените в браузъра трябва да се рестартира VS, за да може той да прихване поне веднъж промените от системата. Ако и това не помогне, след 1-я рестарт на компютъра, ще се отразят системните редакции.



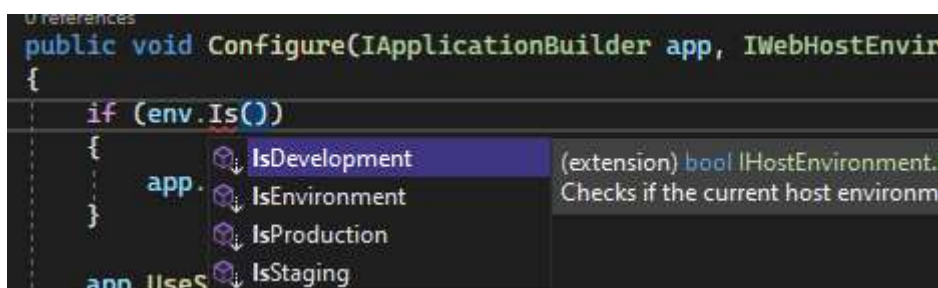
Когато имаме в програмата VS Staging, а в OC Development, програмата е с приоритет и закрива системата.



Последно, в Startup.cs, имаме if с резултати в стойности 1 за истина и 0 за грешка.



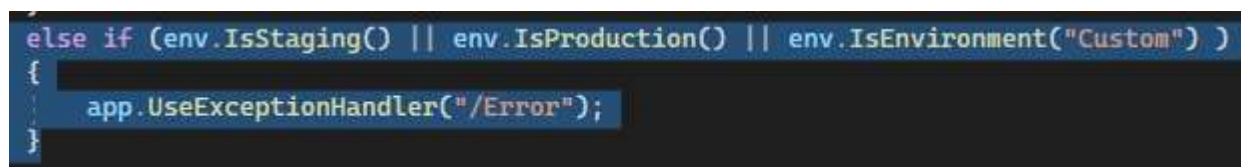
Освен IsDevelopment имаме избор IsProduction, IsStaging.



4-тият избор IsEnvironment се използва за custom персонализиран с име по наш избор:



Друг блок, който при мен в Windows 7 не се изпълни, но в 10 stana:



```
if (env.IsDevelopment()) // bool, IsStaging, IsProduction
{
    app.UseDeveloperExceptionPage();
}
```

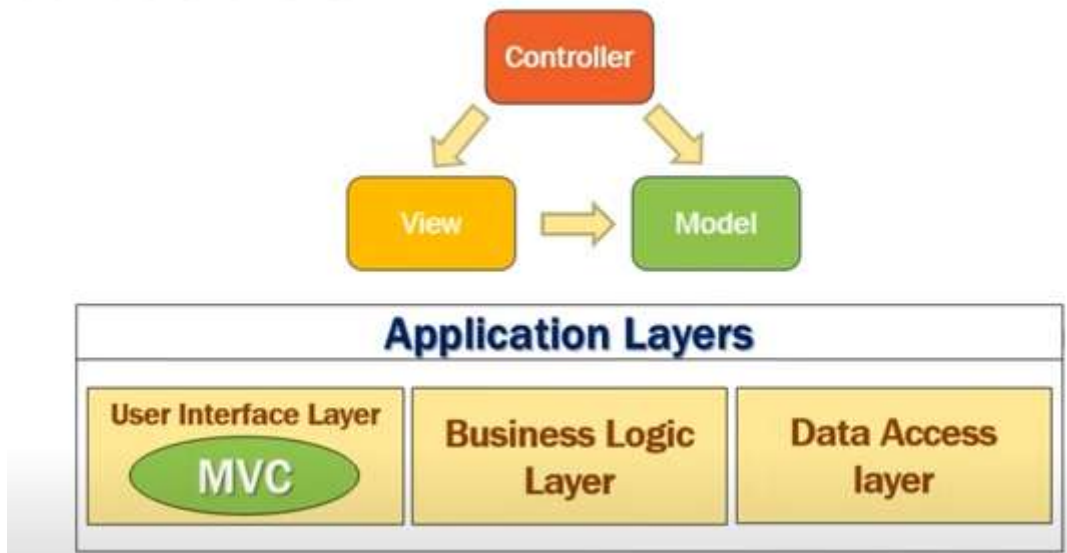
Възможно е потребителска среда **IsEnvironment("UAT")**

```
if (env.IsDevelopment()) // bool, IsStaging, IsProduction, IsEnvironment("UAT")
{
    app.UseDeveloperExceptionPage();
}
else if (env.IsStaging() || env.IsProduction ||
env.IsDevelopment("UAT"))
{
    app.UseExceptionHandler("/Error.");
}
```

Локалните настройки засенчват глобалните

15 – MVC tutorial

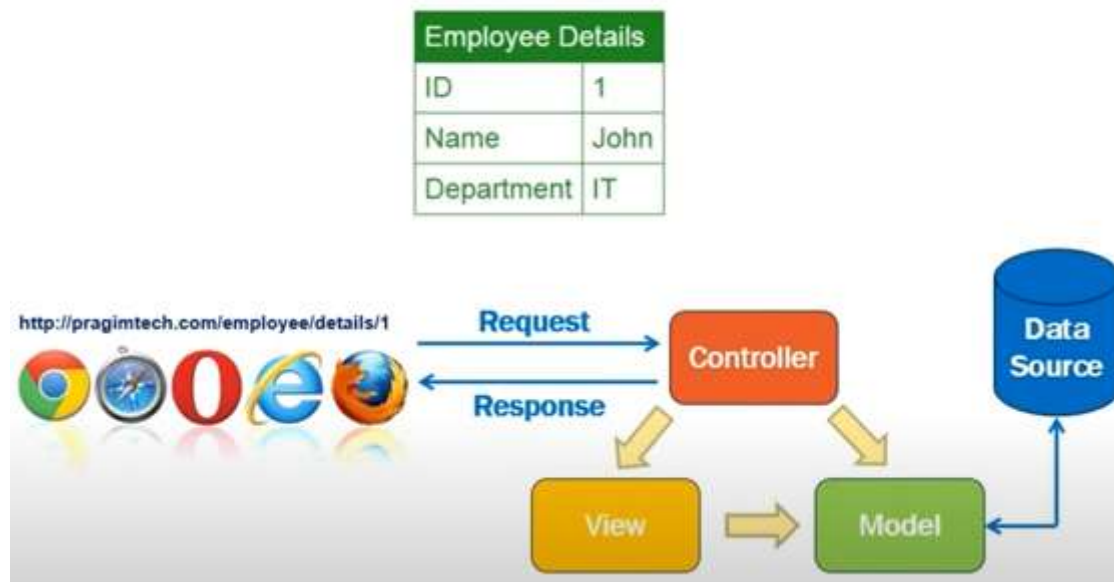
What is MVC



MVC = architectural design pattern for implementing the user interface layer of an application.

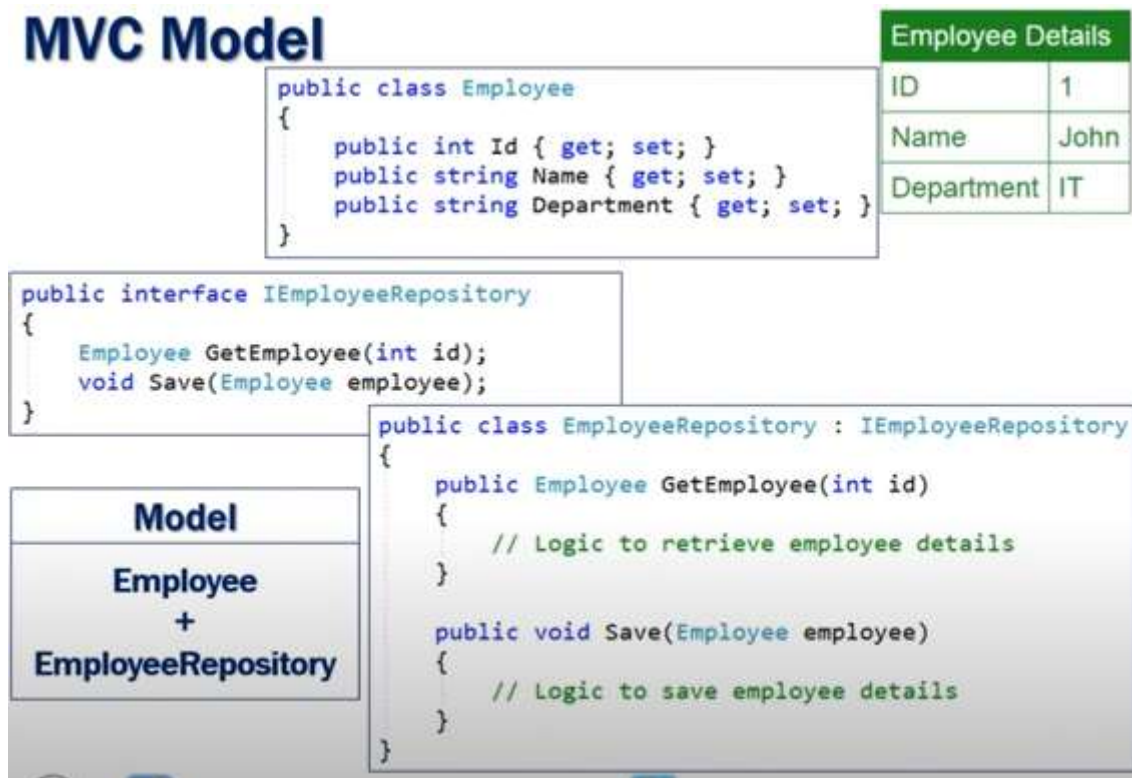
Application layers = User Interface (MVC) + Business Logic (Domain) + Data Access

How MVC Works



Логиката във View трябва да се сведе до минимум, колкото да покаже данните.

MVC Model

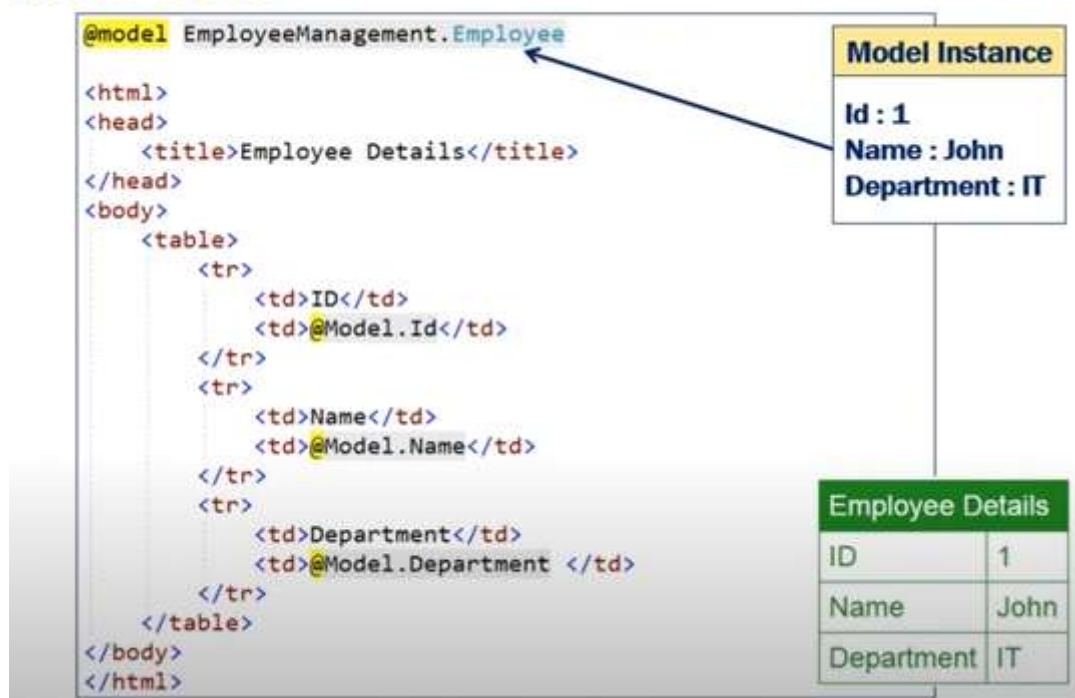


EmployeeRepository = Interface for EmployeeRepository

Може без интерфейс чисто технологично да се имплементира приложението, само с данни и обработка, но този допълнителен интерфейс ни позволява dependency injections (зависими вмъквания) за по-голяма гъвкавост, свобода и тестване. л

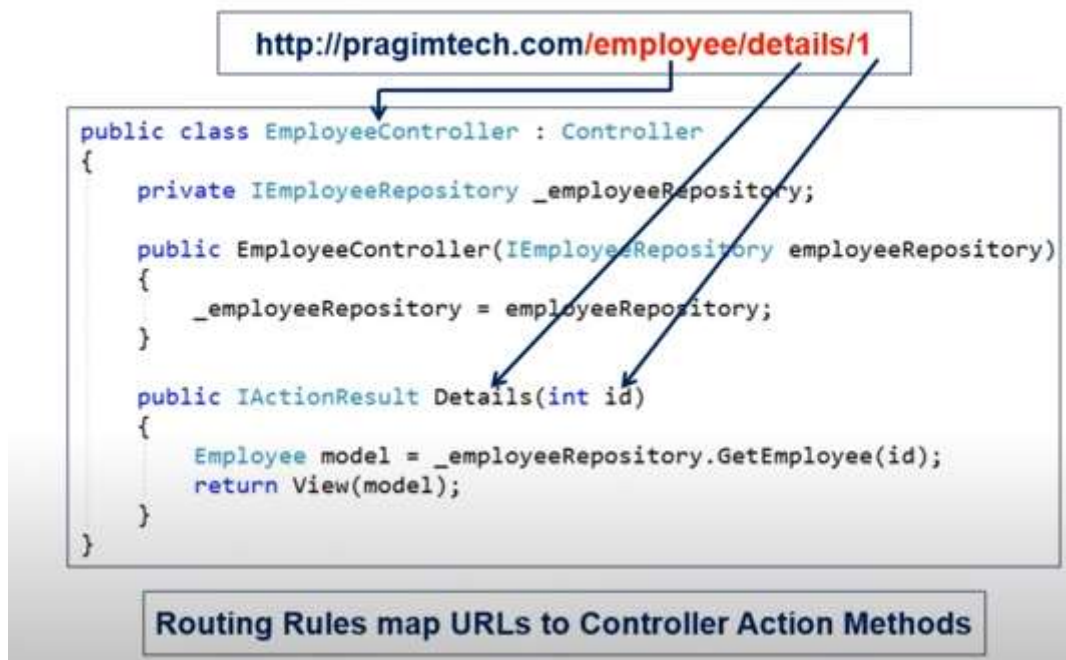
View – трябва да е възможно най-опростен HTML template.

MVC View



Model -----> data -----> View (user) http -----> Controller

MVC Controller



* Routing rules map URLs to Controller Action Method

http request ----- Controller (builds the model) ----- response to user action

Summary

MVC is an architectural design pattern for implementing User Interface Layer of an application

- **Model** : Set of classes that represent data + the logic to manage that data
- **View** : Contains the display logic to present the Model data provided to it by the Controller
- **Controller** : Handles the http request, work with the model, and selects a view to render that model

MVC = архитектурен дизайн за интерфейс потребител-приложение

* Model = състои се задължително от 2 неща: класове за данни + логика за управлението им

* View = логика за дисплей на данните от модела чрез контролера

* Controller = обработва потребителските заявки чрез модела

16 – Setup MVC

Setup MVC in ASP.NET Core

➤ Step 1 : Add the MVC Services to the Dependency Injection Container

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvc();
}
```

➤ Step 2 : Add MVC middleware to the Request Pipeline

```
public void Configure(IApplicationBuilder app, IHostingEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }

    app.UseStaticFiles();
    app.UseMvcWithDefaultRoute();
}
```

- 1: services
- 2: pipeline

B Startup

```
1:     public void ConfigureServices(IServiceCollection services)
      {
          services.AddMvc();
      }

...

2:     app.UseStaticFiles();
      app.UseMvcWithDefaultRoute(); //Short-circle последователност
      app.UseRouting();
```

Може само UseMvc, но With DefaultRoute търси Index-секция в HomeController
HomeController.cs, но понеже не го намира, при зададената последователност предава по
pipeline към следващия middleware, който е terminal – WriteAsync(“Hello world!”).
WithDefaultRoute след версия .Net 3.0 не се използва и дори не разрешава да отида до
дефиницията му, но в туториъла - Д.б. -> Go to Definition

```
'{controller=Home}/{action=Index}/{id?}'.
```

Важното е UseStaticFiles да бъде преди UseMvc, понеже статичните файлове (картинки и прочие)
ще бъдат прихванати и обработени и следващия middleware UseMvc няма да се използва, а ще
бъде превключен на къса верига, избягвайки излишна обработка. От друга страна, при друга
заявка, статичните файлове ще бъдат пропуснати до следващия middleware UseMvc, който ще ги
обработи.

```

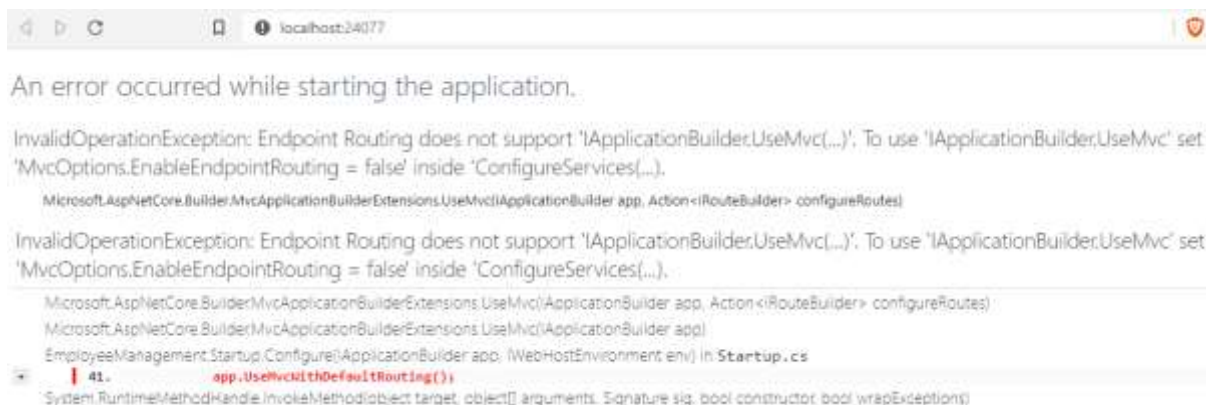
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvc();
}

// This method gets called by the runtime. Use this method to configure
0 references
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    if (env.IsDevelopment()) // bool, IsStaging, IsProduction, IsEnviron
    {
        app.UseDeveloperExceptionPage();
    }

    app.UseStaticFiles(); //Short-circle ordering
    app.UseMvcWithDefaultRouting();
    app.UseRouting();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapGet("/", async context =>
        {
            await context.Response.WriteAsync("Hello world!");
        });
    });
}

```



.net core 3 , MVC , Using 'UseMvcWithDefaultRoute' to configure MVC is not supported while using Endpoint Routing

Disable endpoint Routing.

services.AddMvc(option => option.EnableEndpointRouting = false)

При новите версии,

app.UseMvcWithDefaultRouting(); е заменено с

```

app.UseMvc(rb =>
{
    rb.MapRoute(
        name: "default",
        template: "{controller}/{action}/{id?}",

```



```
        defaults: new { controller = "Home", action = "Index" });  
    });
```

<https://blog.2mas.xyz/asp-net-5-adding-mvc-to-an-application/>

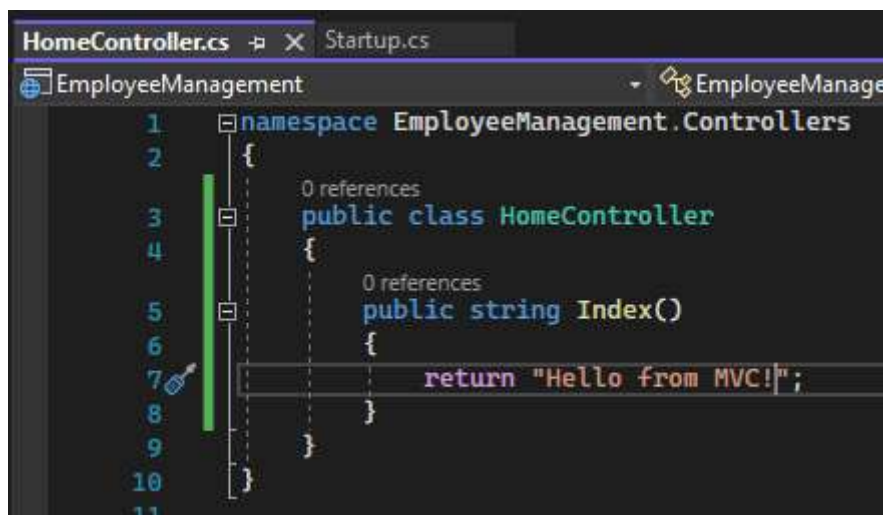
Тогава при:

```
public void ConfigureServices(IServiceCollection services)  
{  
    services.AddMvc(option => option.EnableEndpointRouting = false);  
}  
  
// This method gets called by the runtime. Use this method to configure the HTTP request pipeline  
// 0 references  
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)  
{  
    if (env.IsDevelopment()) // bool, IsStaging, IsProduction, IsEnvironment("UAT")  
    {  
        app.UseDeveloperExceptionPage();  
    }  
  
    app.UseStaticFiles(); //Short-circle ordering  
    app.UseMvc(rb => //app.UseMvcWithDefaultRouting(); error after .net 3.8  
    {  
        rb.MapRoute(  
            name: "default",  
            template: "{controller}/{action}/{id?}",  
            defaults: new { controller = "Home", action = "Index" });  
    });  
    app.UseRouting();  
  
    app.UseEndpoints(endpoints =>  
    {  
        endpoints.MapGet("/", async context =>  
        {  
            await context.Response.WriteAsync("Hello world!");  
        });  
    });  
}
```

Условия за използване на MVC:

- 1) ConfigureServices -> `services.AddMvc();`
- 2) Configure pipeline middleware -> `app.UseMvc();`

WithDefaultRoute търси Index метод в HomeController. Нека ги добавим в стандартна папка Controllers, нов клас HomeController.cs, метод Index



```
1 namespace EmployeeManagement.Controllers
2 {
3     0 references
4     public class HomeController
5     {
6         0 references
7         public string Index()
8         {
9             return "Hello from MVC!";
10        }
11    }
```

Резултат е „Hello from MVC!“, вместо Hello from default.html, Hello from index.html, Hello world! Това е така, т.к. UseMvcWithDefaultRoute се намира преди терминалния middleware и когато неговото условие е изпълнено, той се стартира и другия след него няма вече шанс да се изпълни.

В **туториъла** при <http://localhost:24077/about/index> се стига до Hello world, но при мен не, може би защото съм изключила endpoint ? – не, Hello world се изпълнява при закоментиране на блока код за UseMvc

Проверявам с коментиране на блок код

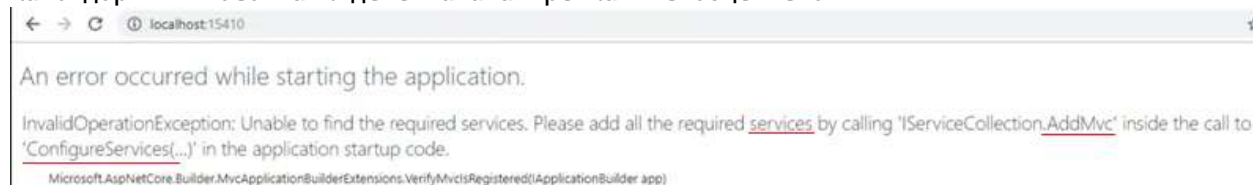
Ctrl + K (key) + C (comment) -> //закоментира всеки ред код

Ctrl + K (key) + U (uncomment) -> //премахва коментара на всеки ред код

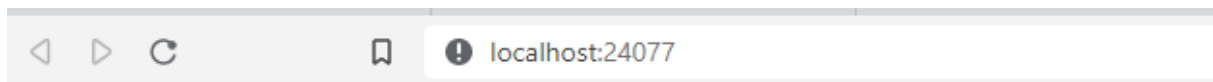
При премахване на option => option.EnableEndpointRouting = false

Дава грешка, както и при премахване на целия ред
services.AddMvc(option => option.EnableEndpointRouting = false);

Като дори ни посочва къде е нашата грешка в съобщението



Интересното е, че страницата за грешки се извиква след UseMvc, но при разминаване на if между заложения Environment в Startup и ENVIRONMENT_VARIABLE извежда страница с почти никаква информация за грешката.



HTTP Error 500.30 - ASP.NET Core app failed to start

Common solutions to this issue:

- The app failed to start
- The app started but then stopped
- The app started but threw an exception during startup

Troubleshooting steps:

- Check the system event log for error messages
- Enable logging the application process' stdout messages
- Attach a debugger to the application process and inspect

For more information visit: <https://go.microsoft.com/fwlink/?LinkID=2028265>

```
public void ConfigureServices(IServiceCollection services)
{
    //services.AddMvc(option => option.EnableEndpointRouting = false);
}

// This method gets called by the runtime. Use this method to configure
0 references
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    if (env.IsDevelopment()) // bool, IsStaging, IsProduction, IsEnviron
    {
        app.UseDeveloperExceptionPage();
    }
}
```

При закоментиране на целия блок Endpoint продължава да извежда правилно Hello from MVC! Това е така, т.к. сме използвали WithDefaultRoute, който търси Index метод в HomeController. Нека ги добавим в стандартна папка Controllers, нов клас HomeController.cs, метод Index

```
//app.UseEndpoints(endpoints =>
//{
//    endpoints.MapGet("/", async context =>
//    {
//        await context.Response.WriteAsync("Hello world!");
//    });
//});
```

Т.е. в главния URL се изпълнява "Hello from MVC", защото условията на extension-UseMvcWithDefaultRoute()-middleware са изпълнени и като short-circle, след него WriteAsync("Hello wodl!") няма шанс да се изпълни.

При заобикаляне на about-контролер, който не съществува в адрес-бара:

localhost.../about/index -> превключва на следващия middleware "Hello world!". Ако отстраним целия терминален middleware ('Ctrl'+ 'X') и рефрешнем браузъра /about/index ('Ctrl'+ 'R') -> Error

404

Превключва .Use ->

.Use ->

без изтрет терминал – нищо 404

WithDefaultRoute - /home/index -> "Hello from MVC!"

localhost: -> "Hello from MVC!"

17 – Mvc vs Mvc Core

services.  Add.MvcCore(); - само ядро
Add.Mvc(); - ядро + всичко

ASP.NET Core AddMvc vs AddMvcCore

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvc();
}
```

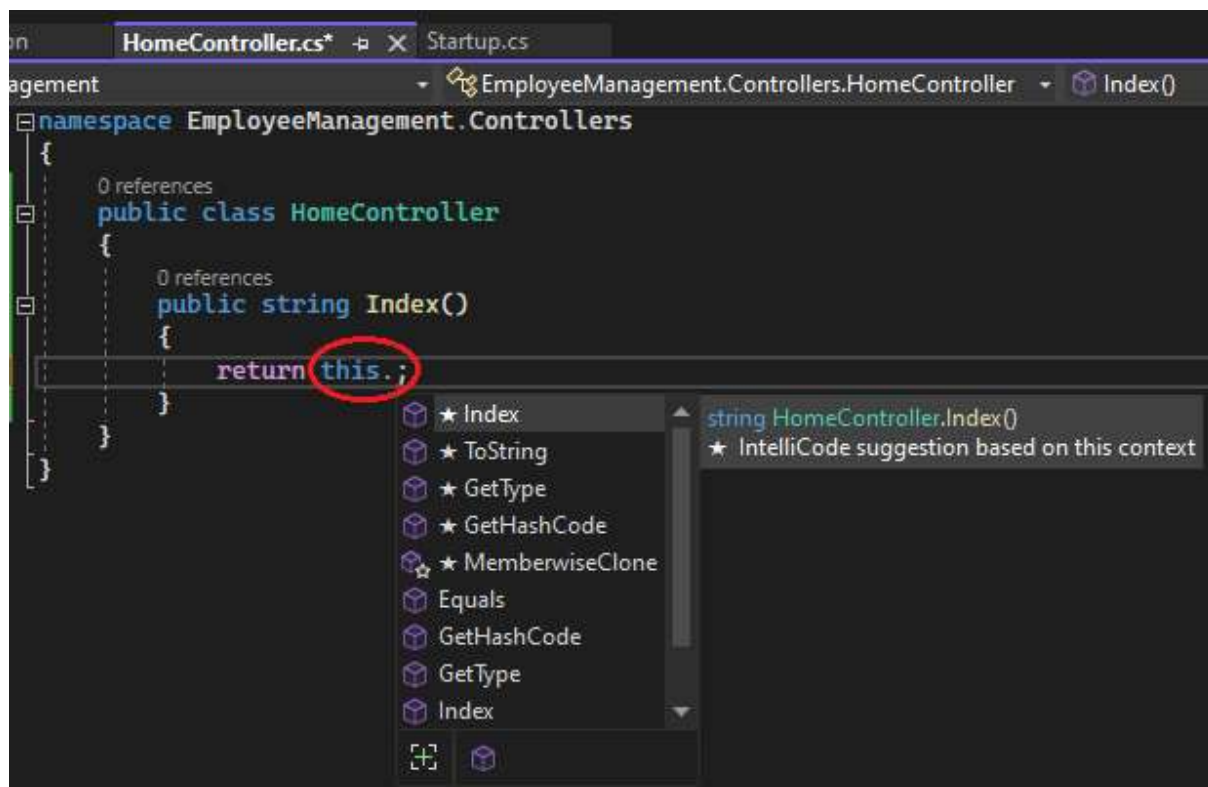
```
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvcCore();
}
```

➤ **AddMvcCore()** method only adds the core MVC services

➤ **AddMvc()** method adds all the required MVC services

➤ **AddMvc()** method calls **AddMvcCore()** method internally

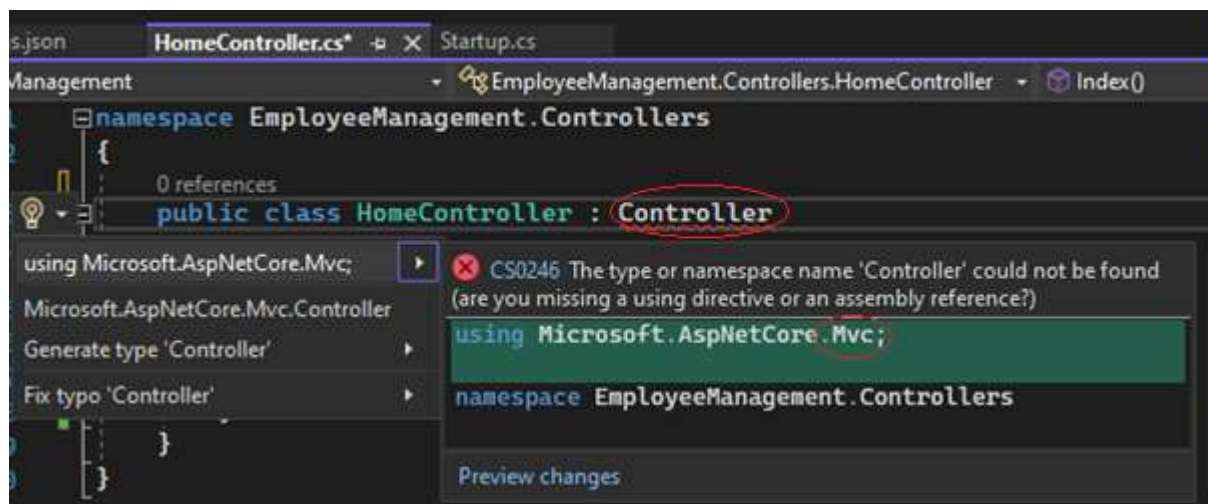
(public method | private field)



Не се включва Intellisense след this.

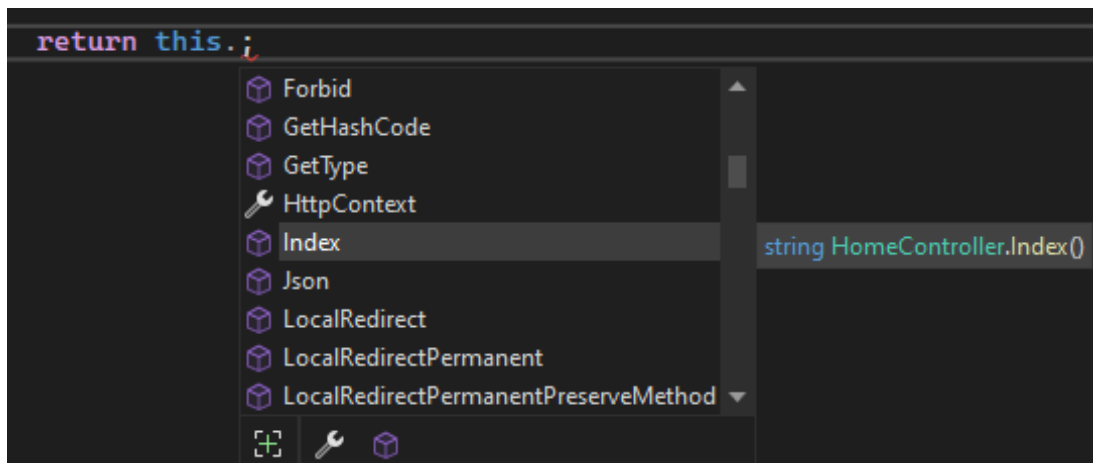
API = Application User Interface (подсказки)

Искаме да изучим повече функции за Mvc с включване на вграден клас Controller



Всички типове резултати като ViewResult, JsonResult, PartialViewResult ... се предоставят от този вграден клас : Controller.

След това вече се включват вградените възможности:



Вграденият клас Controller предоставя възможност за работа с Json тип данни

```
0 references
public JsonResult Index()
{
    return Json(new { id = 1, name="Pragim" }); //new anonymus object
}
```

```
public JsonResult Index()
{
    return Json(new { id = 1, name="Pragim" }); //new anonymus object
}
```

⏪ ⏩ ↻ 📖 ! localhost:24077

Run -> {"id":1,"name":"Pragim"}

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvcCore(option => option.EnableEndpointRouting = false);
}
```

В туториъла извежда грешка, защото ядрото Core предоставя по-малко базови функции (Не работи с Json. При мен, в случая с наличния .net 5.0 няма проблем.

Линкът от

<https://github.com/aspnet/mvc/blob/release/2.2/src/microsoft.aspnetcore.mvc/mvcservicesollectionextensions.cs> в туториъла не работи вече.


```
er Error X Mvc/MvcServiceCollectionExtensions X +
GitHub, Inc. [US] | https://github.com/aspnet/Mvc/blob/release/2.2/src/Microsoft.AspNetCore.Mvc/MvcServiceCollectionExtensions.cs

    /// <summary>
    /// Extension methods for setting up MVC services in an <see cref="IServiceCollection" />.
    /// </summary>
    public static class MvcServiceCollectionExtensions
    {
        /// <summary>
        /// Adds MVC services to the specified <see cref="IServiceCollection" />.
        /// </summary>
        /// <param name="services">The <see cref="IServiceCollection" /> to add services to.</param>
        /// <returns>An <see cref="IMvcBuilder"/> that can be used to further configure the MVC services.</returns>
        public static IMvcBuilder AddMvc(this IServiceCollection services)
        {
            if (services == null)
            {
                throw new ArgumentNullException(nameof(services));
            }

            var builder = services.AddMvcCore();

            builder.AddApiExplorer();
            builder.AddAuthorization();
        }
    }
}
```

Вътрешно търсене на AddMvc

```
https://github.com/aspnet/Mvc/search?q=AddMvcCore&unscoped_q=AddMvcCore
9 20 // Act
21 builder.AddViews();

src/Microsoft.AspNetCore.Mvc.Core/DependencyInjection/MvcCoreServiceCollectionExtensions.cs
Showing the top two matches Last indexed on 20 Nov 2018
37 /// <returns>An <see cref="IMvcCoreBuilder"/> that can be used to further configure the
MVC services.</returns>
38 /// <remarks>
39 /// The <see cref="MvcCoreServiceCollectionExtensions.AddMvcCore(IServiceCollection)"/>
approach for configuring
40
41 /// registered. <see
cref="MvcCoreServiceCollectionExtensions.AddMvcCore(IServiceCollection)"/> will register
42 /// the minimum set of services necessary to route requests and invoke controllers. It is
not expected that any
```

При което се вижда, че предоставя само основните services, без JsonResult.

18 – Model

Model in ASP.NET Core MVC



Искаме да извлечем данни от дадената БД-таблица и да ги изведем на екрана чрез уеб страница **Модел = данни + логика**

=> представя + управлява

Модел = 1 клас за съхранение на данните (таблица БД) + 1 клас за логиката + 1 интерфейс на самата логика (абстракция) =>

модел = 2 класа + 1 интерфейс



Извикваме интерфейса в HomeController

MVC Model

```

public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Department { get; set; }
}

public interface IEmployeeRepository
{
    Employee GetEmployee(int id);
    void Save(Employee employee);
}

public class EmployeeRepository : IEmployeeRepository
{
    public Employee GetEmployee(int id)
    {
        // Logic to retrieve employee details
    }

    public void Save(Employee employee)
    {
        // Logic to save employee details
    }
}

```

Employee Details	
ID	1
Name	John
Department	IT

Model
Employee
+
EmployeeRepository

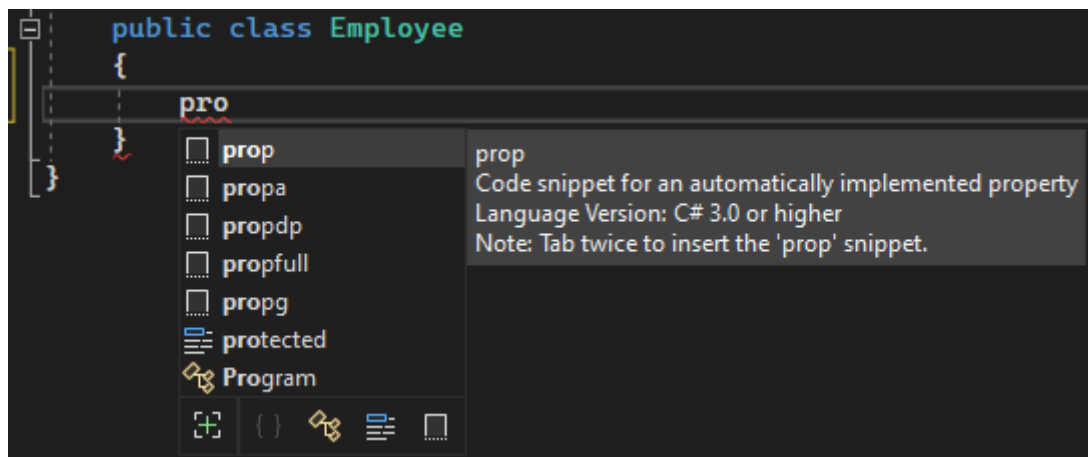
Models
C# Employee.cs - нов клас

Не е задължително моделите и контролерите да са в едноименни папки, те могат да бъдат навсякъде в проекта, но е добра практика да се съхраняват по този начин за по-лесно управление.

При Enter след къдравата скоба за попълване имаме подсказка:

```
1 namespace EmployeeManagement.Models
2 {
3     0 references
4     public class Employee
5     {
6         public int Id { get; set; }
7     }
```

Но искаме property:



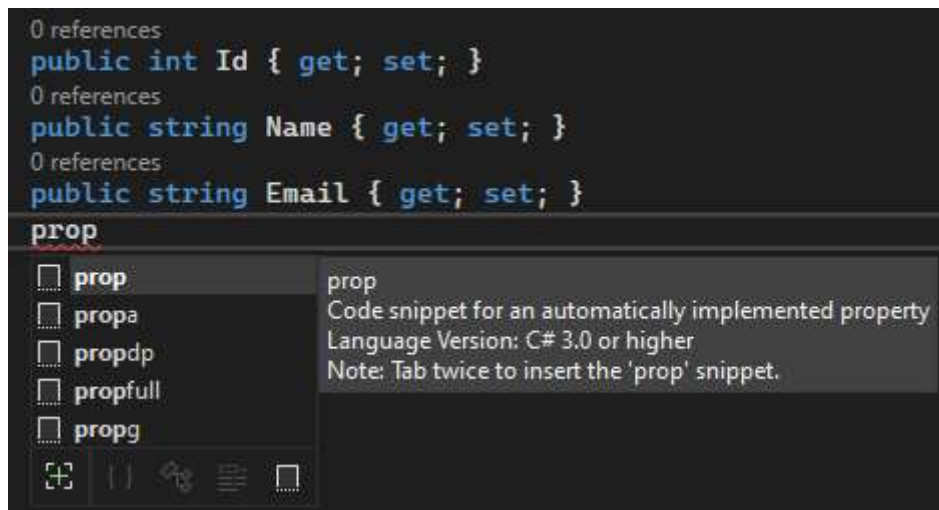
Tab x 2:

```
public class Employee
{
    0 references
    public int MyProperty { get; set; }
}
```

С единичен Tab прехвърляме само целите думи за редакция

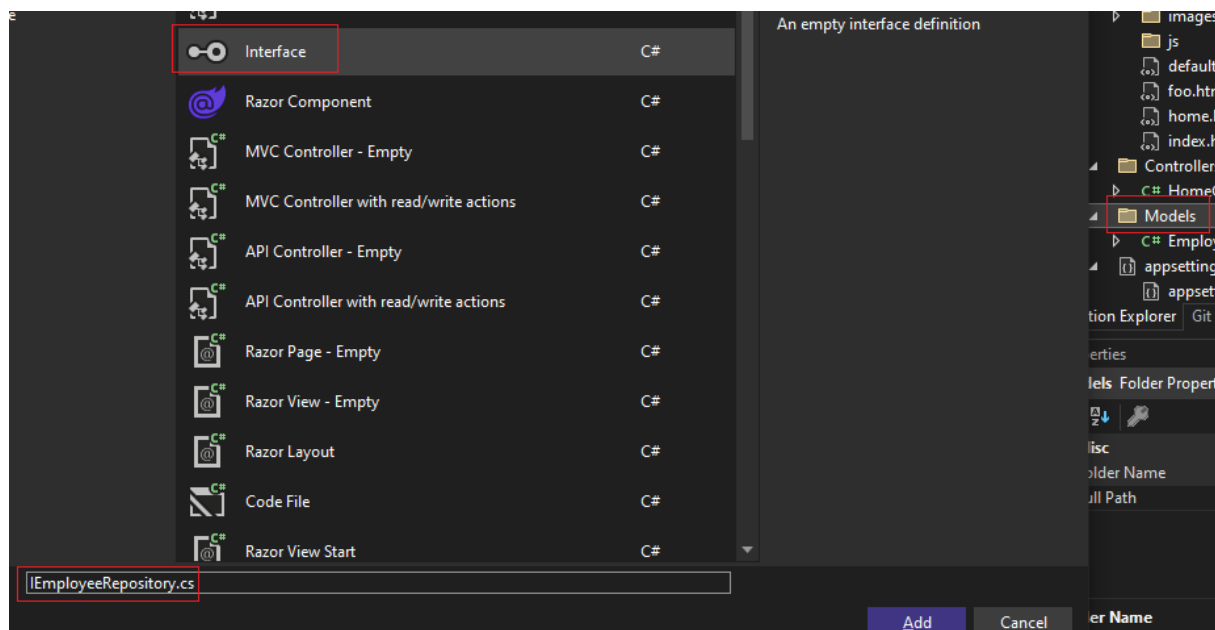
```
0 references
public int Id { get; set; }
public string Name { get; set; }
}
```

Автоматично дописва:



```
namespace EmployeeManagement.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Email { get; set; }
        public string Department { get; set; }
    }
}
```

T.e. ID, Name, Email, Department = property inside the class for DB-table



```
namespace EmployeeManagement.Models
{
    public interface IEmployeeRepository
    {
        Employee GetEmployee (int Id); // declaration of method
    }
}
```

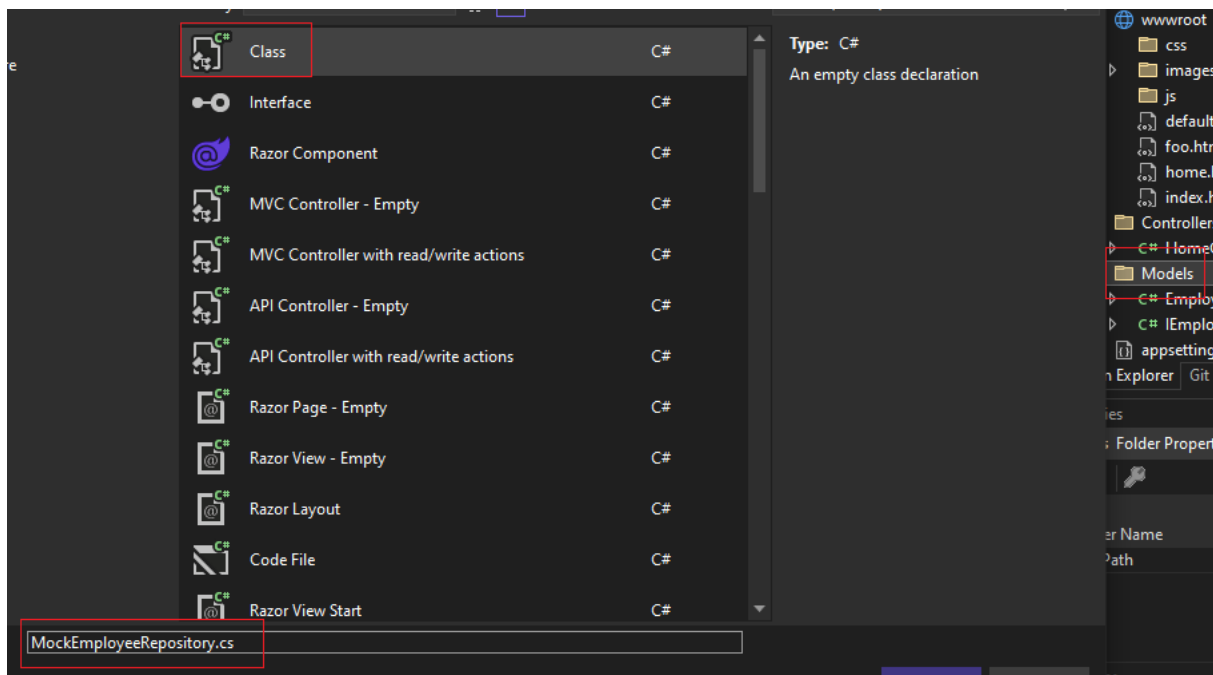
Към момента има само 1 метод GetEmployee в интерфейса, но той ще бъде допълван с още: safe, update, delete.

За удобство, всички интерфейси започват с 'I' –

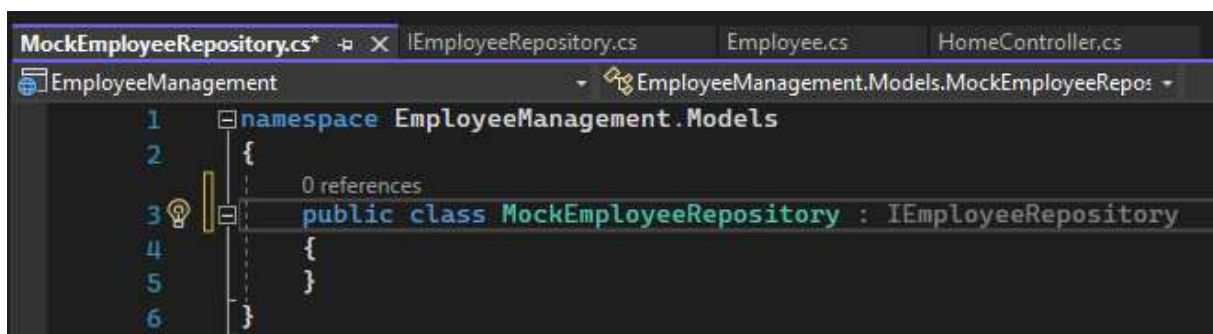
IEmployeeRepository = InterfaceEmployeeRepository.

Няма почти никакъв код, но се използва като интерфейс междинно за по-голяма гъвкавост.

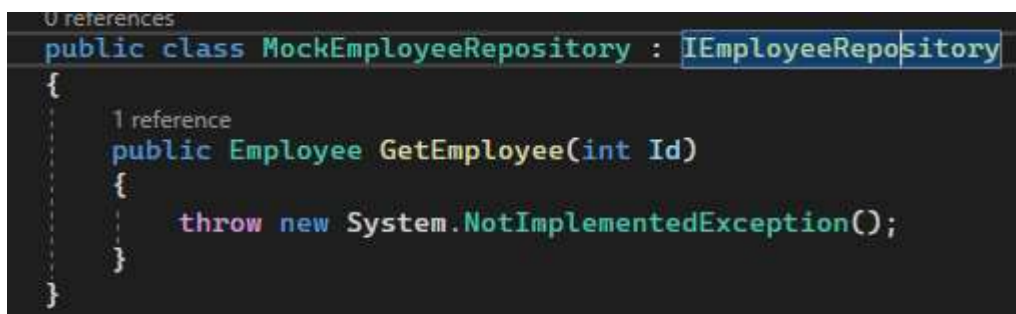
Цялата работа се извършва от MockEmployeeRepository.cs. Бихме могли принципно да използваме само MockEmployeeRepository.cs и програмираме само в него без интерфейс, но тогава няма да може да използваме dependency injection, но тогава приложението ще бъде много трудно за промяна, управление и тестване.



Последователност, добра за подсказките: БД, интерфейс, логика (която ще се свърже автоматично с интерфейса)



Веднага се включва подсказка за връзката с интерфейса., а след това Ctrl+. автокорекция:



Изключението е автоматично вмъкнато и го презаписваме, но първо допълваме над него сурови данни:

```
public class MockEmployeeRepository : IEmployeeRepository
{
    private List<Employee> _employeeList; // field
    1 reference
    public MockEmployeeRepository() // ctor
    {
        _employeeList = new List<Employee>()
        {
            new Employee() { Id = 1, Name = "Mary", Department = "HR", Email = "Mary@abv.bg" },
            new Employee() { Id = 2, Name = "John", Department = "IT", Email = "John@abv.bg" },
            new Employee() { Id = 3, Name = "Sam", Department = "IT", Email = "Sam@abv.bg" }
        };
    }

    2 references
    public Employee GetEmployee(int Id) //incoming id
    {
        return _employeeList.FirstOrDefault( e => e.Id == Id); //matches with the incoming id
    }
}
```

```
using System.Collections.Generic;
```

```
namespace EmployeeManagement.Models
```

```
{
    public class MockEmployeeRepository : IEmployeeRepository
    {
        private List<Employee> _employeeList; // list with Employee-type data
        public MockEmployeeRepository() // ctor
        {
            _employeeList = new List<Employee>() // hard-code data type Employee
            {
                new Employee() {Id = 1, Name = "Mary", Department = "HR", Email = "Mary@abv.bg"},
                new Employee() {Id = 2, Name = "John", Department = "IT", Email = "John@abv.bg"},
                new Employee() {Id = 3, Name = "Sam", Department = "IT", Email = "Sam@abv.bg"}
            };
        }

        public Employee GetEmployee(int Id) // incoming Id
        {
            return _employeeList.FirstOrDefault( e => e.Id == Id);
        }
    }
}
```

Връща резултат служителя от списъка, чието Id съвпада с подаденото Id.

Към момента, цялата работа се извършва от клас MockEmployeeRepository, който съхранява логиката.

Създаваме нов ctor в HomeController за връзка с интерфейса:

```
public class HomeController : Controller
{
    private IEmployeeRepository _employeeRepository; // autofield

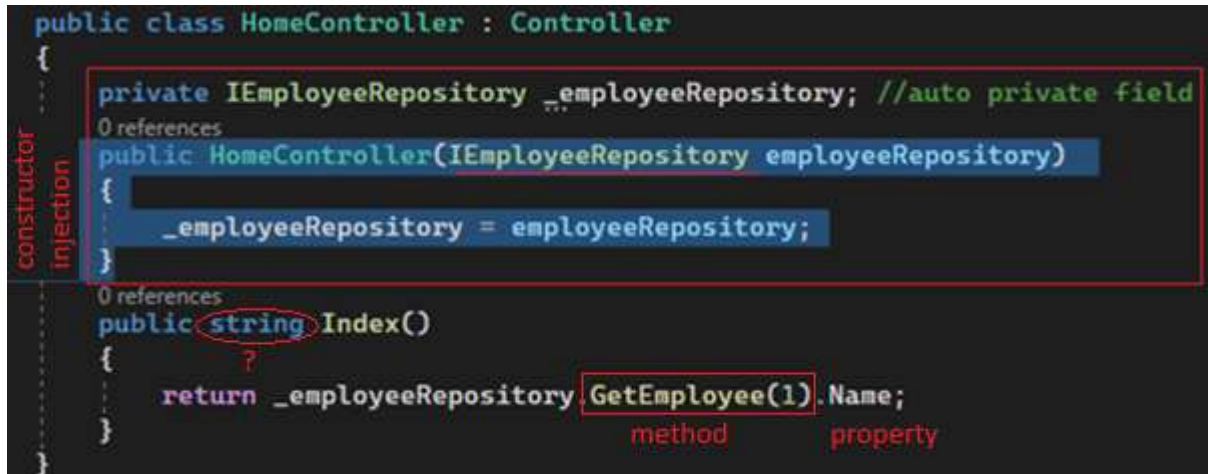
    public HomeController(IEmployeeRepository
employeeRepository)//ctor+depend
    {
        _employeeRepository = employeeRepository; // injected parameter!
    }
}
```

```

    } //ctor inhection

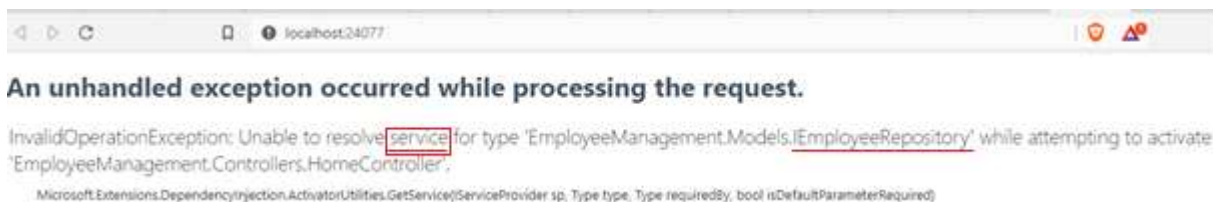
    public string Index()
    {
        return _employeeRepository.GetEmployee(1).Name;
    }
}

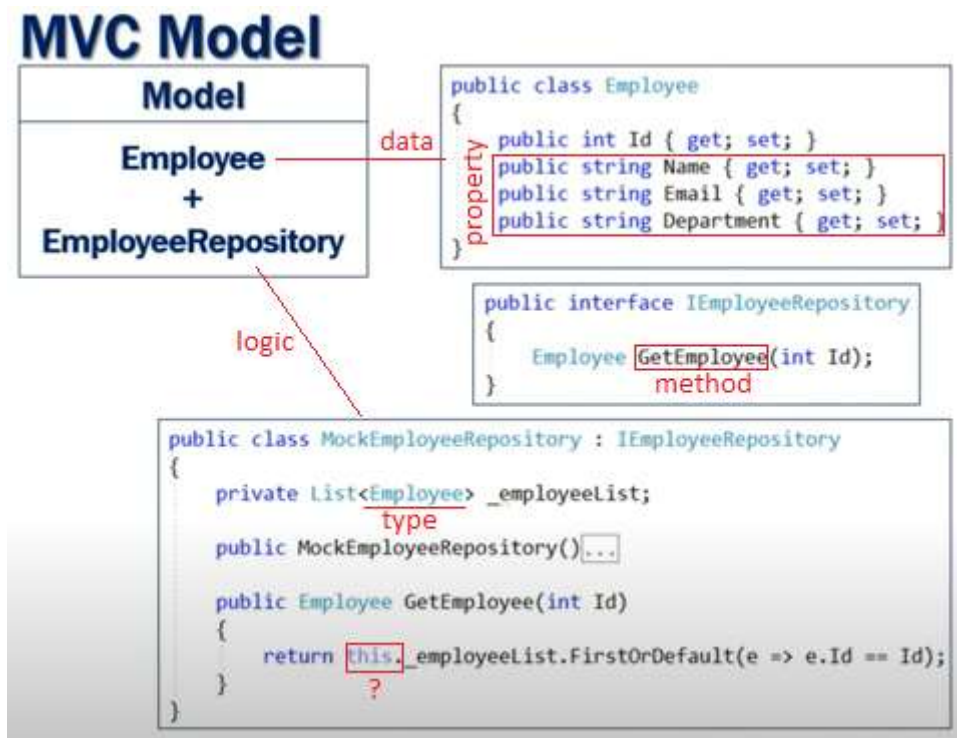
```



String? Защо да не е `ViewResult`? – трябва да създадем нов html документ във View. Със string не се налага – просто го извежда, изпълнява действието в базова стр.

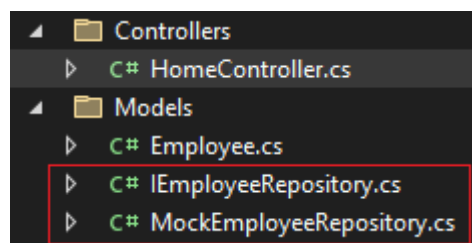
Правим връзка между най-важният `HomeController` и интерфейса чрез инжектиране с конструктор. Искаме да изведем резултат името на първия hard-code служител. Оградила съм стандартния constructor injection. Резултат:





19 – Dependancy injection

За да коригираме грешката при извикването на интерфейса в HomeController, добавяме един единствен ред код за типа зависимост между двата класа:

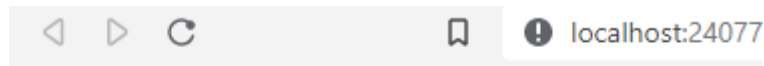


In Startup.cs - **add dependency** to dependency injection container: (ctrl. – namespace) ConfigureServices отговаря за вградените services във framework + нашите custom. В него параметър services е от тип IServiceCollection, който е интерфейс. Благодарение на него можем да добавим връзка тип Singleton между първо интерфейса IEmployeeRepository – и второ, имплементацията на този интерфейс MockEmployeeRepository.

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvcCore(option => option.EnableEndpointRouting = false);
    services.AddSingleton<IEmployeeRepository, MockEmployeeRepository>();
    // if someone (like HomeController) requests this IEmployeeRepository service,
    // create an instance of this MockEmployeeRepository class and inject it.
}
```

Към момента имаме само 1 имплементация на MockEmployeeRepository.

Резултат Ctrl + R (refresh):



Mary

Има и друго решение, което е по-лесно. Вместо:

```
public HomeController(IEmployeeRepository employeeRepository)
{
    _employeeRepository = employeeRepository;
}
```

Можем да не използваме интерфейса, а нова ключова дума MockEmployeeRepository:

```
public HomeController()
{
    _employeeRepository = new MockEmployeeRepository();
}
```

Програмата работи и извежда същият резултат, но прекалено силно обвързва клас MockEmployeeRepository – HomeController. Ако впоследствие се наложи да редактираме изпълнението, би следвало да използваме друга ключова дума като SequelEmployeeRepository (продължение) или друга. Към момента изглежда приемливо, т.к. имаме само 1 контролер, но при стандартен проект с по 50-60-100 контролера, в които използваме тази ключова дума, кодът навсякъде трябва да се редактира, което е не просто досадна работа, а предполагаща грешки, докато с DIC (dependency injection container), единствено трябва да го добавим в реда в Startup.cs, само на едно място, което прави нещата удобни за редакция, гъвкави и лесни за тестване.

```
services.AddSingleton<IEmployeeRepository, MockEmployeeRepository>();
```

* Singleton – first requested this same instance is then used by all the subsequent requests => created only one time per application & used throughout the app lifetime.

Този ред код показва, че ако някой като нашия HomeController.cs или друг, заявява request като IEmployeeRepository – service, тогава създава отделен случай instance на този MockEmployeeRepository-class и инжектира този instance.

To Register with Dependency Injection Container

- AddSingleton()
- AddTransient()
- AddScoped()

Benefits of DI

- Loose Coupling
- Easy to Unit Test

Singleton – единичен/самостоятелен: създава се само веднъж и използва по време на живот на приложението;

Transient – преходен: създава се и използва всеки път когато е извикан;

Scoped – обхват: създава се веднъж на заявка за всеки пример
(в MVC създава 1 instance за всеки http-request).

20 – Controller

Контролерите са с наставка/суфикс Controller и имат набор от public action methods.

Home|Controller --→ метод Index()

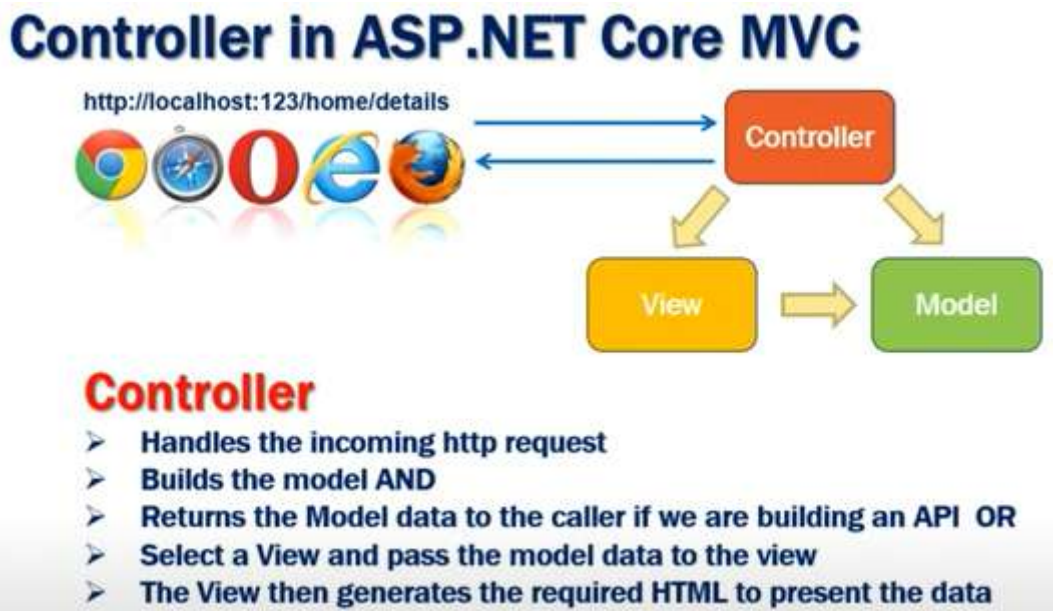
Employees|Controller

Products|Controller...

HTTP request -----→ Controller

A Controller class usually has a set of public (action) methods.

mapping localhost:1234/home/details = details public action method in HomeController class



ReadOnly – предотвратява инцидентно добавяне на нова променлива или метод в контролера.

Към момента имаме само Index method, нека добавим Details метод:

```

public class HomeController : Controller
{
    private readonly IEmployeeRepository _employeeRepository;
    0 references
    public HomeController()
    {
        _employeeRepository = new MockEmployeeRepository();
    }
    0 references
    public string Index()
    {
        return _employeeRepository.GetEmployee(1).Name;
    }
    0 references
    public JsonResult Details()
    {
        Employee model = _employeeRepository.GetEmployee(1);
        return Json(model);
    }
}

```

method

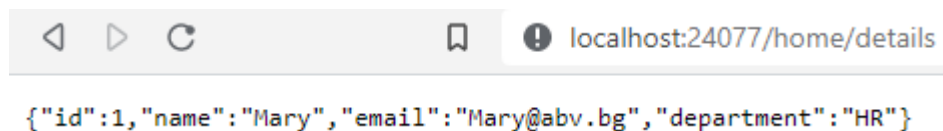
hard-coded instead of dynamically from URL

Вижда се dependency injection на интерфейс IEmployeeRepository в HomeController чрез ctor.

```

public JsonResult Details()
{
    Employee model = _employeeRepository.GetEmployee(1);
    return Json(model);
}

```



localhost:24077/home/details

```

{"id":1,"name":"Mary","email":"Mary@abv.bg","department":"HR"}

```

Виждаме резултатът от страницата като Json данни. При тях се пренебрегва форматирането на съдържанието, освен header-а. Ако искаме да има форматиране, сменяме формата:

```

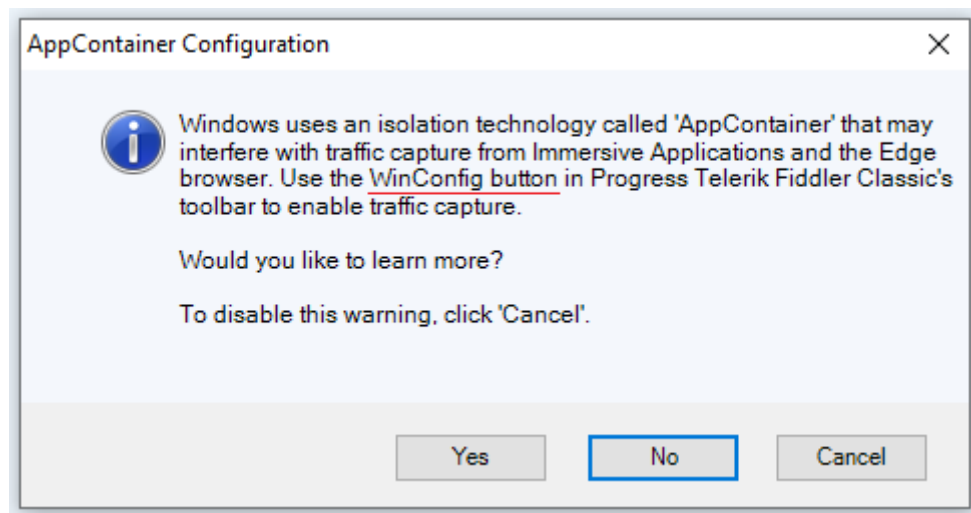
public ObjectResult Details()
{
    Employee model = _employeeRepository.GetEmployee(1);
    return new ObjectResult(model);
}

```

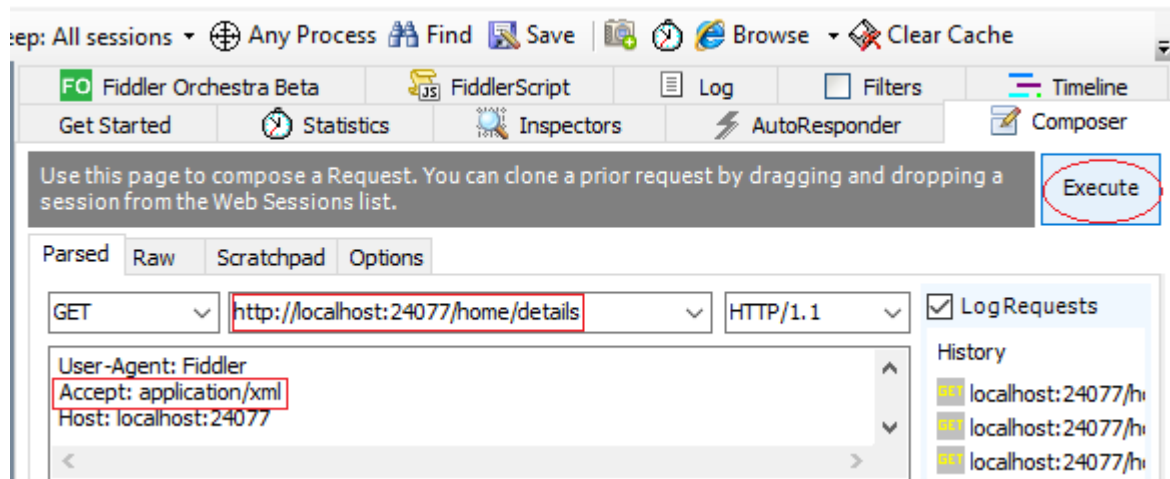
```

public ObjectResult Details()
{
    Employee model = _employeeRepository.GetEmployee(1);
    return new ObjectResult(model);
}

```

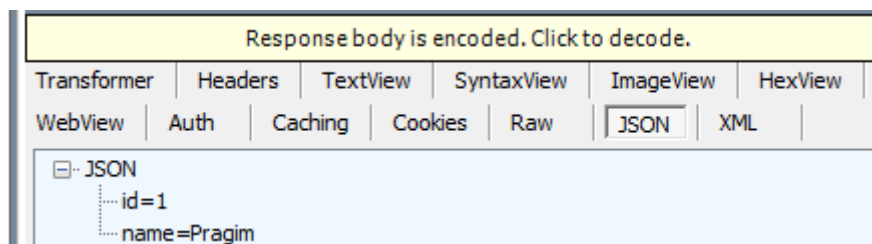
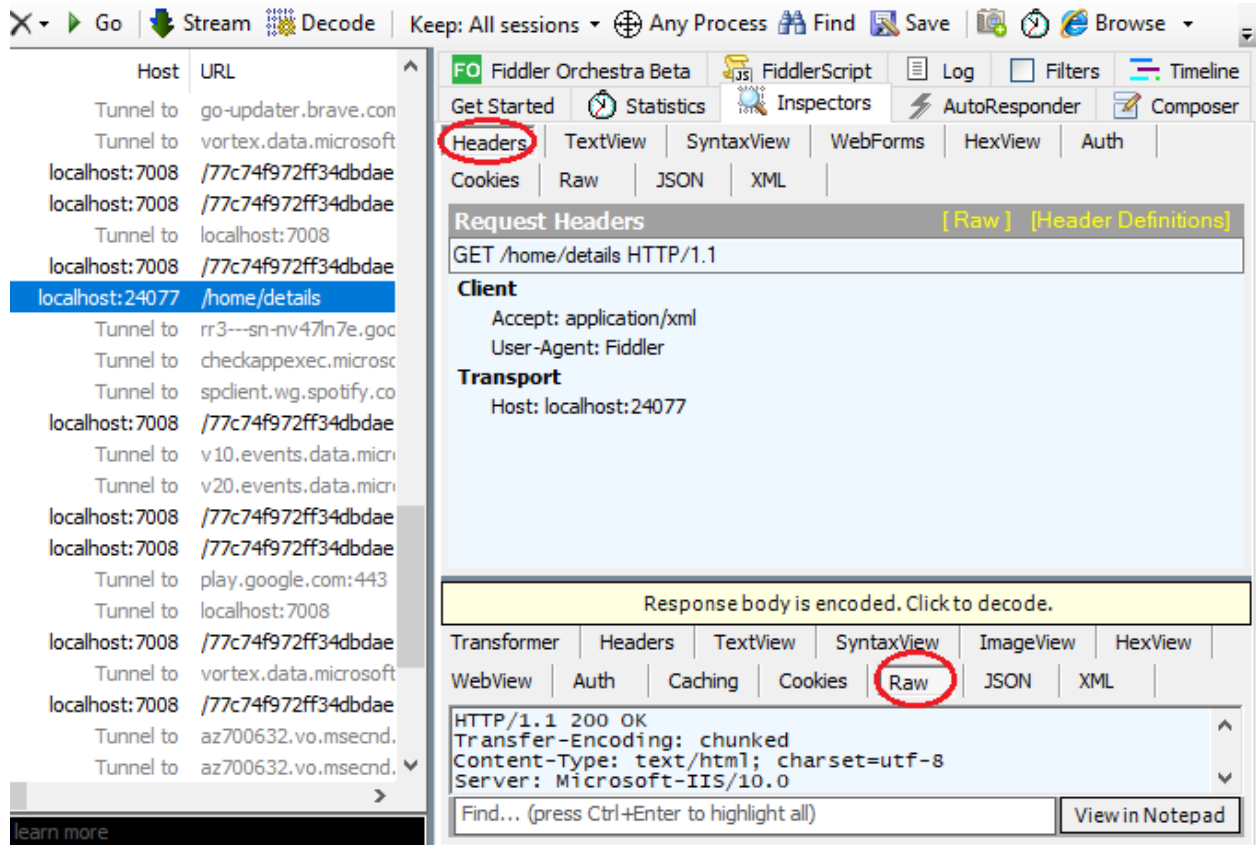


<https://www.telerik.com/download/fiddler> – когато искаме приемлив header + данни в XML-формат.



Инсталираме <https://www.telerik.com/download/fiddler/first-run>
изследваме <http://localhost:24077/home/details>
Accept: application/xml

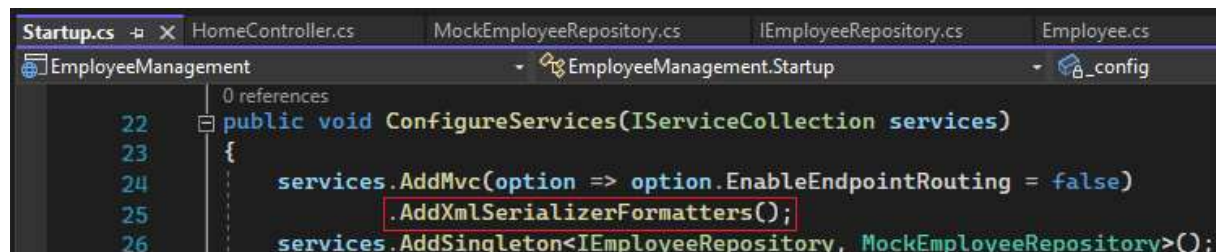
кликваме вляво на адреса



Виждаме данните обаче отново като Json, а не XML. За да стане това, добавяме разширение:

`.AddXmlSerializerFormatters();`

Програмата работи и с MCV извън Core, преди ми даваше отклонение в тази версия.



`services.AddMvc(option => option.EnableEndpointRouting = false)`
`.AddXmlSerializerFormatters();`

Отново Composer->Execute->последния линк вляво е нашият->Headers->Raw.

```
<html>
<head>
  <meta name="viewport" content="width=device-width" />
  <title>Employee Details</title>
</head>
<body>
  <div>

    <h3>Employee Details</h3>

    <div>
      Name: Mary
    </div>
    <div>
      Email: Mary@abv.bg
    </div>
    <div>
      Department: HR
    </div>

  </div>

  <script src="/js/CustomScript.js"></script>
</body>
</html>
```

Transformer Headers TextView SyntaxView ImageView HexView
WebView Auth Caching Cookies Raw JSON XML

html
head
 meta name="viewport" content="width=device-width" />
 title
 Employee Details
body
 div
 h3
 Employee Details
 div
 Name: Mary
 div
 Email: Mary@abv.bg
 div
 Department: HR
 script src="/js/CustomScript.js"></script>

Expand All Collapse

Първият път не ми се получи, но сега съм по-напред в туториъла и при повторението се виждат данните в тези формати.

21 – Views

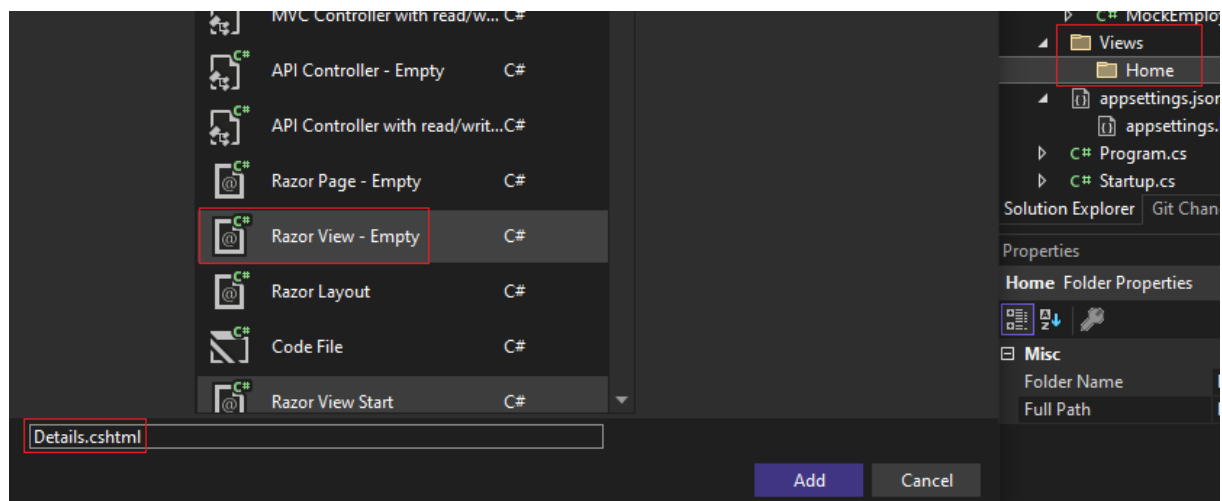
- * .cshtml = CS (C sharp) + html
- * Model -> Controller -> View (logic only to display data)
- * HTML template + razor markup

Нека променим типа на изведените данни:

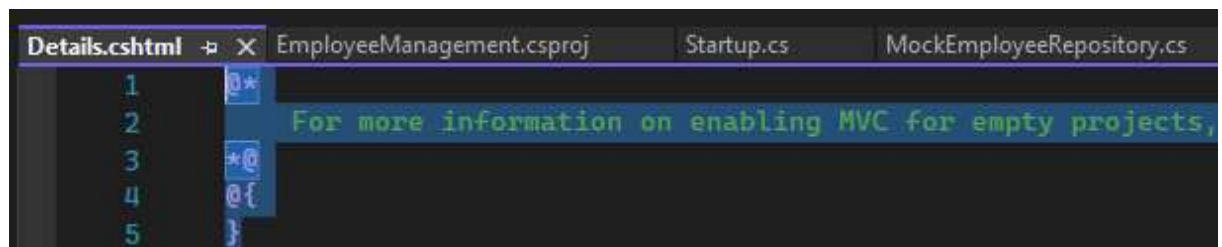
```
0 references
public IActionResult Details()
{
    Employee model = _employeeRepository.GetEmployee(1);
    return View(model);
}
```



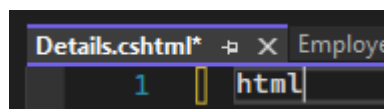

По подразбиране търси резултат тип View файлове .cshtml (c sharp) в папка View, подпапка Home заради контролера, в който се намира и action Details. Нека го създадем.



Изтриваме кода по подразбиране:

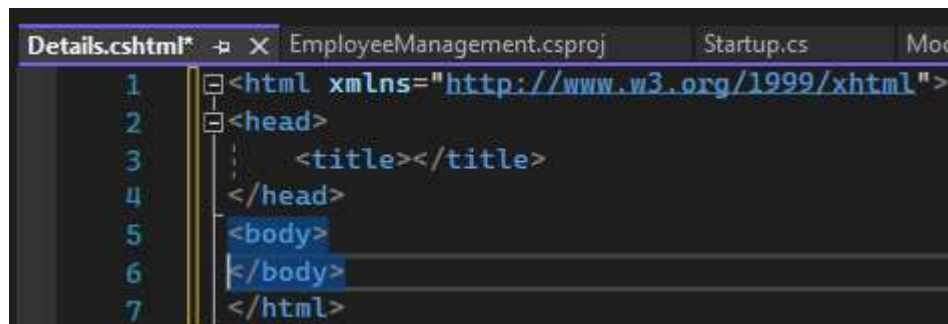
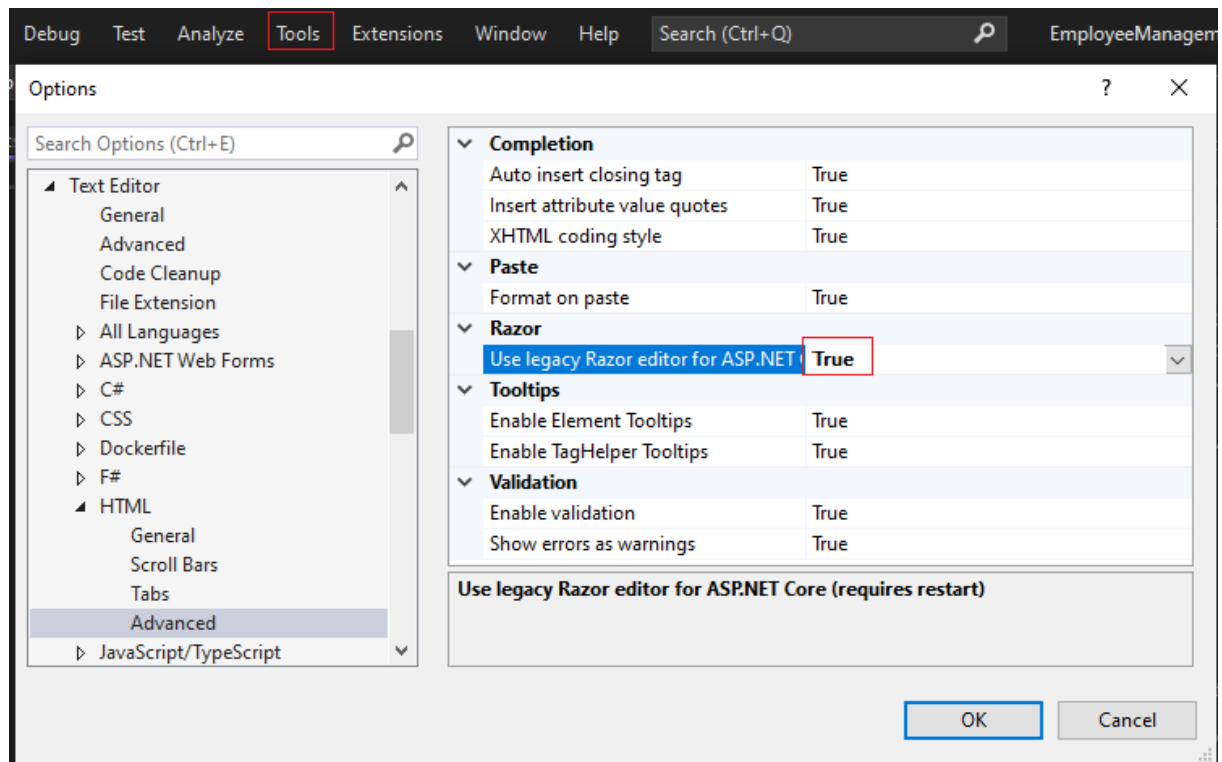


В по-старите версии при изписване на html + Tab x 2 -> автоматично се запълва код, но при мен с windows 7 нямаше проблем, а с 10 не се получаваше.



Накрая се оказа, че тази настройка по подразбиране е False и трябва да се включи и рестартира VS.

(Tools -> Options -> Text Editor -> HTML -> Advanced -> Use legacy Razor editor for ASP.NET Core) -> True



```
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title></title>
</head>
<body>
  <h1>Details.cshtml in View/Home</h1>
</body>
</html>
```

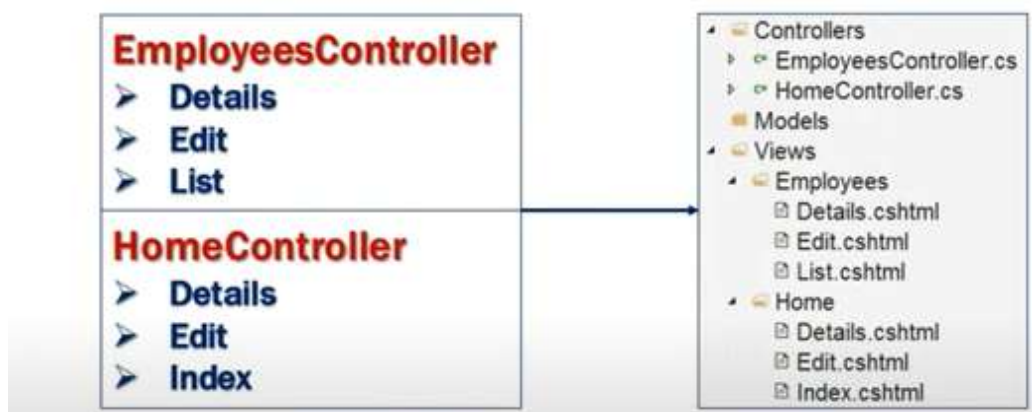
Вече няма грешка:



Details.cshtml in View/Home

Views in ASP.NET Core MVC

- A view file has .cshtml extension
- A view is an HTML template with embedded Razor markup
- Contains logic to display Model data



Това е структура по подразбиране, която може да се промени с допълнителни настройки.

22 – Customize View discovery

За да включим Intellisense, изтриване (и я дописваме пред model:

```
_employeeRepo { } Microsoft
}
MiddlewareFilterAttribute
MockEmployeeRepository
public string Ind[ ] model
{
    return _emplo
}
ModelBinderAttribute
ModelBinderFactory
ModelMetadataTypeAttribute
{ } Models
Employee mode
return View (model);
```

(parameter) object model
The model that is rendered by the view.

▲ 2 of 4 ▼ ViewResult Controller.View(object model)
Creates a ViewResult object by specifying a model to be rendered by the view.
model: The model that is rendered by the view.

Виждаме, че има 4 Overloaded versions of View() method.

1 – без параметър

```
View (model);
```

▲ 1 of 4 ▼ ViewResult Controller.View()
Creates a ViewResult object that renders a view to the response.

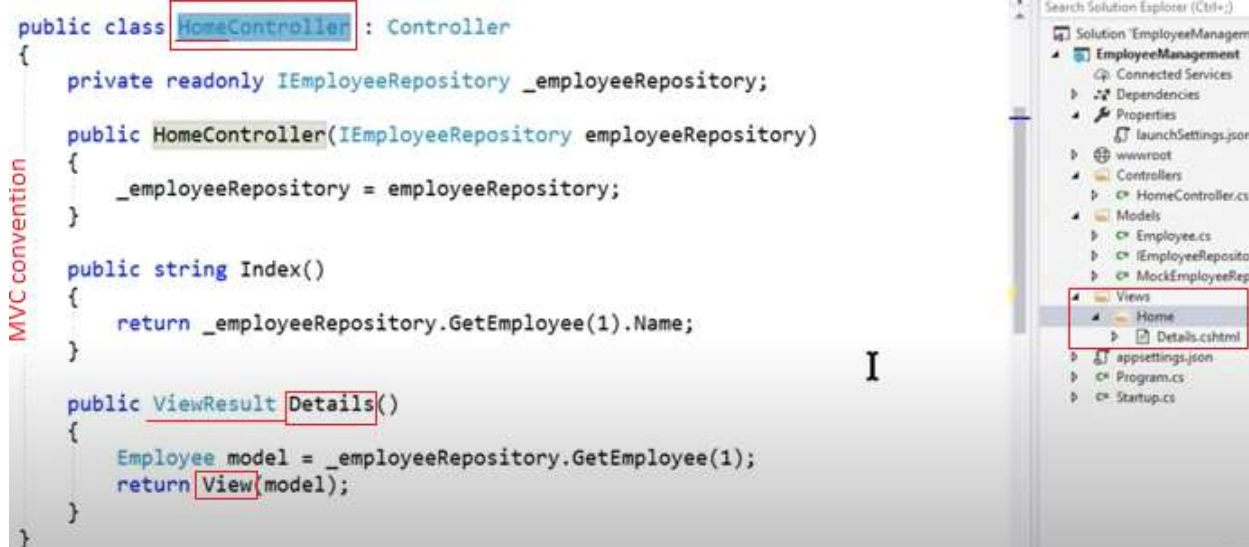
2 – с един параметър

```
View (model);
```

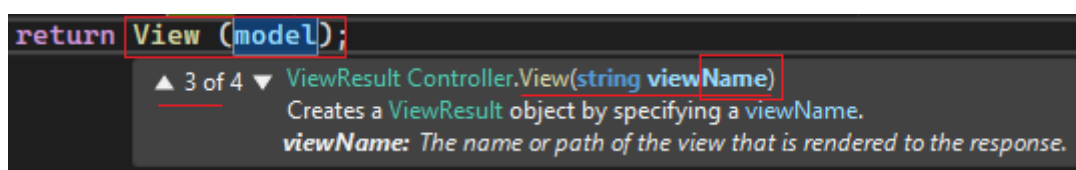
▲ 2 of 4 ▼ ViewResult Controller.View(object model)
Creates a ViewResult object by specifying a model to be rendered by the view.
model: The model that is rendered by the view.

View discovery convention:

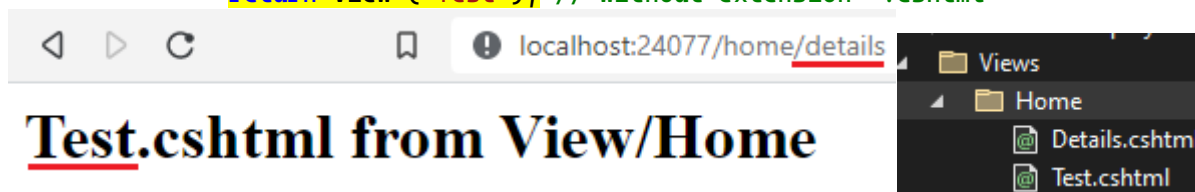
MVS търси по подразбиране документ View със същото име като action метода в папка с името на контролера, т.е. Details.cshtml (action) тип View (папка) в Home (Controller).



Можем да я персонализираме, като сменим конвенцията по подразбиране.

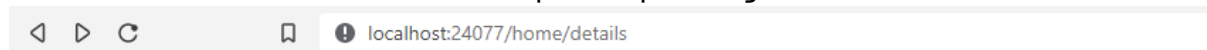


```
return View("Test"); // without extension *.cshtml
```



Всичко това се изпълнява от HomeController в Details, така че, когато зададем директория http://localhost:24077/home/test където всъщност се намира файла, връща липсващ 404.

Това е относителен адрес. Ако зададем разширение Test.cshtml връща грешка, но от DeveloperExceptionPage:



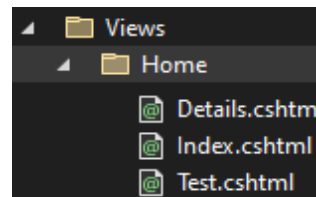
An unhandled exception occurred while processing the request.

InvalidOperationException: The view 'Test.cshtml' was not found. The following locations were searched: /Test.cshtml

Microsoft.AspNetCore.Mvc.ViewEngines.ViewEngineResult.EnsureSuccessful(IEnumerable<string> originalLocations)

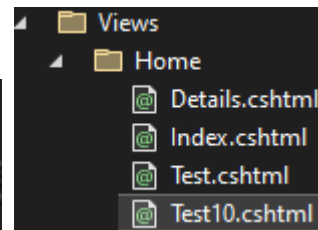
Когато си ретроспектирах упражнението, създадох тестови action:

```
0 references
public IActionResult Test()
{
    return View();
}
```



Друг опит:

```
public IActionResult Test()
{
    return View("Test10");
}
```



Което е еднакво с абсолютния

```
return View("~/Views/Home/Test10.cshtml");
```

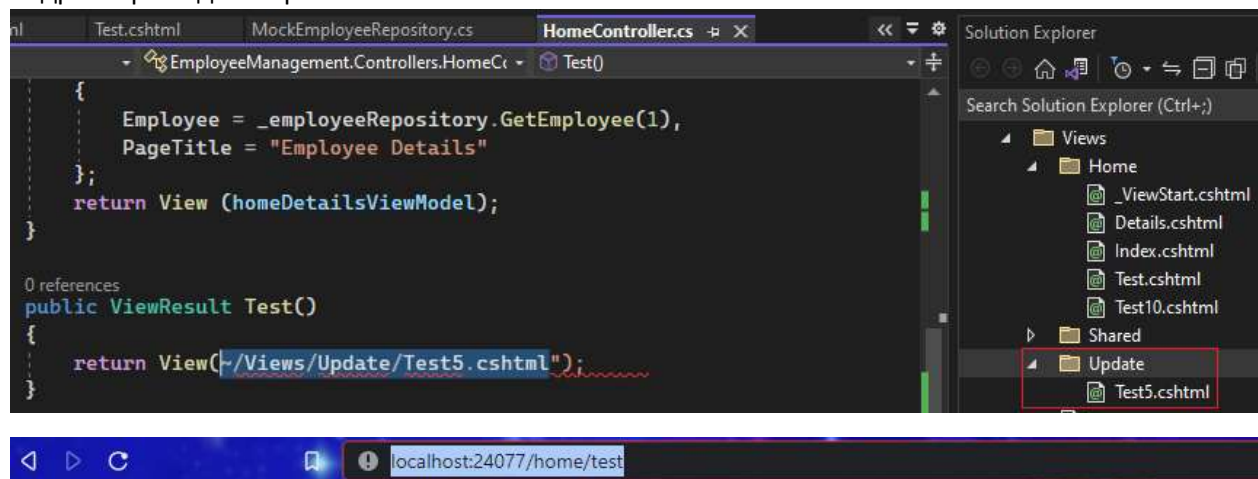
И относителния

```
return View("../Views/Home/Test");
```



Test10

Всички опити, при които се използват някакъв тестови View извън папката на Home, по подразбиране дават грешки.



An unhandled exception occurred while processing the request.

InvalidOperationException: The view '/Update/Test5' was not found. The following locations were searched:
/Update/Test5

Microsoft.AspNetCore.Mvc.ViewEngines.ViewEngineResult.EnsureSuccessful(IEnumerable<string> originalLocations)

Липсва разширението .cshtml при търсене в грешката.

Тази папка Update все едно не съществува и търси във Views, но не я вижда:


```

0 references
public ActionResult Test()
{
    return View("../Update/Test5");
}

```

localhost:24077/home/test

An unhandled exception occurred while processing the request.

InvalidOperationException: The layout view '_NonAdminLayout' could not be located. The following locations were searched:
 /Views/Home/_NonAdminLayout.cshtml
 /Views/Shared/_NonAdminLayout.cshtml

Явно е свързано с някой от следващите уроци, в които вече имам Layout структура.

```

_ViewStart.cshtml
1  @if (User.IsInRole("Admin"))
2  {
3      Layout = "_AdminLayout";
4  }
5  else
6  {
7      Layout = "_NonAdminLayout";
8  }
9
_ViewImports.cshtml
_ViewStart.cshtml

```

```

return View("../Views/Update/Test5");

```

Fix formatting
 Introduce constant
 Introduce parameter for "'../Views/Update/Test5'"
 Add argument name 'viewName'
 Convert to raw string
 Use expression body for methods
 Suppress or Configure issues

IDE0055 Fix formatting
 Lines 30 to 32

```

{
    return View("../Views/Update/Test5");
}

```

 Preview changes
 | Document | Project | Solution

```

public ActionResult Test() => View("~/Views/Update/Test5.cshtml");

```

Introduce constant
 Introduce parameter for "'~/Views/Update/Test5.cshtml'"
 Add argument name 'viewName'
 Convert to raw string
 Use block body for methods
 Suppress or Configure issues

IDE0052 Use block body for methods
 Lines 30 to 30

```

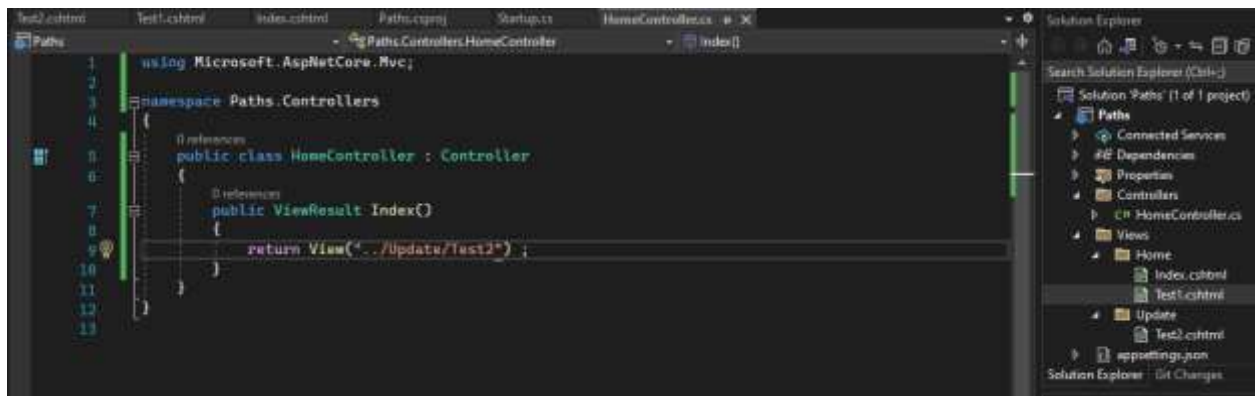
public ActionResult Test() => View("~/Views/Update/Test5.cshtml");
public ActionResult Test()
{
    return View("~/Views/Update/Test5.cshtml");
}

```

 Preview changes
 | Document | Project | Solution

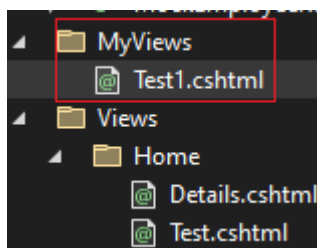
Views
 Home
 _ViewStart.cshtml
 Details.cshtml
 Index.cshtml
 Test.cshtml
 Test10.cshtml
 Shared
 Update
 Test5.cshtml
 _ViewImports.cshtml
 _ViewStart.cshtml

Направих нов проект, в който пътищата работят:



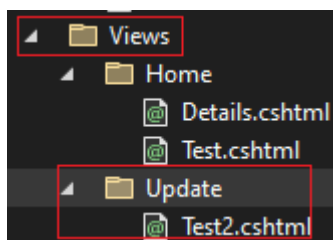
Това са стари скрийншоти от Windows 7, при които тръгваше без грешки.

`return View ("MyViews/Test1.cshtml");` като абсолютен адрес с разширение може да използваме и знаците за връщане в главна директория `~/` или `../`



```
return View ("~/MyViews/Test1.cshtml");
```

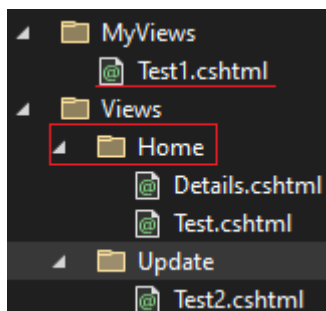
root directory



```
return View ("../Update/Test2");
```

1 директория ↑ от Home

И понеже е относителен адрес, би трябвало дори така да работи: `return View ("../Update");`



```
return View ("~/MyViews/Test1.cshtml");
```

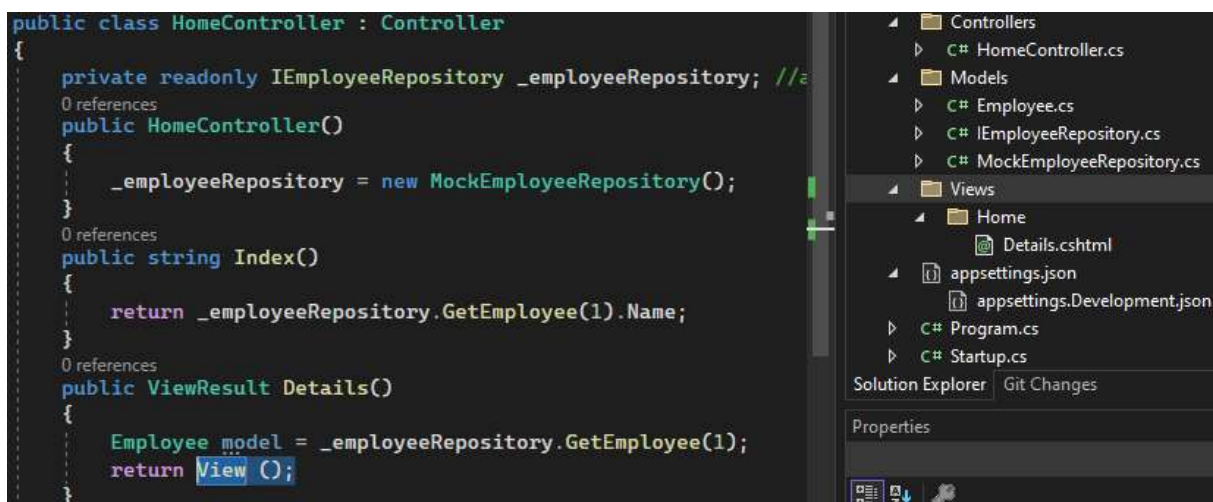
```
return View ("../../MyViews/Test1");
```

Т.е. използваме ли `~` за root, адресът е абсолютен `.cshtml`, иначе не се използва разширение.

Customize View Discovery

- **View() or View(object model)** : Looks for a view file with the same name as the action method
- **View(string viewName)**
 - Looks for a view file with your own custom name
 - You can specify a view name or a view file path
 - View file path can be absolute or relative
 - With absolute path **.cshtml** extension must be specified
 - With relative path do not specify the file extension **.cshtml**
- **View(string viewName, object model)**

Изтриваме всички тестови файлове, папки и код и според tutorиъла, в Details() оставяме View() без параметри.



```
public class HomeController : Controller
{
    private readonly IEmployeeRepository _employeeRepository; //0 references
    public HomeController()
    {
        _employeeRepository = new MockEmployeeRepository();
    }
    public string Index()
    {
        return _employeeRepository.GetEmployee(1).Name;
    }
    public ViewResult Details()
    {
        Employee model = _employeeRepository.GetEmployee(1);
        return View();
    }
}
```

The Solution Explorer on the right shows the following structure:

- Controllers
 - C# HomeController.cs
- Models
 - C# Employee.cs
 - C# IEmployeeRepository.cs
 - C# MockEmployeeRepository.cs
- Views
 - Home
 - Details.cshtml
 - appsettings.json
 - appsettings.Development.json
 - C# Program.cs
 - C# Startup.cs

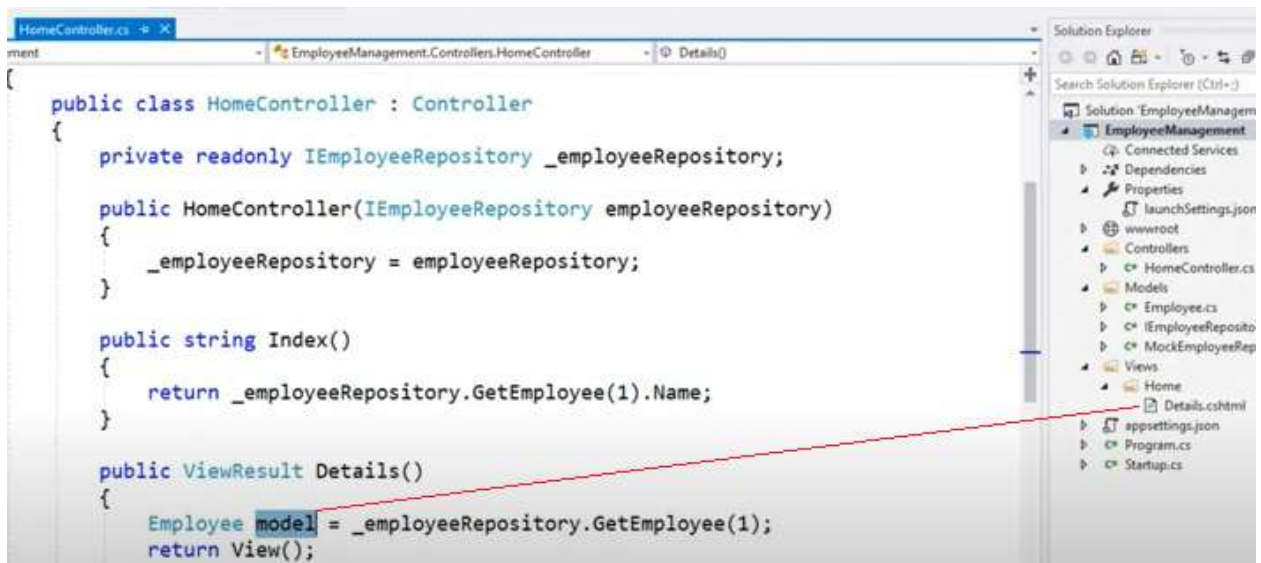
23 – Passing data to View

Passing Data to View from Controller

Different ways of passing data to a View from a Controller

- ViewData
 - ViewBag
 - Strongly Typed View
- Loosely Typed View

Искаме да използваме този Employee model object от HomeController, Details() в Details.cshtml-view.

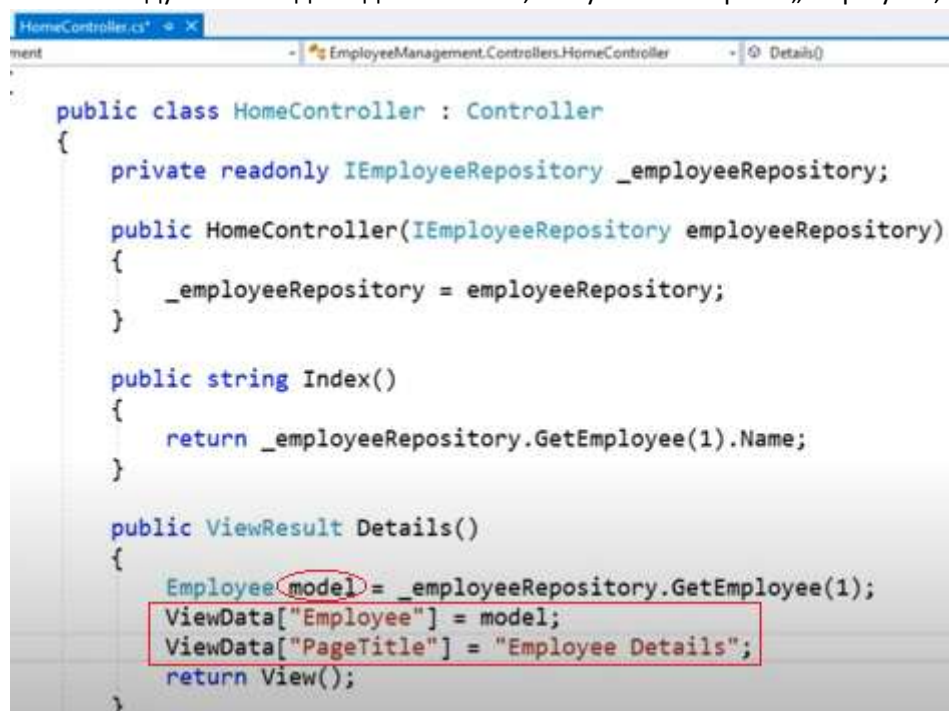


За целта ще използваме ViewData:

ViewData

- Dictionary of weakly typed objects
- Use string keys to store and retrieve data
- Dynamically resolved at runtime
- No compile - time type checking and Intellisens

За целта ключовите думи могат да бъдат всякакви, в случая са избрани „Employee”, „PageTitle”



Те са статични [...],

На Employee е присвоен обектът model, който е от сложен custom type Employee class.

На PageTitle е присвоена стойност "Employee Details".

Така използваме ViewData Dictionary - контейнери за данни с KeyWords, за да ги използваме за предаване другаде.

```
Employee model = _employeeRepository.GetEmployee(1);  
ViewData ["Employee"] = model;  
ViewData["PageTitle"] = "Employee Details";  
return View ();
```

```
ViewData ["Employee"] = model;  
ViewData["PageTitle"] = "Employee Details";
```

Първото нещо е да сменим

<h1> -> <h3>, това е Razor View, в което може да комбинираме C# + HTML

@ - означава начало на C# код @{ блок C# код }



В случая на PageTitle се използва string тип данни, затова не е нужно изречно да бъде обяснен какъв тип данни е, това е автоматично направено, но ако се използва какъвто и да е друг тип данни, той трябва обезателно да бъде упоменат. Такъв е `var employee = ViewData["Employee"] as EmployeeManagement.Models.Employee;` като сме посочили къде се намира този сложен custom type object class

```
<html xmlns="http://www.w3.org/1999/xhtml">  
<head>  
  <title></title>  
</head>  
<body>  
  <h3>@ViewData["PageTitle"]</h3>  
  
  @{  
    var employee = ViewData["Employee"]  
    as EmployeeManagement.Models.Employee;  
  }  
  <div>  
    Name: @employee.Name  
  </div>  
  <div>  
    Email: @employee.Email  
  </div>  
  <div>  
    Department: @employee.Department  
  </div>
```



```
</body>  
</html>
```



Employee Details

Name: Mary
Email: Mary@abv.bg
Department: HR

Понеже ViewData ["Employee"] е произволен текст, пробвах със смяна на ViewData ["Employee1"] на двете места в HomeController + Details и се получи.

По същият начин пробвах да сменя новата локална променлива в Details: var employee -> employee5 на всички места, в които се среща и няма проблеми. Т.е. макар да се повтарят навсякъде из файловете Employee, employee, това са локални променливи, които важат само за отделните файлове. За да се използват за прехвърляне на данни, използваме ViewData, ViewBag, др. Проблемът е, че те са лесни за печатни грешки (typographical errors) с този loosely type без Intellisense. Ако сгрешим и оставим Employee на едно място, Build-а е успешен, т.е. няма да разберем за нея в CompileTime, чак после в браузъра в Runtime и страницата ще крашне, т.е. съкращава времето без проверка за грешки Intellisense, не се компилира, а динамично се изпълнява по време на стартиране, затова принципно ViewData се избягва.

ViewData използва произволни random string keys, които при грешка с изписването се компилират, а тя се открива чак при динамично презареждане на страницата в runtime, като крашне. Затова ViewBag е по-добър.

24 – ViewBag

ViewBag = обвивка около ViewData

* ViewData използва статични string keys [" "]

```
ViewData ["Employee"] = model;  
ViewData ["PageTitle"] = "Employee Details";
```

* ViewBag използва dynamic properties .

```
ViewBag.Employee = model;  
ViewBag.PageTitle = "Employee Details";
```

При ViewData, ако е string, няма нужда да се указва по подразбиране, но когато типът е различен, както в случая Employee е сложен тип данни/клас, трябва да го укажем. При ViewBag не е необходимо, затова изтриваме целия блок код:

```
@{  
    var employee = ViewData["Employee"] as EmployeeManagement.Models.Employee;  
}
```

```

<body>
  <h3>@ViewBag.PageTitle</h3>

  <div>
    Name: @ViewBag.Employee.Name
  </div>
  <div>
    Email: @ViewBag.Employee.Email
  </div>
  <div>
    Department: @ViewBag.Employee.Department
  </div>
</body>

```

```

<body>
  <h3>@ViewBag.PageTitle</h3>

  <div>
    Name: @ViewBag.Employee.Name
  </div>
  <div>
    Email: @ViewBag.Employee.Email
  </div>
  <div>
    Department: @ViewBag.Employee.Department
  </div>
</body>

```

Също вероятността за грешки е голяма. Ако дадем курсора върху данните, единственото, което виждаме като информация е, че е динамичен тип (с присвояване .), дори не виждаме типа дали е string, integer или сложен като Employee:

```

<h3>@ViewBag.PageTitle</h3>
<div>
  Name: @ViewBag.Employee.Name

```

dynamic
Represents an object whose operations will be resolved at runtime.

Също като оставим само ., виждаме, че няма Intellisense:

```

<h3>@ViewBag.</h3>
<div>
  Name: @Vi
</div>
<div>
  Email: @V
</div>
<div>
  Departmen
</div>

```

If the a element has an href attribute, then it represents a hyperlink (a hypertext anchor) labeled by its contents.

Microsoft.AspNetCore.Mvc.Razor.TagHelpers.UrlResolutionTagHelper
Microsoft.AspNetCore.Mvc.Razor.TagHelpers.TagHelper implementation targeting elements containing attributes with URL expected values. Resolves URLs starting with '~/ (relative to the application's 'webroot' setting) that are not targeted by other Microsoft.AspNetCore.Mvc.Razor.TagHelpers.TagHelpers. Runs prior to other Microsoft.AspNetCore.Mvc.Razor.TagHelpers.TagHelpers to ensure application-relative URLs are resolved.

ee.Department

Connected Services

- C# IEmployeeRepo
- C# MockEmployee
- Views
- Home

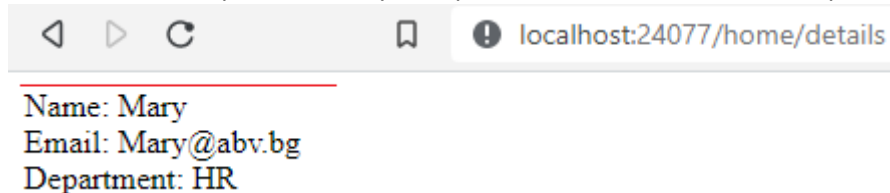
Отново, ако допуснем печатна грешка, компилирането е успешно:

```
<body>
  <h3>@ViewBag.PageTitle</h3>
  <div>
    Name: @ViewBag.Employee.Name
  </div>
  <div>
    Email: @ViewBag.Employee.Email
  </div>
  <div>
    Department: @ViewBag.Employee.Department
  </div>
</body>
</html>
```

No issues found

om: Build
Management -> D:\Exercises\CS_Begin\EmployeeManagement\bin\Debug\ne
Management -> D:\Exercises\CS_Begin\EmployeeManagement\bin\Debug\ne
Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====

и чак в Runtime я откриваме, в случая просто липсва заглавието на страницата:



Пробвах същият вид грешка с ViewData – също данните се зареждат и просто липсва заглавието на страницата „Employee Details”.

Ето защо употребата на ViewBag за заглавие на страниците е приемлива – пренася се малък текст информация, която не е фатално да бъде сгрешена и не се отразява върху съдържанието, а просто няма да се зареди самото заглавие.

Ако сгрешим обаче дори само на едно място в частта със съдържанието (липсваща буква ,е’), цялата страница крашва:

```
<div>
  Name: @ViewBag.Employee.Name
</div>
<div>
  Email: @ViewBag.Employee.Email
</div>
```

An unhandled exception occurred while processing the request.

RuntimeBinderException: Cannot perform runtime binding on a null reference

CallSite.Target(Closure, CallSite, object)

Stack Query Cookies Headers Routing

RuntimeBinderException: Cannot perform runtime binding on a null reference

CallSite.Target(Closure, CallSite, object)

System.Dynamic.UpdateDelegates.UpdateAndExecute1<T0, TRet>(CallSite site, T0 arg0)

AspNetCore.Views_Home_Details.<ExecuteAsync>b__8_10 in Details.cshtml

9. Name: @ViewBag.Employee.Name

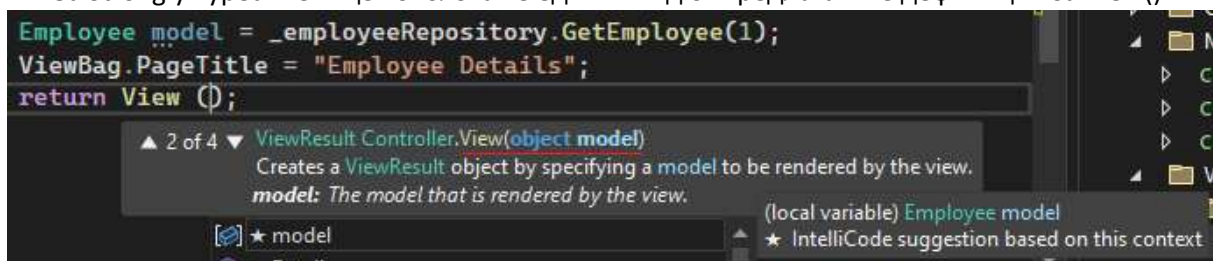
Не може да изпълни (perform) Employee -> връща null, и тази null се опитва да я присвои на Name.

ViewBag v/s ViewData

- ViewBag Is a wrapper around ViewData
- Creates a loosely typed view
- ViewData uses string keys to store and retrieve data
- ViewBag used dynamic properties to store and retrieve data
- Resolved dynamically at runtime
- No compile - time type checking and intellisense
- Preferred approach to pass data from a controller to a view is by using a Strongly Typed View

25 – Strongly Typed View

За Strongly Typed View ще използваме единия вид от предлаганите дефиниции за View():



HomeController:

```
public ViewResult Details()
{
    Employee model = _employeeRepository.GetEmployee(1);
    ViewBag.PageTitle = "Employee Details";
    return View(model);
}

public ViewResult Details()
{
```

В съдържанието Details е с главна буква Model, но все още не се включва IntelliSense:

Защото Details view все още не е Strongly Typed View.

```
<body>
  <h3>@ViewBag.PageTitle</h3>

  <div>
    Name: @Model.Name
  </div>
  <div>
    Email: @Model.Email
  </div>
  <div>
    Department: @Model.Department
  </div>
</body>
```

Department: @Model.Department

div>

>

 dynamic
Represents an object whose operations will be resolved at runtime.

The screenshot shows the Visual Studio interface. On the left, the Solution Explorer displays the 'Department' project structure, including files like '@', 'abbr', 'address', 'applet', 'area', 'article', and 'article'. On the right, the Output window shows the documentation for the 'Microsoft.AspNetCore.Razor.TagHelpers.UrlResolutionTagHelper'. The documentation states: 'If the element has an href attribute, then it represents a hyperlink (a hypertext anchor) labeled by its contents. Microsoft.AspNetCore.Razor.TagHelpers.UrlResolutionTagHelper Microsoft.AspNetCore.Razor.TagHelpers.TagHelper implementation targeting elements containing attributes with URL expected values. Resolves URLs starting with '/' (relative to the application's 'webroot' setting) that are not targeted by other Microsoft.AspNetCore.Razor.TagHelpers.TagHelpers. Runs prior to other Microsoft.AspNetCore.Razor.TagHelpers to ensure application-relative URLs are resolved.'

За целта на първи ред използваме с малка буква @model + директория @model.EmployeeManagement.Models.Employee т.е. указваме откъде да се включи библиотеката, класът, в който се намира този сложен тип

данни, нещо като namespace

Strongly Type = ред 1: @model EmployeeManagement.Models.Employee – дава големи възможности за писане на код без грешки с избор на всички допустими опции и корекции на грешки за по-добра производителност.

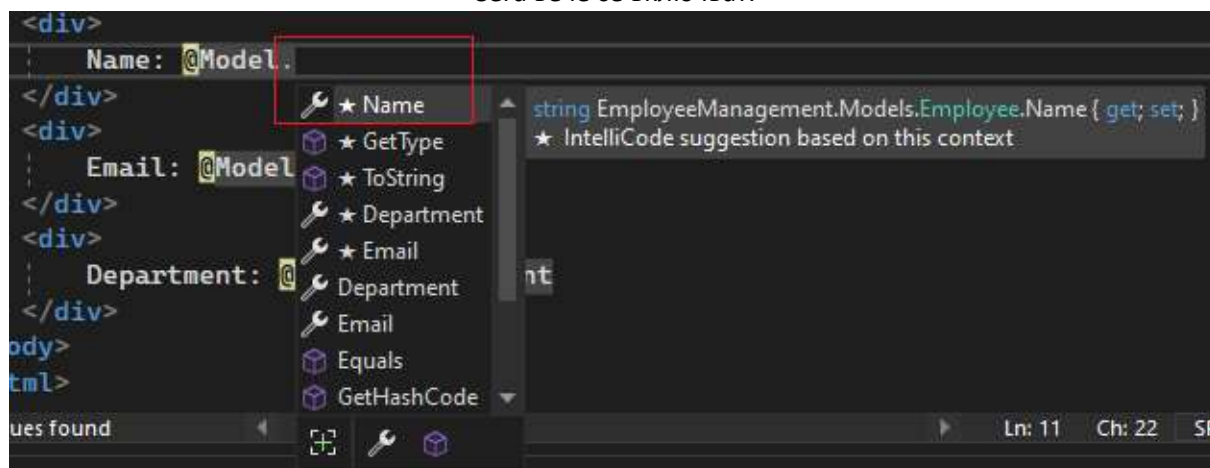
Strongly Typed View provides compile-time type checking + Intellisense.

```
@model EmployeeManagement.Models.Employee

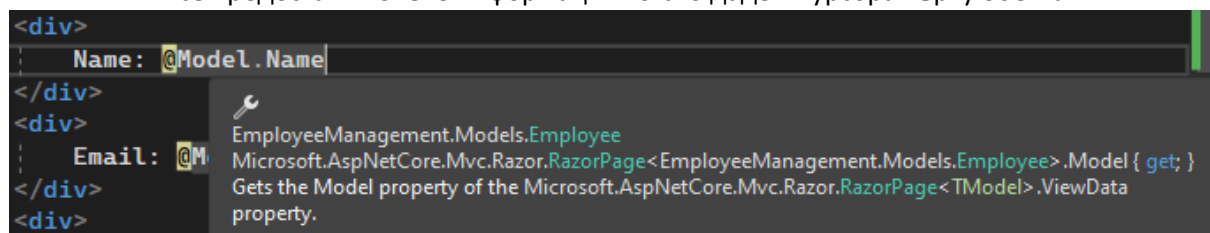
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title></title>
</head>
<body>
  <h3>@ViewBag.PageTitle</h3>

  <div>
    Name: @Model.Name
  </div>
  <div>
    Email: @Model.Email
  </div>
  <div>
    Department: @Model.Department
  </div>
</body>
</html>
```

Сега вече се включват:



И се предоставя повече информация когато дадем курсора върху обекта:



Когато сгрешим, получаваме червено подчертаване:

```
<div>
    Name: @Model.Nam
</div>
```

To create a Strongly Typed View

- Specify model type in the view using @model directive

```
@model EmployeeManagement.Models.Employee
```

- To access the model object properties we use @Model

```
<div>
    Name : @Model.Name
</div>
<div>
    Email : @Model.Email
</div>
```

малка буква 'm'
+
голяма буква 'M'

- Strongly Typed View provides **compile-time type checking and intellisense**

26 – ViewModel

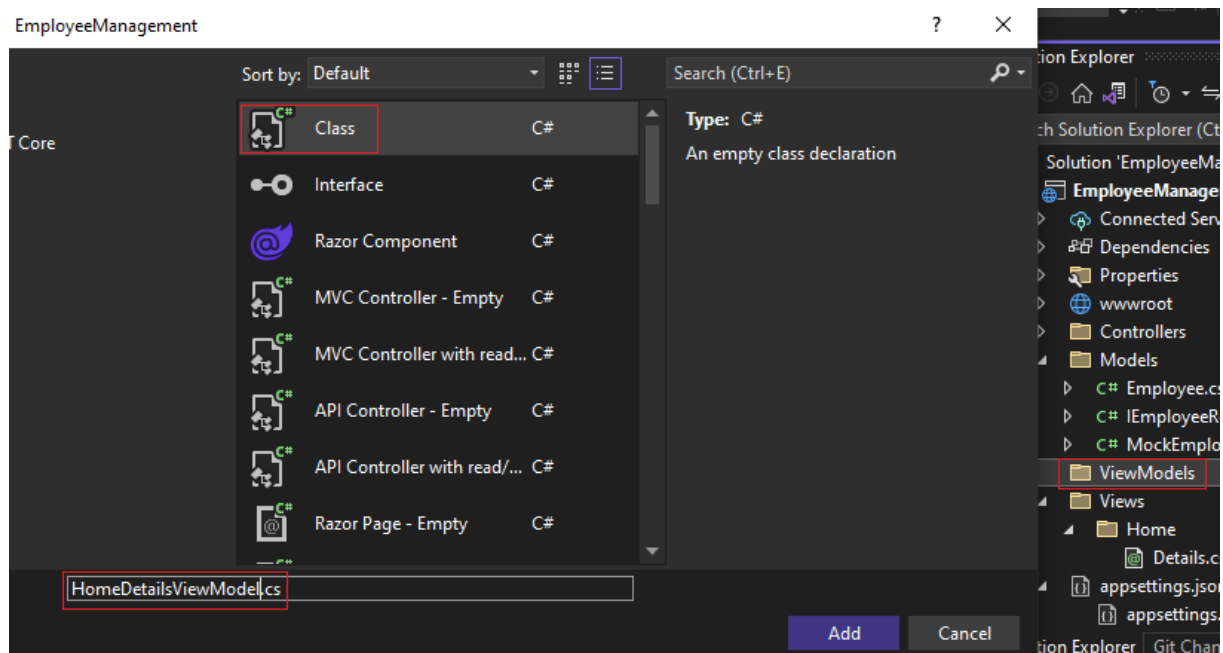
За по-добра производителност избягваме ViewData и ViewBag като девелъпъри. Все още използваме обаче ViewBag за PageTitle. Начинът едновременно да предадем данни и за PageTitle + сложен тип данни като клас Employee, е да ги обединим в специфичен модел ViewModel, който ще ни позволи включване на Intellisense.

В момента в HomeController използваме все още 2 типа данни за прехвърляне:

```
public IActionResult Details()
{
    Employee model = _employeeRepository.GetEmployee(1);
    ViewBag.PageTitle = "Employee Details";
    return View(model);
}
```

Няма как на тип Employee да присвоим PageTitle, затова използваме новия тип ViewModel, който ще ги обедини двата в по-сложния тип. Той може да бъде поставен навсякъде в проекта, но за по-добра организация създаваме отделна папка ViewModels. Въпросът е не просто да се обединят двата типа данни в един нов, а да включим Intellisense, за да предотвратяваме грешки по време на писането на кода.

HomeDetailsViewModel = PageTitle + Employee

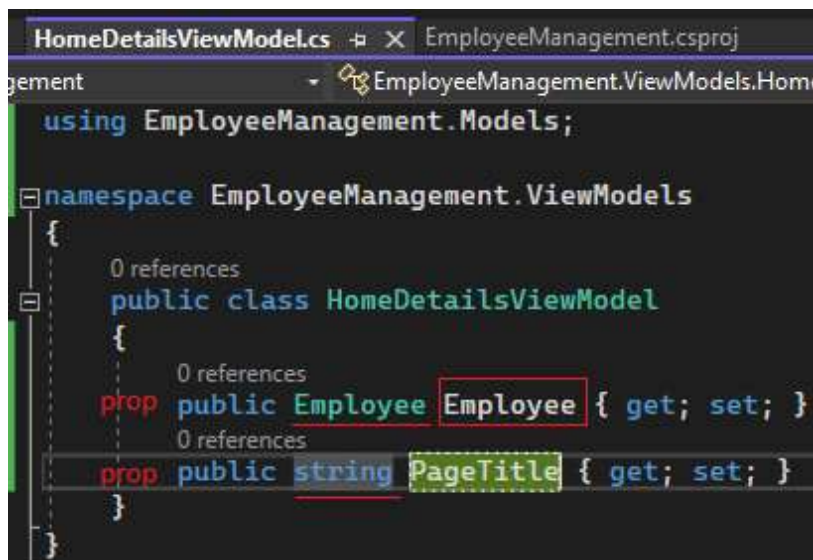


Име **HomeController+Details** action method + **ViewModel** = HomeDetailsViewModel.cs
за пренос на данни между контролера и View.

ViewModels = Data Transfer Objects = DTO.

A namespace is a declarative region that provides a scope to the identifiers (the names of types, functions, variables, etc) inside it.

Винаги първо създаваме „библиотеката“, за да може после да я извикаме като декларация с възможност корекции на грешки в контролера или където е необходимо.



Използваме команда `prop` + `Tab`, после за смяна на другите команди навигираме с `Tab`. Единият тип данни е `Employee`, другият `string`. Едната променлива е наречена `Employee` отново, което не одобрявам като постоянни повторения, другата е `PageTitle`.

```
using EmployeeManagement.Models;

namespace EmployeeManagement.ViewModels
{
    public class HomeDetailsViewModel
```

```

{
    public Employee Employee { get; set; }
    public string PageTitle { get; set; }
}

```

```

public IActionResult Details()
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(1),
        PageTitle = "Employee Details"
    };
    Employee model = _employeeRepository.GetEmployee(1);
    ViewBag.PageTitle = "Employee Details";
    return View(model);
}

```

Тук вече може да извикаме namespace HomeDetailsViewModel - Ctrl+. Intellisense.

```

public IActionResult Details()
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(1),
        PageTitle = "Employee Details"
    };
    return View(homeDetailsViewModel);
}

```

```

public IActionResult Details()
{
    HomeDetailsViewModel homeDetailsViewModel
        = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(1),
        PageTitle = "Employee Details"
    };
    return View(homeDetailsViewModel);
}

```

Имаме възможност за успешно стартиране, като в началната страница всичко е наред и зарежда Mary, но дава грешка, че типът данни, който е предаден, е от HomeDetailsViewModel, а изисква да е тип Employee.



An unhandled exception occurred while processing the request.

InvalidOperationException: The model item passed into the ViewDataDictionary is of type 'EmployeeManagement.ViewModels.HomeDetailsViewModel', but this ViewDataDictionary instance requires a model item of type 'EmployeeManagement.Models.Employee'.

Microsoft.AspNetCore.Mvc.ViewFeatures.ViewDataDictionary.EnsureCompatible(object value)

защото в Details сме задали тип Employee, а вече използваме по-сложния, разширен, надграден

```
HomeDetailsViewModel.cs EmployeeManagement.csproj Details.cshtml X HomeController.cs
@model EmployeeManagement.Models.Employee
<html xmlns="http://www.w3.org/1999/xhtml" class="EmployeeManagement.Models.Employee">
```

Докато в същото време в HomeController предаваме

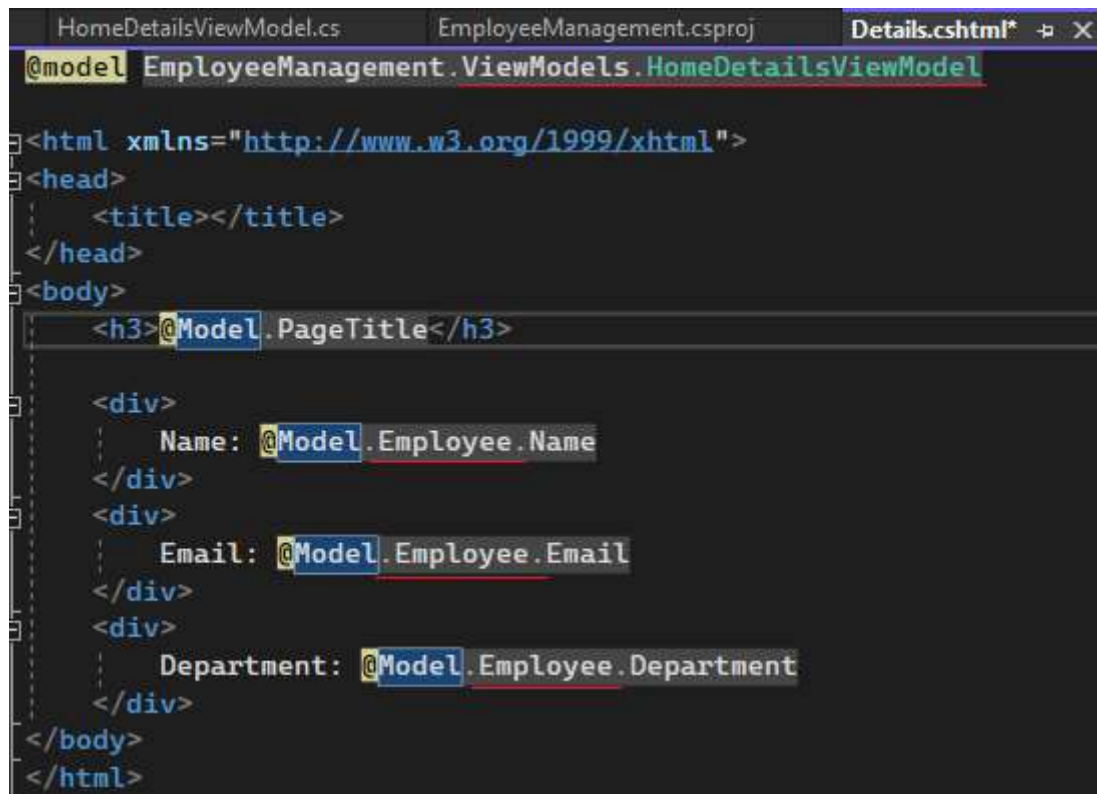
```
return View (homeDetailsViewModel);
```

Така че го сменяме, но веднага надолу се подчертават с червено за грешки параметрите:

```
HomeDetailsViewModel.cs EmployeeManagement.csproj Details.cshtml X
@model EmployeeManagement.ViewModels.HomeDetailsViewModel
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title></title>
</head>
<body>
<h3>@ViewBag.PageTitle</h3>
<div>
Name: @Model.Name
</div>
<div>
Email: @Model.Email
</div>
<div>
Department: @Model.Department
</div>
</body>
</html>
```

Това е така, защото Model няма свойствата Name, Email, Department. Те принадлежат на Employee, който е едно ниво под него, защото вътре в самия HomeDetailsViewModel = Employee(Name, Email, Department) + PageTitle и използваме подсказките, за да ги присвоим, т.е. не можем да го прескочим като тип в типа.

Също вече не е необходимо да прехвърляме PageTitle чрез ViewBag, защото сме го направили част от декларирания в началото Model, към който също е включен Intellisense.



```
HomeDetailsViewModel.cs EmployeeManagement.csproj Details.cshtml*
@model EmployeeManagement.ViewModels.HomeDetailsViewModel

<html xmlns="http://www.w3.org/1999/xhtml">
<head>
    <title></title>
</head>
<body>
    <h3>@Model.PageTitle</h3>

    <div>
        Name: @Model.Employee.Name
    </div>
    <div>
        Email: @Model.Employee.Email
    </div>
    <div>
        Department: @Model.Employee.Department
    </div>
</body>
</html>
```

@model EmployeeManagement.ViewModels.HomeDetailsViewModel

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
    <title></title>
</head>
<body>
    <h3>@Model.PageTitle</h3>

    <div>
        Name: @Model.Employee.Name
    </div>
    <div>
        Email: @Model.Employee.Email
    </div>
    <div>
        Department: @Model.Employee.Department
    </div>
</body>
</html>
```

Така страницата зарежда и PageTitle + Details.

We create a "View Model" when a Model object does not contain all the data a view needs

```
public class HomeDetailsViewModel
{
    public Employee Employee { get; set; }
    public string PageTitle { get; set; }
}
```

```
public IActionResult Details()
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(1),
        PageTitle = "Employee Details"
    };

    return View(homeDetailsViewModel);
}
```

27 – List view

Искаме да изобразим всички служители в една html таблица:

List View in ASP.NET Core MVC

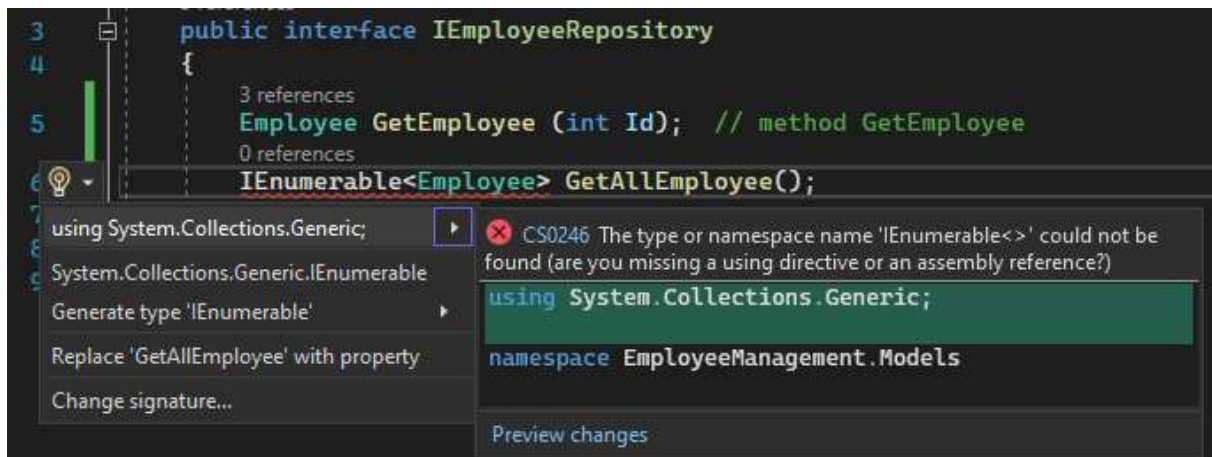
ID	Name	Department
1	Mary	HR
2	John	IT
3	Sam	IT

До момента IEmployeeRepository има само един единствен метод GetEmployee

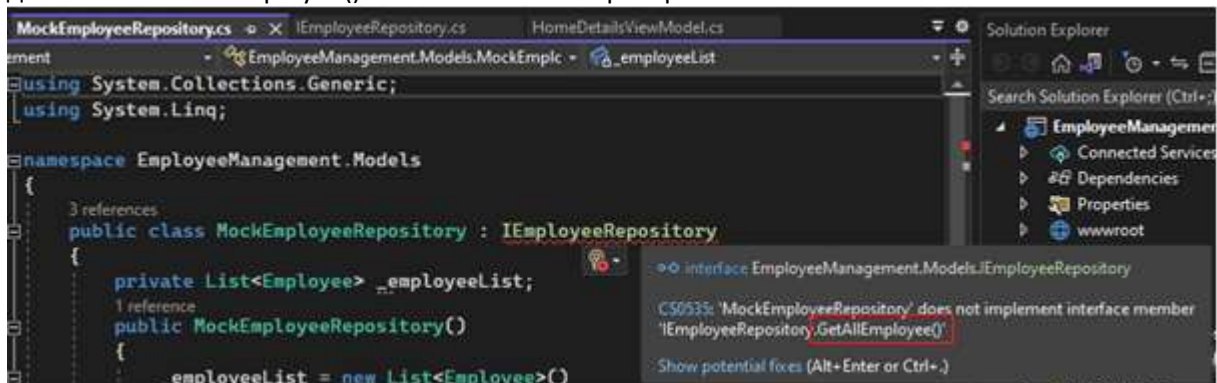
```
public interface IEmployeeRepository
{
    // 3 references
    Employee GetEmployee (int Id); // method GetEmployee
}
```

който връща служител по неговото id.

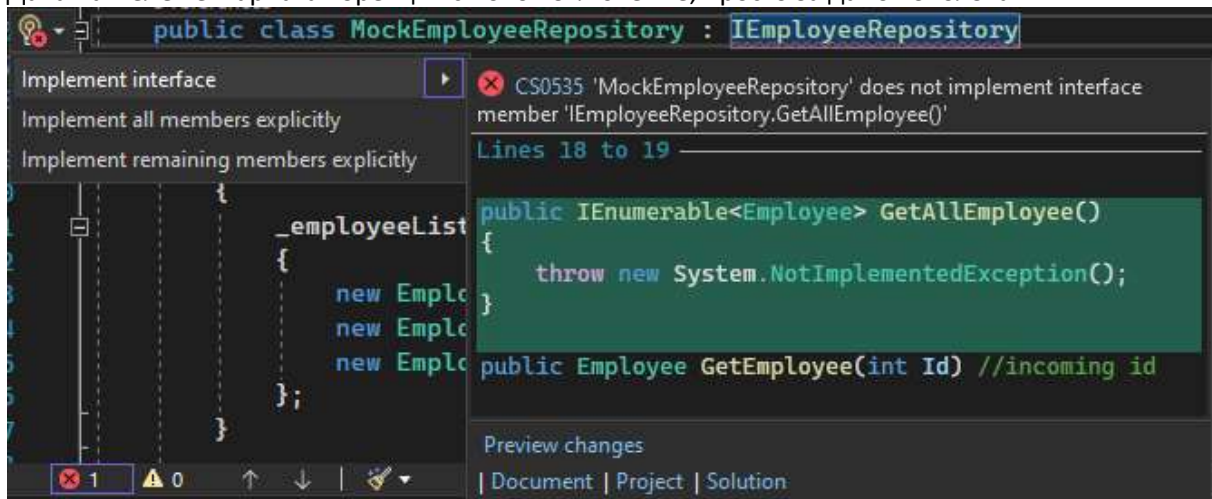
За да направим метод, който връща всички служители по списък, той трябва да е тип IEnumerable в който вкарваме лист от <Employee> с нова променлива, която наричаме:



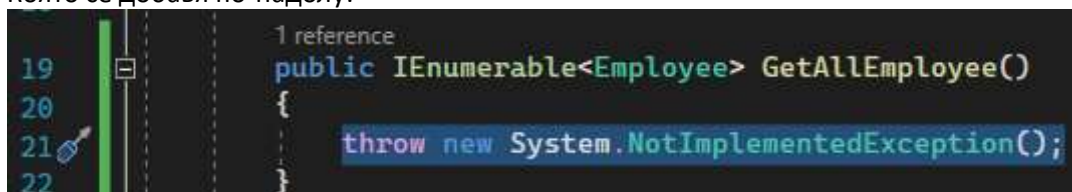
`IEmployeeRepository` има само 1 имплементация (изпълнение) и тя е в `MockEmployeeRepository`, но дава вече грешка в нея с червено подчертаване, защото все още сме й предоставили имплементация само на единия `GetEmployee()`, а на другия нов метод, който току-що й добавихме `GetAllEmployee()` липсва. Когато я коригираме автоматично с `Ctrl`.



Дава най-елементарната корекция с ново изключение, просто за да го използва:



Която се добавя по-надолу:

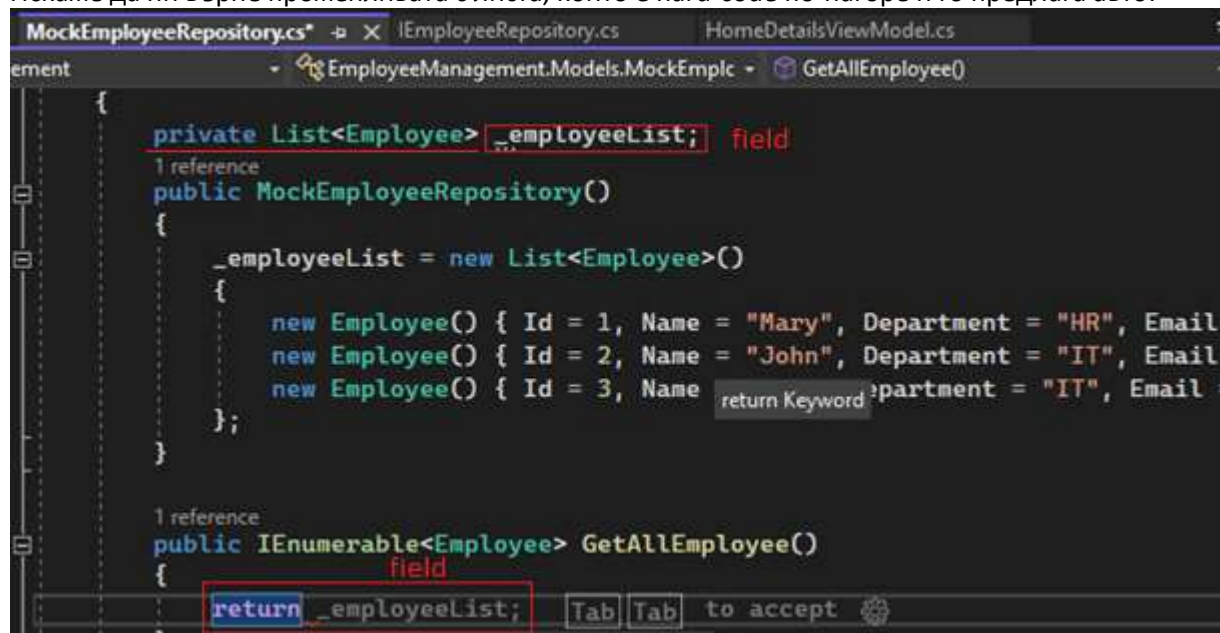


Всичко, което искаме този нов метод да направи, е да ни върне списъка от листа на всички служители.

* SQL server DB – произнася го като sequel (продължение).

По-нататък ще направим имплементацията на списъка вместо с hard-code в лист, с извличане на данни от SQL server DB.

Искаме да ни върне променливата с листа, който е hard-code по-нагоре и го предлага авто:



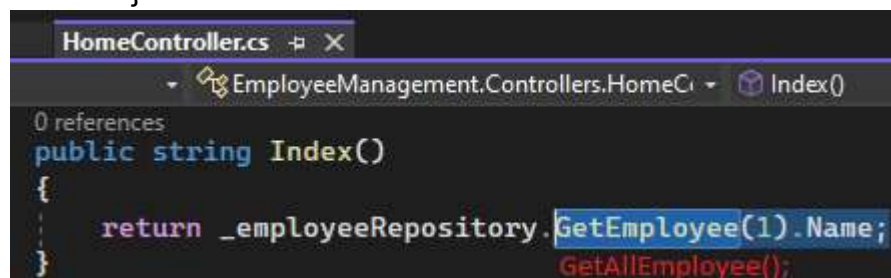
```
MockEmployeeRepository.cs
EmployeeManagement.Models.MockEmplic
GetAllEmployee()

{
    private List<Employee> _employeeList; field
    1 reference
    public MockEmployeeRepository()
    {
        _employeeList = new List<Employee>()
        {
            new Employee() { Id = 1, Name = "Mary", Department = "HR", Email
            new Employee() { Id = 2, Name = "John", Department = "IT", Email
            new Employee() { Id = 3, Name
            return Keyword
        };
    }

    1 reference
    public IEnumerable<Employee> GetAllEmployee()
    {
        return _employeeList;
    }
}
```

Т.е. return на този private field на метода, за да върне целия списък със служителите.

```
public IEnumerable<Employee> GetAllEmployee()
{
    return _employeeList;
}
```



```
HomeController.cs
EmployeeManagement.Controllers.HomeC
Index()

0 references
public string Index()
{
    return _employeeRepository.GetEmployee(1).Name;
    GetAllEmployee();
}
```

Искаме вместо 1 потребител с id = 1 да ни върне всички.

Съхраняваме този лист в нова променлива model.

```
public ActionResult Index()
{
    var model = _employeeRepository.GetAllEmployee();
    return View(model);
}
```

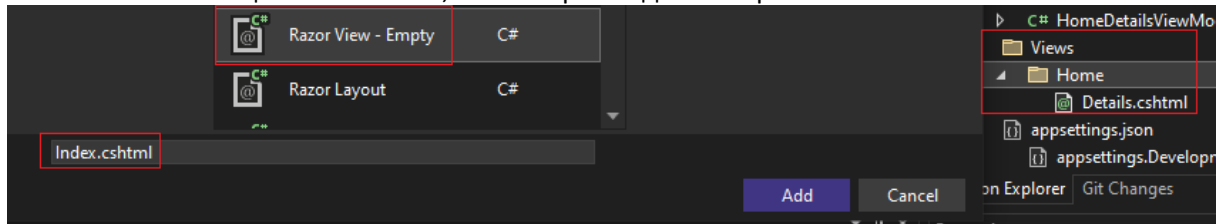
View изисква ActionResult, а не string, както досега, така че го сменяме.

Като string не е имало проблем да бъде изписан в Index() action, но View изисква самостоятелна страница .cshtml.

```
public ActionResult Index()
{
    var model = _employeeRepository.GetAllEmployee();
    return View(model);
}

0 references
(local variable) System.Collections.Generic.IEnumerable<Employee> model
```


За нея обаче все още нямаме View, така че трябва да го направим.

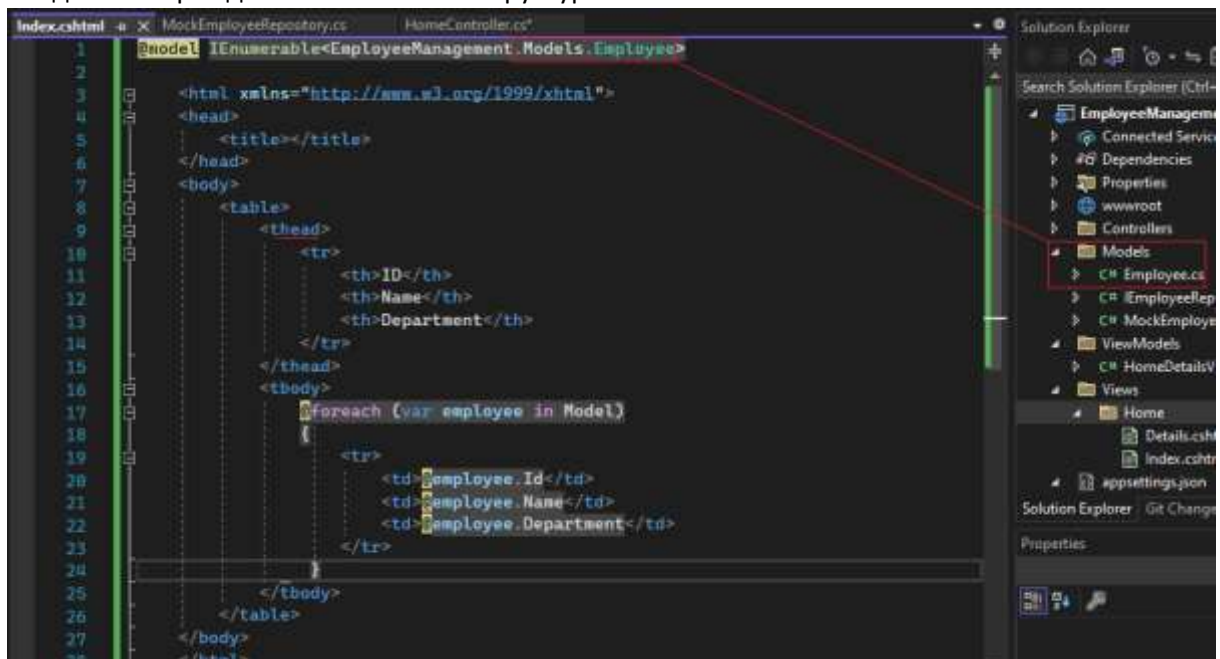


За Intellisense, моделът за нашето View е от IEnumerable.

Досега изписвахме `IEnumerable<Employee>`, но понеже се касае за namespace, е необходим пълният адрес:

`@model IEnumerable<EmployeeManagement.Models.Employee>`

След което вграждаме базовата html-структура с html + Tab x 2:



Влагаме таблица с header + цикъл за body:

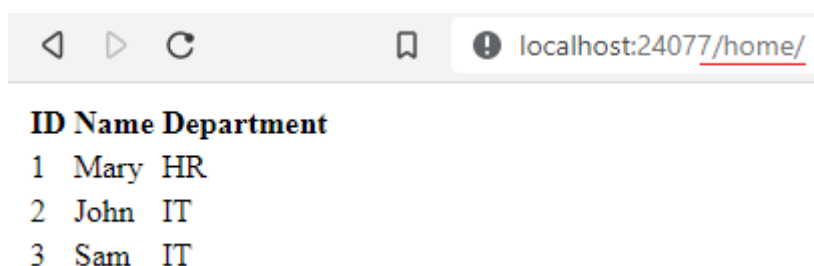
```
<table>
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Department</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var employee in Model) // loop
        {
            <tr>
                <td>@employee.Id</td>
                <td>@employee.Name</td>
                <td>@employee.Department</td>
            </tr>
        }
    </tbody>
</table>
```

Интересният момент тук е поредното объркващо повторение на думата на променливата employee, но няма как тя да бъде пренесена от друг документ тук, тя е локална променлива и като такава, можем да я кръстим както си искаме.

Освен това, за да се извика C# код с `@Model` в съдържанието, очевидно не е задължително те да бъдат слети. Възможно е да има други структури между тях като това условие за цикъл `@.....Model`

```
@foreach (var employee in Model)
{
    <tr>
        <td>@employee.Id</td>
        <td>@employee.Name</td>
        <td>@employee.Department</td>
    </tr>
}
```

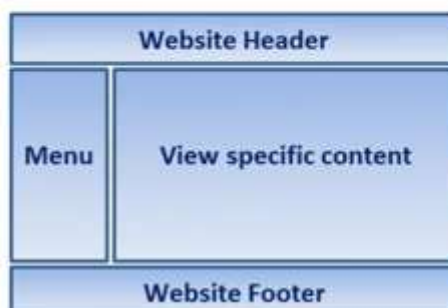
Извличайки данните от `employee` in `Model` генерираме по 1 нов ред в цикъла `foreach` loop, всеки от които с по 3 клетки table data за съответните параметри в него, така че за всеки `employee` object in `Model`, а `<tr>` with 3 `<td>` will be dynamically generated.



По-нататък ще инсталираме Bootstrap за по-добър стил.

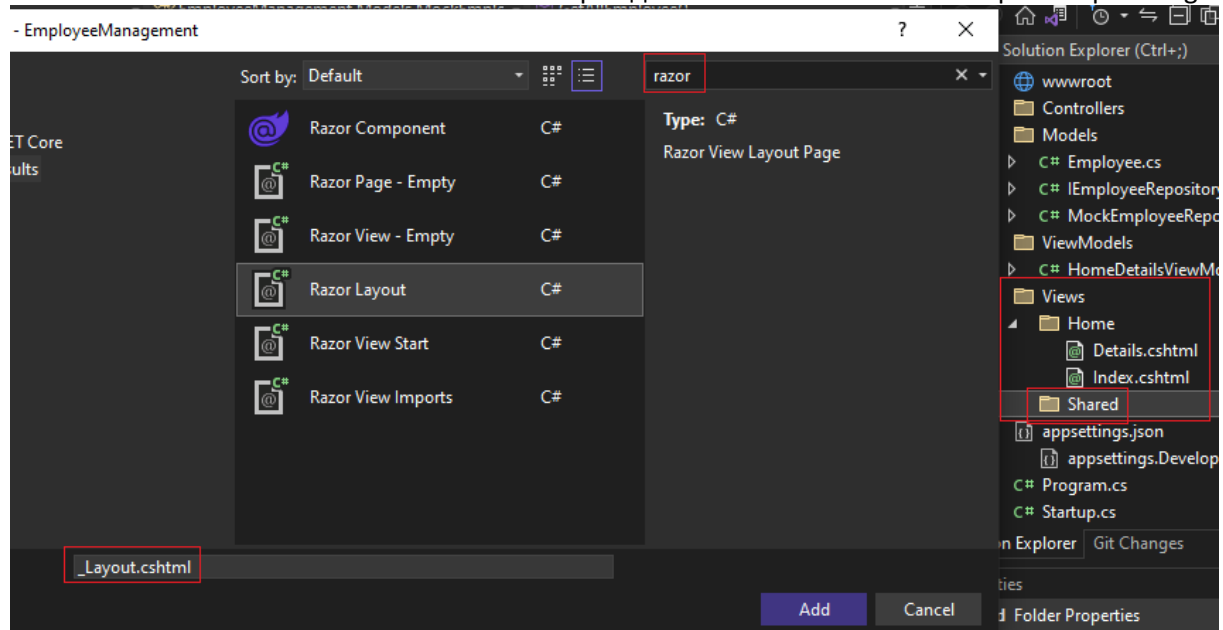
28 – Layout

Layout View in ASP.NET Core MVC



Целта на тази Layout-структура е да не се налага повторение на HTML-кода за всяка страница в приложението, с което управлението би станало кошмар. В тях се повтаря голяма част от html-кода, който може да бъде изнесен в отделен Layout (както във Photoshop – всеки нов елемент в отделен Layout) и в `.cshtml` да остане само уникалната част, специфична само за конкретната страница. Така, ако се наложи да добавим или премахнем елемент от менюто например, няма да се налага да го правим във всички страници, а само на едно място в Layout, като промяната ще се отрази незабавно върху всички страници в цялото приложение. Обикновено поставяме всички Layout във `Views/Shared`-папка, така че нека я добавим. Всички Layout-оформления са със `.cshtml` разширения и може да ги възприемаме като главна рамка, master-page на останалите html-forms. До момента имаме 2 View файла за `Index.cshtml` + `Details.cshtml` в папка `Views/Home`,

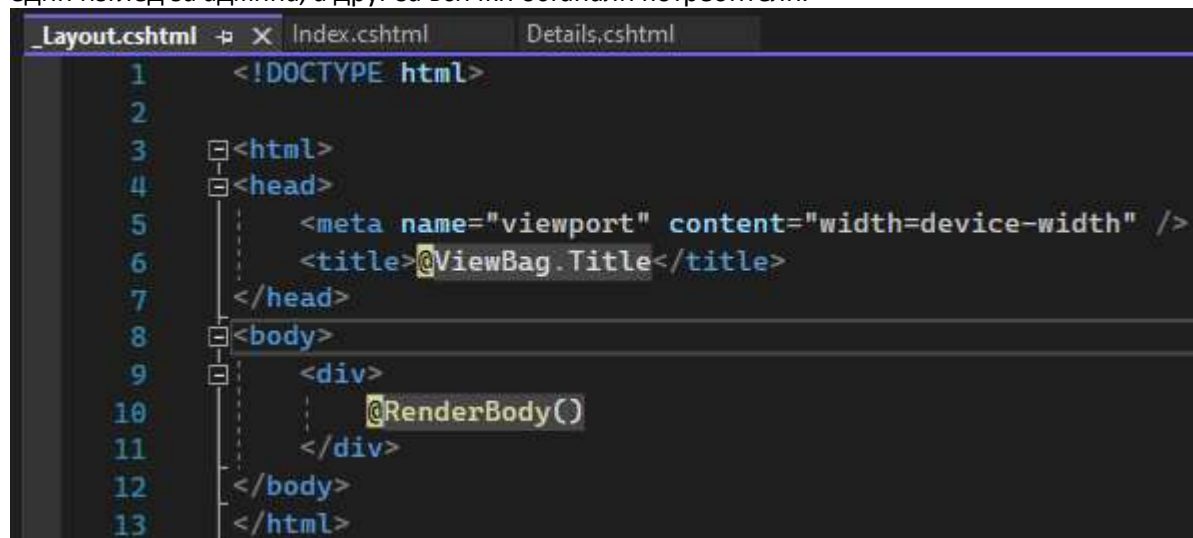
съседна по стандарт на Views/Shared, ето защо ни се показваше като основна в търсенията, когато се зареждаше DeveloperExceptionPage.



Layout.cshtml по подразбиране започва с '', което показва, че целта на файла е да бъде обработен, без да бъде показван директно в браузъра (в противен случай, само разширението .cshtml може да ни подведе), по същият начин са следващите _ViewStart.cshtml, _ViewImports.cshtml.

_Layout.cshtml не е специфичен за даден контролер като предишните 2 - Index, Details, които принадлежат на Home, затова сме го поставили в папка Shared.

Възможно е да имаме няколко _Layout файла с различни имена в едно приложение, например един изглед за админа, а друг за всички останали потребители.



<!DOCTYPE html>

```
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>@ViewBag.Title</title>
</head>
<body>
    <div>
        @RenderBody()
    </div>
```

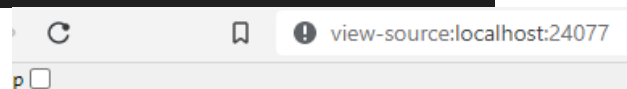
```
</body>  
</html>
```

Целият html-код, който е вложен по подразбиране като тагове html, head, body и се повтарят в останалите Index.cshtml, Details.cshtml, може да бъде премахнат, за да не се повтаря в тях. @ViewBag.Title – използва се за PageTitle, вместо да hard-code-нем заглавията на всички html-страници, по-добре да използваме ViewBag за толкова минимално количество информация, т.к. рисковете при грешки в него биха били с леки последици (просто няма да се зареди заглавието, а не да крашне цялата страница).

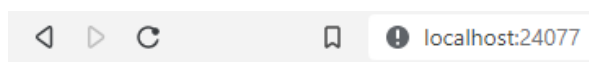
@RenderBody() – на негово място се инжектира целия специфичен индивидуален html-код, който евентуално бихме искали да се изведе в структурата на извиканите отделни страници.

```
Index.cshtml  Details.cshtml
@model IEnumerable<EmployeeManagement.Models.Employee>

<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Department</th>
    </tr>
  </thead>
  <tbody>
    @foreach (var employee in Model)
    {
      <tr>
        <td>@employee.Id</td>
        <td>@employee.Name</td>
        <td>@employee.Department</td>
      </tr>
    }
  </tbody>
</table>
```



```
<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Department</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>1</td>
      <td>Mary</td>
      <td>HR</td>
    </tr>
    <tr>
      <td>2</td>
      <td>John</td>
      <td>IT</td>
    </tr>
    <tr>
      <td>3</td>
      <td>Sam</td>
      <td>IT</td>
    </tr>
  </tbody>
</table>
```



ID Name Department

1 Mary HR
2 John IT
3 Sam IT

Зарежда правилен резултат дори без _Layout, но при View Source е с изрязаните <html> тагове.

The screenshot shows a web browser window with the address bar displaying 'localhost:24077/home/details'. The page content includes a title 'Employee Details' and three lines of text: 'Name: Mary', 'Email: Mary@abv.bg', and 'Department: HR'. To the right, the browser's developer tools are open, showing the 'view-source' view of the page. The source code is as follows:

```
<h3>Employee Details</h3>
<div>
  Name: Mary
</div>
<div>
  Email: Mary@abv.bg
</div>
<div>
  Department: HR
</div>
```

At the top of the image, a separate window shows the C# code for the `Details.cshtml` file, which uses the `@Model` property to render the page title and employee details:

```
@Model.EmployeeManagement.ViewModels.HomeDetailsViewModel

<h3>@Model.PageTitle</h3>

<div>
  Name: @Model.Employee.Name
</div>
<div>
  Email: @Model.Employee.Email
</div>
<div>
  Department: @Model.Employee.Department
</div>
```

Липсващите `<html>` тагове не са прихванати, защото не сме указали използването на `_Layout`.

```
@{
    Layout = "~/Views/Shared/_Layout.cshtml";
    ViewBag.Title = "Employee List";
}
```

Това е код, който към момента поставяме във всеки `.cshtml`-файл, но при приложение с 100 страници, това е неуместно и ще разгледаме как може да се избегне, за по-лесна редакция при необходимост по-късно.


```
Index.cshtml X Details.cshtml
@model IEnumerable<EmployeeManagement.Models.Employee>

@{
    Layout = "~/Views/Shared/_Layout.cshtml";
    ViewBag.Title = "Employee List";
}

<table>
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Department</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var employee in Model)
        {
            <tr>
                <td>@employee.Id</td>
                <td>@employee.Name</td>
                <td>@employee.Department</td>
            </tr>
        }
    </tbody>
</table>
```

Демонстративно се забравя символ @ в блок с код @{}, което може да се случи при копиране:

```
3  @
4  block
5  {
6  }

Layout = "~/Views/Shared/_Layout.cshtml";
@ViewBag.Title = "Employee Details";
```

Internal Server Error x view-source:localhost:15410 x +

localhost:15410

An error occurred during the compilation of a resource required to process this request. Please review the following specific error details and modify your source code appropriately.

Generated Code

Invalid expression term '='

38. EndContext();

Show compilation source

The screenshot shows a web browser window with the address bar displaying 'view-source:localhost:24077'. The source code is visible, showing an HTML document with a title 'Employee List' and a table with three columns: ID, Name, and Department. The table contains three rows of data: (1, Mary, HR), (2, John, IT), and (3, Sam, IT). Below the source code, a small window shows the rendered page, which displays the same table of employee data.

ID	Name	Department
1	Mary	HR
2	John	IT
3	Sam	IT

Във View Source <html>-автоматично е добавен, забелязваме и Title, но той не се извежда в браузъра, но стандартно озаглавява името на подпрозореца. Съответно

The screenshot shows a code editor with the file 'Details.cshtml' open. The code is a Razor view for 'EmployeeDetailsView'. It includes a layout directive, a ViewBag.Title assignment, and a series of divs displaying employee details: Name, Email, and Department. The code is as follows:

```
@model EmployeeManagement.ViewModels.HomeDetailsViewModel

@{
    Layout = "~/Views/Shared/_Layout.cshtml";
    ViewBag.Title = "Employee Details";
}

<h3>@Model.PageTitle</h3>

<div>
    Name: @Model.Employee.Name
</div>

<div>
    Email: @Model.Employee.Email
</div>

<div>
    Department: @Model.Employee.Department
</div>
```

Много странно тук в туториъла е оставено повторение на заглавие на страницата по два различни начина: с ViewBag.Title и по-надолу с <h3>@Model.PageTitle</h3>

localhost:24077/home/details

Employee Details

Name: Mary
Email: Mary@abv.bg
Department: HR

```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width" />
  <title>Employee Details</title>
</head>
<body>
  <div>
    <h3>Employee Details</h3>
    <div>
      Name: Mary
    </div>
    <div>
      Email: Mary@abv.bg
    </div>
    <div>
      Department: HR
    </div>
  </div>
```

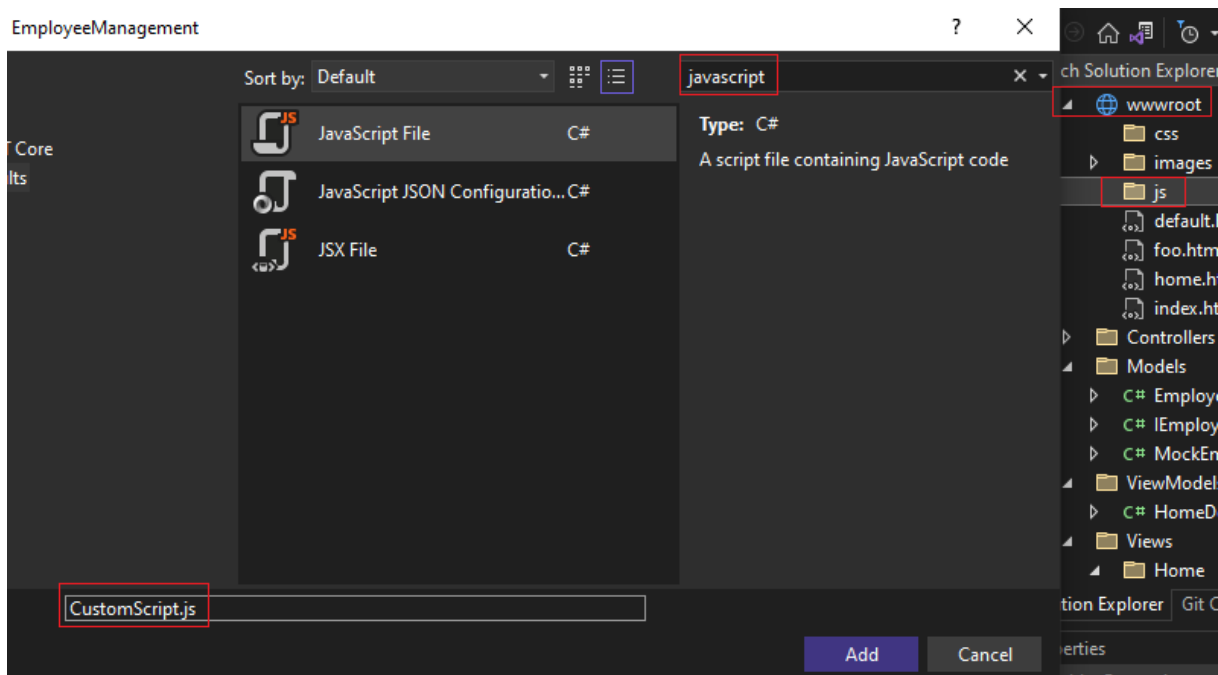
Когато добавим _Layout, при View Source автоматично се зареждат всички html тагове без видимост на C# - той е заменен с обикновени тагове като <title>, <div>.

Layout View in ASP.NET Core MVC

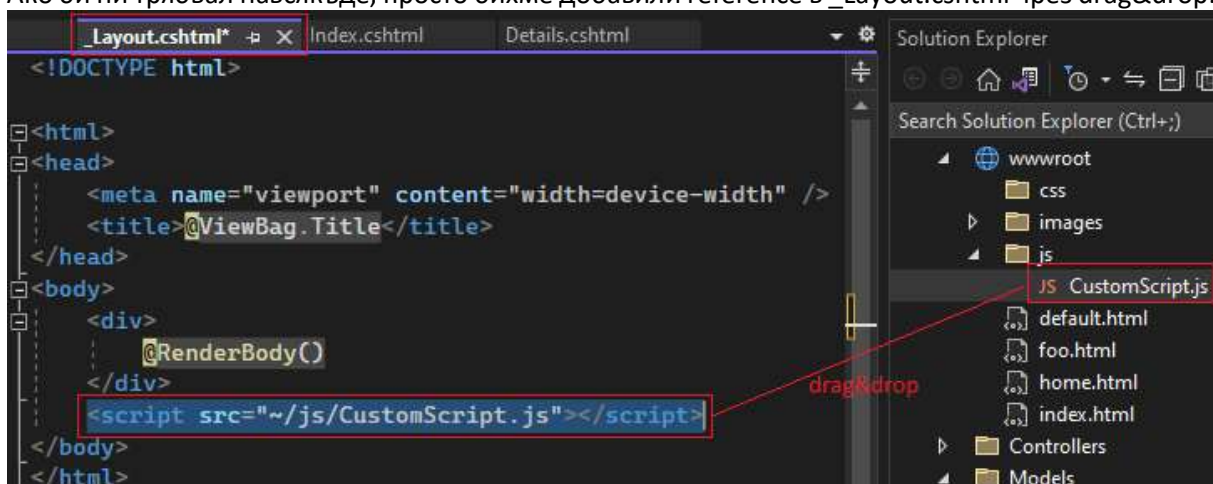
- Consistent look and behaviour for all the views in a web application
- Similar to master page in ASP.NET Web Forms
- File on the file system with **.cshtml** extension
- Default name is **_Layout.cshtml**
- Usually placed in **Views/Shared** folder
- An application can have multiple layout views

29 – Sections in Layout Page

Ако искаме да използваме даден JavaScript файл, може да го провлечим в _Layout (съответно да се изпише адреса към него в кода) и директно да го използваме навсякъде. Ако обаче искаме само в определени страници да използваме JS, трябва да укажем къде да бъде позволен и къде не.

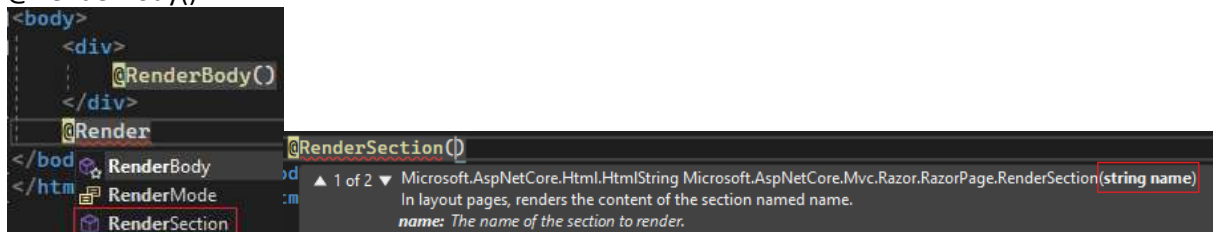


Ще използваме CustomScript.js само в Details.cshtml и никъде другаде. Ако би ни трябвал навсякъде, просто бихме добавили reference в _Layout.cshtml чрез drag&drop:

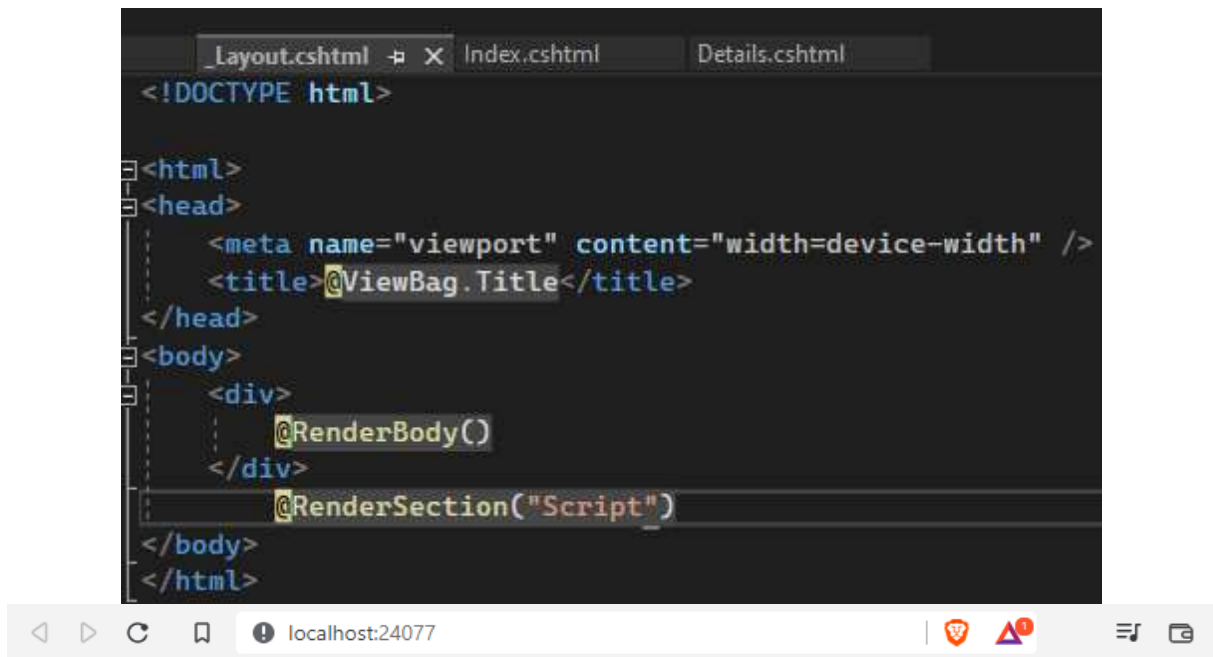


Като добра практика е всички .js да се вмъкват на дъното на страницата преди `</body>`, защото излишно бавят зареждането, когато не са необходими веднага. Като цяло те утежняват. На нас обаче не ни трябва към момента в _Layout, а само ще го извикаме в Details, затова премахваме авто-кода, който интерактивно провлачихме.

Вместо него, искаме да вмъкнем `@RenderSection()`, който е с подобни свойства на `@RenderBody()`.



Задаваме име „Scripts“

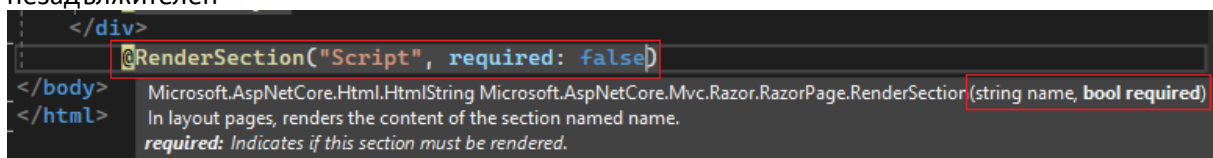


An unhandled exception occurred while processing the request.

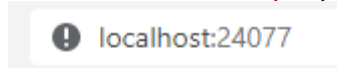
InvalidOperationException: The layout page '/Views/Shared/_Layout.cshtml' cannot find the section 'Script' in the content page '/Views/Home/Index.cshtml'.

Microsoft.AspNetCore.Mvc.Razor.RazorPage.RenderSectionAsyncCore(string sectionName, bool required)

Извежда грешка, защото в Index.cshtml не сме задали `@RenderSection("Script")`, защото не искаме да я използваме, в момента скрипта е задължителен в _Layout. Правим го незадължителен



I `@RenderSection("Script", required: false)`



ID Name Department

1 Mary HR

2 John IT

3 Sam IT

```

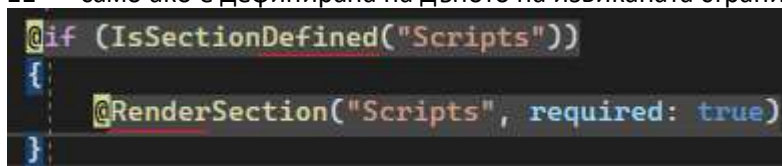
</tr>
</tbody>
</table>
</div>

```

Грешката вече я няма, а при View Source не виждаме .js преди `</body>`

* начин да направим задължително `@RenderSection("Script", required: true)`

II – само ако е дефинирана на дъното на извиканата страница (Details), я обработи:



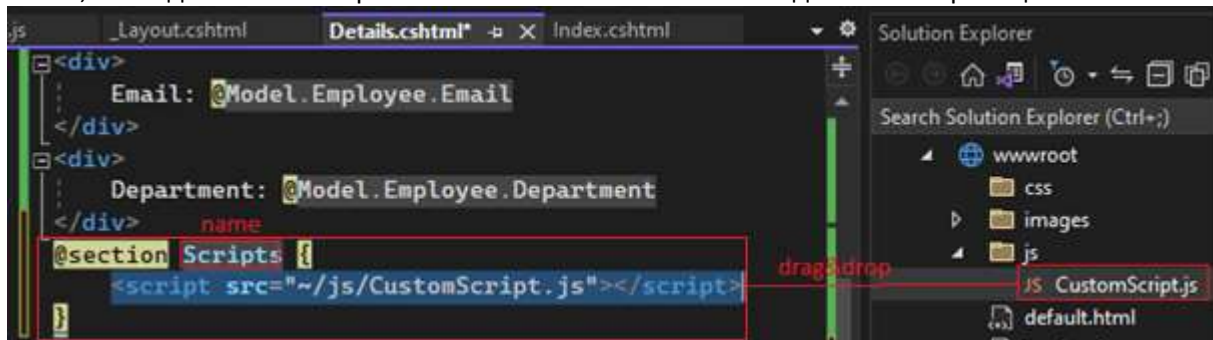
```

@if (IsSectionDefined("Scripts"))
{
    @RenderSection("Script", required: true)
}

```


}

В момента и двете страници зареждат без проблеми в браузъра. Целта ни от самото начало обаче, беше да се извика скрипта само в Details. Това става на дъното на страницата с блок:



```
@section Scripts {  
    <script src="~/js/CustomScript.js"></script>  
}
```

Когато заредя страница Index, зарежда правилно, там не е дефиниран и извикан никакъв JS код, а във View Source липсва крайния ред за JS.

При Details също зарежда, като той вече се вижда:

```
    Department: HR  
</div>  
  
</div>  
  
<script src="/js/CustomScript.js"></script>  
@model EmployeeManagement.ViewModels.HomeDetailsViewModel  
  
{  
    Layout = "~/Views/Shared/_Layout.cshtml";  
    ViewBag.Title = "Employee Details";  
}  
  
<h3>@Model.PageTitle</h3>  
  
<div>  
    Name: @Model.Employee.Name  
</div>  
<div>  
    Email: @Model.Employee.Email  
</div>  
<div>  
    Department: @Model.Employee.Department  
</div>  
@section Scripts {  
    <script src="~/js/CustomScript.js"></script>  
}
```

Sections in Layout Page

- A Section in a Layout View provides a way to organize where certain page elements should be placed
- A section can be **optional** or **mandatory**
- A section in the layout view is rendered at the location where **RenderSection()** method is called

```
<html>
<head>
  <title>@ViewBag.Title</title>
</head>
<body>
  <div>
    @RenderBody()
  </div>

  @RenderSection("Scripts", required: false)
</body>
</html>
```

30 – ViewStart.cshtml

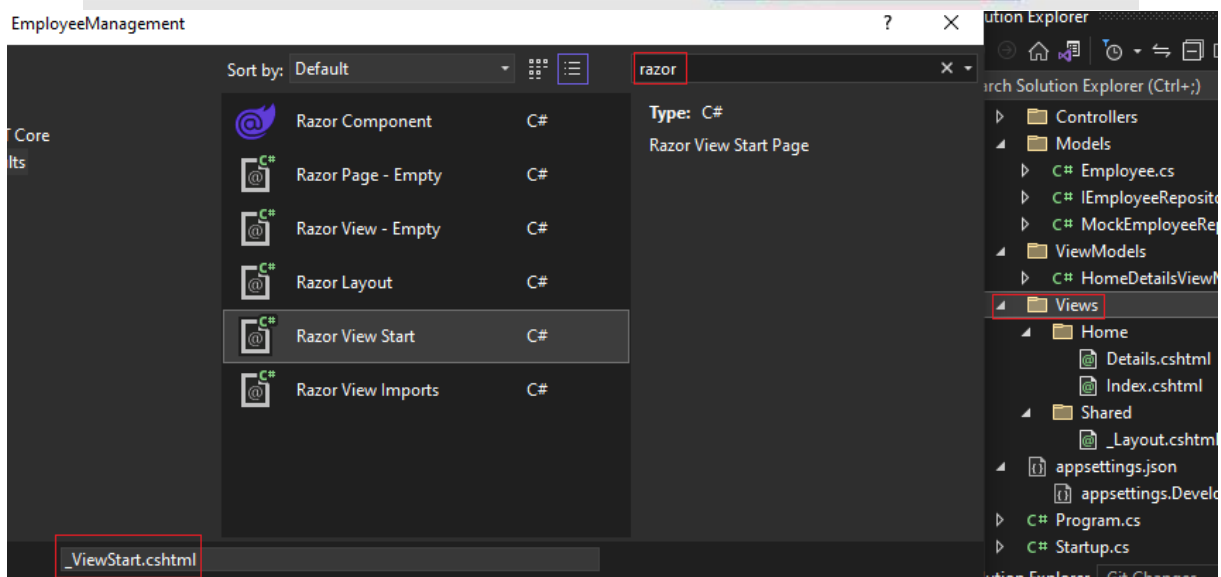
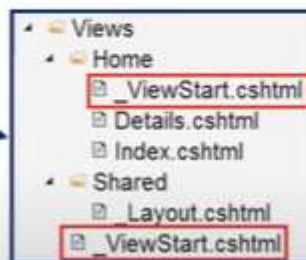
За да се избегне повторението на блоков код с извикване на един и същи _Layout във всеки индивидуален View, се използва _ViewStart.cshtml

```
@{
    Layout = "~/Views/Shared/_Layout.cshtml";
    ViewBag.Title = "Employee Details";
}
```

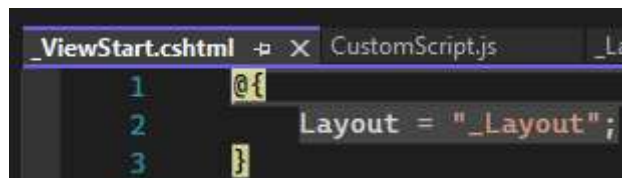
Кодът във _ViewStart се изпълнява преди всички останали Views.cshtml и има йерархична структура.

_ViewStart in ASP.NET Core MVC

- Code in **ViewStart** is executed before the code in an individual view
- Move the **common code** such as setting the Layout property to ViewStart
- ViewStart reduces **code redundancy** and **improves maintainability**
- ViewStart file is hierarchical



DRY = Don't Repeat Yourself, избягва се code duplication, заради който управлението на голямо приложение може да се превърне в кошмар. Ако в даден момент решим да използваме друг Layout, то ще трябва да го сменяме ръчно във всеки индивидуален View. За целта се използва специалният вид Layout: `_ViewStart.cshtml`, който се поставя в началото на страницата, за да може да се зареди преди регулярните Layout. Обикновено се поставя директно в папка Views. Автокод:



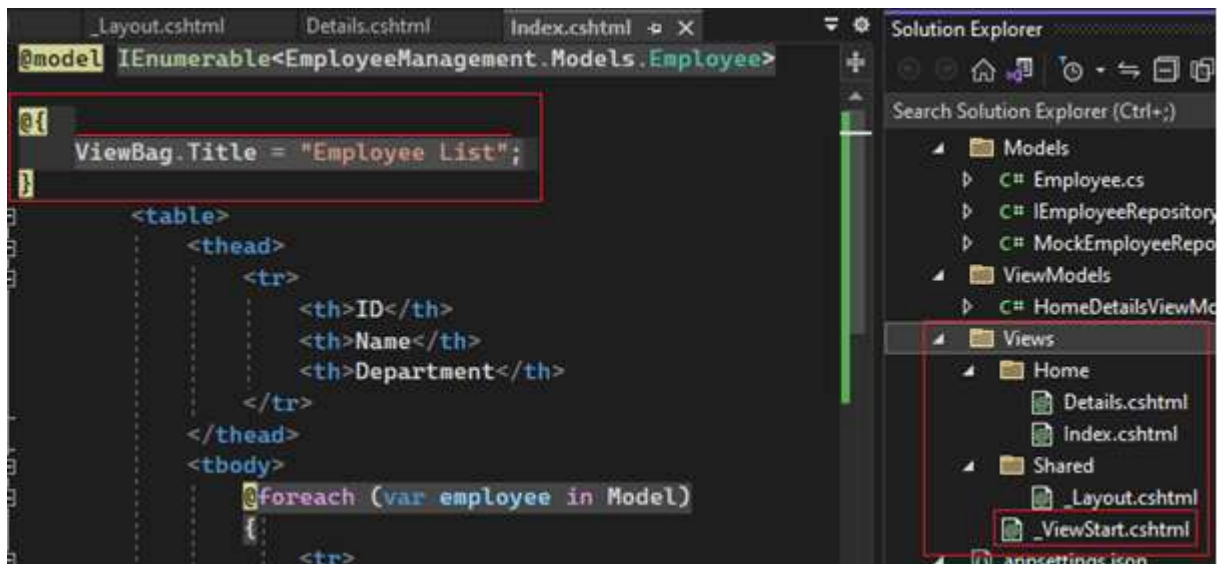
По този начин, управлението става много по-лесно, т.к. трябва да се смени само този 1 ред код, ако искаме да използваме друг `_Layout.cshtml`, вместо навсякъде.

Тази линия повтарящ се в Index, Details вече може да бъде отстранена:

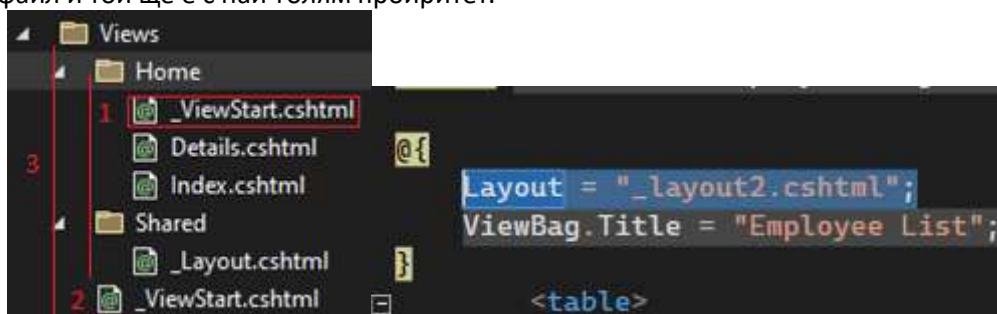
`Layout = "~/Views/Shared/_Layout.cshtml";`

`Layout = "~/Views/Shared/_Layout.cshtml";`

`_ViewStart.cshtml` не се извиква никъде, въпреки това, се изпълнява автоматично.



Доказателство за това е зареждането правилно на страниците в браузъра и View Source. Можем да имаме повече от 1 _ViewStart.cshtml, като другите имат йерархична структура в субдиректориите в подпапките на Views, като принципът остава непроменен: локалната засенчва глобалната. В същото време може да добавим и Layout property в индивидуален View-
_cshtml-файл и той ще е с най-голям приоритет.

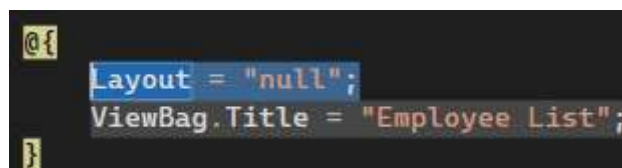


Т.е. по приоритет:

- 1) – вътре във файла, локално, с най-висок приоритет
- 2) в Home, Shared - папки
- 3) във Views – папка, глобално е с най-слаб приоритет, стандартно се поставя в нея.

Също можем локално във файл да занулим, т.е. да прекратим използването на всякакво Layout оформление:

Layout = "null";



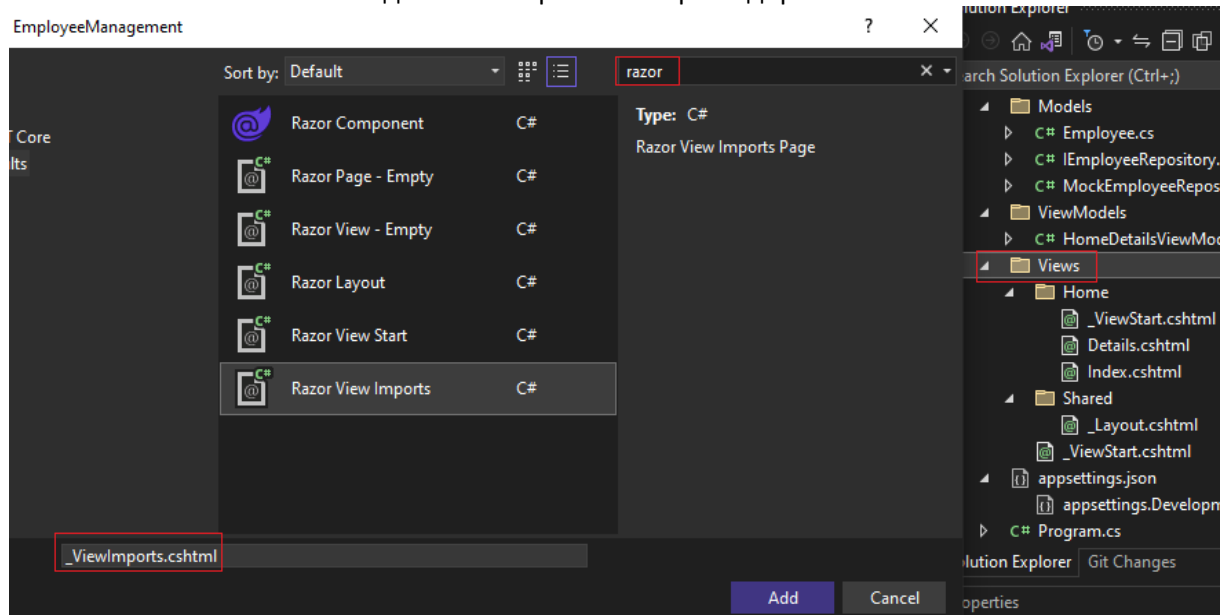
Изтриваме го, за да не пречи.

Понеже може да използваме много Layout, самият _ViewStart.cshtml може да съдържа логика за динамичен избор по време на RunTime. Редактираме външния _ViewStart, за да не пречи.

```
@{
    if (User.IsInRole("Admin"))
    {
        Layout = "_AdminLayout";
    }
    else
    {
        Layout = "_NonAdminLayout";
    }
}
```

31 – ViewImports.cshtml

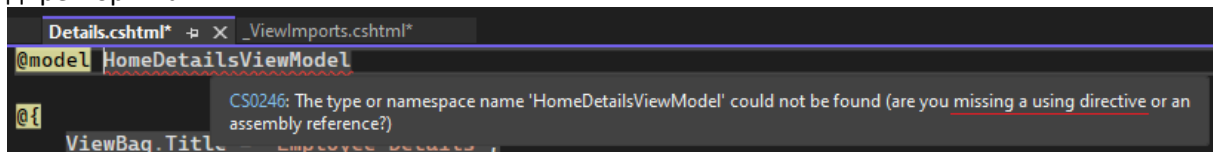
_ViewImports.cshtml – използва се, за да се декларират повтарящите се namespaces на 1 място, вместо всеки път във всеки отделен View файл. Намира се директно във Views папката.



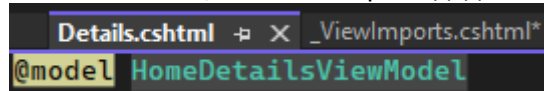
Така в самите View файлове ще останат за изписване само специфичните поддиректории за деклариране, а не пълните пътища до тях:

Cut: EmployeeManagement.ViewModels. -> Paste in _ViewImports.cshtml

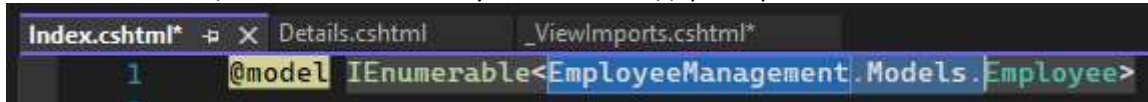
Сега, когато погледнем в Details.cshtml, червеното подчертаване ни казва, че има грешка в директорията.



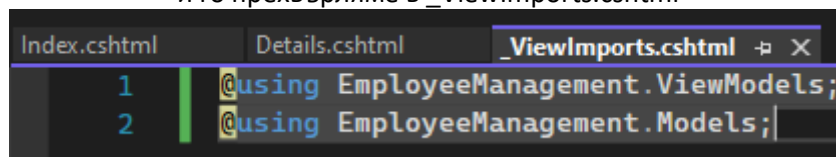
Тази грешка се изчиства от само себе си, колкото по-рано дадем safe на ViewImports.cshtml.



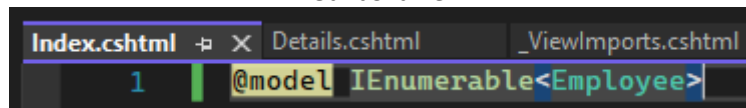
По същия начин скъсяваме указването на директорията на Index.cshtml:



и го прехвърляме в _ViewImports.cshtml

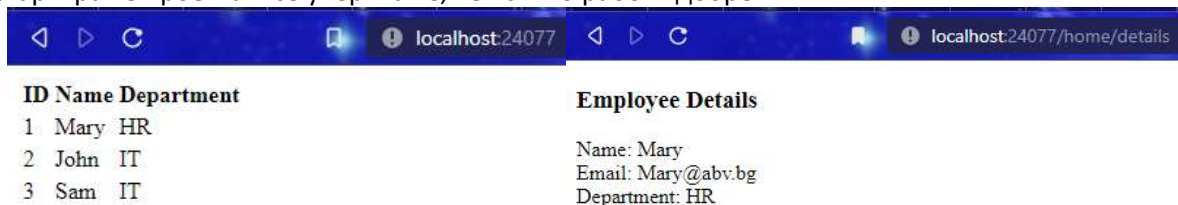


Запазваме:



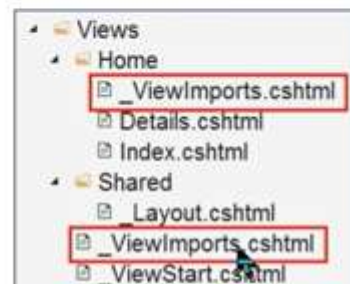
При мен, дори след запазване на промените в 3-те файла и стартиране на проекта, червеното подчертаване не се автоотстранява, докато не изтрия директорията и я напиша отново. Сега всеки View, който се намира в папката Views, в която се намира и нашия _ViewImports, ще има достъп до тези създадени и декларираны от нас namespaces, без да се налага да изписваме целия път до тях.

Стартираме проекта и се уверяваме, че всичко работи добре:



_ViewImports in ASP.NET Core MVC

- _ViewImports file is placed in the **Views** folder
- Used to include the **common namespaces**
- To include the **common namespaces** use **@using** directive
- Other **supported directives**
 - @addTagHelper
 - @removeTagHelper
 - @tagHelperPrefix
 - @model
 - @inherits
 - @inject
- _ViewImports file is **hierarchical**



- _ViewImports.cshtml също има йерархична структура като _ViewStart.cshtml:
- 1) индивидуален файл – максимален приоритет, презаписва средата над него.
 - 2) локална папка (Home, Shared)
 - 3) глобална папка (Views)

32 – Conventional Routing in MVC

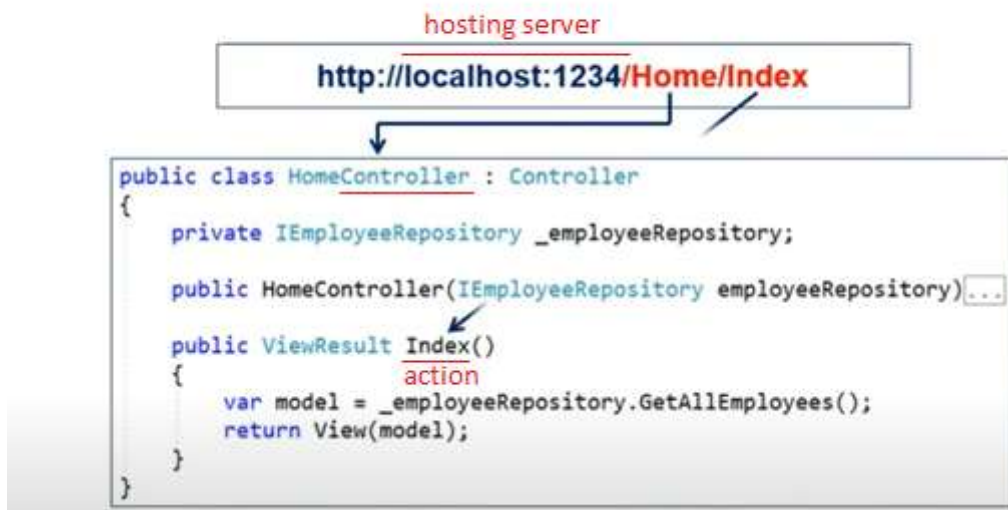
Routing in ASP.NET Core MVC

Routing Techniques

- Conventional Routing
- Attribute Routing



Routing in ASP.NET Core MVC





Демонстрираме:

The screenshot shows the Visual Studio IDE with the `HomeController.cs` file open. The `Details` method is highlighted, and the `id` parameter is circled in red. The code is as follows:

```
public ActionResult Index()
{
    var model = _employeeRepository.GetAllEmployee();
    return View(model);
}

public ActionResult Details()
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(1),
        PageTitle = "Employee Details"
    };
    return View(homeDetailsViewModel);
}
```

Вместо hard coding с конкретна стойност число, подаваме параметър:

The screenshot shows the Visual Studio IDE with the `HomeController.cs` file open. The `Details` method is highlighted, and the `id` parameter is circled in green. The code is as follows:

```
public ActionResult Index()
{
    var model = _employeeRepository.GetAllEmployee();
    return View(model);
}

public ActionResult Details(int id)
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(id),
        PageTitle = "Employee Details"
    };
    return View(homeDetailsViewModel);
}
```

Проверяваме приложението – в Index работи безпроблемно, там нищо не сме променяли, както и при `/home/details/1`,



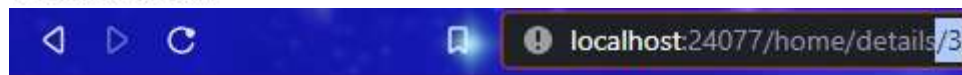
Employee Details

Name: Mary
Email: Mary@abv.bg
Department: HR



Employee Details

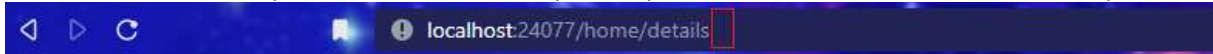
Name: John
Email: John@abv.bg
Department: IT



Employee Details

Name: Sam
Email: Sam@abv.bg
Department: IT

но като зададем адреса без последния параметър /1, а само /home/details, извежда грешка:



An unhandled exception occurred while processing the request.

NullReferenceException: Object reference not set to an instance of an object.
AspNetCore.Views_Home_Details.ExecuteAsync() in **Details.cshtml**, line 10

Stack Query Cookies Headers Routing

NullReferenceException: Object reference not set to an instance of an object.

AspNetCore.Views_Home_Details.ExecuteAsync() in **Details.cshtml**

10. Name: @Model.Employee.Name

Ние не сме дефинирали незадължителния характер на тази променлива за action Details:


```
Startup.cs | HomeController.cs | Index.cshtml | Details.cshtml
EmployeeManagement | EmployeeManagement.Startup | Configure(IApplicationBuilder app, IWebHostEnvironment env)

0 references
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    if (env.IsDevelopment()) // bool, IsStaging, IsProduction, IsEnvironment("UAT")
    {
        app.UseDeveloperExceptionPage();
    }

    app.UseStaticFiles(); //Short-circle ordering
    app.UseMvc(rb => //app.UseMvcWithDefaultRoute(); error after .net 3.0
    {
        rb.MapRoute(
            name: "default",
            template: "{controller}/{action}/{id?}",
            defaults: new { controller = "Home", action = "Index" });
    });

    app.UseRouting();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapGet("/", async context =>
        {
            await context.Response.WriteAsync("Hello world!");
        });
    });
};
```

Което в туториъла е със стария UseMvcWith DefaultRoute

```
Startup.cs | HomeController.cs
EmployeeManagement | EmployeeManagement.Startup | Configure(IApplicationBuilder app, IWebHostEnvironment env)

25 public void ConfigureServices(IServiceCollection services)
26 {
27     services.AddMvc().AddXmlSerializerFormatters();
28     services.AddSingleton<IEmployeeRepository, MockEmployeeRepository>();
29 }
30
31 // This method gets called by the runtime. Use this method to configure the application.
32 public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
33 {
34     if (env.IsDevelopment())
35     {
36         app.UseDeveloperExceptionPage();
37     }
38
39     app.UseStaticFiles();
40     app.UseMvcWithDefaultRoute();
41 }
42
```

Демонстративно закоментира WithDefaultRoute и използва обикновения Use.Mvc

```
//app.UseMvcWithDefaultRoute();
app.UseMvc();
```



This localhost page can't be found

No web page was found for the web address: <http://localhost:15410/home/details/2>

[Search Google for localhost 15410 home details](#)

HTTP ERROR 404


```

app.UseStaticFiles(); //Short-circle ordering
app.UseMvc(routes => //app.UseMvcWithDefaultRouting(); error after .net 3.0
{
    routes.MapRoute(

```

Виждаме, че променливата routes е от тип IRouteBuilder.

```

app.UseStaticFiles();
//app.UseMvcWithDefaultRoute();
app.UseMvc(routes => {
    routes.MapRoute("default", "{controller}/{action}/{id}");
});

```

Параметър id, който задаваме като число в URL, е обвързан с параметъра в Details с процес, наречен Model Binding. В случая, адресът без id - /home/details извежда грешка, липсваща страница, т.к. сме задали id да бъде задължителен параметър. Правим го опционен със знак '?'

При мен, обаче, този код дава редица грешки при всякакви опити, заради версията:

```

app.UseStaticFiles(); //Short-circle ordering
app.UseMvc(routes => //app.UseMvcWithDefaultRouting(); error after .net 3.0
{
    routes.MapRoute("default", "{controller}/{action}/{id?}");
});
app.UseRouting();

```

Home – дава Hello world,

Details – не се зарежда,

id – зарежда 3-те.

Единственият начин да сработи целият проект е, да оставя кода по решението с MVC с променлива тип IRouteBuilder (rb или routes) и направя промяна в HomeController:

```

app.UseStaticFiles(); //Short-circle ordering
app.UseMvc(rb => //app.UseMvcWithDefaultRouting(); error after .net 3.0
{
    rb.MapRoute(
        name: "default",
        template: "{controller}/{action}/{id?}",
        defaults: new { controller = "Home", action = "Index" });
});
app.UseRouting();

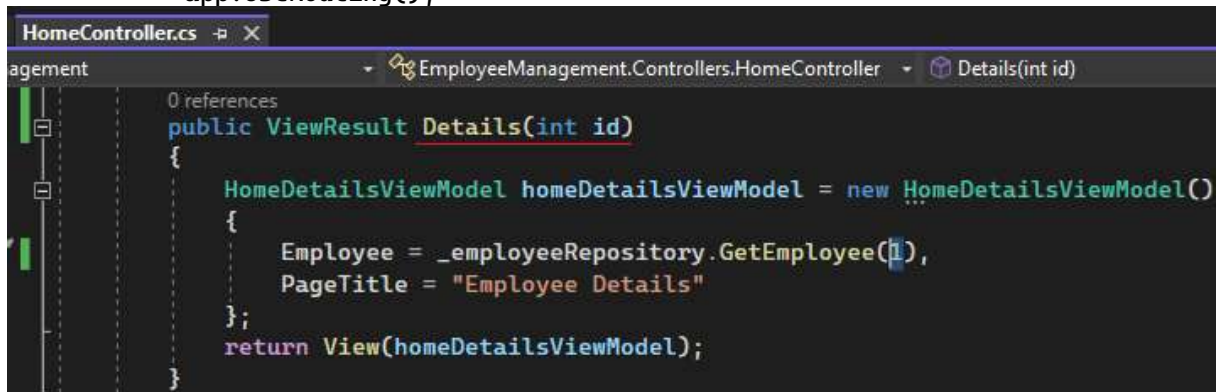
```

```

app.UseStaticFiles(); //Short-circle ordering
app.UseMvc(rb => //app.UseMvcWithDefaultRouting(); after .net 3.0
{
    rb.MapRoute(
        name: "default",
        template: "{controller}/{action}/{id?}",
        defaults: new { controller = "Home", action = "Index" });
});

```

```
app.UseRouting();
```



```
HomeController.cs
EmployeeManagement.Controllers.HomeController
Details(int id)
0 references
public IActionResult Details(int id)
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(1),
        PageTitle = "Employee Details"
    };
    return View(homeDetailsViewModel);
}
```

hard-code със стойност 1, за да се зарежда автоматично първият служител, когато директорията е /home/details, като в останалите случаи /1,/2,/3 няма проблеми. Index също си зарежда.

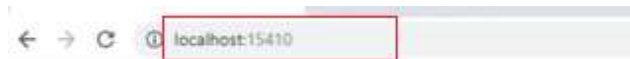


Това hard code (1) е момент, с който ще трябва да се съобразявам, защото в противен случай директно /home/details няма да се изпълнява, независимо, че id? е optional.

```
public IActionResult Details(int id)
{
    HomeDetailsViewModel homeDetailsViewModel = new
HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(1),
        PageTitle = "Employee Details"
    };
    return View(homeDetailsViewModel);
}
```



Когато се въведе id, което не присъства в списъка, като 6 в случая, се извежда служител 1. В tutorиъла, когато се навигира до root URL localhost:1234, извежда 404, но при мен работи вече, защото аз съм ги задава с друг синтаксис:



This localhost page can't be found

No web page was found for the web address: <http://localhost:15410/>

Search Google for localhost 15410

HTTP ERROR 404

```
app.UseStaticFiles();
//app.UseMvcWithDefaultRoute();
app.UseMvc(routes => {
    routes.MapRoute("default", "{controller=Home}/{action=Index}/{id?}");
});
```

Тест:

```
app.UseMvc(rb => //app.UseMvcWithDefaultRouting(); error after .net 3.0
{
    rb.MapRoute("default", "{controller=Home}/{action=Index}/{id?}");
});
```

И понеже работи безпроблемно, излиза, че наистина двата кода са идентични:

```
app.UseMvc(rb => //app.UseMvcWithDefaultRouting(); error after .net 3.0
{
    rb.MapRoute(
        name: "default",
        template: "{controller}/{action}/{id?}",
        defaults: new { controller = "Home", action = "Index" });
});
```

По-съкратен:

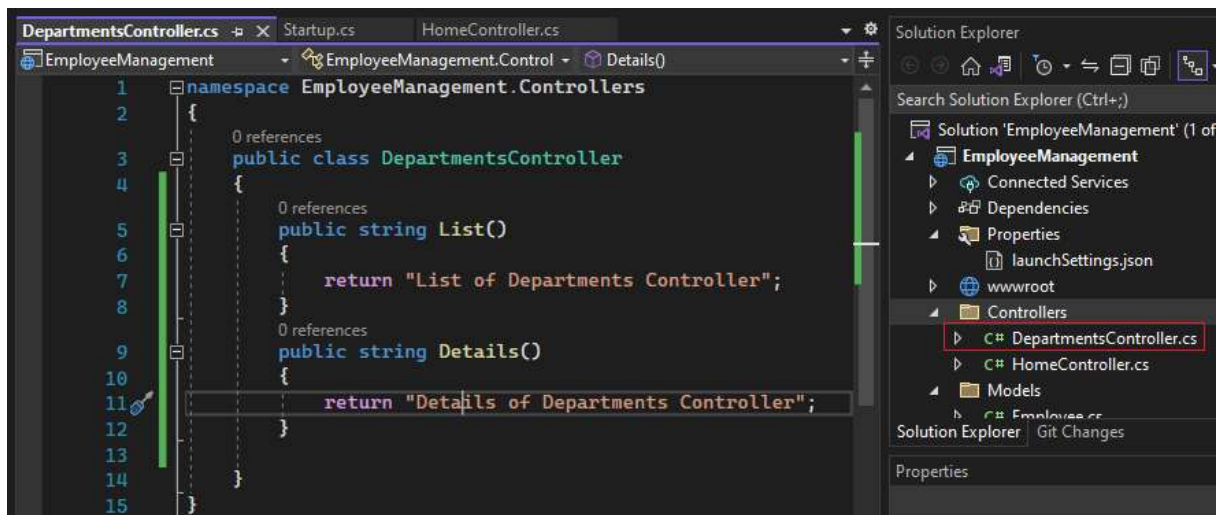
```
app.UseMvc(rb => //app.UseMvcWithDefaultRouting(); error after .net 3.0
{
    rb.MapRoute("default", "{controller=Home}/{action=Index}/{id?}");
});
```

Проверката е, че това е идентичен код с различни изписвания:

```
app.UseStaticFiles();
app.UseMvcWithDefaultRoute();
app.UseMvc(routes => {
    routes.MapRoute("default", "{controller=Home}/{action=Index}/{id?}");
});
```

(extension) IApplicationBuilder IApplicationBuilder.UseMvcWithDefaultRoute()
Adds MVC to the IApplicationBuilder request execution pipeline with a default route
named 'default' and the following template: '{controller=Home}/{action=Index}/{id?}'.

Демонстративно създаваме друг контролер DepartmentsController:



```

namespace EmployeeManagement.Controllers
{
    public class DepartmentsController
    {
        public string List()
        {
            return "List of Departments Controller";
        }
        public string Details()
        {
            return "Details of Departments Controller";
        }
    }
}

```

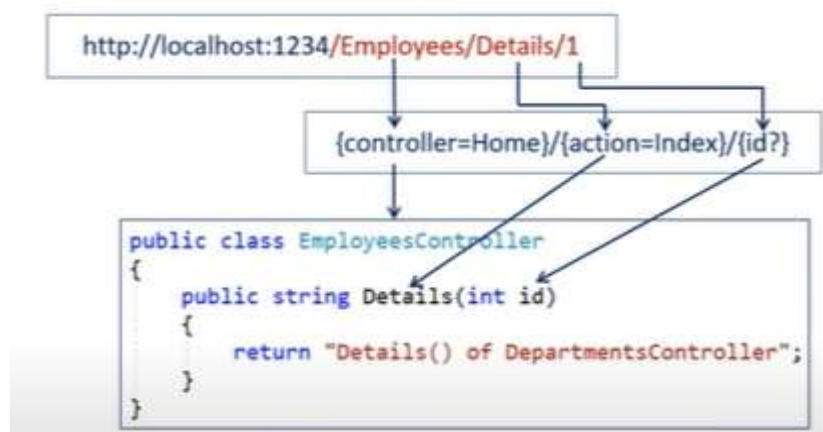


List of Departments Controller



Details of Departments Controller

conventional routing



33- Attribute routing

Ctrl+K+C – закоментираме блока код от предходния урок, за да демонстрираме UseMvc:

```
app.UseStaticFiles(); //Short-circle ordering
//app.UseMvc(rb => //app.UseMvcWithDefaultRouting(); error after .net 3.0
//{
//    rb.MapRoute("default", "{controller=Home}/{action=Index}/{id?}");
//});
app.UseMvc();

app.UseRouting();
```

Всички страници дават 404, без root URL, който изписва Hello World, вероятно заради командата UseRouting(), която има в .net 5.0, а в туториъла липсва, но без нея извежда ExceptionPage, а не 404, така че ще я оставя без да я закоментирам.



Искаме в root URL да се извежда съдържанието на HomeController/Index action. Attribute routing може да се постави на което и да е място в HomeController: директно в класа или в някой от принадлежащите му вложени методи, непосредствено в реда над конкретния метод, за когото се касае. Това става с route attribute [Route("")] // root URL = empty string ""

```
HomeController.cs* X
EmployeeManagement.Controllers.Ho

[Route("")] // root URL = empty string ""
0 references
public IActionResult Index()
```

По същият начин, искаме root URL да се зареди, когато зададем /home, затова добавяме:

```
[Route("Home")] // /home

[Route("")] // root URL = empty string ""
[Route("Home")] // /home
0 references
public IActionResult Index()

[Route("Home/Index")]

[Route("")] // root URL = empty string ""
[Route("Home")] // /home
[Route("Home/Index")]
0 references
public IActionResult Index()
```

Който и от 3-те директории да бъде указан, все да отвежда в началния root URL.



ID Name Department

1	Mary	HR
2	John	IT
3	Sam	IT

Нека подадем задължителен параметър id (без ?):

```
[Route("Home/Details/{id}")]
0 references
public IActionResult Details(int id) parameter
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(1) hard code
        PageTitle = "Employee Details"
    }
}
```

При това положение, /details = 404, а всички останали /details/1,2,3 връщат все hard-code /1

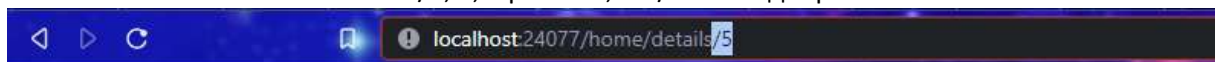


Employee Details

Name: Mary
Email: Mary@abv.bg
Department: HR

```
Employee = _employeeRepository.GetEmployee(id),
```

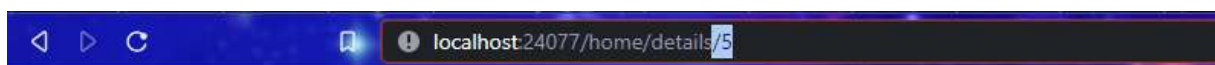
Тогава /1, 2, 3 работят, но /5 извежда грешка



An unhandled exception occurred while processing the request.

NullReferenceException: Object reference not set to an instance of an object.

AspNetCore.Views_Home_Details.ExecuteAsync() in Details.cshtml, line 10



An unhandled exception occurred while processing the request.

NullReferenceException: Object reference not set to an instance of an object.

AspNetCore.Views_Home_Details.ExecuteAsync() in Details.cshtml, line 10

Отменяме задължителния характер на id с ?, но не само на едно място, а при всяко срещане на

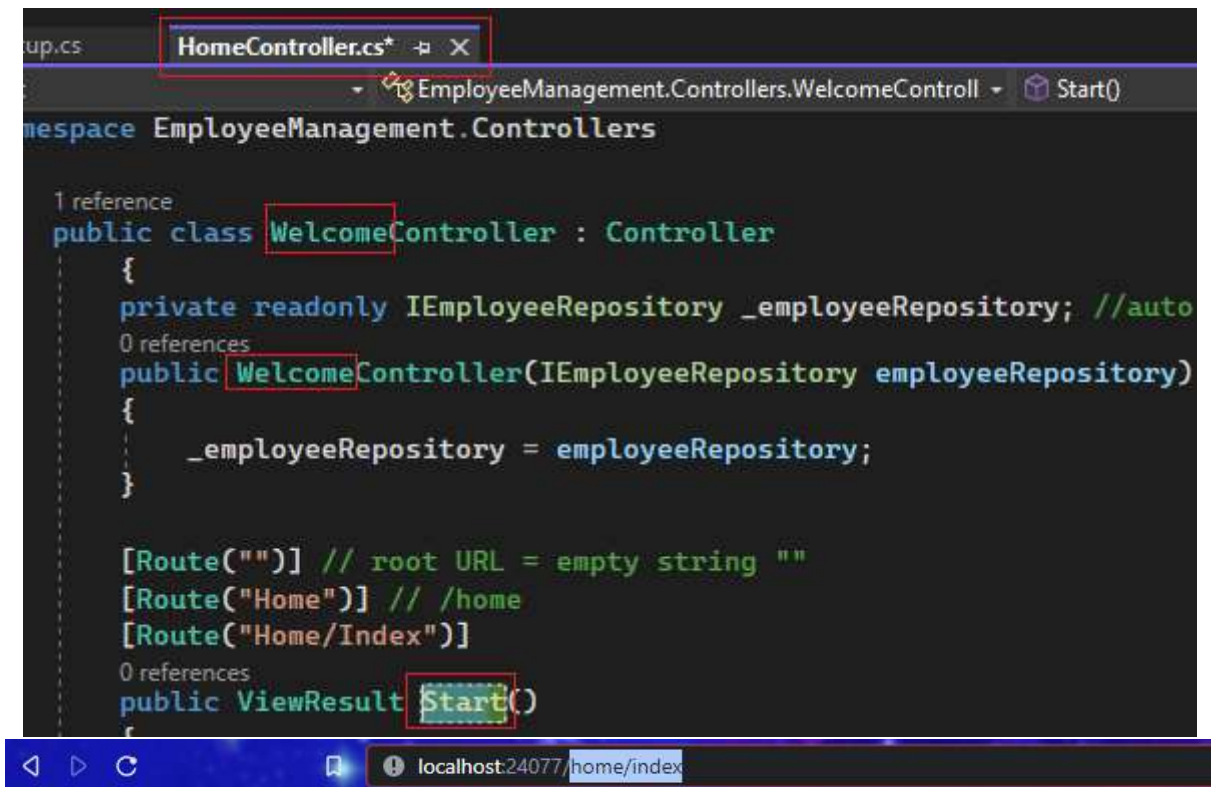
id:

```
[Route("Home/Details/{id?}")]
0 references
public IActionResult Details(int? id)
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(id ?? 1),
    }
}
```

If id is not null, use this value,
If id is null, use this default value of 1

Всичко работи, дори /details, единствено при подаване на грешна стойност /5 ExceptionPage.

При Attribute routing, името на контролера и вложените в него методи, не играят никаква роля за подадения в URL адрес. Сменяме имената им демонстративно:



```
namespace EmployeeManagement.Controllers
{
    1 reference
    public class WelcomeController : Controller
    {
        private readonly IEmployeeRepository _employeeRepository; //auto
        0 references
        public WelcomeController(IEmployeeRepository employeeRepository)
        {
            _employeeRepository = employeeRepository;
        }

        [Route("")] // root URL = empty string ""
        [Route("Home")] // /home
        [Route("Home/Index")]
        0 references
        public IActionResult Start()
    }
}
```

An unhandled exception occurred while processing the request.

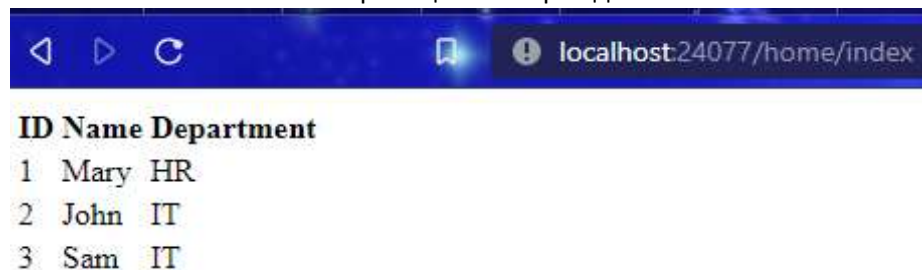
InvalidOperationException: The view 'Start' was not found. The following locations were searched:
/Views/Welcome/Start.cshtml
/Views/Shared/Start.cshtml
/Pages/Shared/Start.cshtml

Така че коригираме пътя до нея:

```
up.cs | HomeController.cs | X
EmployeeManagement.Controllers.WelcomeControll | Start()
namespace EmployeeManagement.Controllers
{
    1 reference
    public class WelcomeController : Controller
    {
        private readonly IEmployeeRepository _employeeRepository; //auto
        0 references
        public WelcomeController(IEmployeeRepository employeeRepository)
        {
            _employeeRepository = employeeRepository;
        }

        [Route("")] // root URL = empty string ""
        [Route("Home")] // /home
        [Route("Home/Index")]
        0 references
        public ViewResult Start()
        {
            var model = _employeeRepository.GetAllEmployee();
            return View("~/Views/Home/Index.cshtml", model);
        }
    }
}
```

И страницата се зарежда:



Т.е. имената на класовете и методите вътре в контролерите нямат значение, ако чрез Attribute routing сме задали пътя по този начин. Ctrl+Z.

Досега използвахме [Route] атрибутите само пред методите в класа и като резултат имаме много повторения на Home, затова може да го изнесем директно в Controller class.

По този начин комбинираме [Route] от контролера с [Route] в индивидуалните actions.


```

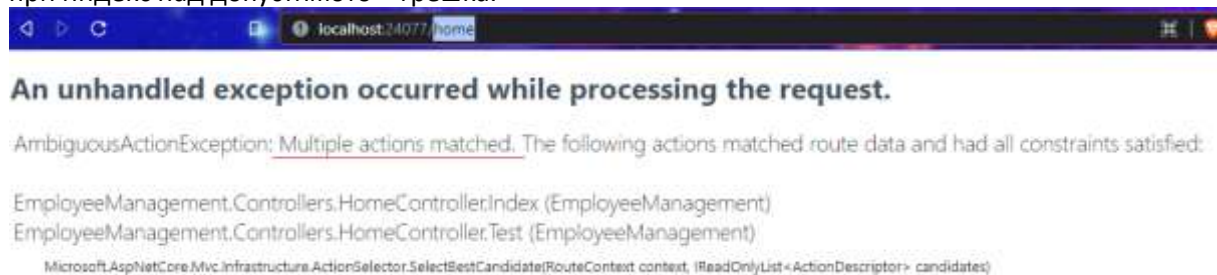
Startup.cs | HomeController.cs
EmployeeManagement.Controllers
{
    [Route("Home")]
    1 reference
    public class HomeController : Controller
    {
        private readonly IEmployeeRepository
        0 references
        public HomeController(IEmployeeReposi
        {
            _employeeRepository = employeeRep
        }

        [Route("")]
        [Route("Index")]
        0 references
        public IActionResult Index()
        {
            var model = _employeeRepository.G
            return View(model);
        }

        [Route("Details/{id?}")]
        0 references
        public IActionResult Details(int? id)
    }
}

```

При мен в тази ситуация root дава Hello world,
при home/index, /details, details/1,2,3 работи правилно,
при индекс над допустимото – грешка.



За да оправим root, който се явява над home, който сме декларирали в началото на контролера, трябва да изведем едно ниво над home чрез ~/

```

[Route("~/")] //root URL
[Route("")] // home = empty string
[Route("Index")]

```

localhost:24077

ID	Name	Department
1	Mary	HR
2	John	IT
3	Sam	IT

В тази ситуация root работи, освен /home както преди малко, независимо от размятането на [Route].

[Route] – атрибутите поддържат също token replacement. Това е мощна техника, т.к.

впоследствие, ако се наложи да сменим имената на контролерите или методите, няма да се налага да променяме routing конфигурацията.

```
[Route("[controller]")]

Startup.cs HomeController.cs
EmployeeManagement.Controllers.HomeController
namespace EmployeeManagement.Controllers
{
    [Route("[controller]")] // token replacement
    public class HomeController : Controller
    {
        private readonly IEmployeeRepository _employeeRepository;
        public HomeController(IEmployeeRepository employeeRepository)
        {
            _employeeRepository = employeeRepository;
        }

        [Route("")] // home = empty string
        [Route("~/")] //root URL
        [Route("[action]")] // token replacement
        public ActionResult Index()
        {
            var model = _employeeRepository.GetAllEmployee();
            return View(model);
        }

        [Route("[action]/{id?}")] // token replacement
        public ActionResult Details(int? id)
```

```
[Route("")] // home = empty string
[Route("~/")] //root URL
[Route("[action]")] // token replacement
```

По същия начин /home, details/5 не работят.

Ако закоментирам [Route("")] за home, се включва 3-я метод Test, който създадох последен. Също странно е, защо при details/4 и друго число извън допустимите, не се прихваща и използва като 1, щом като логиката е за false Null?

```
namespace EmployeeManagement.Controllers
{
    [Route("[controller]")] // token replacement
    public class HomeController : Controller
    {
        private readonly IEmployeeRepository _employeeRepository;
        public HomeController(IEmployeeRepository employeeRepository)
        {
            _employeeRepository = employeeRepository;
        }

        [Route("")] // home = empty string
        [Route("~/")] //root URL
        [Route("[action]")] // token replacement
        public ActionResult Index()
```



```

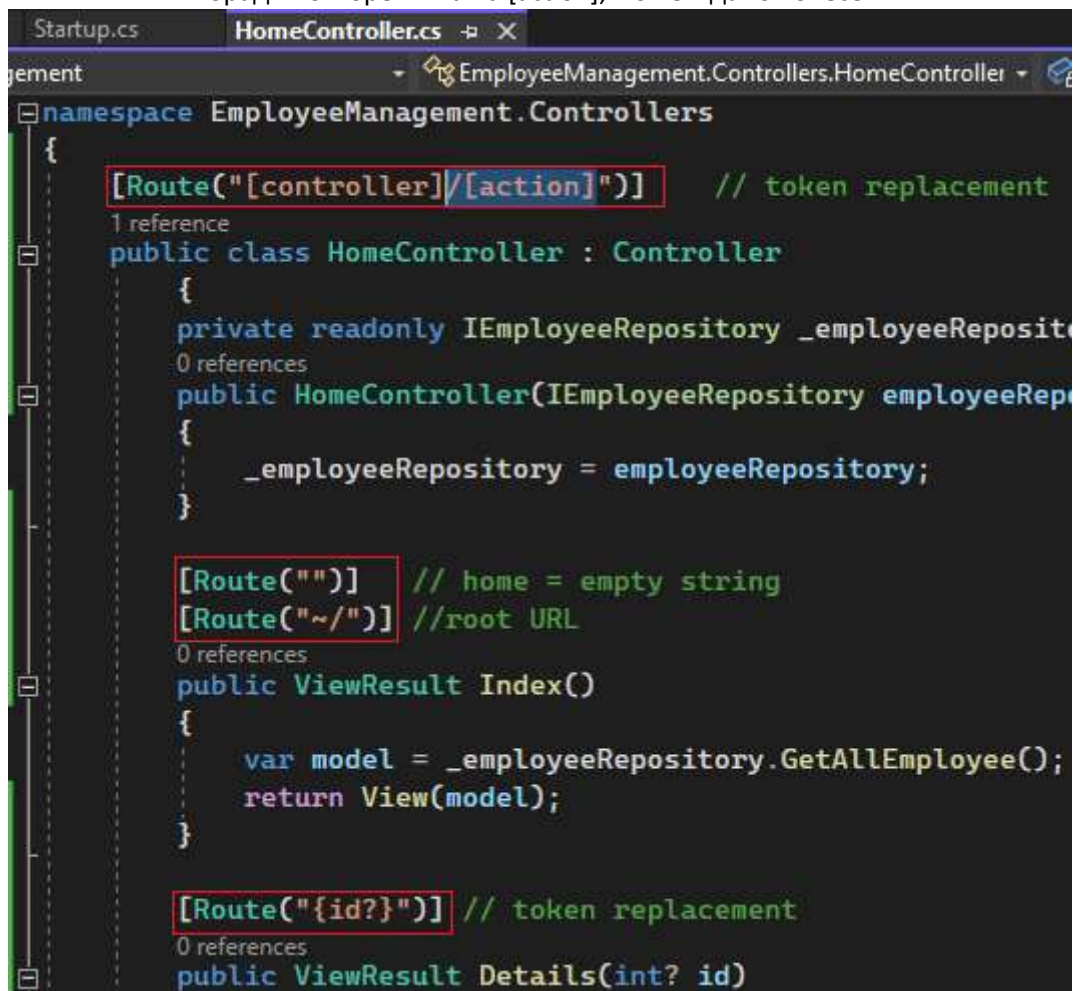
{
    var model = _employeeRepository.GetAllEmployee();
    return View(model);
}

[Route("[action]/{id?}")] // token replacement
public ActionResult Details(int? id)
{
    HomeDetailsViewModel homeDetailsViewModel = new
HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(id??1), //C# coeasing
operator
        PageTitle = "Employee Details"
    };
    return View(homeDetailsViewModel);
}

public ActionResult Test()
{
    return View("Test");
}

```

Поради повторенията на [action], можем да го изнесем.



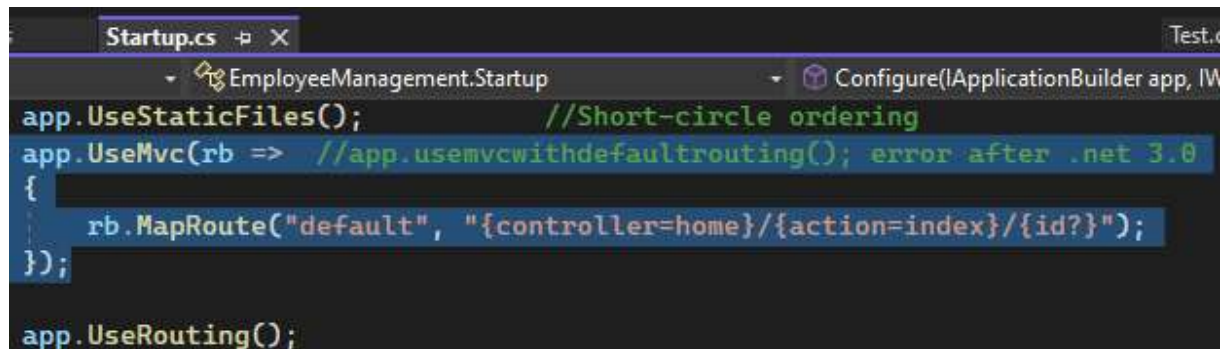
Отново работи, но /home този път извежда не Multiple actions matched, а 404. Това е защото очаква едновременно име на controller/action едновременно. Понеже не може да се направи action? Незадължителен (това го бях пробвала при MVC routing), единственият начин остава Index да се направи default. В предишните версии е било допустимо присвояването:

`[Route("[controller]/[action=index]")]`. Един от хитрите начини да се направи това, е да се добави

```
[Route("~/Home")] // Index - default
[Route("~/")] //root URL
0 references
public IActionResult Index()
```

Тогава дори `/home` работи, но за изненада, точно `/home/index` не работи :D

Изтриваме всички attribute routing и оставяме приложението на досегашния conventional MVC, който в `Startup.cs` с `Ctrl+K+U` разкоментираме като блоков коментар.



```
Startup.cs
EmployeeManagement.Startup
Configure(IApplicationBuilder app, IW
app.UseStaticFiles(); //Short-circle ordering
app.UseMvc(rb => //app.usemvcwithdefaultrouting(); error after .net 3.0
{
    rb.MapRoute("default", "{controller=home}/{action=index}/{id?}");
});
app.UseRouting();
```

С което всичко работи без грашка, с изключение на `/details/5`, който извежда `NullReference`, т.к. сме hard code-нали само 3 служителя и грешката не е прихваната.

Attribute Routing in ASP.NET Core MVC

- With Attribute Routing, **[Route]** attribute is used to define the Routes
- Route attribute can be applied on the **Controller** or the **Controller Action Methods**
- With attribute routing, routes are placed **next to the action methods** that will actually use them
- Attribute routes offer a bit **more flexibility** than conventional routes

Conventional MVC routing – http pages

Attribute routing `[Route]` – rest API (application programming interface)

34 – Install and use Bootstrap

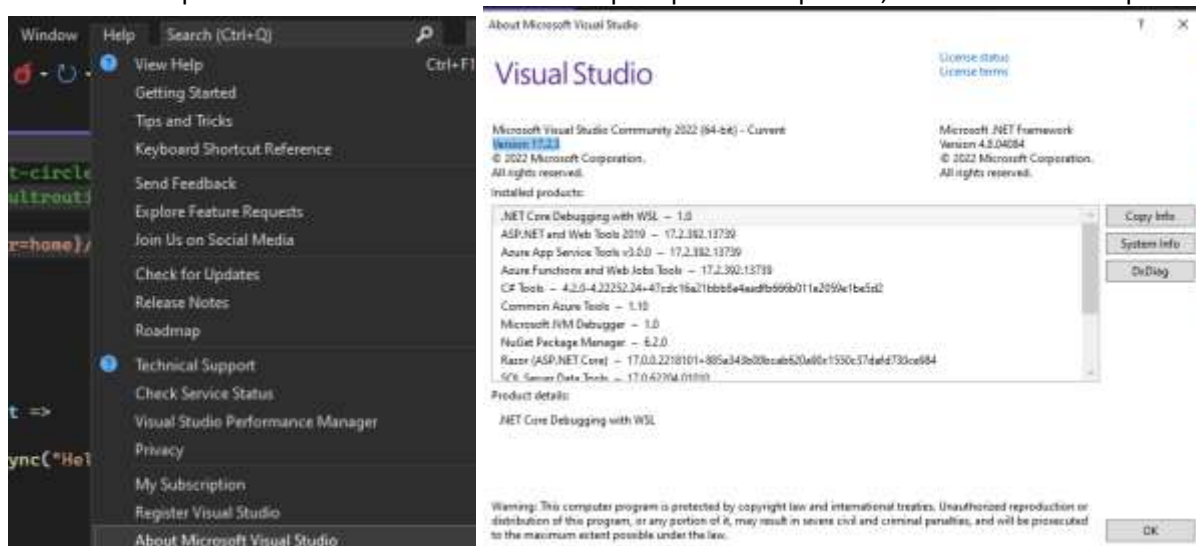
Има много потребителски пакети, които предлагат инсталация на bootstrap, JQuery:

Tools to Install Client-Side Packages

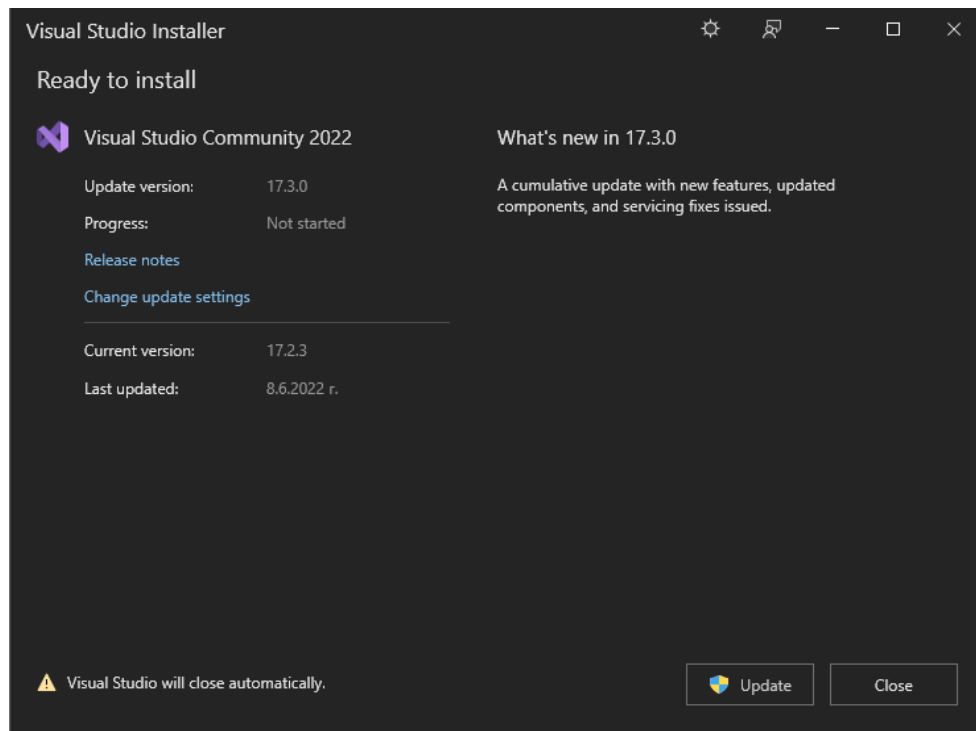


Ние ще използваме Library Manager (LibMan).

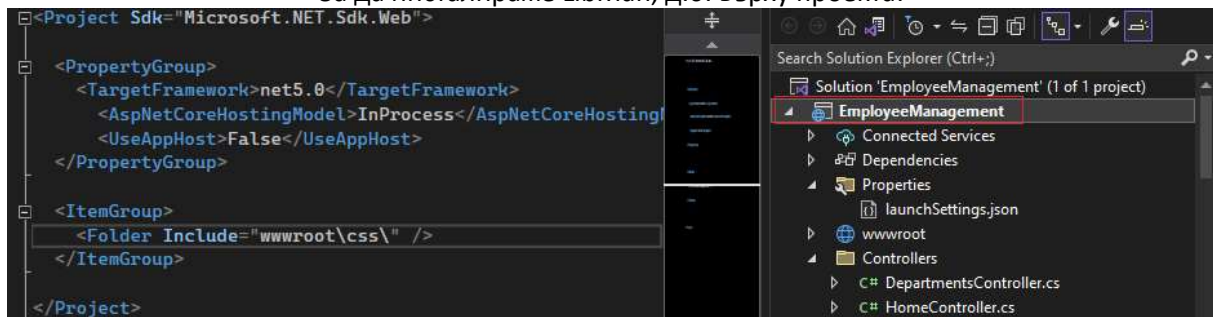
От меню Help -> About Microsoft Visual Studio проверяваме версията, която сме инсталирали:



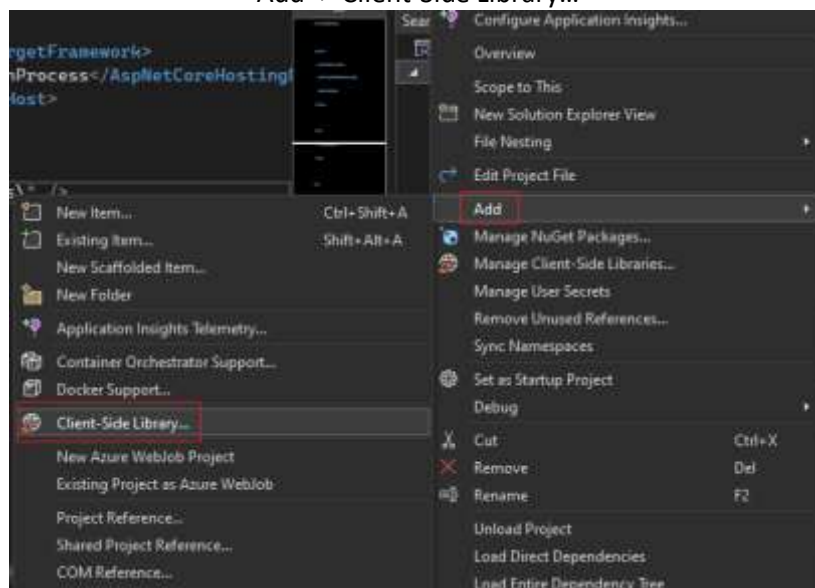
Евентуално може от Help -> Check for Updates да я обновим.



За да инсталираме LibMan, д.б. върху проекта:

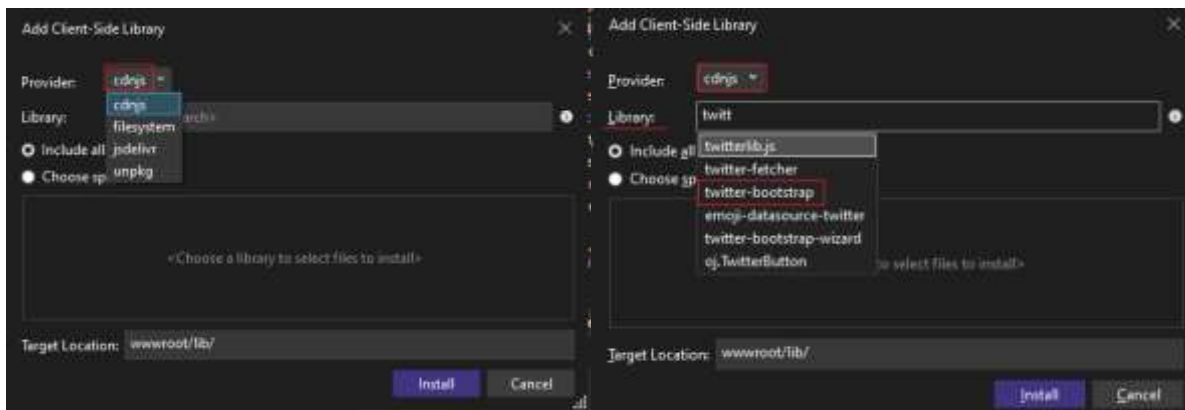


Add -> Client-Side Library...

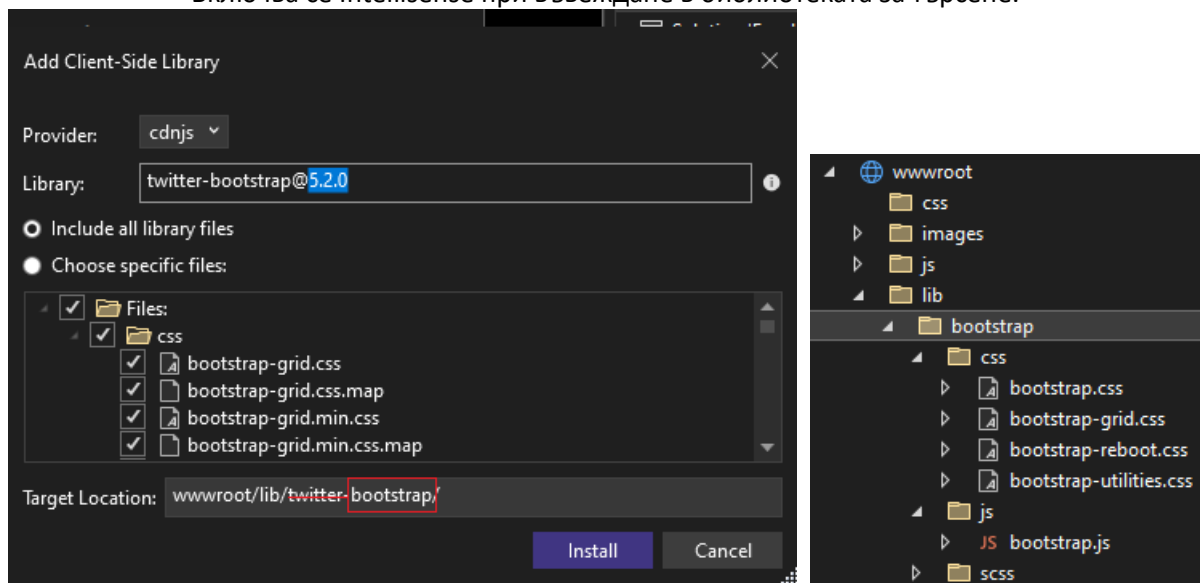


Имаме възможности за инсталация

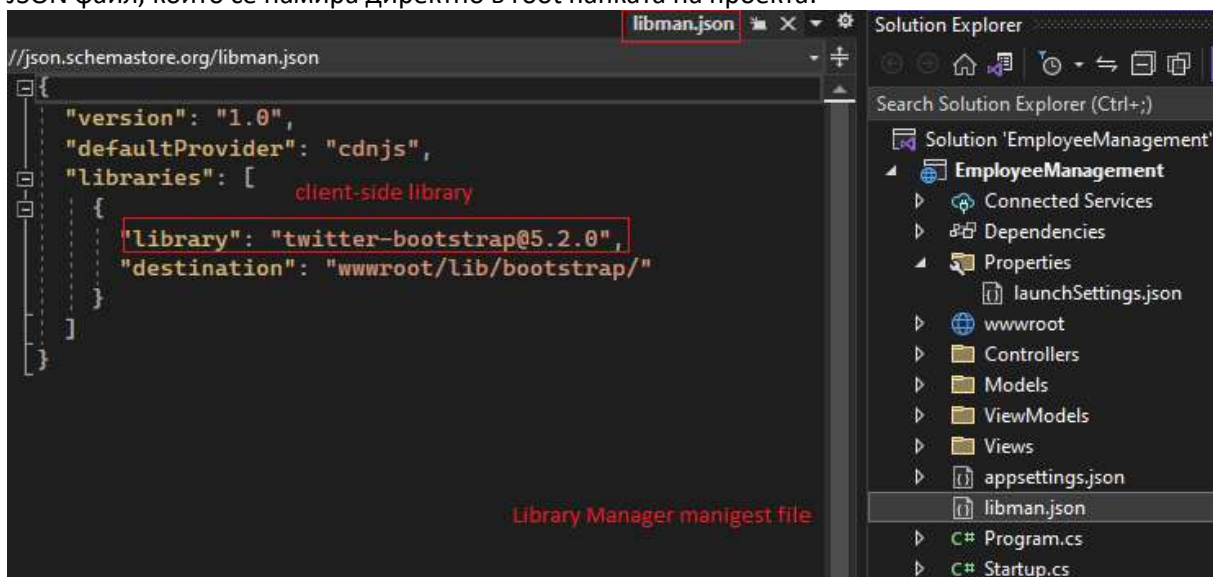
- * от CDN – Content Delivery Network (for better performance)
- * от файловата система



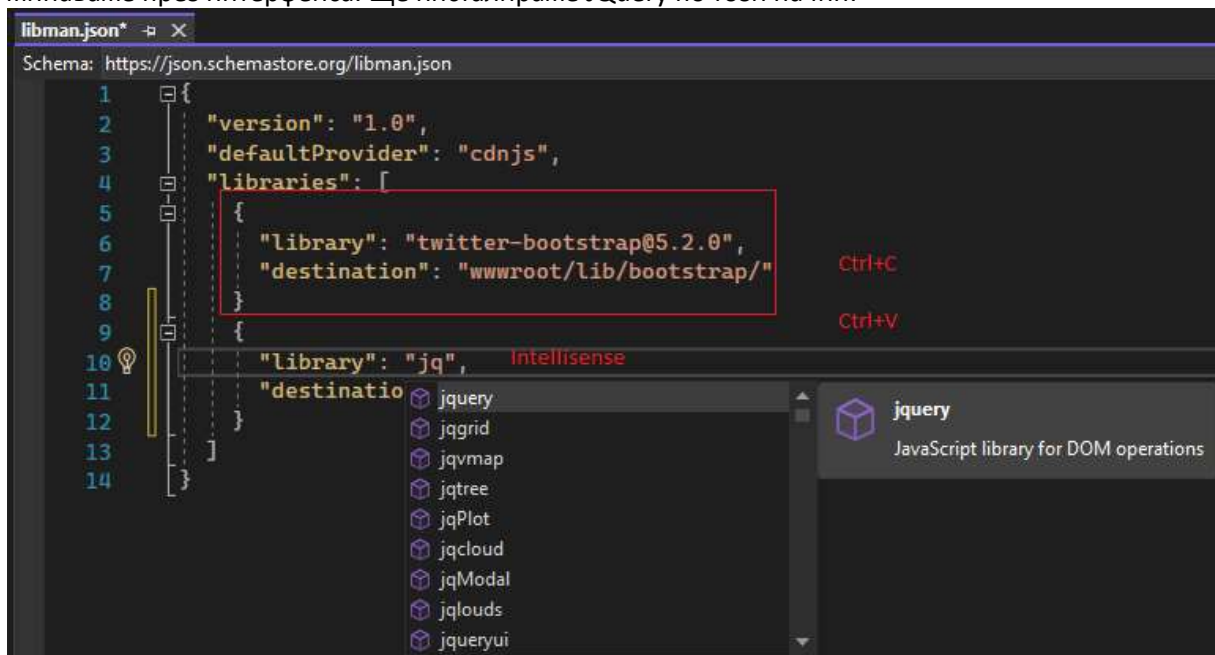
Включва се Intellisense при въвеждане в библиотеката за търсене.



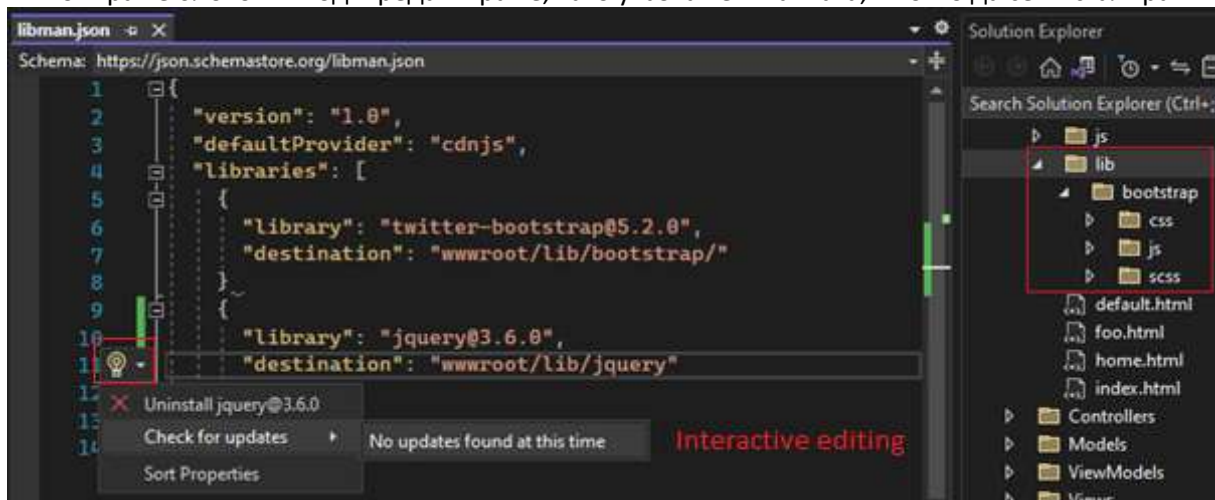
Инсталира се последната версия, но ако искаме по-стара, при изтриване на числата след @ и въвеждане на по-малко (ако вместо 5, въведем 4), се включва Intellisense и предлага избор между по-старите версии. Това не се получи при мен – освен 5.2.0 нямам избор на по-стара. Папката, в която се инсталират, е познатата ни в проекта `wwwroot`, в която бяха статичните файлове. За улеснение оставяме просто `/lib/bootstrap`, която веднага се създава с вградени автоматично `css`, `js` файлове и папки. За да управлява тези библиотеки и пакети, LibMan създава JSON файл, който се намира директно в `root` папката на проекта:



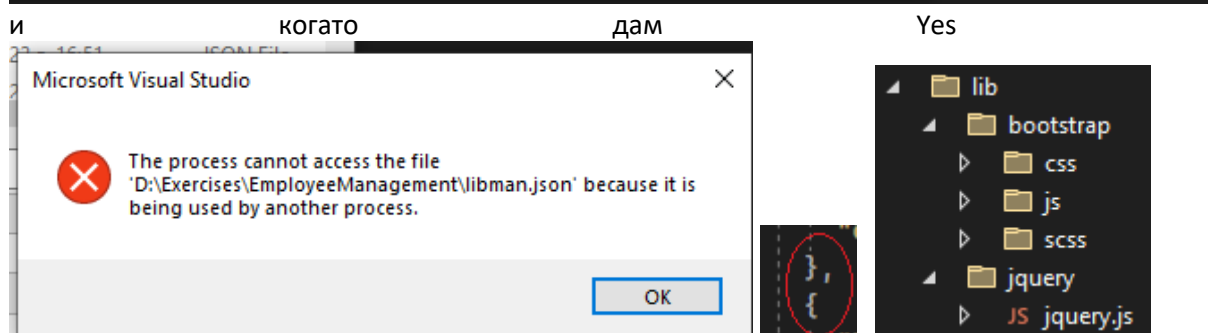
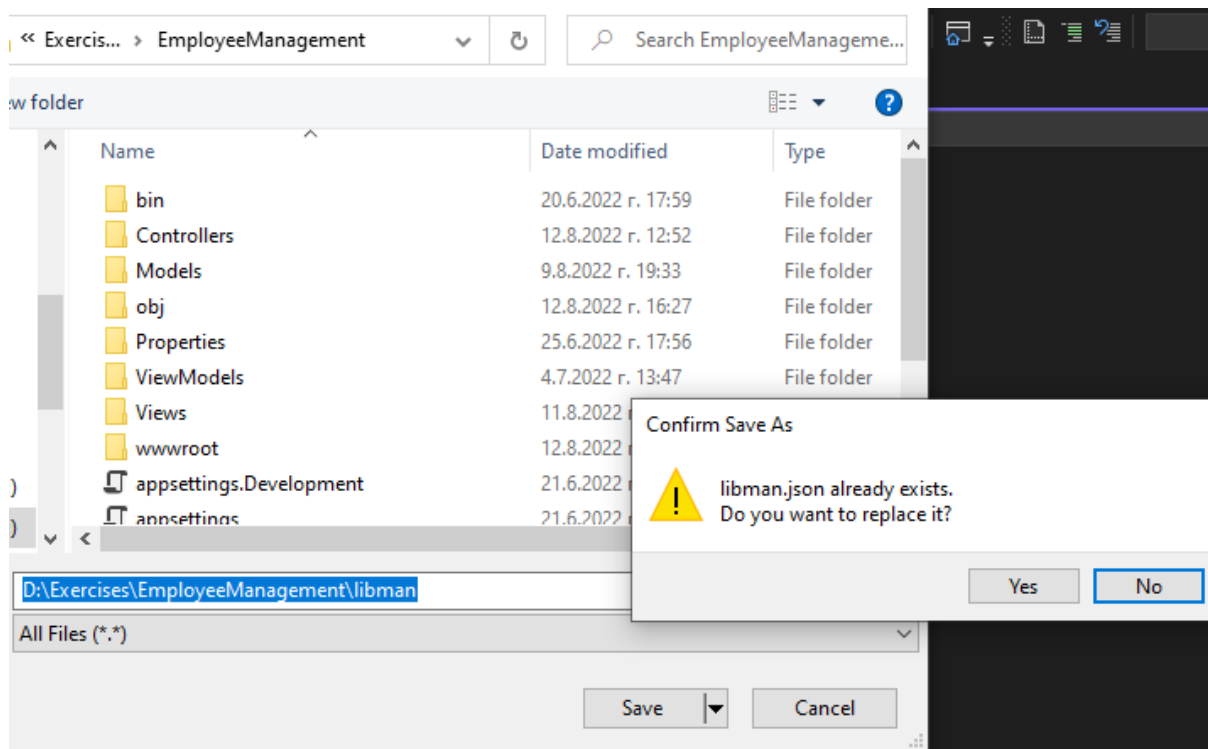
libman.json = Library Manager manifest file.
Той съдържа entry for client-side library, който току-що инсталирахме.
С редактирането му, можем директно да инсталираме/деинсталираме библиотеки, без да минаваме през интерфейса. Ще инсталираме JQuery по този начин.



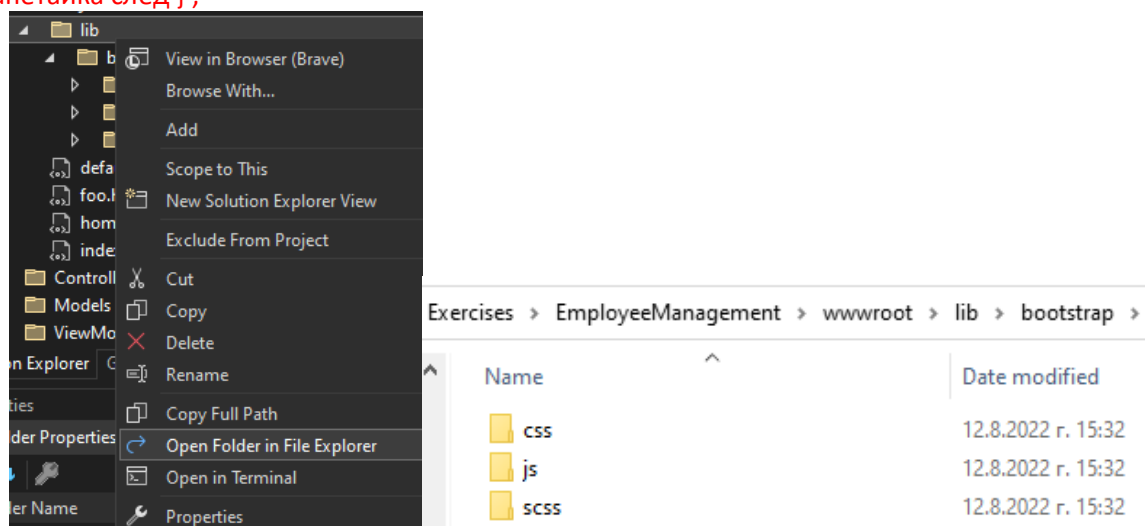
Копираме блоковия код и редактираме, като указваме и папката, в която да се инсталира:



През цялото време се включва Intellisense, обаче новата папка така и не я виждам нито в проекта на програмата, нито в Windows Explorer, а дава, че е инсталирана, щом има опция Uninstall... Деинсталирах и инсталирах наново, но пак не се появява автоматично директорията.
Не ми позволява да запазя промените в libman.js -> Safe

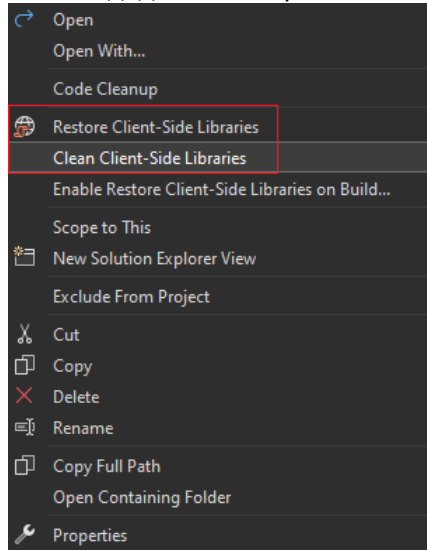


Предполагам, че заради това не се получават редакциите. Намерих си грешката: **забравила съм запетайка след },**



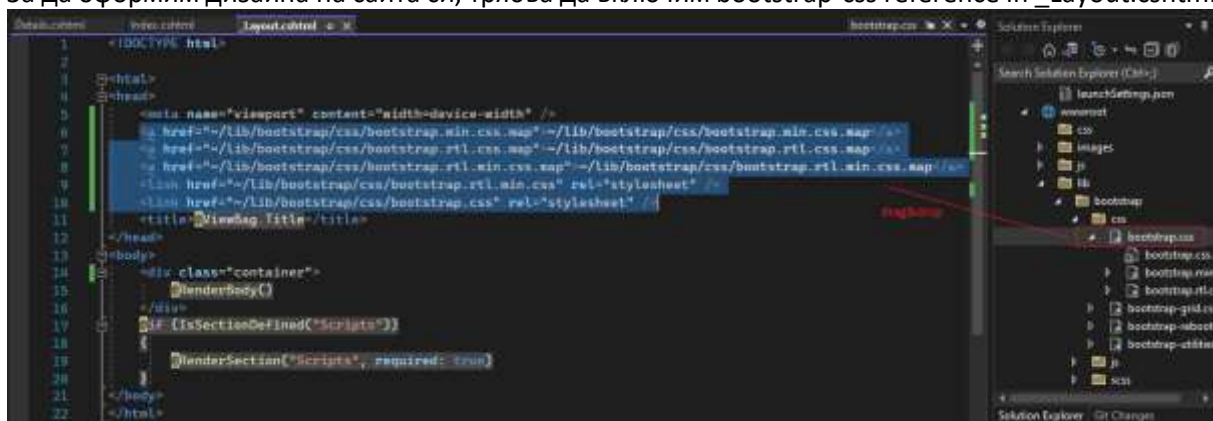
При д.б. върху libman.json имаме възможност да изтрием с Clean и възстановим Restore като

записи всички създадени от потребителя библиотеки:



с което всички инсталирани в папка lib трябва да изчезнат от VS, но при мен не се получи. Записите им обаче са останали в libman.json, така че при Restore могат да се възстановят (и lib папка се създава наново, но при мен – не!).

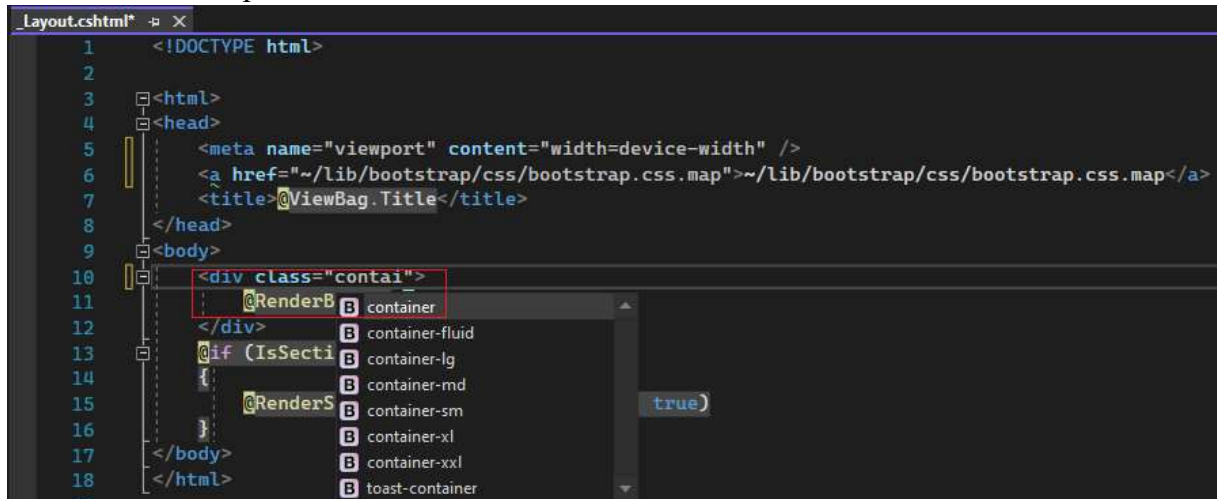
Чрез редакцията му, можем uninstall, update, downgrade версии, независимо дали само с текстова редакция на файла, или интерактивно с подсказката в началото на реда. За да оформим дизайна на сайта си, трябва да включим bootstrap-css reference in _Layout.cshtml



Изтривам текстовата част на линковете, за да не се показва в браузъра:

```
<a href="~/lib/bootstrap/css/bootstrap.min.css.map"></a>
<a href="~/lib/bootstrap/css/bootstrap.rtl.css.map"></a>
<a href="~/lib/bootstrap/css/bootstrap.rtl.min.css.map"></a>
<link href="~/lib/bootstrap/css/bootstrap.rtl.min.css" rel="stylesheet" />
<link href="~/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
```

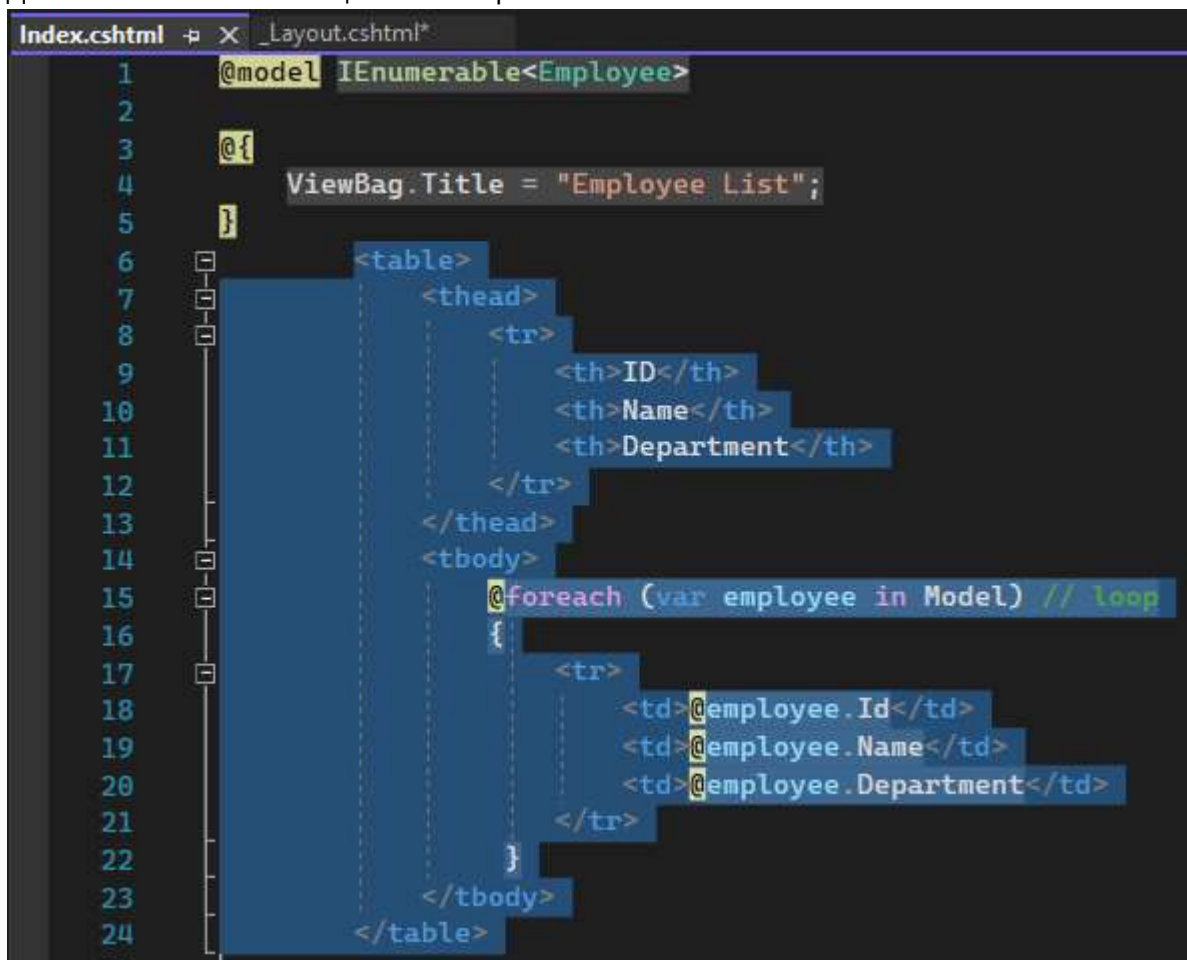
Kudvenkat bootstrap tutorial link.



```
1 <!DOCTYPE html>
2
3 <html>
4 <head>
5   <meta name="viewport" content="width=device-width" />
6   <a href="~/lib/bootstrap/css/bootstrap.css.map"~/lib/bootstrap/css/bootstrap.css.map</a>
7   <title>@ViewBag.Title</title>
8 </head>
9 <body>
10  <div class="contai">
11    @RenderB container
12  </div>
13  @if (IsSecti
14  {
15    @RenderS
16  }
17 </body>
18 </html>
```

Имаме IntelliSense, с която задаваме контейнер на съдържанието.

Добавяме към html-таблицата bootstrap класове за стил:



```
1 @model IEnumerable<Employee>
2
3 @{
4   ViewBag.Title = "Employee List";
5 }
6
7 <table>
8   <thead>
9     <tr>
10      <th>ID</th>
11      <th>Name</th>
12      <th>Department</th>
13    </tr>
14   </thead>
15   <tbody>
16     @foreach (var employee in Model) // loop
17     {
18       <tr>
19         <td>@employee.Id</td>
20         <td>@employee.Name</td>
21         <td>@employee.Department</td>
22       </tr>
23     }
24   </tbody>
25 </table>
```

```

Details.cshtml | Index.cshtml * X _Layout.cshtml
@model IEnumerable<Employee>

@{
    ViewBag.Title = "Employee List";
}

<div class="card-deck">
    @foreach (var employee in Model)
    {
        <div class="card m-3">
            <div class="card-header">
                <h3>@employee.Name</h3>
            </div>
            
            <div class="card-footer text-center">
                <a href="#" class="btn btn-primary">View</a>
                <a href="#" class="btn btn-primary">Edit</a>
                <a href="#" class="btn btn-danger">Delete</a>
            </div>
        </div>
    }
</div>

```

A card in Bootstrap 4 is a bordered box with some padding around its content. It includes options for headers, footers, content, colors, etc.

Intellisense не ми позволява **card-deck** в Bootstrap 5, затова оставям card-group.

```

Index.cshtml X _Layout.cshtml
@model IEnumerable<Employee>

@{
    ViewBag.Title = "Employee List";
}

<div class="card-group">
    @foreach (var employee in Model)
    {
        <div class="card m-3">
            <div class="card-header">
                <h3>@employee.Name</h3>
            </div>
            
            <div class="card-footer text-center">
                <a href="#" class="btn btn-primary">View</a>
                <a href="#" class="btn btn-primary">Edit</a>
                <a href="#" class="btn btn-danger">Delete</a>
            </div>
        </div>
    }
</div>

@model IEnumerable<Employee>

@{
    ViewBag.Title = "Employee List";
}

<div class="card-group">

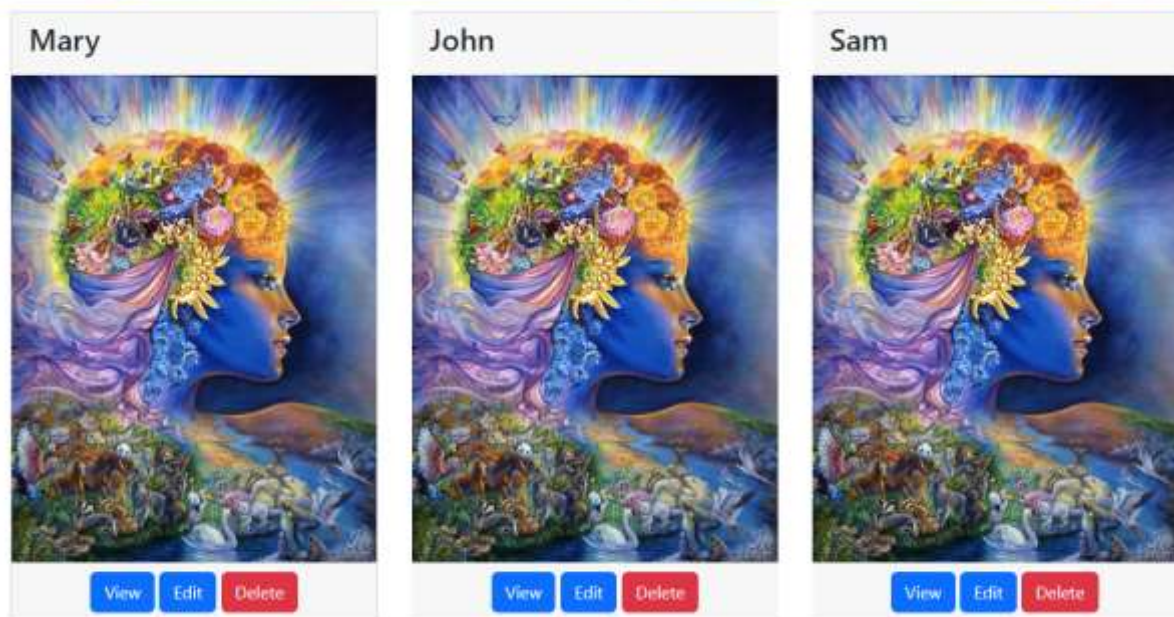
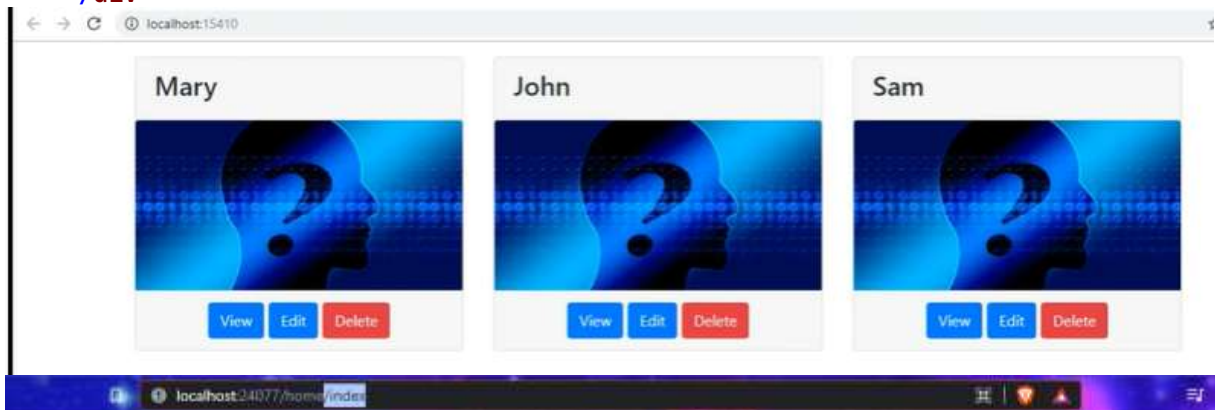
```



```

@foreach (var employee in Model)
{
    <div class="card m-3">
        <div class="card-header">
            <h3>@employee.Name</h3>
        </div>
        
        <div class="card-footer text-center">
            <a href="#" class="btn btn-primary">View</a>
            <a href="#" class="btn btn-primary">Edit</a>
            <a href="#" class="btn btn-danger">Delete</a>
        </div>
    </div>
}
</div>

```



```

Details.cshtml  X Index.cshtml  _Layout.cshtml
1      @model HomeDetailsViewModel
2
3      @{
4          ViewBag.Title = "Employee Details";
5      }
6
7      <h3>@Model.PageTitle</h3>
8
9      <div>
10         Name: @Model.Employee.Name
11     </div>
12     <div>
13         Email: @Model.Employee.Email
14     </div>
15     <div>
16         Department: @Model.Employee.Department
17     </div>
18     @section Scripts {
19         <script src="~/js/CustomScript.js"></script>
20     }

```

Заместваме маркираното с:

```

Details.cshtml* X Index.cshtml  _Layout.cshtml
1      @model HomeDetailsViewModel
2      @{
3          ViewBag.Title = "Employee Details";
4      }
5      <div class="row justify-content-center m-3">
6          <div class="col-sm-8">
7              <div class="card">
8                  <div class="card-header">
9                      <h1>@Model.Employee.Name</h1>
10                 </div>
11
12                 <div class="card-body text-center">
13                     
14
15                     <h4>Employee ID: @Model.Employee.Id </h4>
16                     <h4>Employee Email: @Model.Employee.Email </h4>
17                     <h4>Employee Department: @Model.Employee.Department </h4>
18                 </div>
19                 <div class="card-footer text-center">
20                     <a href="#" class="btn btn-primary">Back</a>
21                     <a href="#" class="btn btn-primary">Edit</a>
22                     <a href="#" class="btn btn-danger">Delete</a>
23                 </div>
24             </div>
25         </div>
26     </div>
27     @section Scripts {
28         <script src="~/js/CustomScript.js"></script>
29     }

```

@model HomeDetailsViewModel

@{

ViewBag.Title = "Employee Details";

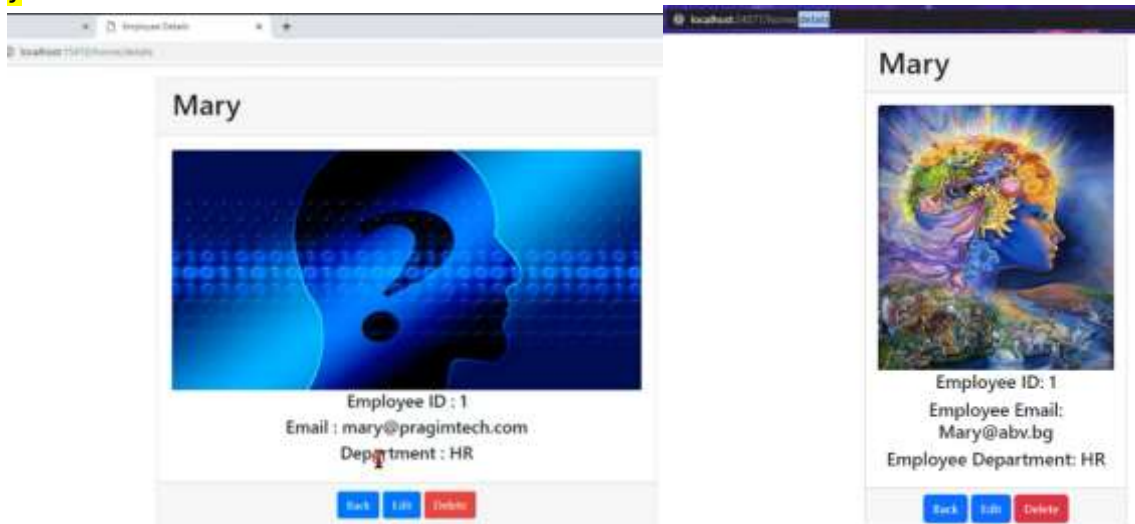
```

}
<div class="row justify-content-center m-3">
  <div class="col-sm-8">
    <div class="card">
      <div class="card-header">
        <h1>@Model.Employee.Name</h1>
      </div>

      <div class="card-body text-center">
        

        <h4>Employee ID: @Model.Employee.Id </h4>
        <h4>Employee Email: @Model.Employee.Email </h4>
        <h4>Employee Department: @Model.Employee.Department </h4>
      </div>
      <div class="card-footer text-center">
        <a href="#" class="btn btn-primary">Back</a>
        <a href="#" class="btn btn-primary">Edit</a>
        <a href="#" class="btn btn-danger">Delete</a>
      </div>
    </div>
  </div>
</div>
<@section Scripts {
  <script src="~/js/CustomScript.js"></script>
}>

```



Заради моята картинка направих леки корекции на кода:

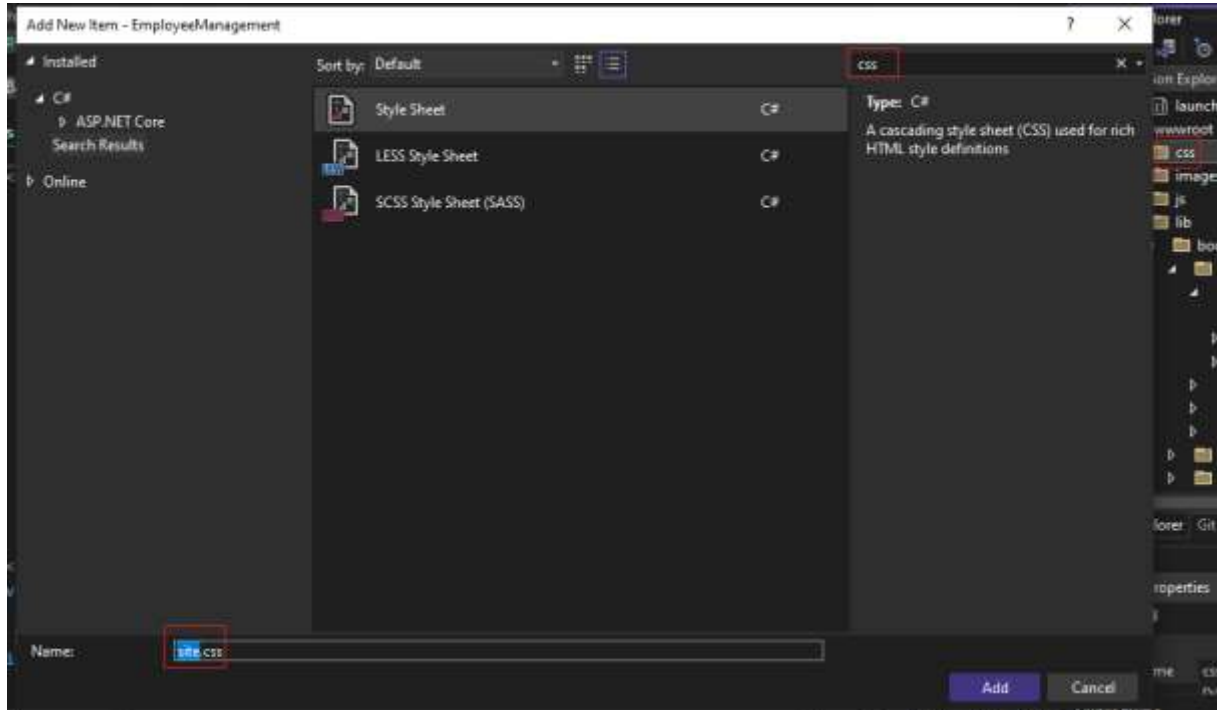
```

<div class="row justify-content-center m-1">
  <div class="col-sm-4">

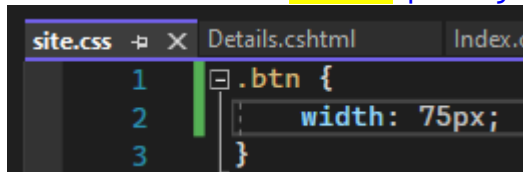
    

```

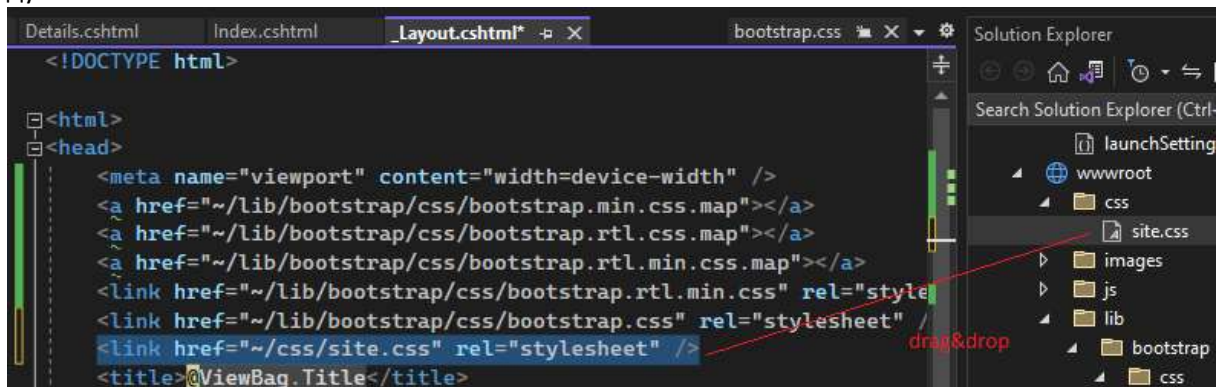
Добавяме и CSS:



`<a href="#" class="btn btn-primary">Back</a>`



за да може всички бутони да не са с еднаква дължина, а не всеки авто според дължината на думата.



При провлачване се пействат много редове код, като се оставя само последният, който е необходим.

Library Manager - LibMan



- Light-weight, client side library acquisition tool
- Downloads from the file system or from a CDN
- Visual Studio 2017 version 15.8 or later required
- libman.json is the library manager manifest file
- Use GUI or libman.json file to manage client side packages

35 – Tag Helpers

Tag Helpers in ASP.NET Core

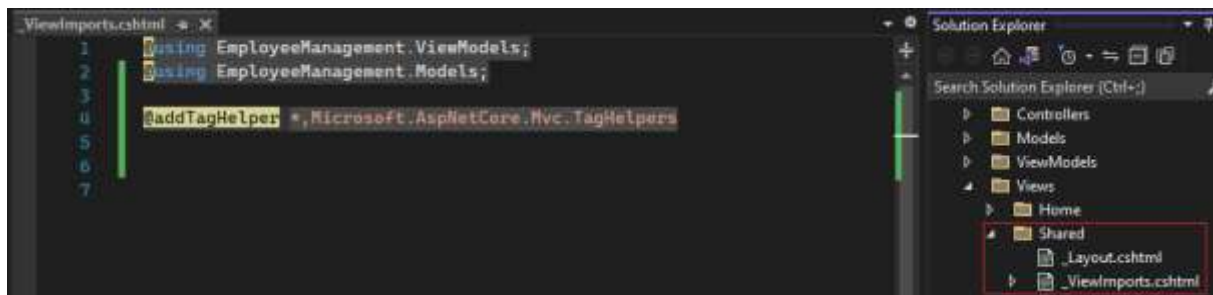
- Server side components
- Processed on the server to create and render HTML elements
- Similar to HTML helpers
- Built-in tag helpers - Generating links, creating forms, loading assets etc.

- Importing tag helpers

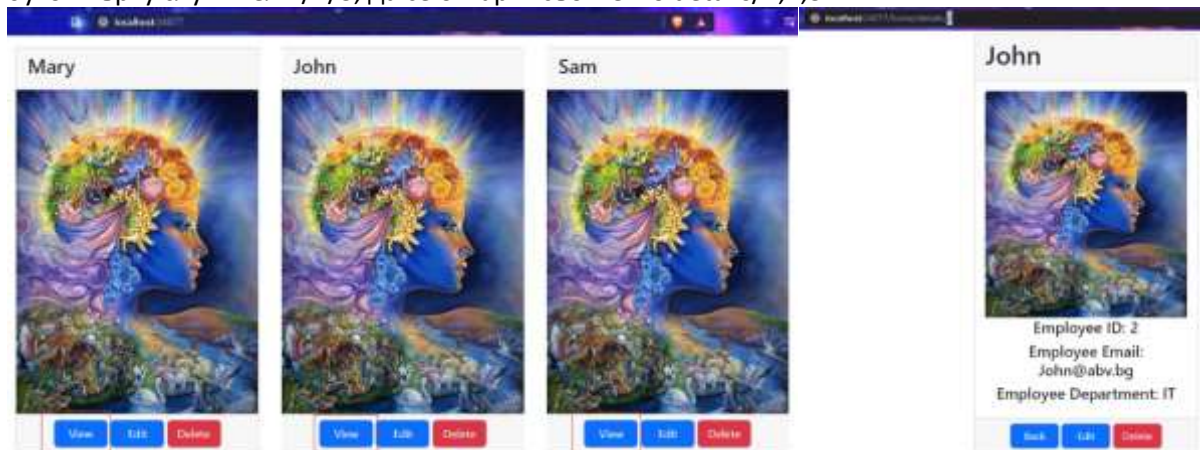
@addTagHelper [] Microsoft.AspNetCore.Mvc.TagHelpers

All Assembly

@addTagHelper *,Microsoft.AspNetCore.Mvc.TagHelpers



Към момента в Index.cshtml anchor-element е празен #, искаме при натискане на съответния бутон върху служител 1/2/3, да се отворят съответно details/1,2,3.



За целта адресираме `<a href="#" class="btn btn-primary">View</a>`

```

<div class="card m-3">
  <div class="card-header">
    <h3>Mary</h3>
  </div>
  
  <div class="card-footer text-center">
    <a href="/home/details/1" class="btn btn-primary">View</a>
    <a href="#" class="btn btn-primary">Edit</a>
    <a href="#" class="btn btn-danger">Delete</a>
  </div>
</div>
<div class="card m-3">
  <div class="card-header">
    <h3>John</h3>
  </div>
  
  <div class="card-footer text-center">
    <a href="/home/details/2" class="btn btn-primary">View</a>
    <a href="#" class="btn btn-primary">Edit</a>
    <a href="#" class="btn btn-danger">Delete</a>
  </div>
</div>
<div class="card m-3">
  <div class="card-header">
    <h3>Sam</h3>
  </div>
  
  <div class="card-footer text-center">
    <a href="/home/details/3" class="btn btn-primary">View</a>
  </div>
</div>

```

Има няколко начина, Като се включва Intellisense:

- 1) `<a href="/home/details/@employee.Id" class="btn btn-primary">View</a>`
- 2) `@Html.ActionLink("View", "details", "home", new { id = employee.Id })`

което е само текст без btn

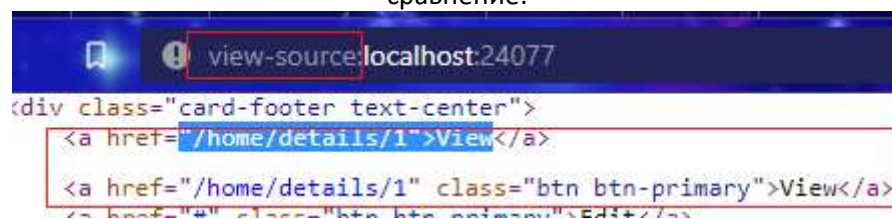
```

@Html.ActionLink("View", "details", "home", new { id = employee.Id })
      text      action      controller      anonymous object
<a href="/home/details/@employee.Id" class="btn btn-primary">View</a>

```

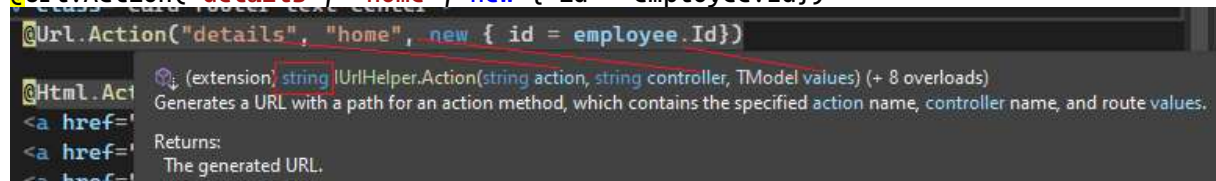


сравнение:

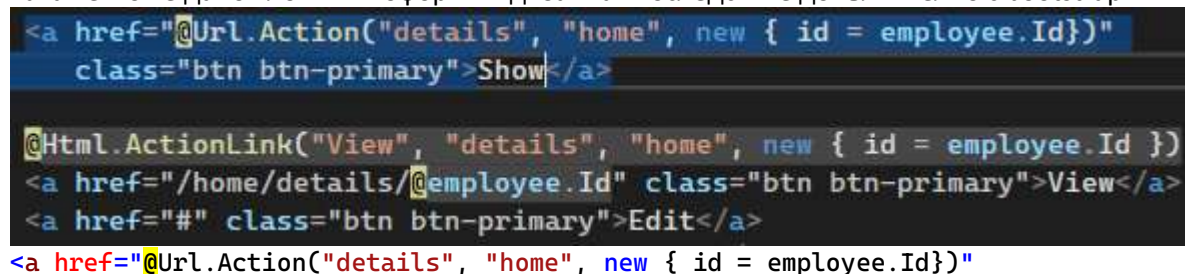


3) текстов стринг, който може да бъде вложен в <a>

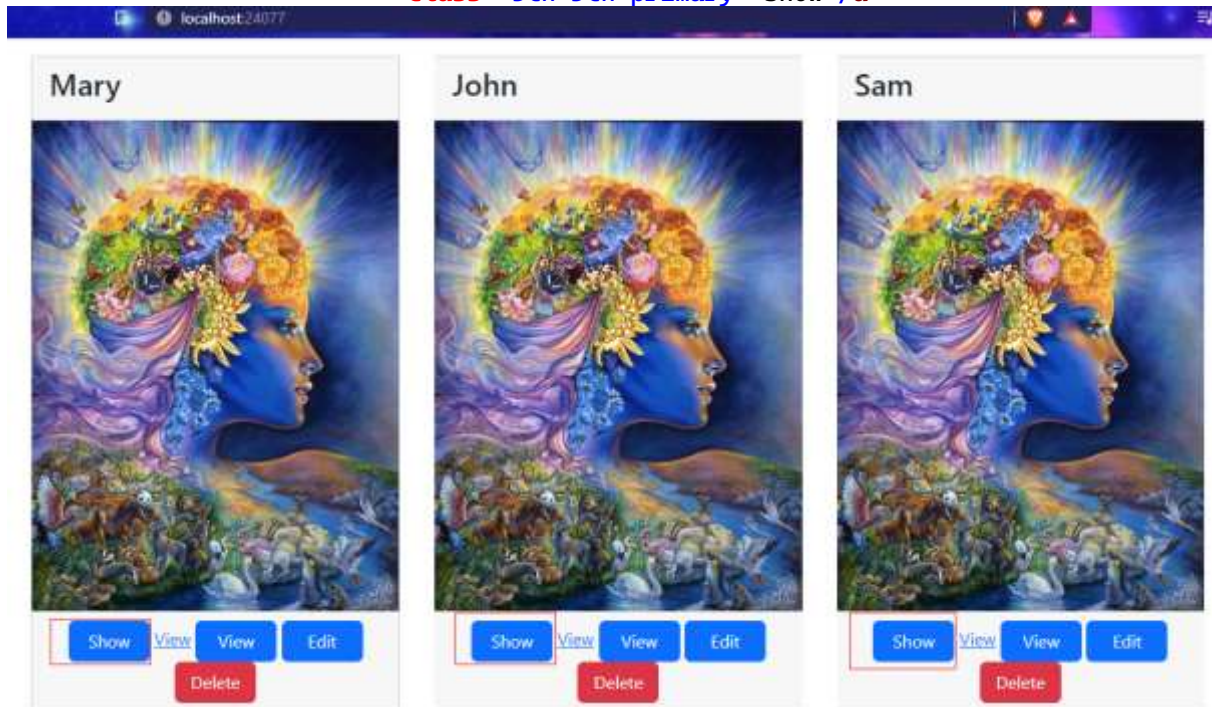
@Url.Action("details", "home", new { id = employee.Id})



Така че може да го вложим и оформим дизайна впоследствие допълнително с bootstrap:



`class="btn btn-primary">Show</a>`



4) TagHelper – ще използваме нея и изтрием останалите:

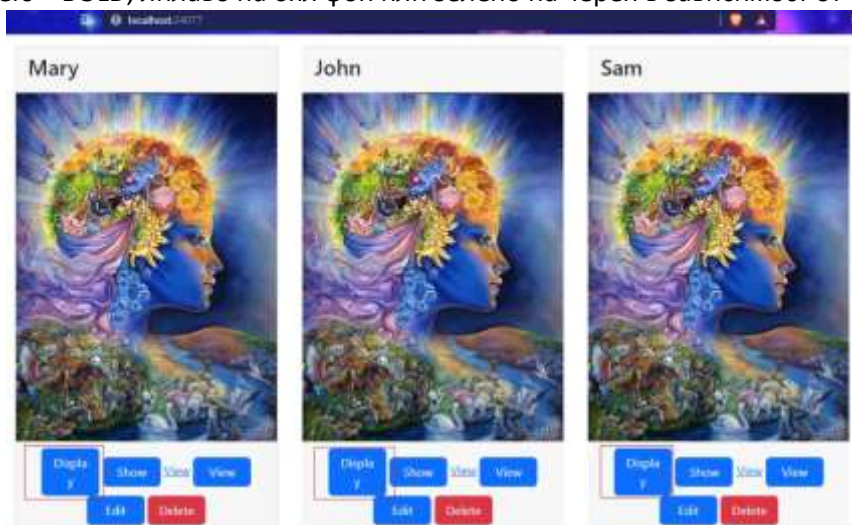
```
<a asp-controller="home" asp-action="details" asp-route-id="@employee.Id"
class="btn btn-primary">Display</a>
the name of the parameter

<a href="@Url.Action("details", "home", new { id = employee.Id})"
class="btn btn-primary">Show</a>

@Html.ActionLink("View", "details", "home", new { id = employee.Id })
<a href="/home/details/@employee.Id" class="btn btn-primary">View</a>
<a href="#" class="btn btn-primary">Edit</a>
<a href="#" class="btn btn-danger">Delete</a>
```

`<a asp-controller="home" asp-action="details" asp-route-id="@employee.Id"
class="btn btn-primary">Show`

* TagHelpers – BOLD, лилаво на бял фон или зелено на черен в зависимост от фона на VS.



Изтриваме останалите и сменяме надписа на View:

```
<a asp-controller="home" asp-action="details" asp-route-id="@employee.Id"
  class="btn btn-primary">View</a>
<a href="#" class="btn btn-primary">Edit</a>
<a href="#" class="btn btn-danger">Delete</a>
```

Details.cshtml:

```
Details.cshtml  Index.cshtml
<a asp-controller="home" asp-action="index" class="btn btn-primary">Back</a>
<a href="#" class="btn btn-primary">Edit</a>
<a href="#" class="btn btn-danger">Delete</a>
```

Понеже и двата Index + Details принадлежат на HomeController, не е задължително да се указва `asp-controller="home"`, а понеже Index няма други параметри, не е необходимо да уточняваме `id` допълнително след него.

Tag Helpers in ASP.NET Core

Manually generating links

```
<a href="/home/details/@employee.Id">View</a>
```

Using Tag Helpers

```
<a asp-controller="home" asp-action="details"
  asp-route-id="@employee.Id">View</a>
```

36 – Why use TagHelpers

* Ако, когато приключим проекта, получим задание да променим линка на адреса с добавка на основния, например от `rooturl` -> `abv/rooturl`.

* Първо се изпълняват TagHelpers, после route-templates.

Why use Tag Helpers

```
app.UseMvc(routes =>
{
    routes.MapRoute("default", "{controller=Home}/{action=Index}/{id?}");
});
```

Route Template

`<a href="/home/details/@employee.Id">View</a>`
↓
`<a href="/home/details/1">View</a>`

✓

`<a asp-controller="home" asp-action="details" asp-route-id="@employee.Id">View</a>`
↓
`<a href="/home/details/1">View</a>`

✓

В даден момент, когато цялата работа по проекта е свършена, решаваме, че всички линкове трябва да започнат от сайт с името на компанията pragim.

Why use Tag Helpers

```
app.UseMvc(routes =>
{
    routes.MapRoute("default", "pragim/{controller=Home}/{action=Index}/{id?}");
});
```

Route Template

`<a href="/home/details/@employee.Id">View</a>`
↓
`<a href="/home/details/1">View</a>`

✗

`<a asp-controller="home" asp-action="details" asp-route-id="@employee.Id">View</a>`
↓
`<a href="/pragim/home/details/1">View</a>`

✓

TagHelpers cancel the route-templates before they generates the links.

TagHelpers looks at the route-template and injects the name of the controller at this location, where we have controller, then injects the name of the action and respectively the parameter.

Демонстрираме го:

```
app.UseMvc(rb => //app.usemvcwithdefaultrouting(); error after .net 3.0
{
    rb.MapRoute("default", "pragim/{controller=home}/{action=index}/{id?}");
});

<a asp-controller="home" asp-action="details" asp-route-id="@employee.Id"
class="btn btn-primary">|Helper</a>
<a href="/home/details/@employee.Id" class="btn btn-primary">View</a>

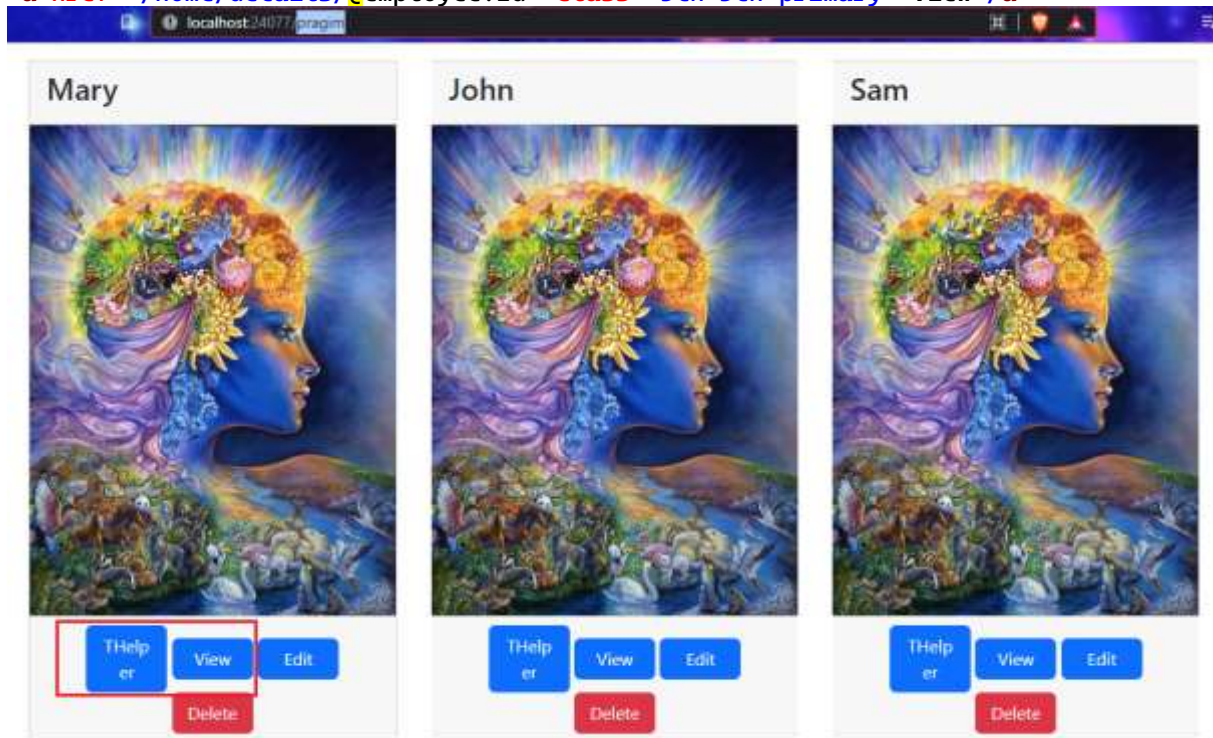
<a asp-controller="home" asp-action="details" asp-route-id="@employee.Id"
```



```

class="btn btn-primary">THelper</a>
<a href="/home/details/@employee.Id" class="btn btn-primary">View</a>

```

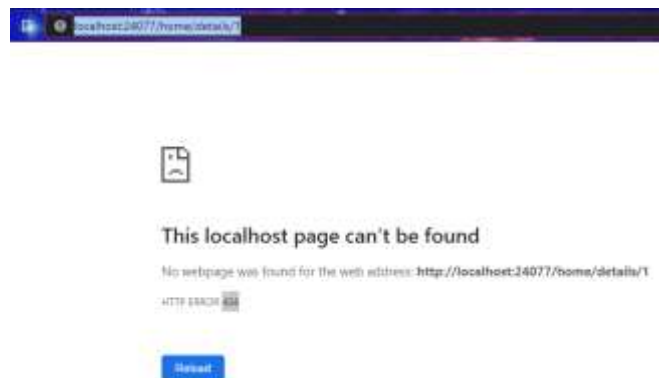


```

<a class="btn btn-primary" href="/pragim/home/details/1">THelper</a>
<a href="/home/details/1" class="btn btn-primary">View</a>

```

THelper – работи, докато другият линк не.



TagHelpers generate links based on the application routes templates. If we later change the route-templates, the links generated by tag-helpers will automatically reflect those changes and the generated links just work.

TagHelpers can generate forms. When this form is post back to the server, this values will be automatically handled and the associated validation messages will be displayed.

37 – ImageTagHelper

Повечето браузъри изтеглят картинката веднъж и после я запазват и ползват наготово от кеш-паметта cache при сърфирането. В повечето случаи не е проблем, т.к. картинките не се сменят често. В този случай, само Refresh Ctrl+R не е достатъчен.

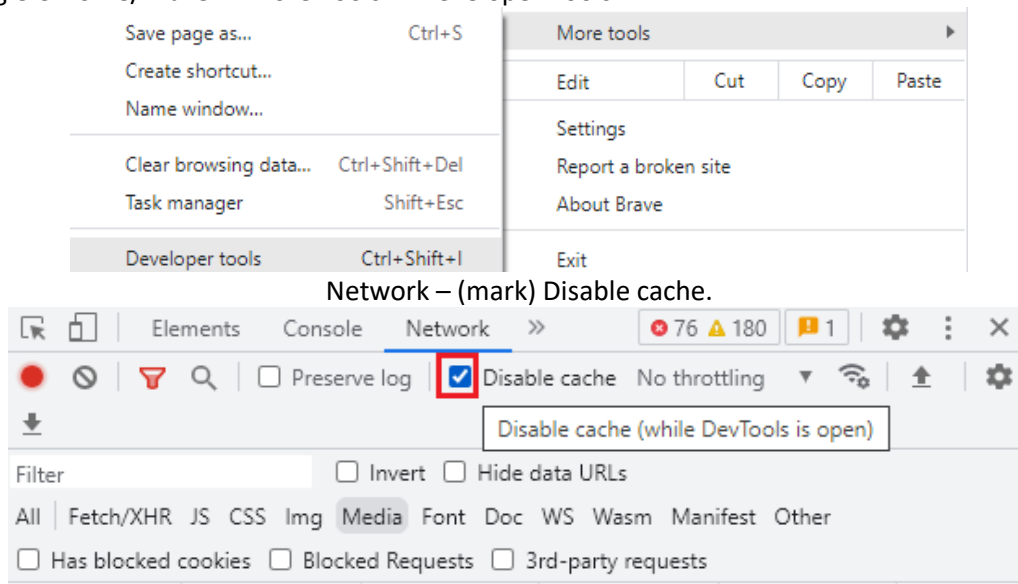


сменяме картинката

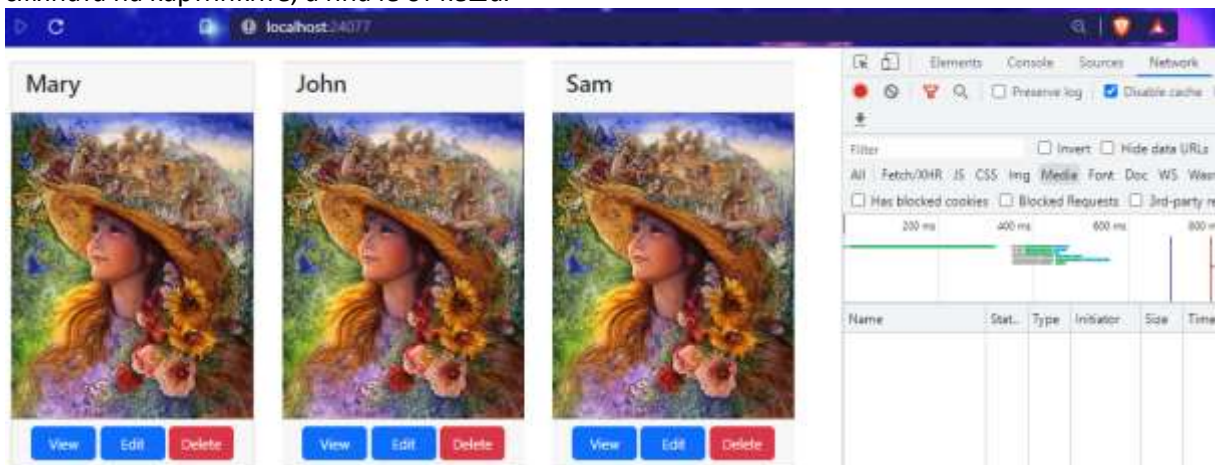
Промяната не се отразява от браузъра дори след Refresh или рестартиране на VS.

Начини:

1) Google Chrome/Brave -> More Tools -> Developer Tools F12



Така всеки път, при всяко посещение/Refresh на страницата, картинките се изтеглят от сървъра, но за производителност това не е добре. По-добре би било изтеглянето да става само при смяната на картинките, а иначе от кеша.



2) TagHelper appendix:

```

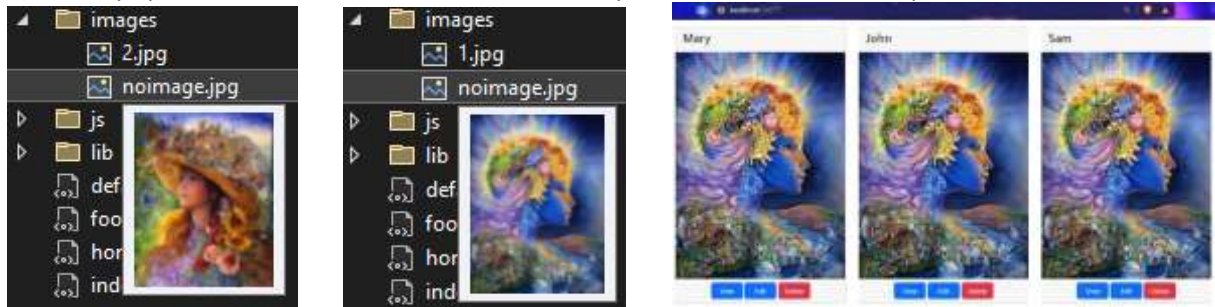
```

``
 със стойност true добавя уникален апендикс към адреса на изтегленото изображение от сървъра и го използва от кеша, докато не се смени изображението. Тогава сменя апендикса. Така няма нужда от настройката Disable cache във всеки браузър на всеки потребител.
 Refresh, ViewSource:

``

Дори когато името на картинката е същото, но съдържанието (файла) ѝ се смени, рекалкулира нов уникален стринг за брауъра и показва промените. Ако картинката е същата, уникалният апендикс се запазва след Refresh.

Демонстрираме смяната на имената, Refresh – промените веднага са отразени:



Хеша е сменен (от последни символи BE->08):

``

Ако директно ViewSource се Refresh многократно, cache-value остава непроменен, докато не се смени картинката.

ASP.NET Core Image tag helper

``

➤ Image Tag Helper enhances the `<img>` tag to provide cache-busting behaviour for static image files

➤ Based on the content of the image, a unique hash value is calculated and appended to image URL

``

➤ Each time the image on the server changes a new hash value is calculated and cached

38 – Environment TagHelper

1) Искане на локалната машина (our own server) при * Development -> bootstrap.css

2) В останалите случаи при * Staging

* Production -> bootstrap.min.css

* UAT

и ако CDN (Content Delivery Network) is down -> bootstrap.min.css


```

1 <html>
2 <head>
3   <meta name="viewport" content="width=device-width" />
4   <link href="~/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
5   <link href="~/css/site.css" rel="stylesheet" />
6   <title>@ViewBag.Title</title>
7 </head>

```

Коментарите в C# се поставят между @* *@

```

3 <html>
4 <head>
5   <meta name="viewport" content="width=device-width" />
6   <link href="~/lib/bootstrap/css/bootstrap.rtl.min.css" rel="stylesheet" />
7   <link href="~/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
8   <link href="~/css/site.css" rel="stylesheet" />
9   <title>@ViewBag.Title</title>
10 </head>
11 <body>

```

Properties node:

```

10 "profiles": {
11   "IIS Express": {
12     "commandName": "IISExpress",
13     "launchBrowser": true,
14     "environmentVariables": {
15       "ASPNETCORE_ENVIRONMENT": "Development"
16     },
17     "MyKey": "Value of MyKey from launchsett
18   },

```

Staging/Production in OS.

ASP.NET Core Environment Tag Helper

- The application environment name is set using **ASPNETCORE_ENVIRONMENT** variable

```

<environment include="Development">
  <link rel="stylesheet"
    href="~/lib/bootstrap/css/bootstrap.css" />
</environment>

```

- Environment tag helper supports rendering different content depending on the application environment

For the development environment render this content that is enclosed within the <environment> tag

```

<environment include="Development">
  <link href="~/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
</environment>

```

```

<environment include="Development">
  <link href="~/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
</environment>

```

include and exclude attributes

I

```
<environment include="Staging,Production">
  <link rel="stylesheet"
    href="https://CDN_URL/css/bootstrap.min.css"
    integrity="sha384-Integrity_Hash.....">
</environment>
```

or

except

II

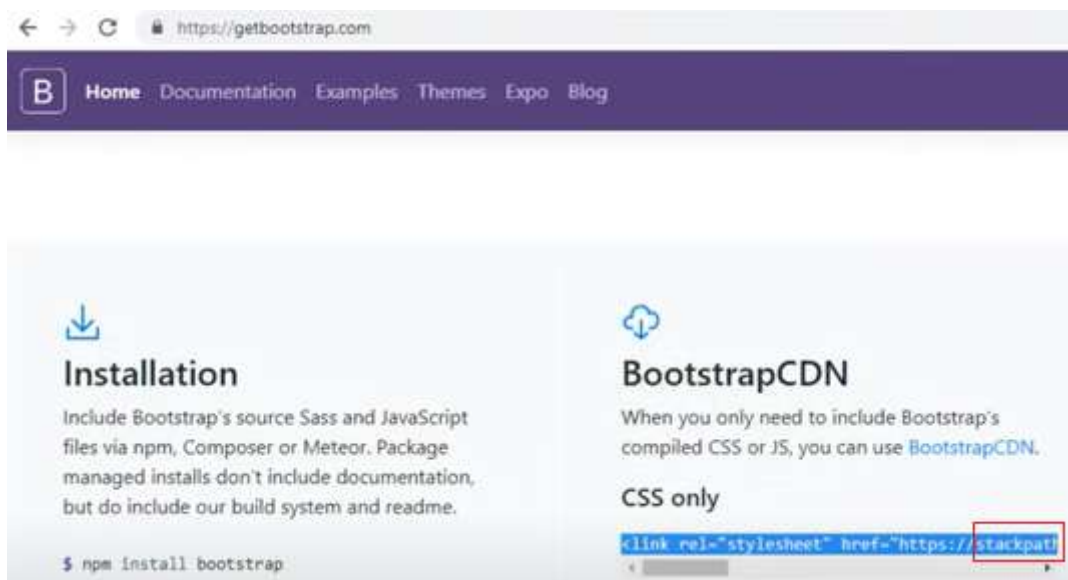
```
<environment exclude="Development">
  <link rel="stylesheet"
    href="https://CDN_URL/css/bootstrap.min.css"
    integrity="sha384-Integrity_Hash.....">
</environment>
```

The value of **"include"** and **"exclude"** attribute can be either a single hosting environment name or a comma-separated list of hosting environment names

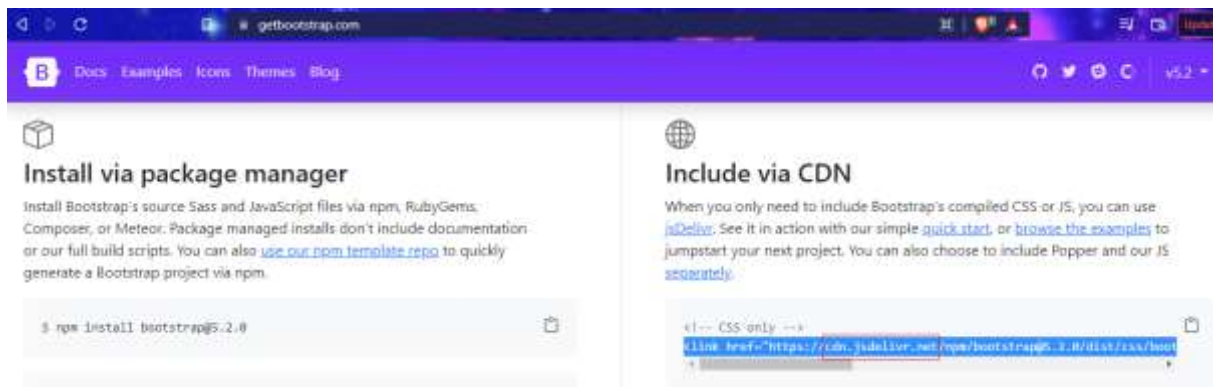
Include - For Staging or Production use minified css bootstrap file from CDN URL link (изброяваме, разделени със запетайки).

Exclude - „с изключение на“ е удобен, когато впоследствие се наложи да добавим нов environment like UAT, няма да се налага да правим никаква промяна. При include ще трябва да добавим новата UAT разделена със запетая от останалите.

<https://getbootstrap.com/>

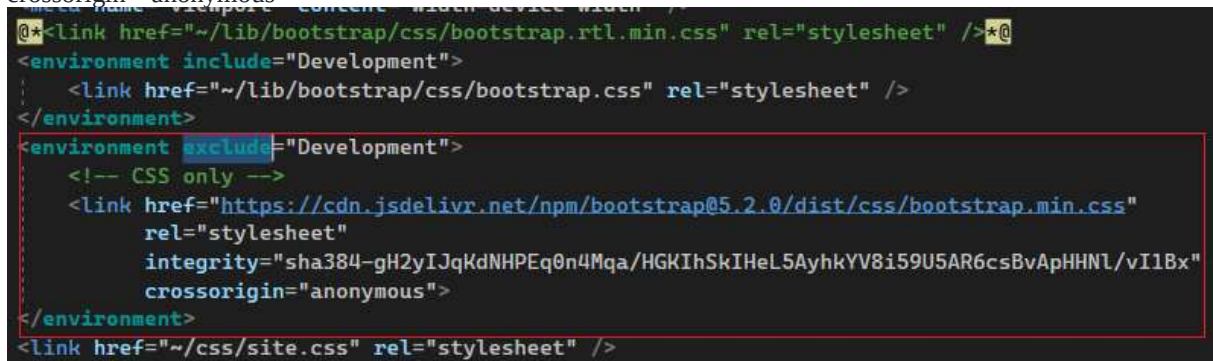


Забелязвам, че в новата версия линкът е различен:



<!-- CSS only -->

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-gH2yIJqKdNHPEq0n4Mqa/HGKIhSkIHeL5AyhkYV8i59U5AR6csBvApHHNL/vi1Bx" crossorigin="anonymous">



<environment exclude="Development">

<!-- CSS only -->

<link

href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/css/bootstrap.min.css"

rel="stylesheet"

integrity="sha384-

gH2yIJqKdNHPEq0n4Mqa/HGKIhSkIHeL5AyhkYV8i59U5AR6csBvApHHNL/vi1Bx"

crossorigin="anonymous">

</environment>



<meta name="viewport" content="width=device-width" />

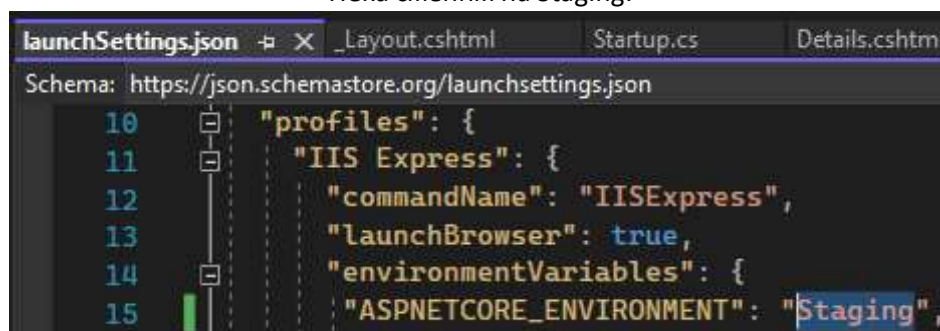
<link href="/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />

our own server/local machine

<link href="/css/site.css" rel="stylesheet" />

<title>Employee List</title>

Нека сменим на Staging:



Веднага се зарежда bootstrap.min.css



```
<meta name="viewport" content="width=device-width" />

<!-- CSS only -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/css/bootstrap.min.css"
      rel="stylesheet"
      integrity="sha384-gH2yIjQkdNHPEq0n4Mqa/HGKIhSkIHeL5AyhkYV8i59U5AR6csBvApHHNl/vI1Bx"
      crossorigin="anonymous">

<link href="/css/site.css" rel="stylesheet" />
```

integrity – SubResource integrity (SRI) check - проверка за сигурност дали файлът е изтегнен от сървъра или има злонамерено вмешателство.

SubResource Integrity (SRI)

```
<environment exclude="Development">
  <link rel="stylesheet"
        href="https://CDN URL/css/bootstrap.min.css"
        integrity="sha384-Integrity_Hash.....">
</environment>
```

- The **"integrity"** attribute on the **<link>** element is used for Subresource Integrity check
- SRI is a security feature that allows a browser to check if the file being retrieved **has been maliciously altered**

Integrity съдържа хеш стойност. Когато браузърът сваля .min. файла от линка на CDN, преизчислява хеш стойността и я сравнява със стойността, която ние сме задали. Ако съвпадат, знаем, че изтегления файл не е бил променен. Ако не съвпадат, браузърът ще блокира изтеглянето.

При мен има още един нов атрибут:

```
integrity="sha384-gH2yIJqKdNHPEq0n4Mqa/HGKIhSkIHeL5AyhkYV8i59U5AR6csBvApHHNl/vI1Bx"
crossorigin="anonymous">
```

CORS is **Cross Origin Resource Sharing**. It is a W3C standard that allows a server to make cross-domain calls from the specified domains, while rejecting others. By default, due to browser security, it prevents a web page from making one domain Ajax request to another domain.

The **`crossorigin`** attribute, valid on the `<audio>`, `<img>`, `<link>`, `<script>`, and `<video>` elements, provides support for [CORS](#), defining how the element handles cross-origin requests, thereby enabling the configuration of the CORS requests for the element's fetched data. Depending on the element, the attribute can be a CORS settings attribute.

The **`crossorigin`** content attribute on media elements is a CORS settings attribute.

These attributes are enumerated, and have the following possible values:

`anonymous`

Request uses CORS headers and credentials flag is set to `'same-origin'`. There is no exchange of **user credentials** via cookies, client-side SSL certificates or HTTP authentication, unless destination is the same origin.

`use-credentials`

Request uses CORS headers, credentials flag is set to `'include'` and **user credentials** are always included.

`""`

Setting the attribute name to an empty value, like `crossorigin` or `crossorigin=""`, is the same as **`anonymous`**.

An invalid keyword and an empty string will be handled as the **`anonymous`** keyword.

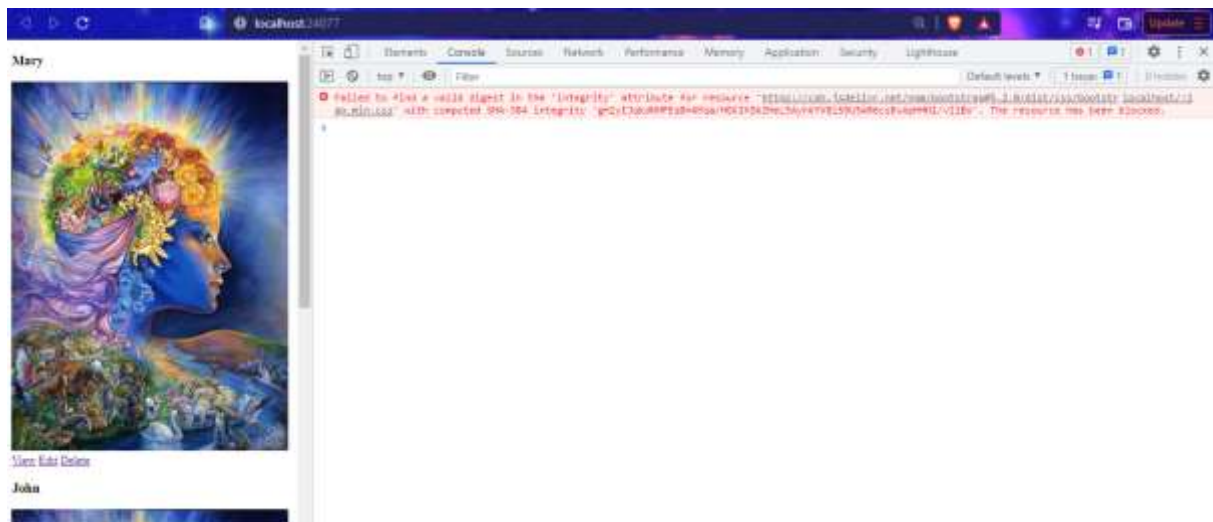
За да демонстрираме злонамерена намера, ще променим последната стойност на `integrity`:

```
integrity="sha384-gH2yIJqKdNHPEq0n4Mqa/HGKIhSkIHeL5AyhkYV8i59U5AR6csBvAphHNL/vI1Bx"
```

`_wrong_hash`:

```
integrity="sha384-gH2yIJqKdNHPEq0n4Mqa/HGKIhSkIHeL5AyhkYV8i59U5_wrong_hash"
```

Refresh:



Bootstrap стиловете вече не работят, защото изтеглянето им от CDN сървъра е блокирано.

If the CDN:

- * is down
- * the app is not able to reach the CDN
- * someone has maliciously altered the bootstrap file

the browser will fail to download the bootstrap file from CDN and the app will not work.

So we want in this case the app to go back to our own server and load the bootstrap.min from our server like the environment is Staging, Production, UAT....

Т.е. ако проверката между линк CDN-integrity не съвпада, приложението със стиловете не се зарежда в браузъра правилно и искаме да се върне и изтегли .min от нашия собствен сървър, както би направило, ако environment не е development.

CDN and Fallback Source

```
<environment exclude="Development">
  <link rel="stylesheet"
    href="https://CDN_URL/css/bootstrap.min.css"
    integrity="sha384-Integrity_Hash...."
    our server asp-fallback-href="~/lib/bootstrap/css/bootstrap.min.css"
    asp-fallback-test-class="sr-only" sr-server
    asp-fallback-test-property="position"
    asp-fallback-test-value="absolute"
    asp-suppress-fallback-integrity="true">turn off checking
</environment> checking we trust in our own server
```

➤ Use **asp-fallback-href** attribute to specify a fallback source

➤ Use **asp-suppress-fallback-integrity** attribute to turn off integrity check for files downloaded from fallback source

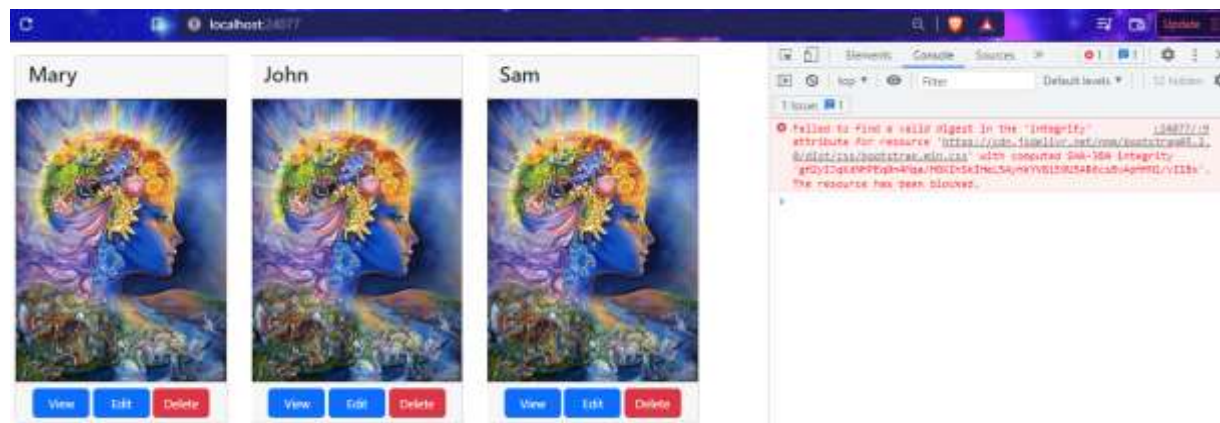
3 - test – helpers – тестват дали браузърът успешно е изтеглил bootstrap от CDN като инжектират custom JavaScript в браузъра, който проверява последователно за тези {–class–property–value} и ако те съществуват, т.е. създадени са, изтеглянето на bootstrap от CDN е успешно. Ако те не съществуват, т.е. браузърът не е изтеглил успешно bootstrap от CDN, изтегля .min от нашия

собствен сървър, при което по подразбиране проверката е вярна, integrity = "true", защото няма вероятност от злонамерена намеса в нашия сървър. Нуждаем се от integrity проверка само за изтегляне от външния сървър на 3-и лица, какъвто е CDN в случая. Ние имаме доверие на файловете, които се изтеглят от нашия сървър.

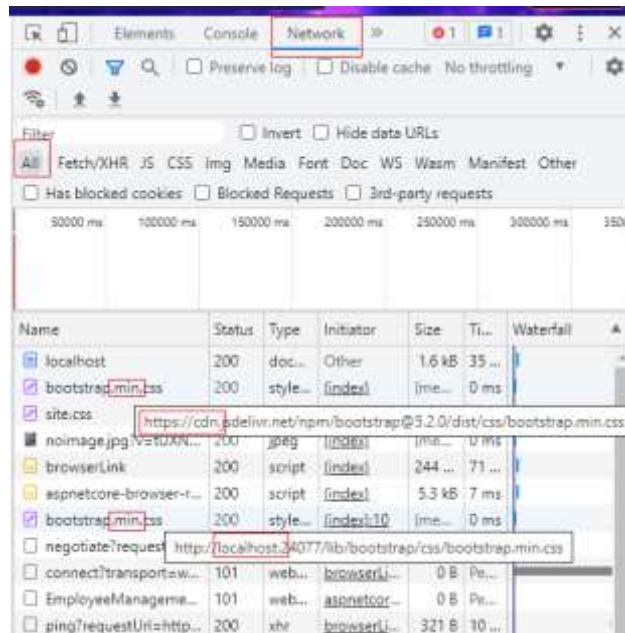
integrity = "true" – проверката е вярна за нашия сървър,
= "false" – или без integrity – (по подразбиране) = провери.

```
<environment include="Development">
  <link href="/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
</environment>
<environment exclude="Development">
  <!-- CSS only -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/css/bootstrap.min.css"
    rel="stylesheet"
    integrity="sha384-gH2yIJqKdNHPEq0n4MqA/HGKIhSkIHeL5AyhkYV8i59U5_wrong_hash"
    crossorigin="anonymous"
    asp-fallback-href="/lib/bootstrap/css/bootstrap.min.css"
    asp-fallback-test-class="sr-only"
    asp-fallback-test-property="position"
    asp-fallback-test-value="absolute"
    asp-suppress-fallback-integrity="true">
</environment>
```

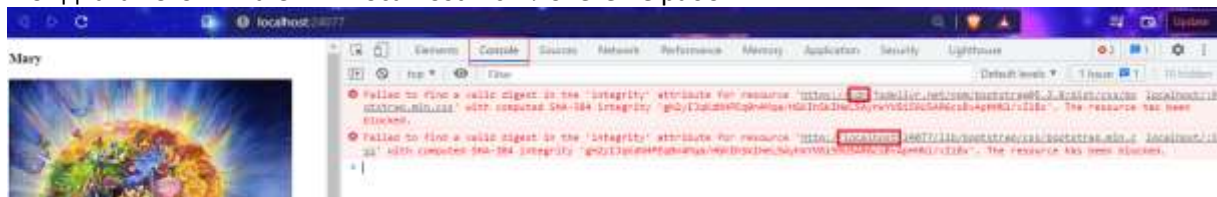
```
<environment include="Development">
  <link href="/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
</environment>
<environment exclude="Development">
  <!-- CSS only -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/css/bootstrap.min.css"
rel="stylesheet"
integrity="sha384-
gH2yIJqKdNHPEq0n4MqA/HGKIhSkIHeL5AyhkYV8i59U5_wrong_hash"
crossorigin="anonymous"
asp-fallback-href="/lib/bootstrap/css/bootstrap.min.css"
asp-fallback-test-class="sr-only"
asp-fallback-test-property="position"
asp-fallback-test-value="absolute"
asp-suppress-fallback-integrity="true">
</environment>
```



Изтеглянето е блокирано, защото хеш стойността не съвпада, затова е пренасочено към изтегляне на .min от нашия сървър и стиловете работят.



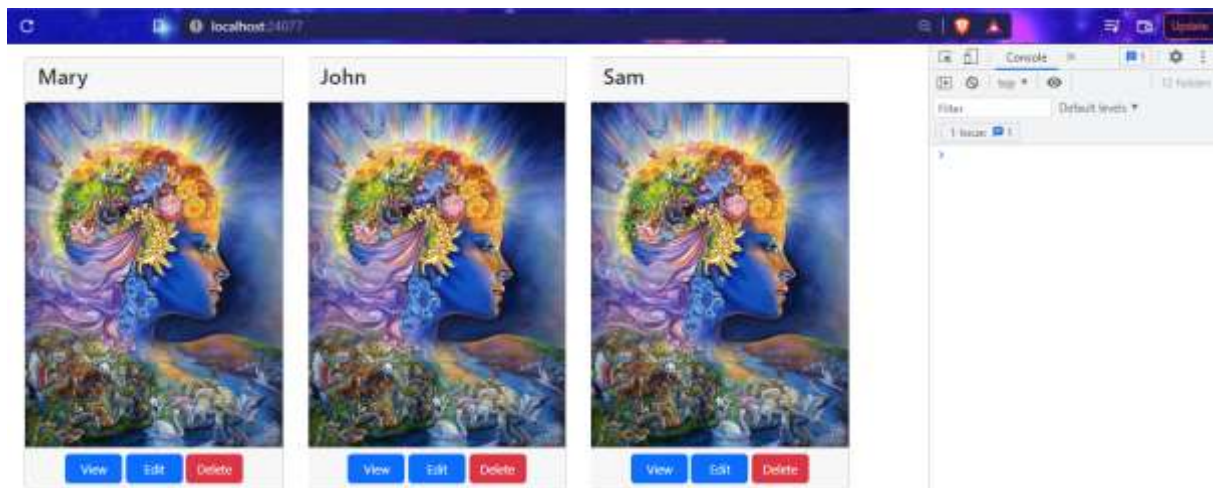
Ако зададем `asp-suppress-fallback-integrity="false">` ще извършва проверка дори при изтегляне на файла от нашия сървър и понеже сме задали `_wrong_hash`, изтеглянето се проваля и от двата източника CDN и localhost и стиловете не работят.



Коригираме

ВСИЧКО:

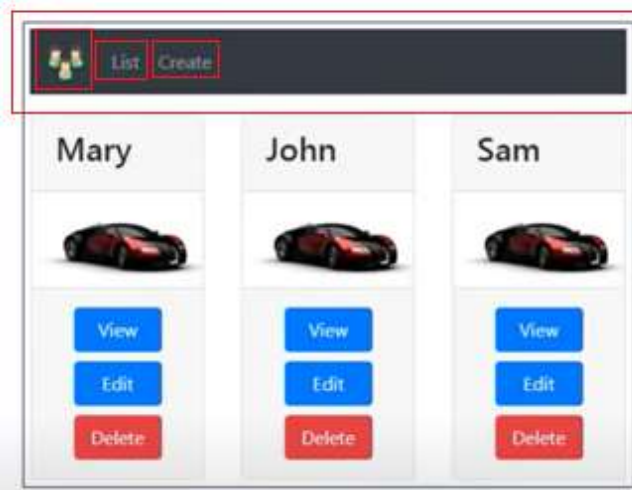
```
<environment include="Development">
  <link href="~/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
</environment>
<environment exclude="Development">
  <!-- CSS only -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/css/bootstrap.min.css"
rel="stylesheet"
integrity="sha384-
gH2yIJqKdNHPEq0n4MqahHGKIhSkiHeL5AyhkYV8i59U5AR6csBvApHHNL/vI1Bx"
crossorigin="anonymous"
asp-fallback-href="~/lib/bootstrap/css/bootstrap.min.css"
asp-fallback-test-class="sr-only"
asp-fallback-test-property="position"
asp-fallback-test-value="absolute"
asp-suppress-fallback-integrity="true">
</environment>
```



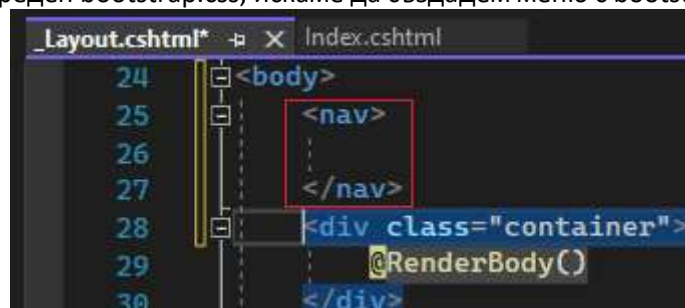
Работи добре.

launchsettings.json -> "ASPNETCORE_ENVIRONMENT": "Development",

39 – Bootstrap Navigation Menu On a Large Screen Device



Вече имаме зареден bootstrap.css, искаме да създадем меню с bootstrap-navbar-class:



Първо <nav>-елемент, после в него влагаме bootstrap class.

<nav> - navigation menu

 - unordered list

 - list item

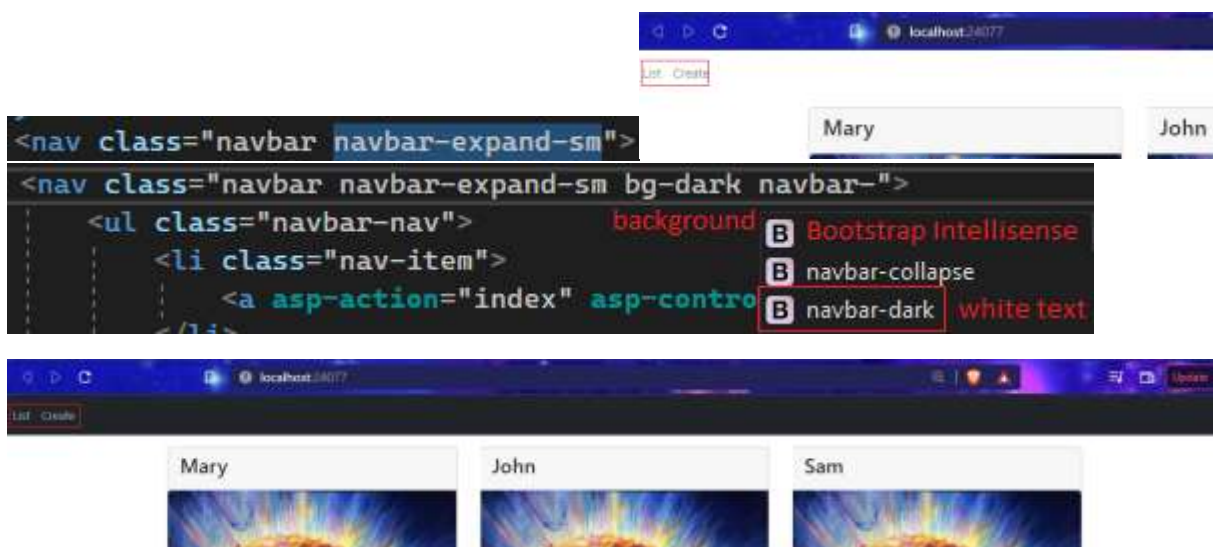
<a> - anchor

```
Layout.cshtml* X Index.cshtml
<body>
  <nav class="navbar">
    <ul class="navbar-nav">
      <li class="nav-item">
        <a asp-action="index" asp-controller="home" class="nav-link">List</a>
      </li>
      <li class="nav-item">
        <a asp-action="create" asp-controller="home" class="nav-link">Create</a>
      </li>
    </ul>
  </nav>
  <div class="container">
    @RenderBody()
  </div>
</body>
```

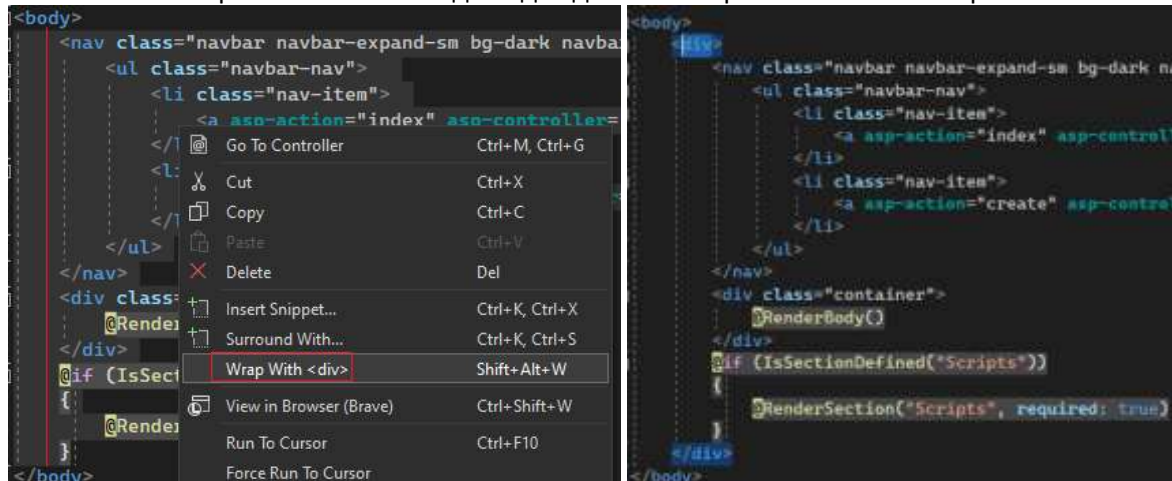
```
<nav class="navbar">
  <ul class="navbar-nav">
    <li class="nav-item">
      <a asp-action="index" asp-controller="home" class="nav-link">List</a>
    </li>
    <li class="nav-item">
      <a asp-action="create" asp-controller="home" class="nav-link">Create</a>
    </li>
  </ul>
</nav>
```



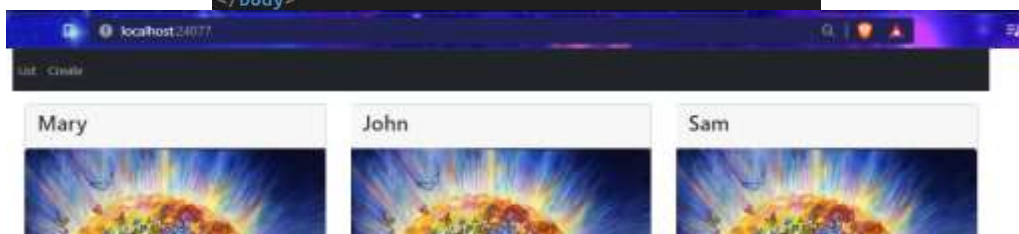
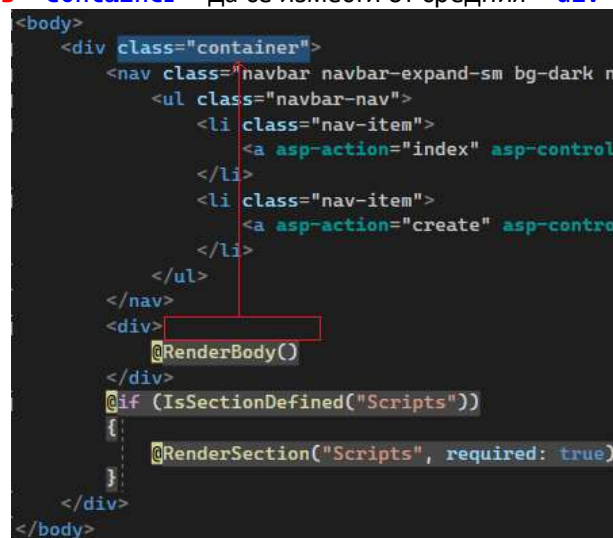
горизонтально меню: `<nav class="navbar navbar-expand-sm">`



Ширината на менюто да бъде еднаква с ширината на контейнера.



след което `class="container"` да се измести от средния `<div>` в новия нагоре.



Искаме и лого в навигационното меню:

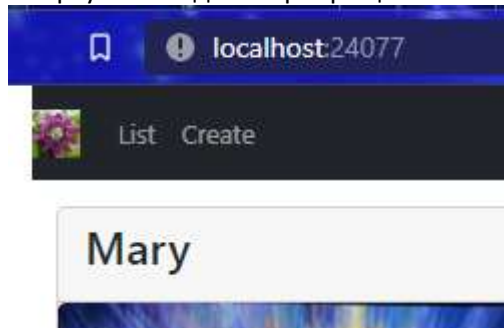
isk (D:) > Exercises > EmployeeManagement > wwwroot > images



Избирам по-малката големина .JPG

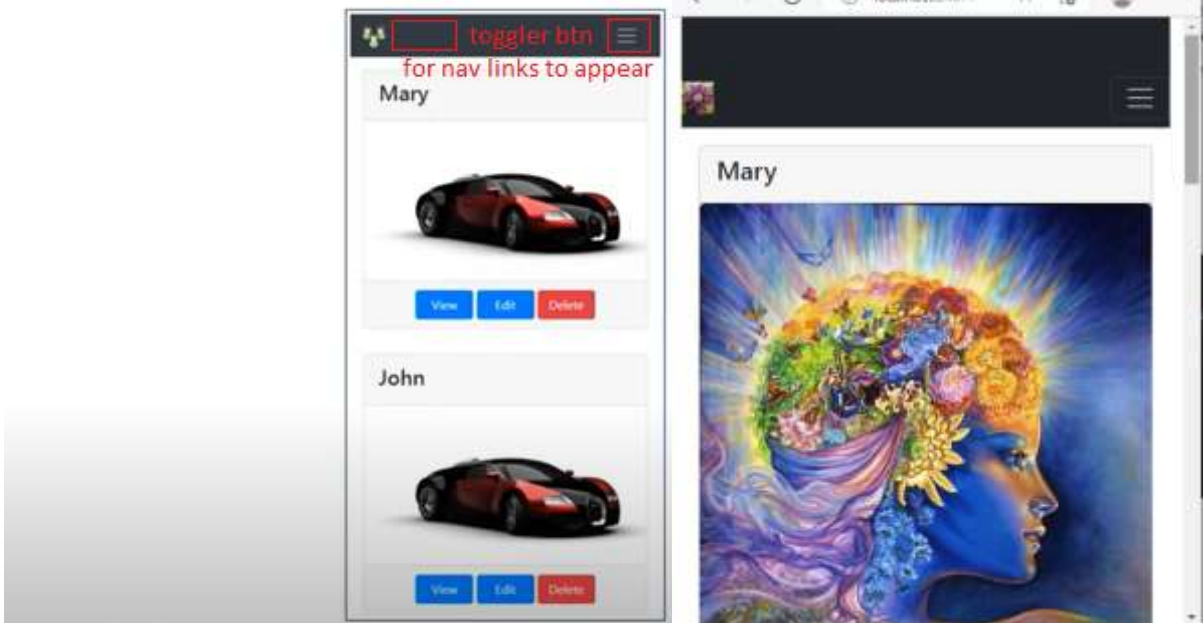

```
<nav class="navbar navbar-expand-sm bg-dark navbar-dark">
  <a class="navbar-brand" asp-action="index" asp-controller="home">
    
  </a>
</nav>
```

Искаме при натискане върху логото да ни препраща линк към началната страница.



```
<div class="container">
  <nav class="navbar navbar-expand-sm bg-dark navbar-dark">
    <a class="navbar-brand" asp-action="index" asp-controller="home">
      
    </a>
    <ul class="navbar-nav">
      <li class="nav-item">
        <a asp-action="index" asp-controller="home" class="nav-
link">List</a>
      </li>
      <li class="nav-item">
        <a asp-action="create" asp-controller="home" class="nav-
link">Create</a>
      </li>
    </ul>
  </nav>
  <div>
    @RenderBody()
  </div>
  @if (IsSectionDefined("Scripts"))
  {
    @RenderSection("Scripts", required: true)
  }
</div>
```


`</div>` On a Small Screen Device



При мен работи само в Edge и пак стилирането не е много добре.

= id selector.

```
<nav class="navbar navbar-expand-sm bg-dark navbar-dark">
  <a class="navbar-brand" asp-action="index" asp-controller="home" >
    
  </a>
  <button type="button" class="navbar-toggler" data-toggle="collapse" data-target="#collapsibleNavbar">
    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse" id="collapsibleNavbar">
    <ul class="navbar-nav">
      <li class="nav-item">
        <a asp-action="index" asp-controller="home" class="nav-link">List</a>
      </li>
      <li class="nav-item">
        <a asp-action="create" asp-controller="home" class="nav-link">Create</a>
      </li>
    </ul>
  </div>
</nav>
```

Тук се получи един много важен момент, който забелязвам и други пъти в рамките на VisualStudio – дори да има верен код, докато не се използва Intellisense, много пъти остава неразпознат и не работи, затова трябва да се обръща внимание дали цветовете са разпознати.

```
Details.cshtml _Layout.cshtml _ViewStart.cshtml _ViewStart.cshtml Test.cshtml
<script src="~/lib/bootstrap/js/bootstrap.esm.min.js"></script>
</environment>
<link href="~/css/site.css" rel="stylesheet" />
<title>@ViewBag.Title</title>
</head>
<body>
  <div class="container">
    <nav class="navbar navbar-expand-sm bg-dark navbar-dark">
      <a class="navbar-brand" asp-action="index" asp-controller="home" ></a>
      <a class="navbar-brand" asp-action="index" asp-controller="home" >
        
      </a>
```

```
</body>
<div class="container">
```

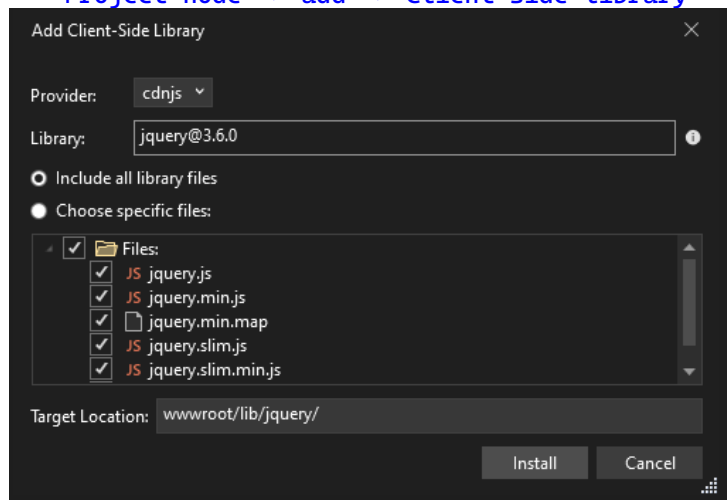
```

<nav class="navbar navbar-expand-sm bg-dark navbar-dark">
  <a class="navbar-brand" asp-action="index" asp-controller="home" >
    
  </a>
  <button type="button" class="navbar-toggler" data-toggle="collapse"
data-target="#collapsibleNavbar">
    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse" id="collapsibleNavbar">
    <ul class="navbar-nav">
      <li class="nav-item">
        <a asp-action="index" asp-controller="home" class="nav-
link">List</a>
      </li>
      <li class="nav-item">
        <a asp-action="create" asp-controller="home" class="nav-
link">Create</a>
      </li>
    </ul>
  </div>
</nav>
<div>
  @RenderBody()
</div>
@if (IsSectionDefined("Scripts"))
{
  @RenderSection("Scripts", required: true)
}
</div>
</body>

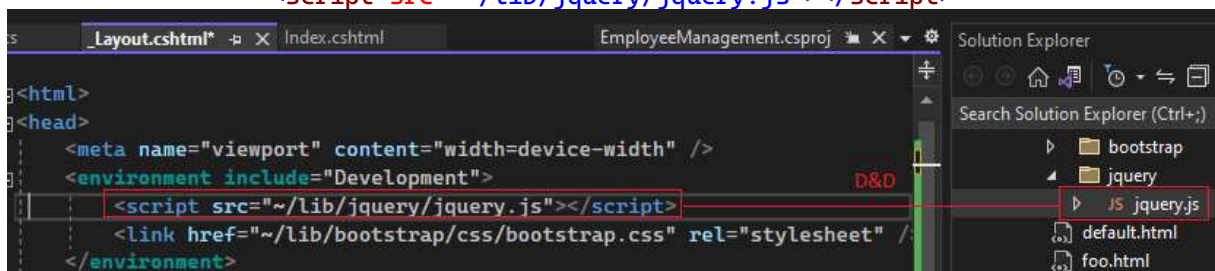
```

Toggle btn не работи, защото му е необходима JQuery функционалност.

Project node -> add -> Client side library



```
<script src="~/lib/jquery/jquery.js"></script>
```



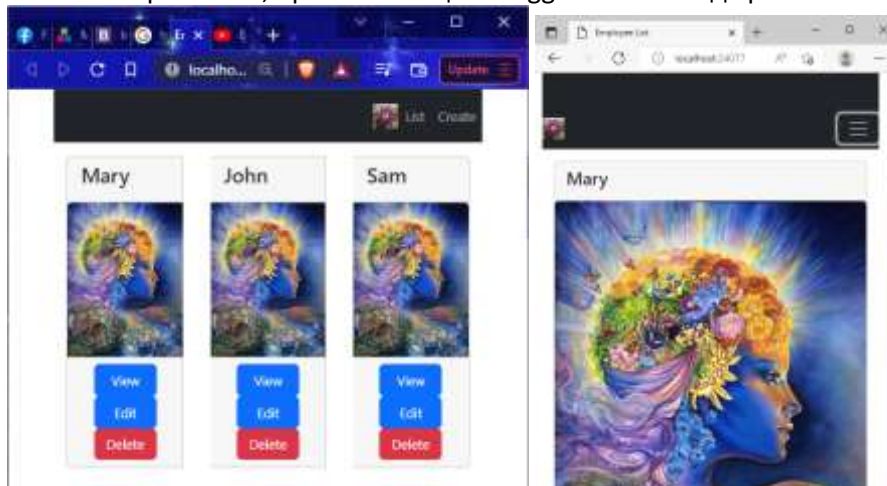
Трябва ни също и Bootstrap JavaScript.

```
<meta name="viewport" content="width=device-width" />
<environment include="Development">
  <link href="~/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
  <script src="~/lib/jquery/jquery.js"></script>
  <script src="~/lib/bootstrap/js/bootstrap.js"></script>
</environment>
<environment exclude="Development">
```

Последователността е много важна: JQuery трябва да бъде зареден преди bootstrap.js!

```
<link href="~/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
<script src="~/lib/jquery/jquery.js"></script>
<script src="~/lib/bootstrap/js/bootstrap.js"></script>
```

Ако се размести, превключващият Toggler btn няма да работи.



При мен в Brave така и не се показва Toggler, но бутоните са подредени и искаме разстояния между тях. В Edge може да бъде селектиран Toggler, но не се разлива.

Сору->Paste за двата вида Environment, съответно, единият е min.

```
<script src="~/lib/jquery/jquery.slim.min.js"></script>
<script src="~/lib/bootstrap/js/bootstrap.esm.min.js"></script>
```

```
<environment include="Development">
  <link href="~/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
  <script src="~/lib/jquery/jquery.js"></script>
  <script src="~/lib/bootstrap/js/bootstrap.js"></script>
</environment>
<environment exclude="Development">
  <!-- CSS only -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/css/bootstrap.min.css"
    rel="stylesheet"
    integrity="sha384-gH2yIJqKdNHPEq0n4MqA/HGKIhSkIHeL5AyhkyV8i59U5AR6csBvApHHNL/vI1Bx"
    crossorigin="anonymous"
    asp-fallback-href="~/lib/bootstrap/css/bootstrap.min.css"
    asp-fallback-test-class="sr-only"
    asp-fallback-test-property="position"
    asp-fallback-test-value="absolute"
    asp-suppress-fallback-integrity="true">
  <script src="~/lib/jquery/jquery.slim.min.js"></script>
  <script src="~/lib/bootstrap/js/bootstrap.esm.min.js"></script>
</environment>
```

```
<a asp-controller="home" asp-action="details" asp-route-id="@employee.Id"
  class="btn btn-primary m-1">View</a>
<a href="#" class="btn btn-primary m-1">Edit</a>
```



```
<a href="#" class="btn btn-danger m-1">Delete</a>
```

```
Index.cshtml X margin-1
class="btn btn-primary m-1">View</a>
<a href="#" class="btn btn-primary m-1">Edit</a>
<a href="#" class="btn btn-danger m-1">Delete</a>
```



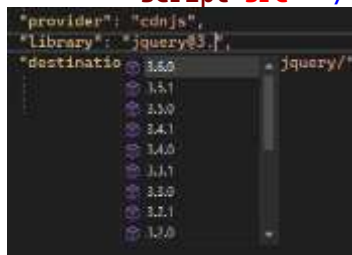
Margin m-1

Засега само List работи от менюто, следващият път ще добавим Create action in HomeController + Create View.

Brave, Mozilla, Chrome показват по еднакъв начин сайта, Edge по-различен.

При мен Toggler btn не работи дори с инсталация на по-стара версия на JQuery 3.3.1 като в туториъла и при правилна подредба:

```
<link href="~/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
<script src="~/lib/jquery/jquery.js"></script>
<script src="~/lib/bootstrap/js/bootstrap.js"></script>
```



```
<link href="~/lib/bootstrap/css/bootstrap.css" rel="stylesheet" />
<script src="~/lib/jquery/jquery.js"></script>
<script src="~/lib/bootstrap/js/bootstrap.js"></script>
```

Впоследствие откраднах от другия туториъл C# crash course автокода за toggler:

```
<body>
<div class="container-fluid">
<nav class="navbar navbar-expand-sm navbar-toggleable-sm bg-dark navbar-dark border-bottom box-shadow mb-3">
<a class="navbar-brand" asp-area="" asp-controller="home" asp-action="index">

</a>
<button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target=".navbar-collapse"
aria-controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle navigation">
<span class="navbar-toggler-icon"></span>
</button>
<div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
<ul class="navbar-nav flex-grow-1">
<li class="nav-item">
<a class="nav-link text-light" asp-area="" asp-controller="home" asp-action="index">List</a>
</li>
<li class="nav-item">
<a class="nav-link text-light" asp-area="" asp-controller="home" asp-action="create">Create</a>
</li>
</ul>
</div>
</nav>

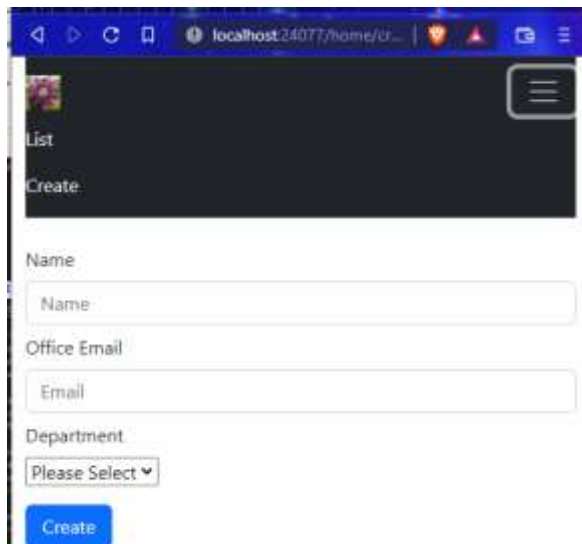
<div>
RenderBody()
</div>
```

```
<div class="container-fluid">
<nav class="navbar navbar-expand-sm navbar-toggleable-sm bg-dark navbar-dark
border-bottom box-shadow mb-3">
```

```

<a class="navbar-brand" asp-area="" asp-controller="home" asp-action="index">
    
</a>
<button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target=".navbar-collapse" aria-controls="navbarSupportedContent" aria-
expanded="false" aria-label="Toggle navigation">
    <span class="navbar-toggler-icon"></span>
</button>
<div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
    <ul class="navbar-nav flex-grow-1">
        <li class="nav-item">
<a class="nav-link text-light" asp-area="" asp-controller="home" asp-
action="index">List</a>
        </li>
        <li class="nav-item">
<a class="nav-link text-light" asp-area="" asp-controller="home" asp-
action="create">Create</a>
        </li>
    </ul>
</div>
</nav>

```

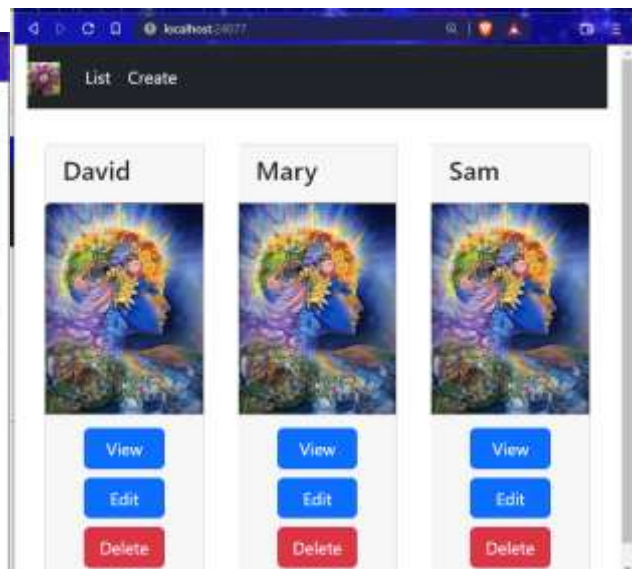


List
 Create

Name

Office Email

Department
 Please Select ▼



40 – Form TagHelpers

Form Tag Helpers in ASP.NET Core



<Form> е достатъчен за създаване на форма. Останалите са за контролни елементи.

Form Tag Helpers in ASP.NET Core

List Create

Name

Email

Department

Create

Форма с всички необходими данни, които трябва да бъдат прихванати за създаването на нов служител. Досега „Create“ не работи при клик – 404, сега ще създадем action Create in HomeController + Create View in Home Folder.

```
1 @model Employee
2
3 @f
4 ViewBag.Title = "Create Employee";
5
6 <form asp-controller="Home" asp-action="Create" method="post">
7
8 </form>
```

```
0 references
public IActionResult Create()
{
    return View();
}
```

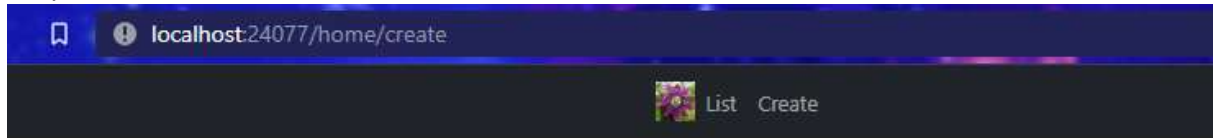
Веднага включваме първо @model = Strongly type view:

* Intellisense

* autocomplete

* compile-time error checking.

Резултат:



ViewSource: `<form method="post" action="/Home/Create">`

В този адрес се изпълнява Action method in HomeController, който извиква съответното Create View, като при попълването на формата и натискането на бутона, се изпълнява метод post, чрез който се създава нов служител и това става в същия URL. Най-интересният момент тук е, че дори от таг `<form asp-controller="Home" asp-action="Create" method="post">` да бъдат премахнати `asp-controller="Home" asp-action="Create"`, всичко това ще бъде обслужено пак в същия /home/create, защото по този начин е инициализирано в HomeController и Create View.

`<input asp-for="Name" />` С което вече имаме input елемент.



т.е. въвежда се текст, който има value = empty string, атрибути id = name = "Name", заради процес, който се нарича Model Binding.

`<label asp-for="Name"></label>` - полето се нарича Name, то автоматично ще бъде показано като текст Name и свързано с input след него. Това означава, че когато се кликне върху Name, автоматично ще се активира полето за въвеждане на текст от input след него, като курсорът се появи.



```

37 <form method="post" action="/Home/Create">
38   <label for="Name">Name</label>
39   <input type="text" id="Name" name="Name" value="" />

```

```

<div>
  <label asp-for="Name"></label>
  <input asp-for="Name" />
</div>

```

```

Create.cshtml  Details.cshtml  _Layout.cshtml  Index.cshtml  HomeController.cs
1  @model Employee
2
3  @{
4      ViewBag.Title = "Create Employee";
5  }
6  <form asp-controller="Home" asp-action="Create" method="post">
7      <div>
8          <label asp-for="Name"></label>
9          <input asp-for="Name" />
10     </div>
11     <div>
12         <label asp-for="Email"></label>
13         <input asp-for="Email" />
14     </div>
15     <div>
16         <label asp-for="Department"></label>
17         <select asp-for="Department" ></select>
18     </div>
19 </form>

```

<select> има няколко начина за въвеждане на опциите:

- * html – hard code
- * data from DB table
- * enum

Ние ще използваме Enum: Models->add-> Class-> Dept.cs, създавайки нов сложен тип данни.

```

Dept.cs  Create.cshtml  Details.cshtml  _Layout.cshtml
EmployeeManagement  Employee
namespace EmployeeManagement.Models
{
    1
    2
    3  public enum Dept
    4  {
    5      None,
    6      HR,
    7      IT,
    8      Payroll
    9  }
10
}

```

```

namespace EmployeeManagement.Models
{
    public enum Dept
    {
        None,
        HR,
        IT,
        Payroll
    }
}

```

Въвеждаме го там, където ни трябваше в **<select>**

```

<div>
  <label asp-for="Department"></label>
  <select asp-for="Department" asp-items="Html.GetEnumSelectList<Dept>()"></select>
</div>

```

System.Collections.Generic.Enumerable<Microsoft.AspNetCore.Mvc.Rendering.SelectListItem>
Microsoft.AspNetCore.Mvc.TagHelpers.SelectTagHelper.Items
A collection of Microsoft.AspNetCore.Mvc.Rendering.SelectListItem objects used to populate the <select> element with <optgroup> and <option> elements.

```

<select asp-for="Department" asp-items="Html.GetEnumSelectList<Dept>()"></select>

```

Целият код на Create:

@model Employee

@{

ViewBag.Title = "Create Employee";

}

<form asp-controller="Home" asp-action="Create" method="post">

<div>

<label asp-for="Name"></label>

<input asp-for="Name" />

</div>

<div>

<label asp-for="Email"></label>

<input asp-for="Email" />

</div>

<div>

<label asp-for="Department"></label>

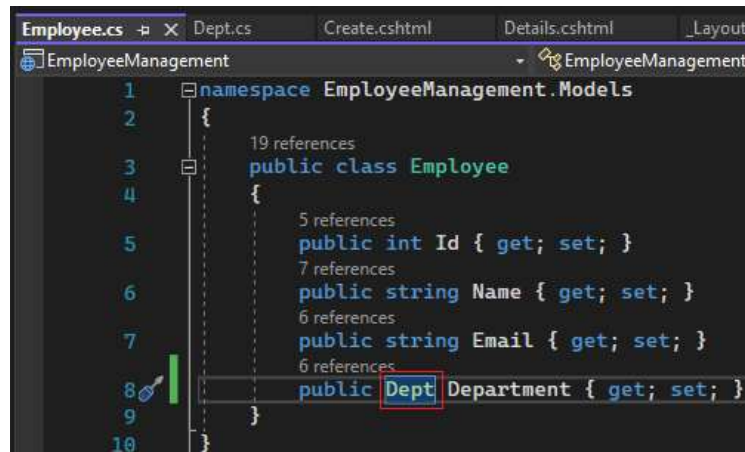
<select asp-for="Department"

asp-items="Html.GetEnumSelectList<Dept>()">

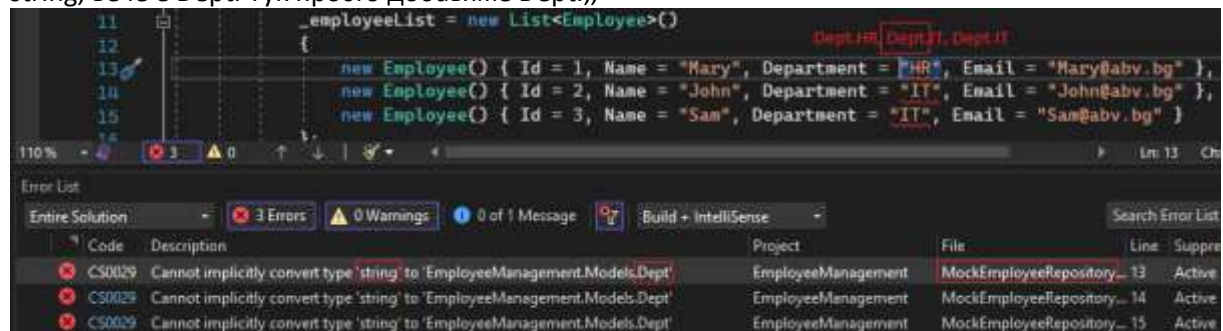
</select>

</div>

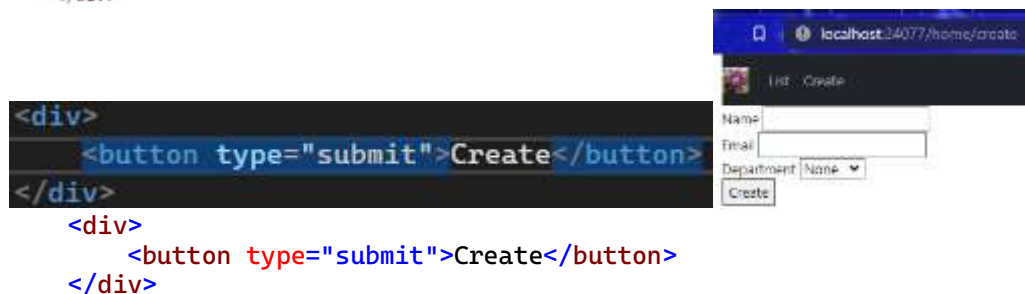
</form>



При изпълнение получаваме 3 грешки в MockEmployeeRepository, които при клик ни отвеждат в реда на кода. Причината е, че string не може да се преобразува в Dept. Преди Department беше string, вече е Dept. Тук просто добавяме Dept.,,



```
public MockEmployeeRepository() // ctor
{
    _employeeList = new List<Employee>()
    {
        new Employee() { Id = 1, Name = "Mary", Department = Dept.HR,
            Email = "Mary@abv.bg" },
        new Employee() { Id = 2, Name = "John", Department = Dept.IT,
            Email = "John@abv.bg" },
        new Employee() { Id = 3, Name = "Sam", Department = Dept.IT,
            Email = "Sam@abv.bg" }
    };
}
```



Оформяме с bootstrap:

@model Employee

@{

ViewBag.Title = "Create Employee";

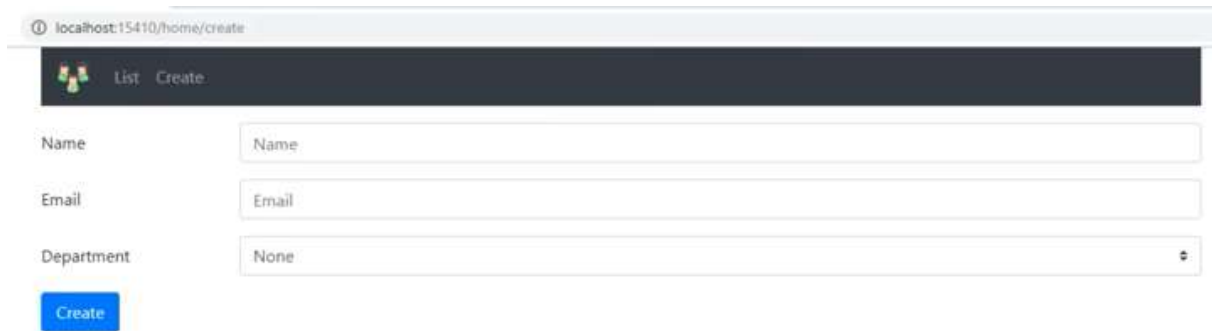
}

```

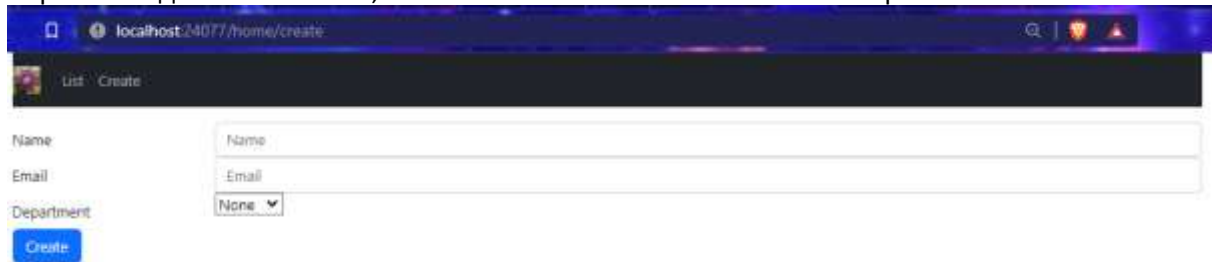
<form asp-controller="Home" asp-action="Create" method="post" class="mt-3">
    <div class="form-group row">
        <label asp-for="Name" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10">
            <input asp-for="Name" class="form-control" placeholder="Name"/>
        </div>
    </div>
    <div class="form-group row">
        <label asp-for="Email" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10">
            <input asp-for="Email" class="form-control" placeholder="Email"/>
        </div>
    </div>
    <div class="form-group row">
        <label asp-for="Department" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10">
            <select asp-for="Department" class="custom-select mr-sm-2"
                asp-items="Html.GetEnumSelectList<Dept>()">
            </select>
        </div>
    </div>
    <div class="form-group row">
        <div class="col-sm-10">
            <button type="submit" class="btn btn-primary">Create</button>
        </div>
    </div>
</form>

```

Трябва да се получи:

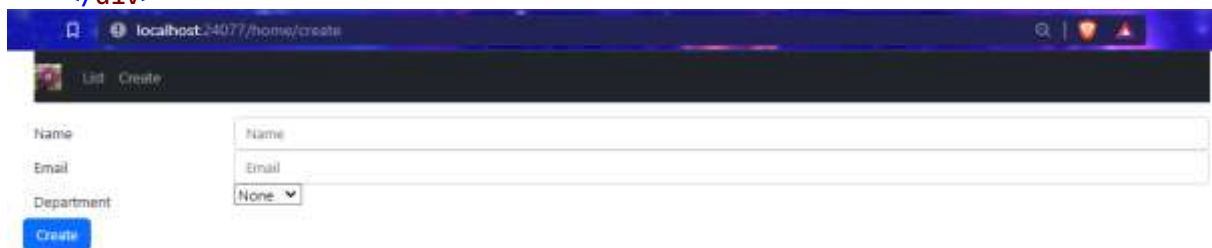


Department дава отклонение, понеже `class="custom-select"` не се разпознава.



Логото не е добре позиционирано при мен – няма отстояние, но засега го оставям така. Направи ми впечатление, че има два последователни `<div>` за бутона, а за тест, като ги обединя в един `<div class=`

```
<div class="form-group row col-sm-10">  
  <button type="submit" class="btn btn-primary">Create</button>  
</div>
```



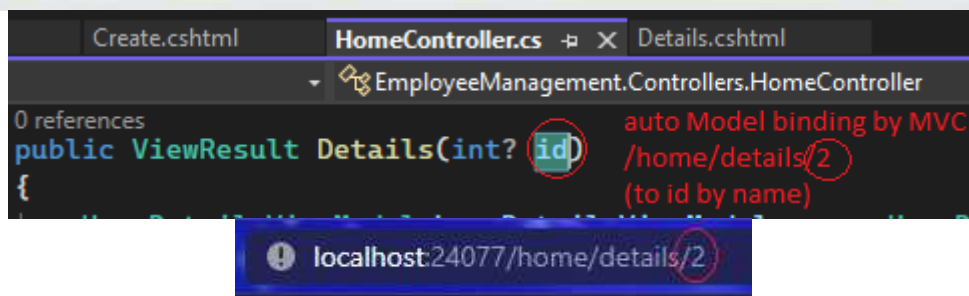
Остава да се занимава с `class="custom-select"` за Department по-късно.

41 – Model Binding

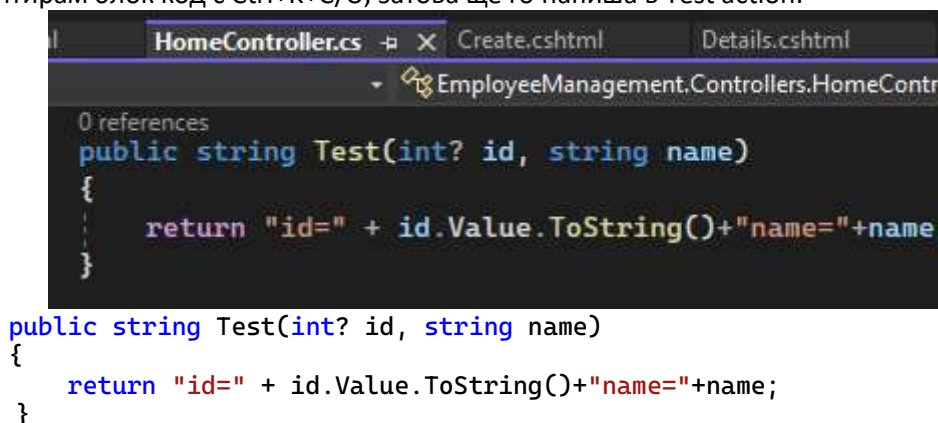
ASP.NET Core Model Binding

Model binding maps data in an HTTP request to controller action method parameters

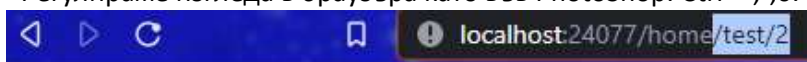
The **action parameters** may be **simple** types such as integers, strings, etc or **complex** types like Customer, Employee, Order etc



С това автоматично прихващане и разпознаване, MVC ни спестява много код. Демонстративно изменяме кода, включвайки нов параметър, но аз предпочитам да не закоментирам и разкоментирам блок код с Ctrl+K+C/U, затова ще го напиша в Test action:



Регулираме изгледа в браузъра като във PhotoShop: Ctrl++, -, 0.



id=2name= към момента name=0

В URL адреса може да добавим string query: /home/details/2?name=abv

localhost:24077/home/test/2?name=abv

id= 2 name= abv

localhost:24077/home/test/2?name1=abv

id= 2 name=

localhost:24077/home/test/2?name=abv&id=5

id= 2 name= abv последователност: прихваща първата стойност

ASP.NET Core Model Binding

To bind the request data to the controller action method parameters, model binding looks for data in the HTTP request



Премахваме стойността 2 и прихваща id=5, нищо, че е зададено след name:

localhost:24077/home/test/?name=abv&id=5

id= 5 name= abv

За да създадем нов служител, използваме action Create, който трябва да се изпълнява 2 пъти: единият за get, другият за post в рамките на 1 и същи URL адрес:

В противен случай ще отчете грешка, че има 2 Create, които неразлично изпълняват условията (Ambiguous – двусмислие)

localhost:24077/home/create

An unhandled exception occurred while processing the request.

AmbiguousActionException: Multiple actions matched. The following actions matched route data and had all constraints satisfied:

EmployeeManagement.Controllers.HomeController.Create (EmployeeManagement)

EmployeeManagement.Controllers.HomeController.Create (EmployeeManagement)

[HttpGet]

```
public IActionResult Create()
{
    return View();
}
```

[HttpPost]

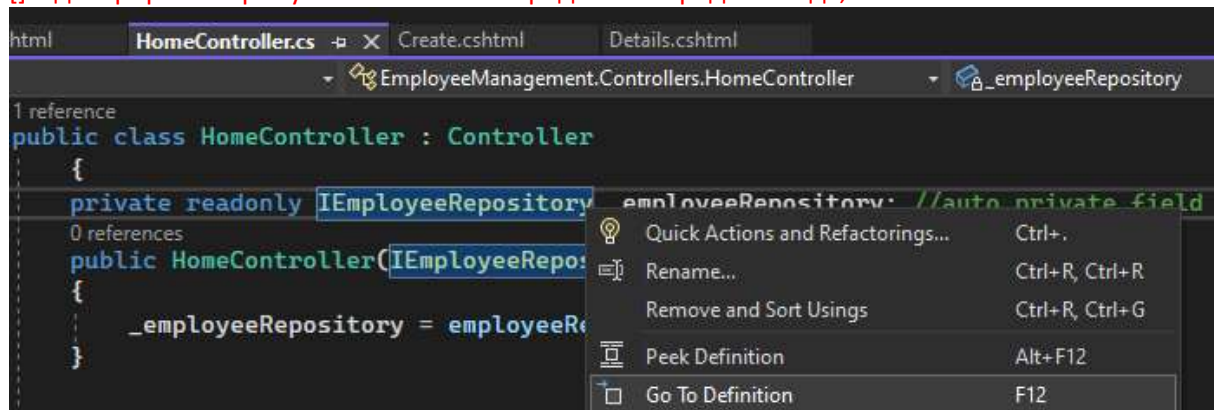
```
public IActionResult Create()
```

```

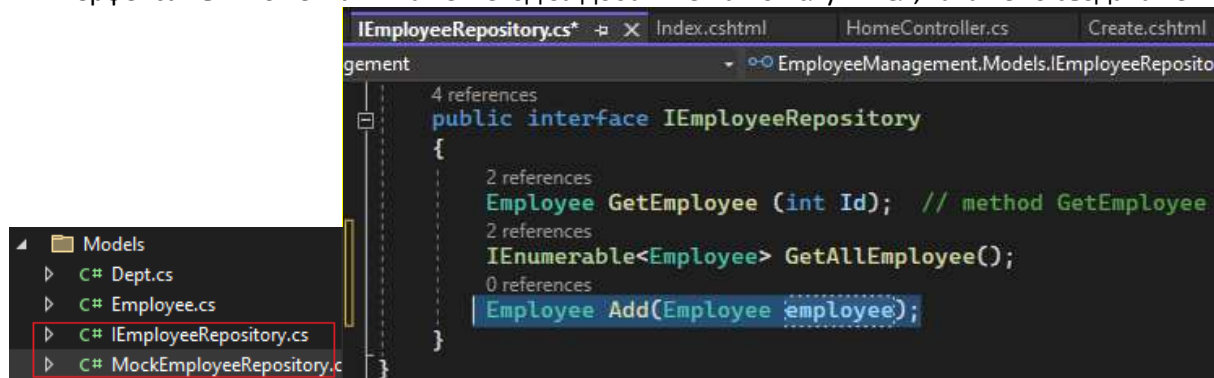
{
    return View();
}

```

[] – декорираме атрибутите винаги непосредствено преди метода, за който се отнасят.

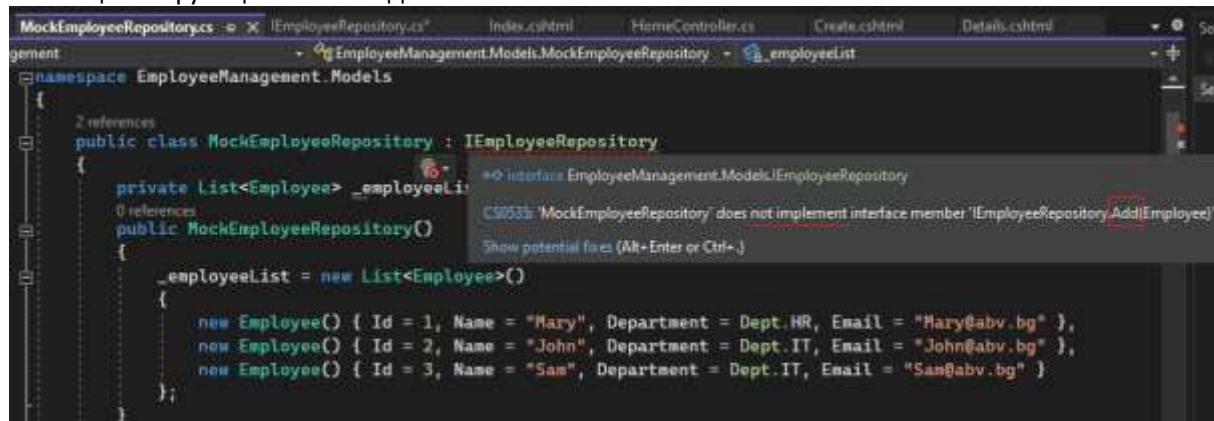


В интерфейса към момента нямаме метод за добавяне на нов служител, така че го създаваме:

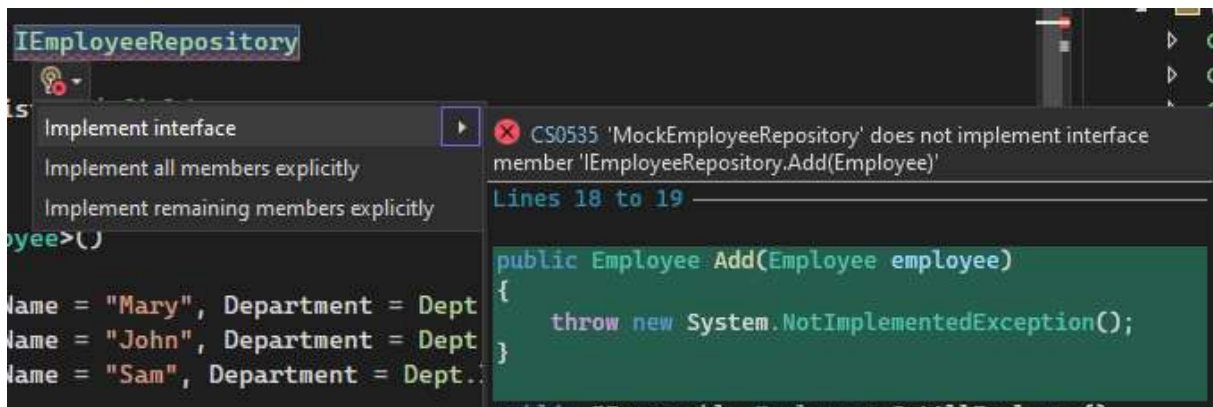


Employee Add(Employee employee);

Следва да създадем имплементация за него в MockEmployeeRepository, където се изпълнява почти целия функционален код.



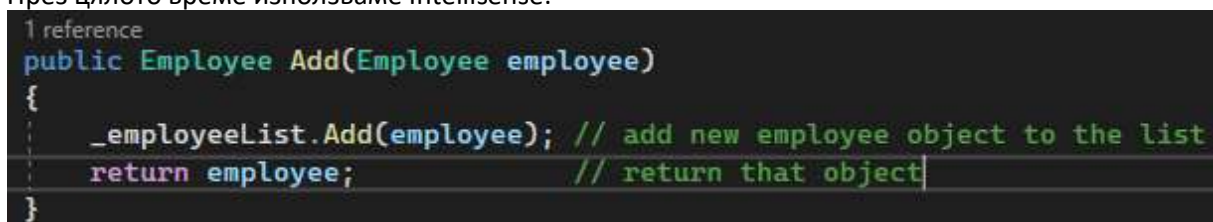
Автокорекцията е с най-простата имплементация чрез употребата на изключение:



Ние обаче искаме да добавим нов член към съществуващия преди него лист, т.е. искаме да добавим employee -> `_employeeList`

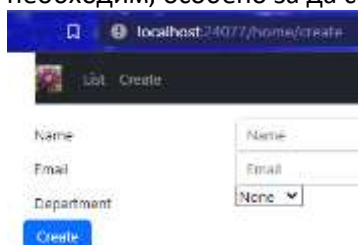


През цялото време използваме Intellisense:



```
public Employee Add(Employee employee)
{
    _employeeList.Add(employee); // add new employee object to the list
    return employee;             // return that object
}
```

Във формата за попълване има само Name, Email, Department, но скрит остава id, който също е необходим, особено за да се добави към горния списък:



За целта използваме Max method, като към най-голямото Id от списъка +1 и новата му стойност се предава като Id на новият employee object.


```
MockEmployeeRepository.cs* Index.cshtml HomeController.cs Create.cshtml Details.cshtml
EmployeeManagement.Models.MockEmployeeRepository Add(Employee employee)

_employeeeList = new List<Employee>()
{
    new Employee() { Id = 1, Name = "Mary", Department = Dept.HR, Email = "Mary@abv.bg" },
    new Employee() { Id = 2, Name = "John", Department = Dept.IT, Email = "John@abv.bg" },
    new Employee() { Id = 3, Name = "Sam", Department = Dept.IT, Email = "Sam@abv.bg" }
};

1 reference
public Employee Add(Employee employee)
{
    employee.Id = _employeeList.Max(e => e.Id)+1; // max id form the list + 1 = new id
    _employeeList.Add(employee); // add new employee object to the list
    return employee; // return that object
}
```

```
public Employee Add(Employee employee)
{
    employee.Id = _employeeList.Max(e => e.Id)+1; // max id + 1 = new id
    _employeeList.Add(employee); // add new employee object to the list
    return employee; // return that object
}
```

```
MockEmployeeRepository.cs HomeController.cs* MockEmployeeRepository.cs Index.cshtml Create.cshtml Det
EmployeeManagement.Controllers.HomeController _employeeRepository
1 reference
public class HomeController : Controller
{
    private readonly IEmployeeRepository _employeeRepository; //auto private field
    0 references
    public HomeController(IEmployeeRepository employeeRepository)
    {
        _employeeRepository = employeeRepository;
    }

    0 references
    public ViewResult Index()
    {
        var model = _employeeRepository.GetAllEmployee();
        return View(model);
    }

    0 references
    public ViewResult Details(int? id)
    {
        HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
        {
            Employee = _employeeRepository.GetEmployee(id ?? 1), //c# colesing operator
            PageTitle = "Employee Details"
        };
        return View(homeDetailsViewModel);
    }
}
```

Като погледнем, към момента всички action methods използват `_employeeRepository` от частното поле, създадено още в началото на `HomeController` за тази цел. Използваме инжектирания `IEmployeeRepository` и добавяме този `_employeeRepository`:

```
_employeeRepository.Add(employee);
```

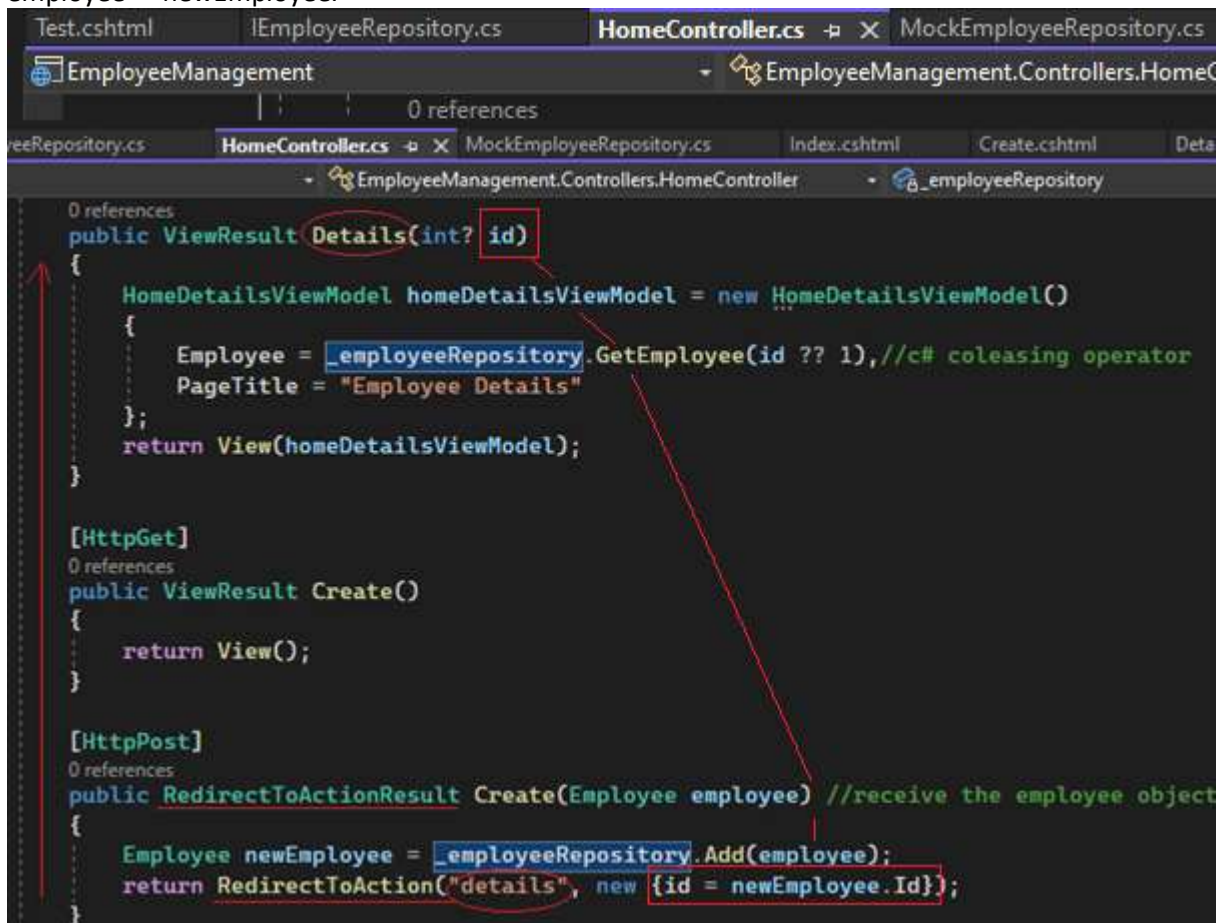
като му прехвърляме `employee` (т.е. той прихваща `employee`), на който вече сме присвоили новото `id`.

```
Employee newEmployee = _employeeRepository.Add(employee);
```

Съхраняваме този нов служител от клас `Employee` в нова променлива, която наричаме

newEmployee.

След това пренасочваме (redirect) потребителя към Details action, за да види новите данни на новия служител, който току-що сме създали. Също му подаваме ново Id, което се взема от employee -> newEmployee.



```
0 references
public ViewResult Details(int? id)
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(id ?? 1), //c# coalescing operator
        PageTitle = "Employee Details"
    };
    return View(homeDetailsViewModel);
}

[HttpGet]
0 references
public ViewResult Create()
{
    return View();
}

[HttpPost]
0 references
public RedirectToActionResult Create(Employee employee) //receive the employee object
{
    Employee newEmployee = _employeeRepository.Add(employee);
    return RedirectToAction("details", new {id = newEmployee.Id});
}
```

Т.е. искаме в HomeController да се изпълни Create, която чрез извикване на Add да създаде нов служител и като се попълни формата ѝ + автоматично новото id, да съхрани новите данни, като ги добави в _employeeRepository, а после да прехвърли потребителя към details. Самата функция RedirectToAction изисква да се подаде името на action метода, към който пренасочваме и променлива, която ще се подаде, а според характеристиките на details, се подава id. Ние подаваме id чрез създаването на нов анонимен {} обект.

new {anonymus object} include a property with name id, and the value for this is come from the property Id of this newEmployee object.

RedirectToAction изисква тип RedirectToActionResult.

```
[HttpGet]
public ViewResult Create()
{
    return View();
}

[HttpPost]
public RedirectToActionResult Create(Employee employee)
{
    Employee newEmployee = _employeeRepository.Add(employee);
    return RedirectToAction("details", new {id = newEmployee.Id});
}
```

}

localhost:24077/home/create

List Create

Name David

Email david@abv.bg

Department IT

Create

localhost:24077/Home/Details/4

List Create

David



Employee ID:
EmployeeManagement.Models.E
mployee

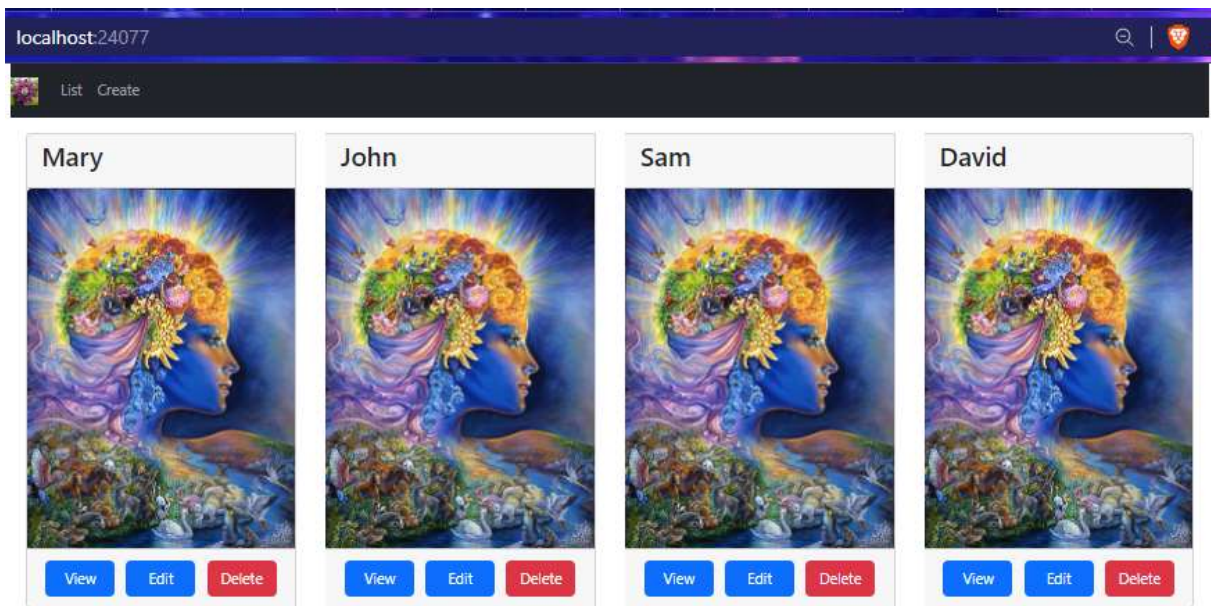
Employee Email: david@abv.bg
Employee Department: IT

Back Edit Delete

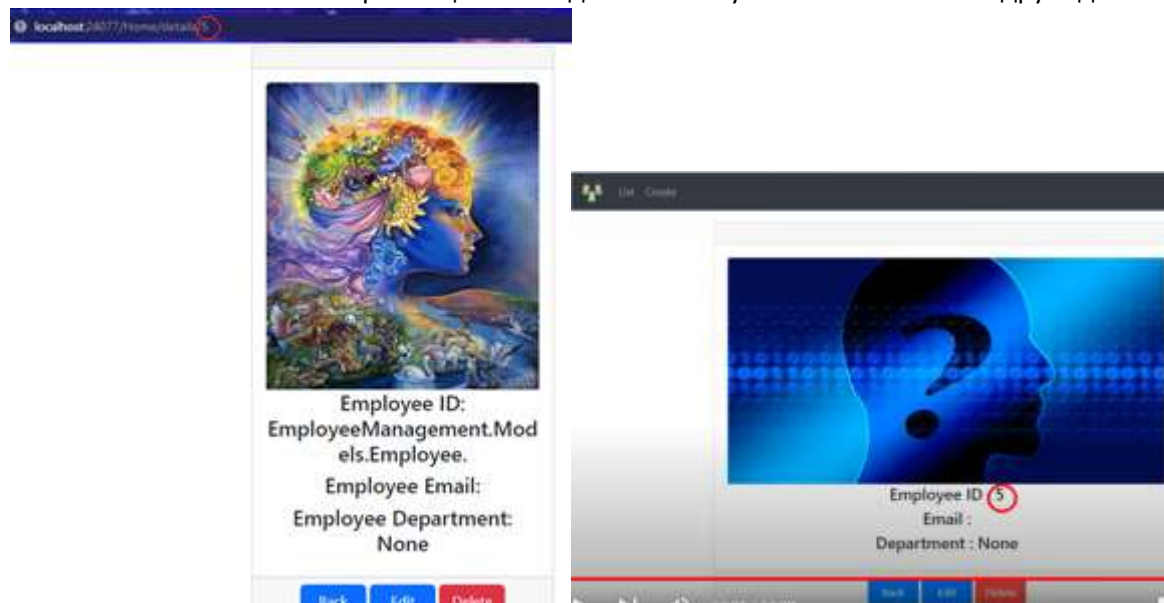
Имената, които задаваме на формата са много важни – те трябва да съвпадат с предварително дефинираните в Employee (Name-Name, Email-Email, Department-Department), за да може да се осъществи адекватен мапинг.

```
namespace EmployeeManagement.Models
{
    25 references
    public class Employee
    {
        8 references
        public int Id { get; set; }
        7 references
        public string Name { get; set; }
        6 references
        public string Email { get; set; }
        6 references
        public Dept Department { get; set; }
    }
}
```

```
<label class="col-sm-2 col-form-label" for="Name">Name</label>
<div class="col-sm-10">
    <input class="form-control" placeholder="Name" type="text" id="Name" name="Name" value="" />
</div>
</div>
<div class="form-group row">
    <label class="col-sm-2 col-form-label" for="Email">Email</label>
    <div class="col-sm-10">
        <input class="form-control" placeholder="Email" type="text" id="Email" name="Email" value="" />
    </div>
</div>
<div class="form-group row">
    <label class="col-sm-2 col-form-label" for="Department">Department</label>
    <div class="col-sm-10">
        <select class="custom-select mr-sm-2" data-val="true" data-val-required="The Department field is required." id="Department" name="Department">
```

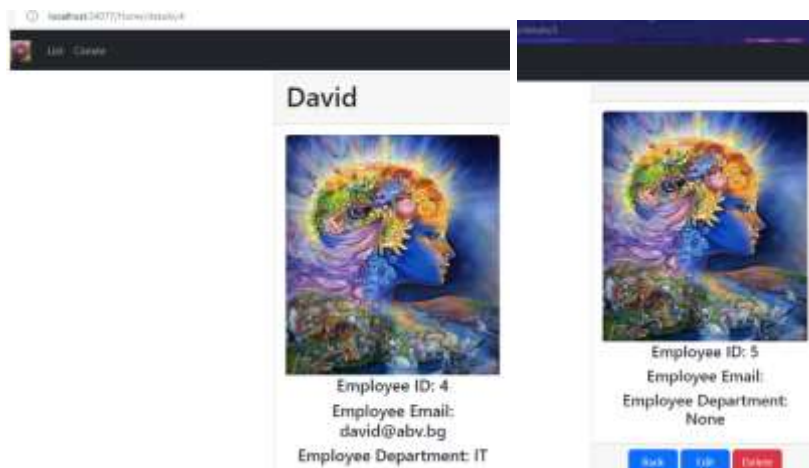
Вече създаваме нови служители, но нямаме валидация, затова може да попълним с празни полета и пак автоматично прихваща id и създава нови служители без имена и други данни.



Вижда се, че в браузъра id = 5, но при мен не се изписва. Когато попълня формата в единия браузър, промените се виждат и в другия.

Намерих си грешката – без да искам нещо съм натиснала и изтрила .ID, .Email от Details

```
<h4>Employee ID: @Model.Employee.Id</h4>
<h4>Employee Email: @Model.Employee.Email </h4>
<h4>Employee Department: @Model.Employee.Department </h4>
```



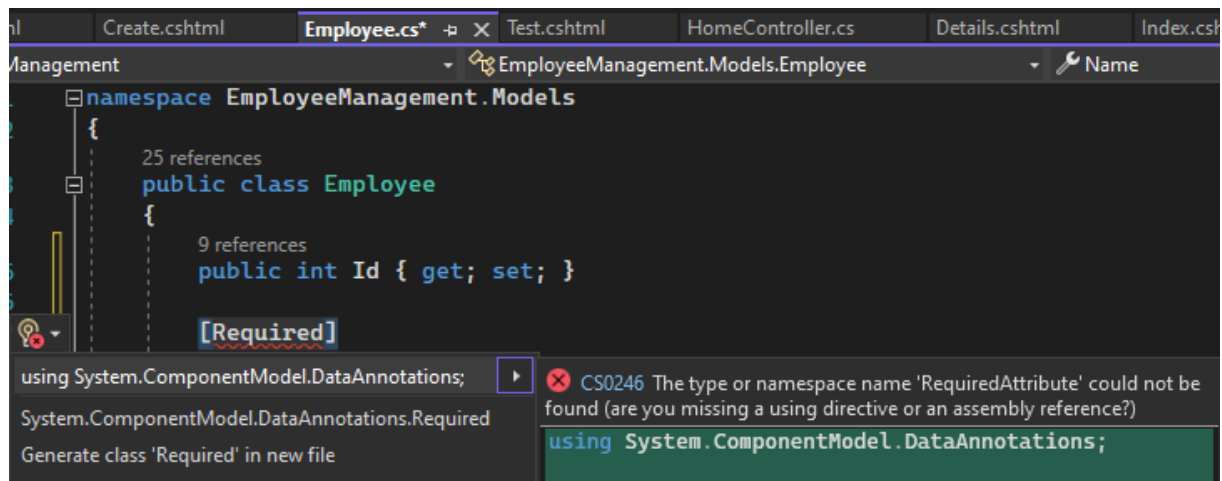
Към момента може да добавяме служител без попълнени полета. В следващия урок ще забраним това с model validation.

42 – Model Validation

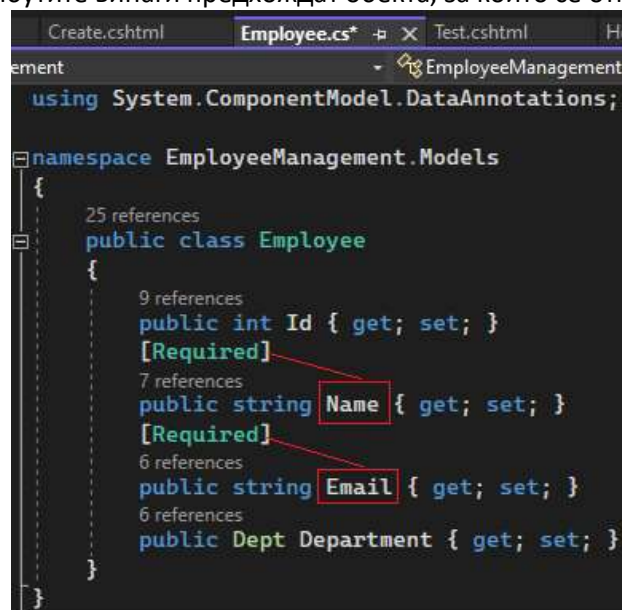
Към момента имаме възможност за създаването на нови служители без попълнени данни. За да се избегне това, се изисква валидация.

ASP.NET Core Model Validation

За да направим поле Name, което е свързано с Employee.Name задължително, е необходим **[Required]** attribute, който съответно да извика namespace

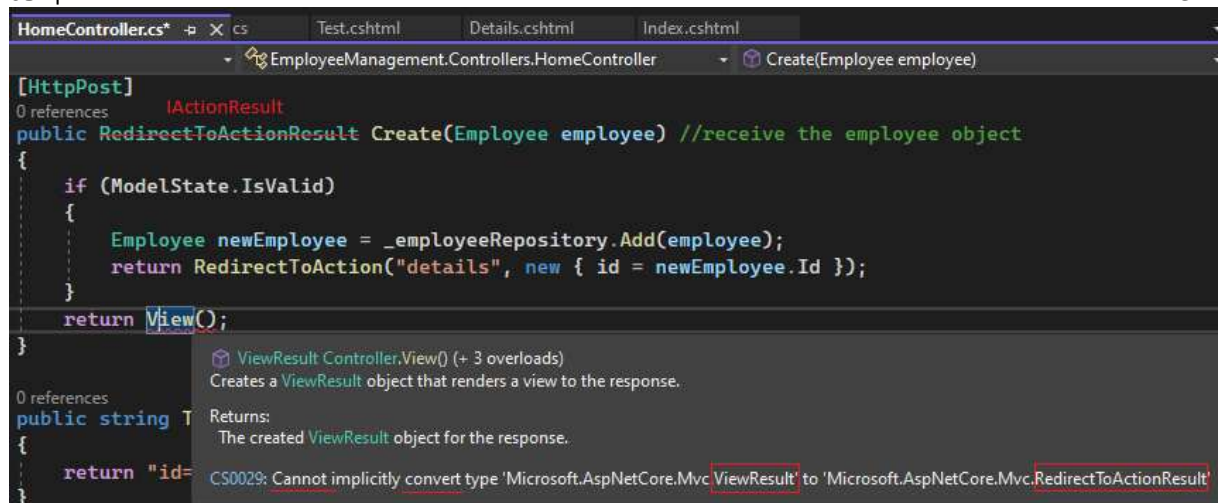


Атрибутите винаги предхождат обекта, за който се отнасят>



На този етап, ако се попълни празно поле Name, неговата стойност ще бъде Null и няма да се изпълни валидацията.

За да провери контролера дали валидацията е успешна или не, всичко, което трябва да направим, е да добавим if – ако валидацията е успешна, добавя нов служител, ако не, връща същия View.



```

[HttpPost]
public IActionResult Create(Employee employee) {

```

```

    if (ModelState.IsValid)
    {
        Employee newEmployee = _employeeRepository.Add(employee);
        return RedirectToAction("details", new { id = newEmployee.Id });
    }
    return View();
}

```

* IActionResult = RedirectToActionResult + ViewResult

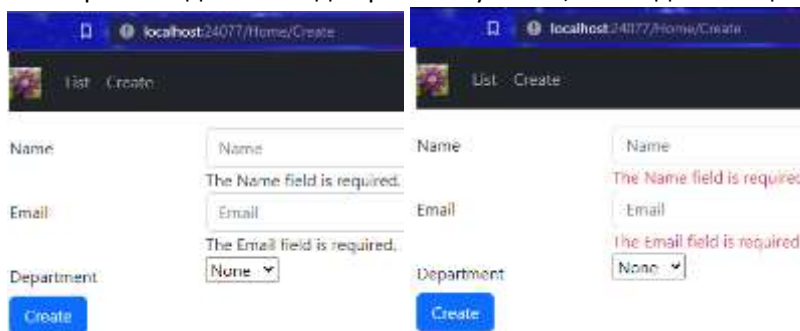
Остава да се изведе съобщение за грешка чрез C# asp-validation-for. Ако валидацията не се изпълни за Name property, на мястото на позиционирането на таг ще се изведе съобщение за грешка.

```

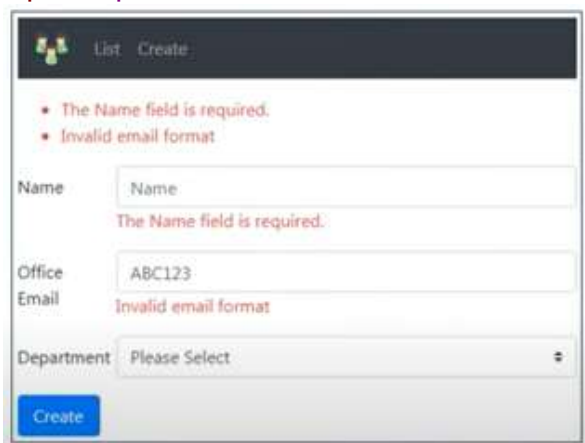
<form asp-controller="Home" asp-action="Create" method="post" class="mt-3">
  <div class="form-group row">
    <label asp-for="Name" class="col-sm-2 col-form-label"></label>
    <div class="col-sm-10">
      <input asp-for="Name" class="form-control" placeholder="Name"/>
      <span asp-validation-for="Name"></span>
    </div>
  </div>

```


Сега при опит да се въведе празен служител, извежда съобщения за грешки.



class="text-danger" прави текста червен.



Built-in Validation Attributes

- RegularExpression
- Required
- Range
- MinLength
- MaxLength
- Compare

Към момента обаче, форматът на Email не е валиден.

[RegularExpression(@"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+\$")]

```
[Required]
[RegularExpression(@"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$")]
7 references
public string Email { get; set; }
```

Резултат при грешно попълване на email:

Name	Name
	The Name field is required.
Email	AbC1
	The field Email must match the regular expression '^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+\$'.
Department	None
	Create

За да променим съобщението, добавяме Property `ErrorMessage = "Invalid email format".`

```
[RegularExpression(@"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$")]
ErrorMessage = "Invalid email format"]
```

Към момента, текстът Email е автоматично прихванат, но ако желаем да го променим на Office email, използваме друг атрибут `[Display (Name="Office Email")]`

Name	Name	Name	Name
	The Name field is required.		
Office Email	AbC1	Office Email	AbC1
	Invalid email format		Invalid email format
Department	None	Department	None
	Create		Create

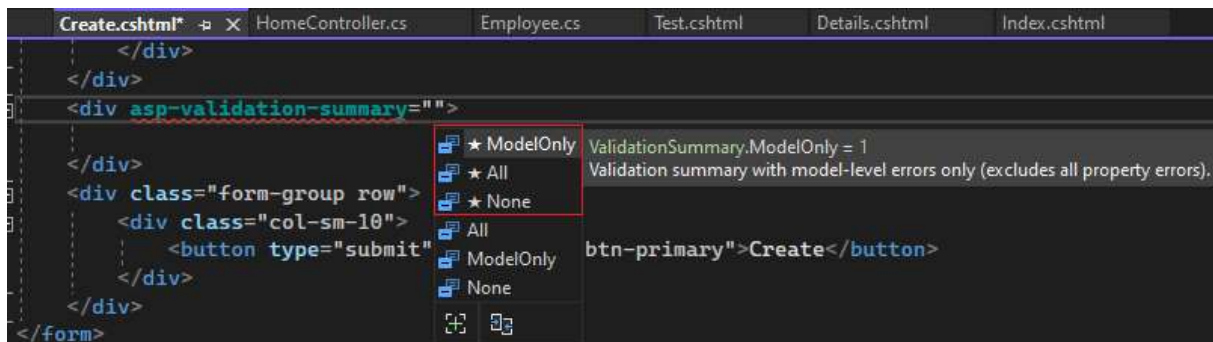
Последно искаме да допуснем максимална дължина от 50 символа за името с персонализирано съобщение за грешка.

```
[MaxLength(50, ErrorMessage = "Name cannot exceed 50 characters")]
```

При мен обаче така и не извежда подобна грешка, просто не допуска да надхвърля при самото въвеждане повече от 50 символа. Когато опитам ограничение с 60 символа, по същият начин не ми позволи да ги надхвърля още при въвеждането.

```
public class Employee
{
    public int Id { get; set; }
    [Required]
    [MaxLength(60, ErrorMessage = "Name cannot exceed 50 characters")]
    public string Name { get; set; }
    [Required]
    [RegularExpression(@"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$",
        ErrorMessage = "Invalid email format")]
    [Display(Name="Office Email")]
    public string Email { get; set; }
    public Dept Department { get; set; }
}
```

Към момента под всяко поле за въвеждане се извежда съответната грешка, но може да обобщим грешките на едно място точно над бутона чрез summary:



`<div asp-validation-summary="All"></div>`

Name
The Name field is required.

Office Email
Invalid email format

Department

- The Name field is required.
- Invalid email format

[Create](#)

съответно отново може да променим цвета до червен с `class="text-danger"`.

ASP.NET Core Model Validation

Step 1 : Apply **Validation Attributes** on Properties

```
public class Employee
{
    public int Id { get; set; }
    [Required, MaxLength(50)]
    public string Name { get; set; }
    [Display(Name = "Office Email")]
    [RegularExpression(@"^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+$",
        ErrorMessage = "Invalid email format")]
    [Required]
    public string Email { get; set; }
    public Dept Department { get; set; }
}
```

Step 2 : Use **ModelState.IsValid** property to check if validation has failed or succeeded

```
[HttpPost]
public IActionResult Create(Employee employee)
{
    if (ModelState.IsValid)
    {
        Employee newEmployee = _employeeRepository.Add(employee);
        return RedirectToAction("details", new { id = newEmployee.Id });
    }

    return View();
}
```

Step 3 : Use **asp-validation-for** and **asp-validation-summary** tag helpers to display validation errors

```
<div asp-validation-summary="All">
</div>

<div>
    <label asp-for="Name"></label>
    <div>
        <input asp-for="Name">
        <span asp-validation-for="Name"></span>
    </div>
</div>

<div>
    <label asp-for="Email"></label>
    <div>
        <input asp-for="Email">
        <span asp-validation-for="Email"></span>
    </div>
</div>
```

Validation: I Attributes [Required]
If check: II ModelState.IsValid
Errors: III asp-validation-for / -summary

44 – AddSingleton vs AddScoped vs AddTransient


```

{
    private List<Employee> employeeList; // field
    0 references
    public MockEmployeeRepository() // ctor
    {
        _employeeList = new List<Employee>()
        {
            new Employee() { Id = 1, Name = "Mary", Department = Dept.HR, Email = "Mary@abv.bg" },
            new Employee() { Id = 2, Name = "John", Department = Dept.IT, Email = "John@abv.bg" },
            new Employee() { Id = 3, Name = "Sam", Department = Dept.IT, Email = "Sam@abv.bg" }
        };
    }

    2 references
    public Employee Add(Employee employee)
    {
        employee.Id = _employeeList.Max(e => e.Id)+1; // max id form the list + 1 = new id
        _employeeList.Add(employee); // add new employee object to the list
        return employee; // return that object
    }
}

```

_employeeList is in-memory list = при всеки рестарт на приложението, промените в частното поле _employeeList се губят.

Когато се стартира приложението и се извика конструкторът MockEmployeeRepository(), частното поле _employeeList се инициализира с тези 3 служителя, с които започва работата му.

```

oller.cs  Startup.cs  EmployeeRepository.cs  MockEmployeeRepository.cs
EmployeeManagement.Startup  ConfigureServices(IServiceCollection)

0 references
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvc(option => option.EnableEndpointRouting = false)
        .AddXmlSerializerFormatters();
    services.AddSingleton<IEmployeeRepository, MockEmployeeRepository>();
    // if someone (like HomeController) requests this IEmployeeRepository service,
    // create an instance of this MockEmployeeRepository class and inject it.
    AddScoped
    AddTransient
}

```

```

public class HomeController : Controller
{
    private readonly IEmployeeRepository _employeeRepository; //auto private field
    0 references
    public HomeController(IEmployeeRepository employeeRepository)
    {
        0 references
        //dependency injection
        _employeeRepository = employeeRepository;
    }
    0 references
    public IActionResult Index() ...
    0 references
    public IActionResult Details(int? id) ...

    [HttpGet]
    0 references
    public IActionResult Create() ...

    [HttpPost]
    0 references
    public IActionResult Create(Employee employee) //receive the employee object
    {
        if (ModelState.IsValid)
        {
            Employee newEmployee = _employeeRepository.Add(employee);
            return RedirectToAction("details", new { id = newEmployee.Id });
        }
        return View();
    }
}

```

A red arrow originates from the `IEmployeeRepository` parameter in the `HomeController` constructor and points to the `_employeeRepository` field. Another red arrow originates from the `_employeeRepository` field and points to the `Add(employee)` call inside the `Create` method.

Временно не желаем при създаването на нов служител да бъдем пренасочвани към страница Details, а да останем в същата Create, затова закоментираме пренасочването:

```

[HttpPost]
0 references
public IActionResult Create(Employee employee) //receive the employee object
{
    if (ModelState.IsValid)
    {
        Employee newEmployee = _employeeRepository.Add(employee);
        // return RedirectToAction("details", new { id = newEmployee.Id });
    }
    return View();
}

```

A red arrow points from the `employee` parameter in the `Create` method to the `employee` argument in the `Add(employee)` call. Another red arrow points from the `Add(employee)` call to the commented-out `RedirectToAction` line.

Освен това, искаме да ни извежда общият брой на всички служители:

```
@model Employee
[Inject] IEmployeeRepository _employeeRepository

@{
    ViewBag.Title = "Create Employee";
}

<form asp-controller="Home" asp-action="Create" method="post" class="mt-3">
    <div class="form-group row"></div>
    <div class="form-group row"></div>
    <div class="form-group row"></div>
    <div asp-validation-summary="All" class="text-danger"></div>
    <div class="form-group row"></div>
    <div class="form-group row">
        <div class="col-sm-10">
            Total Employees Count = @_employeeRepository.GetAllEmployee().Count()
        </div>
    </div>
</form>
```

[Inject] IEmployeeRepository _employeeRepository

Total Employees Count = @_employeeRepository.GetAllEmployee().Count()

Когато натиснем Create btn, и обработи заявката, нашето приложение се нуждае от IEmployeeRepository на 2 места:

- 1) в HomeController/Create [HttpPost], съответно ни трябва IEmployeeRepository service, т.е. _employeeRepository
 - 2) от същото в Create.cshtml, за да изчислим общия брой на всички служители.
- За целта към момента използваме AddSingleton in Startup.

Name	
Office Email	
Department	Please Select
Create	

Total Employees Count = 3

Какво всъщност се случва: ние сме на адрес /home/create, съответно първият [HttpGet] Create прихваща нашата заявка, а единственото, което прави, е да ни препрати в Create.cshtml. Когато разгледаме неговия код, съответно, още в началото, той има зависимост към IEmployeeRepository_employeeRepository.

В Startup имаме този код:

```
services.AddSingleton<IEmployeeRepository, MockEmployeeRepository>();
```

като, вместо да се създава нова инстанция за употребата на Mock, реизползва отново същата IEmployeeRepository_employeeRepository в HomeController и я инжектира в Create.cshtml, който я използва, за да изчисли Count().

To serve this request an instance of the HomeController is created first. HomeController has a dependency on IEmployeeRepository. This is the first time, the instance of the service is requested. So asp.net core creates an instance of the service and injects it into the HomeController.

Попълваме данните веднъж и независимо колко пъти натискаме Create със същите данни,

броят на служителите под него се увеличава (без да ни препраща към Details).

```
services.AddSingleton<IEmployeeRepository, MockEmployeeRepository>();
```

Name	David
Office Email	david@abv.bg Singleton
Department	IT Home -> Create
	3 + 1 = 4
	4 + 1 = 5
	5 + 1 = 6
	6 + 1 = 7
Total Employees Count = 4 5, 6, 7, 8, ...	

AddSingleton() Създава инстанция на _employeeRepository service и се използва през целия живот от всички заявки на приложението.

* Създава се, запазва се, създава се нова, запазва се.

```
services.AddScoped<IEmployeeRepository, MockEmployeeRepository>();
```

Name	David
Office Email	david@abv.bg Scoped
Department	IT Home -> Create
	3 + 1 = 4
	3 + 1 = 4
	3 + 1 = 4
	3 + 1 = 4
Total Employees Count = 4 4, 4, 4, 4, 4, ...	

AddScoped() При всяко натискане на бутона, създаваме нова http заявка, която от своя страна създава нова инстанция на _employeeRepository service. Това е причината служител, създаден с една заявка, да не може да бъде видян от следващата заявка. Т.е. нова инстанция при всяка нова заявка.

* Създава се, запазва се веднъж, създава се наново всеки път.

```
services.AddTransient<IEmployeeRepository, MockEmployeeRepository>();
```

Name	David
Office Email	david@abv.bg Transient
Department	IT Home -> Create
	3 + 1 = 3
	3 + 1 = 3
	3 + 1 = 3
	3 + 1 = 3
Total Employees Count = 3 3, 3, 3, 3, 3 ...	

AddTransient() всеки път се създава нова инстанция без да се запазва с всяка нова заявка.

AddSingleton AddScoped AddTransient

With a **singleton** service, there is only a single instance. An instance is created, when the service is first requested and that single instance is used by all http requests throughout the application

With a **scoped** service we get the same instance within the scope of a given http request but a new instance across different http requests

With a **transient** service a new instance is provided every time an instance is requested whether it is in the scope of the same http request or across different http requests

45 – Introduction to Entity Framework

What is Entity Framework Core

- **ORM (Object-Relational Mapper)**
- **Lightweight, Extensible, and Open Source**
- **Works Cross Platform**
- **Microsoft's Official Data Access Platform**

ORM stands for Object-Relational Mapper and it enables developers to work with a database using business objects. As a developer we work with the application business objects and the ORM generates the SQL that the underlying database understands. In-short, an ORM, eliminates the need for most of the data-access code that developers usually need to write.

Domain Classes

```
public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public Department Department { get; set; }
}
```

```
public class Department
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Location { get; set; }
}
```

ORM спестява много код за read, insert, update, delete DB tables.

```
public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public Department Department { get; set; }
}

public class Department
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Location { get; set; }
}
```



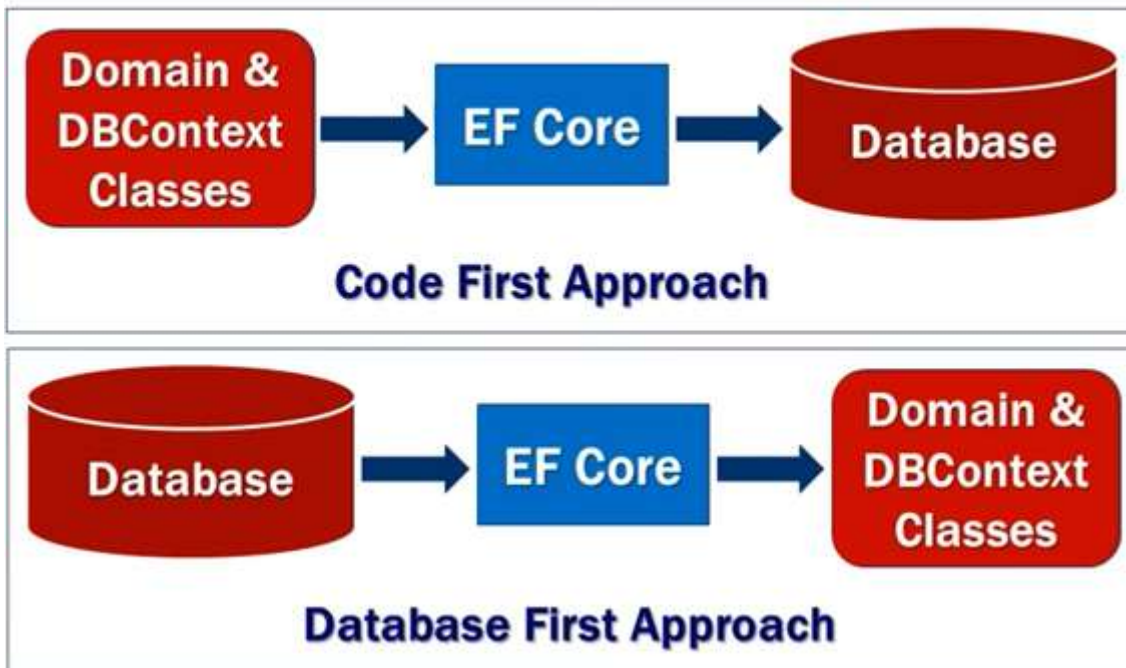
Entity Framework Core



Database

Има два начина за връзка между класовете и ДБ: Първо код, после БД или обратното:

Code First & Database First Approach



Plug-in libraries = DB Providers (NuGet Packages): <https://docs.microsoft.com/en-us/ef/core/providers/?tabs=dotnet-core-cli> – съдържат цялата характерна функционалност, необходима за осъществяване на специфичните услуги.

	NuGet Package	Supported database engines	Maintainer / Vendor	Notes / Requirements	Built for versions
	Microsoft.EntityFrameworkCore.SqlServer [↗]	SQL Server 2012 onwards	EF Core Project [↗] (Microsoft)		6.0
	Microsoft.EntityFrameworkCore.Sqlite [↗]	SQLite 3.7 onwards	EF Core Project [↗] (Microsoft)		6.0
	Microsoft.EntityFrameworkCore.InMemory [↗]	EF Core in-memory database	EF Core Project [↗] (Microsoft)	Limitations	6.0
	Microsoft.EntityFrameworkCore.Cosmos [↗]	Azure Cosmos DB SQL API	EF Core Project [↗] (Microsoft)		6.0
	Npgsql.EntityFrameworkCore.PostgreSQL [↗]	PostgreSQL	Npgsql Development Team [↗]		6.0
	Pomelo.EntityFrameworkCore.MySql [↗]	MySQL, MariaDB	Pomelo Foundation Project [↗]		6.0
	MySql.EntityFrameworkCore [↗]	MySQL	MySQL project [↗] (Oracle)		6.0

EF Core Database Providers



46 – Install Entity Framework Core

Install Entity Framework Core

Single Layer Web Application

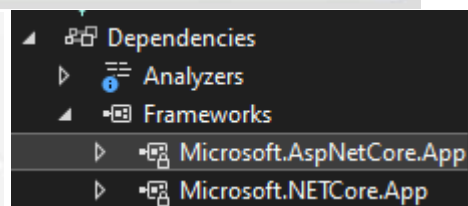
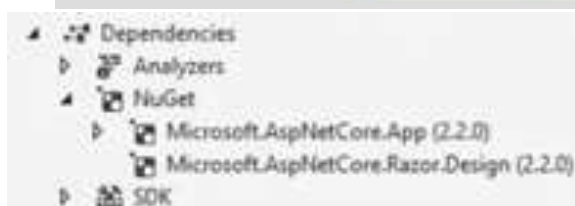
Multi Layer Web Application

Multi Layer Web Application

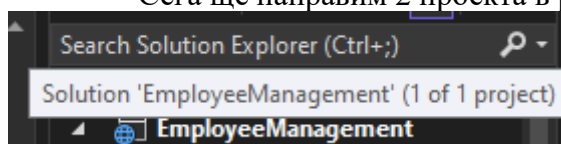
Presentation Layer

Business Layer

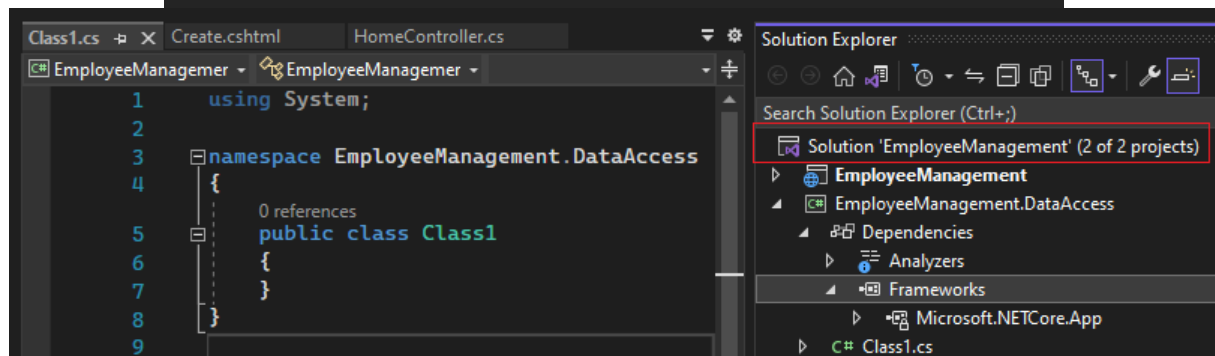
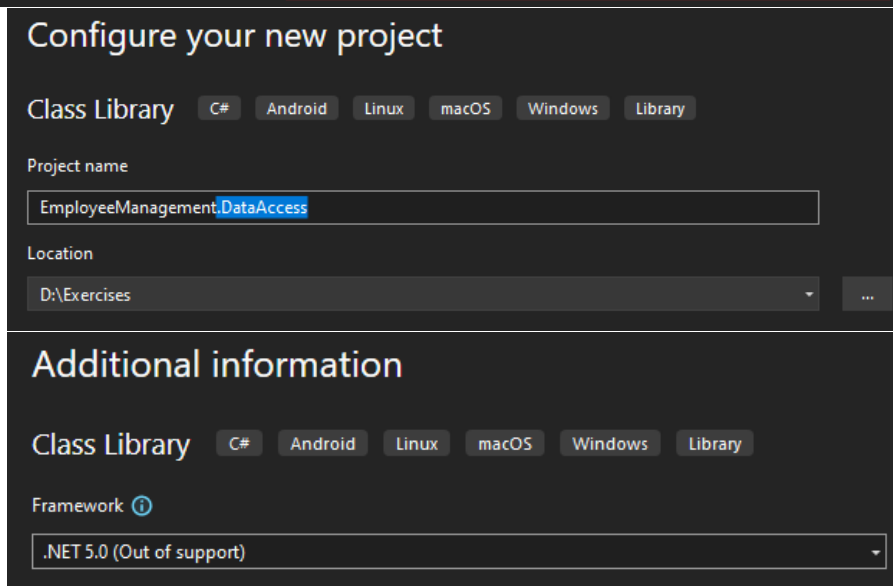
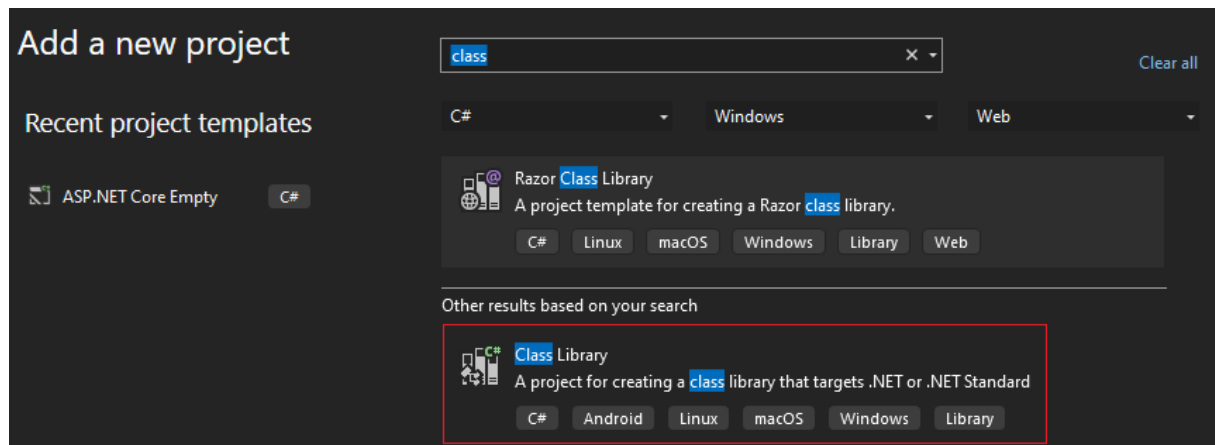
Data Access Layer



Сега ще направим 2 проекта в рамките на 1 Solution:



Solution -> Add -> New Project



За да използваме SqlServer за нашите приложения, трябва да инсталираме NuGet

Entity Framework Core Packages

Microsoft.EntityFrameworkCore.SqlServer

SqlServer = Relational + ()

Microsoft.EntityFrameworkCore.Relational

надграждане

Microsoft.EntityFrameworkCore Ядро

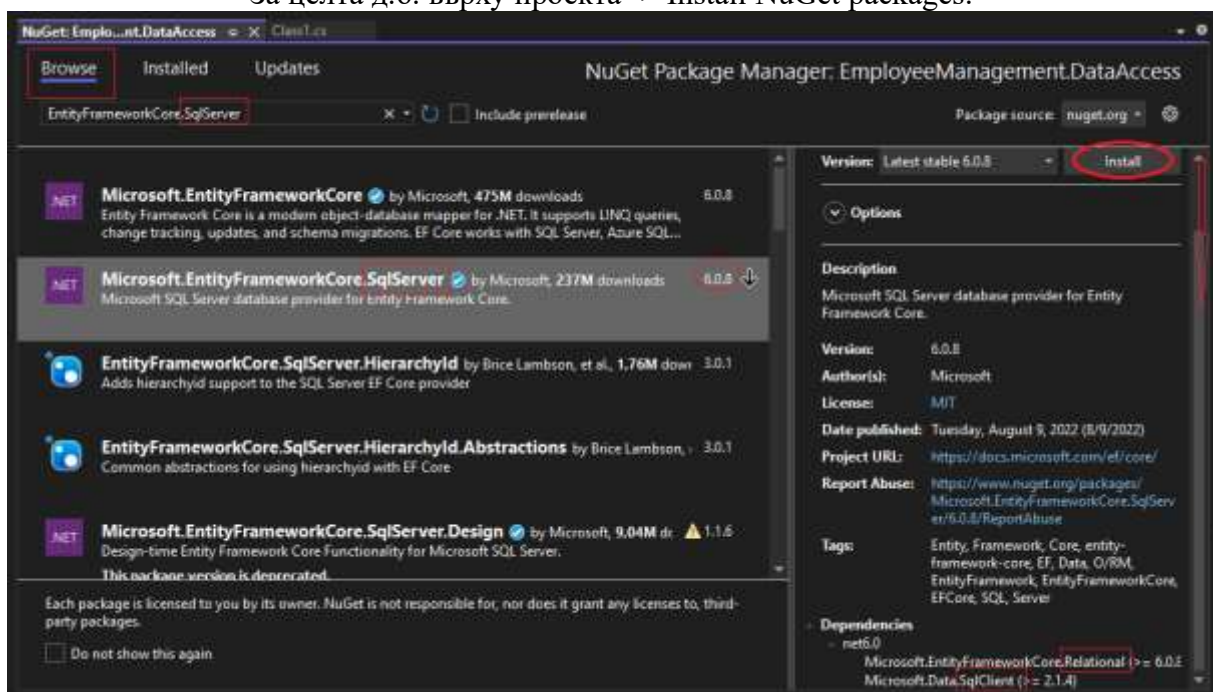
packages:

SqlServer включва в себе си Relational, а той от своя страна Core

$\{[(\text{Core})+\text{Relational}]+\} = \text{SqlServer}$

Т.е. ако инсталираме първо SqlServer, няма да се налага да инсталираме останалите, понеже ги включва в себе си, но ако инсталираме първо Core, ще се наложи да го надграждаме до Relational и SqlServer.

За целта д.б. върху проекта -> Install NuGet packages:



Търсим SqlServer, като виждаме, че в зависимостите, той съдържа Relational + SqlClient

docs.microsoft.com/en-us/ef/core/providers/?tabs=dotnet-core-cli

NuGet Package	Supported database engines	Maintainer Vendor
Microsoft.EntityFrameworkCore.SqlServer	SQL Server 2012 onwards	EF Core Project (Microsoft)
Microsoft.EntityFrameworkCore.Sqlite	SQLite 3.7 onwards	EF Core Project (Microsoft)
Microsoft.EntityFrameworkCore.InMemory	EF Core in-memory database	EF Core Project (Microsoft)
Microsoft.EntityFrameworkCore.Cosmos	Azure Cosmos DB SQL API	EF Core Project (Microsoft)
Npgsql.EntityFrameworkCore.PostgreSQL	PostgreSQL	Npgsql Development Team
Pomelo.EntityFrameworkCore.MySql	MySQL MariaDB	Pomelo Foundation Project
MySQL.EntityFrameworkCore	MySQL	MySQL project (Oracle)

С инсталацията на пакета, автоматично се включват и всички зависимости.

EntityFrameworkCore.MySql x Include prerelease Package source: nuget.org

Microsoft.EntityFrameworkCore by Microsoft, 475M downloads 6.0.8
Entity Framework Core is a modern object-database mapper for .NET. It supports LINQ queries, change tracking, updates, and schema migrations. EF Core works with SQL Ser...

Pomelo.EntityFrameworkCore.MySql by Laurents Meyer, Caleb Lloyd, Yuko Z 6.0.2
Pomelo's MySQL database provider for Entity Framework Core.

Each package is licensed to you by its owner. NuGet is not responsible for, nor does it grant any licenses to, third-party packages.

☐ Do not show this again

Date published: Sunday, July 24, 2022 (7/24/2022)

Project URL: <https://github.com/PomeloFoundation/Pomelo.EntityFrameworkCore.MySql>

Report Abuse: <https://www.nuget.org/packages/Pomelo.EntityFrameworkCore.MySql/6.0.2/ReportAbuse>

Tags: pomelo, mysql, mariadb, EntityFrameworkCore, entity-framework-core, ef, efcore, core, orm, sql

Dependencies

- net6.0
 - Microsoft.EntityFrameworkCore.Relational >= 1
 - Microsoft.Extensions.DependencyInjection.Abstractions >= 2.1.2
 - Microsoft.Enti

Този урок бе с демонстрационна цел, затова д.б. върху новия проект -> Remove.

Заедно с това, трябва да изтрием и проекта от файловата система, затова д.б. върху Solution -> Open Folder in File Explorer -> Delete.

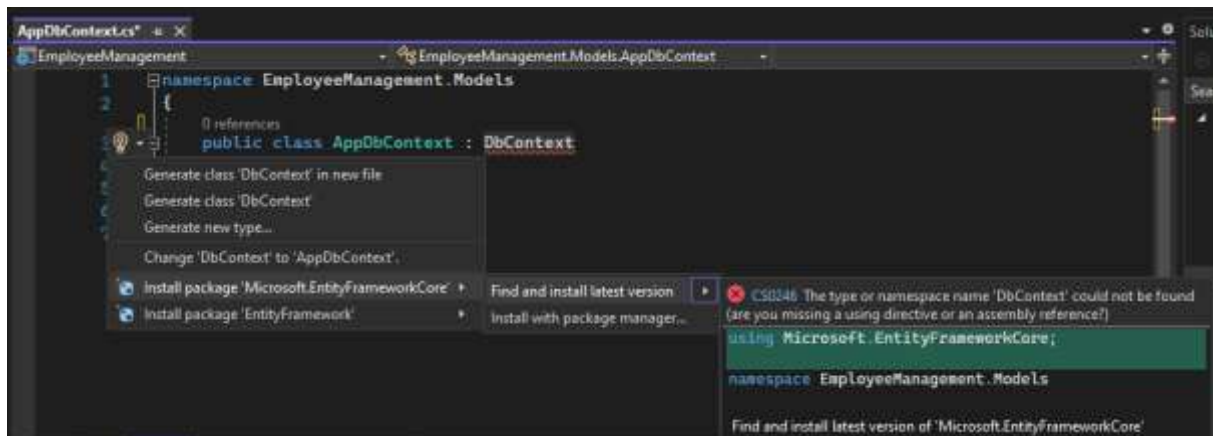
До момента, приложението съхраняваше данните в in-memory list, а вече ще научим как да ги съхраняваме чрез MySQL server.

47 — DbContext in Entity Framework Core

Вграденият DbContext class служи за връзка с БД: Models -> Add -> Class -> по избор AppDbContext.cs

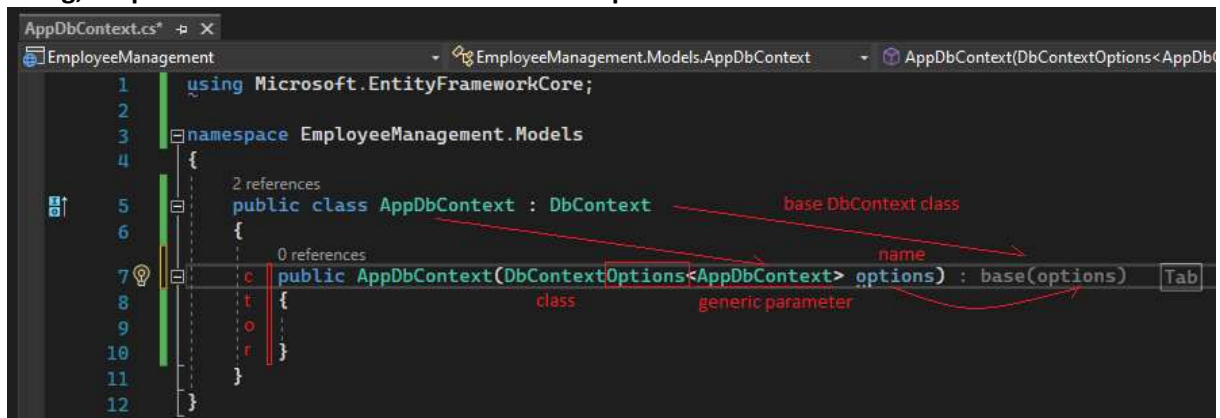
Искаме да извикаме вграден клас DbContext, но автоматичното namespace не ми е позволено, защото EntityFrameworkCore не е инсталиран. Решавам да инсталирам направо SqlServer, който

го включва в себе си, като за целта проектът се изменя на .net 6.0, т.к. 5.0 вече е извън поддръжка.



С което вече успешно се извиква namespace.

За да бъде използваем този `DbContext` вграден клас, е необходимо да се извика `DbContext options class`, като неговата инстанция носи конфигурационната информация, като `connection string`, `Db providers` и т.н. Най-лесният начин е чрез `ctor`:



Entity Framework Core DbSet

- The `DbContext` class includes a `DbSet<TEntity>` property for each entity in the model.
- At the moment in our application we have, only one entity class - `Employee`.
- So in our `AppDbContext` class we only have one `DbSet<Employee>` property.


```

public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
{
}
}
class
prop public DbSet<Employee> Employees { get; set; } Tab to accept
}
type new property name

```

- We will use this `DbSet` property `Employees` to query and save instances of the `Employee` class.
- The LINQ queries against the `DbSet<TEntity>` will be translated into queries against the underlying database.

DbContext in Entity Framework Core

```

public class AppDbContext : DbContext
{
}

```

To use the **DbContext** class in our application, create a class that derives from the **DbContext** class

DbContext in Entity Framework Core

```

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
    {
    }
}
keyword

```

To pass configuration information to the **DbContext** use **DbContextOptions** instance

DbContext in Entity Framework Core

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
    {
    }

    public DbSet<Employee> Employees { get; set; }
}
```

- The **DbContext** class includes a **DbSet<TEntity>** property for each entity in the model
- We will use this **DbSet** property **Employees** to query and save instances of the **Employee** class
- The LINQ queries against the **DbSet<TEntity>** will be translated into queries against the underlying database

using Microsoft.EntityFrameworkCore;

```
namespace EmployeeManagement.Models
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Employee> Employees { get; set; }
    }
}
```

48 — Sql Server

Искаме да използваме MSSqlServer with EntityFrameworkCore. Това става чрез ConfigureServices – Startup.cs

Миналия урок създадохме AppDbContext class, който трябва да регистрираме чрез Dependency injection in Startup.cs.

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContext<>
    services.AddDbContextFactory<>
    services.AddDbContextPool<>
    // if som AddPooledDbContextFactory<>
```

With **DbContext** pooling, an instance from the **DbContext** pool is provided if available, rather than creating a new instance.

Всеки път когато инстанция на DbContext е заявена, вместо да се създаде нова инстанция, Asp.NetCore проверява дали тя съществува в DbContextPool и ако е налична, връща нея. От гледна точка на производителност, AddDbContextPool() е по-добър от AddDbContext(). След това трябва да вмъкнем необходимия provider със съответното namespace, за да се включи и Intellisense.

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContextPool<AppDbContext>(options => options.UseSqlServer());
    services.AddMvc(option => option.EnableEndpointRouting = false)
        .AddXmlSerializerFormatters();
    services.AddTransient<IEmployeeRepository, MockEmployeeRepository>();
}

options.UseSqlServer();
```

in (extension) DbContextOptionsBuilder DbContextOptionsBuilder.UseSqlServer
 ([System.Action<Microsoft.EntityFrameworkCore.Infrastructure.SqlServerDbContextOptionsBuilder>? sqlServerOptionsAction = null]) (+ 2 overloads)
 Configures the context to connect to a Microsoft SQL Server database, but without initially setting any System.Data.Common.DbConnection or connection string.
 The connection or connection string must be set before the DbContext is used to connect to a database. Set a connection using RelationalDatabaseFacadeExtensions.SetDbConnection

Вместо да вложим този connection string вътре в UseSqlServer(), нека използваме appsettings.json.

```
appsettings.json
{
  "Logging": {
    "LogLevel": {
      "Default": "Warning"
    }
  },
  "AllowedHosts": "*",
  "MyKey": "Value of MyKey from appsettings.json",
  "ConnectionStrings": {
    "EmployeeDBConnection": "server=(localdb)\\MSSQLLocalDB;database=EmployeeDB;Trusted_Connection=true"
  }
}
```

name automatically installed with VS custom name integrated WS authentication
 JSON format

В класически Asp.Net, съхраняваме конфигурационна информация в web.config файл, който е в XML формат. В Asp.Net.Core обаче, имаме различни конфигурационни източници, един от които е appsettings.json, който ги съхранява в JSON формат.

EmployeeDB още не е създадена.

```
"ConnectionStrings": {
  "EmployeeDBConnection":
    "server=(localdb)\\MSSQLLocalDB;database=EmployeeDB;Trusted_Connection=true"
}
```

Сега трябва да го укажем в IConfiguration:

```
private IConfiguration _config; // auto generated private field;

public Startup(IConfiguration config) //+ auto MS namespace, parameter named config...

// This method gets called by the runtime. Use this method to add services to the container. ...
public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContextPool<AppDbContext>(
        options => options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection")));
    services.AddMvc(option => option.EnableEndpointRouting = false)
        .AddXmlSerializerFormatters();
    services.AddTransient<IEmployeeRepository, MockEmployeeRepository>();
}
```

services.AddDbContextPool<AppDbContext>(

options =>
 options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection"));

Using SQL Server with EF Core

```
public class Startup
{
    private IConfiguration _config;

    public Startup(IConfiguration config)
    {
        _config = config;
    }

    public void ConfigureServices(IServiceCollection services)
    {
        services.AddDbContextPool<AppDbContext>(
            options => options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection")));

        services.AddMvc().AddXmlSerializerFormatters();
        services.AddTransient<IEmployeeRepository, MockEmployeeRepository>();
    }

    // Rest of the code
}
```

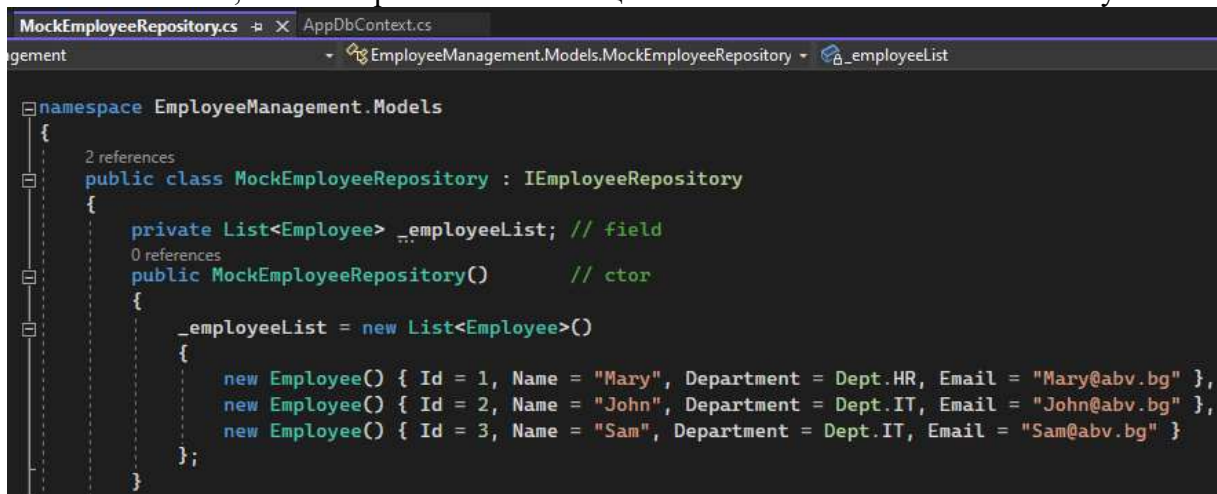
What is the difference

```
"server=(localdb)\\MSSQLLocalDB;database=EmployeeDB;Trusted_Connection=true"
```

```
Trusted_Connection=true
Integrated Security=SSPI
Integrated Security=true
```

- All the 3 settings specify the same thing
- Use Integrated Windows Authentication instead SQL Server authentication

Към момента, нашето приложение все още използва този hard code in-memory list:



```
namespace EmployeeManagement.Models
{
    2 references
    public class MockEmployeeRepository : IEmployeeRepository
    {
        private List<Employee> _employeeList; // field
        0 references
        public MockEmployeeRepository() // ctor
        {
            _employeeList = new List<Employee>()
            {
                new Employee() { Id = 1, Name = "Mary", Department = Dept.HR, Email = "Mary@abv.bg" },
                new Employee() { Id = 2, Name = "John", Department = Dept.IT, Email = "John@abv.bg" },
                new Employee() { Id = 3, Name = "Sam", Department = Dept.IT, Email = "Sam@abv.bg" }
            };
        }
    }
}
```

В следващото видео ще направим SqlServer repository, от което могат да се извличат и обработват данни чрез local server DB.

– Repository Pattern

Repository Pattern is an abstraction of Data Access Layer. It hides the details of how exactly the data is saved or retrieved from the underlying data source. The details of how the data is stored and retrieved is in the respective repository.

CRUD operations = Create, Retrieve, Update, Delete.

В най-простата си форма, Repository Patterns съдържа 2 участника: интерфейс + клас, който имплементира този интерфейс.

Към момента, ние работим с този

Employee.cs тип, затова сме нарекли интерфейса IEmployeeRepository.

ако работим с Product.cs тип, ще го наречем IProductRepository;

съответно за Customer.cs тип => ICustomerRepository.

```
public interface IEmployeeRepository
{
    2 references
    Employee GetEmployee (int Id); // method GetEmployee
    3 references
    IEnumerable<Employee> GetAllEmployee();
    2 references
    Employee Add(Employee employee);
}
```

Към момента нямаме всички CRUD операции. Искаме да добавим Update + Delete.

```
Employee Add(Employee employee);
Employee Update(Employee employee); Tab to accept
```

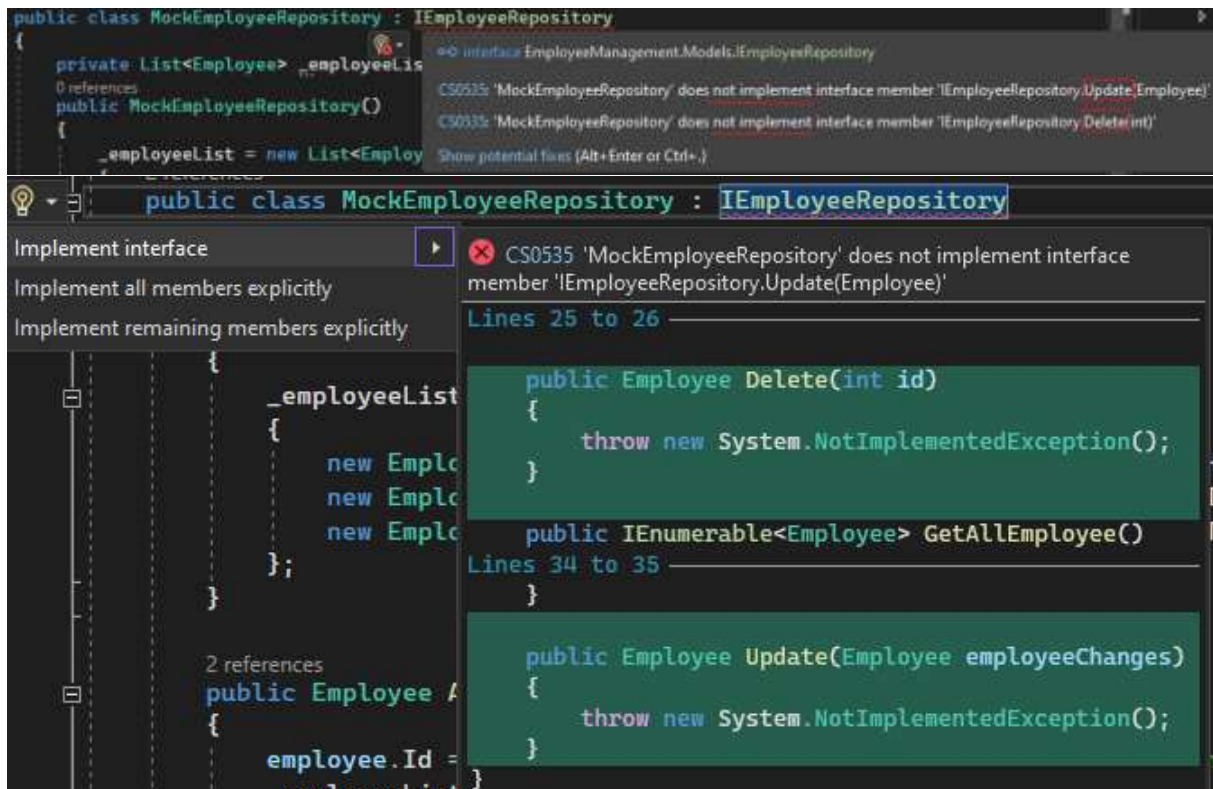
Intellisense предлага последователно Update, Delete

```
public interface IEmployeeRepository
{
    2 references
    Employee GetEmployee (int Id); // Read
    3 references
    IEnumerable<Employee> GetAllEmployee();
    2 references
    Employee Add(Employee employee); // Create
    0 references
    Employee Update(Employee employeeChanges); // Update
    0 references
    Employee Delete(int id); // Delete
    type name
    pass parameter
    type name
}
```

IEmployeeRepository interface supports the following operations:

- * Get all the employees
- * Get a single employee by id
- * Add a new employee
- * Update an employee
- * Delete an employee

Важният момент тук е, че Repository интерфейсът съдържа само операции, които са поддържани от този Repository. Той не съдържа детайли КАК тези операции се изпълняват. Тези подробности се намират в съответния Repository class, в нашия случай MockEmployeeRepository.cs



private List<Employee> _employeeList;

Първо искаме от private field _employeeList да изтрием елемент, затова извикваме FirstOrDefault, който намира елемента по подаденото id.

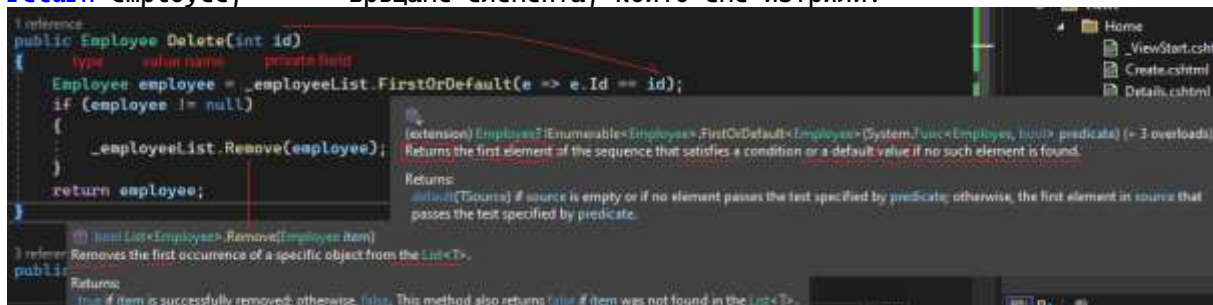
_employeeList.FirstOrDefault(e => e.Id == id);

После запазваме елемента, който е резултат от кода, като го съхраняваме в променлива, която наричаме employee, от тип Employee.

if (employee != null) – ако сме намерили такъв елемент

_employeeList.Remove(employee); – го премахваме от листа (частното поле).

return employee; – връщаме елемента, който сме изтрили.



```

public Employee Delete(int id)
{
    Employee employee = _employeeList.FirstOrDefault(e => e.Id == id);
    if (employee != null)
    {
        _employeeList.Remove(employee);
    }
    return employee;
}

```

За Update кодът е почти същият, затова го копираме и поставяме за леки промени – прихващаме Id на новата променлива, която е въведена employeeChanges. Нейните променливи се съхраняват в employee, като тя приема съответните стойности на въведените.

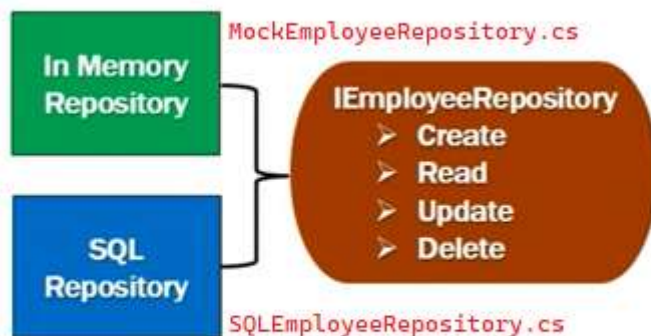

```

public Employee Update(Employee employeeChanges)
{
    Employee employee = _employeeList.FirstOrDefault(e => e.Id == employeeChanges.Id);
    if (employee != null)
    {
        employee.Name = employeeChanges.Name;
        employee.Email = employeeChanges.Email;
        employee.Department = employeeChanges.Department;
    }
}

public Employee Update(Employee employeeChanges)
{
    Employee employee = _employeeList.FirstOrDefault(
e => e.Id == employeeChanges.Id);
    if (employee != null)
    {
        employee.Name = employeeChanges.Name;
        employee.Email = employeeChanges.Email;
        employee.Department = employeeChanges.Department;
    }
    return employee;
}

```

Repository Pattern in ASP.NET Core



```

public class HomeController : Controller
{
    private readonly IEmployeeRepository _employeeRepository;
}

```

Искаме вместо в паметта, данните да се съхраняват в SqlServer, затова добавяме нов клас.

Models folder -> Add Class -> SQLEmployeeRepository.cs

Искаме той да имплементира интерфейса IEmployeeRepository, затова го извикваме:

```
SQLEmployeeRepository.cs* | IEmployeeRepository.cs* | Startup.cs* | MockEmployeeRepository.cs
EmployeeManagement | EmployeeManagement.Models | SQLEmployeeRepository.cs
0 references
public class SQLEmployeeRepository : IEmployeeRepository
{
    2 references
    public Employee Add(Employee employee)
    {
        throw new NotImplementedException();
    }
    1 reference
    public Employee Delete(int id)
    {
        throw new NotImplementedException();
    }
    3 references
    public IEnumerable<Employee> GetAllEmployee()
    {
        throw new NotImplementedException();
    }
    2 references
    public Employee GetEmployee(int Id)
    {
        throw new NotImplementedException();
    }
}
```

За да може този клас да работи с данни от SQL Server DB, ни е необходима връзка с DbContext class, която имплементирахме като AppDbContext.cs по-рано.

```
SQLEmployeeRepository.cs* | AppDbContext.cs* | IEmployeeRepository.cs* | MockEmployeeRepository.cs
EmployeeManagement | EmployeeManagement.Models | AppDbContext.cs
using Microsoft.EntityFrameworkCore;

namespace EmployeeManagement.Models
{
    3 references
    public class AppDbContext : DbContext
    {
        0 references
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        {
        }
        0 references
        public DbSet<Employee> Employees { get; set; }
    }
}
```

За да инжектираме AppDbContext в SQLEmployeeRepository, ни е необходим ctor:

```

1 reference
public class SQLEmployeeRepository : IEmployeeRepository
{
    private readonly AppDbContext context; ← auto field
    0 references
    public SQLEmployeeRepository(AppDbContext context) ← inject class, name of parameter
    {
        this.context = context; ← ctrl.
    }
}

```

След това ни трябва имплементации за всички CRUD methods.

```

SQLEmployeeRepository.cs  MockEmployeeRepository.cs
EmployeeManagement.Models.SQLEmployeeRepository  Delete(int id)
private readonly AppDbContext context;

0 references
public SQLEmployeeRepository(AppDbContext context)
{
    this.context = context;
}

2 references
public Employee Add(Employee employee)
{
    context.Employees.Add(employee); ← public DbSet<Employee> Employees { get; set; }
    context.SaveChanges(); // method for DB
    return employee;
}

1 reference
public Employee Delete(int id)
{
    Employee employee = context.Employees.Find(id); // Find(x => x.Id == id).Delete();
    if(employee != null) ← if we found it
    {
        context.Employees.Remove(employee);
        context.SaveChanges();
    }
    return employee;
}

public IEnumerable<Employee> GetAllEmployee()
{
    return context.Employees;
}

2 references
public Employee GetEmployee(int Id)
{
    return context.Employees.Find(Id);
}

1 reference
public Employee Update(Employee employeeChanges)
{
    var employee = context.Employees.Attach(employeeChanges);
    employee.State = Microsoft.EntityFrameworkCore.EntityState.Modified;
    context.SaveChanges();
    return employeeChanges;
}

```

enum Microsoft.EntityFrameworkCore.EntityState
The state in which an entity is being tracked by a context.

public class SQLEmployeeRepository : IEmployeeRepository

```

{
    private readonly AppDbContext context;

    public SQLEmployeeRepository(AppDbContext context)
    {
        this.context = context;
    }

    public Employee Add(Employee employee)
    {
        context.Employees.Add(employee);
        context.SaveChanges(); // method for DB
        return employee;
    }

    public Employee Delete(int id)
    {
        Employee employee = context.Employees.Find(id);
        //Find(x => x.Id == id).Delete();
        if(employee != null)
        {
            context.Employees.Remove(employee);
            context.SaveChanges();
        }
        return employee;
    }

    public IEnumerable<Employee> GetAllEmployee()
    {
        return context.Employees;
    }

    public Employee GetEmployee(int Id)
    {
        return context.Employees.Find(Id);
    }

    public Employee Update(Employee employeeChanges)
    {
        var employee = context.Employees.Attach(employeeChanges);
        employee.State = Microsoft.EntityFrameworkCore.EntityState.Modified;
        context.SaveChanges();
        return employeeChanges;
    }
}

```


Repository Pattern in ASP.NET Core



```
public void ConfigureServices(IServiceCollection services)
{
    // Other Code...
    services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();
}
```

Към момента имаме 2 имплементации на Repository – Mock (in-memory) и SQL. В HomeController не е указано коя да се използва, виждаме само `private readonly IEmployeeRepository _employeeRepository;` Коя да избере контролерът, се задава в Startup.cs -> ConfigureServices

```
Startup.cs
public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContextPool<AppDbContext>(
        options => options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection")));

    services.AddMvc(option => option.EnableEndpointRouting = false)
        .AddXmlSerializerFormatters();
    services.AddTransient<IEmployeeRepository, MockEmployeeRepository>();
}
```

Към момента сме задали Mock.

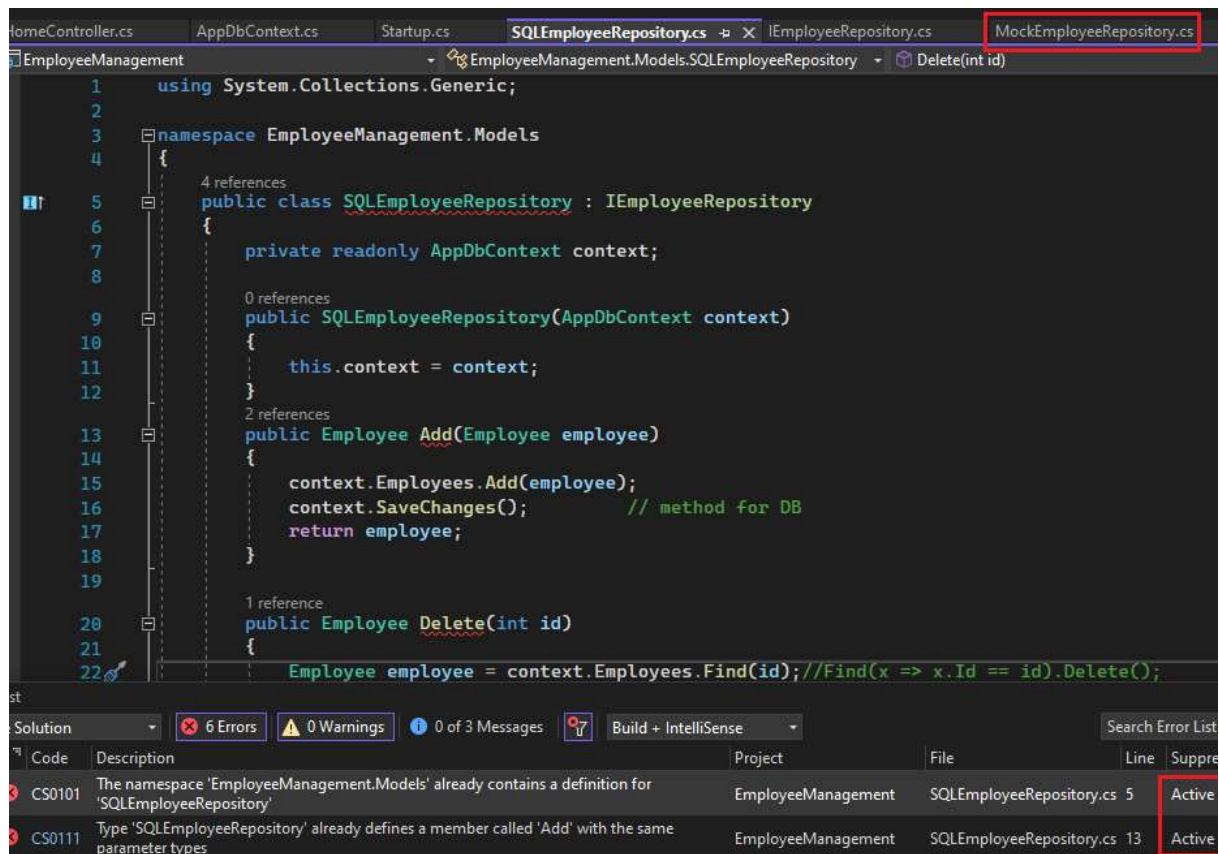
```
services.AddTransient<IEmployeeRepository, SQLEmployeeRepository>();
```

С промяна в тази една линия код, нашето приложение вече използва БД. Това е силата и гъвкавостта на dependency injection.

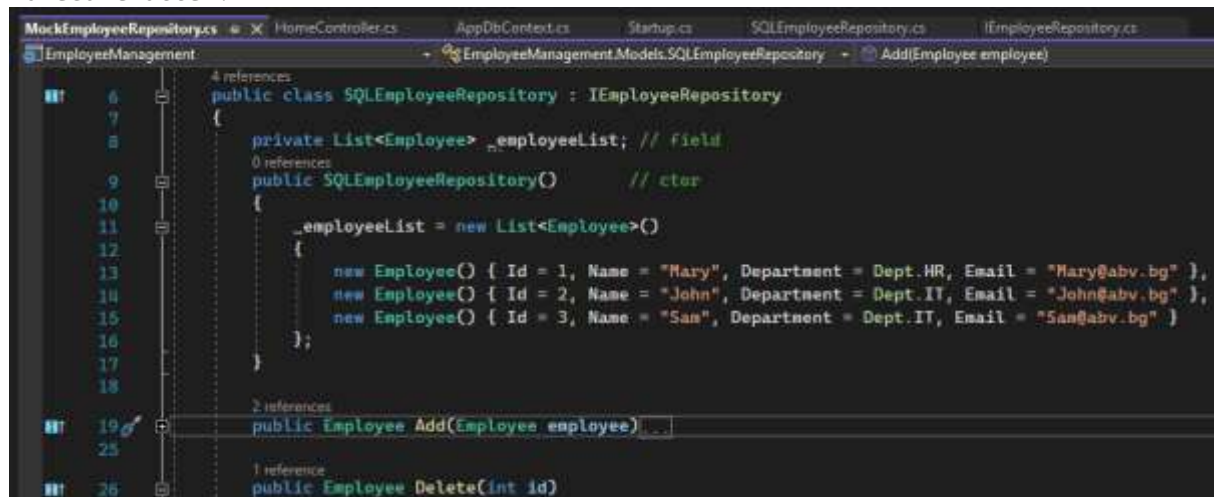
```
services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();
services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();
```

Променяме и връзката между двата Repository на Scoped, за регистрация на нашата услуга service, защото искаме инстанцията на IEmployeeRepository class да бъде активна и достъпна по време на целия обхват на дадена HTTP заявка. Когато нова заявка бъде подадена, искаме да се създаде нова инстанция, която от своя страна също да бъде активна и достъпна по време на целия обхват на тази HTTP заявка.

Предимствата са, че кодът е много по-изчистен и гъвкав, особено, когато се наложи използване на OracleDB. При тестване е много по-удачно да се смени и използва in-memory, преди да се включи реална BD, което също става с лесна промяна в един ред код.



Излязоха 6 грешки, които помислих, че са от отворения Мокс, но не са. Маркира повторените CRUD methods в двата файла Мокс и SQL като дублиращи се, въпреки, че вече използвам само SQL навсякъде, а Мокс не е упоменат никъде, дори в HomeController.



localhost:24077/Home/Create

List Create

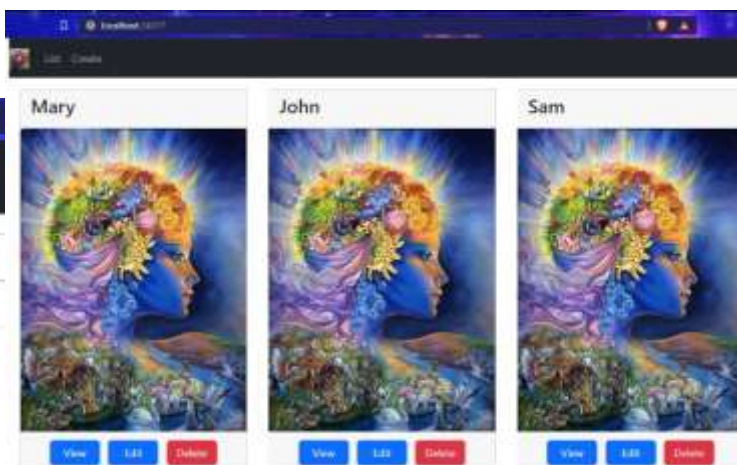
Name: David

Office Email: david@abv.bg

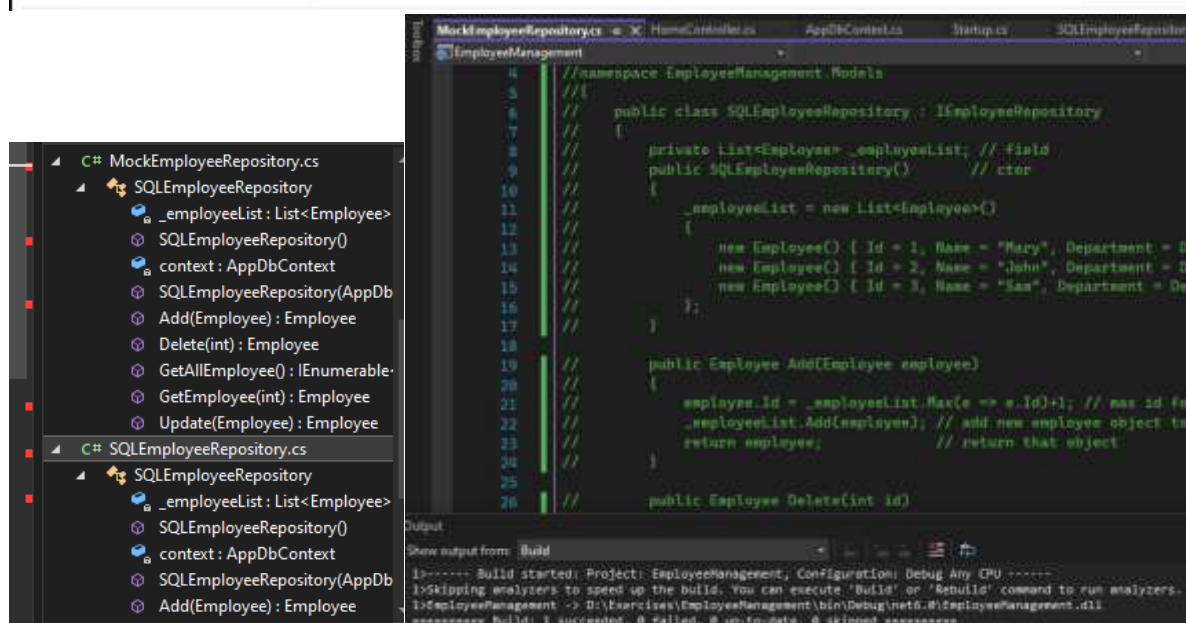
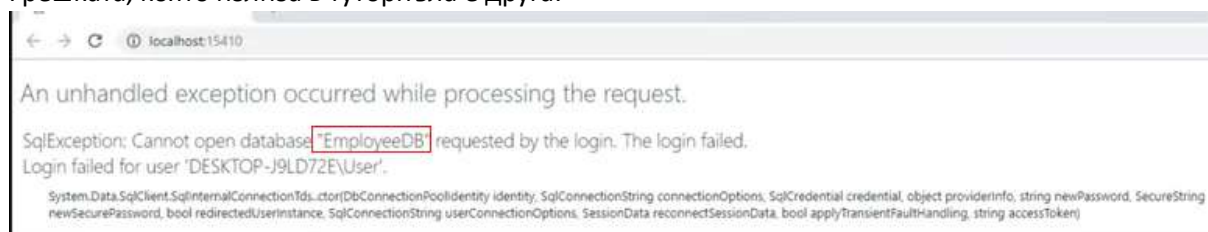
Department: IT

Create

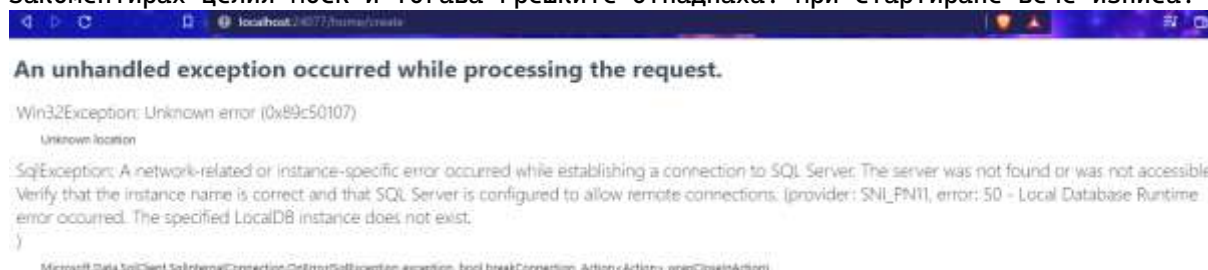
Total Employees Count = 4



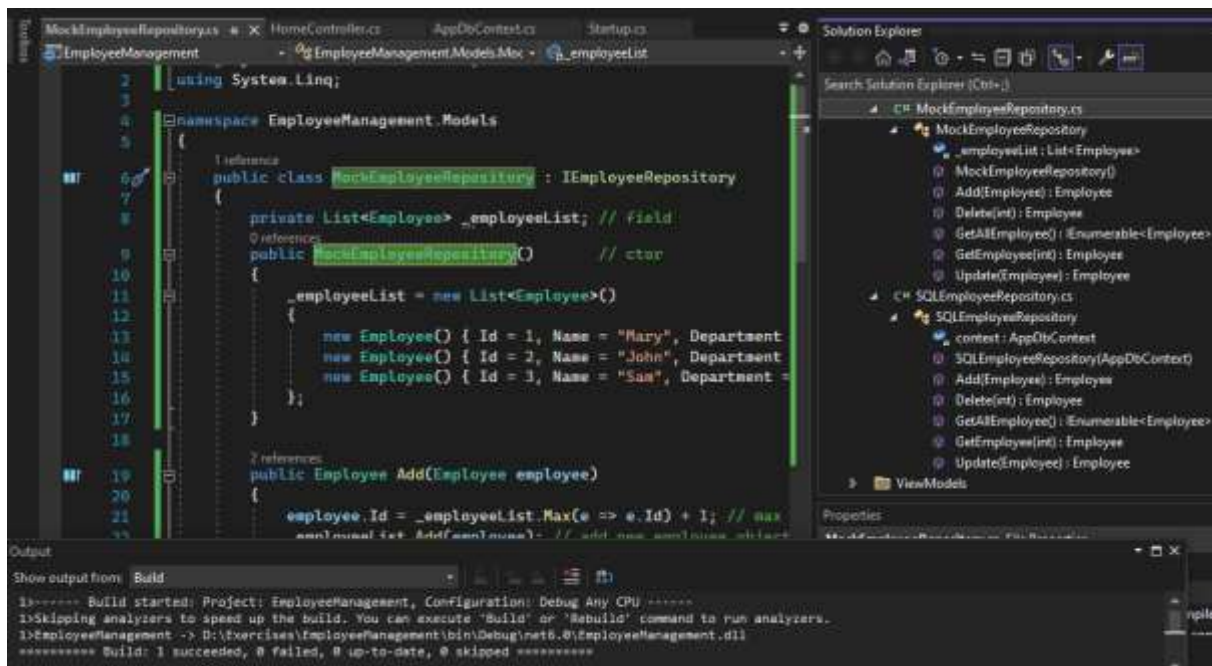
Създава нов служител, но не го съхранява нито в In-memory, защото никъде не се извиква Mock, нито с SQL, защото EmployeeDB не е създадена.
Грешката, която излиза в туториъла е друга:



Закоментирах целия Mock и тогава грешките отпаднаха. При стартиране вече изписа:



Като отворих Mock, се оказа, че на 2 места автоматично е бил сменен с SQL



Изписаната грешка, че локална БД липсва е обяснима – все още не сме я създали.

50 – Entity Framework Core migrations

Entity Framework Core Migrations

Migration keeps the database schema and application model classes in sync

За да създадем използваема БД, единият начин е първо да се създаде миграция към нея, а след това самата БД.

appsettings.json

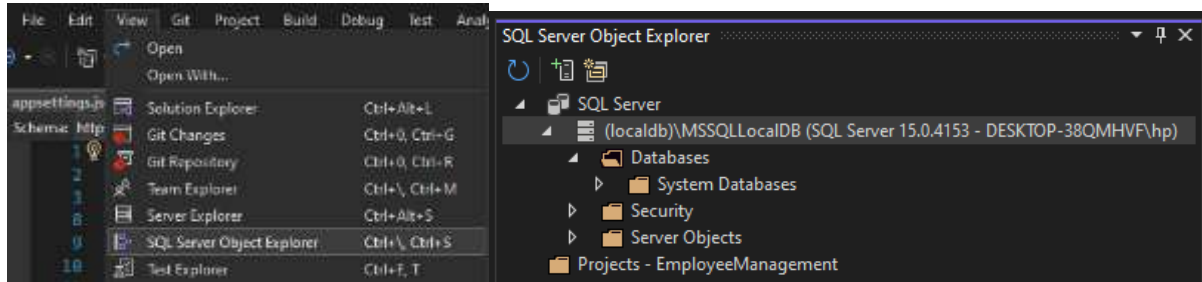
```
"ConnectionStrings": {
  "EmployeeDBConnection": "server=(localdb)\\MSSQLLocalDB;database=EmployeeDB;Trusted_Connection=true"
}

"ConnectionStrings": {
  "EmployeeDBConnection":
"server=(localdb)\\mssqllocaldb;database=EmployeeDB;Trusted_Connection=True;
MultipleActiveResultSets=true"
}
```

В новите версии е с повече код:

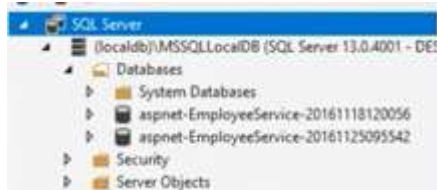
```
"server=(localdb)\\mssqllocaldb;database=EmployeeDB;Trusted_Connection=True;MultipleActiveResultSets=true"
```


View -> SQL Server Object Explorer:



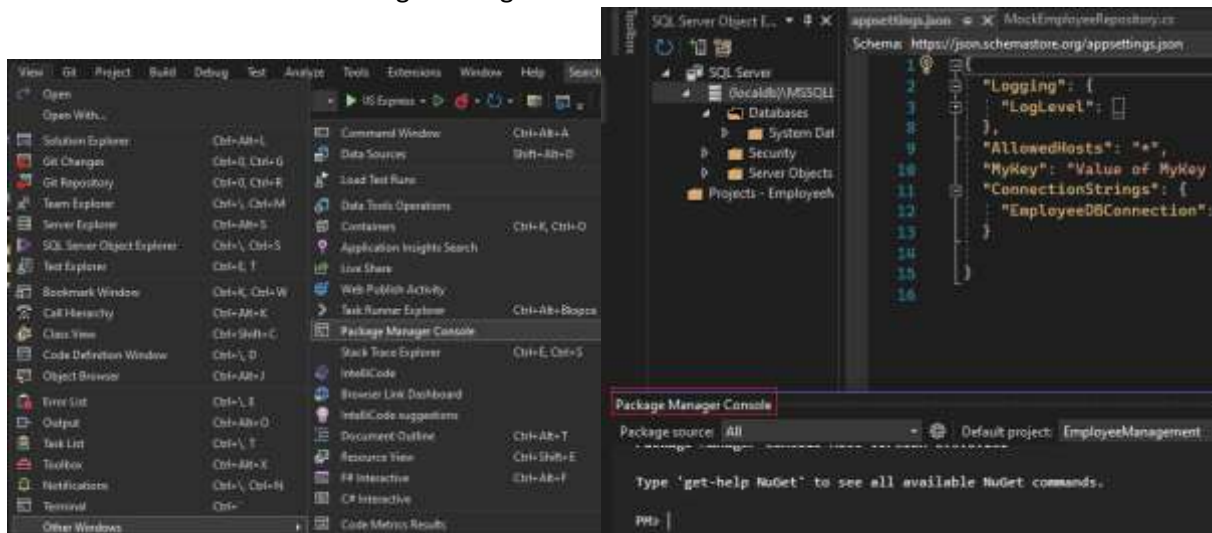
Не е създадена EmployeeDB.

При мен обаче липсват инсталации, които има в туториъла:

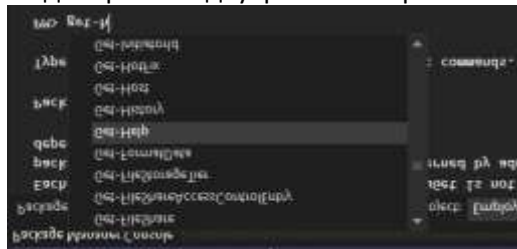


За да създадем миграция, може да използваме Package Manager Console или CLI.

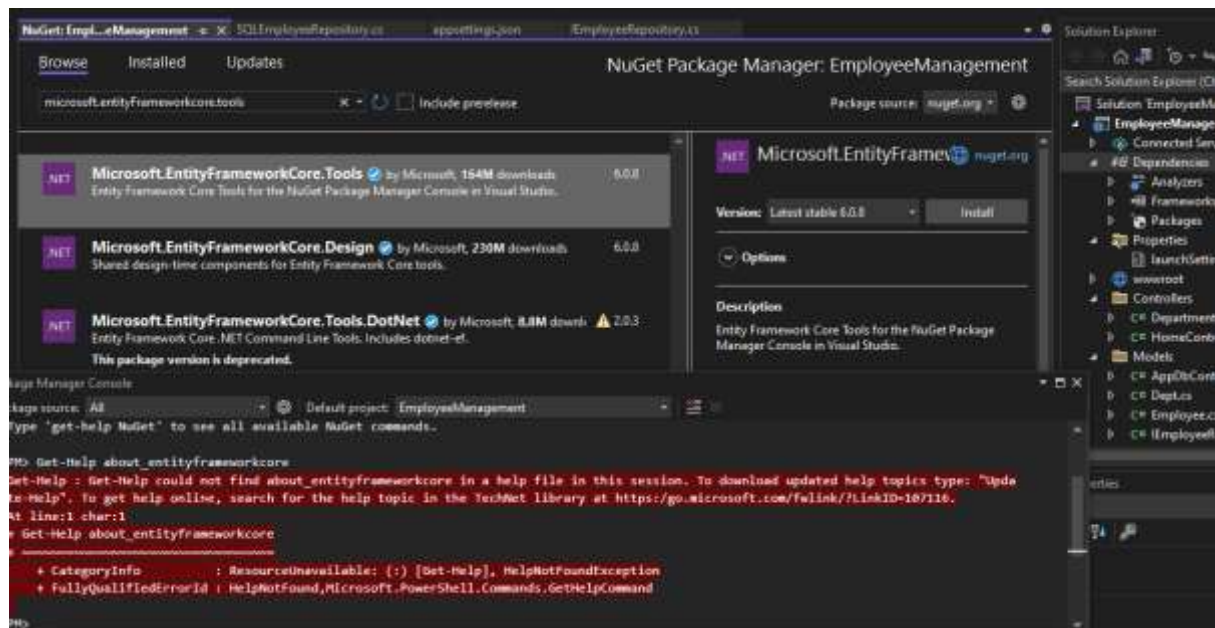
View -> Other Windows -> Package Manager Console:



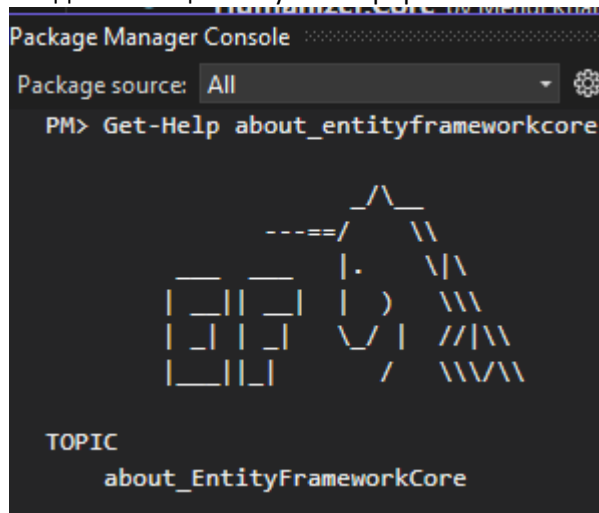
С еднократно и двукратно избиране на Tab се включва и избира от Intellisense:



Командата `get-help about_entityframeworkcore`



След инсталацията му вече проработи:




```

Package Manager Console
Package source: All [v] [g] Default project: EmployeeManagement [v] [x] [m]
-----
Cmdlet      Description
-----
Add-Migration      Adds a new migration.

Bundle-Migration   Creates an executable to update the database.

Drop-Database      Drops the database.

Get-DbContext      Lists and gets information about available DbContext types.

Get-Migration      Lists available migrations.

Optimize-DbContext Generates a compiled version of the model used by the DbContext.

Remove-Migration   Removes the last migration.

Scaffold-DbContext Scaffolds a DbContext and entity types for a database.

Script-DbContext   Generates a SQL script from the DbContext. Bypasses any migrations.

Script-Migration   Generates a SQL script from migrations.

Update-Database    Updates the database to a specified migration.

SEE ALSO
Add-Migration
Bundle-Migration
Drop-Database
Get-DbContext
Get-Migration
Optimize-DbContext
Remove-Migration
Scaffold-DbContext
Script-DbContext
Script-Migration
Update-Database

```

Най-често използваните команди са:

<code>get-help about_entityframeworkcore</code>	Provides entity framework core help
<code>Add-Migration</code>	Adds a new migration
<code>Update-Database</code>	Updates the database to a specified migration

Може да използваме `get-help` пред всичко, за което искаме допълнителна информация, например: `get-help Add-Migration`.

```

Package Manager Console
Package source: All [v] [g] Default project: EmployeeManagement [v] [x] [m]
NAME
Add-Migration

SYNOPSIS
Adds a new migration.

SYNTAX
Add-Migration [-Name] <String> [-OutputDir <String>] [-Context <String>] [-Project <String>] [-StartupProject <String>] [-Namespace <String>] [-Args <String>] [<CommonParameters>]

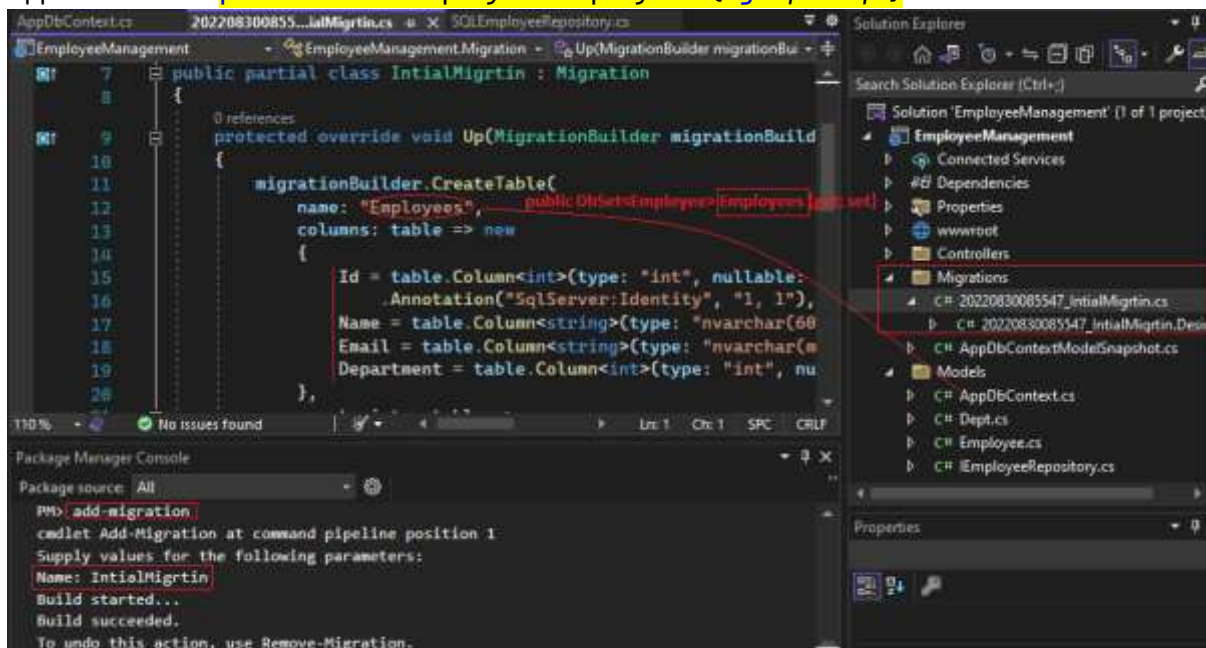
```

Ако се подаде само `add-migration`, ще изиска име:

```
Package Manager Console
Package source: All
Default project:

PM> add-migration
cmdlet Add-Migration at command pipeline position 1
Supply values for the following parameters:
Name:
```

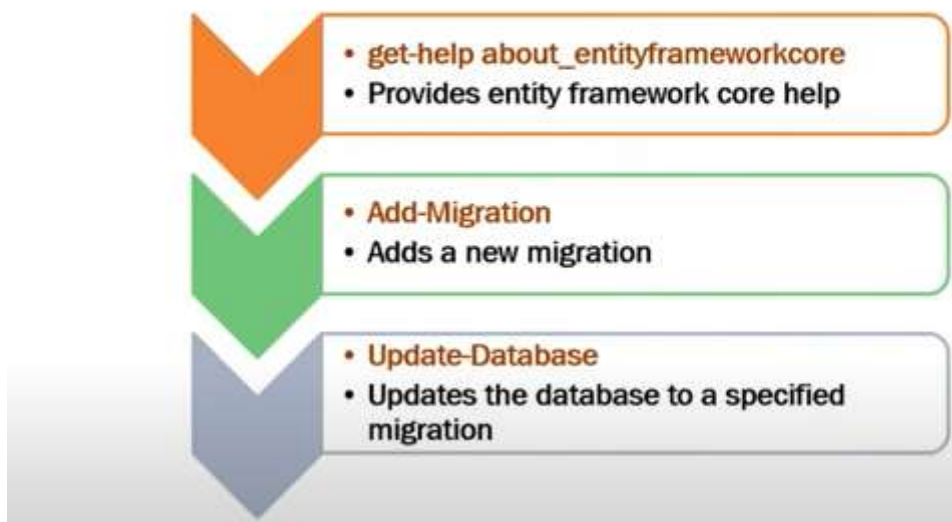
В случая, понеже е първоначална: **InitialMigration**, при което автоматично се създава папка Migrations, която съдържа БД таблица с Employee properties, името на което сме задали в AppDbContext.cs **public DbSet<Employee> Employees { get; set; }**



Вижда се, че таблицата е извлякла дори MaxLength(60) за Name от Employee.cs, както и че типът данни на Department е int.

Създали сме миграцията, но за да се изпълни, трябва да използваме команда **Update-Database**

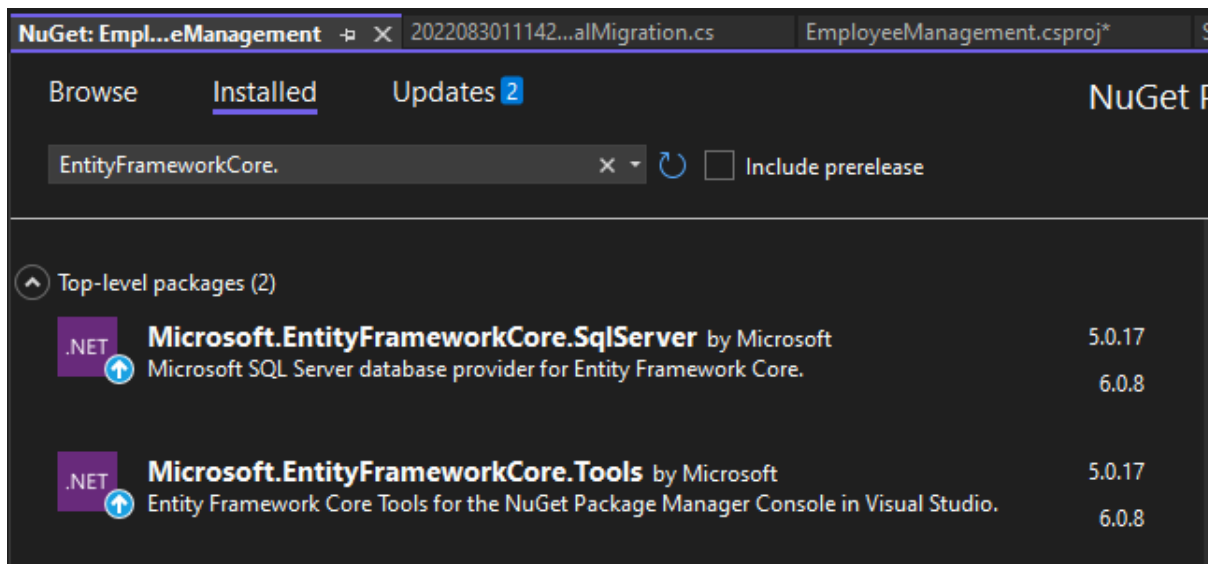
Entity Framework Core Migrations



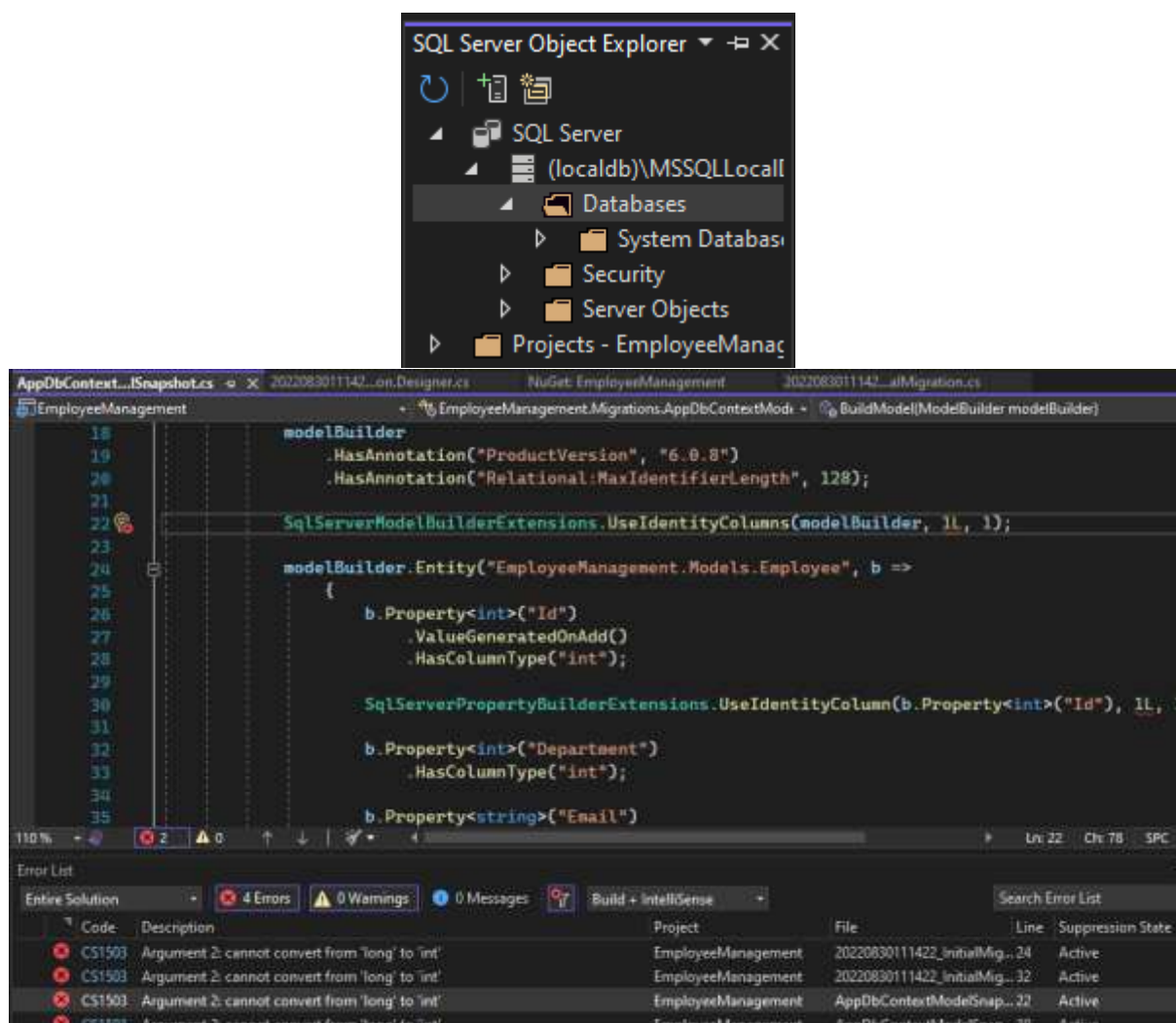
Понеже отново извежда грешка с SQL, инсталирам по-старите версии за 5.0

```
EmployeeManagement | 2022083011142...alMigration.cs | EmployeeMan...ement.csproj* - X
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net5.0</TargetFramework>
    <AspNetCoreHostingModel>InProcess</AspNetCoreHostingModel>
    <UseAppHost>False</UseAppHost>
  </PropertyGroup>
```

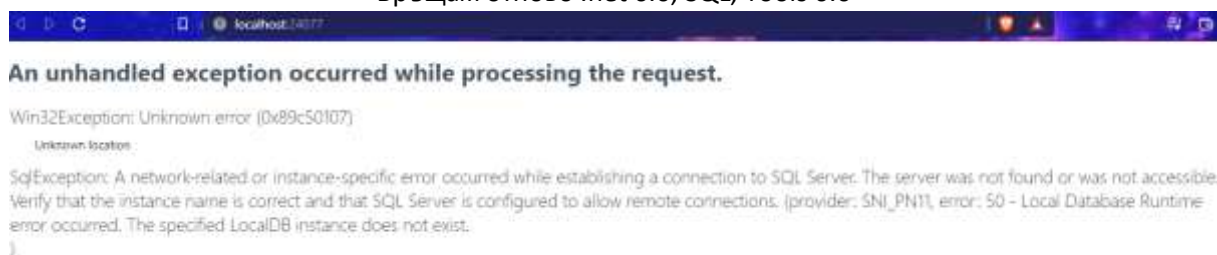


Отново не са инсталирани:



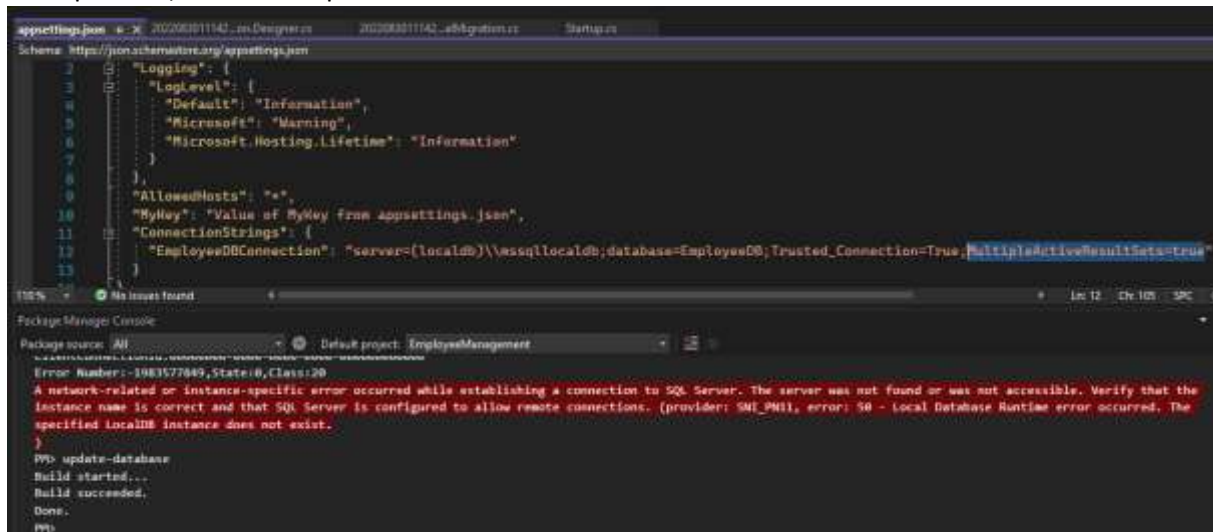
Сайтът стартира без грешка, но не може да добавя към списъка служители.

Връщам отново .net 6.0, SQL, Tools 6.0



При нея трябва да се спомене името на миграцията, но понеже при нас има само една – InitialMigration, използва само нея. Ако не се упомене никоя, използва последно създадената. В кода на конзолата се вижда, че първо се създава Create Database [EmployeeDb], а после таблицата, която е в нея, което може да се потвърди и в SQL Server object explorer. Вече страницата се зарежда, но е празна, защото няма данни в нея. Това се вижда от д.б. dbo.Employees – View Data, в който таблицата е без попълнени все още данни. Служители могат да се добавят интерактивно през сайта или чрез таблицата.

Намерих защо не ми се Update-Databases



The screenshot shows the Visual Studio IDE. The top pane displays the `appsettings.json` file with the following content:

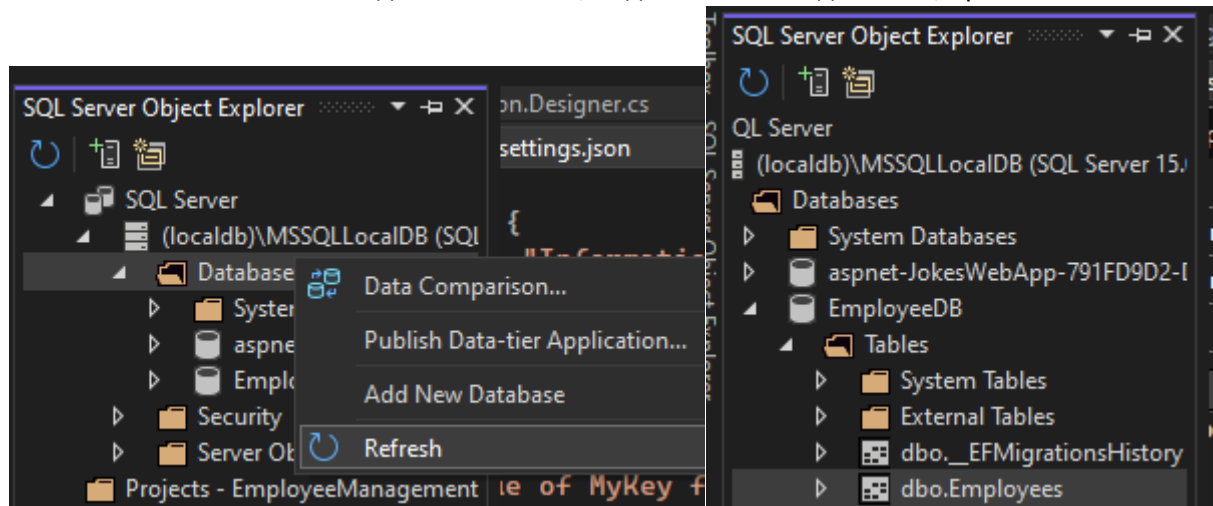
```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*",
  "MyKey": "Value of MyKey from appsettings.json",
  "ConnectionStrings": {
    "EmployeeDBConnection": "server=(localdb)\\mssqllocaldb;database=EmployeeDB;Trusted_Connection=True;MultipleActiveResultSets=true"
  }
}
```

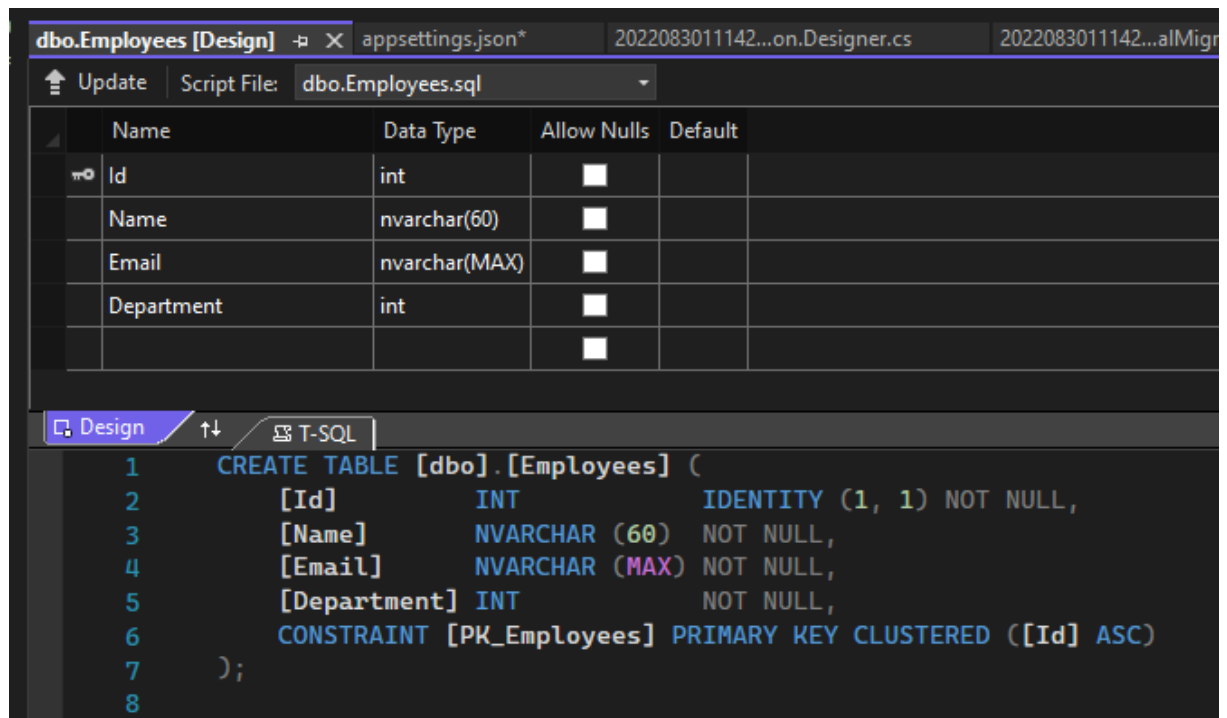
The bottom pane shows the Package Manager Console output for the `EmployeeManagement` project:

```
Package source: All
Default project: EmployeeManagement
Error Number: -198357889, State: 0, Class: 20
A network-related or instance-specific error occurred while establishing a connection to SQL Server. The server was not found or was not accessible. Verify that the instance name is correct and that SQL Server is configured to allow remote connections. (provider: SQLNCLI, error: 50 - Local Database Runtime error occurred. The specified localDB instance does not exist.)
PDB: update-database
Build started...
Build succeeded.
Done.
PDB:
```

"server=(localdb)\\mssqllocaldb;database=EmployeeDB;Trusted_Connection=True;MultipleActiveResultSets=true"

Много е важно да се Refresh DB, за да се появи създаването/Update:





Работи, БД е занулена, bootstrap в случая се е разместил:

The screenshot shows a web application running on localhost:24077. The application has a dark theme and a sidebar with a hamburger menu icon. The main content area displays a form titled 'List' and 'Create'.

The form fields are:

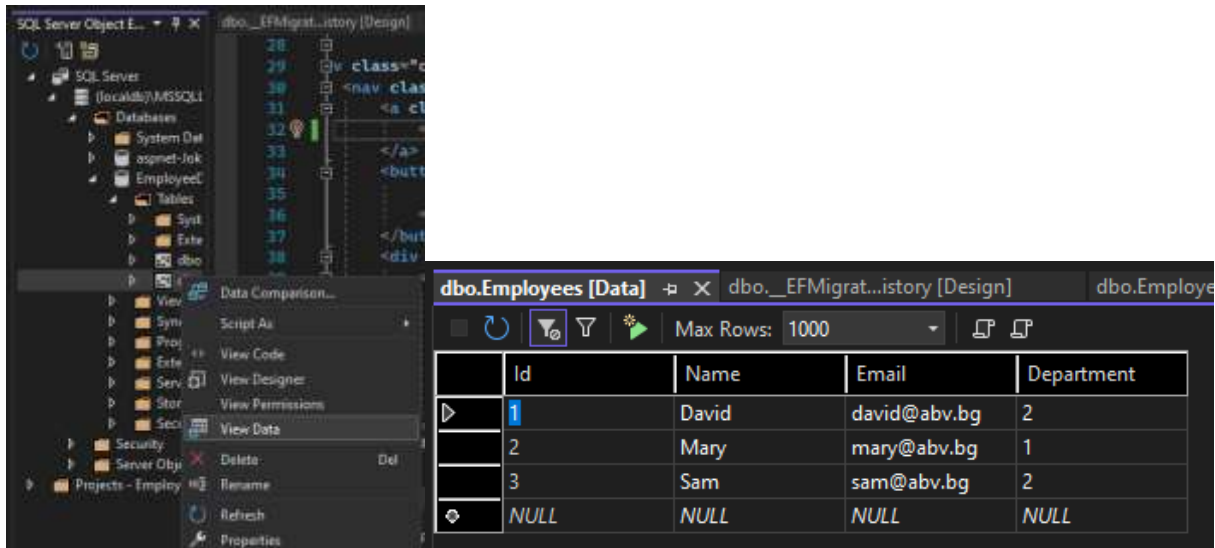
- Name:
- Office Email:
- Department:

Below the form is a blue 'Create' button.

At the bottom of the form, it says 'Total Employees Count = 0'.

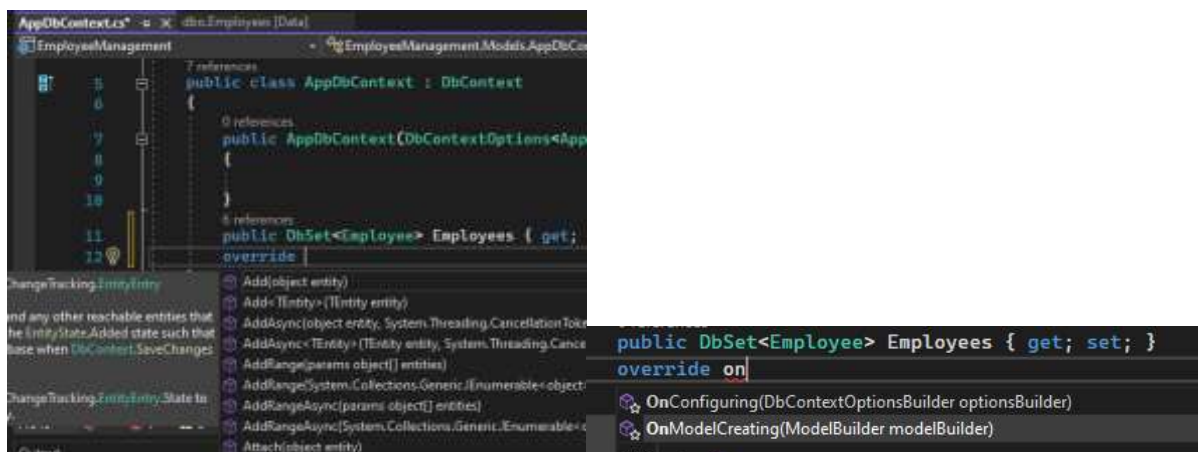
51 - Entity framework core seed data

Table -> View data, въвеждам малко данни през сайта.



	Id	Name	Email	Department
	1	David	david@abv.bg	2
	2	Mary	mary@abv.bg	1
	3	Sam	sam@abv.bg	2
	NULL	NULL	NULL	NULL

Другият начин е през AppDbContext, команда **override** -> **OnModelCreating**



```
public class AppDbContext : DbContext
{
    public DbSet<Employee> Employees { get; set; }
    override
    {
        OnModelCreating(ModelBuilder modelBuilder)
    }
}
```

автокод:

cbzalhawasy;'mxvnm,./vbnm,./qs

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
}
```

В конкретния случай, команда `base.OnModelCreating(modelBuilder);` не ни върши работа, затова я изтриваме.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Employee>().HasData(
        new Employee {
            Id = 1,
            Name = "Mark",
            Department = Dept.IT,
            Email = "mark@abv.bg"
        }
    );
}
```

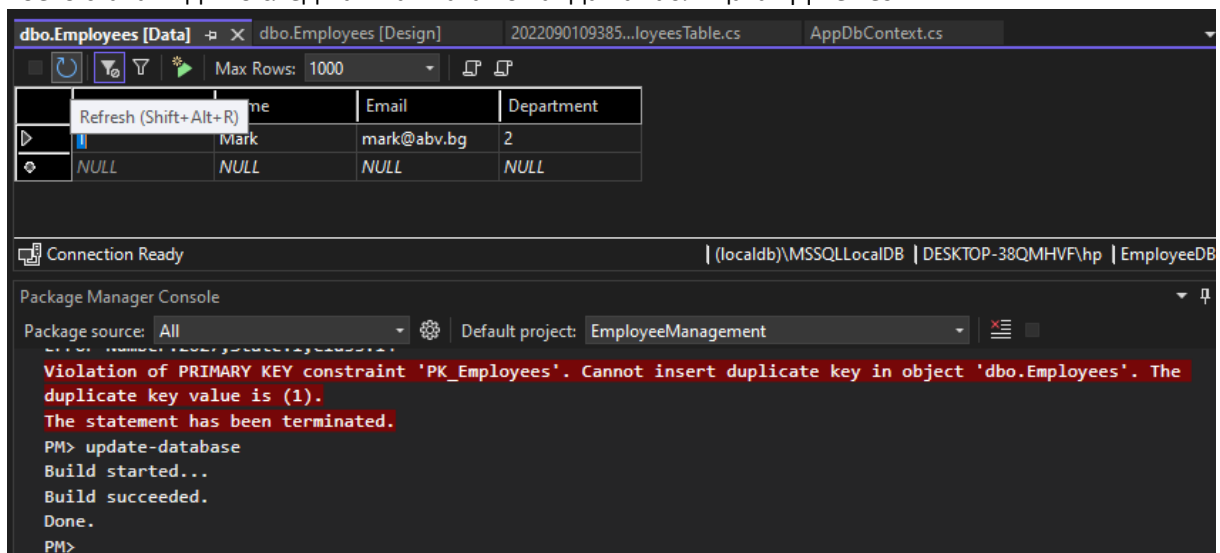
```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Employee>().HasData(
        new Employee {
            Id = 1,
            Name = "Mark",
            Department = Dept.IT,
            Email = "mark@abv.bg"
        }
    );
}
```

За да вмъкнем данните в БД, ни е необходима нова миграция:

View->Other windows -> Package Migrations ->

add-migration SeedEmployeesTable, update-database.

Нарочно въведох Id = 1, за да видя дали ще има презаписване, но при update изведе грешка, затова изтрих таблицата с данните и оставих само AppDbContext. Новият служител се въведе, което става видимо след най-важната команда на таблицата БД Refresh:



The screenshot shows the SQL Server Enterprise Designer interface. The top pane displays the 'dbo.Employees' table with columns: Id, Name, Email, and Department. The table contains one record with Id=2, Name=Mark, Email=mark@abv.bg, and Department=2. The bottom pane shows the Package Manager Console with the following output:

```
PM> update-database
Build started...
Build succeeded.
Done.
PM>
```

The error message in the console is: "Violation of PRIMARY KEY constraint 'PK_Employees'. Cannot insert duplicate key in object 'dbo.Employees'. The duplicate key value is (1). The statement has been terminated."

това се отразява и в браузъра.

Ако сме допуснали грешка, въвели сме Mark, вместо Mary, тогава редактираме C-кода и правим нова миграция. HasData() позволява въвеждане на много служители.

```

modelBuilder.Entity<Employee>().HasData(
    new Employee {
        Id = 1,
        Name = "Mary",
        Department = Dept.IT,
        Email = "mary@abv.bg"
    },
    new Employee {
        Id = 2,
        Name = "John",
        Department = Dept.HR,
        Email = "john@abv.bg"
    }
);

```

	Refresh (Shift+Alt+R)	Name	Email	Department
1		Mary	mary@abv.bg	2
2		John	john@abv.bg	1
		NULL	NULL	NULL

Add-migration AlterEmployeeSeedData, получавам съобщение за презаписване на БД, update.

An operation was scaffolded that may result in the loss of data. Please review the migration for accuracy. To undo this action, use Remove-Migration.



Колкото повече hard code добавяме като C data -> AppDbContext, толкова по-разпилян ще е кодът.

```

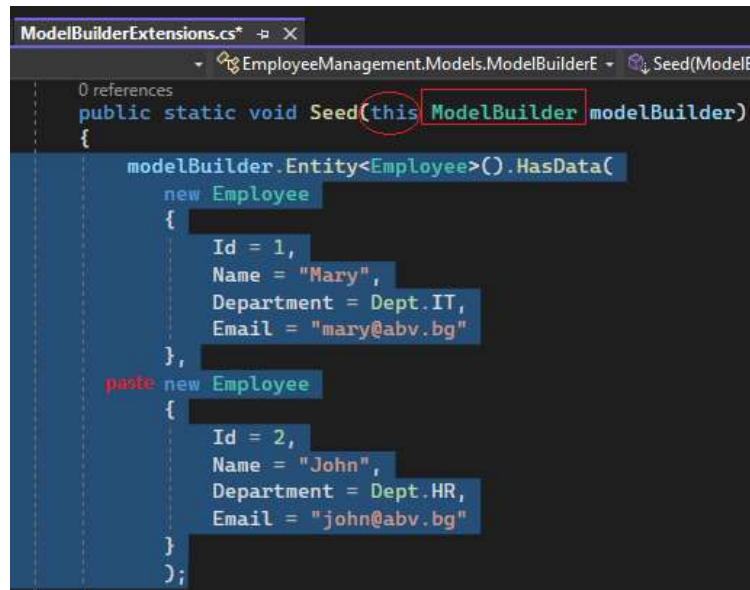
AppDbContext.cs
EmployeeManagement.Models.AppDbContext
OnModelCreating(ModelBuilder modelBuilder)
6 references
public DbSet<Employee> Employees { get; set; }
0 references
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Employee>().HasData(
        new Employee {
            Id = 1,
            Name = "Mary",
            Department = Dept.IT,
            Email = "mary@abv.bg"
        },
        new Employee {
            Id = 2,
            Name = "John",
            Department = Dept.HR,
            Email = "john@abv.bg"
        }
    );
}

```

За да се структурира по-добре, може да се премести в допълнителен modelBuilder class.

Папка Models->Add-> Class->ModelBuilderExtensions

За Extensions е необходим static, освен това, за разширенията е необходимо да се посочи откъде са, чрез ключова дума this и името на класа.



```
ModelBuilderExtensions.cs*
EmployeeManagement.Models.ModelBuilderE
Seed(ModelB

0 references
public static void Seed(this ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Employee>().HasData(
        new Employee
        {
            Id = 1,
            Name = "Mary",
            Department = Dept.IT,
            Email = "mary@abv.bg"
        },
        new Employee
        {
            Id = 2,
            Name = "John",
            Department = Dept.HR,
            Email = "john@abv.bg"
        }
    );
}
```

```
public static void Seed(this ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Employee>().HasData(
        new Employee
        {
            Id = 1,
            Name = "Mary",
            Department = Dept.IT,
            Email = "mary@abv.bg"
        },
        new Employee
        {
            Id = 2,
            Name = "John",
            Department = Dept.HR,
            Email = "john@abv.bg"
        }
    );
}
```

На променливата modelBuilder присвояваме class Seed, който е разширение.


```
AppDbContext.cs
using Microsoft.EntityFrameworkCore;

namespace EmployeeManagement.Models
{
    9 references
    public class AppDbContext : DbContext
    {
        0 references
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        {
        }
        6 references
        public DbSet<Employee> Employees { get; set; }
        0 references
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Seed();
        }
    }
}
```

```
using Microsoft.EntityFrameworkCore;
```

```
namespace EmployeeManagement.Models
{
```

```
    public class AppDbContext : DbContext
    {
```

```
        public AppDbContext(DbContextOptions<AppDbContext> options) :
base(options)
        {
```

```
        }
        public DbSet<Employee> Employees { get; set; }
```

```
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Seed();
        }
    }
```

```
}
```

Entity Framework Core Seed Data

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    { }

    public DbSet<Employee> Employees { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Employee>().HasData(
            new Employee
            {
                Id = 1, Name = "Mark", Department = Dept.IT,
                Email = "mark@pragimtech.com"
            }
        );
    }
}
```

52 – Keeping domain models and database schema in sync

Искаме да има и поле за снимка към Employee, освен id, name, email, department.

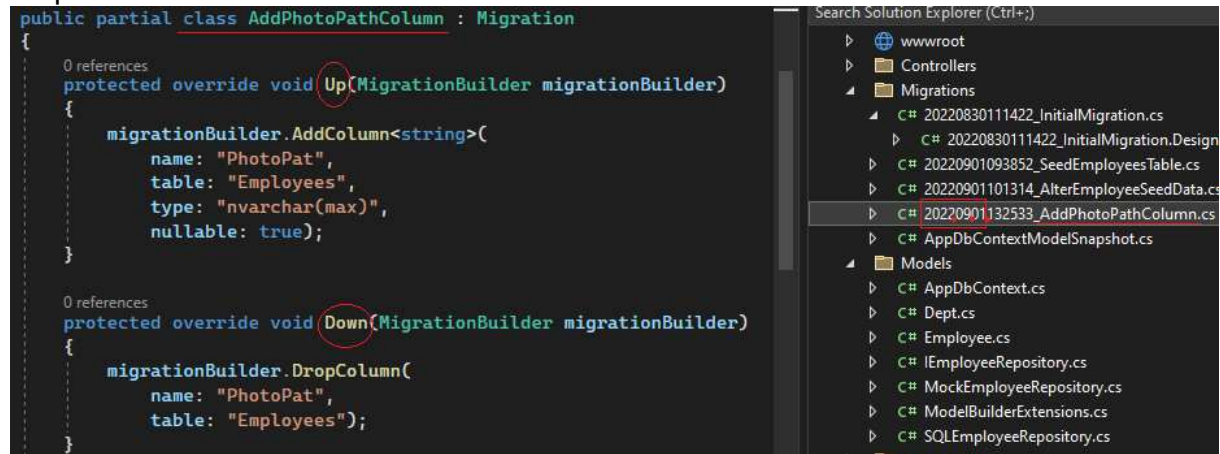
За целта към Employee.cs добавяме

```
public string PhotoPath { get; set; }
```

Все още обаче, в БД няма колона за нея. За целта трябва да добавим миграция.

С демонстративна цел правим печатна грешка, като премахваме PhotoPat_.

View->Other Windows -> Package Manager -> add-migration AddMigrationPhotoColumn -> update-database

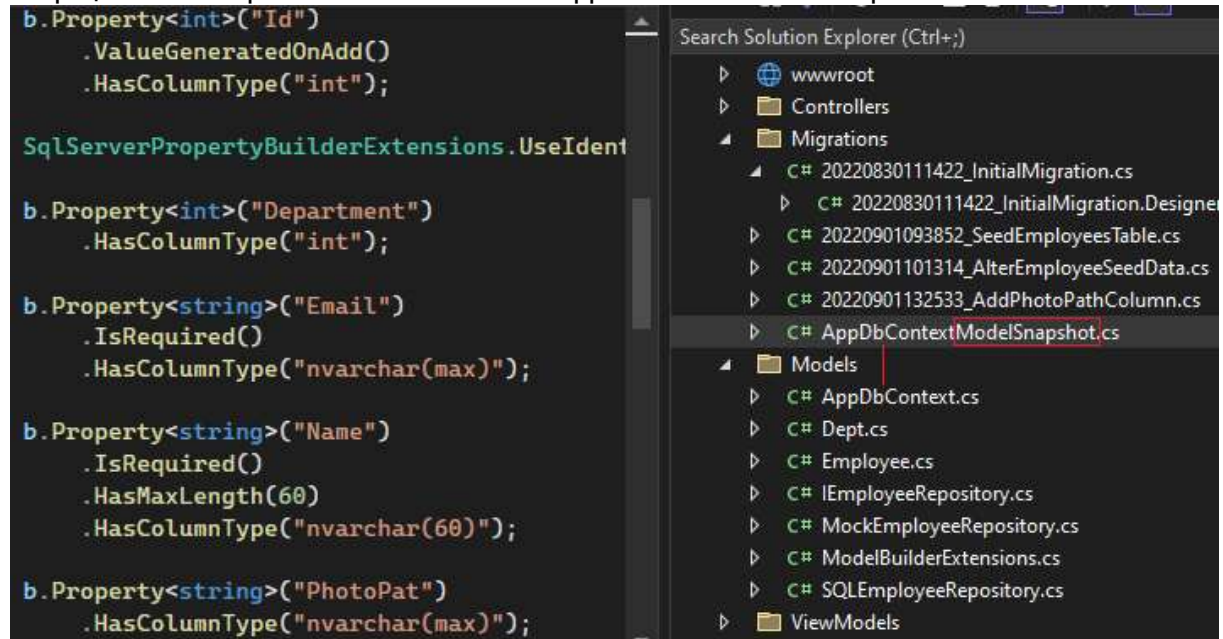


Имената на миграциите започват с датата на създаване.

Името на класа в тях съвпада с името на миграцията и винаги съдържа 2 метода:

Up+Down. Up съдържа промените, които се прилагат, а down ги възстановява до предходното състояние като undo, ако се наложи. Едновременно с това, промяната в

миграциите се отразява автоматично в App.DbContextModelSnapShot.cs



AppDbContextSnapShoot.cs = AppDbContext class + SnapShoot

Създава се още с първата миграция и отразява автоматично всички промени в миграциите. EF Core използва този файл за да определи всички промени.

С демонстративна цел добавяме ново свойство в Employee.cs

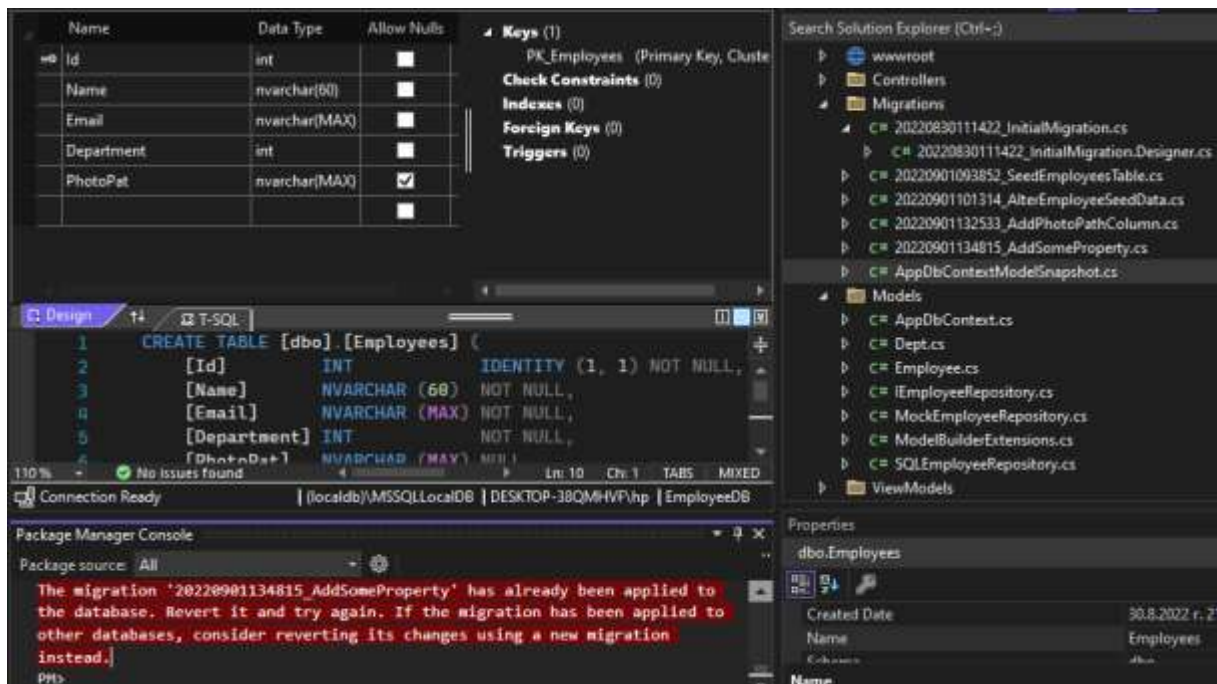
```
public int SomeProperty { get; set; }
```

и нова миграция AddSomeProperty за него. Всеки път, когато добавяме нова миграция, ModelSnapshot се ъпдейтва.



Нека премахнем миграция remove-migration

Ако забравя след създаването да дам update-migration, връща грешка, че току-що е добавена и не може да се премахне.



How do I revert an applied migration in ef core?

If the migration has been applied to other databases, consider reverting its changes using a new migration.

...

1. Revert migration from database: PM> Update-Database <prior-migration-name>
2. Remove migration file from project (or it will be reapplied again on next step)
3. Update model snapshot: PM> Remove-Migration.

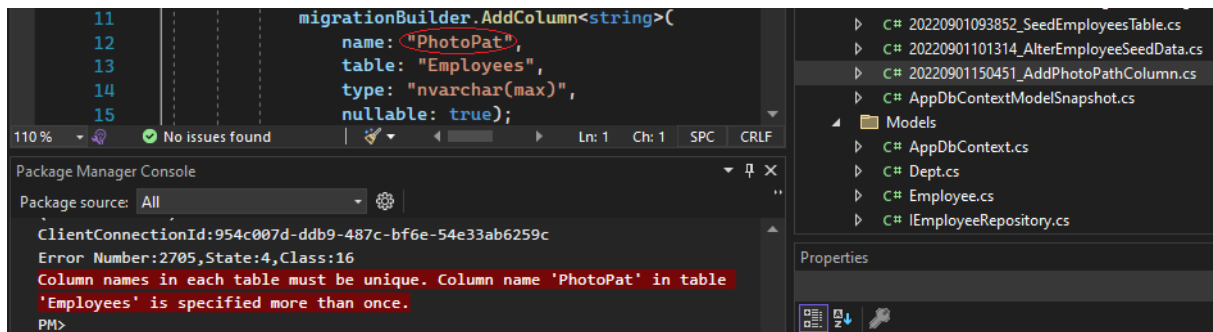


Едва след изтриване от проекта работи **remove-migration** и Snapshot се обновява. Искаме да премахнем и следващата миграция AddPhotoPathColumn по същия начин: първо се изтрива файла от проекта и след това командата PM> Промените в Employee.cs обаче остават и трябва ръчно да се отстранят.

Ако се опитаме да отстраним AlterEmployeeSeedData, ще върне грешка, че вече тази миграция е приложена в БД. Такива миграции не могат да се отстранят лесно. Миграциите се отстраняват само една по една последователно, когато все още не са включени в БД. Нека го демонстрираме:



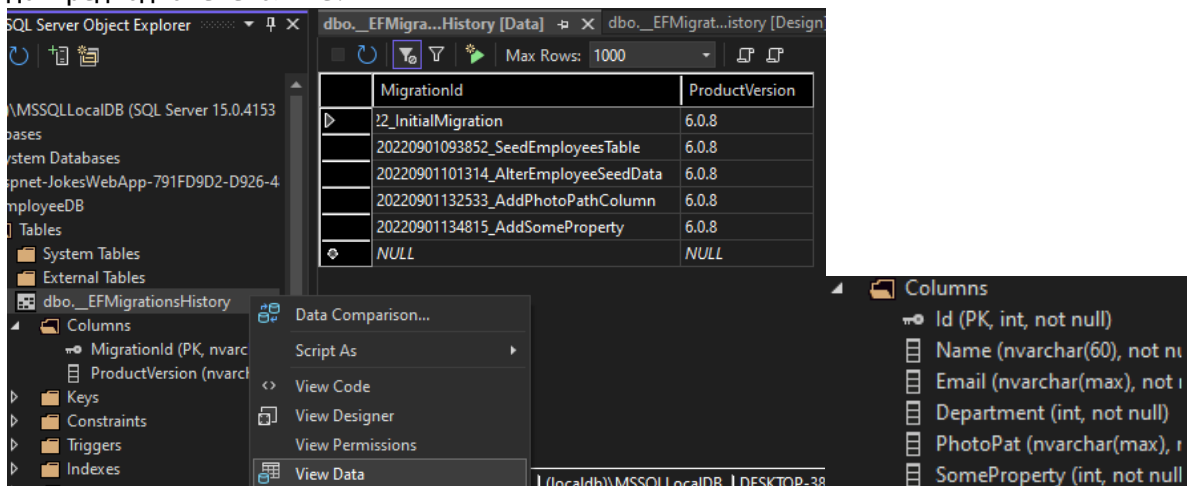
Нека премахнем `SomeProperty` и добавим нова миграция `AddPhotoPathColumn`. След ъпдейт извежда интересно съобщение:



Връщаме `public int SomeProperty { get; set; }`

Връща същата грешка.

За да може да изтрием миграция, която вече е приложена в БД, трябва да я върнем до предходно състояние.



Има папка с историята на редакциите на таблицата. След Refresh -> View Data показва всички миграции, които са били приложени в БД.

Понеже сме допуснали грешка в колона `PhotoPat` и практически искаме да отменим промените в последните 2 миграции, задаваме ъпдейт на миграцията преди тях `AlterEmployeeSeedData`. От таблицата изчезват `AddPhotoPathColumn`, `AddSomeProperty`, което след Refresh се отрязва в изчезването на колоните на БД таблицата вляво. Вдясно обаче, в папка `Migrations`, те все още стоят. Даваме `remove-migration`, като командата отстранява последната миграция `AddSomeProperty`, Следва повторение на команда `remove-migration`, която отстранява и предпоследната команда `AddPhotoPathColumn`. Refresh. Промените са отразени с изчезване на миграциите от папка `Migrations` и промени в `SnapShoot`. Сега можем да коригираме и `Employee.cs`, откъдето започнахме и имаме грешка `PhotoPat`. Ако забравим корекциите там, ще се появи грешка



An unhandled exception occurred while processing the request.

SqlException: Invalid column name 'One'.

Invalid column name 'Som'.

Invalid column name 'Some'.

Microsoft.Data.SqlClient.SqlConnection.OnError(SqlException exception, bool breakConnection, Action<Action> wrapCloseInAction)

Domain Models & Database Schema

- Use **migrations** to keep domain models and database schema in sync
- To add a new migration use **Add-Migration** command
- To update the database with the latest migration use **Update-Database** command
- To remove the latest migration that is not yet applied to the database use **Remove-Migration**
- **__EFMigrationsHistory** table is used to keep track of the migrations that are applied to the database
- **ModelSnapshot.cs** file contains the snapshot of the current model and is used to determine what has changed when adding the next migration

To remove the migration that is already applied to the database

- First use the **Update-Database** command to undo the database changes applied by the migration
- Next, use **Remove-Migration** command to remove the migration code file

Add-migration Name2

Remove-migration.

Add-migration Name2

Update-database

Update-database Name 1

Remove-migration

Ако се даде реален update-database, миграцията се вкарва в БД, с което вече става активна и използваема, но не може да се изтрие лесно. Поглеждаме последователността в dbo_EFMigrationsHistory -> ViewData. Ако имаме миграция 1, миграция 2, миграция 3 и искаме да изтрием миграция 2 и 3, тогава update-database с името на миграция 1 автоматично ще занули промените, нанесени от миграция 2 и 3. Веднага миграция 2 и 3 се отстраняват от таблица MigrationsHistory, а когато дадем refresh на таблицата dboEmployees, колони 2 и 3 вече ги няма.

След това използваме `remove-migration` без име, за да премахне автоматично последната добавена миграция 3, което се вижда вдясно в папка Migrations, след което още веднъж, за да отстрани и предпоследната миграция 2, която също изчезва от папката. Файлът SnapShoot отразява промените, но въведените в модел Employee.cs, първите, които са ръчно необходими за създаване на миграция, са останали, така че коригираме и там.

Need to update database without losing data in Migration

<https://stackoverflow.com/questions/38025541/need-to-update-database-without-losing-data-in-migration/38025775>

53 - File upload

Ще се научим как да прикачваме файл (за снимка).

File Upload in ASP.NET Core MVC



Данните ще бъдат записани в БД, а картинката в сървъра в `wwwroot->images`.

```
Create.cshtml  Employee.cs  ApplicationDbContext...elSnapshot.cs  dbo._EFMigrat...History [Data]
1  @model Employee
2  @inject IEmployeeRepository _employeeRepository
3
4  @{
5      ViewBag.Title = "Create Employee";
6  }
7  <form asp-controller="Home" asp-action="Create" method="post" class="mt-3">
8      <div class="form-group row">
9          <label asp-for="Name" class="col-sm-2 col-form-label"></label>
10         <div class="col-sm-10">
11             <input asp-for="Name" class="form-control" placeholder="Name" />
12             <span asp-validation-for="Name" class="text-danger"></span>
13         </div>
14     </div>
```

До момента, създаването на нов служител става в Create View, пряко обвързан с модел Employee.

За картинка обаче, ще е необходимо да създадем друг ViewModel, който е вид разширение към основния View, специално за целта, по същият начин както създадохме HomeDetailsViewModel.cs, за да допълним към клас Employee + PageTitle.

```

1  using EmployeeManagement.Models;
2
3  namespace EmployeeManagement.ViewModels
4  {
5      5 references
6      public class HomeDetailsViewModel
7      {
8          5 references
9          public Employee Employee { get; set; }
10         1 reference
11         public string PageTitle { get; set; }
12     }

```

Папка ViewModels ->Add->Class->EmployeeCreateViewModel.

Това е необходимо, за да имаме достъп до картинката от сървъра чрез ModelBinding, използвайки тип IFormFile.

Класът, който ще направим, ще е много сходен с Employee, така че копираме наготово от него съдържанието, за да го изменим леко, като отстраним излишния id, вкараме необходимите namespace и изменим името вместо PhotoPath->Photo и тип вместо string ->IFormFile, изискващ

```

public class EmployeeCreateViewModel
{
    [Required]
    [MaxLength(50, ErrorMessage = "Name cannot exceed 50 characters")]
    0 references
    public string Name { get; set; }
    [Required]
    [RegularExpression(@"^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+$",
        ErrorMessage = "Invalid email format")]
    [Display(Name = "Office Email")]
    0 references
    public string Email { get; set; }
    [Required]
    0 references
    public Dept? Department { get; set; }
    0 references
    public IFormFile Photo { get; set; }
}

```

using Microsoft.AspNetCore.Http;
Microsoft.AspNetCore.Http.IFormFile
Generate type 'IFormFile'

CS0246 The type or namespace name 'IFormFile' could not be found (are you missing a using directive or an assembly reference?)
using EmployeeManagement.Models;
using Microsoft.AspNetCore.Http;

IFormFile съдържа много полезни properties, изброени по-надолу.

Ако решим да не създаваме отделен ViewModel, а просто да сменим формата на IFormFile, ще трябва да сменим navigation property, понеже IFormFile is a complex object, което ще усложни обработката в бъдеще, освен това, не искаме цялата тази информация да се съхранява в БД, а само името на файла. Затова в Employee оставяме тип string PhotoPath, която да се съхранява в таблица БД, а отделно правим ViewModel, чрез който да прикачим файл към сървъра като IFormFile Photo.

File Upload in ASP.NET Core MVC

```
public class EmployeeCreateViewModel
{
    // Other Employee Properties

    public IFormFile Photo { get; set; }
}
```

```
public interface IFormFile
{
    string ContentType { get; }
    string ContentDisposition { get; }
    IDictionary Headers { get; }
    long Length { get; }
    string Name { get; }
    string FileName { get; }
    Stream OpenReadStream();
    void CopyTo(Stream target);
    Task CopyToAsync(Stream target, CancellationToken cancellationToken = null);
}
```

```

@model Employee
[Inject] IEmployeeRepository _employeeRepository

[HttpGet]
ViewBag.Title = "Create Employee";

<form asp-controller="Home" asp-action="Create" method="post" class="mt-3">
  <div class="form-group row">
    <label asp-for="Name" class="col-sm-2 col-form-label"></label>
    <div class="col-sm-10"></div>
  </div>
  <div class="form-group row">
    <label asp-for="Email" class="col-sm-2 col-form-label"></label>
    <div class="col-sm-10"></div>
  </div>
  <div class="form-group row">
    <label asp-for="Department" class="col-sm-2 col-form-label"></label>
    <div class="col-sm-10"></div>
  </div>
  <div asp-validation-summary="All" class="text-danger"></div>
  <div class="form-group row">
    <div class="col-sm-10"></div>
  </div>
  <div class="form-group row">
    <div class="col-sm-10">
      Total Employees Count = @_employeeRepository.GetAllEmployee().Count()
    </div>
  </div>
</form>

```

В Create.cshtml, правим връзка към model вместо Employee, да бъде към по-сложния тип EmployeeCreateViewModel, отстраняваме и този излишен ред @inject, както и брояча.


```

ViewModel.cs      Create.cshtml - X
@model EmployeeCreateViewModel

@{
    ViewBag.Title = "Create Employee";
}

<form asp-controller="Home" asp-action="Create" method="post" class="mt-3">
    <div class="form-group row">
        <label asp-for="Name" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div class="form-group row">
        <label asp-for="Email" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div class="form-group row">
        <label asp-for="Department" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div asp-validation-summary="All" class="text-danger"></div>
    <div class="form-group row">
        <div class="col-sm-10"></div>
    </div>
</form>

```

Допълваме вида на формата `<form enctype="multipart/form-data">` и копираме и изменяме кода на един ред, за да бъде нагледен за прикачването.

```

ViewModel.cs      Create.cshtml - X
@model EmployeeCreateViewModel

@{
    ViewBag.Title = "Create Employee";
}

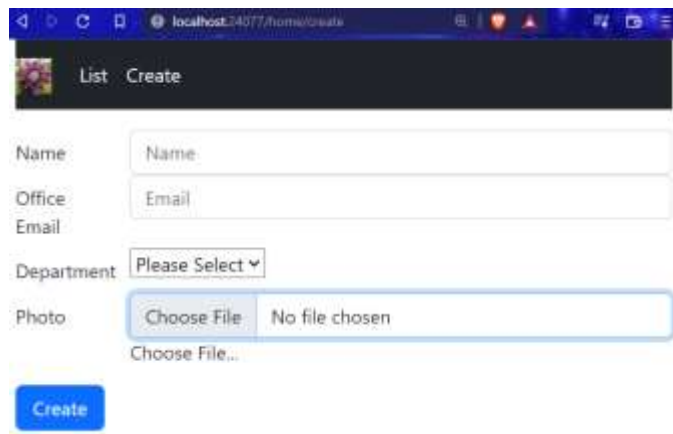
<form enctype="multipart/form-data" asp-controller="Home" asp-action="Create" method="post" class="mt-3">
    <div class="form-group row">
        <label asp-for="Name" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div class="form-group row">
        <label asp-for="Email" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div class="form-group row">
        <label asp-for="Department" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10"></div>
    </div>
    <div class="form-group row">
        <label asp-for="Photo" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10">
            <div class="custom-file">
                <input asp-for="Photo" class="form-control custom-file-input" />
                <label class="custom-file-label">Choose File...</label>
            </div>
        </div>
    </div>
    <div asp-validation-summary="All" class="text-danger"></div>

```

```

<div class="form-group row">
    <label asp-for="Photo" class="col-sm-2 col-form-label"></label>
    <div class="col-sm-10">
        <div class="custom-file">
            <input asp-for="Photo" class="form-control custom-file-input" />
            <label class="custom-file-label">Choose File...</label>
        </div>
    </div>
</div>

```

Никъде не сме уточнили input element, type file.

Това идва от съвпаденията на

```
public IFormFile Photo { get; set; }
```

```
<=>
```

```
<input asp-for="Photo"
```

```
<div class="form-group row">
```

```
<label class="col-sm-2 col-form-label" for="Photo">Photo</label>
```

```
<div class="col-sm-10">
```

```
<div class="custom-file">
```

```
<input class="form-control custom-file-input" type="file" id="Photo" name="Photo" />
```

```
<label class="custom-file-label">Choose File...</label>
```

```
</div>
```

```
</div>
```

```
</div>
```

Photo



```
@if (IsSectionDefined("Scripts"))
{
    @RenderSection("Scripts", required: true)
}
</div>
</body>
```

В туториъла не се изписва името при избирането на файла, а едва, когато се посочи с мишката в label на полето за file upload control. При моята версия обаче, това става автоматично. В _Layout бяхме дефинирали, че ако дъното на страницата съдържа секция Scripts, тя да се изпълни. В туториъла я извикват за JQuery, за да се изпише името на файла, когато е uploaded, което в моя случай е излишно.

```

<div class="form-group row">
  <label asp-for="Photo" class="col-sm-2 col-form-label"></label>
  <div class="col-sm-10">
    <div class="custom-file">
      <input asp-for="Photo" class="form-control custom-file-input" />
      <label class="custom-file-label">Choose File...</label>
    </div>
  </div>
</div>
<div asp-validation-summary="All" class="text-danger"></div>
<div class="form-group row">
  <div class="col-sm-10">
    <button type="submit" class="btn btn-primary">Create</button>
  </div>
</div>
@section Scripts {
  <script>
    $(document).ready(function () {
      $('.custom-file-input').on("change", function () {
        var fileName = $(this).val().split("\\").pop();
        $(this).next('.custom-file-label').html(fileName);
      });
    });
  </script>
}
</form>

```

```

@section Scripts {
  <script>
    $(document).ready(function () {
      $('.custom-file-input').on("change", function () {
        var fileName = $(this).val().split("\\").pop();
        $(this).next('.custom-file-label').html(fileName);
      });
    });
  </script>
}

```

При натискане на бутон Create, [HttpPost] обработва заявката, като вече имаме по-сложен модел от Employee -> EmployeeCreateViewModel

```

[HttpPost]
0 references
public IActionResult Create(Employee employee) //receive the employee object
{
  if (ModelState.IsValid)
  {
    Employee newEmployee = _employeeRepository.Add(employee);
    // return RedirectToAction("details", new { id = newEmployee.Id });
  }
  return View();
}

```

За целта, за да може при изборът да се запази в папка wwwroot/images, ни трябва

IHostingEnvironment:

```
public class HomeController : Controller
{
    private readonly IEmployeeRepository _employeeRepository; //auto private field
    0 references
    public HomeController(IEmployeeRepository employeeRepository,
                          IHostingEnvironment hostingEnvironment)
    {
    }
}
```

Microsoft.AspNetCore.Hosting; CS0246 The type or namespace name 'IHostingEnvironment' could not be found (are you missing a using directive or an assembly reference?)

Microsoft.Extensions.Hosting;

using EmployeeManagement.ViewModels;

using Microsoft.AspNetCore.Hosting;

using Microsoft.AspNetCore.Mvc;

Създаваме частно поле с променлива

```
public class HomeController : Controller
{
    private readonly IEmployeeRepository _employeeRepository; //auto private field
    private readonly IHostingEnvironment hostingEnvironment; //auto private field
    0 references
    public HomeController(IEmployeeRepository employeeRepository,
                          IHostingEnvironment hostingEnvironment)
    {
        _employeeRepository = employeeRepository;
        this.hostingEnvironment = hostingEnvironment;
    }
}
```

извикваме променливата със свойство WebRootPath, което предоставя абсолютно физическо пространство в wwwroot. Трябва да комбинираме неговия адрес с името на прикачения файл. Искаме всяка картинка да бъде с уникално име, а ако се въведат две с еднакво – последната да презапише предходната. За целта се използва **Guid = Global Unique Identifier**.

За да присвоим стринговата комбинация на пътя от папката+името на променливата, използваме CopyTo method:

```

HomeController.cs | Create.cshtml
EmployeeManagement.Controllers.HomeController | Create(EmployeeCreateViewModel model)
[HttpPost]
0 references
public IActionResult Create(EmployeeCreateViewModel model)
{
    if (ModelState.IsValid)
    {
        string uniqueFileName = null;
        if (model.Photo != null) uploaded
        {
            var name sys class .string method wwwroot/ folder
            string uploadFolder = Path.Combine(hostingEnvironment.WebRootPath + "images");
            var to store = Global Unique Identifier (generated number) as string + _ + name of uploaded photo
            uniqueFileName = Guid.NewGuid().ToString() + '_' + model.Photo.FileName;
            sys class
            string filePath = Path.Combine(uploadFolder, uniqueFileName);
            model.Photo.CopyTo(new FileStream(filePath, FileMode.Create));
        } to create it to the server
        Employee newEmployee = new Employee
        {
            Name = model.Name,
            Email = model.Email,
            Department = model.Department,
            PhotoPath = uniqueFileName
        };
        _employeeRepository.Add(newEmployee);
        return RedirectToAction("details", new { id = newEmployee.Id });
    }
}
FileMode.Create));
C# FileStream.Create
FileMode.Create = 2
Specifies that the operating system should create a new file. If the file already exists, it will be overwritten. This requires System.Security.Permissions.FileIOPermissionAccess.Write permission. FileMode.Create is equivalent to requesting that if the file does not exist, use FileMode.CreateNew; otherwise, use FileMode.Truncate. If the file already exists but is a hidden file, an UnauthorizedAccessException exception is thrown.

```

Оказва се, че трябва да добавя наклонена черта за адреса на папка /images, което ми направи печатление като липса още при писането, но в старите версии е работело без нея.



```

[HttpPost]
public IActionResult Create(EmployeeCreateViewModel model)
{
    if (ModelState.IsValid)
    {
        string uniqueFileName = null;
        if (model.Photo != null)
        {
            string uploadFolder =
            Path.Combine(hostingEnvironment.WebRootPath + "/images");
            uniqueFileName = Guid.NewGuid().ToString() + '_' +
            model.Photo.FileName;
            string filePath = Path.Combine(uploadFolder, uniqueFileName);
            model.Photo.CopyTo(new FileStream(filePath,
            FileMode.Create));
        }
        Employee newEmployee = new Employee
        {

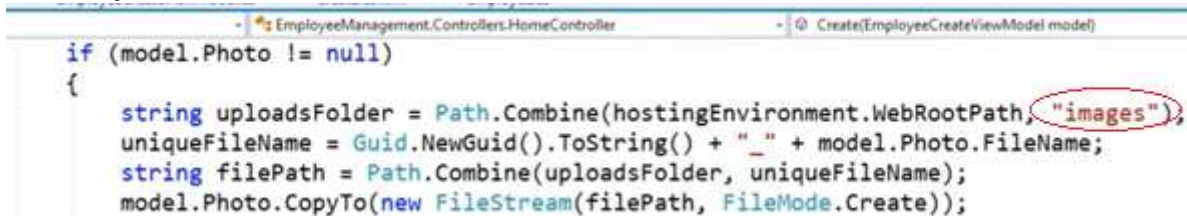
```



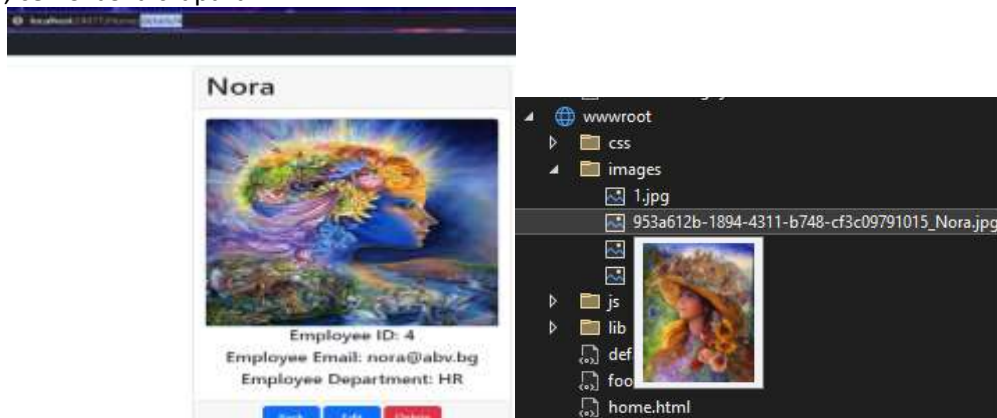
```

        Name = model.Name,
        Email = model.Email,
        Department = model.Department,
        PhotoPath = uniqueFileName
    };
    _employeeRepository.Add(newEmployee);
    return RedirectToAction("details", new { id = newEmployee.Id });
}
return View();
}

```



Когато успява да се създаде новият Employee обаче, въпреки, че картинката е записана в сървъра, се показва старата.



Това е така, защото сме hard code-нали друга по подразбиране.

```

<div class="card-body text-center">
    

    <h4>Employee ID: 4</h4>
    <h4>Employee Email: nora@abv.bg </h4>
    <h4>Employee Department: HR </h4>
</div>
<div class="card-footer text-center">
    <a class="btn btn-primary" href="/">Back</a>
    <a href="#" class="btn btn-primary">Edit</a>
    <a href="#" class="btn btn-danger">Delete</a>
</div>

```


За да го променим, трябва да подадем динамично данните на новата картинка.

```
Employee.cs Details.cshtml* HomeDetailsViewModel.cs
1 @model HomeDetailsViewModel
2 @{
3     ViewBag.Title = "Employee Details";
4     var photoPath = "~/images/" + (Model.Employee.PhotoPath ?? "noimage.jpg");
5 }
6 <div class="row justify-content-center m-1">
7     <div class="col-sm-4">
8         <div class="card">
9             <div class="card-header">
10                 <h1>@Model.Employee.Name</h1>
11             </div>
12             <div class="card-body text-center">
13                 
14                 <h4>Employee ID: @Model.Employee.Id</h4>
15                 <h4>Employee Email: @Model.Employee.Email </h4>
16                 <h4>Employee Department: @Model.Employee.Department </h4>
17             </div>
18         </div>
19     </div>
```

Имаме 2 служителя, които директно са вкарани в БД, без те да имат път по същия начин към профилни снимки, затова слагаме опция ??

dbo.Employees [Data] dbo.Employees [Design] Employee.cs Details.cshtml*					
	Id	Name	Email	Department	PhotoPath
	1	Mary	mary@abv.bg	2	NULL
	2	John	john@abv.bg	1	NULL
	4	Nora	nora@abv.bg	1	953a612b-1894-4311-b748-cf3c09791015_Nora.jpg
	NULL	NULL	NULL	NULL	NULL

```
@model HomeDetailsViewModel
@{
    ViewBag.Title = "Employee Details";
    var photoPath = "~/images/" + (Model.Employee.PhotoPath ?? "noimage.jpg");
}
<div class="row justify-content-center m-1">
    <div class="col-sm-4">
        <div class="card">
            <div class="card-header">
                <h1>@Model.Employee.Name</h1>
            </div>
            <div class="card-body text-center">
                
            </div>
        </div>
    </div>
```

Леки корекции правим и в Index, където също е hard code, но се използва друг модел като списък, лист IEnumerable

```
EmployeeRepository.cs | Index.cshtml* | Details.cshtml* | HomeDetailsViewModel.cs
@model IEnumerable<Employee>

@{
    ViewBag.Title = "Employee List";
}

<div class="card-group">
    @foreach (var employee in Model)
    {
        var photoPath = "~/images/" + (employee.PhotoPath ?? "noimage.jpg");

        <div class="card m-3">
            <div class="card-header">
                <h3>@employee.Name</h3>
            </div>
            
            <div class="card-footer text-center">
                <a asp-controller="home" asp-action="details" asp-route-id="@employee.Id"
                class="btn btn-primary m-1">View</a>
                <a href="#" class="btn btn-primary m-1">Edit</a>
                <a href="#" class="btn btn-danger m-1">Delete</a>
            </div>
        </div>
    }
</div>

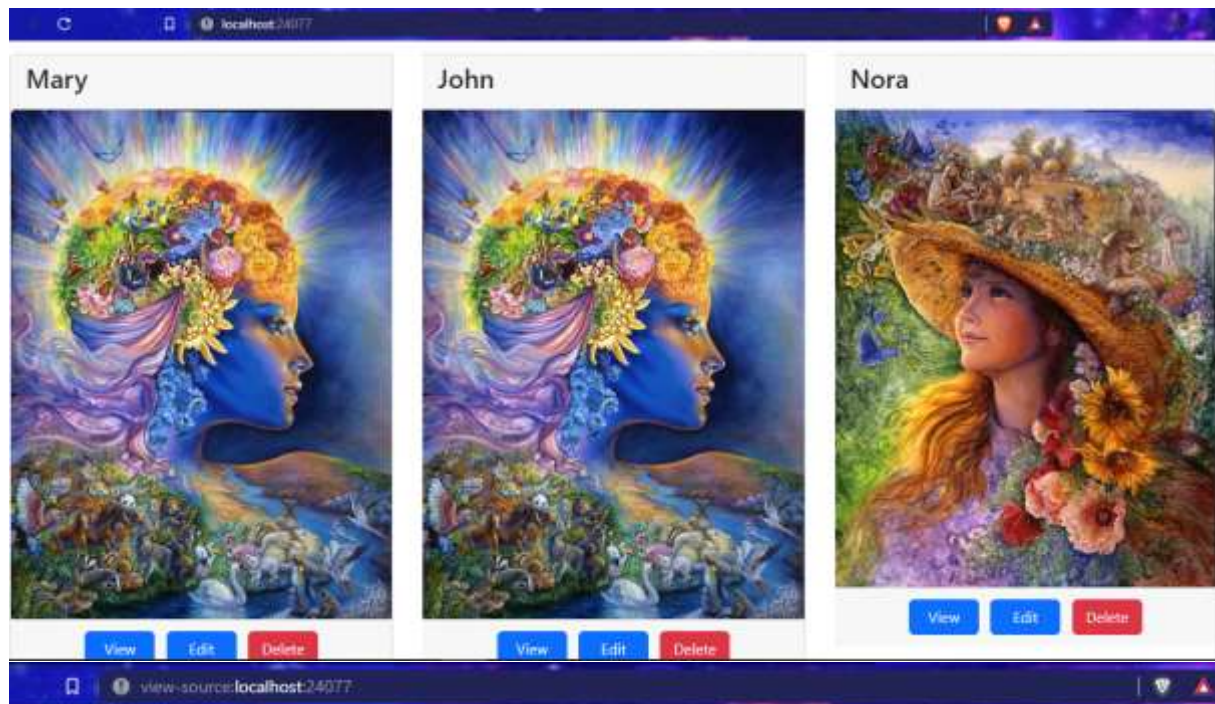
@model IEnumerable<Employee>
```

```
@{
    ViewBag.Title = "Employee List";
}

<div class="card-group">
    @foreach (var employee in Model)
    {
        var photoPath = "~/images/" + (employee.PhotoPath ?? "noimage.jpg");

        <div class="card m-3">
            <div class="card-header">
                <h3>@employee.Name</h3>
            </div>
            
        </div>
    }
</div>
```

И в двата файла създадената photoPath е локална променлива.



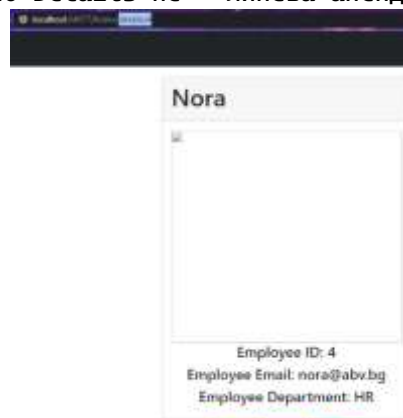
```

<div class="card-header">
  <h3>Mary</h3>
</div>

<div class="card-footer text-center">
  <a class="btn btn-primary m-1" href="/home/details/1">View</a>
  <a href="#" class="btn btn-primary m-1">Edit</a>
  <a href="#" class="btn btn-danger m-1">Delete</a>
</div>
</div>
<div class="card m-3">
  <div class="card-header">
    <h3>John</h3>
  </div>
  
  <div class="card-footer text-center">
    <a class="btn btn-primary m-1" href="/home/details/2">View</a>
    <a href="#" class="btn btn-primary m-1">Edit</a>
    <a href="#" class="btn btn-danger m-1">Delete</a>
  </div>
</div>
<div class="card m-3">
  <div class="card-header">
    <h3>Nora</h3>
  </div>
  

```

Index работи, но Details не – липсва апендикс на версията.




```
view-source:localhost:24077/home/details/4
<li class="nav-item">
  <a class="nav-link text-light" href="/">List</a>
</li>
<li class="nav-item">
  <a class="nav-link text-light" href="/home/create">Create</a>
</li>
</ul>
</div>
</nav>
<div>
  <div class="row justify-content-center m-1">
<div class="col-sm-4">
  <div class="card">
    <div class="card-header">
      <h1>Nora</h1>
    </div>
    <div class="card-body text-center">
      
    </div>
  </div>
</div>
</div>
```

Използваме image TagHelper for cache busting service.

```
<div class="card-body text-center">
  
</div>
```

``

При което се зарежда и тя със същата валидация.

При Index обаче, всички картинки са с различни големина, затова използваме custom css class.

```
site.css
1 .btn {
2   width: 75px;
3 }
4 .imageThumbnail {
5   width: 200px;
6   height: auto;
7 }
```

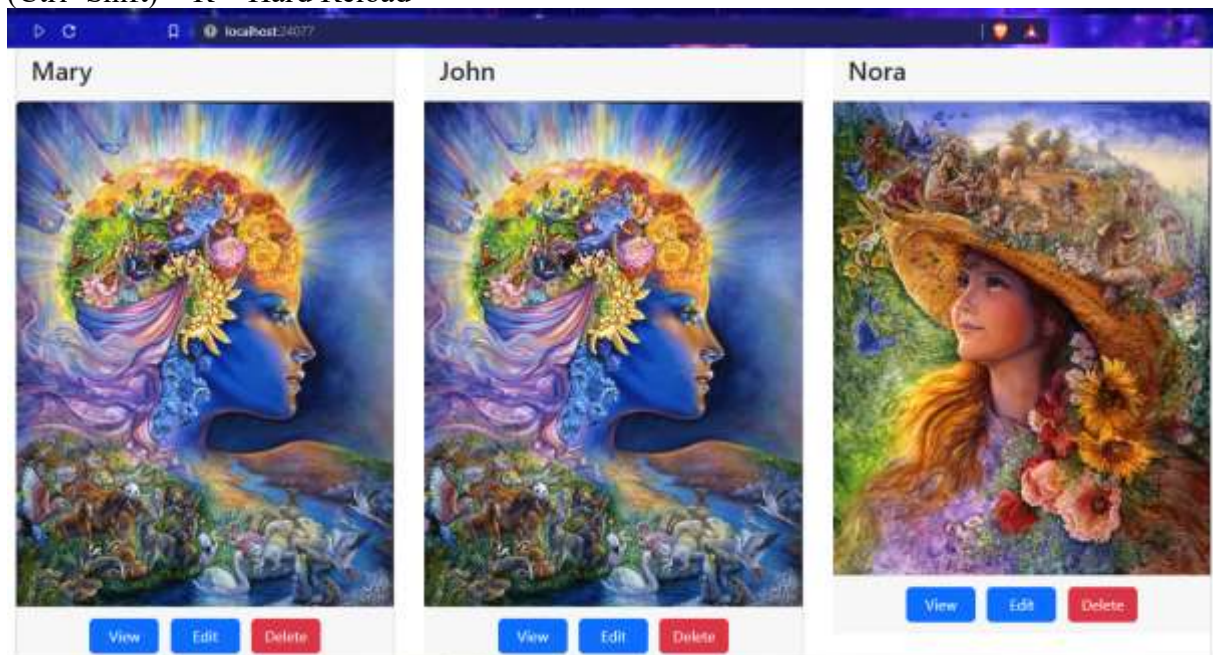
който добавяме в класа на картинката в Index.

```

```

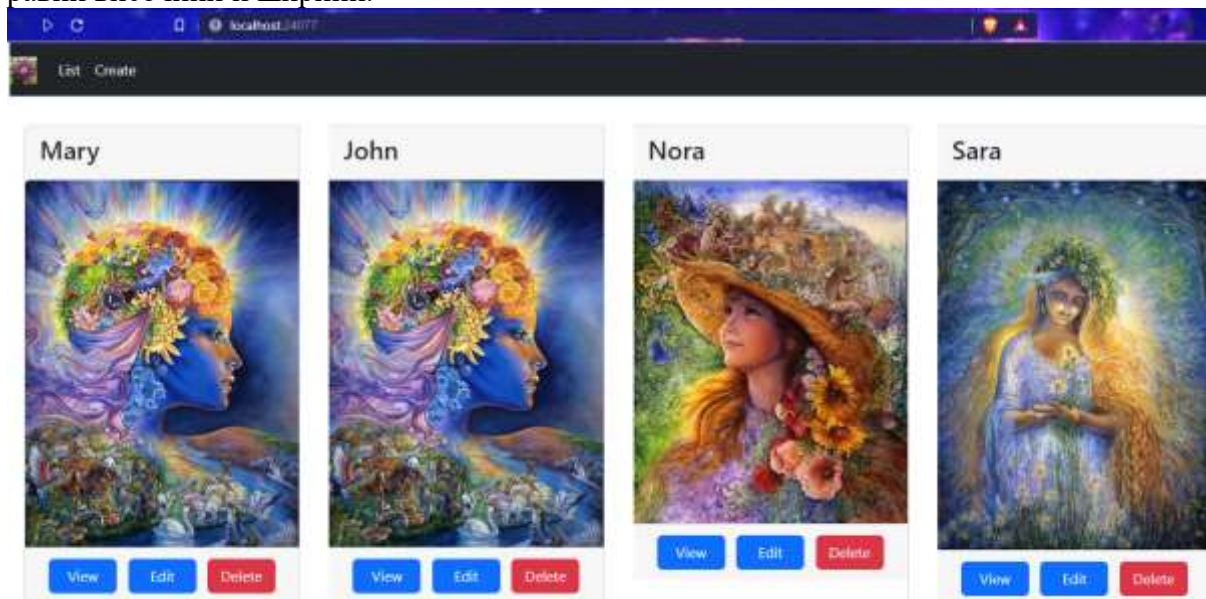
Ctrl+R = Reload

(Ctrl+Shift) + R = Hard Reload

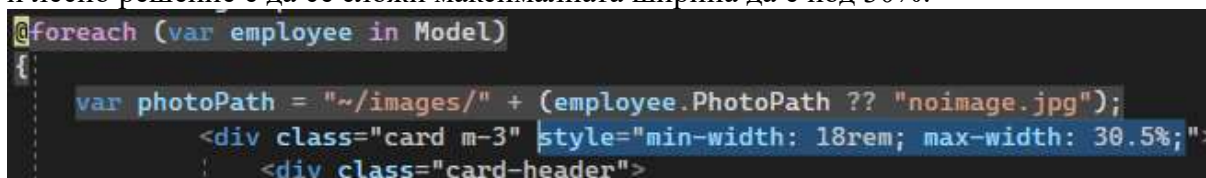


Създаваме нов Employee – явно самата картинка е по-къса, защото останалите са с

равни височини и ширини.

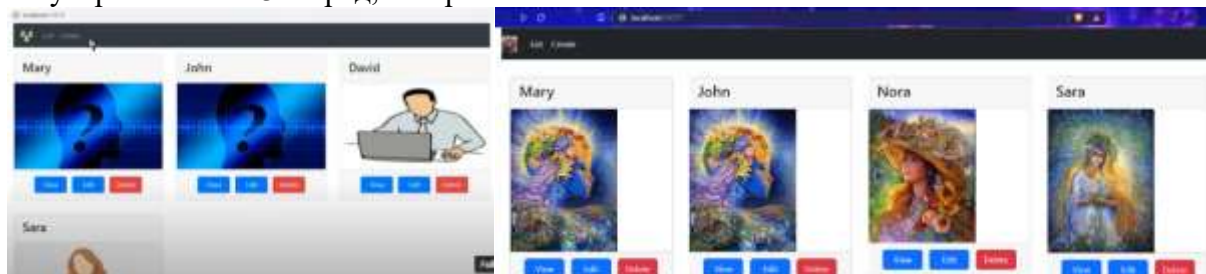


Искаме да има само по максимум 3 картинки на ред. Има няколко начина. Най-бързото и лесно решение е да се сложи максималната ширина да е под 30%:



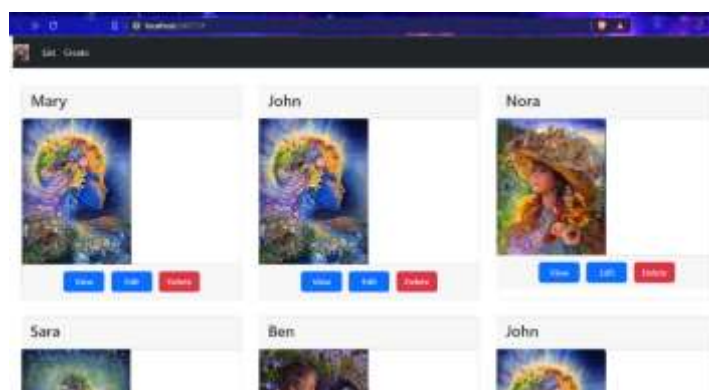
`<div class="card m-3" style="min-width: 18rem; max-width: 30.5%;">`

В туториъла са по 3 на ред, но при мен само си смалиха големините.



При `<div class="card m-3" style="min-width: 20rem; max-width: 30.5%;">` се получават по 3 на моя монитор, също поиграх с justify, но са много видове.

С демонстративна цел създаваме служител без картинка (без да искам повторих името) – noimage.jpg автоматично е присвоена.



В БД се вижда, че е без картинка.

dbo.Employees [Data] X dbo.Employees [Design] site.css Employee.cs IEmployeeRepository.cs					
	Id	Name	Email	Department	PhotoPath
1	1	Mary	mary@abv.bg	2	NULL
2	2	John	john@abv.bg	1	NULL
4	4	Nora	nora@abv.bg	1	953a612b-1894-4311-b748-cf3c09791015_Nora.jpg
5	5	Sara	sara@abv.bg	3	b00b44f3-0a95-4223-8c53-3d1c53634262_Sara.jpg
6	6	Ben	ben@abv.bg	2	694b5a23-c4c6-4d72-ae9d-e0392eefda44_Ben.jpg
7	7	John	john@abv.bg	2	NULL
	NULL	NULL	NULL	NULL	NULL

54 - Upload multiple files

В предходния урок се научихме да прикачваме само един файл. Дори да се опитахме да маркираме два файла с Ctrl + задържане, няма да допусне повече от един селектиран елемент. Това би било възможно само чрез допускане на атрибут multiple.

Upload Multiple Files in ASP.NET Core

```
public class EmployeeCreateViewModel
{
    // Other Properties

    public List<IFormFile> Photos { get; set; }
}
```

```
<input asp-for="Photos" multiple>
```

Supports selecting multiple files

```
Create.cshtml | X |
33 <div class="col-sm-10">
34 <div class="custom-file">
35 <input multiple asp-for="Photo" class="form-control custom-file-input" />
36 <label class="custom-file-label">Choose File...</label>
```

С което вече е допустима селекция на повече елементи:



В кода обаче няма необходимата подготовка за обработката им.

```
viewModel.cs Create.cshtml
@model EmployeeCreateViewModel

@{
    ViewBag.Title = "Create Employee";
}

<form enctype="multipart/form-data" asp-controller="EmployeeManagement" asp-action="Create">
    <div class="form-group row"></div>
    <div class="form-group row"></div>
    <div class="form-group row"></div>
    <div class="form-group row">
        <label asp-for="Photo" class="col-sm-2 col-md-2">Photo</label>
        <div class="col-sm-10 col-md-10">
            <div class="custom-file">
                <input multiple asp-for="Photos" class="custom-file-input" type="file">
                <label class="custom-file-label">Choose file(s)</label>
            </div>
        </div>
    </div>
</form>
```

input element is bound to the Photo property -> EmployeeCreateViewModel.

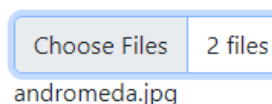
Вместо IFormFile Photo -> List<IFormFile> Photos с автокорекции навсякъде.

```
EmployeeCreateViewModel.cs
public class EmployeeCreateViewModel
{
    public Dept? Department { get; set; }

    public List<IFormFile> Photos { get; set; }
}

// Lines 47 to 54
string uniqueFileName = null;
if (model.Photo != null)
if (model.Photos != null)
{
    string uploadFolder = Path.Combine(hostingEnvironment.WebRootPath + "/images", "uploads");
    uniqueFileName = Guid.NewGuid().ToString() + "_" + model.Photo.FileName;
    uniqueFileName = Guid.NewGuid().ToString() + "_" + model.Photos.FileName;
    string filePath = Path.Combine(uploadFolder, uniqueFileName);
    model.Photo.CopyTo(new FileStream(filePath, FileMode.Create));
    model.Photos.CopyTo(new FileStream(filePath, FileMode.Create));
}
```

Когато проверим, наистина, навсякъде в кода е сменен Photo -> Photos. Когато изберем един файл, виждаме изписано името му, но когато селектираме 2, виждаме името само на единия. В туториъла искаме при избор на 2 и повече файла, да се изписва колко са заредени, вместо името на единия. При мен обаче, става автоматично.



jQuery кодът, който отговаря за това е:

Upload Multiple Files in ASP.NET Core

Photos: 2 files selected Browse...

Photos: david.png Browse...

```
@section Scripts {  
    <script>  
        $(document).ready(function () {  
            $('.custom-file-input').on("change", function () {  
                var fileLabel = $(this).next('.custom-file-label');  
                var files = $(this)[0].files;  
                if (files.length > 1) {  
                    fileLabel.html(files.length + ' files selected');  
                }  
                else if (files.length == 1) {  
                    fileLabel.html(files[0].name);  
                }  
            });  
        });  
    </script>  
}
```

Закоментираме малка част и дописваме:

```
ewModel.cs Create.cshtml -b X  
  
<div asp-validation-summary="All" class="text-danger"></div>  
<div class="form-group row">...</div>  
@section Scripts {  
    <script>  
        $(document).ready(function () {  
            $('.custom-file-input').on("change", function () {  
                var fileName = //$(this).val().split("\\").pop();  
                $(this).next('.custom-file-label').html(fileName);  
                var files = $(this)[0].files;  
                if (files.length > 1) {  
                    fileLabel.html(files.length + ' files selected');  
                }  
                else if (files.length == 1) {  
                    fileLabel.html(files[0].name);  
                }  
            });  
        });  
    </script>  
}
```

```
@section Scripts {  
    <script>  
        $(document).ready(function () {  
            $('.custom-file-input').on("change", function () {  
                var fileName = //$(this).val().split("\\").pop();  
                $(this).next('.custom-file-label').html(fileName);  
                var files = $(this)[0].files;  
                if (files.length > 1) {  
                    fileLabel.html(files.length + ' files selected');  
                }  
            });  
        });  
    </script>  
}
```

```

    }
    else if (files.length == 1) {
        fileLabel.html(files[0].name);
    }
    });
}
</script>
}

```

Понеже това е сложен код, който не сме учили и трудно ще проследим какво прави, нека го изучим чрез конзолата:

```

<script>
$(document).ready(function () {
    $('.custom-file-input').on("change", function () {
        console.log($(this))
        //var fileName = //$(this).val().split("\\").pop();
        // $(this).next('.custom-file-label').html(fileName);
        //var files = $(this)[0].files;
        //if (files.length > 1) {
        //    fileLabel.html(files.length + 'files selected');
        //}
        //else if (files.length == 1) {
        //    fileLabel.html(files[0].name);
        //}
    });
});
</script>

```

Имам предупреждение, че `IHostingEnvironment` е остаряло и в по-новите версии ще бъде сменено с `IWebHostEnvironment`, така че го коригирам.

CS0618 'IHostingEnvironment' is obsolete: 'This type is obsolete and will be removed in a future version. The recommended alternative is Microsoft.AspNetCore.Hosting.IWebHostEnvironment.'

Причината за грешките е, че `Photos` вече не е единичен `FileName`, а `List<IFormFile>`

```

if (model.Photos != null)
{
    string uploadFolder = Path.Combine(hostingEnvironment.WebRootPath + "/images");
    uniqueFileName = Guid.NewGuid().ToString() + '_' + model.Photos.FileName;
    string filePath = Path.Combine(uploadFolder, uniqueFileName);
    model.Photos.CopyTo(new FileStream(filePath, FileMode.Create));
}

```

Description	Project	File	Line	Suppression State
'List<IFormFile>' does not contain a definition for 'FileName' and no accessible extension method 'FileName' accepting a first argument of type 'List<IFormFile>' could be found (are you missing a using directive or an assembly reference?)	EmployeeManagement	HomeController.cs	51	Active

Трябва да модифицираме кода така, че при прикачването на повече файлове, да успява да ги добави към сървъра. За целта ще използваме `foreach` Loop и ще добавим локална променлива файлов тип `photo`, с която ще работим в рамките на цикъла.

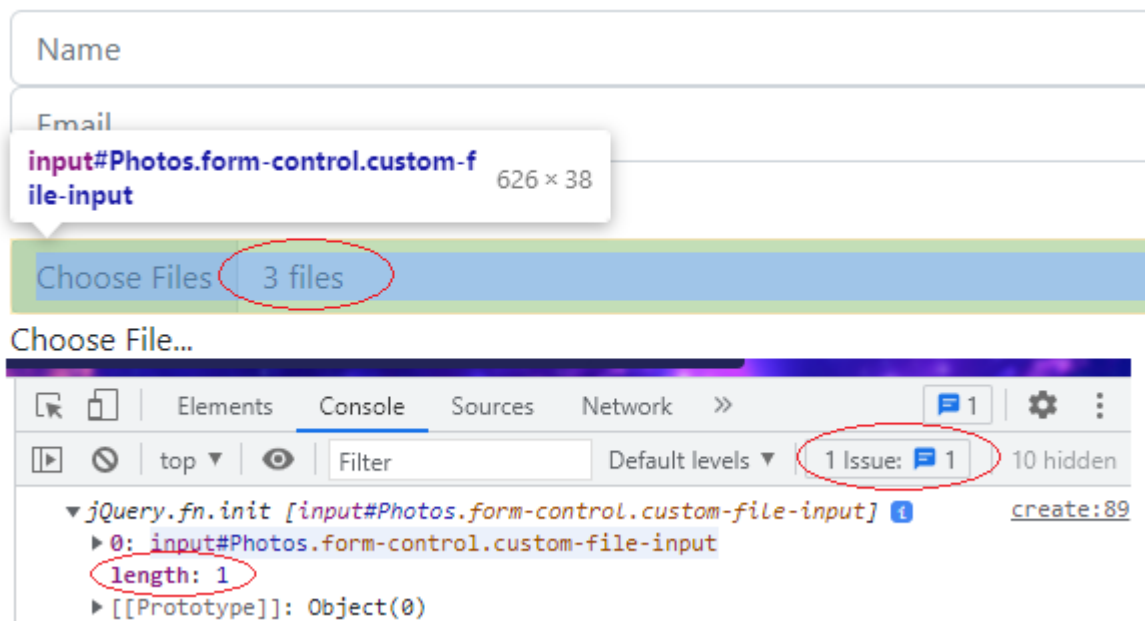
```

if (model.Photos != null && model.Photos.Count > 0)
{
    vartype
    foreach (IFormFile photo in model.Photos)
    {
        string uploadFolder = Path.Combine(hostingEnvironment.WebRootPath + "/images");
        uniqueFileName = Guid.NewGuid().ToString() + '_' + photo.FileName;
        string filePath = Path.Combine(uploadFolder, uniqueFileName);
        photo.CopyTo(new FileStream(filePath, FileMode.Create));
    }
}

```


Зареждаме, за да проверим резултата от `console.log($(this))`

Зареждаме браузъра, натискаме F12 за Developer Tools -> Console, избираме няколко файла за upload:



Виждаме 1 input и при разлистване 3 файла, първият от които е позиция 0

```
▼ files: FileList
  ► 0: File {name: 'Ben.jpg', lastModified: 1544853752000, lastModifiedDate:
  ► 1: File {name: 'Nora.jpg', lastModified: 1544854702000, lastModifiedDate:
  ► 2: File {name: 'Sara.jpg', lastModified: 1544854028000, lastModifiedDate:
  length: 3
```

При това положение, много по-лесно разбираме този JQuery:

```
$(document).ready(function () {
  $('.custom-file-input').on("change", function () {
    var fileLabel = $(this).next('.custom-file-label');
    var files = $(this)[0].files;
    if (files.length > 1) {
      fileLabel.html(files.length + ' files selected');
    }
    else if (files.length == 1) {
      fileLabel.html(files[0].name);
    }
  });
});
```

Съхраняваме файловете в променлива files.

Ако `length > 1` т.е. потребителят е избрал повече от 1 файлове, тогава добавяме надписа „files selected“ (например 2 files selected), който ще се покаже в Label полето.

В противен случай, ако потребителят е избрал само 1 файл, `length = 1` и тогава в Label ще се изпише само името му за позиция 0.

```

<script>
$(document).ready(function () {
    $('.custom-file-input').on("change", function () {
        //console.log($(this))
        var filename = //$(this).val().split("\\").pop();
        $(this).next('.custom-file-label').html(filename);
        var files = $(this)[0].files;
        if (files.length > 1) {
            filelabel.html(files.length + 'files selected');
        }
        else if (files.length == 1) {
            filelabel.html(files[0].name);
        }
    });
});
</script>

```

При мен в конзолата обаче се виждат грешки, а дори без този JQuery приложението работи в тази версия по този начин – име на 1 файл, брой файлове за няколко.

При избор на 1 файл – име:

Choose Files Sara.jpg

Choose File...

```

✖ ▶ Uncaught ReferenceError: filelabel is not defined
    at HTMLInputElement.<anonymous> (create:97:25)
    at HTMLInputElement.dispatch (jquery.js:5430:27)
    at elemData.handle (jquery.js:5234:28)
create:97

```

Избор на 2:

Choose Files 2 files

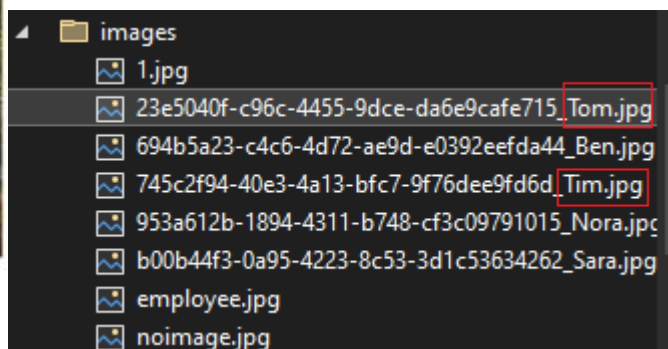
Choose File...

```

✖ ▶ Uncaught ReferenceError: filelabel is not defined
    at HTMLInputElement.<anonymous> (create:94:25)
    at HTMLInputElement.dispatch (jquery.js:5430:27)
    at elemData.handle (jquery.js:5234:28)
create:94

```

Пробваме да създадем нов потребител с 2 прикачени снимки:



Успешно е, а при проверка в папка /images и двете са прикачени, включително в локалната папка. Как обаче на 1 създаден Employee е прикачена само 1 снимка – това е, защото все още имаме връзка 1:1 между името на потребителя и снимката, които се съхраняват в 1 БД.

dbo.Employees [Data] dbo.Employees [Design] HomeController.cs 2022083011142...allMigration.cs					
	Id	Name	Email	Department	PhotoPath
	1	Mary	mary@abv.bg	2	NULL
	2	John	john@abv.bg	1	NULL
	4	Nora	nora@abv.bg	1	953a612b-1894-4311-b748-cf3c09791015_Nora.jpg
	5	Sara	sara@abv.bg	3	b00b44f3-0a95-4223-8c53-3d1c53634262_Sara.jpg
	6	Ben	ben@abv.bg	2	694b5a23-c4c6-4d72-ae9d-e0392eefda44_Ben.jpg
	7	John	john@abv.bg	2	NULL
	8	Tom	tom@abv.bg	3	23e5040f-c96c-4455-9dce-da6e9cafe715_Tom.jpg
	NULL	NULL	NULL	NULL	NULL

Ако искаме един потребител да разполага с няколко снимки, трябва да имаме 2 таблици БД, например една Employees, друга EmployeePhotos, които да бъдат във връзка 1:n (1:много) между тях.

За простота връщаме кода назад, защото искаме само 1 снимка да бъде прикачвана за 1 потребител. Това е кодът, който би ни бил необходим, ако прикачваме няколко файла:

Upload Multiple Files in ASP.NET Core

```
public IActionResult Create(EmployeeCreateViewModel model)
{
    if (ModelState.IsValid)
    {
        string uniqueFileName = null;

        if (model.Photos != null && model.Photos.Count > 0)
        {
            foreach (IFormFile photo in model.Photos)
            {
                string uploadsFolder = Path.Combine(hostingEnvironment.WebRootPath, "images");
                uniqueFileName = Guid.NewGuid().ToString() + "_" + photo.FileName;
                string filePath = Path.Combine(uploadsFolder, uniqueFileName);
                photo.CopyTo(new FileStream(filePath, FileMode.Create));
            }
        }
        // Rest of the code
    }

    return View();
}
```

Начално състояние:

```
EmployeeCreateViewModel.cs | EmployeeManagement.ViewMo
5 references
public IFormFile Photo { get; set; }
```

```

HomeController.cs  X
EmployeeManagement.Controllers.HomeController  Create(EmployeeCreateViewModel model)
string uniqueFileName = null;
if (model.Photo != null)
{
    string uploadFolder = Path.Combine(hostingEnvironment.WebRootPath + "/images");
    uniqueFileName = Guid.NewGuid().ToString() + '_' + model.Photo.FileName;
    string filePath = Path.Combine(uploadFolder, uniqueFileName);
    model.Photo.CopyTo(new FileStream(filePath, FileMode.Create));
}

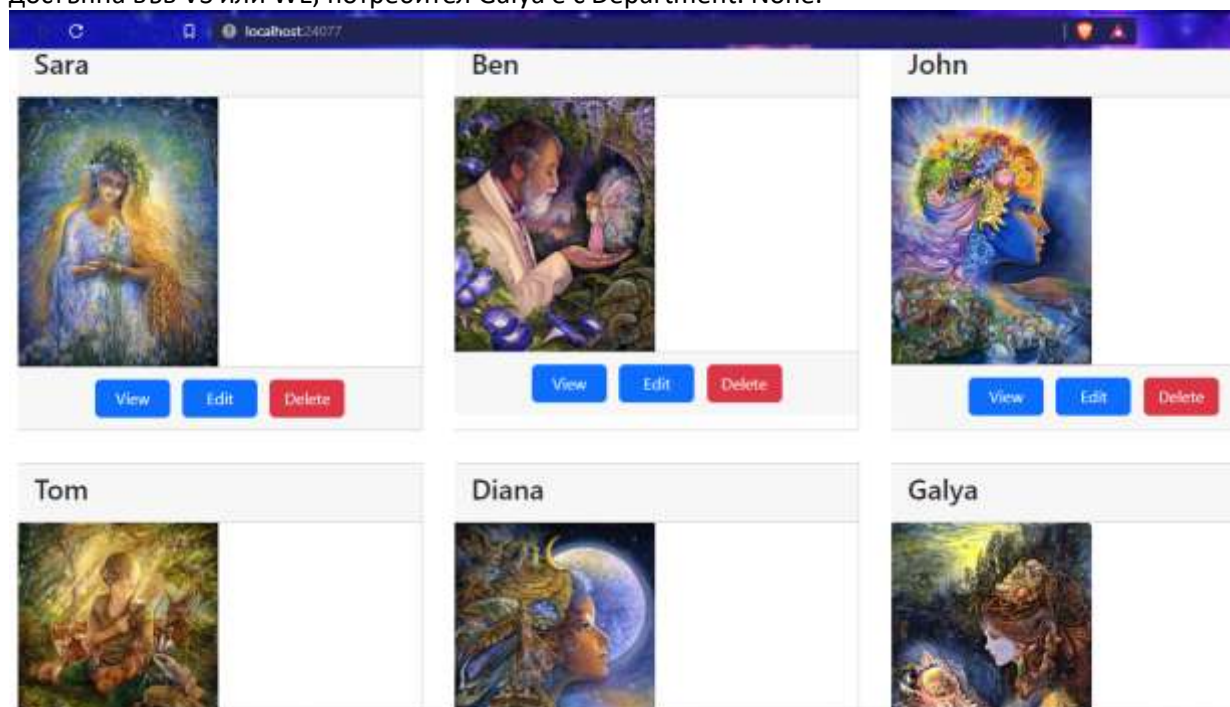
```

```

Create.cshtml  X
@section Scripts {
    <script>
        $(document).ready(function () {
            $('.custom-file-input').on("change", function () {
                var fileName = $(this).val().split("\\").pop();
                $(this).next('.custom-file-label').html(fileName);
            });
        });
    </script>
}

```

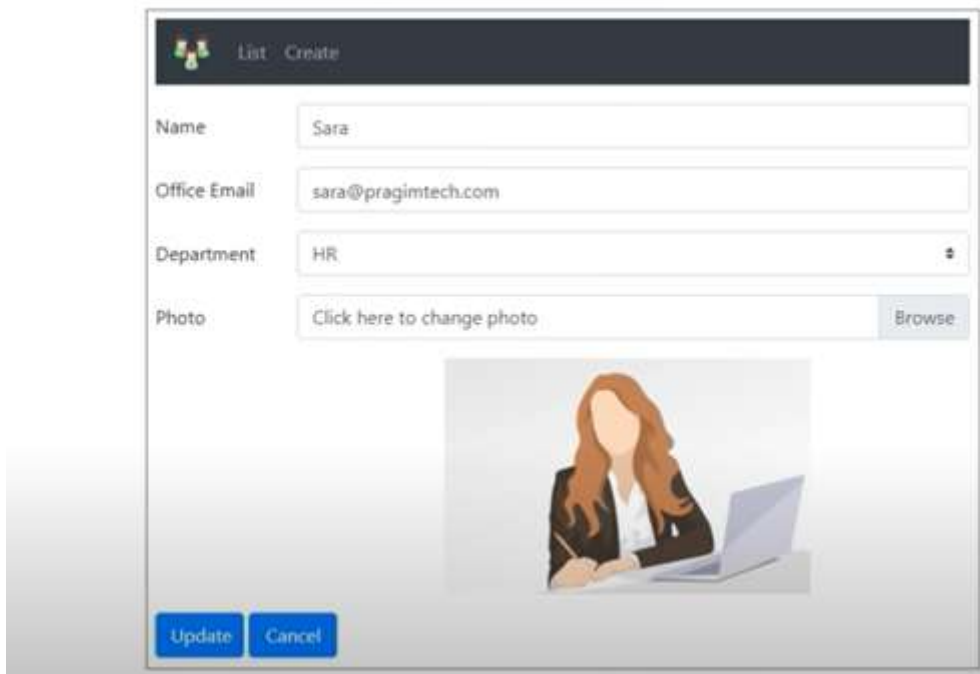
Попълних до момента 10 потребителя, като последната картинка забелязвам, че винаги не е достъпна във VS или WE, потребител Galya е с Department: None.



55 — Edit View

В този урок ще дадем възможност на потребителя да редактира съществуващи данни.

Edit View in ASP.NET Core MVC

The screenshot shows a web application interface for editing a user profile. At the top, there are links for 'List' and 'Create'. The form contains four input fields: 'Name' with the value 'Sara', 'Office Email' with 'sara@pragimtech.com', 'Department' with 'HR', and 'Photo' with a placeholder text 'Click here to change photo' and a 'Browse' button. Below the form are two buttons: 'Update' and 'Cancel'. A placeholder image of a woman working on a laptop is shown below the photo field.

Чрез бутона Update да бъдат изпратени до сървъра и презаписани върху съществуващите записи в БД, включително снимката чрез Edit view + Edit action.

Първото нещо е да направим активен бутон Edit, който да ни пренасочи към нов специален Edit action in HomeController.



Не може едновременно да се използват `href="#"` + `asp-action="edit"`

Редактираме по модела на другия бутон Edit, като уникалните данни на бутона се предават по `asp-route-id="@employee.Id"`

където имаме локална foreach loop variable employee in Model.

```
Details.cshtml | Index.cshtml* X
@model IEnumerable<Employee>

@{
    ViewBag.Title = "Employee List";
}

<div class="card-group">
    @foreach (var employee in Model)
    {
        var photoPath = "~/images/" + (employee.PhotoPath ?? "noimage.jpg");

        <div class="card m-3" style="min-width: 20rem; max-width: 30.5%;">
            <div class="card-header">
                <h3>@employee.Name</h3>
            </div>
            
            <div class="card-footer text-center">
                <a asp-controller="home" asp-action="details" asp-route-id="@employee.Id"
                class="btn btn-primary m-1">View</a>
                <a asp-controller="home" asp-action="edit" asp-route-id="@employee.Id"
                class="btn btn-primary m-1">Edit</a>
                <a href="#" class="btn btn-danger m-1">Delete</a>
            </div>
        </div>
    }
</div>
```

Друг Edit btn се намира в Details view, за който е необходим почти идентичен код, където обаче моделът е друг и трябва да се съобразим с него:

```
Details.cshtml | Index.cshtml*
@model HomeDetailsViewModel

@{
    ViewBag.Title = "Employee Details";
    var photoPath = "~/images/" + (Model.Employee.PhotoPath ?? "noimage.jpg");
}

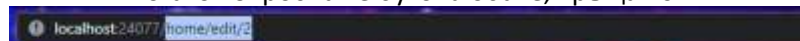
<div class="row justify-content-center m-1">
    <div class="col-sm-4">
        <div class="card">
            <div class="card-header">
                <h1>@Model.Employee.Name</h1>
            </div>

            <div class="card-body text-center">
                

                <h4>Employee ID: @Model.Employee.Id</h4>
                <h4>Employee Email: @Model.Employee.Email </h4>
                <h4>Employee Department: @Model.Employee.Department </h4>
            </div>

            <div class="card-footer text-center">
                <a asp-controller="home" asp-action="index" class="btn btn-primary">Back</a>
                <a asp-controller="home" asp-action="edit" asp-route-id="@Model.Employee.Id"
                class="btn btn-primary">Edit</a>
                <a href="#" class="btn btn-danger">Delete</a>
            </div>
        </div>
    </div>
</div>
```

Когато изпробваме бутона обаче, връща 404

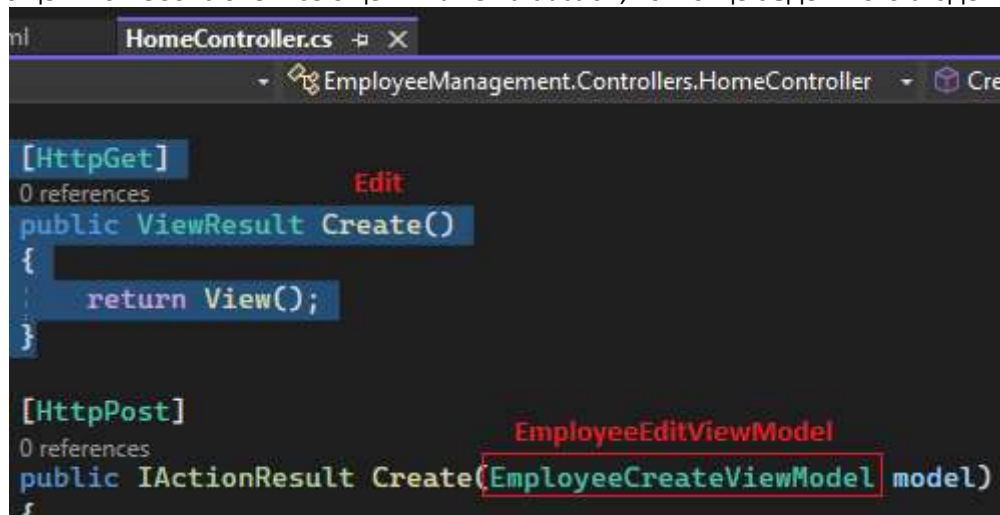


This localhost page can't be found

No webpage was found for the web address: <http://localhost:24077/home/edit/2>

HTTP ERROR 404

защото още в HomeController все още нямаме Edit action, който ще бъде много сходен с Create:



```

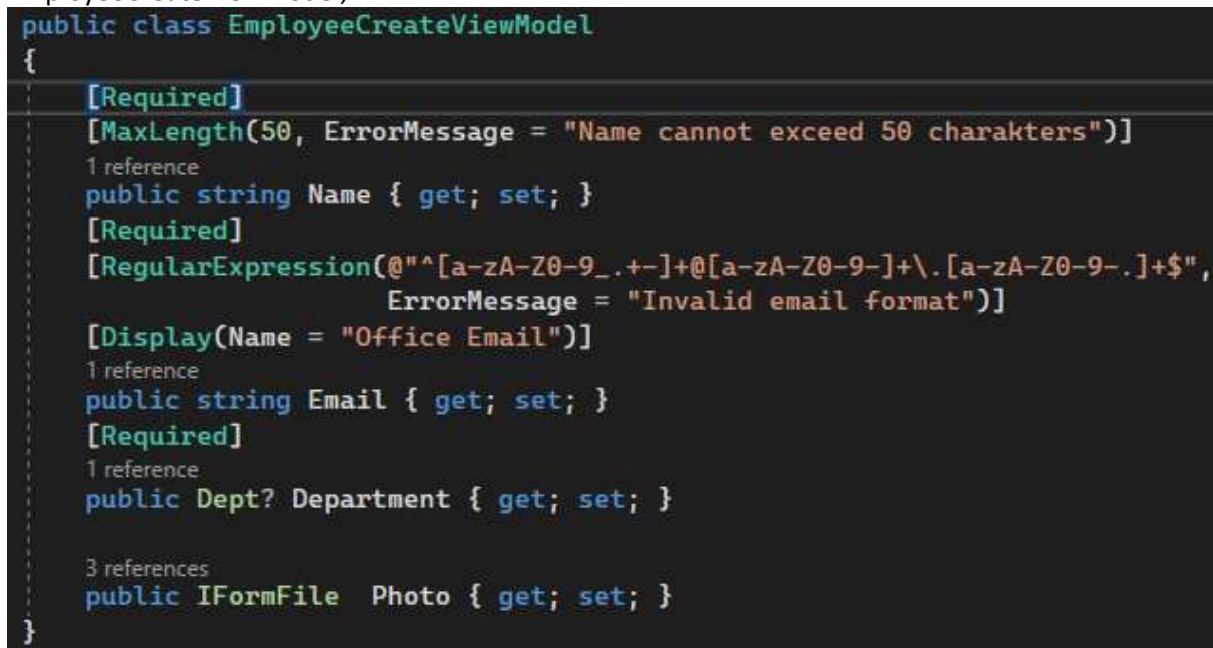
[HttpGet]
0 references
public ActionResult Create()
{
    return View();
}

[HttpPost]
0 references
public ActionResult Create(EmployeeCreateViewModel model)
{
    // ...
}

```

Трябва да може да поеме и прехвърли всички данни, затова създаваме EmployeeEditViewModel.cs class

Понеже за редакцията на формата ни трябва много данни, които съвпадат с EmployeeCreateViewModel,



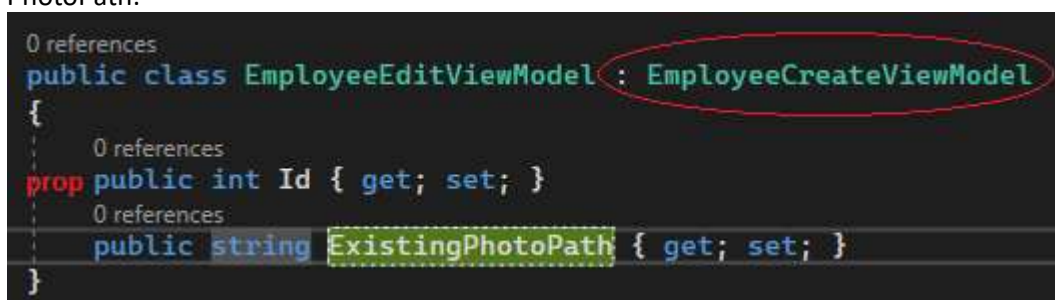
```

public class EmployeeCreateViewModel
{
    [Required]
    [MaxLength(50, ErrorMessage = "Name cannot exceed 50 characters")]
    1 reference
    public string Name { get; set; }
    [Required]
    [RegularExpression(@"^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+$",
        ErrorMessage = "Invalid email format")]
    [Display(Name = "Office Email")]
    1 reference
    public string Email { get; set; }
    [Required]
    1 reference
    public Dept? Department { get; set; }

    3 references
    public IFormFile Photo { get; set; }
}

```

вместо да ги Copy->Paste, най-разумното решение е, те да бъдат наследени, като се извикат. По този начин наследява Name, Email, Department, Photo, но ни трябва и Id и съществуващ PhotoPath:



```

0 references
public class EmployeeEditViewModel : EmployeeCreateViewModel
{
    0 references
    prop public int Id { get; set; }
    0 references
    public string ExistingPhotoPath { get; set; }
}

```


Декларирахме необходимите ни property – свойства, но сега ни трябва данните от БД:

```
HomeController.cs* EmployeeCreateViewModel.cs Details.cshtml Index.cshtml
EmployeeManagement.Controllers.HomeController Create()

0 references service - DB
public HomeController(IEmployeeRepository employeeRepository,
                     IWebHostEnvironment hostingEnvironment)
{
    _employeeRepository = employeeRepository;
    this.hostingEnvironment = hostingEnvironment;
}
0 references
public IActionResult Index()
```

По-надолу вече са actions:

Най-просто е да декларираме id, което автоматично ще бъде прихванато от URL заради MVC, след това създаваме нова локална променлива, тип Employee, която да извлече от БД свойствата по id, а после да ги предаде на създадената за целта инстанция.

```
HomeController.cs* EmployeeCreateViewModel.cs Details.cshtml Index.cshtml
EmployeeManagement.Controllers.HomeController Create()

[HttpGet]
public IActionResult Create()
{
    return View();
}

[HttpGet] localhost:24077/home/edit/2
public IActionResult Edit(int id) MVC - auto GET from URL
{
    new local var Employee employee = _employeeRepository.GetEmployee(id);
    new instance EmployeeEditViewModel employeeEditViewModel = new EmployeeEditViewModel
    {
        new = local . DB
        Id = employee.Id,
        Name = employee.Name,
        Email = employee.Email,
        Department = employee.Department,
        ExistingPhotoPath = employee.PhotoPath
    };
    return View(employeeEditViewModel);
}

[HttpPost]
public IActionResult Create(EmployeeCreateViewModel model)
```

```
[HttpGet]
public IActionResult Edit(int id)
{
    Employee employee = _employeeRepository.GetEmployee(id);
    EmployeeEditViewModel employeeEditViewModel = new
EmployeeEditViewModel
{
    Id = employee.Id,
    Name = employee.Name,
    Email = employee.Email,
    Department = employee.Department,
```

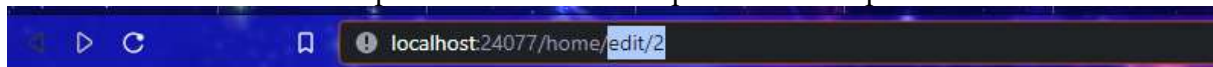


```

        ExistingPhotoPath = employee.PhotoPath
    };
    return View(employeeEditViewModel);
}

```

Този път грешката е позната и приятна за отстраняване



An unhandled exception occurred while processing the request.

InvalidOperationException: The view 'Edit' was not found. The following locations were searched:
 /Views/Home/Edit.cshtml
 /Views/Shared/Edit.cshtml
 /Pages/Shared/Edit.cshtml

Microsoft.AspNetCore.Mvc.ViewEngines.ViewEngineResult.EnsureSuccessful(IEnumerable<string> originalLocations)

Трябва да създадем View->Home->Edit.cshtml, който е много сходен с Create, който ще копиру->paste за леки изменения (Create->Edit):

```

Model.cs Edit.cshtml X HomeController.cs EmployeeCreateViewModel.cs Details.cshtml Index.cshtml
@model EmployeeEditViewModel

@{
    ViewBag.Title = "Edit Employee";
}

<form enctype="multipart/form-data" asp-controller="Home" asp-action="Edit" method="post" class="mt-3">
    <div class="form-group row">
        <input hidden asp-for="Id" />
        <input hidden asp-for="ExistingPhotoPath" />
        <label asp-for="Name" class="col-sm-2 col-form-label"></label>

```

Искаме данните за Id, ExistingPhotoPath да бъдат прихванати, но да не се показват, затова ги поставяме в <input hidden>.

```

wModel.cs Edit.cshtml X Details.cshtml HomeController.cs EmployeeCreateViewModel.cs Index.cshtml
@model EmployeeEditViewModel

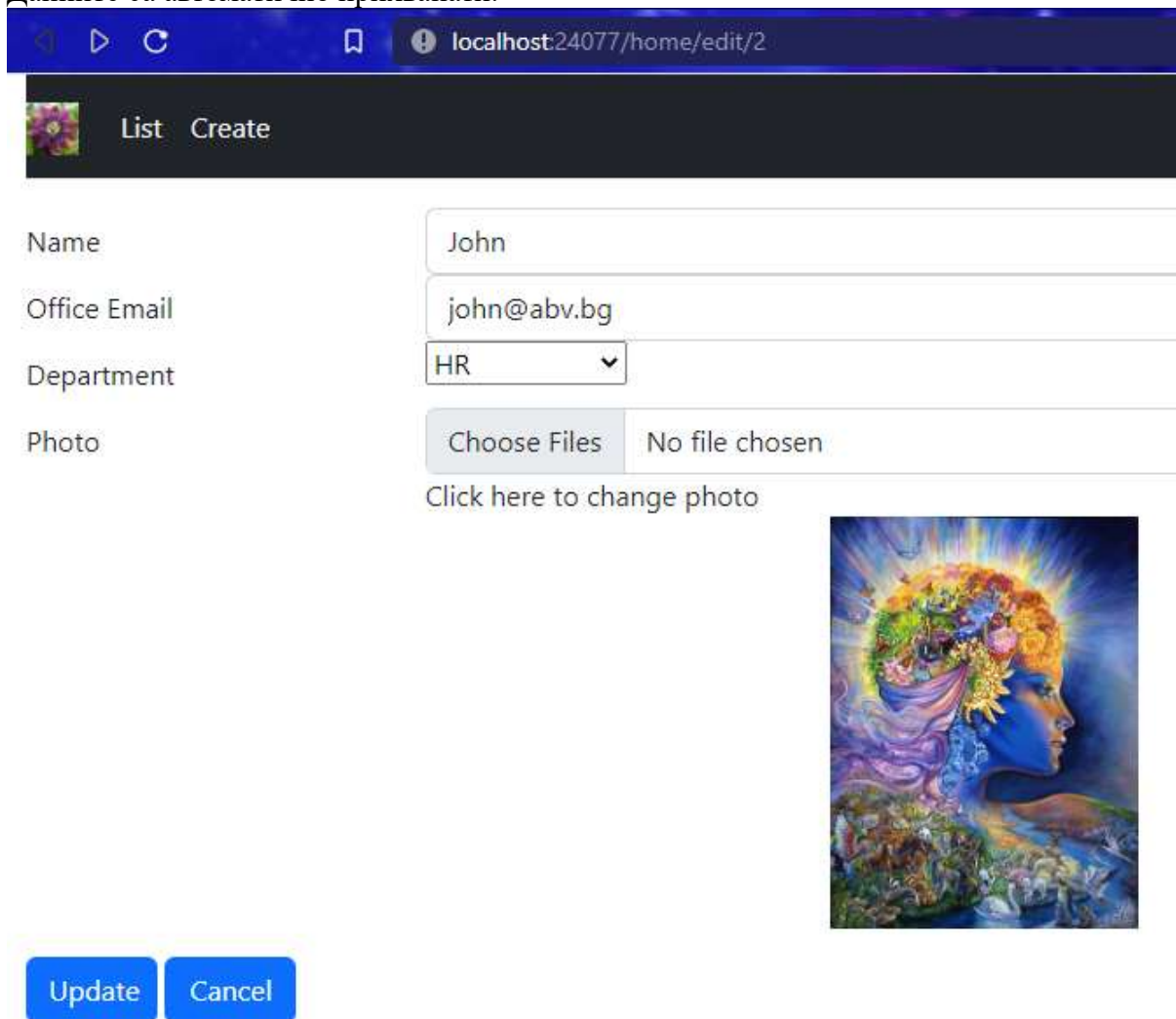
@{
    ViewBag.Title = "Edit Employee";
    var photoPath = "~/images/" + (Model.ExistingPhotoPath ?? "noimage.jpg");
}

<form enctype="multipart/form-data" asp-controller="Home" asp-action="Edit" method="post" class="mt-3">
    <div class="form-group row"></div>
    <div class="form-group row"></div>
    <div class="form-group row"></div>
    <div class="form-group row">
        <label asp-for="Photo" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10">
            <div class="custom-file">
                <input multiple asp-for="Photo" class="form-control custom-file-input" />
                <label class="custom-file-label">Click here to change photo</label>
            </div>
        </div>
    </div>

    <div class="form-group row col-sm-4 offset-4">
        
    </div>
    <div asp-validation-summary="All" class="text-danger"></div>
    <div class="form-group row">
        <div class="col-sm-10">
            <button type="submit" class="btn btn-primary">Update</button>
            <a asp-controller="home" asp-action="index" class="btn btn-primary">Cancel</a>
        </div>

```

Данните са автоматично прихванати.



localhost:24077/home/edit/2

List Create

Name John

Office Email john@abv.bg

Department HR

Photo Choose Files No file chosen

Click here to change photo



Update Cancel

На този етап при клик Edit виждаме попълнените данни на потребителите и имаме възможност за избор на картинка, при Cancel се връщаме в Index, но при натискане на бутон Update връща 404, защото все още нямаме POST Edit в HomeController.

1) рамкиране на свойства - properties (ViewModel)

Step 1 : Create Edit View Model

```
public class EmployeeEditViewModel : EmployeeCreateViewModel
{
    public int Id { get; set; }
    public string ExistingPhotoPath { get; set; }
}

public class EmployeeCreateViewModel
{
    [Required]
    [MaxLength(50, ErrorMessage = "Name cannot exceed 50 characters")]
    public string Name { get; set; }
    [Required]
    [RegularExpression(@"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$",
        ErrorMessage = "Invalid Email Format")]
    [Display(Name = "Office Email")]
    public string Email { get; set; }
    [Required]
    public Dept? Department { get; set; }
    public IFormFile Photo { get; set; }
}
```

2) прихващане и обработка на данни (HomeController)

Step 2 : Implement Edit Action that responds to HttpGet request

```
[HttpGet]
public ActionResult Edit(int id)
{
    Employee employee = _employeeRepository.GetEmployee(id);
    EmployeeEditViewModel employeeEditViewModel = new EmployeeEditViewModel
    {
        Id = employee.Id,
        Name = employee.Name,
        Email = employee.Email,
        Department = employee.Department,
        ExistingPhotoPath = employee.PhotoPath
    };
    return View(employeeEditViewModel);
}
```

3) показване/дисплей .cshtml (View)

Step 3 : Implement Edit View

```
@model EmployeeEditViewModel

@{
    ViewBag.Title = "Edit Employee";
    var photoPath = "~/images/" + (Model.ExistingPhotoPath ?? "noimage.jpg");

    <form asp-controller="home" asp-action="edit" enctype="multipart/form-data" method="post">

        <div asp-validation-summary="All" class="text-danger"></div>

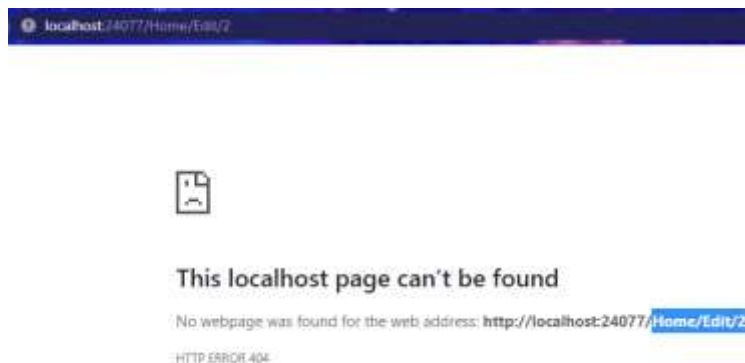
        <input hidden asp-for="Id" />
        <input hidden asp-for="ExistingPhotoPath" />

        Please refer to our blog at the following link for complete view code
        <a href="http://csharp-video-tutorials.blogspot.com/">http://csharp-video-tutorials.blogspot.com/</a>

    </form>
```

56 – HTTP post edit action

Към момента имаме списък със служители, възможност да го редактираме, но при натискане на Update btn, връща 404 резултат, т.к. няма страница, която да изведе променените данни, т.е. разполагаме само с Get – Edit, но не и с Post – Edit. Не сме го направили още.




```
Edit.cshtml | HomeController.cs | Index.cshtml
EmployeeManagement.Controllers.HomeController Edit(int id)

[HttpGet]
0 references
public IActionResult Edit(int id)
{
    Employee employee = _employeeRepository.GetEmployee(id);
    EmployeeEditViewModel employeeEditViewModel = new EmployeeEditViewModel
    {
        Id = employee.Id,
        Name = employee.Name,
        Email = employee.Email,
        Department = employee.Department,
        ExistingPhotoPath = employee.PhotoPath
    };
    return View(employeeEditViewModel);
}
```

HttpPost ще бъде сходен с Create, така че го копираме и изменяме.

```
[HttpPost]
0 references
public IActionResult Edit(EmployeeEditViewModel model)
{
```

В предходното видео създадохме

EmployeeEditViewModel = EmployeeCreateViewModel + PhotoPath.

Ако валидацията е изпълнена, искаме да извлечем наличните данни от БД. В противен случай да се върне страницата с return View(); Извличането става с присвояване на Id от model -> employee, което е достъпно заради Input hidden от Edit.cshtml. Понеже кодът за снимката е еднакъв за Create и Edit, избираме да го откъснем в отделен блок код, който да извикаме на двете места, вместо да го повтаряме същият чрез д.б. Quick Actions and Refactoring.

```
[HttpPost]
public IActionResult Edit(EmployeeEditViewModel model)
{
    if (ModelState.IsValid)
    {
        Employee employee = _employeeRepository.GetEmployee(model.Id);
        employee.Name = model.Name;
        employee.Email = model.Email;
        employee.Department = model.Department;

        string uniqueFileName = null;
        if (model.Photo != null)
        {
            string uploadFolder = Path.Combine(hostingEnvironment.WebRootPath + "/images");
            uniqueFileName = Guid.NewGuid().ToString() + '_' + model.Photo.FileName;
            string filePath = Path.Combine(uploadFolder, uniqueFileName);
            model.Photo.CopyTo(new FileStream(filePath, FileMode.Create));
        }

        Employee newEmployee = new Employee
        {
            Name = model.Name,
            Email = model.Email,
            Department = model.Department,
            PhotoPath = uniqueFileName
        };
        _employeeRepository.Add(newEmployee);
        return RedirectToAction("details", new { id = newEmployee.Id });
    }
}
```

Context menu options:

- Add View...
- Go To View (Ctrl+M, Ctrl+G)
- Quick Actions and Refactorings... (Ctrl+.)
- Rename... (Ctrl+R, Ctrl+R)
- Remove and Sort Usings (Ctrl+R, Ctrl+G)

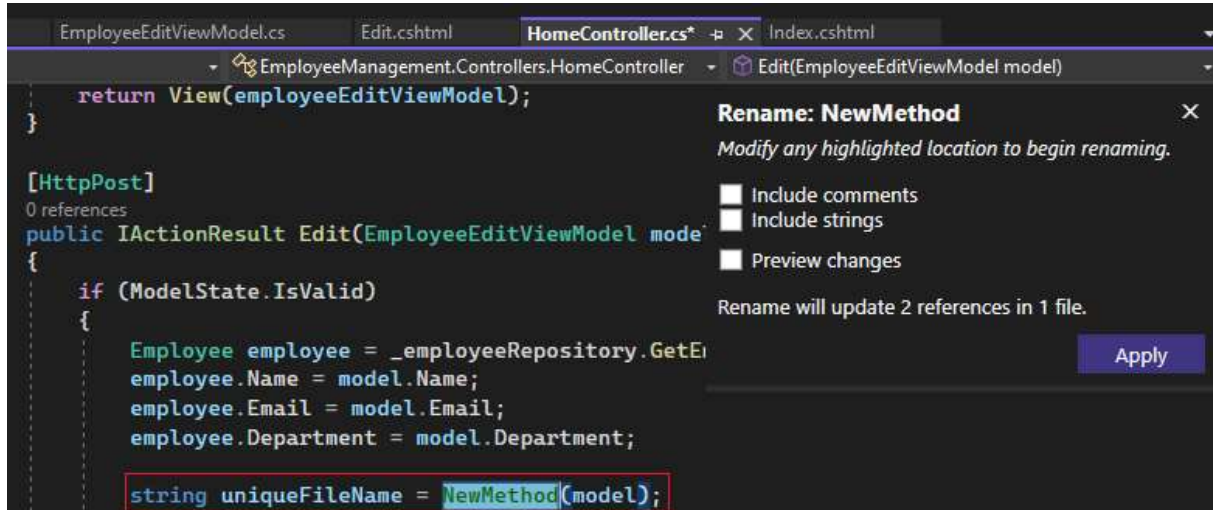
Маркираме редовете на кода, д.б. -> Quick Actions and Refactoring -> Extract method:

Extract method

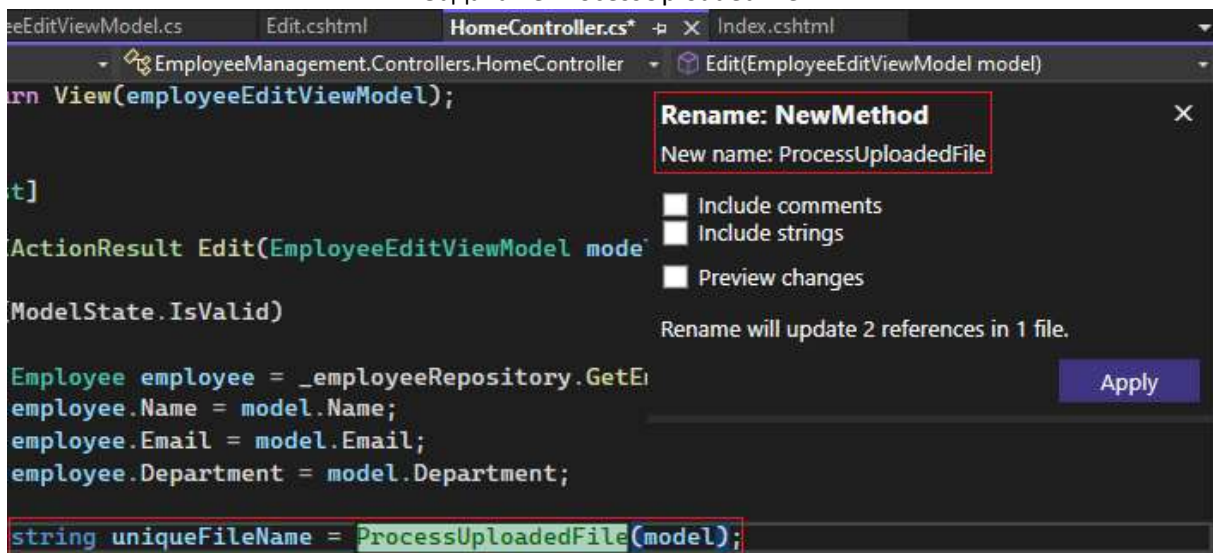
```
private string NewMethod(EmployeeEditViewModel model)
{
    string uniqueFileName = null;
    if (model.Photo != null)
    {
        string uploadFolder = Path.Combine(hostingEnvironment.WebRootPath + "/images");
        uniqueFileName = Guid.NewGuid().ToString() + '_' + model.Photo.FileName;
        string filePath = Path.Combine(uploadFolder, uniqueFileName);
        model.Photo.CopyTo(new FileStream(filePath, FileMode.Create));
    }

    Employee newEmployee = new Employee
    {
        Name = model.Name,
        Email = model.Email,
        Department = model.Department,
        PhotoPath = uniqueFileName
    };
    _employeeRepository.Add(newEmployee);
    return RedirectToAction("details", new { id = newEmployee.Id });
}
```

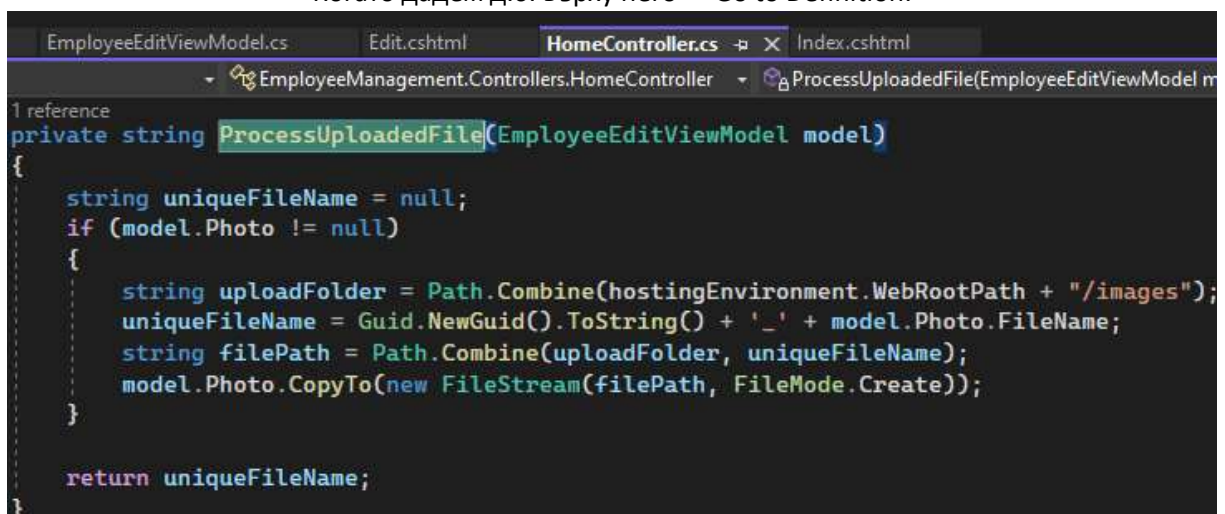
Целият код се замести с един ред, в който трябва да зададем ново име на метода, тип string



Задаваме ProcessUploadedFile



Когато дадем д.б. върху него -> Go to Definition:



Виждаме, че методът е изведен със същите редове код като последен в същия HomeController. За да спестим дубликата на всичките редове код с новия метод, който вече имаме, отиваме да ги заменим в метод Create:


```

[HttpPost]
0 references
public IActionResult Create(EmployeeCreateViewModel model)
{
    if (ModelState.IsValid)
    {
        string uniqueFileName = null;
        if (model.Photo != null)
        {
            string uploadFolder = Path.Combine(hostingEnvironment.WebRootPath + "/images");
            uniqueFileName = Guid.NewGuid().ToString() + '_' + model.Photo.FileName;
            string filePath = Path.Combine(uploadFolder, uniqueFileName);
            model.Photo.CopyTo(new FileStream(filePath, FileMode.Create));
        }
    }
}

```

Виждаме, че на променлива тип string uniqueFileName вместо null, е присвоен новият метод с подадена стойност model:

```

string uniqueFileName = ProcessUploadedFile(model);
Employee newEmployee = new Employee
{
    Name = model.Name,
    Email = model.Email,
}

```

CS1501: Argument 1: cannot convert from 'EmployeeManagement.ViewModels.EmployeeCreateViewModel' to 'EmployeeManagement.ViewModels.EmployeeEditViewModel'

model е с червено подчертаване, защото не може да конвертира от EmployeeCreateViewModel -> EmployeeEditViewModel. Понеже единия клас наследява другия (Edit наследява от Create), всичко, което е необходимо да направим, е върху метода ProcessUploadedFile д.б. -> Go to Definition -> където да сменим единия с другия модел:

```

2 references
private string ProcessUploadedFile(EmployeeEditViewModel model)

```

Т.е. от по-висшестоящия сложен да преминем към по-нисшия.

Ако разгледаме в този вид кода, заради наследяванията, на метод Create подаваме EmployeeCreateViewModel, докато на Edit подаваме EmployeeEditViewModel.

```

[HttpPost]
0 references
public IActionResult Create(EmployeeCreateViewModel model)
{
}

```

и

```

[HttpPost]
0 references
public IActionResult Edit(EmployeeEditViewModel model)
{
}

```

Ако не се предостави нова снимка, искаме да запазим съществуващата.

```

EmployeeEditViewModel.cs Edit.cshtml HomeController.cs Index.cshtml
<div class="custom-file">
    <input multiple asp-for="Photo" class="form-control custom-file-input" />
    <label class="custom-file-label">Click here to change photo</label>
</div>

```

Това може да стане, като погледнем в Edit.cshtml -> Photo property is not NULL.

Тогава искаме да се копира картинката в папката на компютъра чрез ProcessUploadedFile.


```

        if (model.Photo != null)
        {
            employee.PhotoPath = ProcessUploadedFile(model);
        }
        _employeeRepository.Update(employee);
        return RedirectToAction("index");
    }
    return View();
}

```

Остана да проверим дали вече е прикачена снимка и ако има такава, искаме да я изтрием.

```

Edit.cshtml  X HomeController.cs  EmployeeEdit
<input hidden asp-for="Id" />
<input hidden asp-for="ExistingPhotoPath" />

```

```

[HttpPost]
0 references
public IActionResult Edit(EmployeeEditViewModel model)
{
    if (ModelState.IsValid)
    {
        Employee employee = _employeeRepository.GetEmployee(model.Id);
        employee.Name = model.Name;
        employee.Email = model.Email;
        employee.Department = model.Department;
        if (model.Photo != null)
        {
            if(model.ExistingPhotoPath != null)
            {
                string filePath = Path.Combine(hostingEnvironment.WebRootPath,
                                                "images", model.ExistingPhotoPath);
                System.IO.File.Delete(filePath);
            }
            employee.PhotoPath = ProcessUploadedFile(model);
        }
        _employeeRepository.Update(employee);
        return RedirectToAction("index");
    }
    return View();
}

```

```

[HttpPost]
public IActionResult Edit(EmployeeEditViewModel model)
{
    if (ModelState.IsValid)
    {
        Employee employee = _employeeRepository.GetEmployee(model.Id);
        employee.Name = model.Name;
        employee.Email = model.Email;
        employee.Department = model.Department;
        if (model.Photo != null)
        {
            if(model.ExistingPhotoPath != null)
            {
                string filePath =
Path.Combine(hostingEnvironment.WebRootPath, "images", model.ExistingPhotoPath);

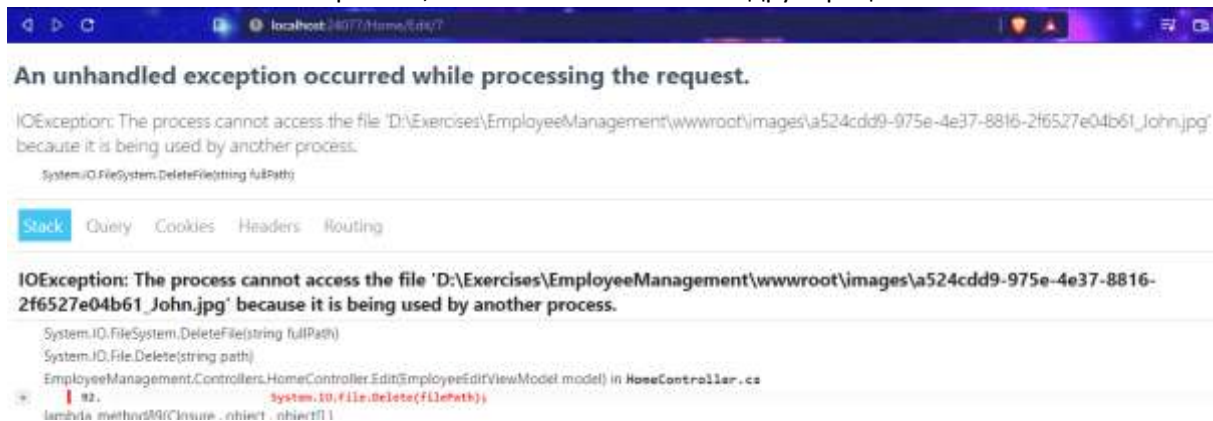
```

```

        System.IO.File.Delete(filePath);
    }
    employee.PhotoPath = ProcessUploadedFile(model);
}
_employeeRepository.Update(employee);
return RedirectToAction("index");
}
return View();
}

```

При мен, снимката се използва от друг процес.



Това е следващият ProcessUploadedFile. Трябва да освободим filePath, която го използва.

```

private string ProcessUploadedFile(EmployeeCreateViewModel model)
{
    string uniqueFileName = null;
    if (model.Photo != null)
    {
        string uploadFolder = Path.Combine(hostingEnvironment.WebRootPath + "/images");
        uniqueFileName = Guid.NewGuid().ToString() + "_" + model.Photo.FileName;
        string filePath = Path.Combine(uploadFolder, uniqueFileName);
        model.Photo.CopyTo(new FileStream(filePath, FileMode.Create));
    }

    return uniqueFileName;
}

```

За да го освободим, го отделяме в цикъл, при края на който, използването се прекратява. Въвеждаме нова променлива fileStream:

```

using (var fileStream = new FileStream(filePath, FileMode.Create))
{
    model.Photo.CopyTo(fileStream);
}

```

Сега вече грешката липсва и освен Update, работи и Delete на снимката от локалния диск.

```

private string ProcessUploadedFile(EmployeeCreateViewModel model)
{
    string uniqueFileName = null;
    if (model.Photo != null)
    {
        string uploadFolder = Path.Combine(hostingEnvironment.WebRootPath
+ "/images");
        uniqueFileName = Guid.NewGuid().ToString() + "_" +
model.Photo.FileName;
        string filePath = Path.Combine(uploadFolder, uniqueFileName);

```

```

        using (var fileStream = new FileStream(filePath,
        FileMode.Create))
        {
            model.Photo.CopyTo(fileStream);
        }

        return uniqueFileName;
    }
}

```

При това можем да избираме вече съществуваща снимка.

HttpPost Edit action

```

[HttpPost]
public IActionResult Edit(EmployeeEditViewModel model)
{
    if (ModelState.IsValid)
    {
        Employee employee = _employeeRepository.GetEmployee(model.Id);
        employee.Name = model.Name; employee.Email = model.Email;
        employee.Department = model.Department;
        if (model.Photo != null)
        {
            if (model.ExistingPhotoPath != null)
            {
                string filePath = Path.Combine(hostingEnvironment.WebRootPath,
                "images", model.ExistingPhotoPath);
                System.IO.File.Delete(filePath);
            }
            employee.PhotoPath = ProcessUploadedFile(model);
        }
        Employee updatedEmployee = _employeeRepository.Update(employee);
        return RedirectToAction("index");
    }
    return View(model);
}

```

ProcessUploadedFile Private Method

```

private string ProcessUploadedFile(EmployeeCreateViewModel model)
{
    string uniqueFileName = null;

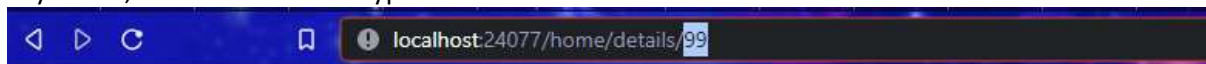
    if (model.Photo != null)
    {
        string uploadsFolder = Path.Combine(hostingEnvironment.WebRootPath, "images");
        uniqueFileName = Guid.NewGuid().ToString() + "_" + model.Photo.FileName;
        string filePath = Path.Combine(uploadsFolder, uniqueFileName);
        using (var fileStream = new FileStream(filePath, FileMode.Create))
        {
            model.Photo.CopyTo(fileStream);
        }
    }

    return uniqueFileName;
}

```

57 – Handling 404 not found

Има два типа грешки за липса, които се появяват: първата е за ресурс. Например към момента имаме 10 служителя, където URL е `http://localhost:24077/home/details/10`. Ако въведе обаче служител, който няма със сигурност:



An unhandled exception occurred while processing the request.

`NullReferenceException: Object reference not set to an instance of an object.`

`AspNetCoreGeneratedDocument.Views_Home_Details.ExecuteAsync() in Details.cshtml, line 4`

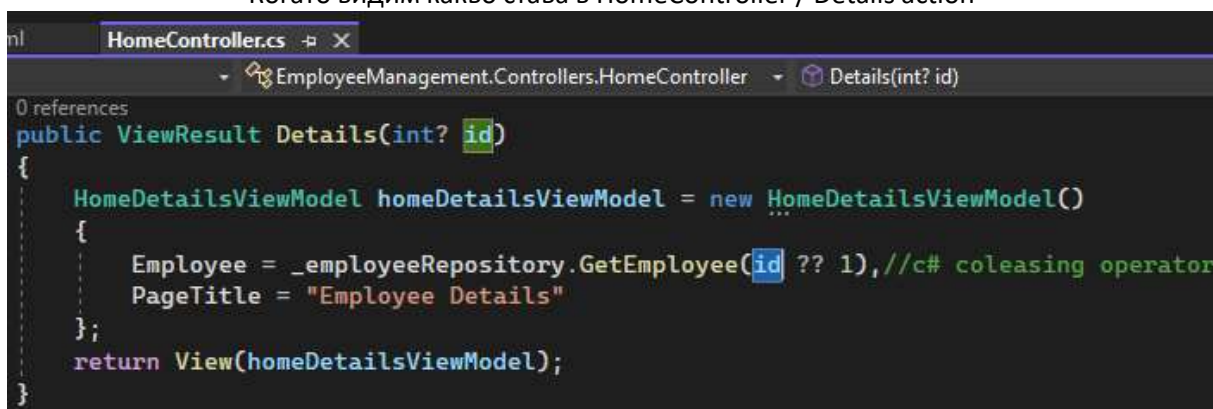
Stack Query Cookies Headers Routing

NullReferenceException: Object reference not set to an instance of an object.

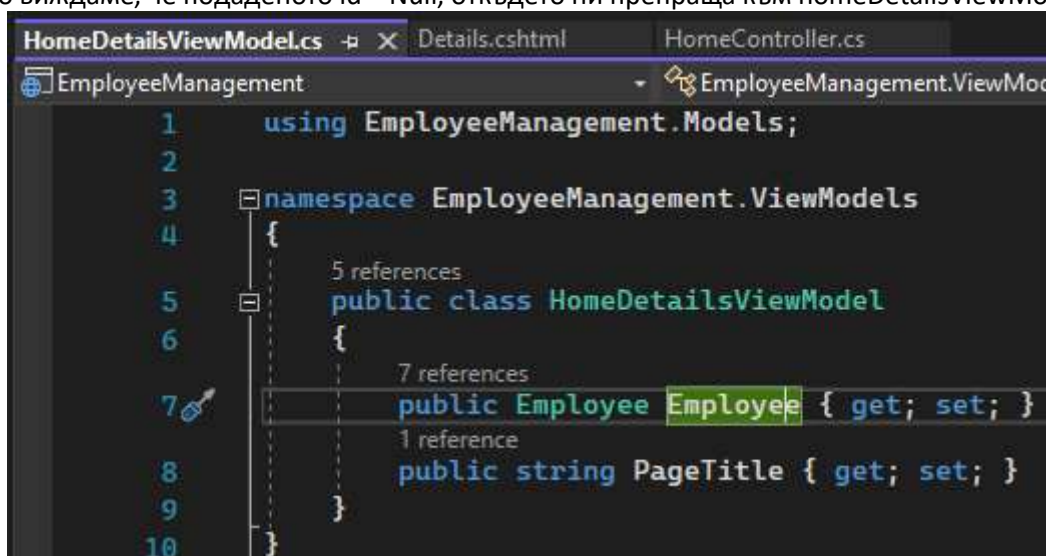
`AspNetCoreGeneratedDocument.Views_Home_Details.ExecuteAsync() in Details.cshtml`

```
4. var photoPath = "~/images/" + (Model.Employee.PhotoPath ?? "noimage.jpg");  
Microsoft.AspNetCore.Mvc.Razor.RazorView.RenderPageCoreAsync(IRazorPage page, ViewContext context)
```

Когато видим какво става в HomeController / Details action



Където виждаме, че подаденото `id = Null`, откъдето ни препраща към `homeDetailsViewModel`:



В което `Employee = Null` и при зареждане на страницата това е първото, което реагира като грешка в линия номер 4:


```

1 @model HomeDetailsViewModel
2 @{
3     ViewBag.Title = "Employee Details";
4     var photoPath = "~/images/" + (Model.Employee.PhotoPath ?? "noimage.jpg");
5 }

```

В този случай може да се предостави линк със списък на номерата на всички достъпни служители, за да се въведе актуален номер. Проверяваме дали има такъв служител с този номер и ако няма, извеждаме нова страница, в която показваме грешния номер.

```

0 references
public ActionResult Details(int? id)
{
    Employee employee = _employeeRepository.GetEmployee(id.Value);
    if (employee == null)
    {
        Response.StatusCode = 404;
        return View("EmployeeNotFound", id.Value);
    }
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = employee,
        PageTitle = "Employee Details"
    };
    return View(homeDetailsViewModel);
}

```

В противен случай, се изпълнява нормалния код, в който се създава нова инстанция за HomeDetailsViewModel, но в нея подаваме employee:

```

public ActionResult Details(int? id)
{
    Employee employee = _employeeRepository.GetEmployee(id.Value);

    if (employee == null)
    {
        Response.StatusCode = 404;
        return View("EmployeeNotFound", id.Value);
    }
    HomeDetailsViewModel homeDetailsViewModel = new
HomeDetailsViewModel()
    {
        Employee = employee,
        PageTitle = "Employee Details"
    };
    return View(homeDetailsViewModel);
}

```

Home->New Razor item -> View -> EmployeeNotFound.
Понеже подаваме id.Value, извикваме @model int, понеже стойността е integer.

```

1 @model int <- id.Value
2
3 @{
4     ViewBag.Title = "404 Error";
5 }

```

Two Types of 404 Errors

Type 1 : Resource with the specified ID does not exist.



Оформяме с bootstrap:

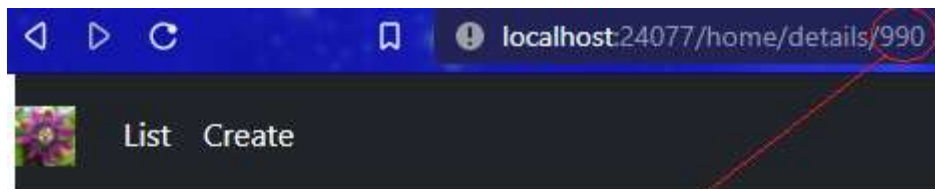
mt-1 = margin top 1

mb-1 = margin bottom 1

hr = horizontal line

```
<div class="alert alert-danger mt-1 mb-1">
  <h4>404 Not Found Error: </h4>
  <hr />
  <h5>
    Employee with ID = @Model cannot be found.
  </h5>
  <a asp-action="index" asp-controller="home" class="btn btn-outline-success">
    Click here to see the list of all employees
  </a>
</div>
```

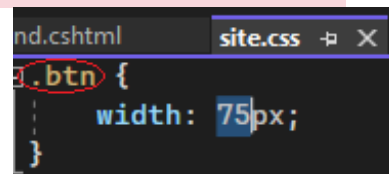
Като презаредим страницата, се вижда съобщението, като независимо кой грешен номер подадем в URL, той автоматично се прихваща и показва като съобщение.



404 Not Found Error:

Employee with ID = 990 cannot be found.

Click
here
to see
the list
of all
employees



Към момента, бутонът е по подразбиране 75px

Нека го коригираме на auto:

`style="width:auto">`

404 Not Found Error:

Employee with ID = 990 cannot be found.

Click here to see the list of all employees

```
@model int

@{
    ViewBag.Title = "404 Error";
}

<div class="alert alert-danger mt-1 mb-1">
    <h4>404 Not Found Error: </h4>
    <hr />
    <h5>
        Employee with ID = @Model cannot be found.
    </h5>
</div>

<a asp-action="index" asp-controller="home" class="btn btn-outline-success" style="width:auto">
    Click here to see the list of all employees
</a>
```

@model int

@{

```

        ViewBag.Title = "404 Error";
    }
    <div class="alert alert-danger mt-1 mb-1">
        <h4>404 Not Found Error: </h4>
        <hr />
        <h5>
            Employee with ID = @Model cannot be found.
        </h5>
    </div>
    <a asp-action="index" asp-controller="home" class="btn btn-outline-success"
    style="width:auto">
        Click here to see the list of all employees
    </a>

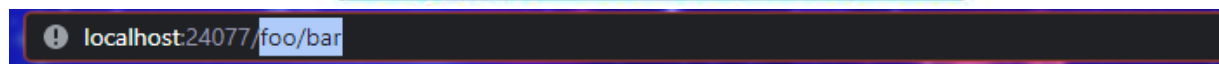
```

Вторият тип грешки е в URL, които връщат празна страница 404.

Two Types of 404 Errors

Type 2 : The URL does not match any route

<http://localhost/foo/bar>



This localhost page can't be found

No webpage was found for the web address: **http://localhost:24077/foo/bar**

HTTP ERROR 404

Reload

Ще ги коментираме в следващото видео.

58 - Centralised 404 error

Two Types of 404 Errors

Type 1 : Resource with the specified ID does not exist.

Type 2 : The URL does not match any route

<http://localhost/foo/bar>

Centralised 404 Error Handling

UseStatusCodePages

UseStatusCodePagesWithRedirects

UseStatusCodePagesWithReExecute

Ще обсъдим генерална грешка в страниците и тези 3 middleware компонента:

За демонстрация сменяме средата от Development -> Production:

```
Startup.cs launchSettings.json HomeContro
nastore.org/launchsettings.json
"ASPNETCORE_ENVIRONMENT": "Production",
```

И избираме последователно 3-те предложения.

```
Startup.cs* launchSettings.json HomeContro
EmployeeManagement.Startup
0 references
public void Configure(IApplicationBuilder
{
    if (env.IsDevelopment()) // bool, Is
    {
        app.UseDeveloperExceptionPage();
    }
    else
    {
        app.UseStatusCodePages();
    }
    app.UseS
    UseStatusCodePages
    UseStatusCodePagesWithRedirects
    UseStatusCodePagesWithReExecute
```

Резултатът от първия е почти идентичен с обикновено съобщение на 404 – няма информация, затова почти не се използва.

localhost:24077/foo/bar

Status Code: 404; Not Found

При втория вариант използваме пренасочване до специален контролер и URL /Error/404, където последния номер ще бъде автоматично генериран.

```

else
{
    app.UseStatusCodePagesWithRedirects("/Error/{0}");
}

```

new Error Controller
auto place Holder

Controller folder -> Add -> New controller: ErrorController.cs

Автокод:

```

using Microsoft.AspNetCore.Mvc;

namespace EmployeeManagement.Controllers
{
    public class ErrorController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }
    }
}

```

Прилагаме персонална логика:

```

using Microsoft.AspNetCore.Mvc;

namespace EmployeeManagement.Controllers
{
    public class ErrorController : Controller
    {
        [Route("/Error/{statusCode}")]
        public IActionResult HttpStatusCodeHandler(int statusCode)
        {
            switch (statusCode)
            {
                case 404:
                    ViewBag.ErrorMessage = "Sorry, the resource you requested could not be found";
                    break;
            }
            return View("NotFound");
        }
    }
}

```

Error 400-500
custom View

Създаваме въпросният View:

Views->Shared-> New item -> Razor View -> NotFound

```

@{
    ViewBag.PageTitle = "Not Found";
}

<h1>@ViewBag.ErrorMessage</h1>

<a asp-action="index" asp-controller="home">
    Click here to navigate to the home page
</a>

```

case 404:
ViewBag.ErrorMessage = "Sorry,
break;

При зареждане на URL: /foo/bar, автоматично се пренасочваше към /error/404, но грешката, която ми извеждаше беше 500:



This page isn't working

localhost is currently unable to handle this request.

HTTP ERROR 500

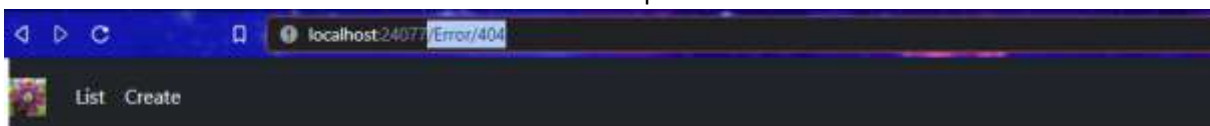
Reload

```
_ViewStart.cshtml NotFound.cshtml
@{
    Layout = "_Layout";
}

@* @if (User.IsInRole("Admin"))
    {
        Layout = "_AdminLayout";
    }
    else
    {
        Layout = "_NonAdminLayout";
    }
* @
```

В коментарите предлагаша преместване вместо в Shared, в друга папка Errors, което не помогна.

Срещнах отговор, че това е заради локалния _ViewStart.cshtml в Home, но извън него, генерално за папка Views се използва другия с админски код, затова го закоментирах, с което и грешката изчезна. Независимо какъв грешен адрес се подаде, имаме пренасочване, както и линка в него работи.



Sorry, the resource you requested could not be found

[Click here to navigate to the home page](#)

Може да използваме и последната опция, Execute, като от потребителска гледна точка, няма да има никаква промяна. Каква е разликата, ще видим в следващото видео.

```
app.UseStatusCodePagesWithReExecute("/Error/{0}");
```

59 - UseStatusCodePagesWithRedirects vs UseStatusCodePagesWithReExecute

```

yeeNotFound.cshtml  ErrorController.cs  Startup.cs  launchSettings.json  HomeController.c
EmployeeManagement.Startup  Configure(IApplicationBuilder a
0 references
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    if (env.IsDevelopment() // bool, IsStaging, IsProduction, IsEnvironment
    {
        app.UseDeveloperExceptionPage();
    }
    else
    {
        app.UseStatusCodePagesWithRedirects("/Error/{0}");
    }

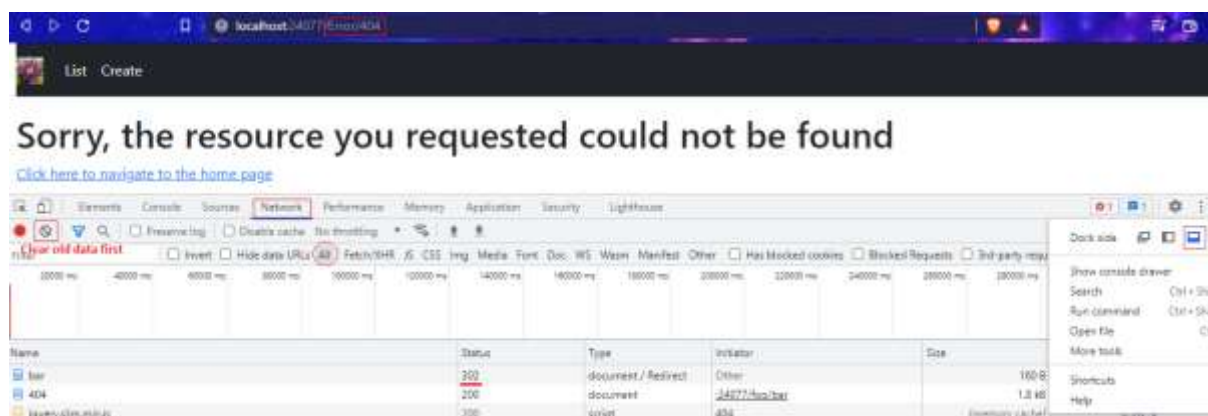
    app.UseStaticFiles(); //Short-circle ordering
    app.UseMvc(rb => //app.usemvcwithdefaultrouting(); error after .net 3.0
    {
        rb.MapRoute("default", "{controller=home}/{action=index}/{id?}");
    });

    app.UseRouting();

    app.UseEndpoints(endpoints =>

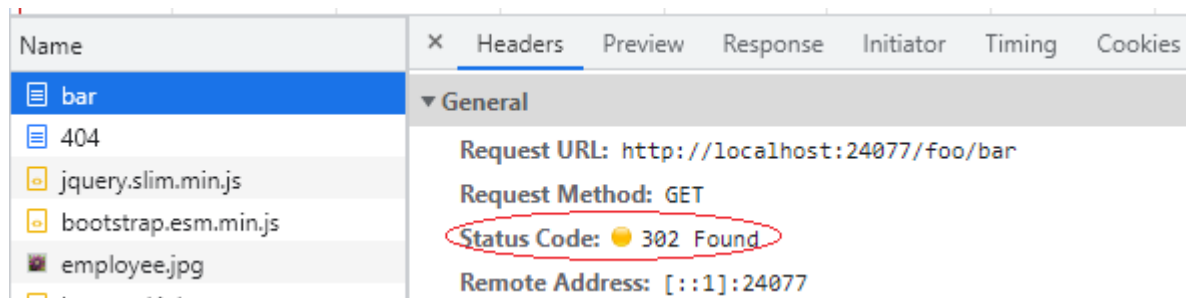
```

Когато пристигне грешен адрес, с Redirects обикаля последователно следващите middleware, вижда, че не е за тях – не е за StaticFiles, докато не стигне до MVCRoute, където съвпада с error/404, после се връща и в обратна посока намира Redirect. Когато занулим и заредим да погледнем какво става в Developer Tools, виждаме резултат bar -> Status 302 Found:



bar	302
404	200
jquery.slim.min.js	200

302 Found



Виждаме последователно с мишката:

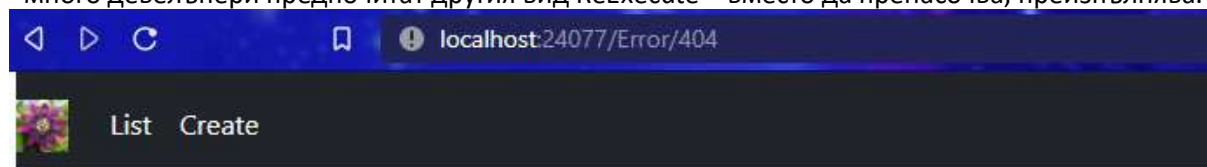
bar -> /foo/bar ... 302 found

404 -> /error/404 ... 200 OK

Т.е. последователно оглежда middleware един след друг и когато се връща, изпълнява MVC route -> NotFound заедно с код 200 OK. Ключовата дума тук е Redirect – пренасочва към /error/404

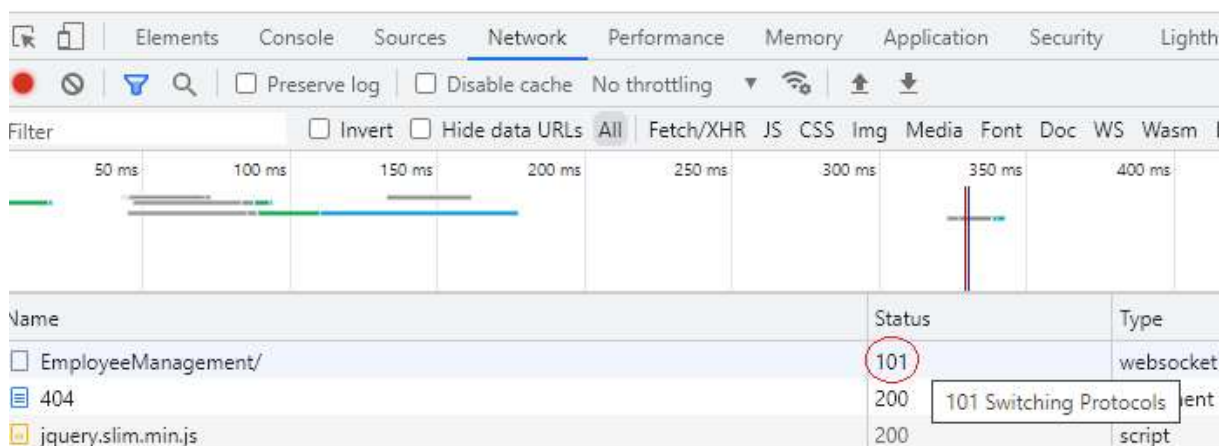
```
app.UseStatusCodePagesWithRedirects("/Error/{0}");
```

Ако обаче наистина се случи реална грешка, семантично всичко ще бъде изпълнено. Ето защо, много девелъпери предпочитат другия вид ReExecute – вместо да пренасочва, изпълнява.

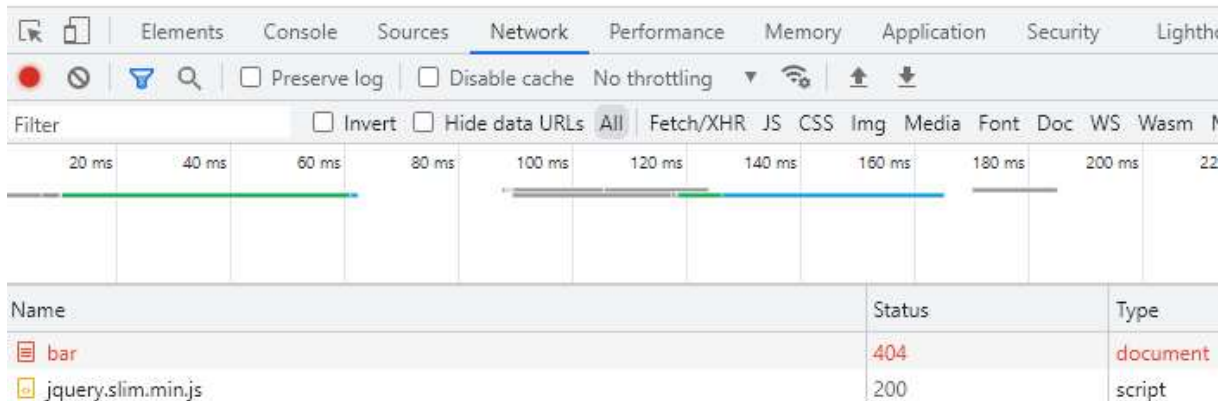


Sorry, the resource you requested cou

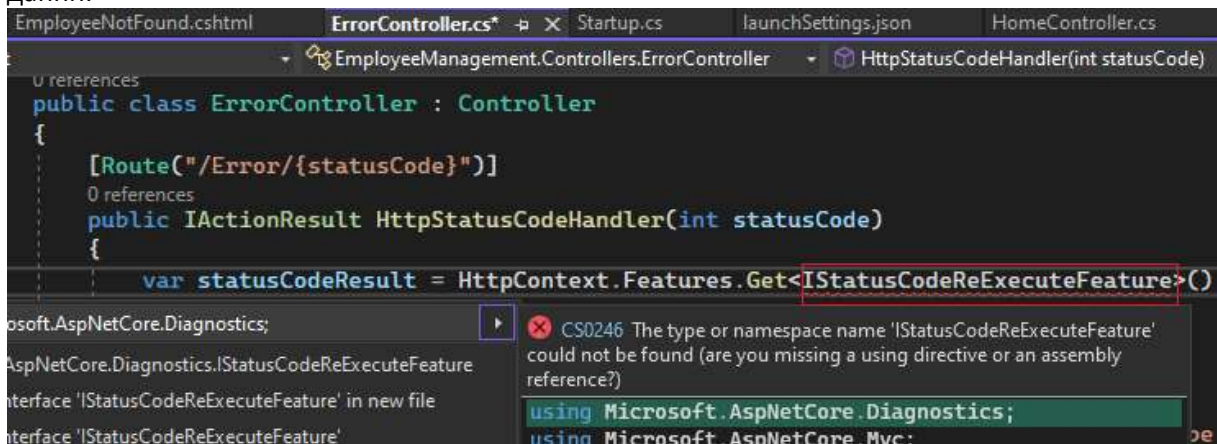
[Click here to navigate to the home page](#)



Вместо 302 имаме 404 – без пренасочване, но страницата пак се изпълнява.



Ехесите обаче има възможност за повече предоставяне на информация чрез прехвърляне на данни:



Въвеждаме 2 променливи за прехвърляне на данни – адрес и заявка:

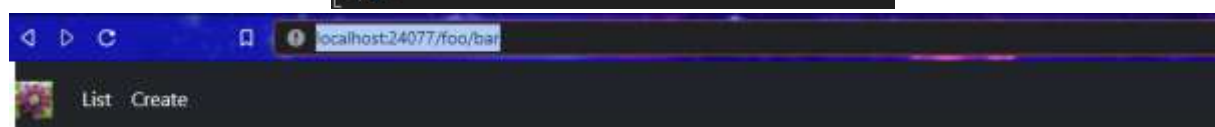
```
eeNotFound.cshtml | ErrorController.cs | Startup.cs | launchSettings.json | HomeController.cs
EmployeeManagement.Controllers.ErrorController | HttpStatusHandler(int statusCode)
[Route("/Error/{statusCode}")]
0 references
public IActionResult HttpStatusHandler(int statusCode)
{
    var statusCodeResult = HttpContext.Features.Get<IStatusCodeReExecuteFeature>();

    switch (statusCode)
    {
        case 404:
            ViewBag.ErrorMessage = "Sorry, the resource you requested could not be found";
            ViewBag.Path = statusCodeResult.OriginalPath;
            ViewBag.QS = statusCodeResult.OriginalQueryString;
            break;
    }
    return View("NotFound");
}
```

```
public IActionResult HttpStatusHandler(int statusCode)
{
    var statusCodeResult =
HttpContext.Features.Get<IStatusCodeReExecuteFeature>();

    switch (statusCode)
    {
        case 404:
            ViewBag.ErrorMessage = "Sorry, the resource you requested
could not be found";
            ViewBag.Path = statusCodeResult.OriginalPath;
            ViewBag.QS = statusCodeResult.OriginalQueryString;
            break;
    }
    return View("NotFound");
}
```

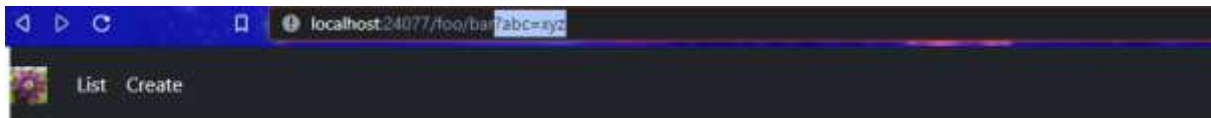
```
nd.cshtml | NotFound.cshtml | ErrorController.cs
@model int
@{
    ViewBag.PageTitle = "Not Found";
}
<h1>@ViewBag.ErrorMessage</h1>
<h1>@ViewBag.Path</h1>
<h1>@ViewBag.QS</h1>
<a asp-action="index" asp-controller="home">
    Click here to navigate to the home page
</a>
```



Sorry, the resource you requested could not be found
/foo/bar

[Click here to navigate to the home page](#)

При това положение, вече можем да видим повече:



Sorry, the resource you requested could not be found

/foo/bar path

?abc=xyz query string

[Click here to navigate to the home page](#)

Извод:

UseStatusCodePagesWithRedirects

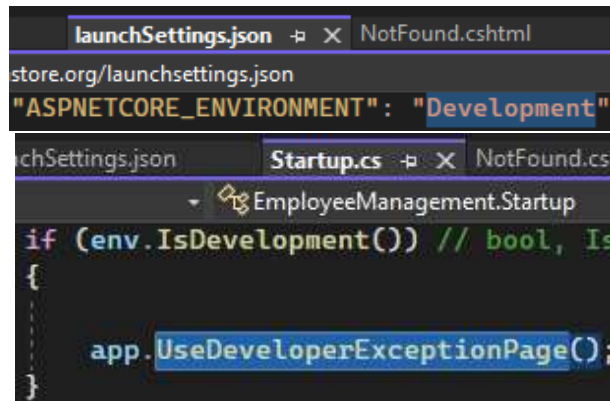
- Because a redirect is issued, the URL in the address bar changes
- Returns a success status code (200) when actually an error occurred which isn't semantically correct

UseStatusCodePagesWithReExecute

- Re-executes the pipeline and returns the original status code (404 for example)
- As it re-executes the pipeline and not issue a redirect request, we also preserve the original URL (/foo/bar) in the address bar

60 - Global exception handling

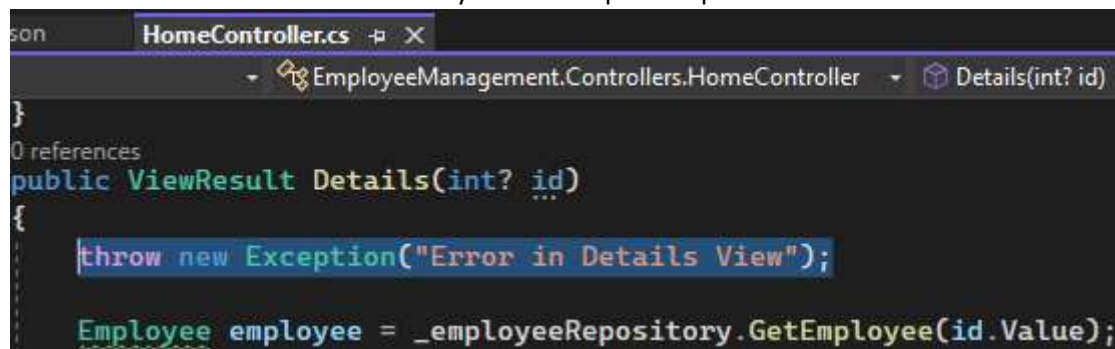
Сменяме средата, за да може в Development да използваме DeveloperExceptionPage. Както показва името ѝ, тя се използва в нея, т.к. предоставя много информация, което би било рисковано да се дава на потребителя в реална среда като Production.



```
launchSettings.json  X  NotFound.cshtml
store.org/launchsettings.json
"ASPNETCORE_ENVIRONMENT": "Development",

chSettings.json  Startup.cs  X  NotFound.cshtml
EmployeeManagement.Startup
if (env.IsDevelopment()) // bool, Is
{
    app.UseDeveloperExceptionPage();
}
```

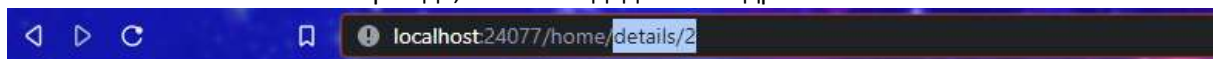
Нека изкуствено направим грешка:



```
son  HomeController.cs  X
EmployeeManagement.Controllers.HomeController  Details(int? id)
}
0 references
public IActionResult Details(int? id)
{
    throw new Exception("Error in Details View");

    Employee employee = _employeeRepository.GetEmployee(id.Value);
}
```

Details се зарежда, но когато дадем за подробности View btn:



An unhandled exception occurred while processing the request.

Exception: Error in Details View

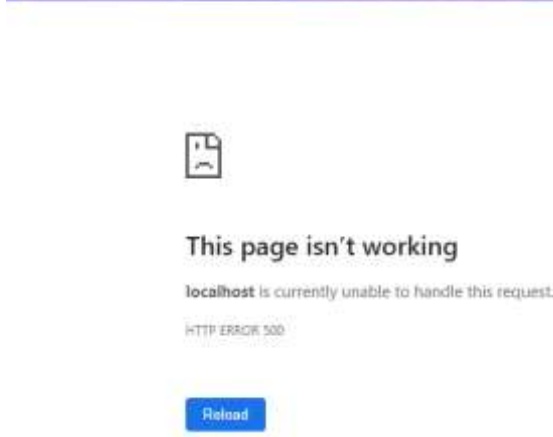
EmployeeManagement.Controllers.HomeController.Details(Nullable<int> id) in HomeController.cs, line 29

Stack Query Cookies Headers Routing

Exception: Error in Details View

EmployeeManagement.Controllers.HomeController.Details(Nullable<int> id) in HomeController.cs
+ 29. throw new Exception("Error in Details View");
lambda_method64(Closure, object, object[])

Ако сменям средата на Production, виждаме 404 error:



```
Startup.cs*
EmployeeManagement.Startup
if (env.IsDevelopment()) // bool, IsStaging, IsProduct
{
    app.UseDeveloperExceptionPage();
}
else
{
    app.UseExceptionHandler("/Error");
    app.UseStatusCodePagesWithReExecute("/Error/{0}");
}
```

Извън Development, в другите среди като Staging, Production, е удачно да се използва ExceptionHandler, в който можем да зададем пътя до страница, която желаем. В нашия случай, това е до ErrorController, който вече сме създали в предходния урок. В него виждаме само един метод, който отговаря за грешка 404, за останалите случаи трябва да създадем нов метод, който да бъде достъпен за всички потребители, независимо admin/user, затова е anonymous.

```
ErrorController.cs
HomeController.cs
Startup.cs*
EmployeeManagement.Controllers.ErrorController
Error()

[Route("Error")]
[AllowAnonymous]
0 references
public IActionResult Error()
{
    var exceptionDetails = HttpContext.Features.Get<IExceptionHandlerPathFeature>();

    ViewBag.ExceptionPath = exceptionDetails.Path;
    ViewBag.ExceptionMessage = exceptionDetails.Error.Message;
    ViewBag.Stacktrace = exceptionDetails.Error.StackTrace;

    return View("Error");
}
```

```
[Route("Error")]
[AllowAnonymous]
public IActionResult Error()
{
    var exceptionDetails =
HttpContext.Features.Get<IExceptionHandlerPathFeature>();

    ViewBag.ExceptionPath = exceptionDetails.Path;
    ViewBag.ExceptionMessage = exceptionDetails.Error.Message;
    ViewBag.Stacktrace = exceptionDetails.Error.StackTrace;

    return View("Error");
}
```

Съответно този създаваме този view Error, тип razor в папка Shared:

```

Error.cshtml | ErrorController.cs | HomeController.cs | Startup.cs
<h3>
    An occurred while processing your request.
    The support team is notified and we are working on the fix
</h3>
<h5>Please, contact us on support@abv.bg</h5>
<hr />
<h3>Exception Details:</h3>
<div class="alert alert-danger">
    <h5>Exception Path</h5>
    <hr />
    <p>@ViewBag.ExceptionPath</p>
</div>
<div class="alert alert-danger">
    <h5>Exception Message</h5>
    <hr />
    <p>@ViewBag.ExceptionMessage</p>
</div>
<div class="alert alert-danger">
    <h5>Exception Stack Trace</h5>
    <hr />
    <p>@ViewBag.StackTrace</p>
</div>

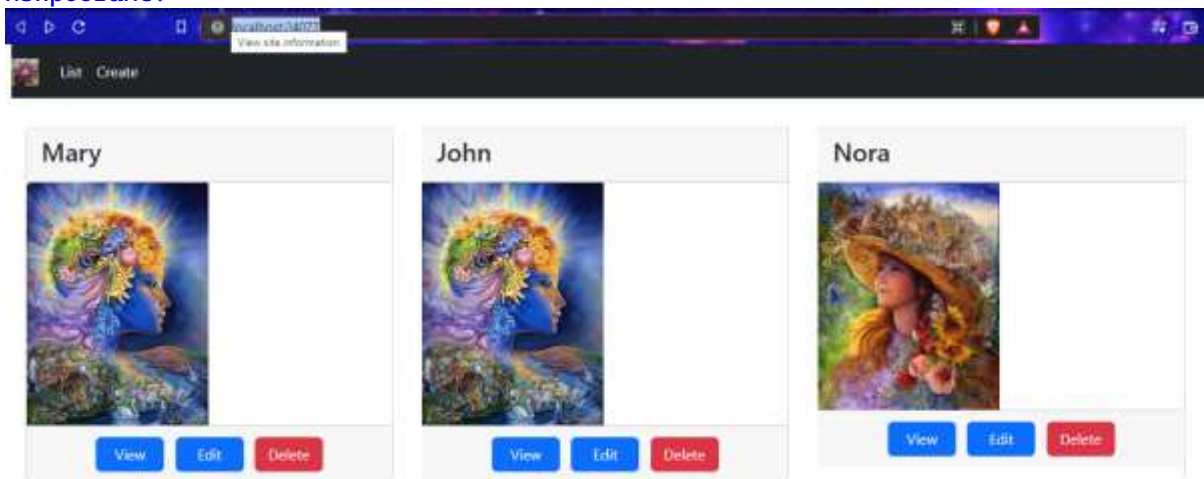
```

```

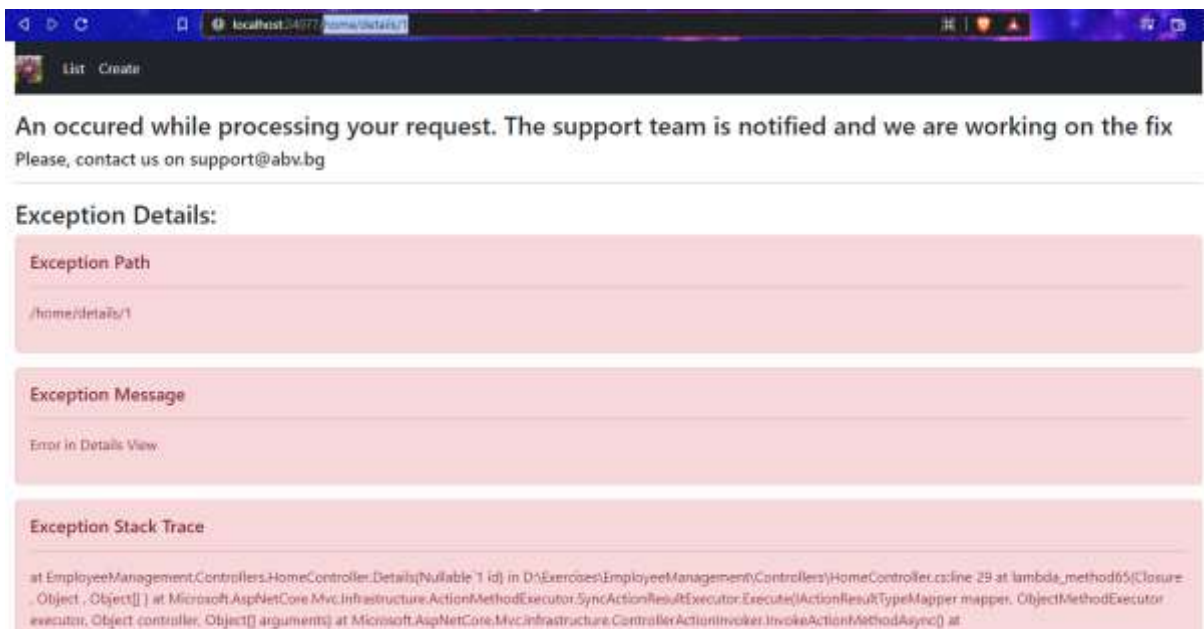
<h3>
    An occurred while processing your request.
    The support team is notified and we are working on the fix
</h3>
<h5>Please, contact us on support@abv.bg</h5>
<hr />
<h3>Exception Details:</h3>
<div class="alert alert-danger">
    <h5>Exception Path</h5>
    <hr />
    <p>@ViewBag.ExceptionPath</p>
</div>
<div class="alert alert-danger">
    <h5>Exception Message</h5>
    <hr />
    <p>@ViewBag.ExceptionMessage</p>
</div>
<div class="alert alert-danger">
    <h5>Exception Stack Trace</h5>
    <hr />
    <p>@ViewBag.StackTrace</p>
</div>

```

Изпробваме:



Като дадем на View бутон, виждаме персонализираното съобщение за грешка:



Трябва да запомним, че НИКОГА не даваме подобни подробни съобщения за грешки в Production environment, а тук е само за демонстрация!

В реално приложение само ще логнем грешката към БД или файл, за да може девелопъра да ги преразгледа и предостави fix.

Имаме 3 стъпки за съобщения за грешки:

Step 1 - Add the Exception Handling Middleware to the Request Processing Pipeline

```
public void Configure(IApplicationBuilder app, IHostingEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }
    else
    {
        app.UseExceptionHandler("/Error");
    }

    // Rest of the code
}
```

Step 2 : Implement ErrorController

```
public class ErrorController : Controller
{
    [AllowAnonymous]
    [Route("Error")]
    public IActionResult Error()
    {
        // Retrieve the exception Details
        var exceptionHandlerPathFeature =
            HttpContext.Features.Get<IExceptionHandlerPathFeature>();

        ViewBag.ExceptionPath = exceptionHandlerPathFeature.Path;
        ViewBag.ExceptionMessage = exceptionHandlerPathFeature.Error.Message;
        ViewBag.StackTrace = exceptionHandlerPathFeature.Error.StackTrace;

        return View("Error");
    }
}
```

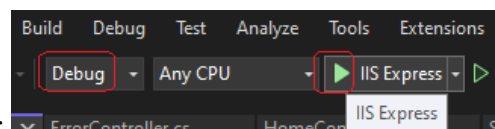
Step 3 : Implement Error View

```
<h3>An occurred processing your request</h3>
<h5>Please contact us on pragim@pragimtech.com</h5>
<hr />
<h3>Exception Details:</h3>
<div class="alert alert-danger">
    <h5>Exception Path</h5><hr />
    <p>@ViewBag.ExceptionPath</p>
</div>

<div class="alert alert-danger">
    <h5>Exception Message</h5><hr />
    <p>@ViewBag.ExceptionMessage</p>
</div>

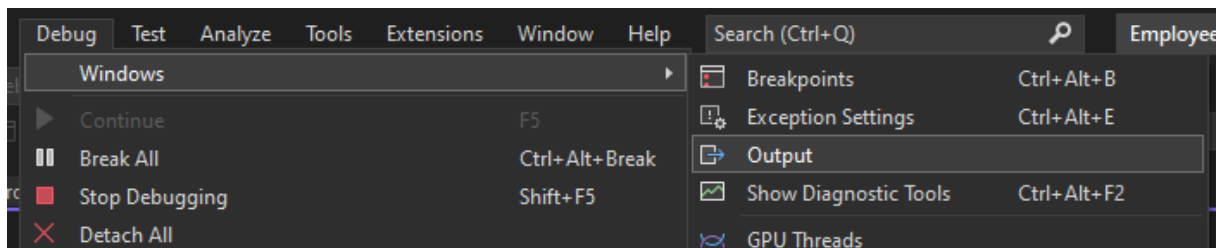
<div class="alert alert-danger">
    <h5>Exception Stack Trace</h5><hr />
    <p>@ViewBag.StackTrace</p>
</div>
```

61 – Logging

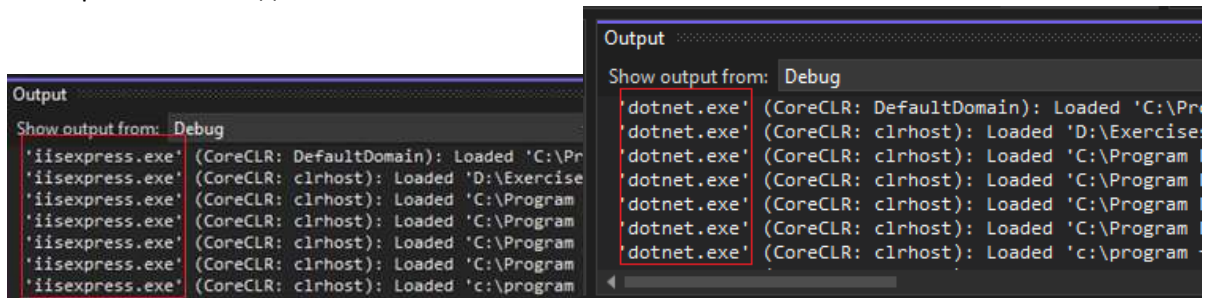


За да разберем процеса, ще стартираме в Debug mode:

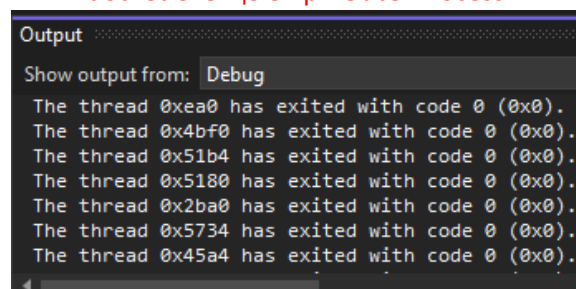
И разгледаме процесите в Output window:



Изхвърлят се много данни:

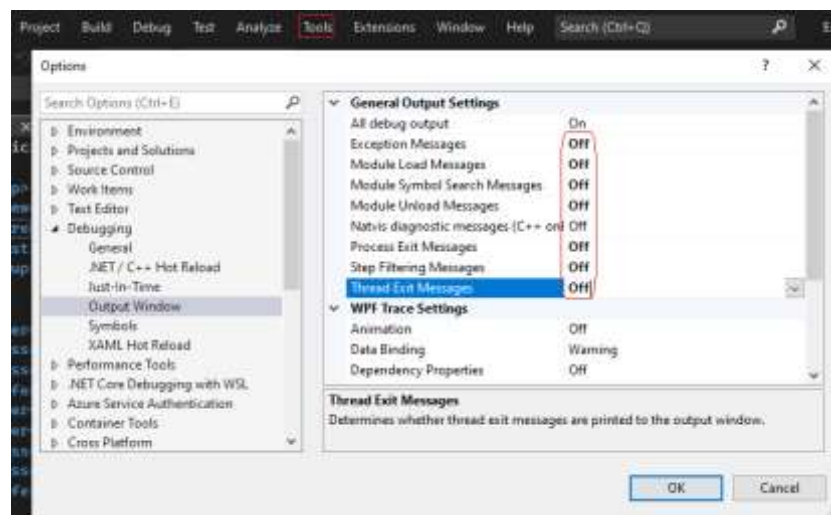


Като в първата част виждаме
iisexpress.exe, защото сме в InProcess hosting,
dotnet.exe ще е при OutOfProcess.

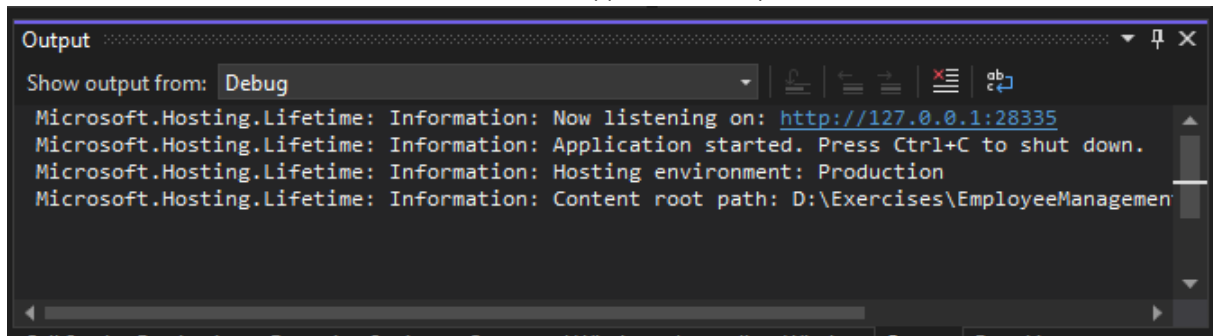


По-надолу code 0.

За да изключим всички тези излишни съобщения:



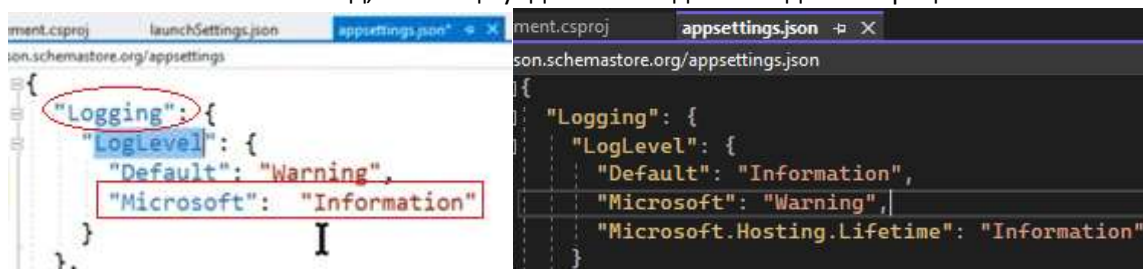
Вече нямаме подобни съобщения:



```
Output
Show output from: Debug
Microsoft.Hosting.Lifetime: Information: Now listening on: http://127.0.0.1:28335
Microsoft.Hosting.Lifetime: Information: Application started. Press Ctrl+C to shut down.
Microsoft.Hosting.Lifetime: Information: Hosting environment: Production
Microsoft.Hosting.Lifetime: Information: Content root path: D:\Exercises\EmployeeManagement
```

Спираме процеса.

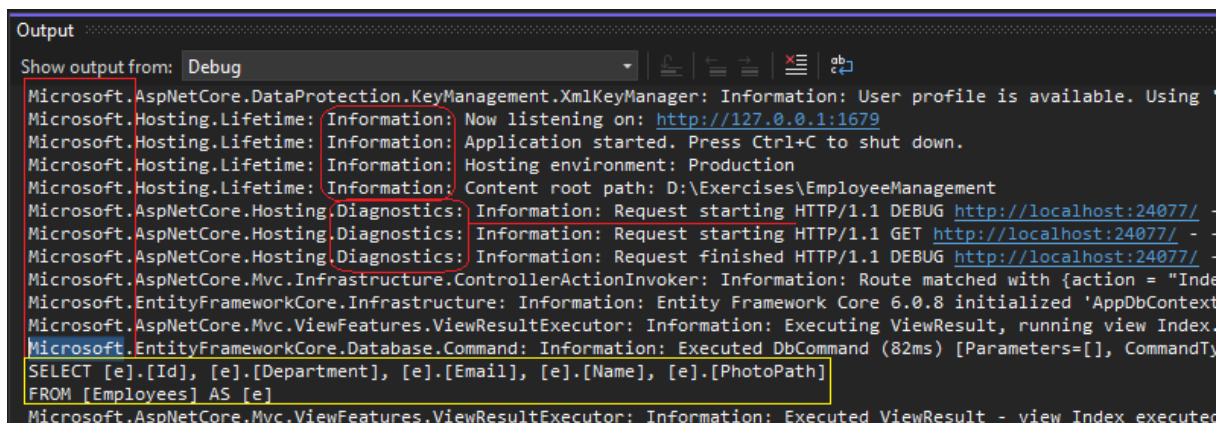
В туториъла нямаме "Microsoft": "Information" и го добавяме, а при мен по подразбиране е с повече код, който ще уеднаквя с видеото за демонстрация.



```
appsettings.json
{
  "Logging": {
    "LogLevel": {
      "Default": "Warning",
      "Microsoft": "Information"
    }
  }
}

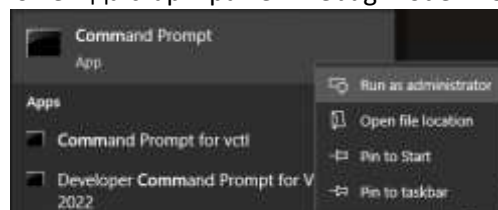
appsettings.json
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  }
}
```

По този начин имаме много информация в Debug mode:



```
Output
Show output from: Debug
Microsoft.AspNetCore.DataProtection.KeyManagement.XmlKeyManager: Information: User profile is available. Using
Microsoft.Hosting.Lifetime: Information: Now listening on: http://127.0.0.1:1679
Microsoft.Hosting.Lifetime: Information: Application started. Press Ctrl+C to shut down.
Microsoft.Hosting.Lifetime: Information: Hosting environment: Production
Microsoft.Hosting.Lifetime: Information: Content root path: D:\Exercises\EmployeeManagement
Microsoft.AspNetCore.Hosting.Diagnostics: Information: Request starting HTTP/1.1 DEBUG http://localhost:24077/ -
Microsoft.AspNetCore.Hosting.Diagnostics: Information: Request starting HTTP/1.1 GET http://localhost:24077/ -
Microsoft.AspNetCore.Hosting.Diagnostics: Information: Request finished HTTP/1.1 DEBUG http://localhost:24077/ -
Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker: Information: Route matched with {action = "Index"}
Microsoft.EntityFrameworkCore.Infrastructure: Information: Entity Framework Core 6.0.8 initialized 'AppDbContext'
Microsoft.AspNetCore.Mvc.ViewFeatures.ViewResultExecutor: Information: Executing ViewResult, running view Index.
Microsoft.EntityFrameworkCore.Database.Command: Information: Executed DbCommand (82ms) [Parameters=[], CommandType=Text, CommandText="SELECT [e].[Id], [e].[Department], [e].[Email], [e].[Name], [e].[PhotoPath] FROM [Employees] AS [e]"]
Microsoft.AspNetCore.Mvc.ViewFeatures.ViewResultExecutor: Information: Executed ViewResult - view Index executed
```

.Net = cross-platform, можем да стартираме в Debug mode in CLI като администратор:




```

C:\Windows\system32>cd /d d:\Exercises\EmployeeManagement
d:\Exercises\EmployeeManagement>dotnet run
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: d:\Exercises\EmployeeManagement

```

След зареждане на адрес Localhost:5000, всяко последващо действие се отразява в CMD, който продължава да слуша в Debug mode и извежда информация:

```

fail: Microsoft.AspNetCore.Diagnostics.DeveloperExceptionPageMiddleware[1]
      An unhandled exception has occurred while executing the request.
      System.Exception: Error in Details View
         at EmployeeManagement.Controllers.HomeController.Details(Nullable`1 id) in d:\Exercises\EmployeeManagement\Controllers\HomeController.cs:line 29
         at lambda_method64(Closure , Object , Object[])
         at Microsoft.AspNetCore.Mvc.Infrastructure.ActionMethodExecutor.SyncActionResultExecutor.Execute(Object mapper, ObjectMethodExecutor executor, Object controller, Object[] arguments)
         at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeActionMethodAsync()
         at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.Next(State& nextActionType, State& nextActionName, State& nextActionArguments, ActionDescriptor action)

```

ASP.NET Core Logging Provider

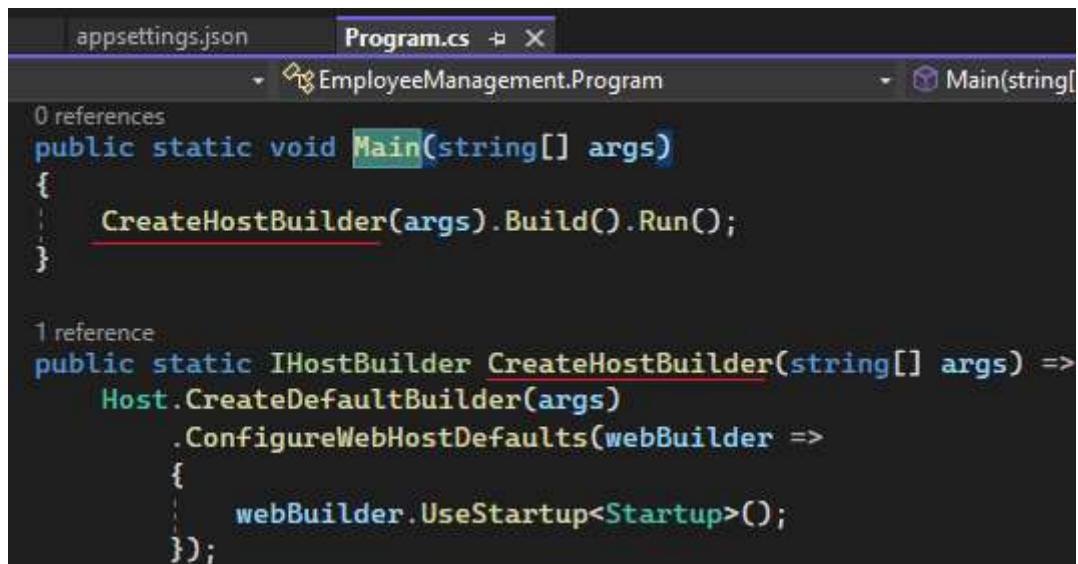
Logging Provider Stores or Displays Logs

Console Logging Provider

Displays logs on the Console

Debug Logging Provider

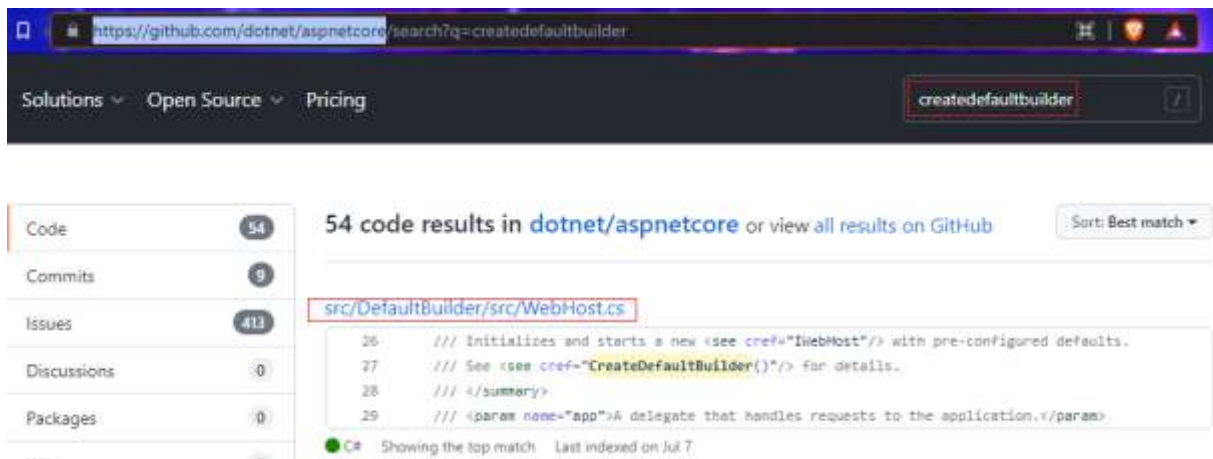
Displays logs on the Debug window in Visual Studio



The screenshot shows a Visual Studio editor window with two tabs: 'appsettings.json' and 'Program.cs'. The 'Program.cs' tab is active, showing the following code:

```
0 references
public static void Main(string[] args)
{
    CreateHostBuilder(args).Build().Run();
}

1 reference
public static IHostBuilder CreateHostBuilder(string[] args) =>
    Host.CreateDefaultBuilder(args)
        .ConfigureWebHostDefaults(webBuilder =>
        {
            webBuilder.UseStartup<Startup>();
        });
```



The screenshot shows a web browser window displaying a GitHub search results page. The address bar shows the URL: <https://github.com/dotnet/aspnetcore/search?q=createdefaultbuilder>. The search results show 54 code results in dotnet/aspnetcore. The top result is highlighted, showing the file path `src/DefaultBuilder/src/WebHost.cs` and the following code snippet:

```
26 /// Initializes and starts a new <see cref="IWebHost"/> with pre-configured defaults.
27 /// See <see cref="CreateDefaultBuilder()"/> for details.
28 /// </summary>
29 /// <param name="app">A delegate that handles requests to the application.</param>
```

Където виждаме, че той отговаря за Logging и се предоставят 3 възможности за целта – Console, Debug, EventSource:

```
}

builder.ConfigureAppConfiguration((hostingContext, config) =>
{
    var env = hostingContext.HostingEnvironment;

    config.AddJsonFile("appsettings.json", optional: true, reloadOnChange: true)
        .AddJsonFile($"appsettings.{env.EnvironmentName}.json", optional: true, reloadOnChange: true);

    if (env.IsDevelopment())
    {
        if (!string.IsNullOrEmpty(env.ApplicationName))
        {
            var appAssembly = Assembly.Load(new AssemblyName(env.ApplicationName));
            if (appAssembly != null)
            {
                config.AddUserSecrets(appAssembly, optional: true);
            }
        }
    }

    config.AddEnvironmentVariables();

    if (args != null)
    {
        config.AddCommandLine(args);
    }
})
.ConfigureLogging((hostingContext, loggingBuilder) =>
{
    loggingBuilder.Configure(options =>
    {
        options.ActivityTrackingOptions = ActivityTrackingOptions.SpanId
            | ActivityTrackingOptions.TraceId
            | ActivityTrackingOptions.ParentId;
    });
    loggingBuilder.AddConfiguration(hostingContext.Configuration.GetSection("Logging"));
    loggingBuilder.AddConsole();
    loggingBuilder.AddDebug();
    loggingBuilder.AddEventSourceLogger();
})
}
```

logging

<https://github.com/dotnet/aspnetcore/blob/c4efbfb1f9a3b6241140150ae9abf219bde16b2/src/DefaultBuilder/src/WebHost.cs>

Но освен тях, има още:

Built-in Logging Providers

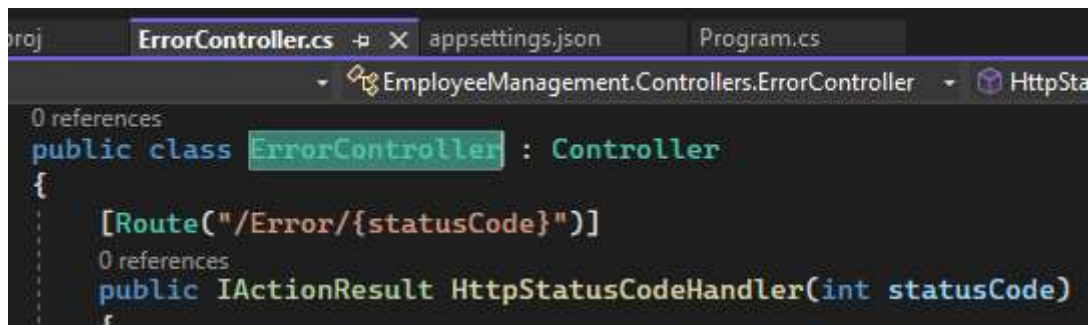
Console	TraceSource
Debug	AzureAppServicesFile
EventSource	AzureAppServicesBlob
EventLog	ApplicationInsights

Third Party Logging Providers

NLog	Gelf
elmah	JSNLog
Serilog	KissLog.net
Sentry	Loggr

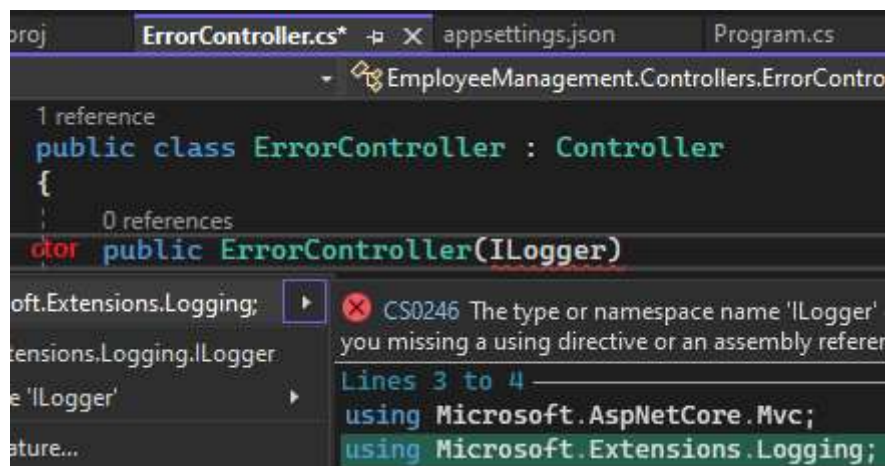
62 - Logging exceptions

В CMD виждаме много информация, а в Debug mode, стартиран от VS – output виждаме също, но за по-голяма пълнота въвеждаме допълнителен код. Към момента, грешките се прихващат от `ErrorHandler`:

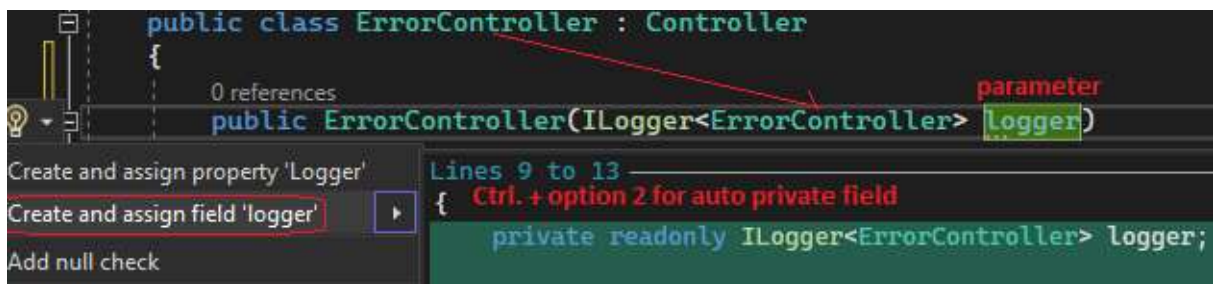


```
public class ErrorController : Controller
{
    [Route("/Error/{statusCode}")]
    public IActionResult HttpStatusCodeHandler(int statusCode)
    {
    }
```

Ако има изключение, ние искаме информация за него. За целта първо трябва да вмъкнем ILoggerInterface, предоставен за целта. С него ще може и да групираме съобщенията за грешки.

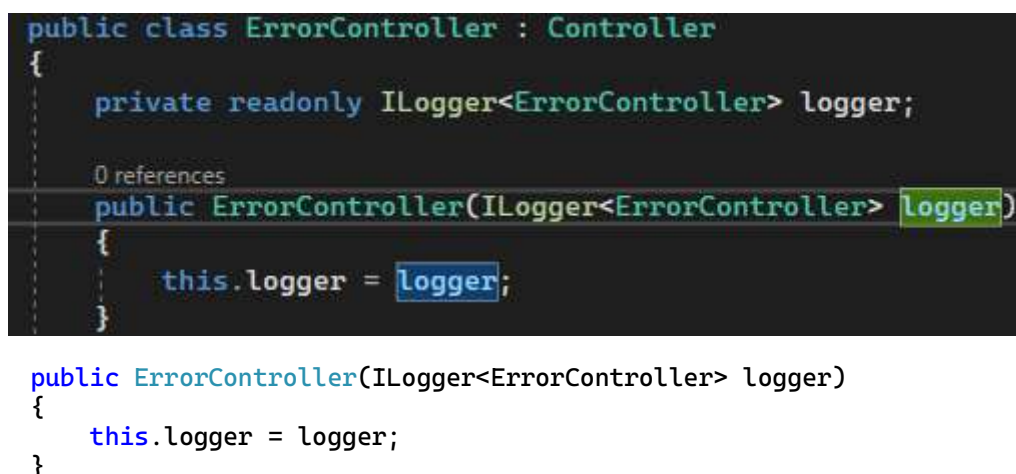


```
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Logging;
```



```
public ErrorController(ILogger<ErrorController> logger)
{
    private readonly ILogger<ErrorController> logger;
```

Вече тази нова променлива от Private field може да се използва навсякъде в кода по-надолу.



```
public class ErrorController : Controller
{
    private readonly ILogger<ErrorController> logger;

    public ErrorController(ILogger<ErrorController> logger)
    {
        this.logger = logger;
    }
}
```

```

ErrorController.cs  appsettings.json  Program.cs
- EmployeeManagement.Controllers.ErrorController  Error()
[Route("Error")]
[AllowAnonymous]
0 references
public IActionResult Error()
{
    var exceptionDetails = HttpContext.Features.Get<IExceptionHandlerPathFeature>();

    logger.LogError($"The path {exceptionDetails.Path} " +
        $"threw an exception {exceptionDetails.Error}");

    ViewBag.ExceptionPath = exceptionDetails.Path;
    ViewBag.ExceptionMessage = exceptionDetails.Error.Message;
    ViewBag.Stacktrace = exceptionDetails.Error.StackTrace;

    return View("Error");
}

```

Използваме го в Error action, където включваме **\$ = string interpolation**, вмъкване на стринг в съобщението за грешка, където параметрите са вкарани **{между}**. При нов ред използваме **+**.

```

logger.LogError($"The path {exceptionDetails.Path} " +
    $"threw an exception {exceptionDetails.Error}");

```

В същото време, вече искаме тези съобщения да се изведат автоматично на екрана, вместо стария View Error, затова отиваме в папка Shared и ги изчистваме (аз закоментирам).

```

gement.csproj  Error.cshtml  ErrorController.cs  appsettings.json  Prog
<h3>
    An occurred while processing your request.
    The support team is notified and we are working on the fix
</h3>
<h5>Please, contact us on support@abv.bg</h5>
<hr />
@* <h3>Exception Details:</h3>
<div class="alert alert-danger">
    <h5>Exception Path</h5>
<hr />

```

В ErrorController имаме по-нагоре още един метод, който отговаря за грешен URL с 404, но вместо информация за грешката, искаме да я регистрираме, така че отново закоментираме:

```

csproj  Error.cshtml  ErrorController.cs  appsettings.json  Program.cs
EmployeeManagement.Controllers.ErrorController  HttpStatusHandler(int statusCode)
[Route("/Error/{statusCode}")]
0 references
public IActionResult HttpStatusHandler(int statusCode)
{
    var statusCodeResult = HttpContext.Features.Get<IStatusCodeReExecuteFeature>();

    switch (statusCode)
    {
        case 404:
            ViewBag.ErrorMessage = "Sorry, the resource you requested could not be found";
            logger.LogWarning($"404 Error Occured. Path = {statusCodeResult.OriginalPath}" +
                $"and QueryString = {statusCodeResult.OriginalQueryString}");
            //ViewBag.Path = statusCodeResult.OriginalPath;
            //ViewBag.QS = statusCodeResult.OriginalQueryString;
            break;
    }

    return View("NotFound");
}

```

logger.LogWarning(\$"404 Error Occured. Path = {statusCodeResult.OriginalPath}" +
 \$"and QueryString = {statusCodeResult.OriginalQueryString}");

Последно изчистваме NotFound View в папка Shared:

```

ment.csproj  NotFound.cshtml  Error.cshtml
@model int
@{
    ViewBag.PageTitle = "Not Found";
}
<h1>@ViewBag.ErrorMessage</h1>
@*<h1>@ViewBag.Path</h1>
<h1>@ViewBag.QS</h1>*@
<a asp-action="index" asp-controller="home">
    Click here to navigate to the home page
</a>

```

```

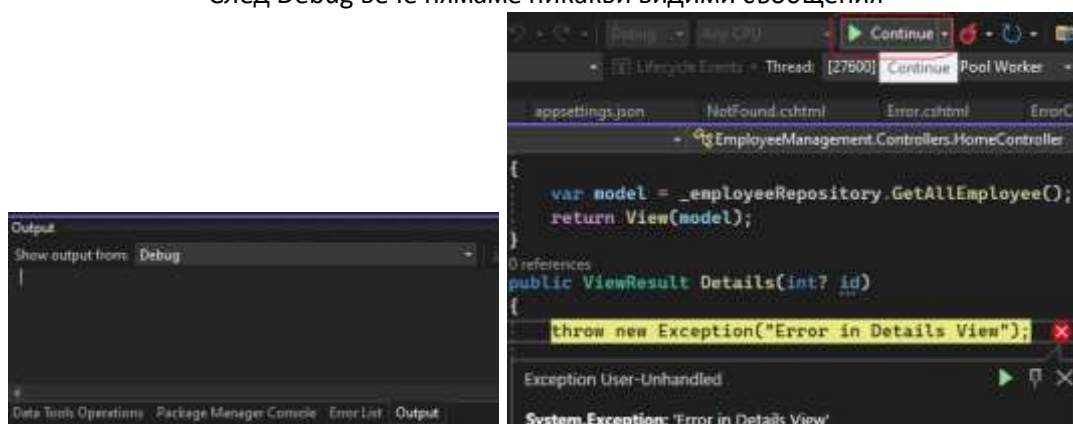
ent.csproj  appsettings.json  NotF
n.schemastore.org/appsettings.json

"Logging": {
  "LogLevel": {
    "Default": "Warning",
    "Microsoft": "Information"
  }
}

```

Когато стартираме в Debug, отново имаме прекалено много съобщения, започващи с Microsoft, затова, за да оставим само предупрежденията, Microsoft: Information -> Warning

След Debug вече нямаме никакви видими съобщения



Където имаме нова възможност за старт: Continue:



An occurred while processing your request. The support team is notified and we are working on the fix
Please, contact us on support@abv.bg

В противен случай би се видяло:



HTTP Error 502.3 - Bad Gateway

The connection with the server was terminated abnormally

Most likely causes:

- The CGI application did not return a valid set of HTTP errors.
- A server acting as a proxy or gateway was unable to process the request due to an error in a parent gateway.

Things you can try:

- Use DebugDiag to troubleshoot the CGI application.
- Determine if a proxy or gateway is responsible for this error.

Detailed Error Information:

Module	AspNetCoreModuleV2
Notification	ExecuteRequestHandler
Handler	aspNetCore
Error Code	0x80072efe

Requested URL	http://localhost:24077/home/details/1
Physical Path	D:\Exercises\EmployeeManagement\home\details\1
Logon Method	Anonymous
Logon User	Anonymous

Виждаме:


```
public class ErrorController : Controller
{
    private readonly ILogger<ErrorController> logger;

    public ErrorController(ILogger<ErrorController> logger)
    {
    }

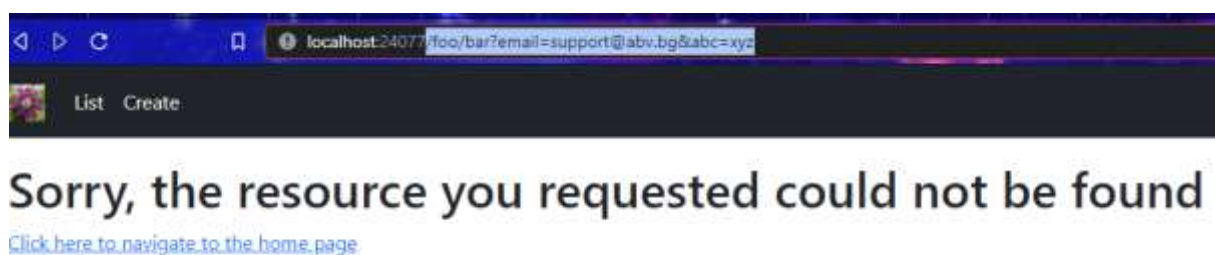
    [Route("/Error/{statusCode}")]
    public IActionResult HttpStatusCodeHandler(int statusCode)
    {
    }

    [Route("Error")]
    [AllowAnonymous]
    public IActionResult Error()
    {
        var exceptionDetails = HttpContext.Features.Get<IExceptionHandlerDetails>();
        logger.LogError($"The path {exceptionDetails.Path} " +
            $"threw an exception {exceptionDetails.Error}");
    }
}
```

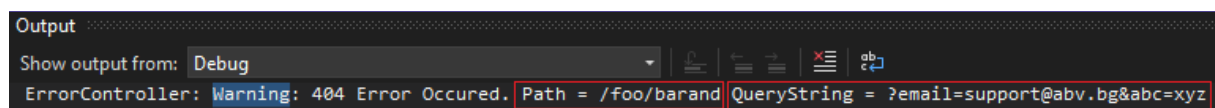
Debug

Microsoft.AspNetCore.Diagnostics.ExceptionHandlerMiddleware: Error: 404 Error Occured. Path = /foo/bar and QueryString = ?email=support@abv.bg&abc=xyz

Още един опит при несъществуващ адрес с променливи:



Където в Debug извежда 1 предупреждение:



Logging Exceptions in ASP.NET Core

- Inject an instance of **ILogger**
- Specify the **type of the Controller** into which **ILogger** is injected as the generic argument

```
public class ErrorController : Controller
{
    private readonly ILogger<ErrorController> logger;

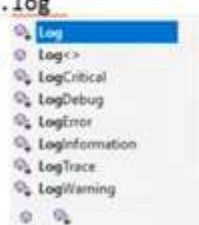
    public ErrorController(ILogger<ErrorController> logger)
    {
        this.logger = logger;
    }

    // Rest of the code
}
```

Logging Exceptions in ASP.NET Core

- Use the injected **ILogger** instance methods to log messages
 - **LogInformation**
 - **LogWarning**
 - **LogError**
 - **Etc....**

```
[Route("Error")]
[AllowAnonymous]
public IActionResult Error()
{
    logger.log
    return
}
```



Next : Logging to a file